DBeaver

Documentação oficial (wiki do projeto DBeaver) — English

Fonte oficial: https://github.com/dbeaver/dbeaver/wiki
Gerado para o projeto doc-frog

Getting started

Table of contents

- Introduction
- First steps
- DBeaver essentials
- Additional resources

Welcome to DBeaver, a powerful and user-friendly database management tool. This guide will help you get started with DBeaver, from installation to basic database operations.

Introduction

DBeaver is an open-source universal database tool that supports a wide variety of databases including SQL, NoSQL, and more. It provides an intuitive interface for database management, development, and analysis.

First steps

To ensure a smooth start with DBeaver, we have outlined the initial steps you need to take. These steps will guide you from download and installation to basic database operations, helping you to become familiar with DBeaver's environment and its powerful features.

- **Download DBeaver**
  Please download DBeaver from the official website. Choose the appropriate version for your operating system
- **Install DBeaver**
  Follow the instructions to install DBeaver on your operating system.
- **Configure DBeaver**
  Choose the language, appearance, optional features, and data collection settings for your first session.
- **Import license**
  If you have a PRO version, learn how to import your license to unlock the full capabilities of DBeaver.
- **Interface overview**
  Get familiar with DBeaver's user interface to navigate through the tool efficiently.
- **Basic operations**
  Walk through the basic steps for your first use of DBeaver.

DBeaver essentials

Refer to these resources for comprehensive support in your database operations.

- **Connections**
  Navigate through your database schemas and objects with ease.
- **Data Editor and Viewer**
  Manage and manipulate your data directly within DBeaver.
- **Database objects**
  Create, modify, and manage your database objects.
- **Run queries**
  Start executing your SQL queries and scripts.
- **Create tables**
  Define and create new tables within your database.
- **Database diagrams**
  Visualize and design your database structure.
- **Schema comparison** **and** **Migration**
  Compare and synchronize database schemas.
- **Export and Import**
  Move data and objects in and out of your database.
- **Backup and Restore databases**
  Protect and recover your database data efficiently.

Additional resources

Here are some resources for additional support and information:

- **DBeaver Blog**
  Read about the latest developments, features, and insights from the DBeaver team on our official blog.
- **YouTube channel**
  Watch tutorials, feature overviews, and other helpful videos to get the most out of DBeaver.

- **Twitter updates**
  Follow DBeaver on Twitter for the latest news, updates, and tips.
- **Edition comparison**
  Compare different DBeaver editions to find the one that fits your needs the best.

Installation guide

Table of contents

- Windows and MacOS installer
- Windows silent install
- ZIP archive
- Debian Package
- RPM Package
- Snap package
- Eclipse plugin
- Automatic updates
- Activate license
- Initial configuration

The installation process depends on the distribution type and your operating system.

DBeaver supports the following operating systems:

- **Microsoft Windows**: Windows 10 or later versions.
- **Windows Server**: Windows Server 2016 or later versions.
- **macOS**: macOS 11 or later versions.
- **Linux**: Most modern distributions.

> Download DBeaver from dbeaver.com.

Windows and MacOS installer

The installer distribution is the recommended method for installing DBeaver on Windows and MacOS. It includes all necessary dependencies. Additionally, the installer automatically upgrades DBeaver to the new version if a previous version is already installed. To install DBeaver, run the installer executable and follow the on-screen instructions.

> **Note**:
>
> - The installer does not modify any system settings or the Java installation.
> - The included JDK is accessible only to DBeaver.
> - DBeaver includes OpenJDK, which is an open-source version of Java. This means you do not need to purchase a separate Java license to use DBeaver.

Windows silent install

Silent mode installs DBeaver without any user interaction. It's useful when you need to deploy it on multiple machines at once.

> For the full list of parameters and return codes, see Windows silent install.

ZIP archive

To install DBeaver manually without using an installer:

1. Extract the contents of the archive.

   > **Note**:
   >
   > - Do not unzip the archive over a previous version of DBeaver. If you already have any version of DBeaver extracted in the same location, remove it before unzipping the new version.
   > - All configurations, scripts, and other necessary data are stored in a separate location (usually in the user's home directory), so uninstalling the program does not affect them.

2. Run the **dbeaver** executable.

Debian Package

To install DBeaver using a Debian package:

1. Run `sudo dpkg -i dbeaver-<version>.deb`.
2. Execute `dbeaver &`.

RPM Package

To install DBeaver using an RPM package:

1. Run `sudo rpm -ivh dbeaver-<version>.rpm`.
2. Execute `dbeaver &`.

> **Note**: To upgrade DBeaver to a newer version, use the `-Uvh` parameter. For instance: `sudo rpm -Uvh dbeaver-<version>.rpm`.

Snap package

To install the `dbeaver-ce` snap package:

```
snap install dbeaver-ce --stable
```

> For information on connecting snap interfaces and known issues, see Snap installation.

Eclipse plugin

You can also install DBeaver as a plugin in Eclipse-based IDEs. For information on installation methods, see Install DBeaver as an Eclipse plugin.

Automatic updates

This feature is available only on Windows and macOS.

Periodically, DBeaver automatically checks for new versions. If the check is not disabled and a new version is available, it will prompt you to decide whether to upgrade. If you choose to upgrade, the installer will be downloaded and launched automatically upon completion.

> **Note**: DBeaver itself will be gracefully closed to avoid data loss.

Activate license

After installation, activate DBeaver to use commercial features:

- Standard license
- Microsoft Store

Initial configuration

After installation and license activation, start DBeaver and complete the initial configuration.

How to import license

Table of contents

- Import from email
- Import from the personal account
- Activate paid subscription
- First-time activation on startup
- Insert the license key into the License Manager
- Import on startup
- License Manager
- Troubleshooting
- Unable to activate a monthly subscription license

> **Note**: If you installed DBeaver from Microsoft Store, your license is managed differently. See Microsoft Store license.

DBeaver offers the following activation options:

- **Free 2-week trial**: explore all features before buying.
- **Paid subscription**: a subscription for commercial use.
- **Academic license**: free access for students and teachers.
- **Early Access Program (EAP)**: access to pre-release builds.

After your license is issued, you'll receive the license key by email. It's also available in your personal account on our website. You can import the key directly from the email or from your personal account. From the personal account, you can also assign license managers, monitor usage, and reassign a license.

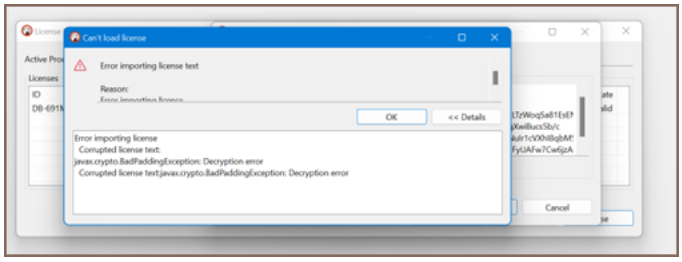
> For information on managing licenses in your account, see License management in the user portal.

Import from email

1. Copy-paste the license key

Note: You need to copy-paste the full license text (not just the license ID). The license text starts with "–" and ends with "==" characters.

Sometimes an email client can corrupt the formatting of the license key that can cause an error.

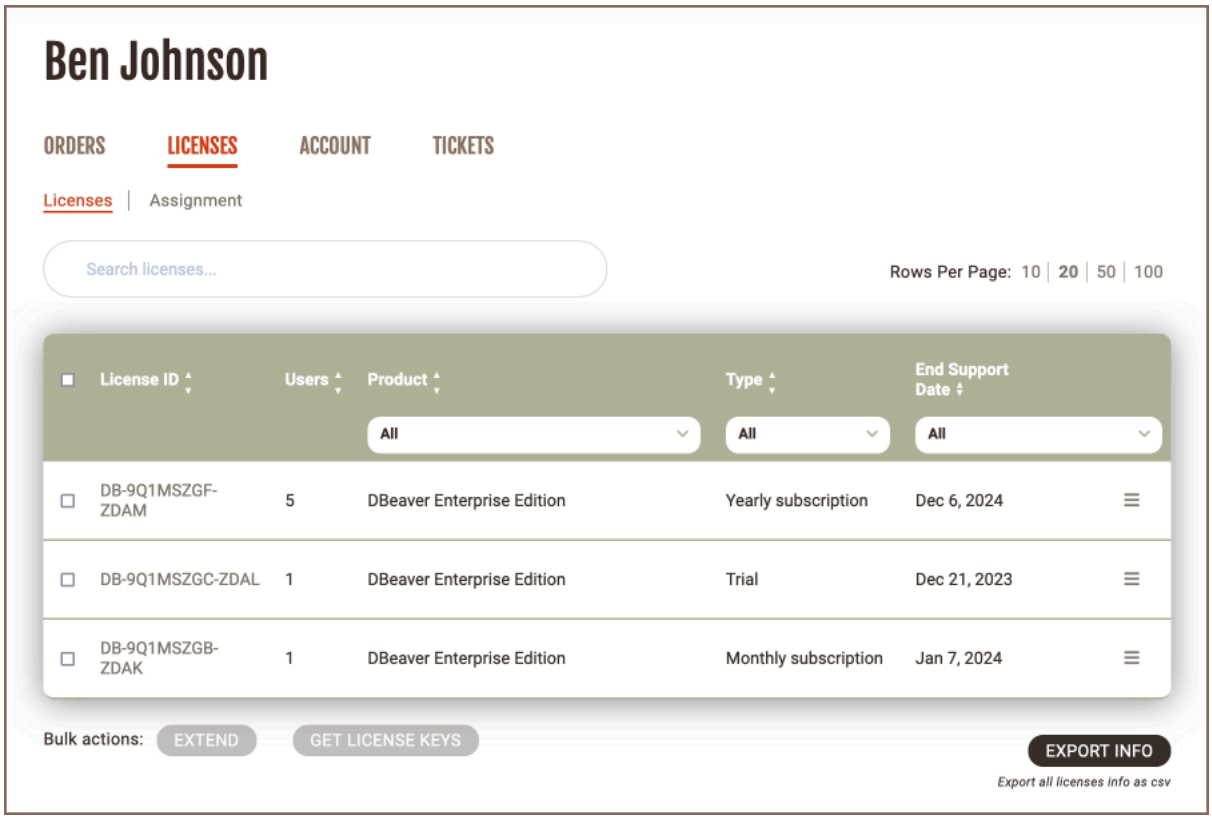


Therefore, you need to import your license key from your personal account on our site https://dbeaver.com/signin/.

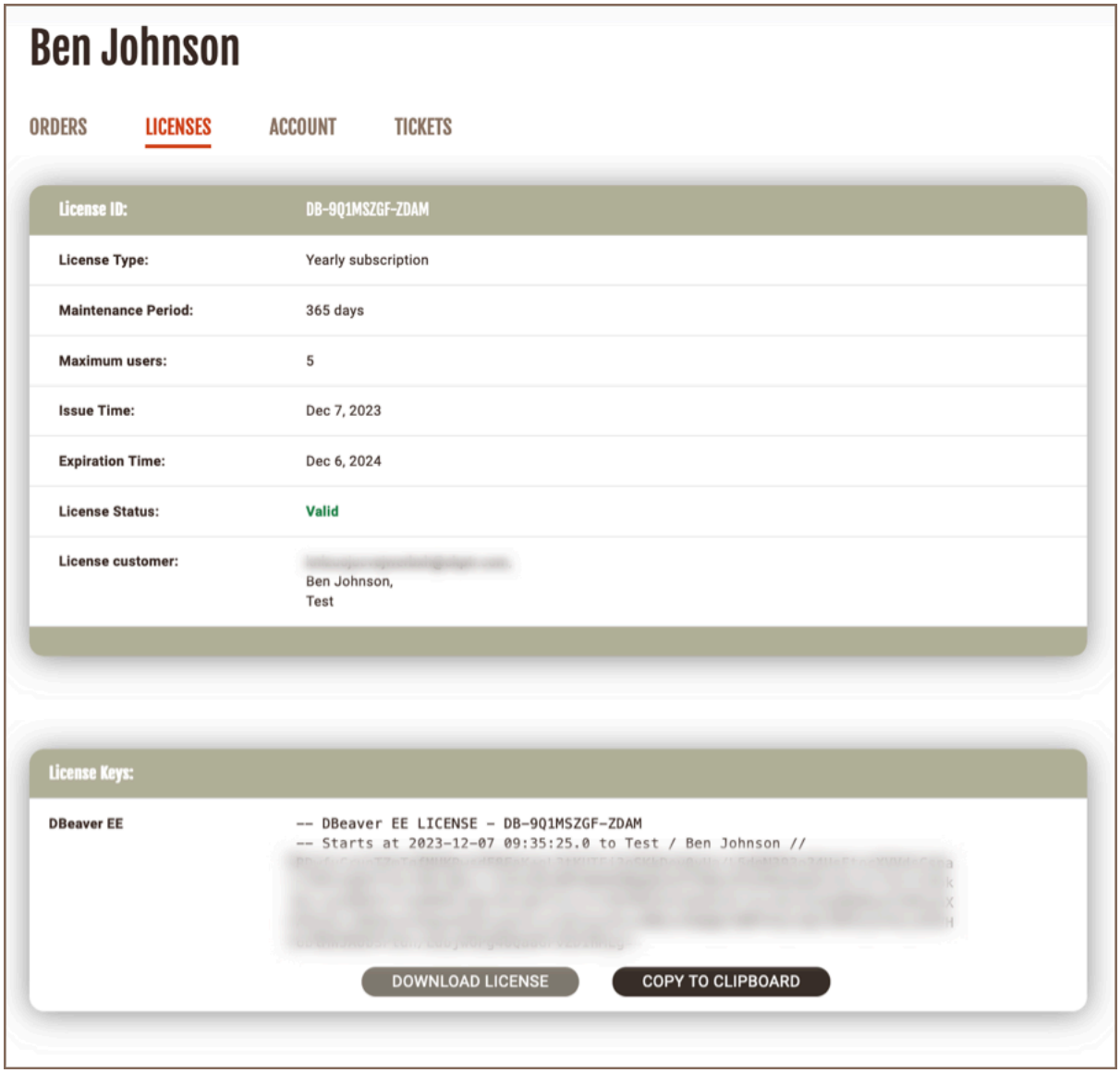
2. Insert the copied license key to License Manager in DBeaver

Import from the personal account

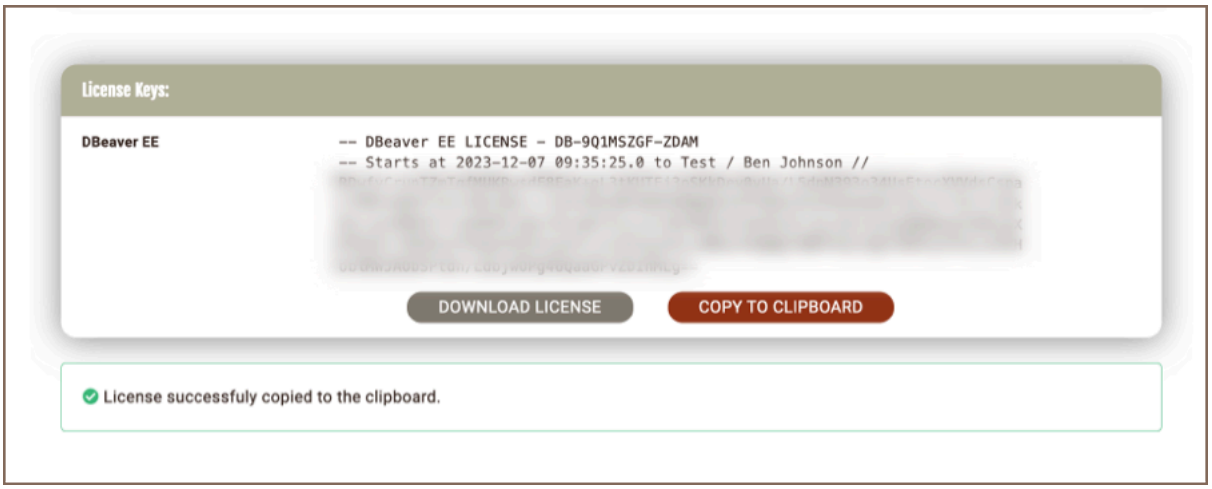
1. Sign in
2. Open the **Licenses** tab, where you can find all your licenses



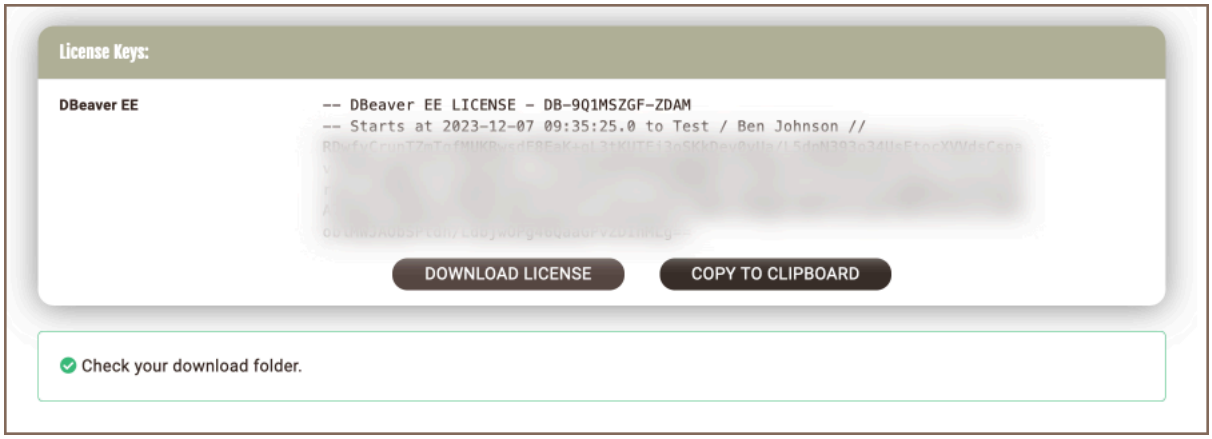
3. Click the license ID link. Here you can find your license status, type, maintenance period, and end support date



4. At the bottom of the page you can find the license key required to start using DBeaver. There are two options how to copy your license key from the personal account:
    - press the **COPY TO CLIPBOARD** button, then press **OK**. The license text will be copied to the clipboard



    - press the **DOWNLOAD LICENSE** button, then press **OK**. The `.txt` file with your license key will be downloaded to your download folder. The file name is License ID, e. g. `DB-841MRZHY-ZH54`.



5. Insert the copied license key to License Manager in DBeaver

Activate paid subscription

Activate your license either during the first startup or when you need to add or replace a license key in an already activated installation. The steps differ depending on whether a license is already present.

First-time activation on startup

Use this option when you start DBeaver for the first time and no license is installed yet.

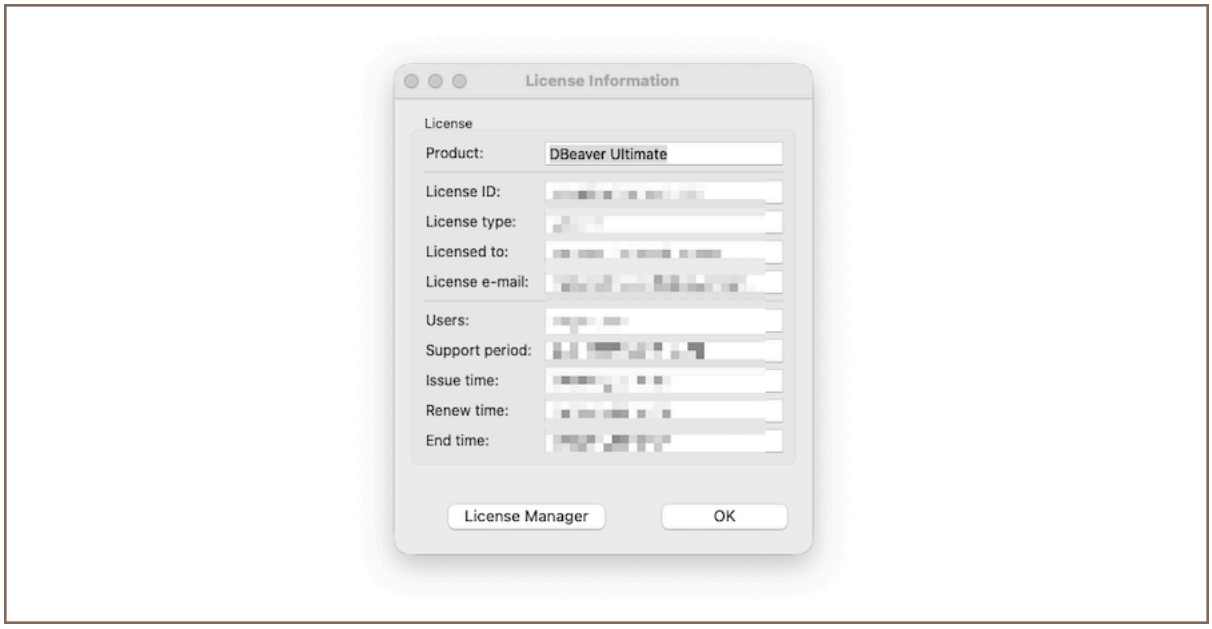
1. Start DBeaver. The **No license found** dialog opens automatically.
2. Select **Import license**.
3. Insert your license key:
    - If you copied the license key to the clipboard, press the **Paste** button and then **Import**.
    - If you downloaded a `.txt` file with the license key, press the **Load** button and then select the file from Downloads. The license key will be pasted automatically.
4. Click **Import**.

> **Tip**: This flow is intended for initial activation. For renewals or replacing an existing license, use the License Manager described below.

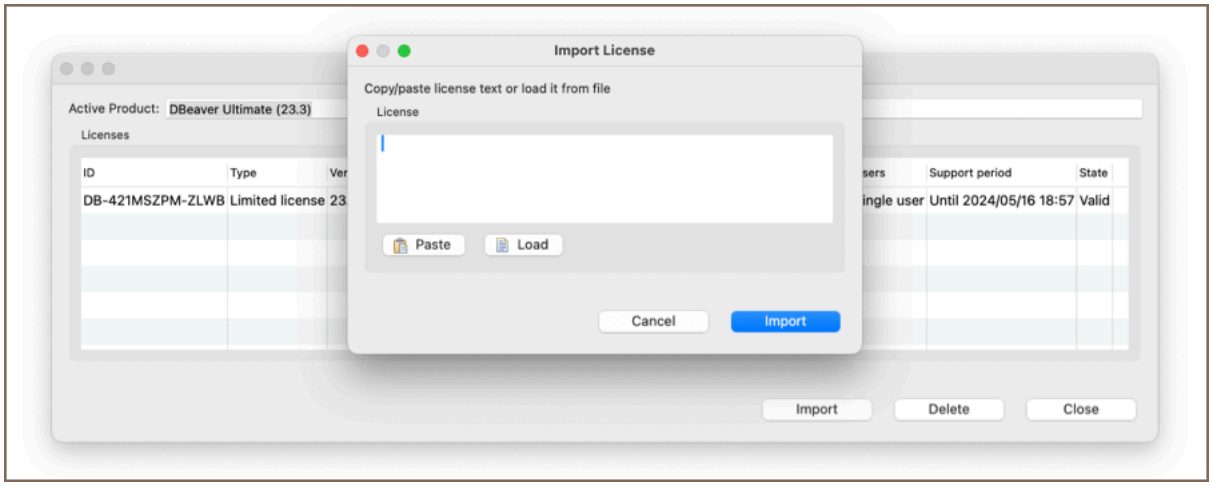
Insert the license key into the License Manager

Use this option to renew an existing license, replace it with a different one, or add a new license to an already activated installation.

1. Go to **Help -> DBeaver License Info** in DBeaver.
2. Open the License Manager.

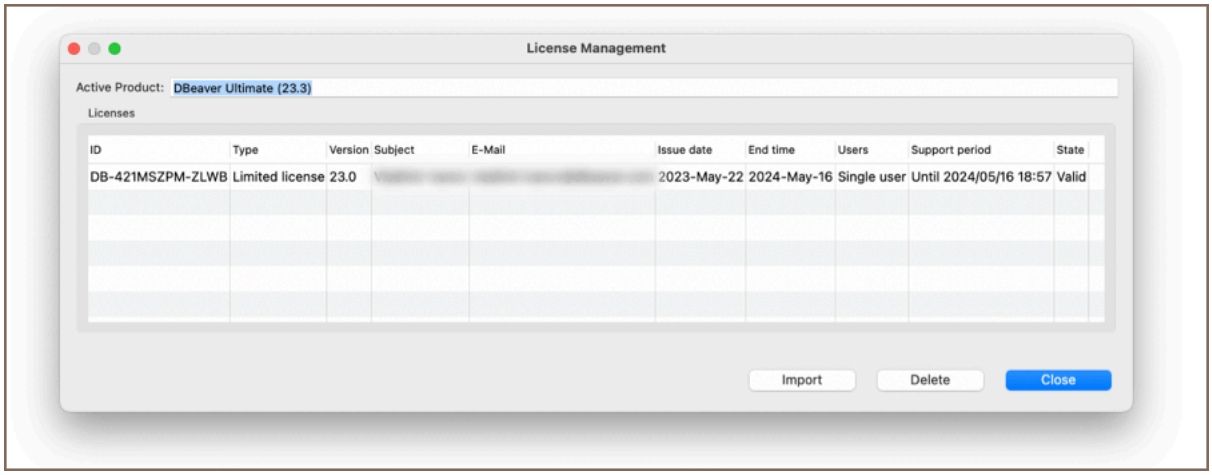


3. Press the **Import** button to paste your license key.



- If you copied the license key to the clipboard, press the Paste button and then Import.
- If you downloaded a `.txt` file with the license key, press the Load button and then select the file from Downloads. The license key will be pasted automatically.

4. Press the Import button and your license will be added to the License Manager.



Import on startup

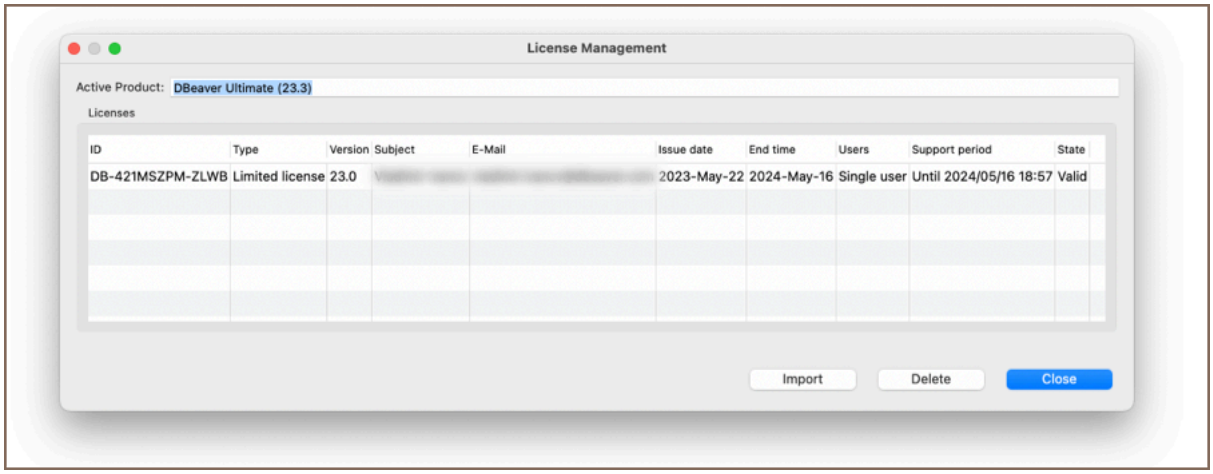
If you want DBeaver to detect the license file automatically (for example, when deploying on multiple machines), you can place the license file in a predefined location before the first launch.

> For details, see License Administration.

License Manager

License Manager provides you with the following information about your licenses:

- **License ID** – e.g. `DB-841MRZHY-ZH54`
- **License type** – Trial, Academic, Subscription, or Standard
- **Version**
- **License owner's name and company**
- **Owner's email**
- **Issue time** – date of initial purchase
- **End time** – date the license expires
- **Number of users** – single user or multi-user license
- **Support period** – duration of access to support system and updates
- **State** – valid or expired



Troubleshooting

Unable to activate a monthly subscription license

A monthly subscription requires internet access on the workstation for the first activation and each prolongation.

If you do not have an active internet connection or work behind a corporate firewall while importing the Subscription license, the following error can appear:

```
Invalid subscription
Can`t find the subscription information for license ‘DB-841MRZHY-ZH54’.
Check your internet connection and/or firewall settings and restart the application.
```

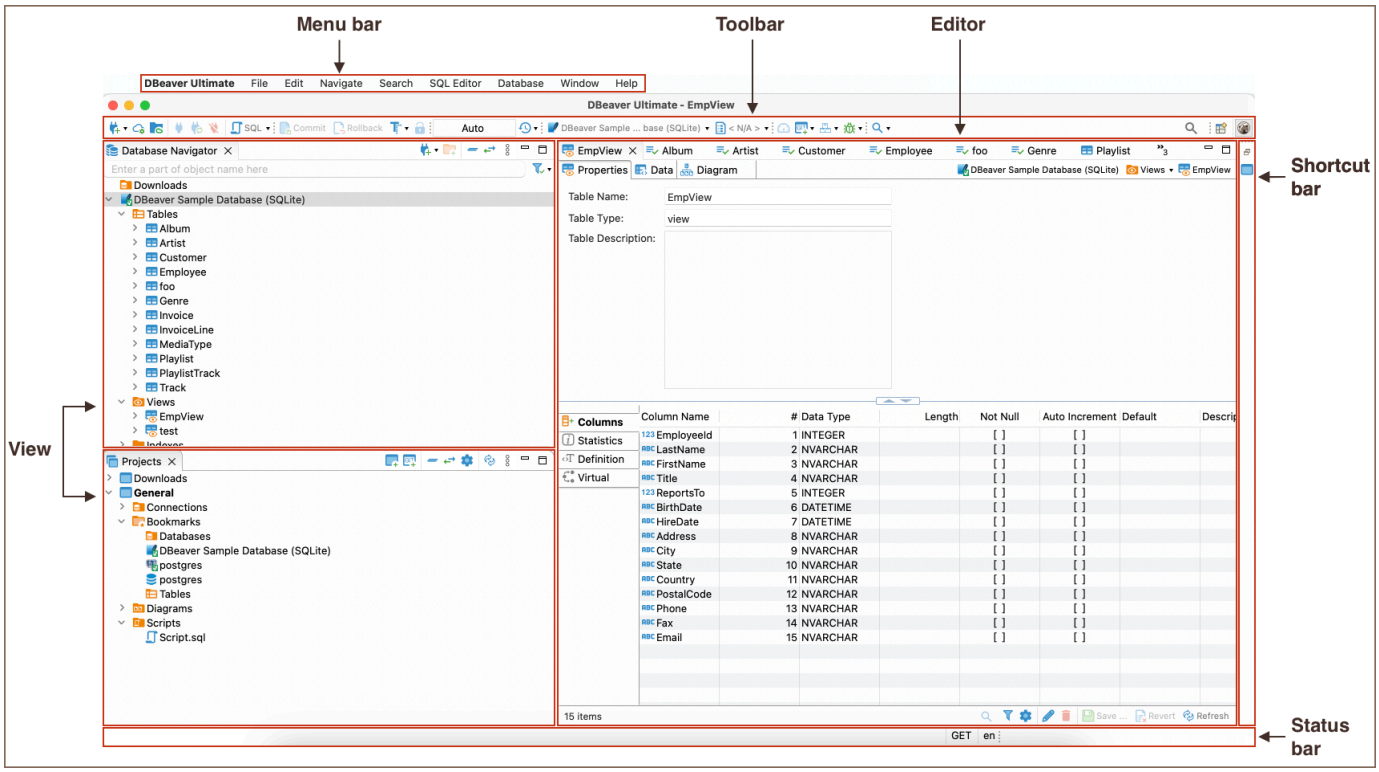
In this case, you need to check that DBeaver has internet access. If you're behind a firewall, you may need to configure a proxy. The UI option might not be available in such cases. For instructions, see Configure proxy using system files.

Application window overview

Table of contents

- Menu Bar
- Toolbar
- Shortcut Bar
- Main area
- Changing the layout
- Managing View and Editors

The DBeaver window contains a menu bar, a toolbar, a main area with one or more editors and views, and a status bar:



Menu Bar

By default, the menu bar contains the following menus:

- The **File** menu contains menu items for creating files, folders, projects, database connections, database projects, Diagrams, and Import and Export items.
- The **Edit** menu contains global commands like Cut, Copy, Paste, Delete and Add Bookmark targeted at the active element.
- The **Navigate** menu allows navigation through scripts and database objects.
- The **Search** menu provides options to search among files database objects and across data.
- The **SQL Editor** menu is for opening SQL Editor and managing its appearance.
- The **Database** menu allows the management of database drivers, connections, and transactions and reconnecting to and disconnecting from a database.
- The **Window** menu includes items to manage the look of DBeaver window: show/hide and minimize/maximize views and editors, display bars, split editors, and manage other preferences.
- The **Help** menu contains links to information and help resources, and menu items to check the version number and availability of updates.

You can customize the menu bar and the list of menu items to display; for this, go to **Window** -> **Customize Perspective** -> **Menu Visibility** tab.

Toolbar

The toolbar contains buttons for the most basic and frequently used commands:

Some buttons are enabled (colored) and others are disabled (grey). The sets of enabled and disabled buttons change depending on which editor is active. Only enabled buttons apply to the active view or editor.

> **Tip**: The toolbar can be customized. For more information, please consult our customization guide.

Shortcut Bar

There are two shortcut bars - one on the left and one on the right side of the main area. Shortcut bars host shortcuts of views and editors and appear if at least one view or editor is minimized, otherwise, they are hidden.

Main area

Views are windows within the main area that provide presentations and ways to navigate the information. For more information about particular views, see the Views article.

Editors are windows in which you can interact with the content of files and databases.

> For more information on Editors, see Database Object Editor.

Both views and editors can appear as separate windows or tabs stacked with other views/editors in a tabbed window. The following image shows the title bar of a tabbed window. They become hidden if tabs do not fit in the title bar of a tabbed window. To see the list of hidden tabs, click the Show List icon that also indicates their number:

Several views and editors can be opened simultaneously, but only one can be active at a time.

You can change the layout of the main area by opening and closing views, docking them in different positions, collapsing them to the shortcut bar, or expanding them to fill the main area and restoring them to the latest docked position.

Changing the layout

You can move views and editors around the main area and dock them in different positions:

- As a tab in a tabbed window
- As a separate window with a vertical or horizontal layout in any zone of the main area

You can also swap locations of two views or editors.

To dock a view to a position in the main area, press and hold the view's title bar, then drag and drop it onto the desired position.

You can resize the view and editor windows. To resize, place the cursor on the window's border until it changes to a double-ended arrow, then click and drag the edge to the needed size.

To close a view or editor, click the Close button, or right-click the title bar of the view / editor, then click one of the options on the context menu (they change depending on the configuration of windows):

- **Close** - to close the active window or tab in a tabbed window.
- **Close Others** (for editors and views that appear as tabs in tabbed windows) - to close all tabs of the current tabbed window except the active tab.
- **Close Tabs to the Right / Left** (also for tabbed windows) - to close all tabs of the current tabbed window that are located to the right / left of the active tab.
- **Close All** - to close all tabs of a tabbed window (close the window).

Managing View and Editors

All views and editors have the **Close**, **Minimize** and **Maximize** buttons:

The Maximize button changes to the Restore button when a view or editor is maximized.

To maximize a view or editor to fill the main area, do one of the following:

- Click the Maximize button in the upper-right corner of the view.
- Double-click the title bar of the view or editor.
- On the **Window** menu, click **Appearance** -> **Maximize Active View or Editor**.

When one view is maximized, other views and editors appear as shortcuts on the shortcut bar.

To restore a maximized view or editor to its latest docked position, double-click its title bar or the **Restore** button in its upper-right corner.

When you minimize a view, it wraps into a shortcut on the shortcut bar:

The shortcuts of views and editors may appear on the left or the right shortcut bar, depending on the latest docked position of the view or editor.

To minimize a view, do one of the following:

- Click the **Minimize** button in the upper-right corner of the view.
- On the Window menu, click **Appearance** -> **Minimize Active View or Editor**.

To restore a minimized view or editor to its previous position, click the Restore button on its shortcut in the shortcut bar. To restore a minimized view or editor to a new position, click the view / editor name button under the restore button.

Basic operations

Table of contents

- First launch
- Connections
- Object editor
- Basic operations
- Object creation
- Working with data in DBeaver UI
  - Additional Resources
- Executing SQL queries

This guide outlines the essential steps to quickly get started with DBeaver.

> To begin working with this guide, it is essential that you install the appropriate version of DBeaver for your operating system. Please download it from the official DBeaver website.

First launch

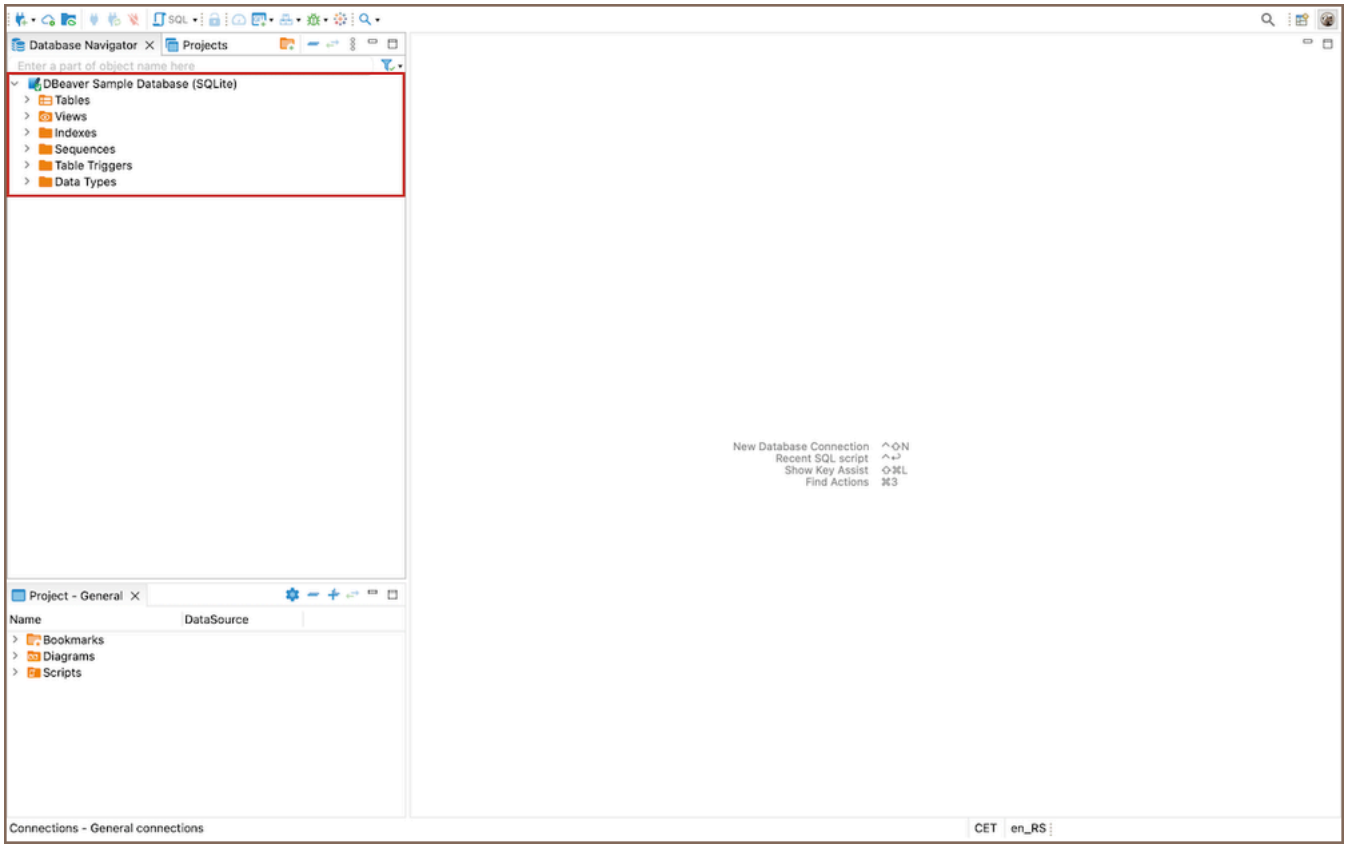
Configure the main settings for your first session in the Product Configuration wizard.

> **Tip**: For a detailed understanding of DBeaver interface, refer to the Application Window Overview article.

Connections

Connections is the central hub for accessing and managing your databases within DBeaver. It allows you to view and interact with the metadata objects of your databases.

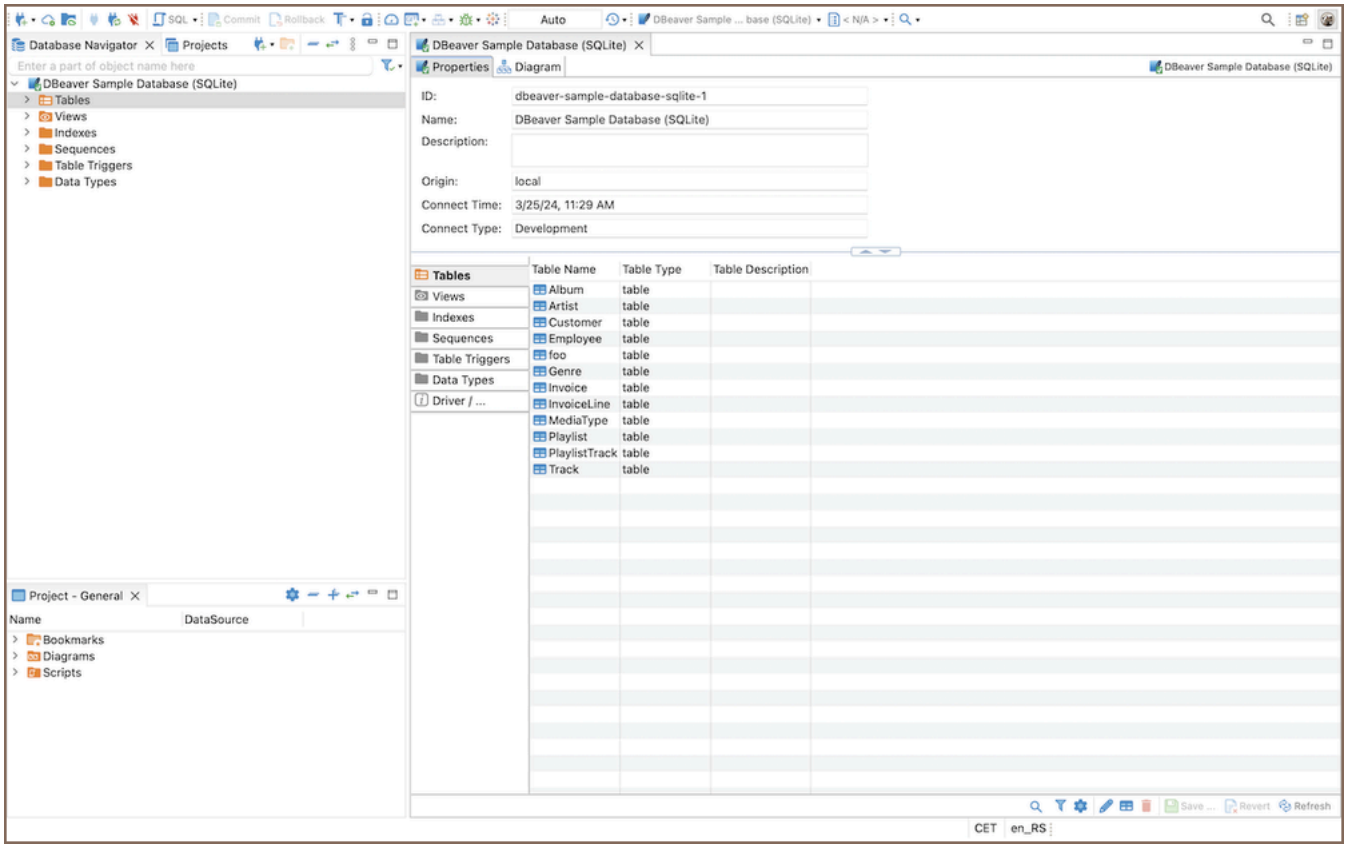
To explore these objects, click on the arrow located next to the database you have created. This action will expand the list of metadata objects available in your database instance.



> **Tip**: For an extensive understanding of the Connections capabilities, refer to the Connections article.

Object editor

From the Connections, you can access the Object Editor, where you have the capability to examine database objects in detail. To view the contents of a table, or to delve into any database object, simply double-click on the desired item. This action will open the Object Editor, providing a comprehensive view and interaction options for the selected object.



> **Tip**: You can also access the Editor using the shortcut (F4). For a comprehensive list of shortcuts and their functions, see the article on Shortcuts.

The Object Editor presents a structured environment where you can access different aspects of your database objects through specialized tabs:

Tab	Description	Reference Article
Properties	Displays the properties of the database object and its sub-entities.	Properties Editor
Data	Opens the Data Editor for tables and views, facilitating data management.	Data Editor
Diagram	Shows Entity-Relationship Diagrams for tables and schemas, visualizing object relationships.	Diagrams

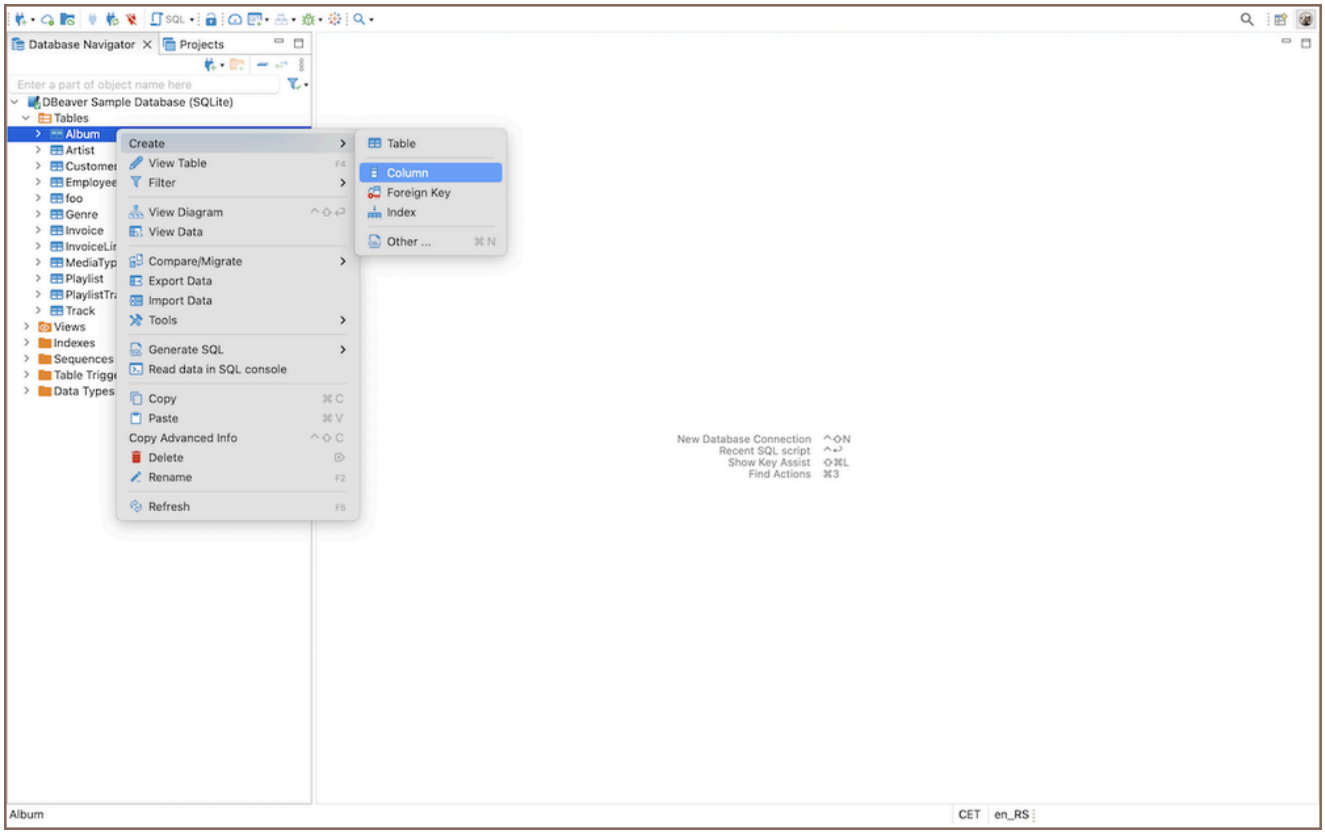
Basic operations

Now, let's explore how to work with data. This section will guide you through the essential processes for working with database objects.

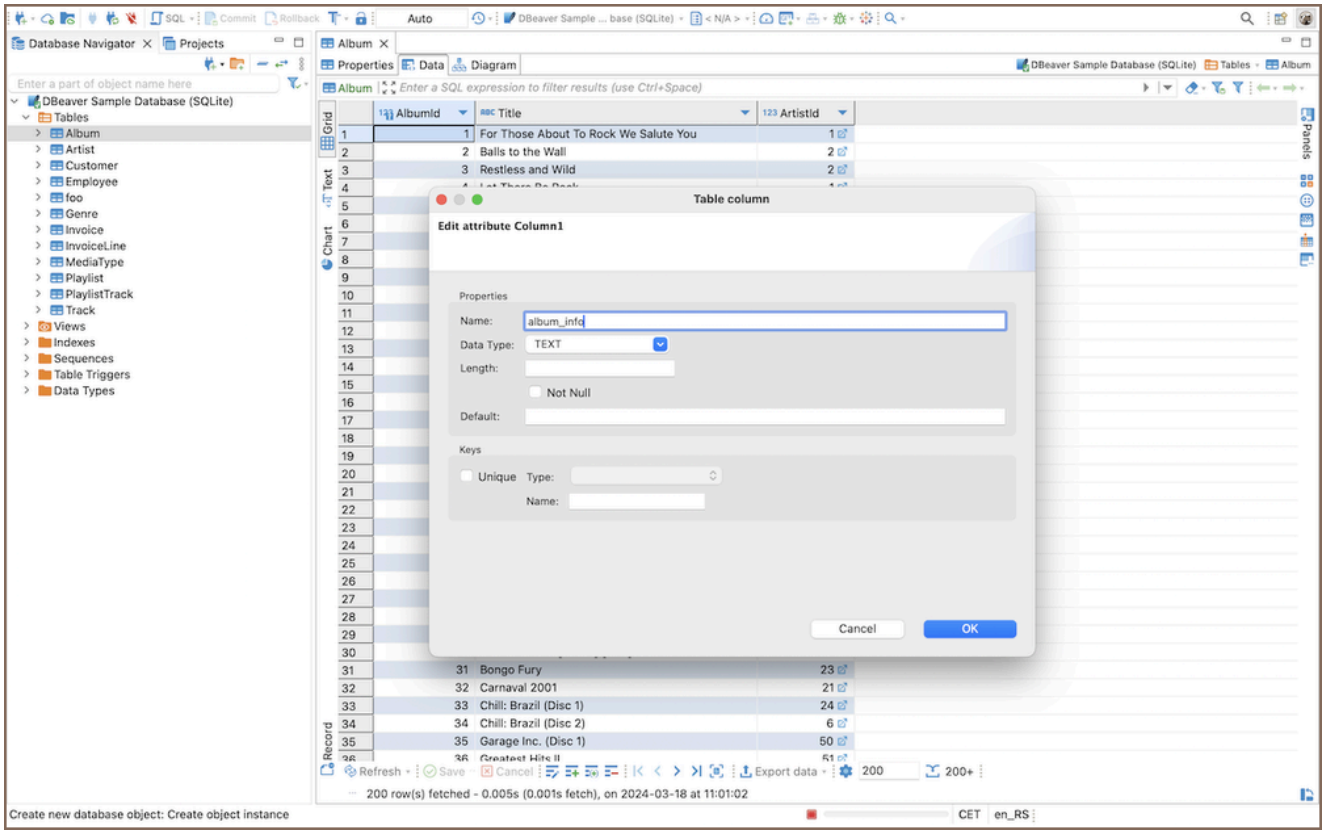
Object creation

Creating new objects within your database is a fundamental skill in database management. This process allows you to add structures like tables, views, or schemas, which are essential for storing and organizing your data effectively.

1. In the **Connections**, locate your database.
2. **Create new object**: Depending on the type of object you wish to create, you might need to navigate to different levels within your database connection.



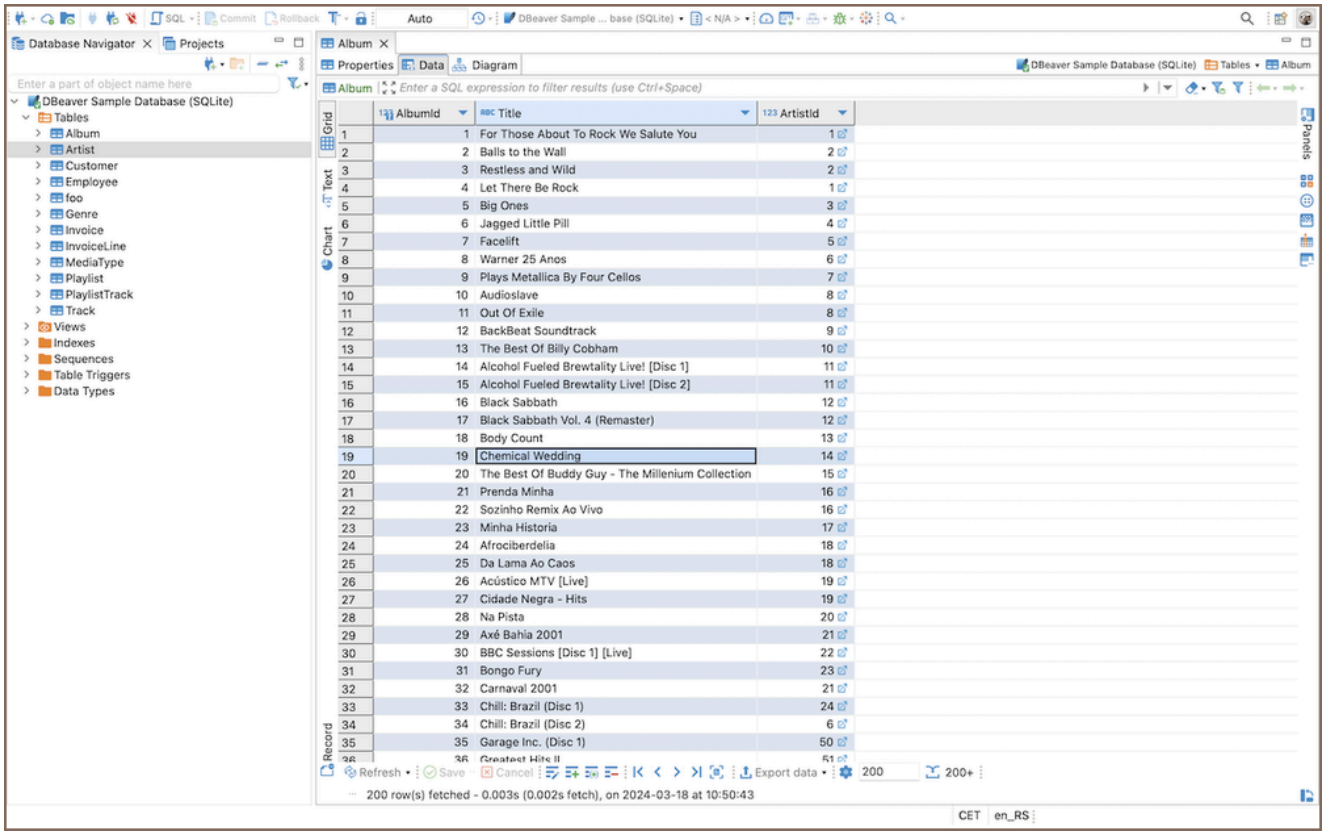
3. First, in the **Connections**, navigate to the level where your new object should be created. This could be at the connection level (for databases and schemas), within a specific database or schema (for tables, views, etc.), or within a table (for columns, foreign keys, indexes, etc.).
4. Right-click on the node (such as a database, schema, table, etc.) where the new object belongs, and select **Create**. A submenu will appear, listing the types of objects you can create at this level.
5. From the submenu, choose the type of object you want to create (e.g., Database, Schema, Table, Column, View, Foreign Key, Index, etc.). If necessary, a dialog will appear, prompting you to enter the details for the new object, such as name, data type, constraints, and other relevant properties.
6. **Fill in the details**: Follow the on-screen instructions to complete the creation of the new object. Some objects may require additional steps or information, such as defining columns for a new table or specifying the reference table and columns for a foreign key.



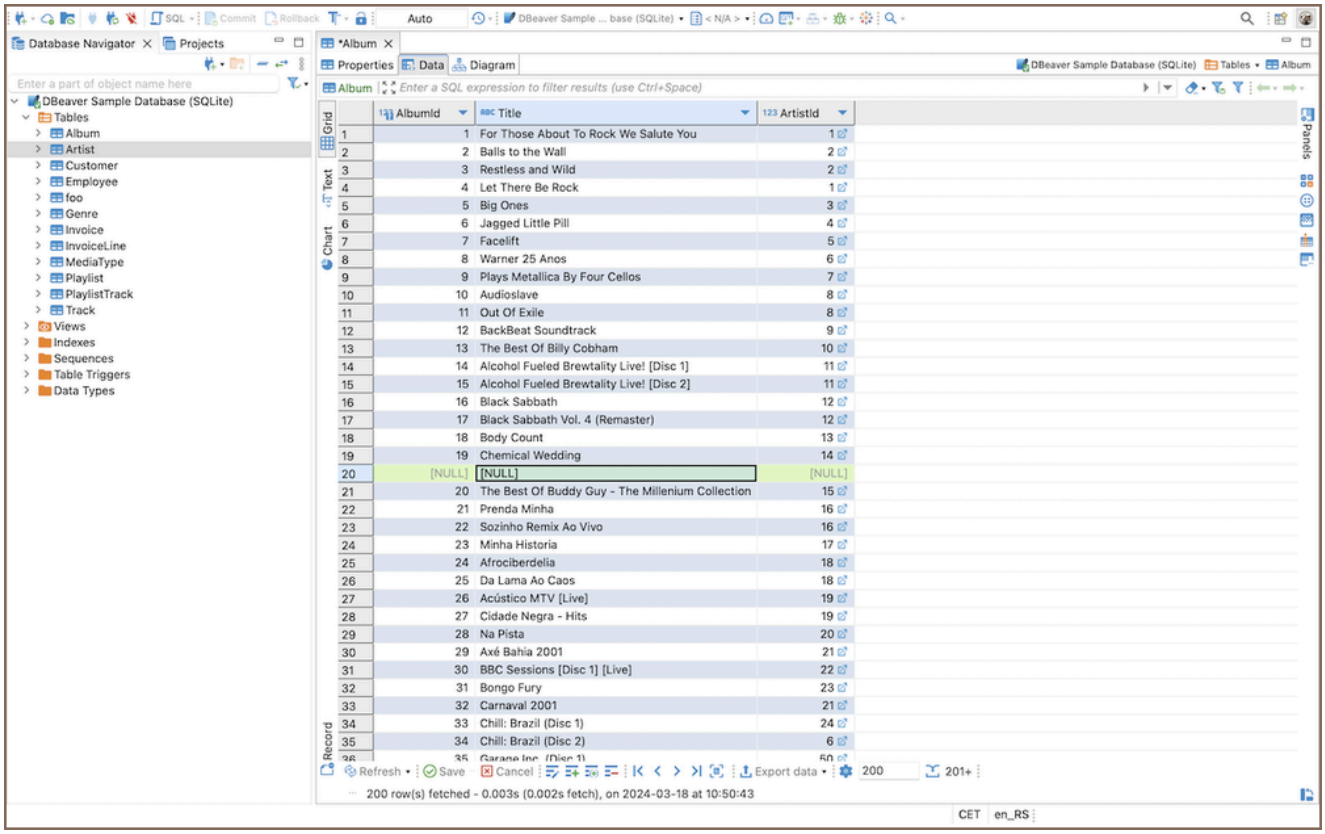
7. **Save**: To complete the creation of the object, click the **Save** button Save .

Working with data in DBeaver UI

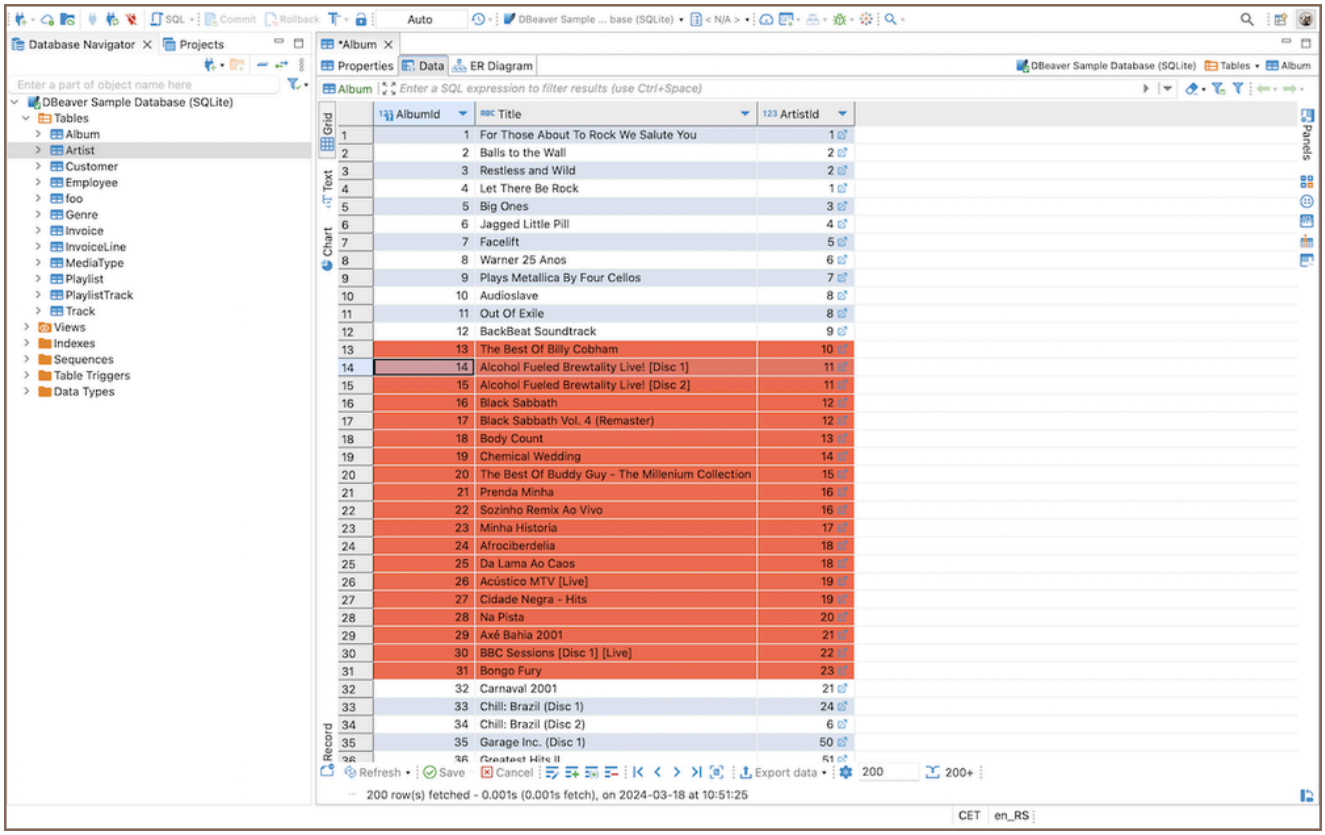
1. **View and Edit Data**: Navigate to your table in the **Connections**, right-click, and select **View Table -> Data** to see the table's contents. Here, you can directly edit the data by typing in new values into the cells, and then save your changes.



2. **Inserting new data**: Use the **Add Row** button in the table data view to add new records. Fill out the new row with your data and click **Save**.



3. **Deleting data**: Select the rows you want to remove and click the **Delete current row** button . Confirm the deletion and save your changes.



Additional Resources

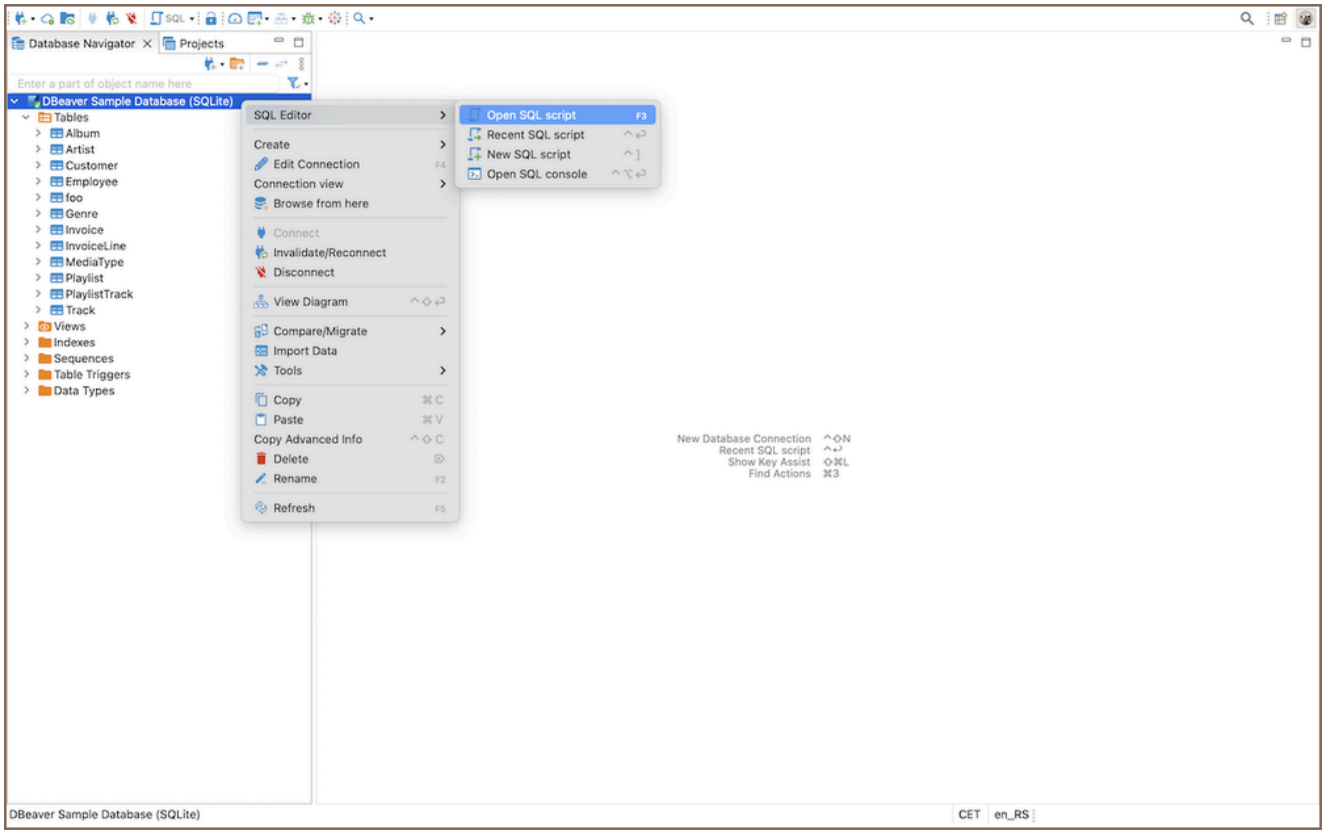
Dive deeper into specific data management topics with these detailed guides:

- New Table Creation
- Creating Columns
- Implementing Constraints
- Utilizing Foreign Keys
- Creating Indexes
- Incorporating Triggers

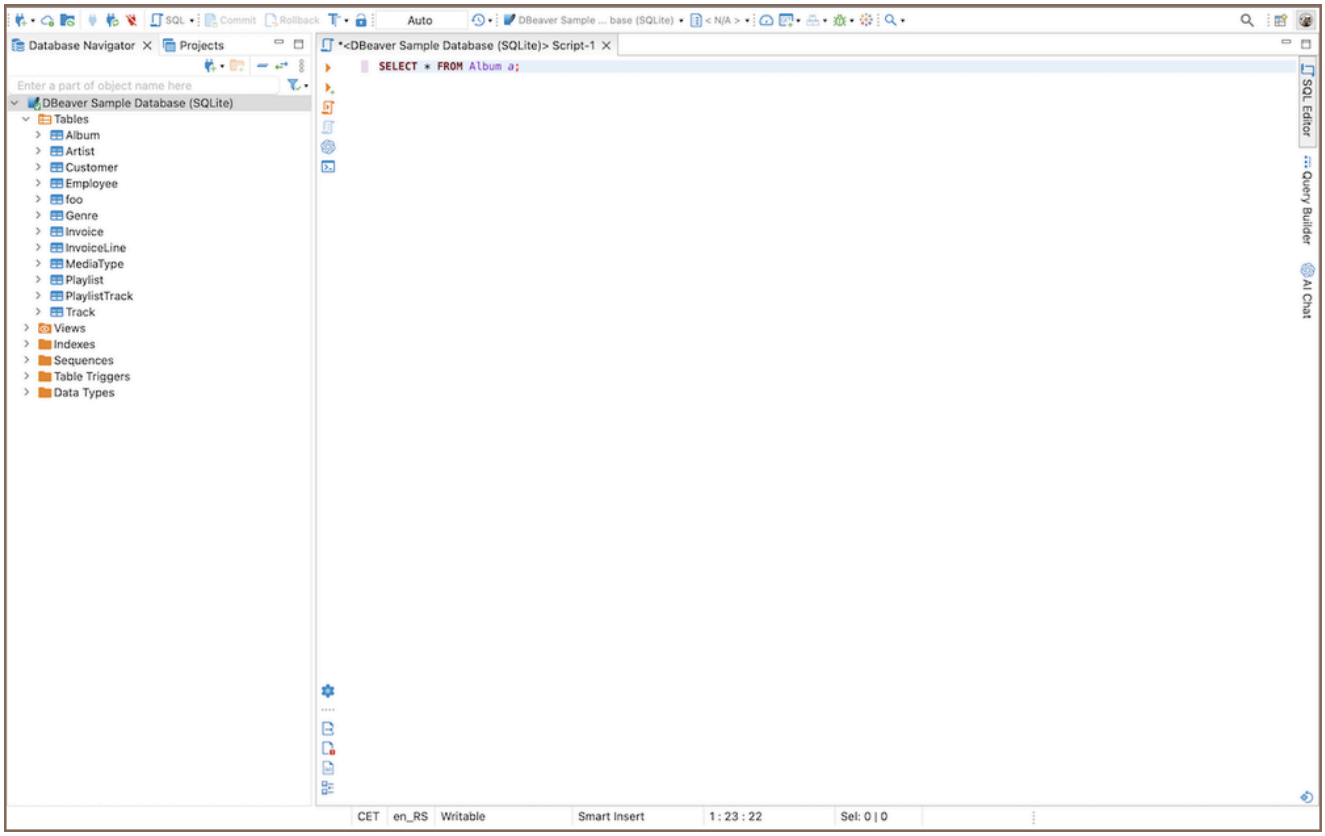
> For more on data management, view our tutorials on YouTube.

Executing SQL queries

1. **Open SQL Editor**: Navigate to the database or schema in the Connections, right-click, and select **SQL Editor -> Open SQL Script**.



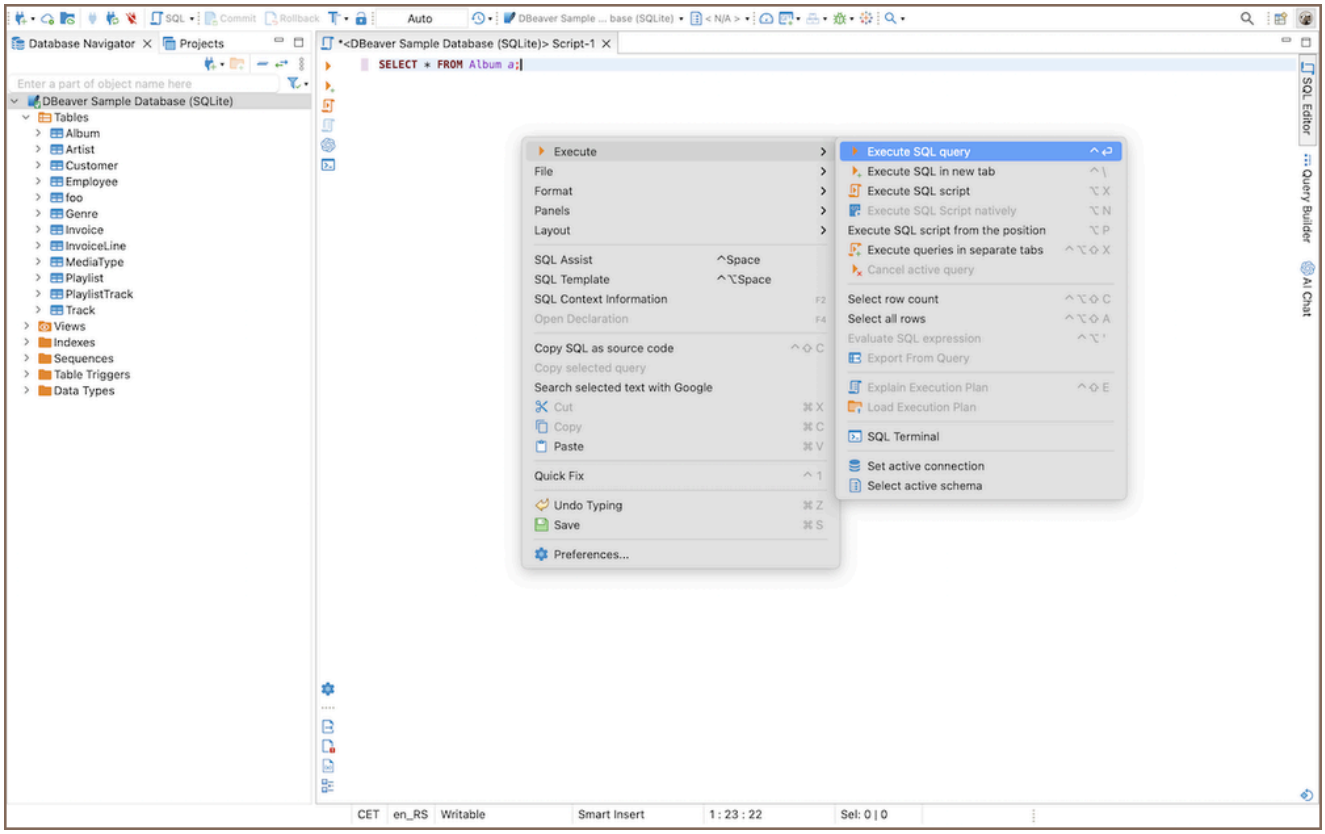
2. **Write your query**: In the SQL Editor, write your SQL query. For example, `SELECT * FROM your_table;`.



> Enhance your query writing with these powerful features: - **AI SQL Assistance**: Utilize AI to suggest optimizations and best practices for your SQL queries. - **SQL Code Editor**: Enhance the behavior of the **Script panel**, focusing on features like displaying problem markers, automatic closing of quotes and

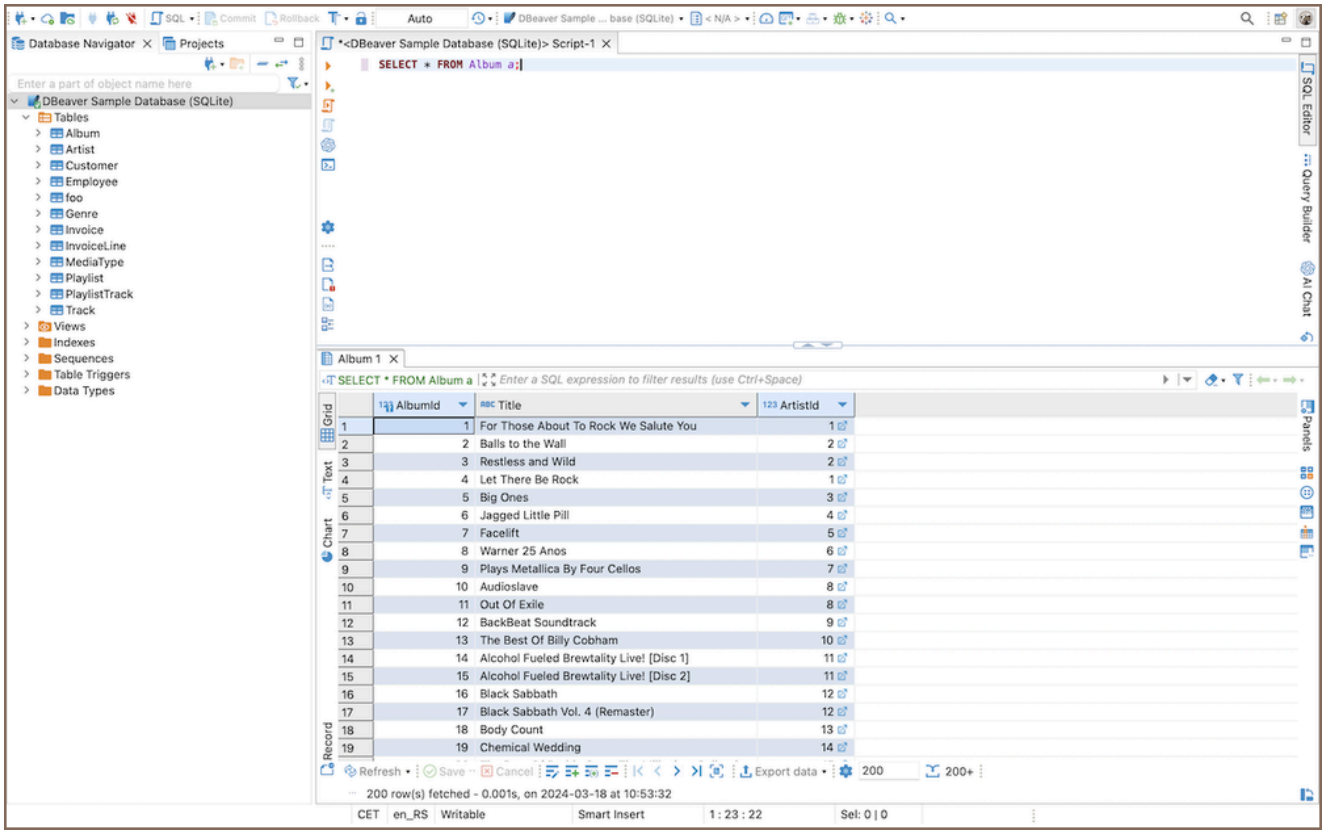
> brackets, semantic analysis, and more. - **Spelling**: Improve the accuracy of your SQL scripts with integrated spelling correction.

3. **Execute query**: To execute a query, press `Ctrl+Enter` or right-click the query and click **Execute -> Execute SQL Statement** on the context menu.



> **Tip**: You can access various execution option, depending on your requirements. For a comprehensive guide on query execution, refer to SQL Execution.

4. **View results**: The results will be displayed in the bottom panel of the **SQL Editor**.



For those who prefer visual learning or seek further instruction on using SQL with DBeaver, we recommend our YouTube playlist. These video guides complement the written documentation and provide practical examples to enhance your understanding and skills in managing databases with DBeaver.

DBeaver configuration

User Interface

Table of contents

- Change the DBeaver theme
- Managing the High Contrast theme in Windows
- Activation of High Contrast theme
- Deactivation of High Contrast theme
- High Contrast theme switching limitations

The interface themes governs the look and feel of the user interface's windows, dialogs, buttons, and all other visual components. By default, DBeaver is configured to use the **Light** theme.

Change the DBeaver theme

1. Go to **Window** -> **Preferences** -> **User Interface** -> **Appearance**.
2. Select from the following available themes:
    - **Light**
    - **Dark**
    - **Classic**

> **Note**: To fully apply the selected theme, you must restart DBeaver.

Managing the High Contrast theme in Windows

Windows users have the additional option of using **High Contrast** themes. This section provides details on how DBeaver interacts with these themes.

Activation of High Contrast theme

When the **High Contrast** theme is activated in Windows, DBeaver automatically switches its theme to **High Contrast**. If DBeaver is already running when the system theme changes, a notification will appear suggesting a restart for the theme to apply correctly.

Deactivation of High Contrast theme

If you deactivate the **High Contrast** theme in Windows, it will automatically be removed from DBeaver's list of available themes after the restart.

Note: After disabling the **High Contrast** theme in Windows, DBeaver will revert to the theme that was previously selected.

High Contrast theme switching limitations

The theme switcher in DBeaver is disabled when the **High Contrast** theme is active in Windows. You cannot switch to a **non-High Contrast** theme in DBeaver unless you disable the **High Contrast** theme in Windows.

Languages

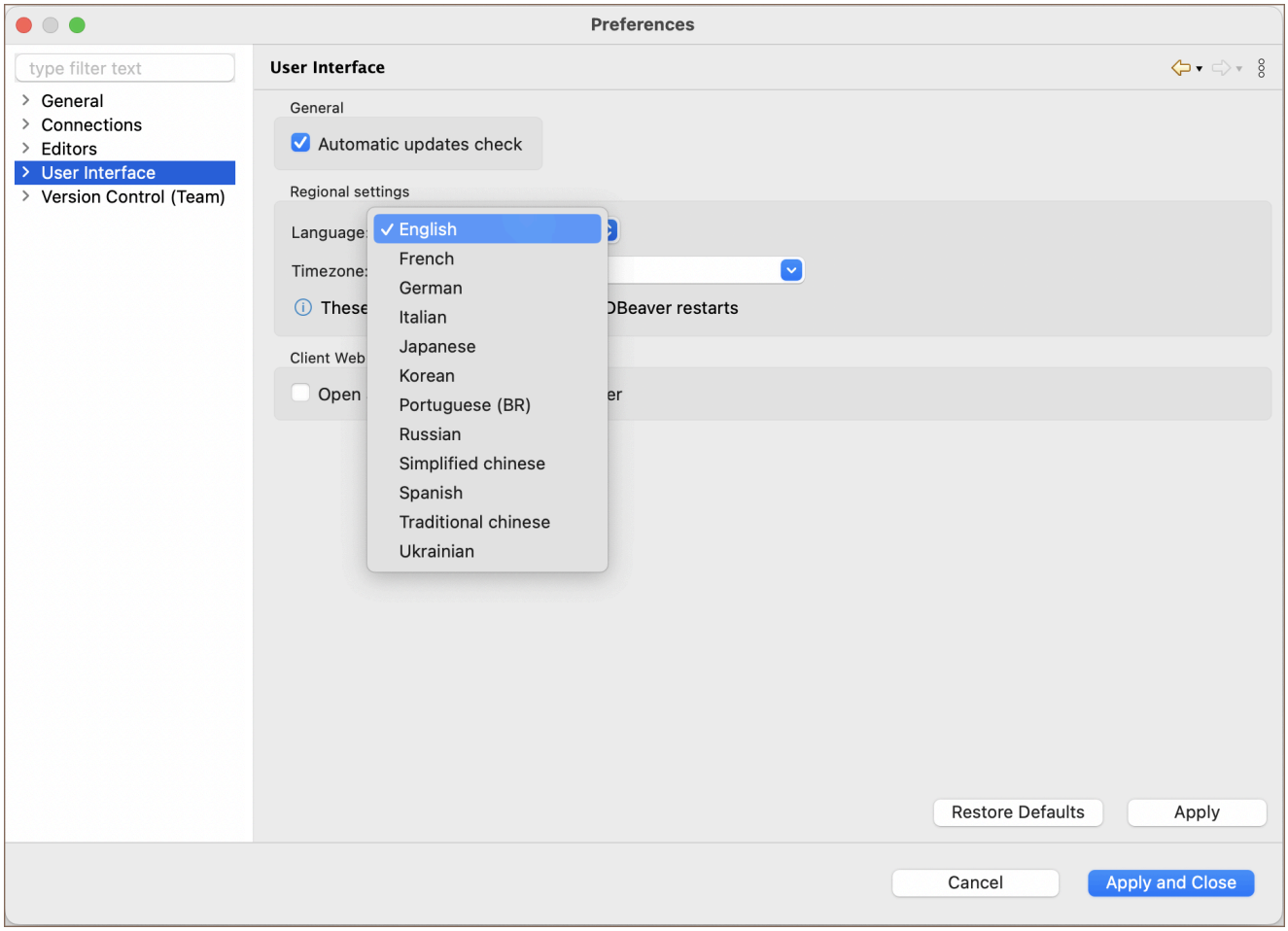
Table of contents

- Changing interface language in preferences
- Changing interface language in configuration file
- Supported languages

The interface language of DBeaver can be customized to suit your preferences. There are two methods to achieve this:

Changing interface language in preferences

1. Navigate to **Window -> Preferences -> User Interface**.



2. Choose your desired language from the drop-down menu.
3. Click the **Apply and Close** button to save your settings.

Note: If DBeaver is installed in a directory without write permissions, you may not be able to change the language directly through the interface. If this occurs, please proceed to modify the configuration file as described below.

Changing interface language in configuration file

1. Locate the `dbeaver.ini` file.

> **Tip**: Detailed instructions on finding `dbeaver.ini` are available in our article.

2. Using a text editor, open this file to make changes.
3. Insert the following lines above the `-vmargs` line:

```
-nl
xx
```

Replace xx with the language code from the list below:

Supported languages

Language	Code
English	`en`
Arabic	`ar`
French	`fr`
German	`de`
Italian	`it`
Japanese	`jp`
Korean	`ko`
Portuguese (BR)	`pt_BR`
Romanian	`ro`
Russian	`ru`

Language	Code
Simplified Chinese	`zh`
Spanish	`es`
Traditional Chinese	`tw`
Ukrainian	`uk`

Accessibility

DBeaver offers you the ability to use most of its features with a keyboard and screen readers. Here you will find a guidelines on how to use DBeaver without a mouse in some common cases.

> **Note**: DBeaver is tested primarily with standard Windows keyboard shortcuts for accessibility.

Table of contents

- General
- Screen readers support
- Structure and navigation
- Shortcuts
- Colors and fonts
  - Themes
  - Fonts
  - Data Colorization
- DBeaver Views and Wizards
- Connections
- New Connection
- Object Properties
- SQL Editor
- Result Set
- Diagrams

General

Screen readers support

DBeaver provides support for various screen readers:

- **NVDA**. Find more details in the NVDA User Guide.
- **JAWS**. Access comprehensive information through the JAWS Documentation.
- **Narrator**. More information available at the Narrator User Guide.

To change the screen reader in DBeaver, navigate to **Window -> Preferences -> User Interface -> Accessibility**.

Structure and navigation

DBeaver is an application consisting of many parts, so-called Views, such as **Connections**, **SQL Editor**, **Data Editor**, etc. A **Toolbar** with many buttons inside each **View** and horizontal and vertical tabs allows you to switch between **Views** and inside them.

Main navigation shortcuts - You can switch Views by using `Ctrl+F7` | `⌘F7`. - You can switch between buttons on the toolbars and menus using **Tab** and the **arrows**.

How to switch tabs - You can switch horizontal tabs by using `Ctrl+PageUp/PageDown`. - Sub-tabs can be switched by using `Alt+PageUp/PageDown`. - If there are vertical tabs inside the view, you can switch them by using `Ctrl+Alt+PageUp/PageDown` or `Ctrl+Shift+PageUp/PageDown`.

When you focus on any interface object, such as a table, column, tab, or cell, you can open the **Context menu** to see all the actions available for that object.

To open the Context menu, use `Shift + F10` | `⇧F10`.

Shortcuts

Most of DBeaver's functions are accessible through special shortcuts. You can find them in **Window -> Preferences –> User Interface –> Keys**. If you mostly use the keyboard, you can switch to the **DBeaver Keyboard Only** scheme, which has additional shortcuts.

Learn more about DBeaver shortcuts

Colors and fonts

Themes

DBeaver supports theme customization to enhance your user experience. Among the available options, DBeaver provides integration for the High-Contrast theme native to Windows systems.

> For more details on this and other themes, see our comprehensive theme guide.

Fonts

To change the interface fonts:

1. Open **Window -> Preferences -> User Interface**.
2. Select the edit icon next to **Main font** or **Monospace font**.
3. Choose a font in the dialog.

To increase the application font size, select the edit icon next to **Main font**, choose a larger size in the font dialog, and select **OK** in **Preferences**.

To open the Colors and Fonts settings, select **Colors and Fonts** on the same page.

Data Colorization

You can color rows in the **Data Grid** by following some rules or even by selecting colors for particular data types.

DBeaver Views and Wizards

Let's see some comments on using the most popular **Views** in DBeaver with the keyboard and screen reader.

Connections

In the **Connections**, you can view and open all the database connections and objects inside them. You can move between database objects and open nodes using arrows.

- To view object properties, use `F4`.
- To open **SQL Editor** associated with the selected object, use `F3`.

See all **Connections** shortcuts

New Connection

To create a new connection, open the **Wizard window** using `Ctrl+N` | `⌘N`, then type **connect** and select **Database connection**.

See **Connection** shortcuts

Object Properties

The **Properties Editor** is used to see all the information about database objects. When you focus on the first row of the table, NVDA reads the contents of the row with the corresponding column names.

- You can open object properties with `F4`.
- To switch between left-side tabs, use `Alt+Shift+Up` | `Alt+Shift+Down`.

See **Properties Editor** shortcuts

SQL Editor

The **SQL Editor** allows you to create and execute SQL scripts associated with a database connection. To open the **SQL Editor**, focus on the connection or table in the **Connections** and press `F3`. To execute the SQL script under the cursor, press `Ctrl+Enter`.

- See **SQL Editor** shortcuts

Result Set

When you place your focus on the first cell in the data table, the NVDA screen reader says the table name and the name of a column consisting of this cell. If you move from one cell to another, the screen reader announces the column names when you change focus.

If you need to work with columns using the keyboard, you can select an entire column, copy the column name, and resize the column using special keyboard shortcuts. Some of them are only available in the **DBeaver Keyboard Only** scheme.

See Result Set shortcuts

Diagrams

Diagrams are fully accessible with the keyboard and the NVDA screen reader.

- Use arrow keys to move between tables.
- Press `Spacebar` to select the current table.
- Press `.` to enter move or resize mode.
- Use arrow keys to move or resize the table, then press `Enter` to confirm.
- While a table is focused, the screen reader announces the table name and its columns.

> For the full list of shortcuts, see Diagram shortcuts.

Toolbar customization

Table of contents

- Top toolbar customization
- SQL Editor toolbar customization
- Data Editor toolbar customization

DBeaver offers highly customizable toolbars to enhance your workflow:

1. **Top toolbar**: This toolbar is the primary control panel located at the top of the DBeaver interface. It provides quick access to various database actions, transactions, and utilities.

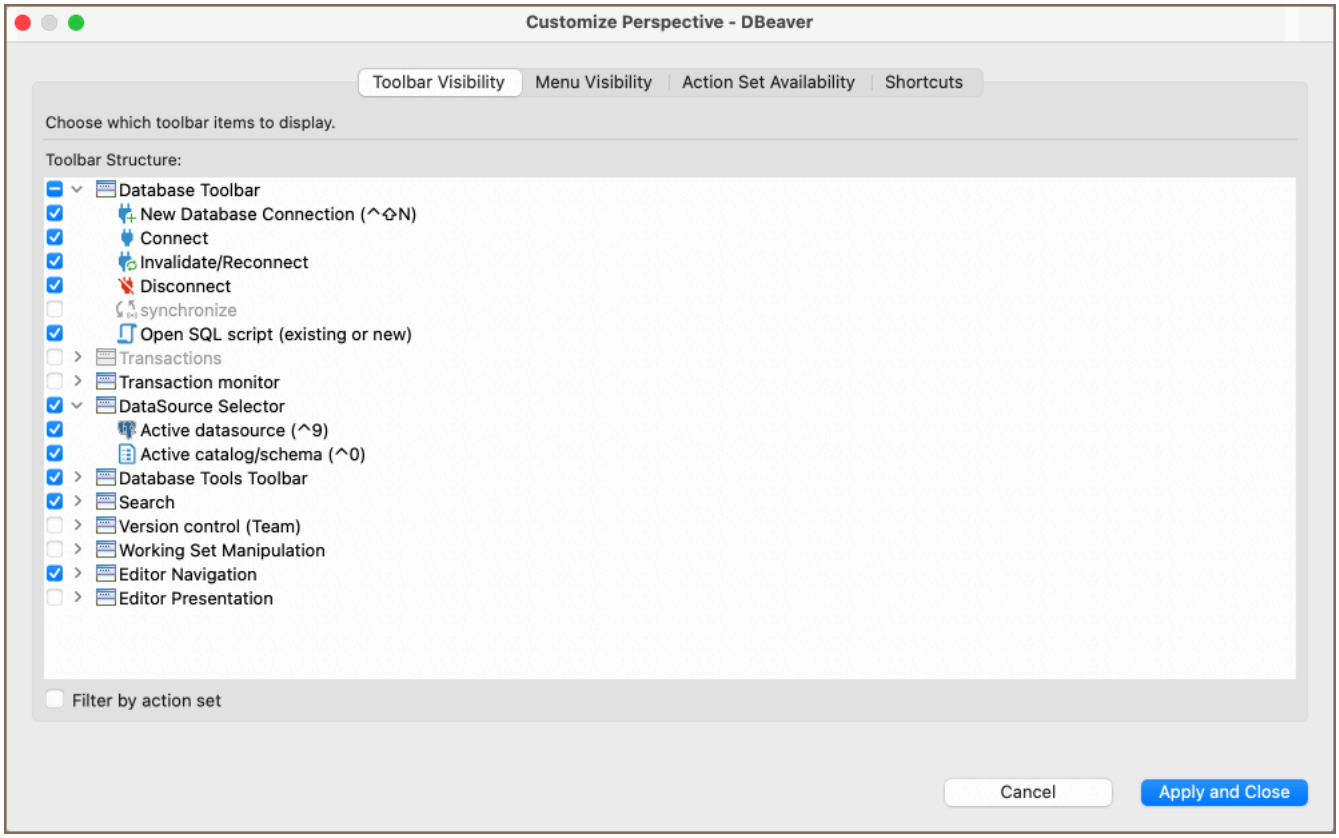
2. **SQL Editor toolbar**: Specifically designed for SQL editing, this toolbar appears when you are working within the SQL editor. It provides functionalities for executing SQL queries, navigating through your code, and other SQL-specific tasks.

3. **Data Editor toolbar**: This toolbar provides abilities for adding, editing, and exporting rows, for navigating and filtering data.

Both toolbars are designed for ease of use and can be customized to fit your specific needs. From basic database operations to advanced debugging and transaction controls, these toolbars can be tailored to streamline your database management activities.

Top toolbar customization

To access toolbar customization, navigate to **Window** -> **Customize Perspective** -> **Toolbar Visibility** tab.



You can customize the Top toolbar by adding or removing various sections or individual elements based on your specific needs. Below is a table summarizing the sections available for customization:

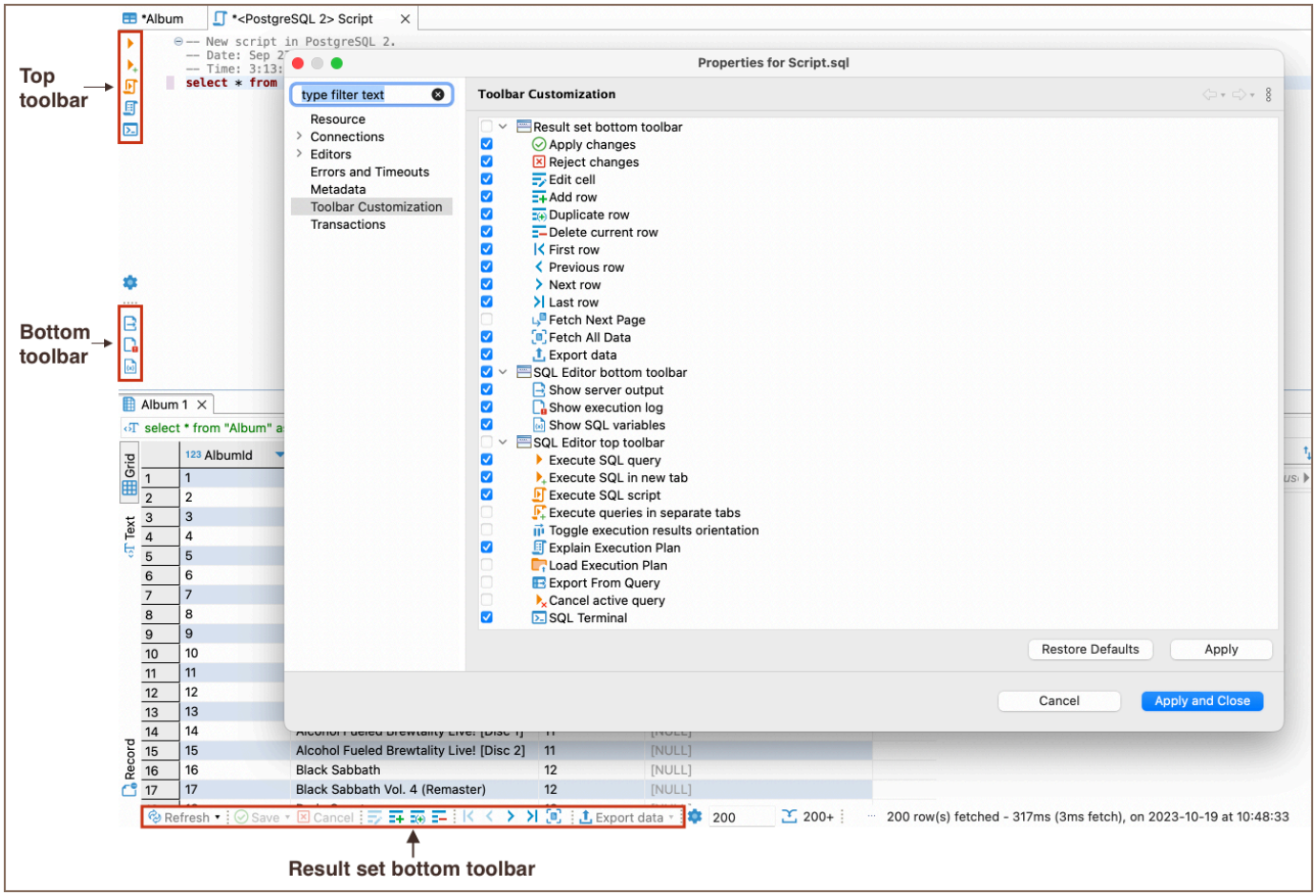
Section	Description
Database Toolbar	Manage database connections and navigation. For further details, see our Connections article.
Transaction	Control database transactions.
Transaction Monitor	Monitor transaction status and logs.
DataSource Selector	Choose active data sources and schemas. Additional information can be found in our article on Active Database.
Database Tools Toolbar	Access dashboards and views. Consult our article on Dashboards for more insights.
Launch	Execute, debug, and profile code.
Debug	Manage debug sessions and control execution.
Git	Perform Git-related activities.
Search	Enable quick search functionalities in code. Further reading is available in our Search article.
Version Control (Team)	Manage and synchronize with version control system. Learn more in our article about Project Team Work.
Working Set Manipulation	Manage working sets for organization.
Editor Navigation	Navigate through editor locations.
Editor Presentation	Customize text display settings in the editor.

> **Note**: Some toolbar sections may not be visible because all their child elements are in an unavailable action set. To make them available, go to the **Action Set Availability** tab and check the checkbox next to the required section.

SQL Editor toolbar customization

The SQL Editor toolbar in DBeaver can be customized to streamline your SQL development workflow. To access toolbar customization options, follow one of these methods:

- Navigate to **Window -> Preferences -> User Interface -> Toolbar Customization**.
- Right-click within the SQL Editor window and choose **Preferences -> Toolbar Customization**.



In the customization window, you can add or remove various sections and buttons. Below is a table summarizing the available sections:

Section	Description
SQL Editor bottom toolbar	Modify technical options at the bottom of the SQL Editor.
SQL Editor top toolbar	Customize query-related options at the top of the SQL Editor.

> **Note**: For detailed explanations of the button functionalities in the SQL Editor, consult our SQL Editor article.

Data Editor toolbar customization

Access the Data Editor toolbar customization options through the following steps:

- Go to **Window -> Preferences -> User Interface -> Toolbar Customization**.
- Alternatively, execute an SQL query in the SQL Editor, then in the Results tab, click on the Configure button and choose Toolbar Customization.

Within the customization menu, you can tailor the **Result set bottom toolbar** to fit your workflow. This section allows you to alter the tools for interacting with the displayed data sets.

Shortcuts

Table of contents

- Connection
- Result Set
- Result Set (Additional shortcuts for DBeaver Keyboard Only key scheme)
- Panels
- Data Editor
- Text Editor
- SQL Editor
- AI Chat
- Diagrams

- Additional association focus shortcuts
- Visual Query Builder
- Search
- Data viewer
- Window
- Connections
- Properties Entity Editor
- Navigation tree
- File
- Run/Debug
- Git
- Project
- Oracle
- Utility
- Help
- References

Here is a complete list of default hotkeys in DBeaver UI for Windows, Linux, and macOS users. It will help you work in DBeaver faster and more efficiently. Remember that you can always change the keyboard shortcut in the DBeaver settings. Go through: **Window -> Preferences -> User Interface -> Keys**. Select command and add a keyboard shortcut to the **Binding row**.

If you mostly use keyboard, to get more shortcuts you can switch default key scheme to **DBeaver Keyboard Only** key scheme in **Window -> Preferences -> User Interface -> Keys** settings.

Connection

Shortcut for Windows/Linux	Shortcut for macOS	Action
F4	F4	Open object editor
Ctrl+Shift+N	Ctrl+Shift+N	Open **Connect to a database** window
Ctrl+Shift+D	^⇧D	Open database meta-object
Ctrl+Alt+Enter	^⌥↩	Open a new SQL console. No script file will be created.
Ctrl+]	^]	Create a new SQL script
F3	F3	Open existing SQL script
Ctrl+Enter	⌘↩	Open most recent SQL script for an active connection
Ctrl+Alt+Shift+K	^4	Commit changes in current session
Alt+`	^V	Shows context menu with database tools
Ctrl+Alt+Shift+U	^⌥⇧U	Generates UUID/GUID and inserts into an active text editor or data editor
Ctrl+Alt+Shift+B	^⌥⇧B	Open database dashboard
Ctrl+Alt+Shift+R	^8	Rollback changes in current session
Ctrl+0	^0	Select active schema for current database
Ctrl+9	^9	Select connection for current editor
Alt+Shift+Down/Up	Alt+⌥+Down/Up	Switch vertical tabs

Result Set

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+Insert	⌥Insert	Add new row
Ctrl+D	^D	Copy values from row above to current row
Ctrl+Alt+D	^⌥D	Copy values from row below to current row
Alt+Delete	⌥⌦	Delete current row
Ctrl+Alt+Insert	^⌥Insert	Duplicate current row
Enter	↩	Edit cell value with inline editor
Esc	Esc	Reset cell to original value
Ctrl+S	^S	Apply data changes
Ctrl+Alt+Shift+T	^⌥⇧T	Set focus to filter editor/data editor
Ctrl+Shift+7	^⇧7	Switch focus to results viewer/active panel
Ctrl+Tab	Ctrl+Tab	Switch focus between the Grid view and the Value panel
Ctrl+Alt+Shift+Enter	^⌥⇧↩	Apply cell changes
Shift+Enter	⇧↩	Edit cell value in separate dialog/editor
Ctrl+Shift+=	^⇧=	Fetch all rows
Ctrl+Alt+N	^⌥N	Fetch next page of results
Ctrl+F11	^F11	Filter by unique attribute values
F11	F11	Filter context menu
Ctrl+Alt+Shift+Left	^⌥⇧←	Move to first row

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Alt+Shift+Right	^⌥⇧→	Move to last row
Alt+Space	⌥Space	Follow foreign key link
Ctrl+Alt+Right	^⌥→	Move to next row
Ctrl+Alt+Left	^⌥←	Move to previous row
Ctrl+Shift+1	^⇧1	Foreign keys and tables referencing current table
Ctrl+R	^R	Reject data changes
Ctrl+Backspace	^⌫	Set cell to default value
Ctrl+`	^`	Switch results presentation
Tab	Tab	Toggle results Grid/Record view
F7	F7	Toggle result panels
Ctrl+2	^2	Toggle results sort order (ascending/descending/default)
Ctrl+Alt+C		Select column(s)
Ctrl+Alt+R		Select row(s)
Shift+F11		Show context menu for column
Ctrl+G		Go to row
Ctrl+Shift+G		Go to column
Alt+9	⌘+⌥+9	Zoom out (decrease default font size)
Alt+0	⌘+⌥+0	Zoom in (increase default font size)

Result Set (Additional shortcuts for DBeaver Keyboard Only key scheme)

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+Shift+Right		Move column(s) right
Alt+Shift+Left		Move column(s) left
Alt+Shift+C		Copy column names
Alt+Shift+H		Hide columns
Alt+Shift+T		Show hidden columns
Alt+Shift+F		Open filters settings

Panels

Shortcut for Windows/Linux	Shortcut for macOS	Action
F7	F7	Toggle results panels
Ctrl+Alt+F6	^+⌥+F6	Show/hide results panel (Calc)
Ctrl+Alt+F5	^+⌥+F5	Show/hide results panel (Grouping)
Ctrl+Alt+F4	^+⌥+F4	Show/hide results panel (Metadata)
Ctrl+Alt+F3	^+⌥+F3	Show/hide results panel (References)
Ctrl+Alt+F2	^+⌥+F2	Show/hide results panel (Value)

Data Editor

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+Insert	⌥Insert	Add row after current row
Shift+Alt+Insert	⇧⌥Insert	Add row before current row
Ctrl+Alt+Insert	^⌥Insert	Duplicate row after current row
Ctrl+Shift+Alt+Insert	^⇧⌥Insert	Duplicate row before current row
Ctrl+Space	^Space	Enable autocomplete
Ctrl+F	⌘F	Find and replace text
Ctrl+Shift+Space	^⇧Space	Show Context Information in Dialogs and Windows
Ctrl+Alt+Space	^⌥Space	Show Context Information (SQL Editor Context)
Ctrl+X	⌘X	Cut the selection to the clipboard
Ctrl+C	⌘C	Copy the selection to the clipboard
Ctrl+V	⌘V	Paste from the clipboard
Delete	⌦	Delete the selection
Ctrl+O	⌘O	Export Diagram
Ctrl+K	⌘K	Find next item
Ctrl+Shift+K	⇧⌘K	Find previous item
Ctrl+J	⌘J	Incremental find

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Shift+J	⇧⌘J	Incremental find reverse
Ctrl+Shift+Q	^⇧Q	Toggles quick diff information display on the line number ruler
Ctrl+1	⌘1	Suggest possible fixes for a problem
Ctrl+Y	⇧⌘Z	Redo the last operation
Ctrl+Z	⌘Z	Undo the last operation
Shift+Delete	⇧⌦	Remove selected Pictogram Elements
Ctrl+A	⌘A	Select all
Alt+Shift+A	⌥⌘A	Toggle block/column selection in the current text editor
Ctrl+Shift+Insert	⇧⌘Insert	Toggle insert mode
Alt+Shift+Y	⌥⌘Y	Toggle word wrap in the current text editor
Ctrl+F5	⌘F5	Update selected Pictogram Elements
Alt+/	^.	Context insensitive completion
Ctrl+Shift+Space	^⇧Space	Context insensitive completion (SQL Editor Context)

Text Editor

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Shift+Y	⇧⌘Y	Changes the selection to lowercase
Ctrl+Shift+X	⇧⌘X	Changes the selection to uppercase
Ctrl+Numpad_Subtract	⌘Numpad_Subtract	Collapses the folded region at the current selection
Ctrl+Shift+Numpad_Divide	⇧⌘Numpad_Divide	Collapses all folded regions
Ctrl+Alt+Down	⌥⌘↓	Duplicates the selected lines and moves the selection to the copy
Ctrl+D	⌘D	Delete a line of text
Ctrl+Delete	⌥⌦	Delete the next word
Ctrl+Backspace	⌥⌫	Delete the previous word
Ctrl+Shift+Delete	⇧⌘⌦	Delete to the end of the line of text
Ctrl+Alt+Up	⌥⌘↑	Duplicates the selected lines and leaves the selection unchanged
Ctrl+Numpad_Add	⌘Numpad_Add	Expands the folded region at the current selection
Ctrl+Numpad_Multiply	⌘Numpad_Multiply	Expands all folded regions
Ctrl+Shift+Enter	⇧⌘↩	Adds a new line above the current line
Shift+Enter	⇧↩	Adds a new line below the current line
Ctrl+Alt+J		Join lines of text (Editing Text)
Ctrl+Shift+J	^⇧J	Join lines of text (SQL Editor Context)
End	⌘→	Go to the end of the line of text
Home	⌘←	Go to the start of the line of text
Alt+Down	^⇧↓	Moves the selected lines down (Editing Text)
Ctrl+Shift+Down	⌥↓	Moves the selected lines down (SQL Editor Context)
Alt+Up	^⇧↑	Moves the selected lines up (Editing Text)
Ctrl+Shift+Up	^⌥↑	Moves the selected lines up (SQL Editor Context)
Ctrl+Right	⌥→	Go to the next word
Ctrl+Left	⌥←	Go to the previous word
Ctrl+Shift+Numpad_Multiply	⇧⌘Numpad_Multiply	Resets the folding structure
Ctrl+Down		Scroll down one line of text
Ctrl+Up		Scroll up one line of text
Shift+End	⇧⌘→	Select to the end of the line of text
Shift+Home	⇧⌘←	Select to the beginning of the line of text
Ctrl+Shift+Right	⌥⇧→	Select the next word
Ctrl+Shift+Left	⌥⇧←	Select the previous word
	⇧↘	Select to the end of the text
	⇧↖	Select to the beginning of the text
F2	F2	Displays information for the current caret location in a focused hover
Ctrl+End	↘	Go to the end of the text
Ctrl+Home	↖	Go to the beginning of the text
Ctrl+Numpad_Divide	⌘Numpad_Divide	Toggles folding in the current editor
Insert	Insert	Toggle overwrite mode

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl++	⌘+	Zoom in text, increase default font size for text editors
Ctrl+=	⌘=	Zoom in text, increase default font size for text editors
Ctrl+-	⌘-	Zoom out text, decrease default font size for text editors

SQL Editor

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+X	⌥X	Execute SQL script
Alt+N	⌥N	Execute SQL script natively
Ctrl+Enter	⌘↩	Execute SQL statement
Ctrl+\	^\	Execute SQL statement in a new tab
Ctrl+Alt+Shift+X	^⌥⇧X	Execute script's statements in separate results tabs
Ctrl+/	^/	Add or remove single line comment
Ctrl+Shift+/	^⇧/	Add or remove multi line comment
Ctrl+Alt+Shift+T	^⌥⇧T	Switch active SQL editor panel
Ctrl+T	^T	Show/hide results panel
Ctrl+Shift+\	^⇧\	Close results tab
Ctrl+Shift+F	^⇧F	Format text
Ctrl+Alt+'	^⌥'	Select value of the selected SQL expression
Ctrl+Shift+E	^⇧E	Explain execution plan
Ctrl+Shift+[	^⇧[	Jump to the corresponding bracket
Ctrl+Shift+]	^⇧]	Select the text between brackets
Ctrl+Alt+Shift+O	^⌥⇧O	Load SQL script from file system
Ctrl+Shift+T	^⇧T	Maximize/normalize results panel
Alt+Down	⌥↓	Switch to the next query
Alt+Up	⌥↑	Switch to the previous query
Ctrl+F2	^F2	Rename current SQL script
F4	F4	Open editor of current (highlighted) database object
Ctrl+Alt+Shift+A	^⌥⇧A	Select and show all rows (no fetch size limit)
Ctrl+Alt+Shift+C	^⌥⇧C	Select row count for query under cursor
Ctrl+Shift+.	^⇧.	Set active connection from Connections selection
Ctrl+Shift+O	^⇧O	Show server output console
Ctrl+Alt+Shift+W	^⌥⇧W	Toggles text editor soft word wrap
Ctrl+Shift+I	^⇧I	Switch presentation to (AI Chat)
Alt+Ctrl+→	⌥+⌘+→	Trigger AI Auto Complete. Press →, `Tab`, or `Enter` to insert the suggestion. Press `Esc` to dismiss it.

AI Chat

These shortcuts work when the AI Chat view is focused.

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Enter	⌘↩	Send the prompt
Ctrl+L	⌘L	Focus the message input field
Ctrl+Shift+M	⇧⌘M	Focus the message list
Ctrl+U	⌘U	Attach a file to the chat
Ctrl+N	⌘N	Create a new conversation
Ctrl+Shift+Delete	⇧⌘⌦	Delete the current conversation
Ctrl+Shift+S	⇧⌘S	Open chat settings
Ctrl+Shift+F	⇧⌘F	Open the **Change scope** drop-down

Diagrams

Shortcut for Windows/Linux	Shortcut for macOS	Action
Arrows	Arrows	Navigate between tables
Shift + Arrows	⇧ + Arrows	Select multiple tables
Ctrl + Arrows + Spacebar	^ + Arrows + Spacebar	Add tables to selection
Spacebar	Spacebar	Select current table
. then Arrows + Enter	. then Arrows + ↩	Move or resize tables
Enter	↩	Focus on attributes

Shortcut for Windows/Linux	Shortcut for macOS	Action
BACKSPACE	⌫	Leave focus
Alt + 1	⌥/Fn + 1	Focus on the diagram
Alt + 2	⌥/Fn + 2	Focus on the palette
Alt + 3	⌥/Fn + 3	Focus/open the outline
Alt + 4	⌥/Fn + 4	Focus on the parameter view
Ctrl + Shift + Enter	^ + Shift + ↩	Open the selected table diagram
Ctrl+P	⌘ + P	Print diagram

Additional association focus shortcuts

Use these shortcuts to navigate between relationship lines of the currently selected table:

- | – focus the next association
- ? – focus the previous association
- \ – cycle through all associations of the selected table

> **Note**: On macOS, using move/resize mode (`.`) may require enabling Accessibility permissions in **System Preferences -> Security & Privacy -> Accessibility**.

Visual Query Builder

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+B	^B	Open Visual Query Builder
Ctrl+Shift+B	^⇧B	Show/hide generated SQL query text
Ctrl+Alt+PageUP/PageDown Ctrl+Shift+PageUP/PageDown	Ctrl+⌥+PageUP/PageDown Ctrl+⇧+PageUP/PageDown	Switch vertical tabs
Alt+1	⌥1	Changes focus to the main tab
Alt+2	⌥2	Changes focus to the palette
Alt+3	⌥3	Changes focus to the outline
Alt+4	⌥4	Changes focus to parameters
Alt+5	⌥5	Switches to Visual Query Builder settings part
Alt+6	⌥6	Switches to query preview

> **Important**: The shortcuts `Alt+1` through `Alt+6` are only effective in the DBeaver Keyboard Scheme.

Search

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+H	^H	Open the Search dialog
Ctrl+Alt+G	⌥⌘G	Searches the files in the workspace for specific text
Ctrl+Alt+Shift+F	⌥⇧⌘L	Quick search in Windows

Data viewer

Shortcut for Windows/Linux	Shortcut for macOS	Action
Tab	Tab	Cycle through hand tool and previously used tool
Ctrl+=	⌘=	Zoom In
Ctrl+-	⌘-	Zoom Out
Alt+Shift+Q, Q	⌥⌘Q Q	Shows a particular view
Alt+Shift+Q, B	⌥⌘Q B	Shows a particular view (Breakpoints)
Alt+Shift+Q, C	⌥⌘Q C	Shows a particular view (Console)
Alt+Shift+Q, L	⌥⌘Q L	Shows a particular view (Error Log)
Alt+Shift+Q, Z	⌥⌘Q Z	Shows a particular view (History)
Alt+Shift+Q, O	⌥⌘Q O	Shows a particular view (Outline)
Alt+Shift+Q, X	⌥⌘Q X	Shows a particular view (Problems)
Alt+Shift+Q, S	⌥⌘Q S	Shows a particular view (Search)
Alt+Shift+Q, Y	⌥⌘Q Y	Shows a particular view (Synchronize)
Alt+Shift+Q, V	⌥⌘Q V	Shows a particular view (Variables)

Window

Shortcut for Windows/Linux	Shortcut for macOS	Action
F12	⌘F12	Activate Editor
	⌘W	Closes the active Dialog
Ctrl+3	⌘3	Find actions. Quickly access UI elements
Ctrl+M	^M	Maximize/restore a state of active view or editor
Ctrl+F6	⌘F6	Switch to the next editor
Ctrl+F8	⌘F8	Switch to the next perspective
Ctrl+F7	⌘F7	Switch to the next view
Ctrl+Shift+F6	⇧⌘F6	Switch to the previous editor
Ctrl+Shift+F8	⇧⌘F8	Switch to the previous perspective
Ctrl+Shift+F7	⇧⌘F7	Switch to the previous view
Ctrl+E	⌘E	Open the editor drop-down list
	⇧F10	Show the context menu
Alt+Shift+F3	⌥⇧F3	Shows contribution information for the currently selected element
Ctrl+Shift+L	⇧⌘L	Show the key assist dialog
Ctrl+F10	⌘F10	Show the context menu for the ruler
Alt+-	⇧⌘F10	Show the system menu
Ctrl+F10	⌘F10	Show the view menu
Ctrl+Shift+E	⇧⌘E	Switch to an editor
Alt+F11	^⌘F	Toggles the window between full screen and normal
Ctrl+_	⇧⌘-	Split or join the currently active editor (Horizontal)
Ctrl+_	⇧⌘[	Split or join the currently active editor (Vertical)

Connections

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Shift+,	^⇧,	Link with editor
Ctrl+Alt+Shift+D	^⌥⇧D	Add Bookmark
Enter	↩	Opens task configuration
Ctrl+Shift+A	^⇧A	Set as default object

Properties Entity Editor

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+Shift+Up	Alt+Shift+Up	Switch to the next entity property left-side tab
Alt+Shift+Down	Alt+Shift+Down	Switch to the previous entity property left-side tab
Ctrl+Alt+PageUP/PageDown Ctrl+Shift+PageUP/PageDown	Ctrl+⌥+PageUP/PageDown Ctrl+Shift+PageUP/PageDown	Switch vertical tabs

Navigation tree

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+Left	⌘[	Move backward in the editor navigation history
Ctrl+Shift+Numpad_Divide	⇧⌘Numpad_Divide	Collapse the current tree
Ctrl+Shift+Numpad_Multiply	⇧⌘Numpad_Multiply	Expand the current tree
Alt+Right	⌘]	Move forward in the editor navigation history
Ctrl+G	^G	Go to a specified line of text in Windows
Ctrl+L	⌘L	Go to a specified line of text (Editing Text)
Ctrl+.	⌘.	Navigate to the next item
Ctrl+Alt+Right	^⌥→	Next edit location
Alt+F7	⌥F7	Switch to the next page
Alt+PageDown	⌥⇟	Switch to the next sub-tab
Ctrl+PageDown	^⇟	Switch to the next tab
Ctrl+Shift+R	⇧⌘R	Open an editor on a particular item
Ctrl+,	⇧⌘.	Navigate to the previous item
Ctrl+Alt+Left	^Q	Previous edit location
Alt+Shift+F7	⌥⇧F7	Switch to the previous page
Alt+PageUp	⌥⇞	Switch to the previous sub-tab
Ctrl+PageUp	^⇞	Switch to the previous tab

Shortcut for Windows/Linux	Shortcut for macOS	Action
Alt+Shift+W	⌥⌘W	Open the Show In menu

File

Shortcut for Windows/Linux	Shortcut for macOS	Action
F2	F2	Rename the selected item
F5	F5	Refresh the selected items
Ctrl+S	⌘S	Save the changes in current file
Ctrl+Shift+S	⇧⌘S	Save changes in all open files
Ctrl+F4	⌘W	Close the active editor
Ctrl+Shift+F4	⇧⌘W	Close all editors
Ctrl+N	⌘N	Open the New item wizard
Alt+Shift+N	⌥⌘N	Open the New menu
Ctrl+O	^O	Open a file
Ctrl+P	⌘P	Print
Alt+Enter	⌘I	Display the properties of the selected item

Run/Debug

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Alt+M	⌥⌘M	Add memory block
Ctrl+W	⌘W	Close the selected rendering
F11	⌘F11	Launch in debug mode
Ctrl+Z	^D	Send end of file
Ctrl+G	⌘G	Go to Address
Ctrl+N	⌘N	Add a new rendering
Ctrl+Alt+N	⌥⌘N	Show renderings from next memory monitor.
Ctrl+Shift+.	⇧⌘.	Load next page of memory
Ctrl+Shift+,	⇧⌘,	Load previous page of memory
Ctrl+F11	⇧⌘F11	Launch in run mode
Ctrl+R	⌘R	Resume and break when execution reaches the current line
Ctrl+Alt+B	^⌥B	Sets whether or not any breakpoint should suspend execution
F5	F5	Step into
F6	F6	Step over
F7	F7	Step return
F8	F8	Resume
Ctrl+F2	⌘F2	Terminate
Ctrl+Shift+B	⇧⌘B	Creates or removes a breakpoint
Ctrl+T	⌘T	Toggle visibility of the Memory Monitors Pane
Shift+F5	⇧F5	Toggles enablement of debug step filters

Git

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+Left	⌘←	Collapse Working Tree
Ctrl+#	⌥⌘3	Commit
Ctrl+C	⌘C	Copy Path to Clipboard
Ctrl+O	⌘O	Show the quick outline for a unified diff
F2	F2	Rename Branch
Ctrl+Shift+K	^⇧K	Commit changes to Git
Ctrl+Shift+U	^⇧U	Update changes from Git

Project

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+B	⌘B	Build all projects

Oracle

Shortcut for Windows/Linux	Shortcut for macOS	Action
Ctrl+F9	^F9	Compile

Utility

Shortcut for Windows&Linux	Shortcut for macOS	Action
Ctrl+Shift+C	^⇧C	Advanced Copy
Ctrl+Alt+Shift+C	^⌥⇧C	Copy data with settings in advanced copy
Ctrl+Shift+V	^⇧V	Advanced paste
CTRL+ALT+PAGE_UP CTRL+ALT+PAGE_DOWN		Switch between tabs in database object Properties view
Alt+Insert	⌘+N	Create Object It is available both in the **Connections** and the **Properties Editor**.

Help

Shortcut for Windows&Linux	Shortcut for macOS	Action
F1	F1	Open the documentation

References

* - Current query is the query under cursor or the selected text. Query is separated from other script queries by delimiter (; by default) or by empty lines.
** - Current script is a set of all queries in the current SQL file. If there is a text selection then only queries in this selection are processed. Queries are separated from each other with a delimiter (; by default).
*** - Current connection detected from active window and selection. If active (focused) window is SQL editor or database object editor then the current connection is the same as in this editor. If the active window is the Connections then the active connection is the "owner" connection of the currently selected element. In other cases there is no current connection and DBeaver will ask you to choose the connection explicitly.

Security

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Master password
- Secure Storage with Secret Providers
- Changing database password
- Password protection for Projects
- Master password for all your Projects
- Password for one project
- Secure authentication
- Kerberos support
- SSO authentication
- Predefined connections
- Connections import
- Read-only connections
- Users roles and permissions
- Configuring preferences
- Roles in Team Edition
- Centralized automatic updates
- License management

The level of security is one of the key questions for enterprises, and the DBeaver team pays a lot of attention to it. One of the best reasons to use PRO versions is to take advantage of its security tools and features, such as password protection, SSO authentication, teams and roles in Team Edition.

This article briefly describes the most important security options available in DBeaver PRO.

Master password

You can enhance security in DBeaver with the Master Password, which encrypts credentials and sensitive data using a secure storage system. This feature ensures that each user's credentials are individually protected by their own Master Password, tying the encryption specifically to their local environment.

> For more details, refer to the Managing Master Password article.

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials securely. For detailed setup instructions, see Secret Providers.

Changing database password

Users can change the current database password directly in DBeaver in the following databases:

- Cockroach
- Exasol
- Greenplum
- Netezza
- Oracle
- PostgreSQL
- Redshift
- Snowflake
- SQL Server
- Vertica.

> Oracle, PostgreSQL, and Netezza allow changing an expired password in DBeaver as well.

- How to change the user password

Password protection for Projects

Master password for all your Projects

You can protect all Projects in your local workspace with a Master password.

You can set this password and store it in DBeaver password provider or use a generated password from your local password provider (for instance, OS X Keystore Integration or Windows integration provider).

Learn more about the Master password for Projects

Password for one project

You can specify a password for any project to protect all the project's configurations.

Learn more about project password

Secure authentication

You can connect to databases using secure authentication via Kerberos or GCP, AWS, and Azure cloud services.

Kerberos support

Kerberos is an authentication protocol, the default authentication technology used in Microsoft Windows.

You can connect via Kerberos using keytab, kinit, or a password. Open the connection settings, choose one of the supported databases and select Kerberos as the authentication method.

Learn more about authentication via Kerberos

SSO authentication

Users can connect to all company services using only one login and password. This is possible if you use SSO - Single Sign-On authentication service.

You do not need to manage, store, and transfer user credentials. When a user connects to the database, DBeaver opens a web browser with SSO authentication.

DBeaver supports the following SSO authentication services:

- AWS SSO
- GCP SSO
- Azure

Predefined connections

Connections import

You can describe all available database connections in configuration files (in JSON format) or import from CSV or XML DBeaver.

Read-only connections

If you want to restrict users from editing connection parameters, you can protect them with passwords.

Users roles and permissions

Configuring preferences

You can customize users preferences before they run DBeaver. For example, you can set the default simple mode for all connections (to show only schemas and tables and hide all system and service objects).

How to manage preferences

Roles in Team Edition

The best way to manage user access, restrictions, and permissions is to use Team Edition.

Team Edition allows you to create users and assign them appropriate roles with predefined capabilities.

You can add Viewers and Editors to work with prepared data, Managers to prepare data for them, Developers to work with scripts and connections, and administrators to manage everything.

Centralized automatic updates

If your team works on Microsoft Windows, you can organize DBeaver mass updates in silent mode, without user input, using the Windows Installer command line options.

Learn more about silent install

License management

You can place the license file in the user's workspace or store it elsewhere, and specify the license path on the command line or in the DBeaver configuration file.

- Learn more about lisense management automation
- How to specify license path in config

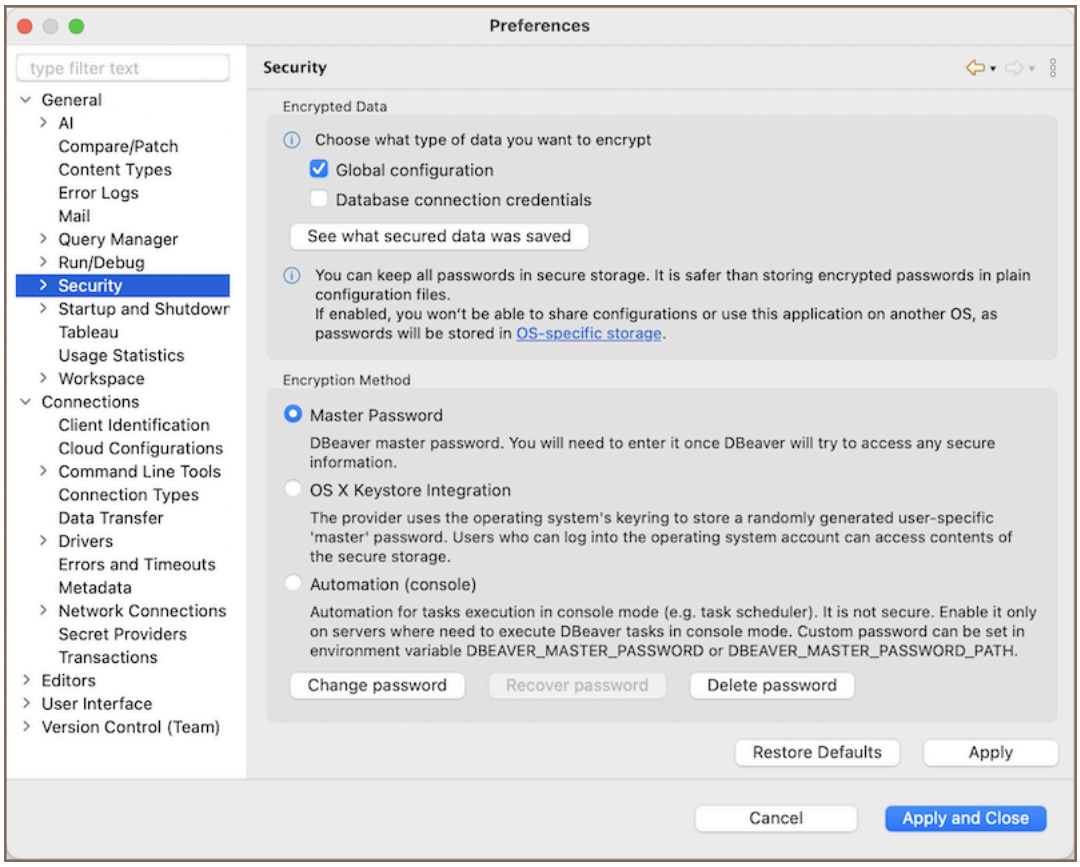
Security in DBeaver PRO

Secure storage

DBeaver has three password provider options:

- **Master password (recommended)** – The most secure option for protecting sensitive data. Learn more ★
- **Automation (console)** – Allows automated tasks to run without password prompts. Learn more ★
- **Integrated security** – Uses your operating system's security system to store passwords securely. Learn more

> **Note**: For the Community Edition, Integrated Security is the only available option.



Secret management

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Supported providers

DBeaver supports external secret managers to store connection credentials outside the application.

Instead of entering connection details manually, you store them in your secret provider and reference the secret by name when creating a connection. DBeaver retrieves the values at runtime and applies them automatically.

Typical workflow:

1. Create a secret in your external provider - see Secret requirements for field requirements.
2. Configure a secret provider in DBeaver - see Set up secret providers.
3. Create a connection and enter the secret name.

> **Tip**: See step-by-step example for creating and using a secret.

Supported providers

Secret manager	Description	Documentation
AWS Secrets Manager	Managed service from AWS for storing and retrieving secrets securely. Requires IAM permissions.	AWS documentation
AWS Systems Manager Parameter Store	AWS service for storing configuration data and secrets. Integrates with IAM and supports encryption via KMS.	AWS documentation
Azure Key Vault	Managed Azure service for storing and accessing secrets. Uses Microsoft Entra ID for access control.	Microsoft documentation
HashiCorp Vault	Open-source tool for managing secrets and protecting sensitive data. Supports fine-grained access control.	HashiCorp documentation
CyberArk	Enterprise-grade security platform designed for managing privileged access and secrets.	CyberArk documentation

Secret manager	Description	Documentation
Google Cloud Secret Manager	Managed service from Google Cloud for storing and accessing secrets. Uses IAM for access control.	Google Cloud documentation

> **Note**: AWS Secrets is available for Lite, Enterprise and Ultimate editions only.

Connection settings

Create connection

Table of contents

- Create a connection
- Preconfigure connections
- Connection settings reference
- Main settings
- Network settings
- Connection details
- Driver properties

Create a database connection from the **Connections** view.

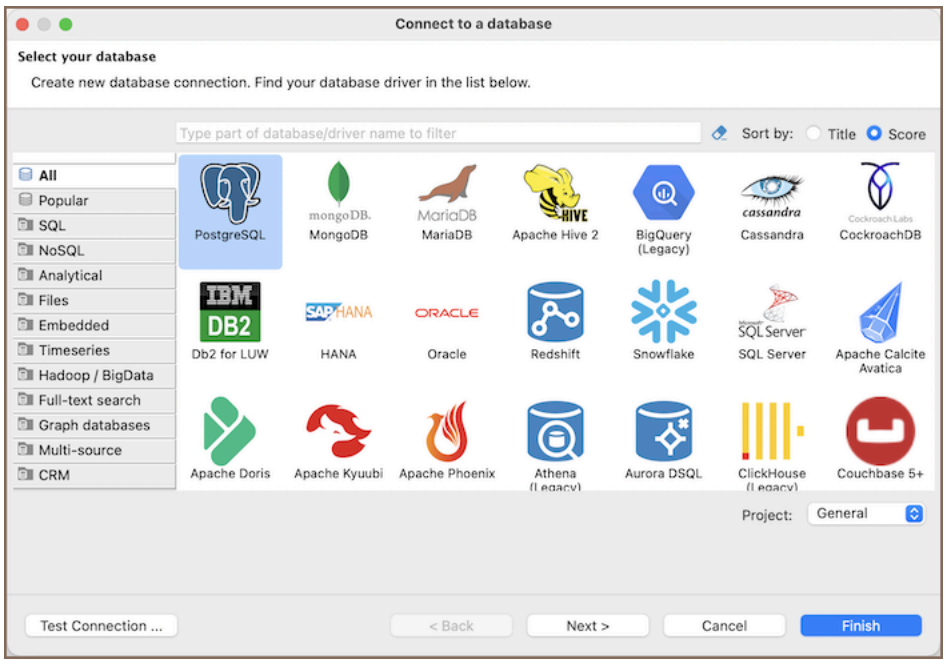
A database connection in DBeaver is a saved configuration that links the application to a database. It includes a driver, main connection fields such as the host, port, and database name, authentication settings, and optional network configuration.

> For information on configuring and managing an existing connection, see Configure connection.

Create a connection

1. Select the **New Connection Wizard** button in the toolbar or the Connections view toolbar.

   Alternatively, on Windows/Linux or macOS, press `Ctrl+Shift+N`.



2. Select a driver.

   To find a driver, browse the database sections, or search for a section using **Search**. You can also sort the list by **Title** or **Score**.

   > For information on managing drivers, see Driver manager.

3. Select **Next**.

4. Enter the main connection and authentication settings.
5. Optionally configure advanced settings.
6. If needed, select the target project for the connection.
7. Select **Test Connection** and **Finish**.

After you save the connection, it appears in **Connections** tree.

Preconfigure connections

Define connection settings in `data-sources.json` to preconfigure connections instead of creating each one manually.

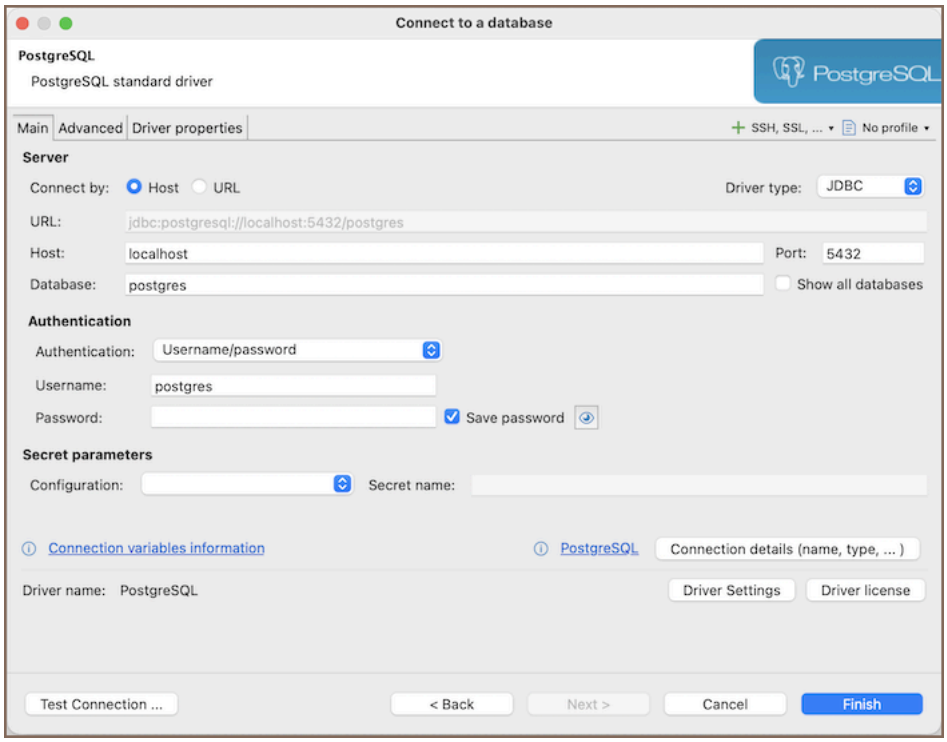
> For information on configuring connections in `data-sources.json`, see Data sources JSON reference.

Connection settings reference

Use the connection dialog to configure the fields and options for new and saved connections. The available settings depend on the selected driver.

Main settings

The **Main** tab contains the primary connection and authentication fields.



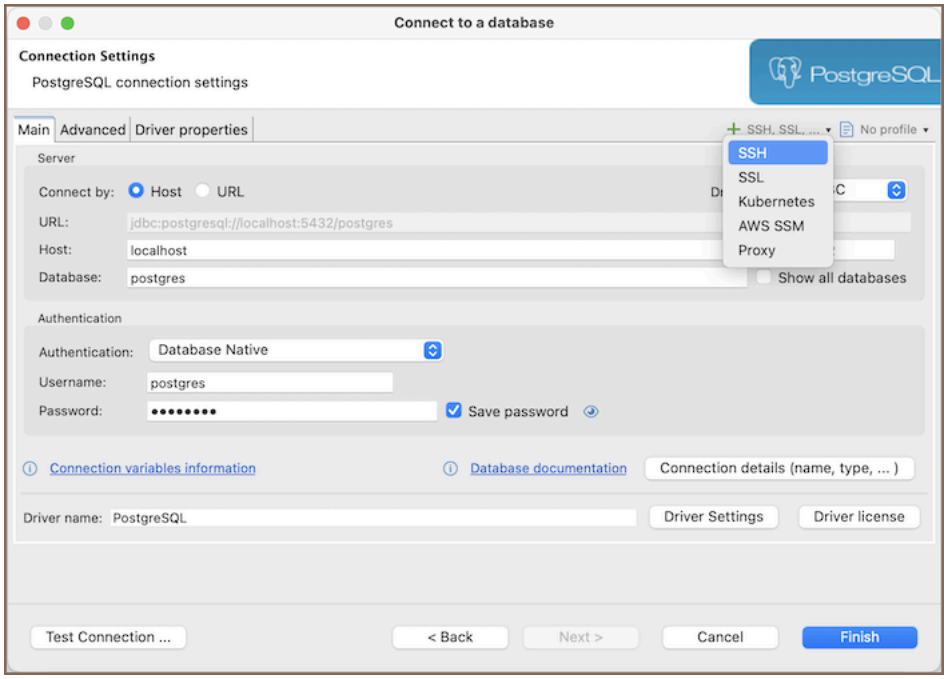
Field	Description
Host	Address of the database server
Port	Port used by the connection
Database name	Database to connect to
Username and password	Credentials used for authentication

> Embedded drivers can use a database path instead of host and port fields. Authentication options depend on the driver.

> For information on authentication methods, see:
>
> - Database username/password
> - Password from shell command
> - DBeaver profile
> - Microsoft Entra ID
> - Kerberos

Network settings

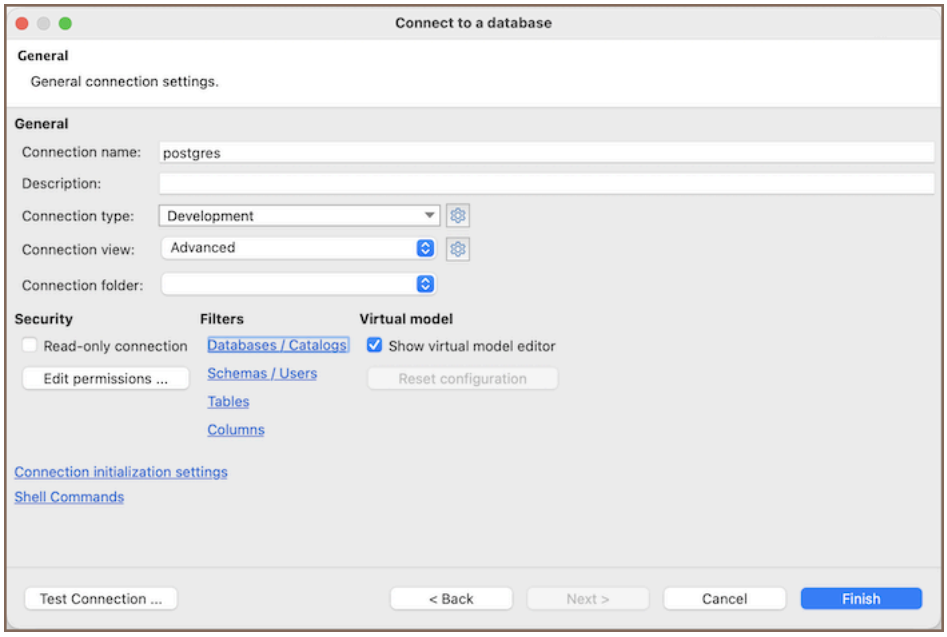
Configure network options in the connection dialog.



For information on network settings, see Network configuration.

Connection details

Click the **Connection details** button to open the **General** settings, which contain connection details and links to additional configuration options.



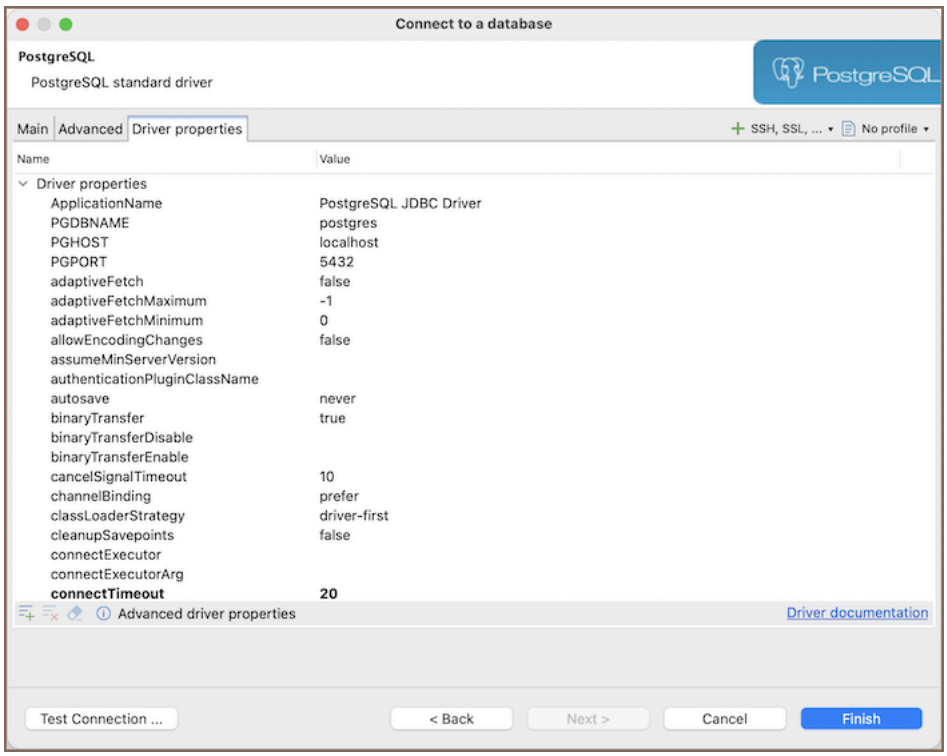
Setting	Description
Connection name	Identifies the connection. You can use variables in the SQL Editor, connection parameters, and driver properties. For information on variables, see SQL Editor
Connection type	Classifies the connection as **Development**, **Test**, or **Production**. **Development** is selected by default. For information on connection types, see Connection types
Description	Stores a description for the connection
Connections view	Selects the simple or advanced view in **Connections**. For information, see Simple and advanced view
Connection folder	Places the connection in a folder. For information, see Group database objects in folders
Security	Restricts read-only access, data editing, structural changes, script execution, and imports. For information, see Managing security restrictions for a database connection
Filters	Includes or excludes visible objects by name patterns. Filters can have a local or global scope. For information, see Configure filters
Virtual model	Defines virtual constraints and foreign keys, column transformers, and row colorings. For information, see Virtual keys
Connection initialization settings	Configures auto-commit, transaction isolation, the default database or schema, keep-alive and idle timeout, and bootstrap SQL. For information, see Configure connection initialization settings

Setting	Description
Shell commands	Runs commands before or after connecting or disconnecting. You can configure timing, the working directory, and wait or process handling. For information, see Working with shell commands in DBeaver

> **Tip**: You can configure separate connections for metadata reads and SQL editors. For information, see Separate connections.

Driver properties

Each driver provides additional properties. Use driver properties to configure driver-specific settings.



> For information on driver-specific settings, see Database drivers.

Network configuration

Table of contents

- Options

To connect to remote databases securely, DBeaver supports several network options. You can create encrypted tunnels, route traffic through proxies, or integrate with infrastructure tools like Kubernetes and AWS SSM.

Options

- **SSH**
- **SSL**
- **Proxy**
- **Kubernetes** ★
- **AWS SSM** ★

> **Tip**: Create network profiles to save and reuse settings across connections.

Auto and manual commit modes

Table of contents

- Control the transaction mode
- Manual commit mode
  - Executing statements
  - Apply or discard changes
  - Pending transactions
  - Transaction log
- Auto-commit mode

    - Executing statements
    - Smart commit mode
    - Transaction isolation level
- Default transaction mode
- Configure default transaction behavior

Transaction modes define how DBeaver manages database changes while you work with SQL. They control when transactions are finalized and when changes become visible to other users.

DBeaver supports two transaction modes: **Auto-commit** and **Manual commit**.

- In **Manual commit** mode, data-changing statements remain in an open transaction. You decide when to finish it. Click **Commit** to apply all pending changes or **Rollback** to discard them.
- In **Auto-commit** mode, each statement is committed immediately after execution. Changes are saved automatically and can't be rolled back.

Control the transaction mode

You can control the transaction mode in two ways:

- **Switch the transaction mode manually**
- **Select the default transaction mode for the connection**

> **Important**: By default, the transaction mode depends on the selected Connection type:
>
> - **Production** connections use **Manual commit**
> - **Development** and **Test** connections use **Auto-commit**

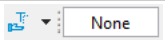
Manual commit mode

To enable manual commit while working in the SQL Editor or Data Editor, click the **Manual commit** button ( ) on the toolbar.

> Use **Manual commit** when you want full control over applying changes.

Executing statements

When you run SQL statements, DBeaver tracks all data-modifying operations inside the current transaction.

Indicator state	Meaning
	Pending modifying statements exist
	No pending changes

Apply or discard changes

Use action buttons to choose what happens with current transaction:

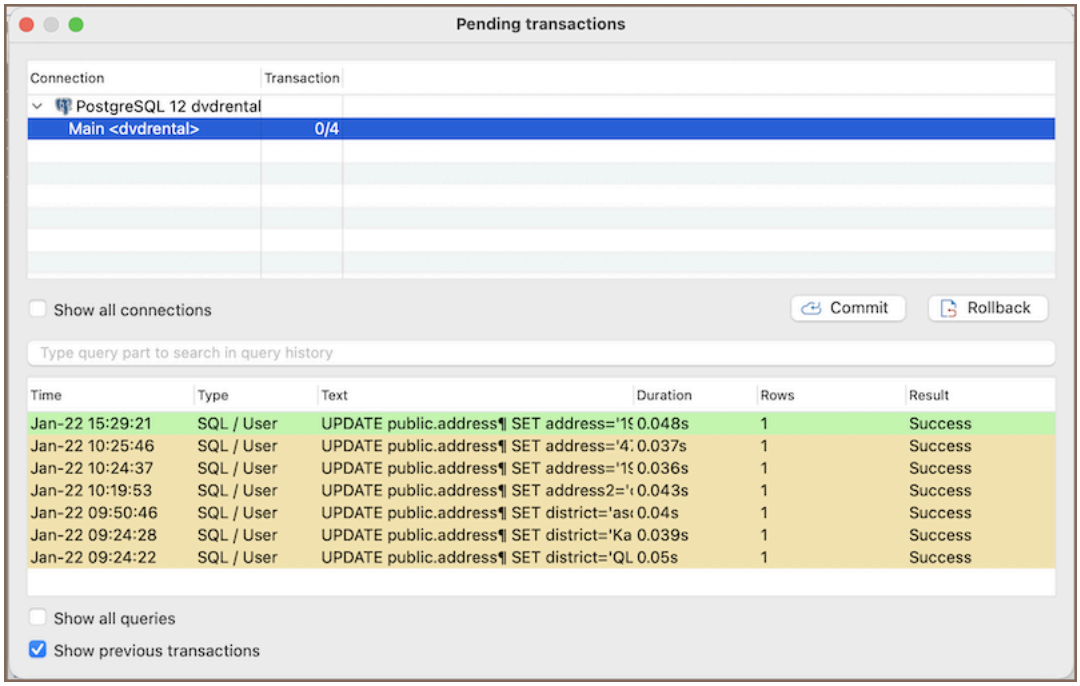
Action	Button	What it does
Commit		Applies all pending changes
Rollback		Discards all pending changes

To review all active transactions before applying or discarding them, use **Pending transactions**.

Pending transactions

This window shows all active transactions for the current session, including those from multiple connections.

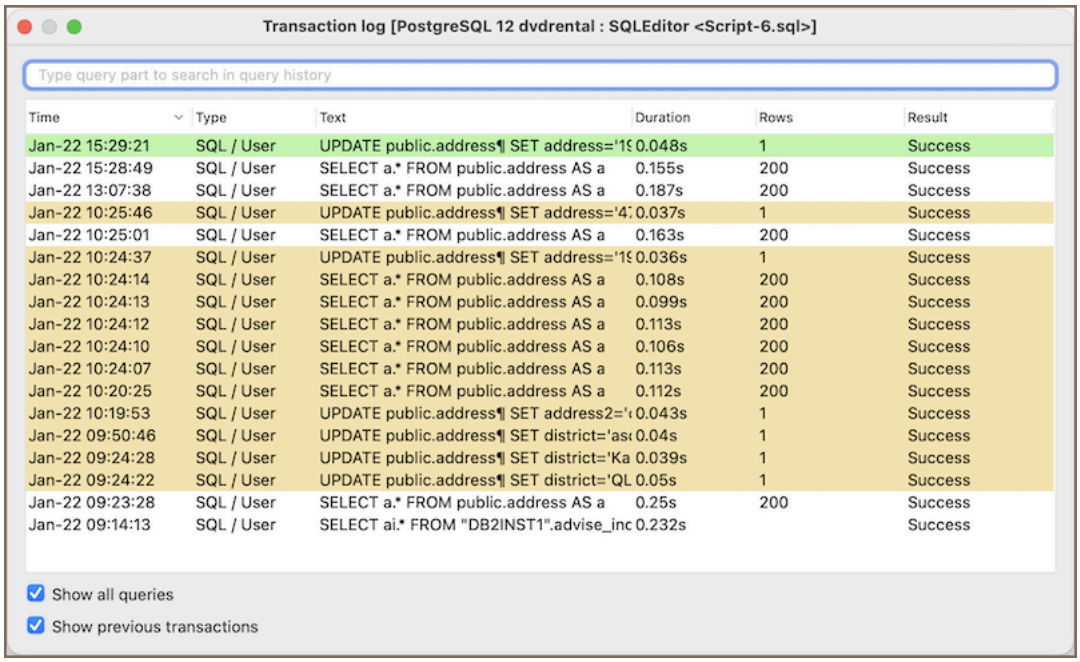
To open the Pending transactions, click the arrow next to the **Transaction Log** button ( ) in the toolbar and choose **Pending transactions**.

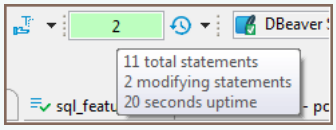


Transaction log

Use the **Transaction log** to review operations executed in the current session.

To open the Transaction log and review executed operations, click the **Transaction log** button ( ) in the toolbar.



> Alternatively, hover over the transaction indicator to see detailed statistics for executed statements:
>
> 
>

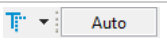
Auto-commit mode

> **Important**: Changes committed in **Auto-commit** mode applied immediately and can't be rolled back.

To enable **Auto-commit** while working in the SQL Editor or Data Editor, click the **Auto-commit** button ( ) on the toolbar.

Executing statements

DBeaver commits each SQL statement automatically.

Indicator state	Meaning
	Auto-commit mode is active

Smart commit mode

Use Smart commit for safer **Auto-commit** behavior. Smart commit reduces the risk of accidental data changes while keeping **Auto-commit** enabled.

When Smart commit is enabled and **Auto-commit** mode is active:

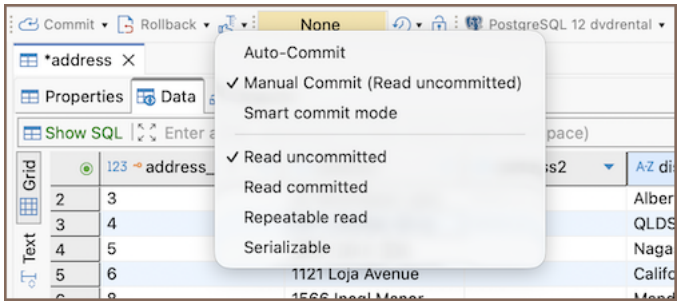
- `SELECT` queries run without starting a transaction
- When you execute a data-modifying statement (`UPDATE`, `INSERT`, `DELETE`, `UPSERT`, `MERGE`, etc), DBeaver switches to **Manual commit** before execution
- The same applies when you edit table data and save changes

> **Tip**: If **Return to Auto-commit on transaction end** is enabled in connection types preferences, DBeaver switches back to **Auto-commit** mode after **Commit** or **Rollback**.

Transaction isolation level

You can set the transaction isolation level for both **Auto-commit** and **Manual commit** modes.

Use the arrow next to the Transaction mode icon to select the required isolation level:

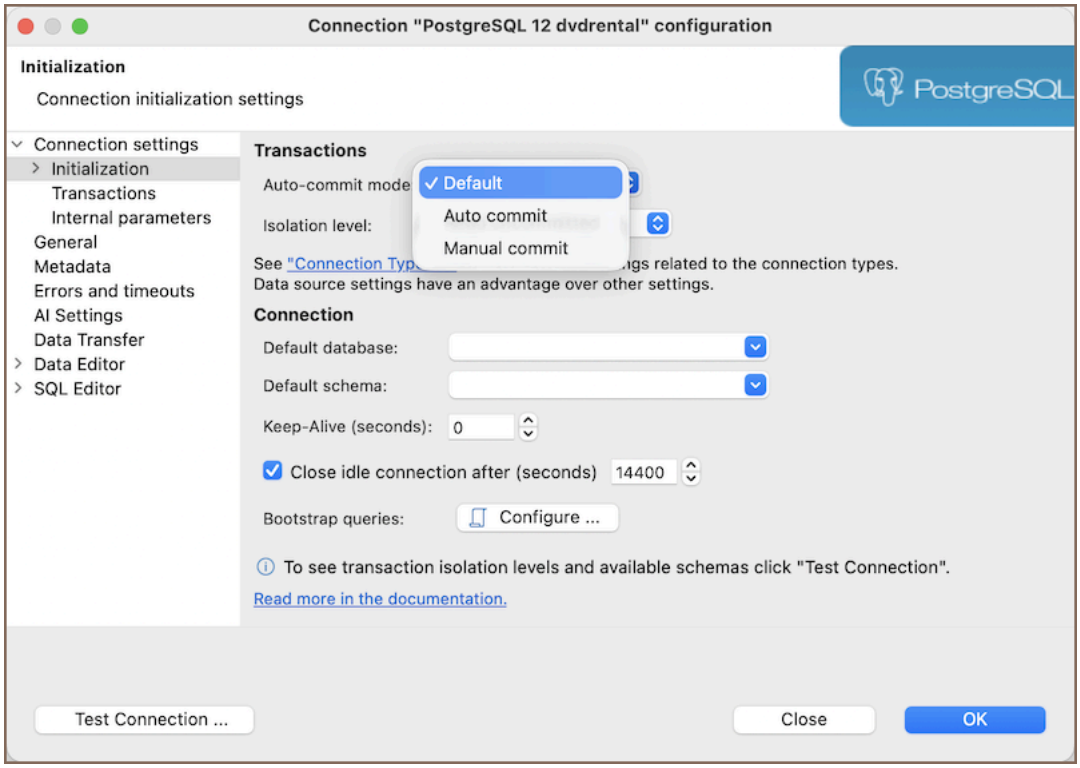


Setting	Description
Read uncommitted	Allows a transaction to read data that has not yet been committed by other transactions.
Read committed	Ensures a transaction only reads data that has been committed at the time the transaction began.
Repeatable read	Guarantees if a row is read twice in the same transaction, the result will be the same.
Serializable	Ensures transactions occur in a completely isolated fashion, equivalent to serial transaction execution.

Default transaction mode

To configure the default transaction mode:

1. Select the required connection in the Connections, then open its context menu and choose **Edit Connection**
2. Go to **Connection settings -> Initialization**



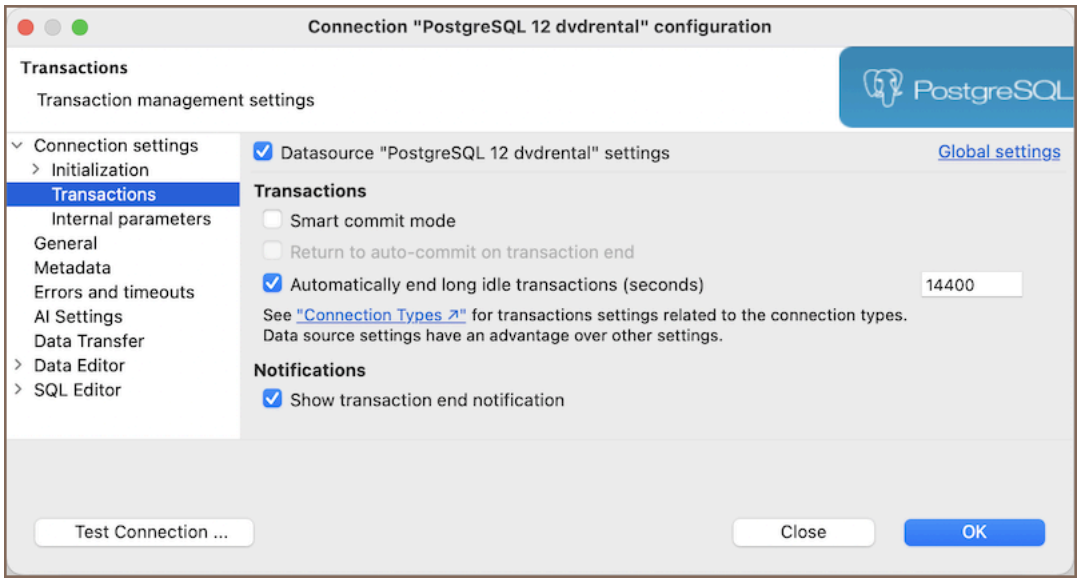
3. On the **Initialization** page, choose how the transaction mode is applied:
    - **Default**
    - **Auto-commit**
    - **Manual commit**

> The **Default** option follows the **Auto-commit by default** setting defined in the Connection type. To edit it, edit the connection type.

Configure default transaction behavior

To configure default transaction behavior:

1. Select the required connection in the Connections
2. Right-click the connection and choose **Edit Connection**
3. Open **Connection settings -> Initialization -> Transactions**.



Setting	Description
Smart commit mode	Enables safer **Auto-commit** behavior. Before executing a data-modifying statement, DBeaver switches to **Manual commit** mode.
Return to auto-commit on transaction end	Switches back to **Auto-commit** after a transaction ends with **Commit** or **Rollback**. Works only with Smart commit mode enabled.

Setting	Description
Automatically end long idle transactions (seconds)	Closes transactions after the specified idle time. No user activity is required before the transaction ends.
Show transaction end notification	Shows a notification when a transaction ends with commit or rollback.

> If you want different default behavior across multiple connections, configure it using Connection Types.

Driver Manager

Table of contents

- Obtaining a JDBC driver
- Driver Manager
- Add a new driver
- Main parameters
- Generic driver
- URL template
- Libraries
  - Maven artifacts
- Driver properties
- Advanced parameters
  - Main parameters
  - Queries
  - DDL
  - Formatting
- Saving the driver and adding a connection
- Driver configuration file
- Edit driver
- Change connection driver
- Update driver manually
  - Update driver via the UI
  - Update driver through the file system
- Update driver automatically
- Contribute a driver configuration

DBeaver supports many types of databases, including SQL, NoSQL, and others, by using pre-configured drivers to establish connections. If a specific database is not supported by default, you can manually add its JDBC driver through the Driver Manager.

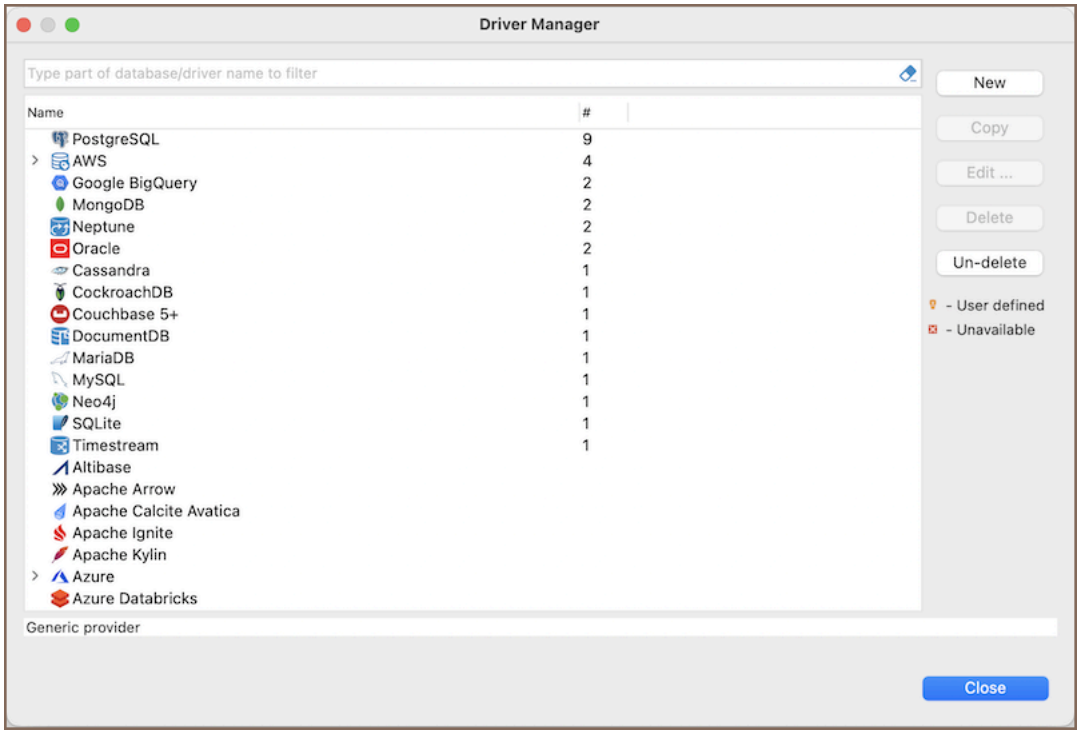
Obtaining a JDBC driver

A JDBC driver is a Java library that lets you connect and work with a specific database. It includes everything needed to use all the features of the database. These drivers are usually provided by the companies that create the databases.

JDBC drivers are made up of one or more `.jar` files. These files contain the code and other important files needed to use the driver. To add a new driver to DBeaver, you first need to get these `.jar` files. You can find them included with your database server or download them from the database provider's website. If you are not sure where to find them, ask your database administrator.

Driver Manager

To begin adding a new JDBC driver, first open the Driver Manager. You can find it by navigating to **Databases -> Driver Manager** from the main menu.



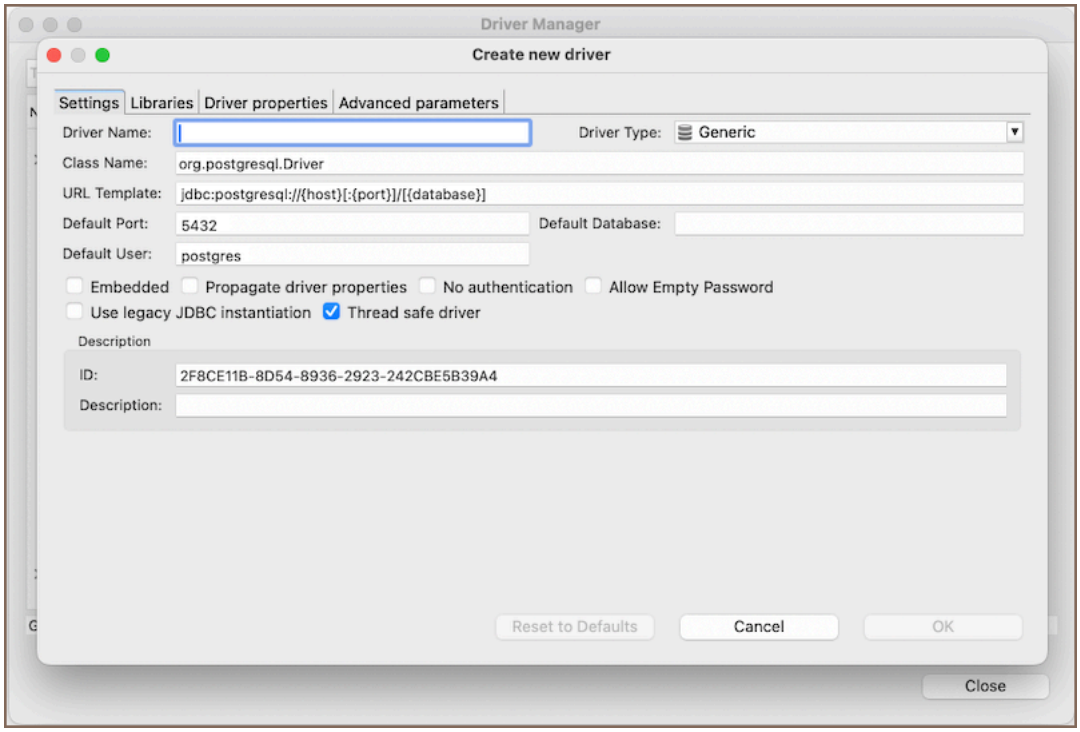
Button	Description
New	Creates a new driver configuration.
Copy	Copies the selected driver configuration.
Edit...	Opens the selected driver configuration for editing.
Delete	Removes a custom driver configuration permanently. For built-in drivers, this only hides them from the list - they aren't deleted.
Un-delete	Opens a list of all hidden built-in drivers. Select the ones you want to restore, the rest stay hidden. There's no time limit.

> **Important**: Deleting a custom driver is permanent. It can't be restored.

Add a new driver

Click the **New** button to start adding a new driver. In the driver edit dialog, you will need to fill all the necessary details to configure the driver.

Main parameters



Parameter	Description
Driver Name	The name you assign to the driver. This can be any descriptive name that helps you identify the driver among others.
Driver Type	Specifies the type of the driver used for connecting to the database. Users typically select the appropriate driver type based on the database being connected to. For unsupported databases, the **Generic** type is often used. For more information see section below.
Class Name	The fully qualified name of the Java class for the JDBC driver. This is the main class that Java uses to interact with the database. It can often be found in the driver's documentation or by inspecting the `.jar` file.
URL Template	A template for the database connection URL. If left empty, you must specify the JDBC URL manually for each connection. It's recommended to provide a template to simplify connection setup. For more information see section below.
Default Port	The port number typically used by the database. DBeaver pre-fills this value when you create a new connection, but you can change it in the connection settings. This is optional and can be left blank if the port is non-standard or varies between installations.
Embedded	Check this option if the database runs embedded within an application. This setting adjusts network and connection management configurations accordingly.
No Authentication	Indicates that the driver does not use authentication. If selected, user and password input fields will be hidden in connection dialogs.
Category	Used to group drivers within DBeaver. This field is deprecated but may still appear in some versions of the software.
ID	A system-generated unique identifier for the driver. This is usually auto-filled.
Description	A brief description of the driver.

Generic driver

The **Generic** driver allows you to connect to databases that do not have a predefined JDBC driver. It works with any database that supports JDBC, giving you control to manually load the required JDBC library.

This driver is useful for connecting to custom or uncommon databases. It ensures flexibility in setting connection parameters, but you must ensure that the JDBC driver you provide is fully compatible with the database to avoid issues with connections or queries.

URL template

JDBC drivers use URLs to connect to databases, similar to web URLs. These URLs typically follow the format `jdbc:vendor:host:port/database`. For example, a PostgreSQL database might use `jdbc:postgresql:localhost:5432/postgres`. Directly editing these long and complex strings can be prone to errors and inconvenient.

DBeaver simplifies this process by constructing the URL from connection parameters like host, port, and database name, which you provide on the connection configuration page. For the example mentioned, the URL template would be: `jdbc:postgresql://{host}:{port}/{database}`

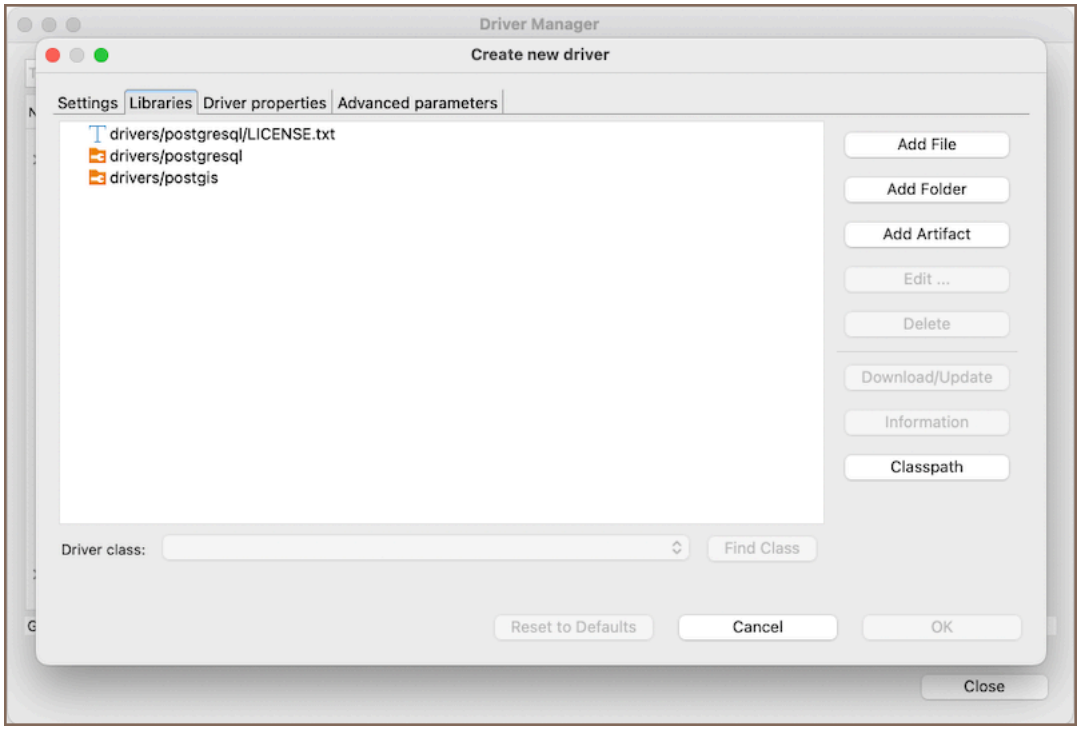
> For more information on the available variables, see Pre-configured variables.

The template only covers connection basics like host, port, and database. If your driver supports additional JDBC URL properties (such as `ssl=true` or `serverTimezone=UTC`), append them manually to the URL on the connection configuration page.

> **Tip**: SSL certificates aren't configured at the driver level. Learn more how to configure SSL Configuration.

Libraries

The Libraries tab displays a list of `.jar` files, binary libraries (such as `.dll` or `.so` files), and other files necessary for the driver. Typically, you only need to add `.jar` files here.



Button	Description
Add File	Adds a single `.jar` file. Useful for including individual JDBC driver files or additional libraries needed.
Add Folder	Adds an entire directory that contains Java classes or resources, facilitating bulk additions.
Add Artifact	Allows for the inclusion of a Maven artifact directly, streamlining dependency management for users who utilize Maven. For more information see the section below.
Edit...	Allows you to modify the selected file or folder properties.
Delete	Enables you to remove a selected file or folder from the list.
Download/Update	Allows for the downloading or updating of the selected library file or artifact.
Information	Provides detailed information about the selected file or artifact, such as version, source, and dependencies.
Find Class	Searches the added `.jar` files for JDBC driver classes and displays all found classes, enabling you to select the correct one for your configuration.
Classpath	Displays the full classpath constructed from the added files and folders, useful for debugging or verification purposes.

Once you have added the necessary `.jar` files, you can locate the JDBC driver classes contained within these files. Click the **Find Class** button, and DBeaver will list all available driver classes found in the added `.jars`. Usually, there is only one driver class per driver, but if multiple classes are listed, you should consult the driver's documentation to identify the correct one.

> **Tip**: If a connection using this driver fails with an error like "Most likely required jar files are missing," one of the driver's dependent `.jar` files is missing from the **Libraries** tab. Add the missing file and try again. The same message can also appear if a `.jar` was built for a newer Java version than the one running DBeaver. If all the files are present and the error persists, check the driver's required Java version.

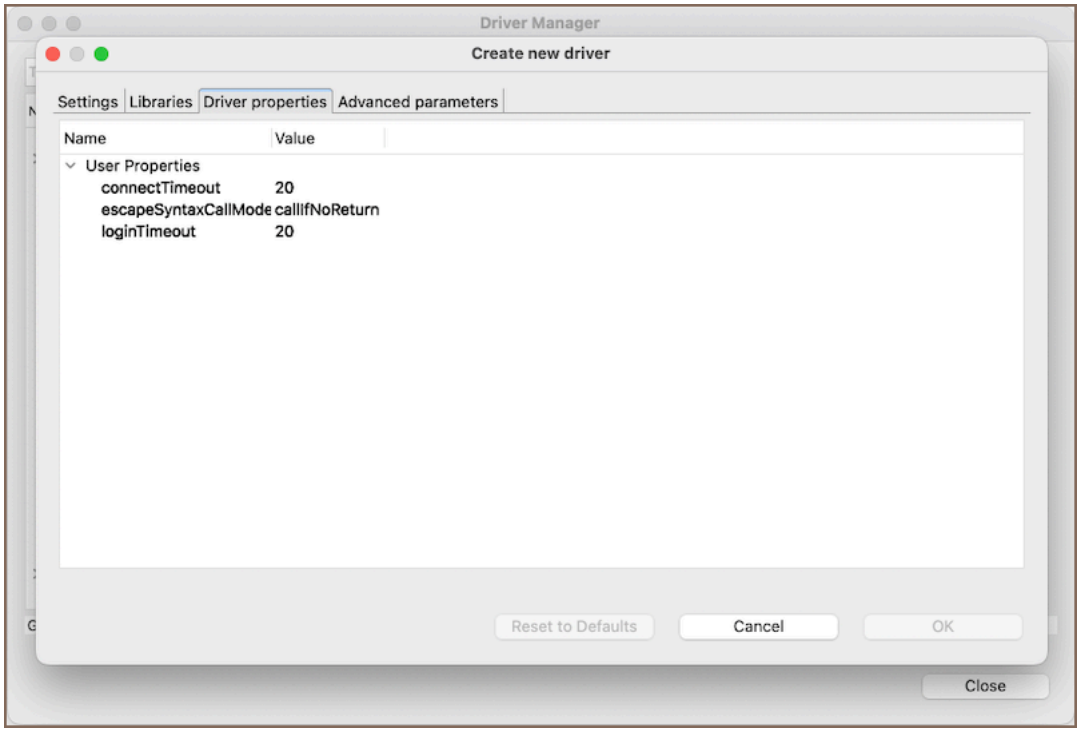
Maven artifacts

DBeaver can download driver `.jars` from the Maven repository, a global collection of Java libraries. If your database driver is listed in this public repository, you can use this feature. Maven artifacts are useful because they let you see all driver versions and change the driver version during runtime without reconfiguring driver properties.

> For information on updating or changing the driver version, see How to update driver version. For additional information, see How to add additional artifacts to the driver.

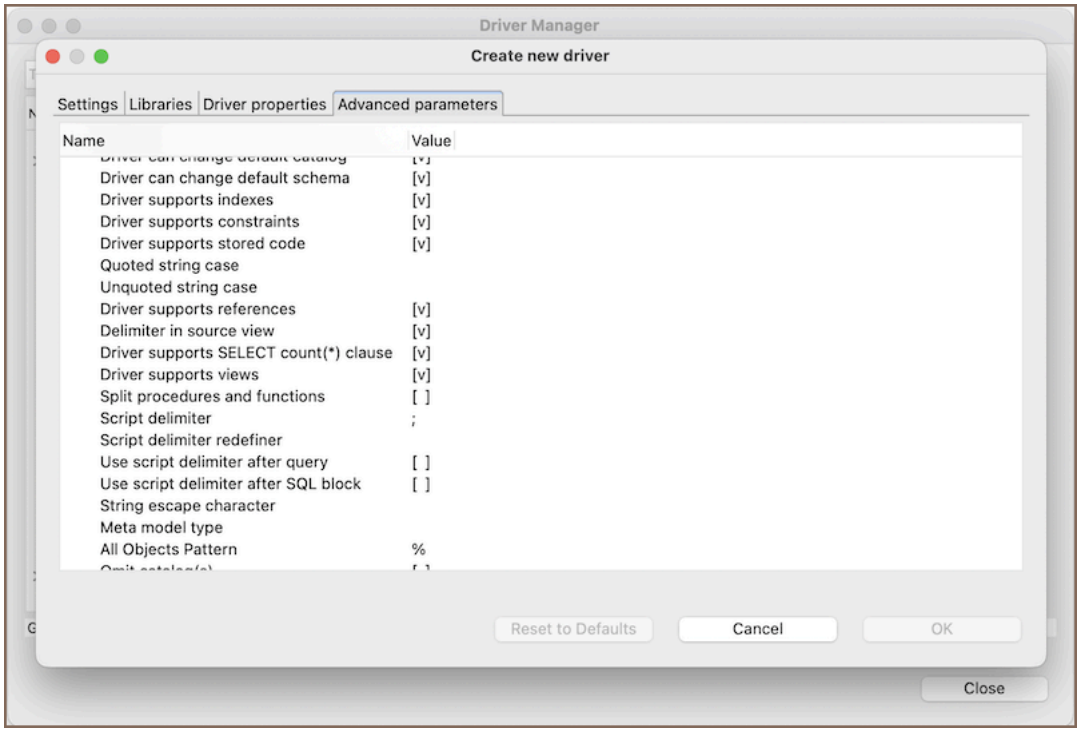
Driver properties

The Driver Properties tab displays default connection properties related to the JDBC driver once a driver file is loaded. This tab allows for the configuration and customization of various driver-specific settings necessary for establishing and managing database connections.



Advanced parameters

For most JDBC drivers, the default advanced properties suffice. However, you may find it beneficial to adjust these settings for performance optimization or to correct structural issues.



Main parameters

Parameter	Description
Driver supports indexes	Indicates if the driver can utilize table indexes.
Driver supports stored code	Supports stored database objects like procedures, functions, and packages.
Driver supports references	Allows the driver to handle table references such as foreign keys.
Driver supports SELECT count(*) clause	Supports the use of the SELECT count(*) SQL clause.
Driver supports views	Enables the driver to manage table views.
Split procedures and functions	Displays procedures and functions in separate folders in the UI.
Script delimiter	Character or string used to separate SQL statements in scripts.
Script delimiter redefiner	SQL clause that changes the script delimiter during execution.

Parameter	Description
Use script delimiter after query	Retains the delimiter after each SQL query within scripts.
Use script delimiter after SQL block	Maintains the delimiter after SQL blocks like `BEGIN/END`.
String escape character	Character used to escape special characters in SQL strings.
Meta model type	Defines the metadata model type, either standard or indexed.
All Objects Pattern	SQL pattern to match all metadata objects.
Omit catalog(s)	Skips catalog (database) information in metadata operations.
Omit single catalog	Hides catalog in the UI if only one exists on the server.
Omit schema(s)	Omits reading schema information from metadata.
Omit single schema	Hides schema in the UI if only one exists.
Use schema filters	Applies JDBC schema filters for databases not supporting catalogs, or filters schemas client-side.
Omit type cache	Avoids using the driver's internal cache for data types.
Shutdown parameter	Parameter for shutting down the database through URL.
Create database parameter	URL parameter used to create a new database.
Driver supports multiple results	Supports multiple result sets per query.
Driver supports result set limit	Allows setting limits on the number of rows in a result set.
Driver supports structure cache	Uses cache for database structure such as columns and keys.
Driver supports TRUNCATE operation	Enables use of the `TRUNCATE` command, faster than using `DELETE`.

Queries

Parameter	Description
Get active database	Retrieves the name of the currently active database.
Set active database	Sets the active database during the session.
Shutdown database	Command to shut down the active database connection.
PING query	SQL query to check the connection state.
Dual table name	Specifies the name of a dummy table used for evaluating expressions.
Active object type	Defines the type of database object that can be selected (e.g., schema, catalog).
Driver supports results scrolling	Enables scrolling through results in the result set.
Quote reserved words	Automatically quotes SQL reserved words when they conflict with identifier names.
Escape LIKE masks in search queries	Escapes special characters in `LIKE` clauses during metadata searches.

DDL

Parameter	Description
Drop column short syntax	Uses a shorter syntax for dropping columns.
Drop column - use brackets	Encloses column names in brackets when dropping.
Use legacy SQL dialect for DDL	Uses a traditional SQL dialect for DDL operations.
Add COLUMN keyword in alter table query	Adds the `COLUMN` keyword in `ALTER TABLE` queries for clarity.

Formatting

Parameter	Description
Timestamp format	The format pattern for timestamp columns.
Date format	The format pattern for date columns.
Time format	The format pattern for time columns.

> For more information on a date/time format pattern, see DateTimeFormatter documentation.

Saving the driver and adding a connection

Once you have completed configuring your driver, click the **Ok** button to save the settings.

To verify it works, create a connection and use **Test Connection**.

Driver configuration file

DBeaver stores every driver definition in a file called `drivers.xml`, in the workspace under `.metadata/.config/`. The Driver Manager reads and writes it for you, so you rarely need to edit it by hand.

> For the file structure, the full attribute reference, and how to reference pre-installed `.jar` files or push a driver configuration to several installations at once, see Managing drivers configuration.

Edit driver

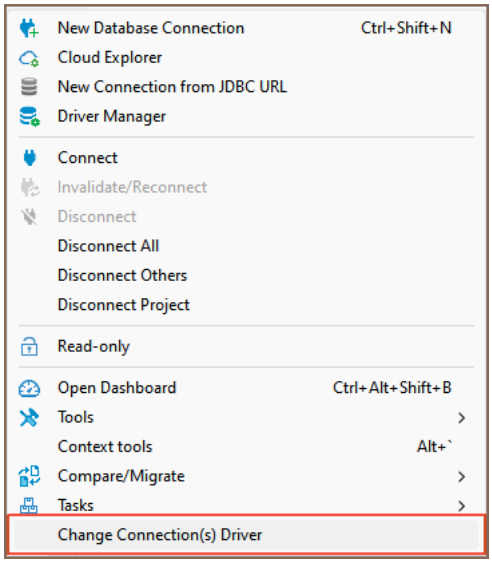
Editing an existing driver can be done through several access points:

- **Driver Manager**: Navigate through **Databases -> Driver Manager** in the main menu to edit any listed driver.
- **Connections**: Right-click on a database connection to open the context menu, then select **Edit Connection** to open the **Connection Properties** dialog. This method allows you to modify the driver settings linked specifically to that connection.

Change connection driver

You can change the driver version for an existing connection:

1. Select a connection in the Connections
2. Go to **Database** in the Menu bar
3. Click **Change connection(s) driver**, then select a different driver.



> **Note**: DBeaver uses two types of drivers: built-in and downloadable (Maven-based).
> You can only change the version for downloadable drivers. To update a built-in driver, update it manually.

Update driver manually

Update driver via the UI

1. In Connections, right-click your connection
2. Click **Edit Connection**
3. In the window that opens, click **Driver Settings**
4. Go to the **Libraries** tab
5. Double-click the driver file - this opens its folder

> If there's no driver listed (for example, you deleted it), click **Add File** or **Add Folder**, then select the driver file manually.

6. Replace the file with the new version. You need to download the `.jar` file manually. Sources vary depending on the driver:
   - **Official vendor site** - some drivers are only available on the database vendor's site.
   - **GitHub releases** - many open-source drivers publish binaries in the **Releases** section.
   - **Maven Central** - some drivers are available at search.maven.org.
   - **Other public repositories** - some projects host drivers on project-specific sites or custom registries.

> **Tip**: You don't need to delete the whole folder - just replace or remove old `.jar` files to avoid conflicts.

7. Click **OK**, then **Test Connection** to verify it works.

Update driver through the file system

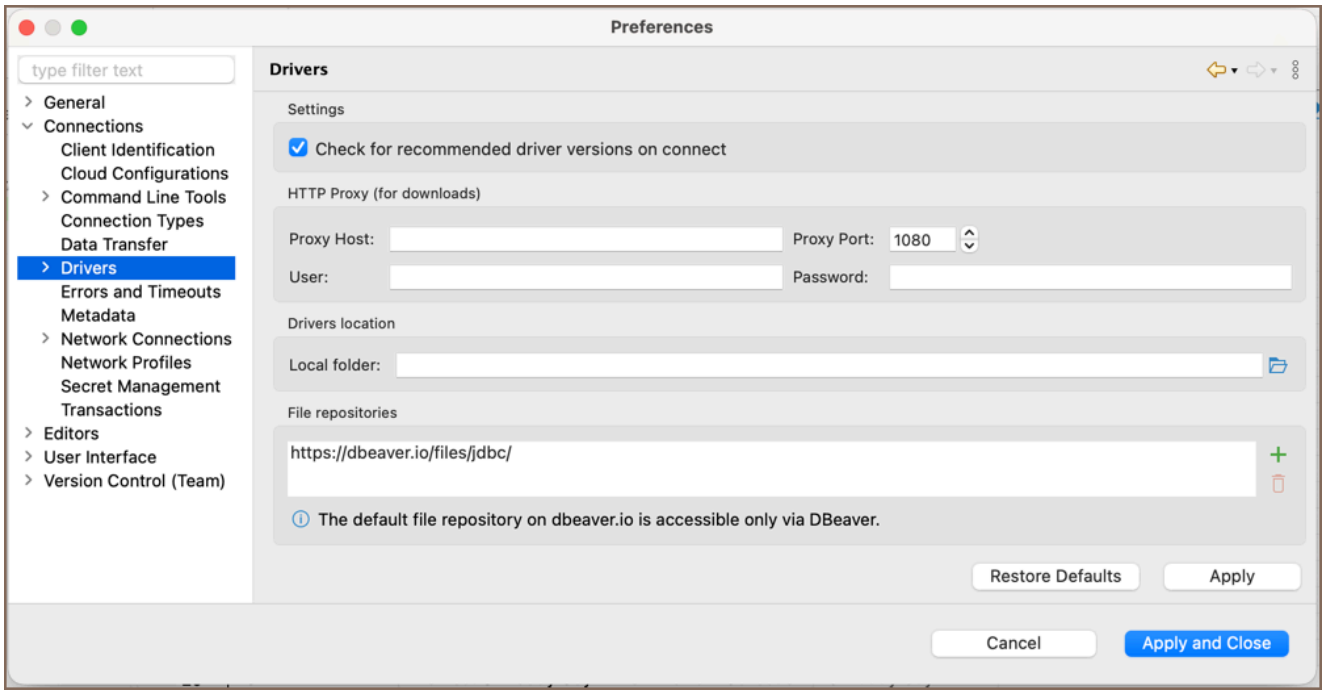
You can update a driver without opening the UI by replacing the `.jar` file directly on disk.

1. Find the driver folder. The location depends on the driver type - for details, see Driver files
2. Download the new `.jar` from the vendor's site, GitHub releases, or Maven Central
3. Replace the old `.jar` with the new one
4. Restart DBeaver.

Update driver automatically

You can configure your drivers to update automatically on startup.

1. Go to **Window -> Preferences -> Connections -> Drivers**



2. Configure driver-related settings:

Setting	Description
Check for new driver versions	Automatically checks for driver updates when DBeaver starts.
Proxy Host	Hostname of the proxy server used for driver downloads.
Proxy Port	Port number for the proxy.
User	Username for proxy authentication (if needed).
Password	Password for proxy authentication.
Local folder	Local folder where downloaded drivers are stored. You can specify a custom directory.
File repositories	URLs of remote repositories used for downloading drivers.

3. Click **Apply and Close**.

Contribute a driver configuration

If you have configured some driver, and it works good, and you think that it makes sense to have this driver configuration in DBeaver - please sent it us. Just create a feature request issue on GitHub and copy/paste driver description in the ticket (in any suitable form).

Thank you :)

Databases support

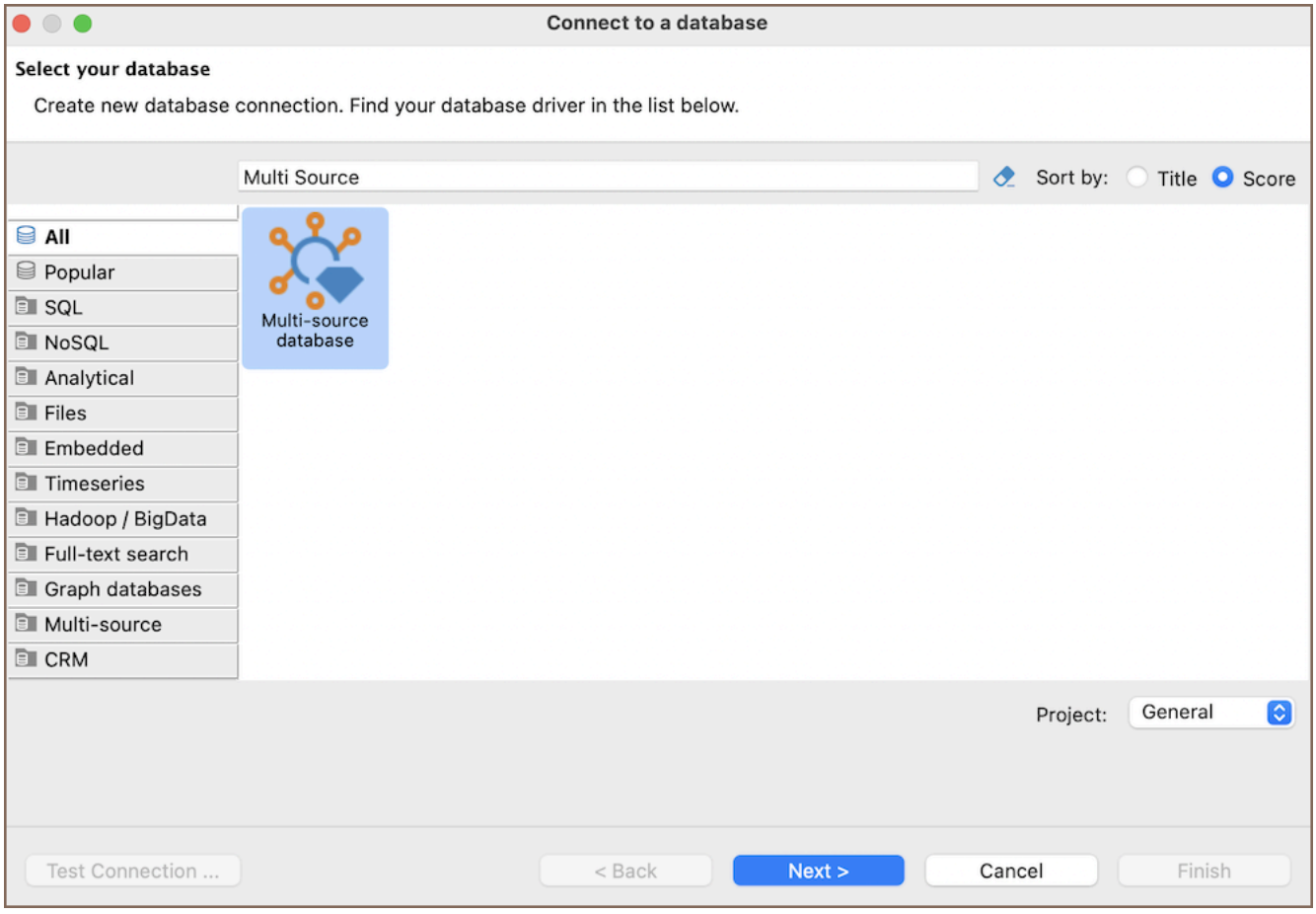
Multi Source Database

Table of contents

- Multi Source database specialty
- Setting up
- Driver properties
- Export schema
- Powering Multi Source Database with DBeaver
- Multi Source Database objects
- Multi Source Database features

> **Note**: This feature is available in Ultimate edition only.

DBeaver supports cross-database queries through Multi Source Database, giving you one SQL editor for data stored across multiple JDBC databases. Combine tables from different database systems in a single query without moving or copying the source data.



> **Important**: Multi Source Database is read-only.

Multi Source database specialty

Multi Source Database is based on Apache Calcite, which provides a virtual SQL layer over existing JDBC connections. In DBeaver, this layer acts as a multi-source connection to the schemas and tables you select.

The driver works with any JDBC data source configured in DBeaver. Use it to run read-only cross-database joins and analytical queries across different database systems. For example, one query can join a PostgreSQL table with a MySQL table.

Multi Source Database is read-only. You can query and join data from source connections, but you can't change source data or database objects through this connection.

> For information about the underlying query engine, see the Apache Calcite documentation.

Setting up

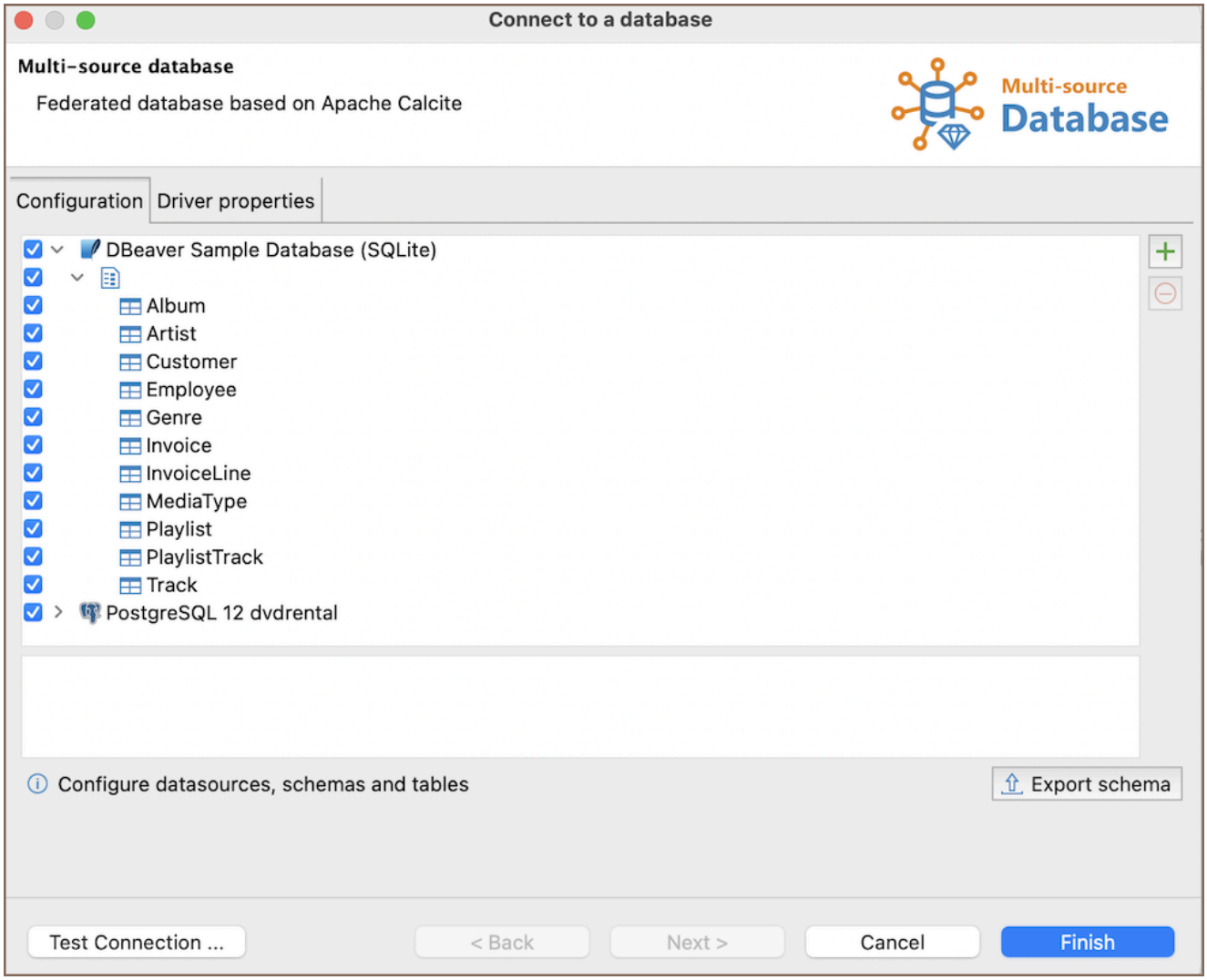
Set up each source connection before you create a Multi Source Database connection. A source connection is a regular database connection in DBeaver that provides data for Multi Source Database. Select an existing connection from Connections, or create and configure a new one. The multi-source connection reuses its settings and credentials.

> All source connections must belong to the same project. You can't add another Multi Source Database connection as a source.

1. Create and test each source connection.
2. On the toolbar, select **Create Multi Source Database** ( ).
    - Alternatively, click the **New Database Connection** button and select **Multi Source Database**.
    - Or select **Database -> Create Multi Source Database**.
3. On the **Configuration** page, select **Add database** ( ), choose a source connection, and confirm your selection.

    > To add a datasource from Connections, drag it to the configuration tree.

4. Expand each source connection and select the schemas and tables to include. Clear a connection checkbox to keep it in the configuration without using it.



> By default, a selected schema includes all its tables. After you select individual tables, the connection includes only that list. Add new source tables to the selection when you want to query them.

5. To change the Apache Calcite settings, open the Driver properties tab.
6. Select **Finish** to save the Multi Source Database connection.

Driver properties

Open the **Driver properties** tab to configure Multi Source Database. The following property is available:

Property	Default	Description
`join.transitive.predicates`	`true`	Infer filtering conditions from joins and push them to the source connections when possible

Export schema

Export the schema when you want to reuse the Multi Source Database configuration in an external Apache Calcite environment. The model describes the JDBC sources, schemas, and selected tables. The classpath file lists the JDBC driver libraries that Apache Calcite needs to connect to those sources.

To export the current configuration, select **Export schema** on the **Configuration** page:

1. Select `JSON` or `YAML` as the model format.
2. Select the export folder. DBeaver creates `calcite-model.json` or `calcite-model.yaml` and `calcite-model.classpath`, then opens the folder.

> **Important**: Exported Calcite models can contain the JDBC URL, user name, and saved password from the included connections. Store exported files securely.

Powering Multi Source Database with DBeaver

Use one SQL editor to query data from the selected source connections. Apache Calcite plans the query and sends applicable operations to the underlying connections.

Queries in a Multi Source Database connection follow Apache Calcite syntax. Source-specific SQL features might not be available in a cross-database query.

> **Example**:
>
> The following query joins customer data from SQLite with booking details from PostgreSQL:
>
> ```sql SELECT c."CustomerId", b."book_ref", b."book_date", b."total_amount" AS "flight_cost" FROM "SQLite"."Customer" AS c CROSS JOIN "postgresql_bookings"."bookings" AS b LIMIT 10;
>
> ```

Multi Source Database objects

DBeaver lets you browse the selected source objects in Connections:

- Schemas
  - Tables
    - Columns
  - Views
  - Procedures
  - Data Types

> **Note**: The connection doesn't expose indexes, table constraints, or references. Relationship-based tools don't have relationship metadata for Multi Source Database objects.

Multi Source Database features

DBeaver provides the following features compatible with Multi Source Database connections, but not exclusive to it:

Category	Feature
Data transfer	Data Import
	Data Export
Schema management	Schema Compare
Data Compare	Data Compare

Classic

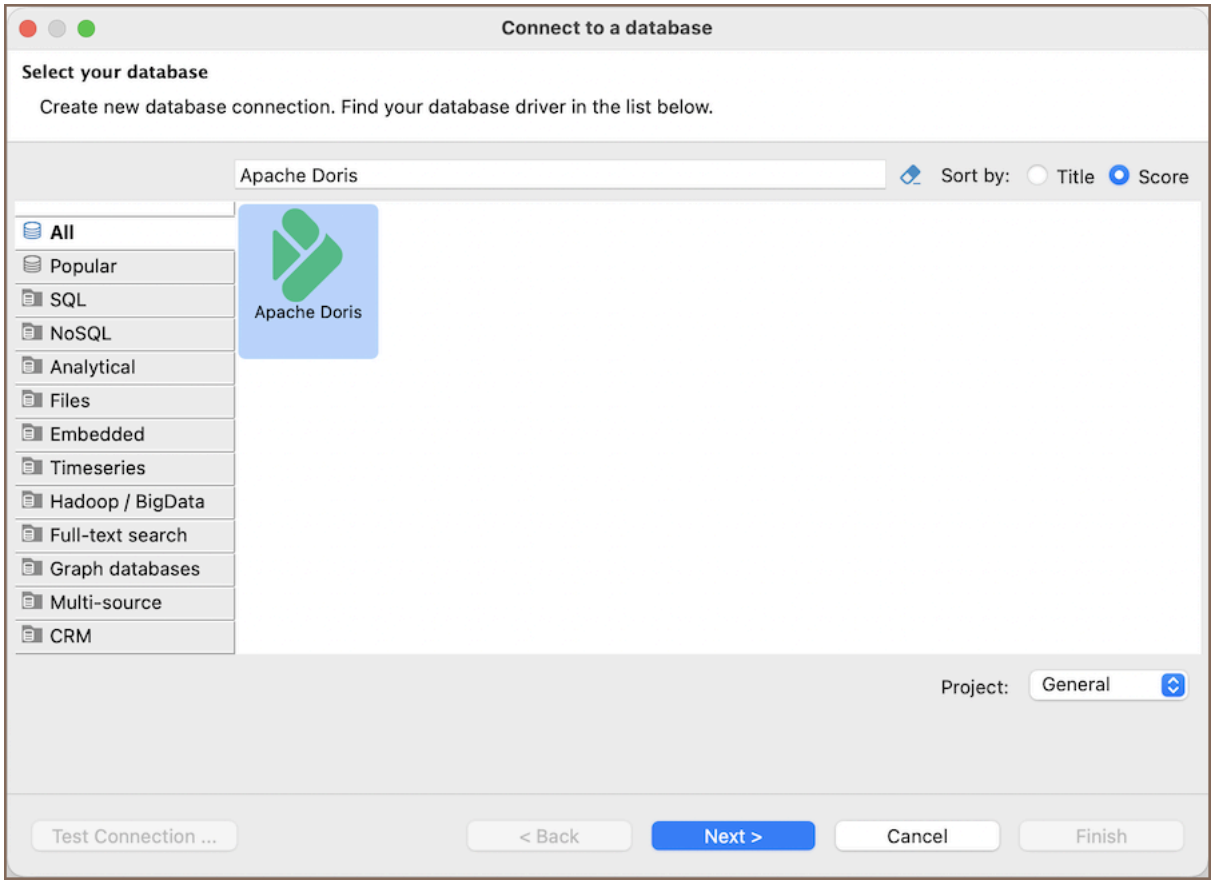
Apache Doris

Table of contents

- Apache Doris specialty
- Setting up
- Apache Doris connection settings
- Connection details
- Apache Doris driver properties
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering Apache Doris with DBeaver
- Apache Doris database objects
- Apache Doris features in DBeaver

This guide provides instructions on how to set up and use Apache Doris with DBeaver.

First of all, start creating a connection in DBeaver and select **Apache Doris**. For information on creating a connection, see Create connection.



> VeloDB Cloud is a managed service built on Apache Doris, so you can connect to it through the same driver in DBeaver.

Apache Doris specialty

Apache Doris is a distributed, MPP-based SQL database for real-time analytics. It uses columnar storage and vectorized query execution to process analytical workloads across backend nodes. Doris supports real-time data ingestion and is compatible with the MySQL protocol, so DBeaver can connect through MySQL Connector/J.

The driver provides access to the built-in internal catalog for Doris tables and external catalogs for querying data from connected sources.

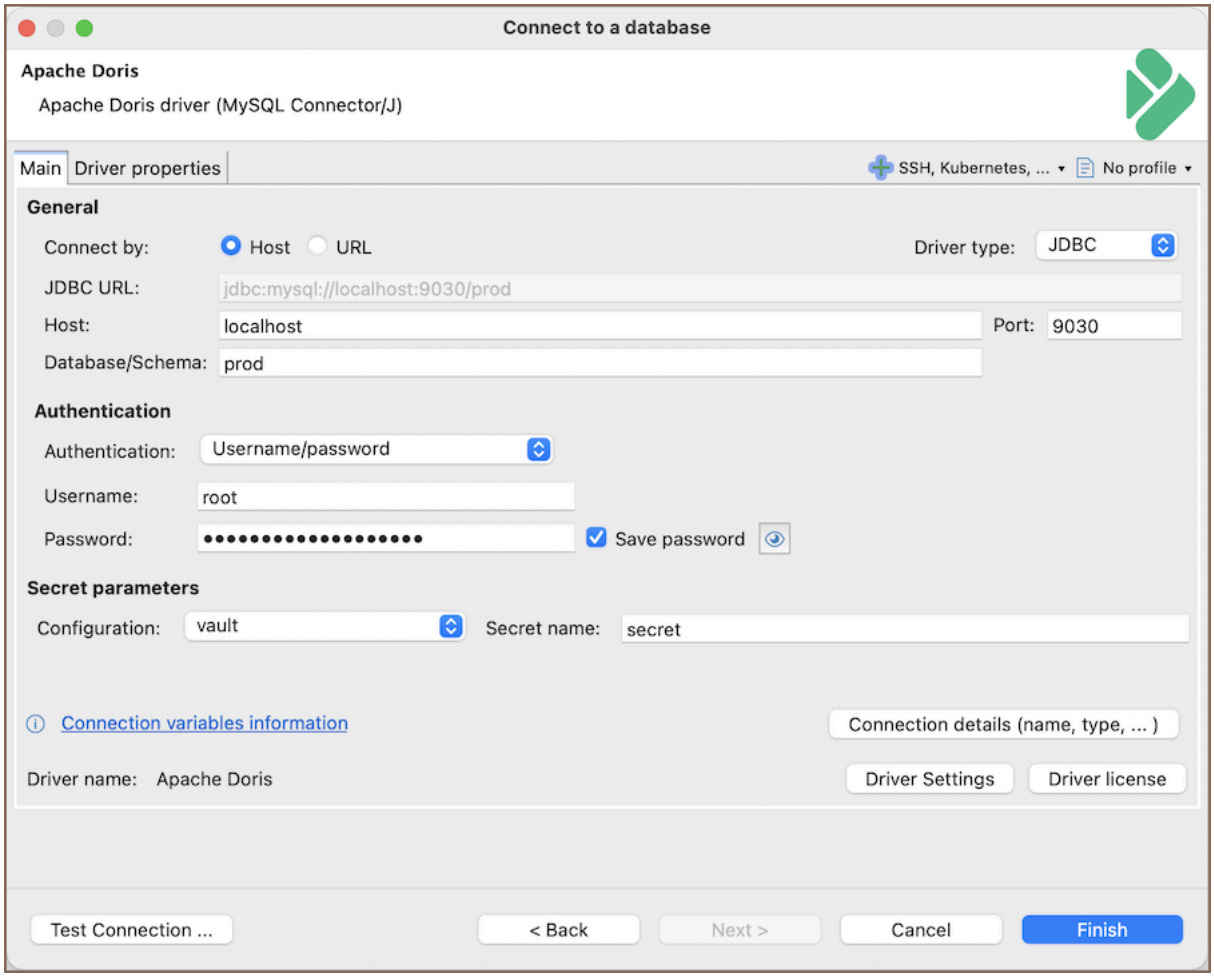
> For more details, see the Apache Doris documentation.

Setting up

Configure a direct connection to Apache Doris and add any required network settings, such as SSH, proxy, or SSL.

Apache Doris connection settings

Use the **Main** tab to configure the initial connection.



The following table describes the connection settings:

Field	Description
Connect by (Host/URL)	Choose whether to connect using a host or a JDBC URL
JDBC URL	Enter the JDBC URL when connecting by URL
Host	Enter the host name or IP address of the Apache Doris frontend node
Database/Schema	Enter a database name, or use `catalog.database` to open a database in an external catalog
Port	Enter the MySQL protocol port, which defaults to `9030`
Authentication	Select username and password, password from shell command, or DBeaver profile ★ authentication
Connection Details	Configure additional connection details
Driver Name	Review the automatically selected driver name
Driver Settings	Configure Apache Doris driver properties

Connection details

Use the **Connection Details** section to configure connection views, security restrictions, filters, initialization, and shell commands. For information on these settings, see the following articles:

- Connection details
- Connections
- Security restrictions
- Filters
- Connection initialization
- Shell commands

Apache Doris driver properties

The Apache Doris driver uses MySQL Connector/J. Configure its properties from the **Driver Settings** page. For information on the available properties, see the MySQL Connector/J configuration properties.

> For information on changing the driver configuration, see Driver manager.

Secure connection configurations

Configure secure access to Apache Doris with the available network settings. For information on each configuration, see the following articles:

- SSH
- Proxy
- Kubernetes ★
- AWS SSM ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Apache Doris with DBeaver

DBeaver provides a host of features designed for Apache Doris databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Apache Doris database objects

DBeaver lets you view and manipulate a wide range of Apache Doris database objects. DBeaver has extensive support for various Databricks metadata types, allowing you to interact with a wide variety of database objects, such as:

- Catalogs
- Databases
  - Tables
  - Columns
  - Views
  - Indexes

Apache Doris features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous additional features compatible with Apache Doris, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Data Visualization	ERD Guide
Data Generation	Mock Data Generation

Apache Hive/Spark/Impala

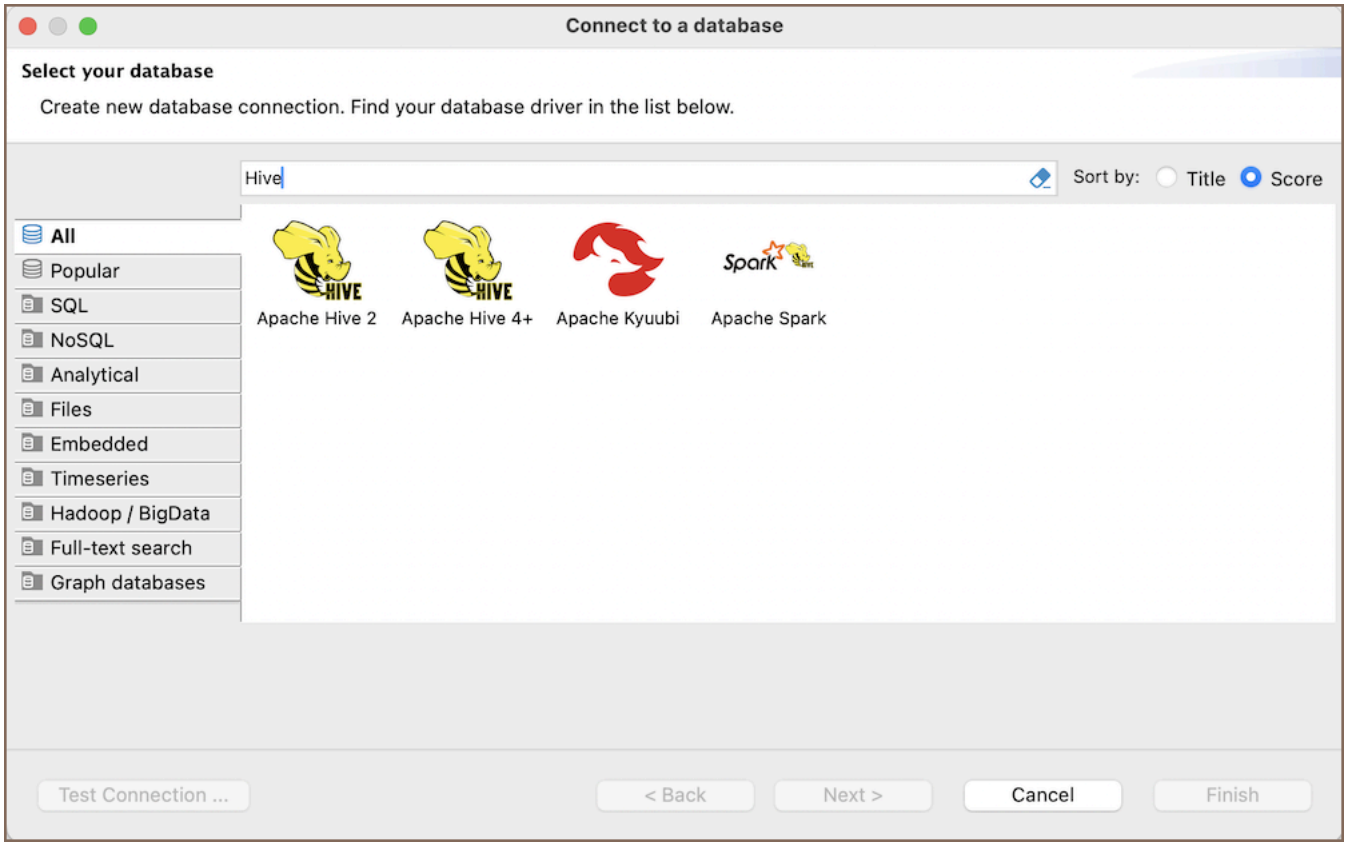
Table of contents

- Apache Hive specialty
- Setting up
- Apache Hive connection settings
- Connection details
- Apache Hive driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Apache Hive with DBeaver
- Apache Hive database objects
- Apache Hive features

This guide provides instructions on how to set up and use Apache Hive with DBeaver.

Before you start, you must create a connection in DBeaver and select Hive. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the Hive server using a specific driver. It supports all versions of Hive, but the correct driver must be selected: use `Hive 2` for versions 2.x and earlier, and `Hive 4` for version 4 and later. DBeaver also supports Hive extensions such as Kyuubi and Spark, depending on your environment configuration. You must select the appropriate driver in the **Connect to a database** window for these extensions.



> **Tip**: Systems like Impala use Hive-compatible drivers, so the standard Hive JDBC driver works in this case.

Apache Hive specialty

Apache Hive is a data warehouse system built on top of Hadoop for querying and analyzing large datasets using a SQL-like language called HiveQL. It's optimized for batch processing and is commonly used for data summarization, ad-hoc queries, and analysis of structured data. Hive translates SQL queries into MapReduce jobs, making it well-suited for handling large-scale data stored in Hadoop Distributed File System (HDFS). Hive is a good fit for OLAP-style workloads and integrates with other big data tools in the Hadoop ecosystem.

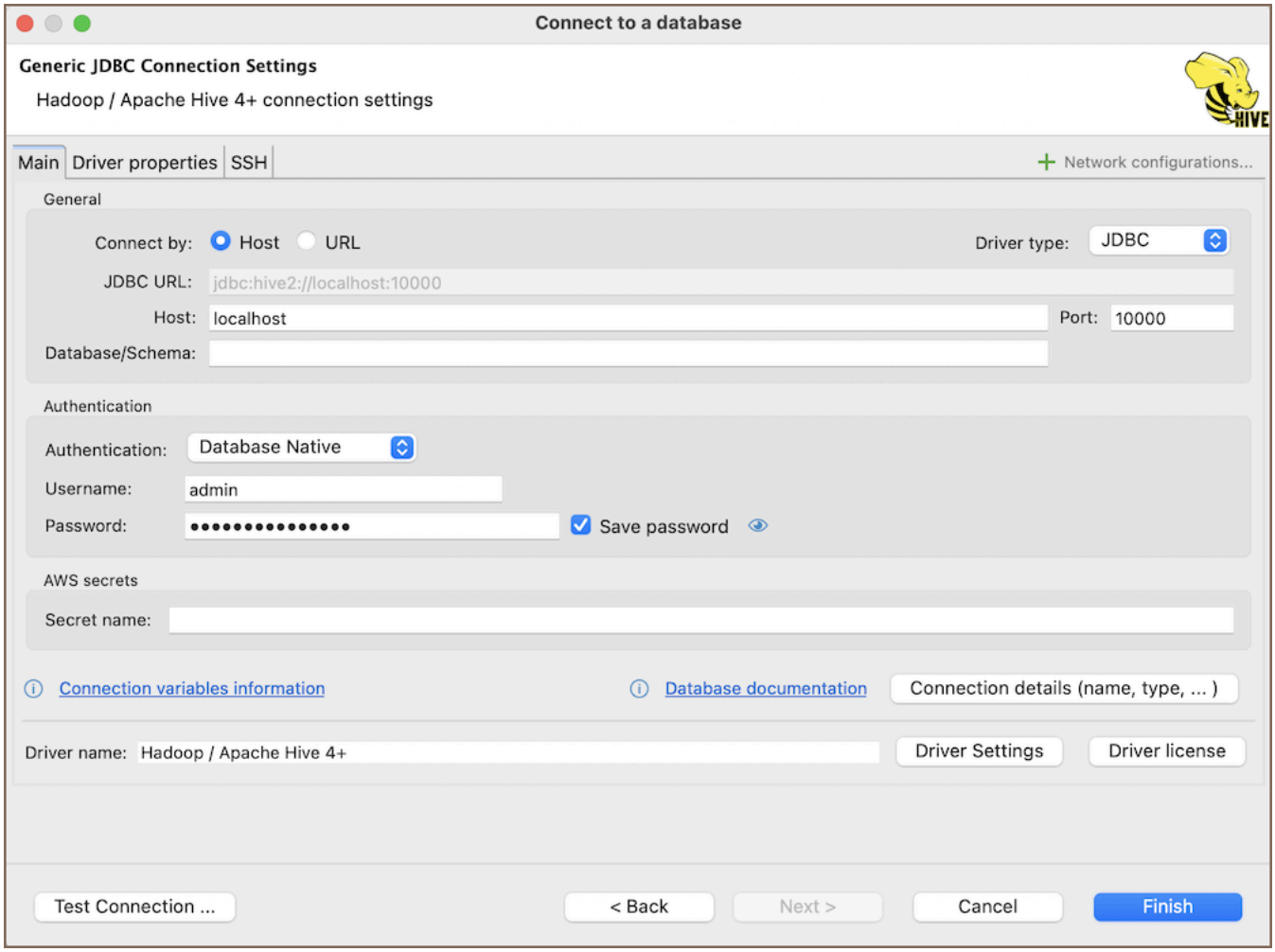
> For more detailed information and a comprehensive understanding of Apache Hive, see the official documentation.

Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH and proxies for Hive.

Apache Hive connection settings

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
URL	If you are connecting via URL, enter the URL of your Hive database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your Hive database here.
Database/Schema	Enter the name of the Hive database you want to connect to.
Port	Enter the port number for your Hive database. The default Hive port is `10000`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Hive database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Apache Hive driver properties

The settings for Hive **Driver properties** enable you to adjust the performance of the Hive driver. These adjustments can influence the efficiency, compatibility, and features of your Hive database.

You can customize the Hive driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Hive database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Hive database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Apache Hive with DBeaver

DBeaver provides a host of features designed for Hive databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Apache Hive database objects

DBeaver lets you view and manipulate a wide range of Hive database objects. DBeaver has extensive support for various Hive metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases/Schemas
- Tables
  - Columns
- Views

> Hive doesn't support referential integrity, so you won't see primary keys or foreign keys.
> Diagrams also aren't relevant.

Apache Hive features

DBeaver is not limited to typical SQL tasks. It also includes features specific to Hive.

Beyond regular SQL operations, DBeaver provides a range of Hive-oriented capabilities, such as:

Category	Feature
Data Types	Hive-specific types like `ARRAY`, `STRUCT`.
File System	HDFS.
Query Language	HiveQL (SQL-like query language).

Additional features compatible with Hive, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Data Management	Data Compare

Cassandra

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

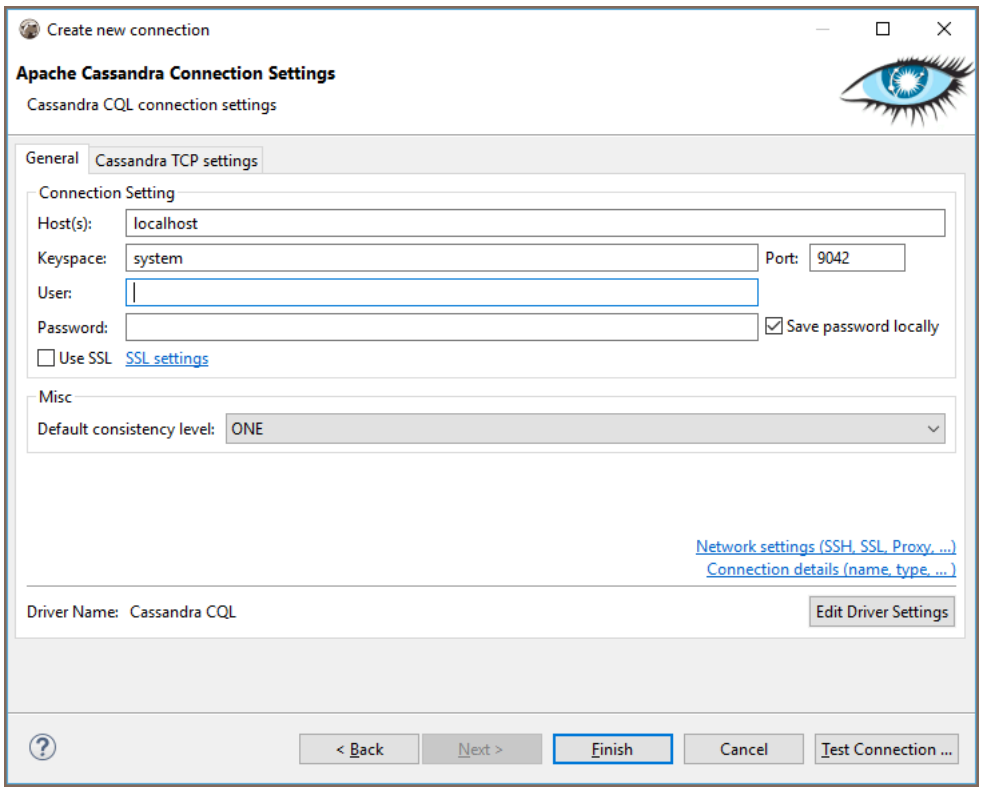
Table of contents

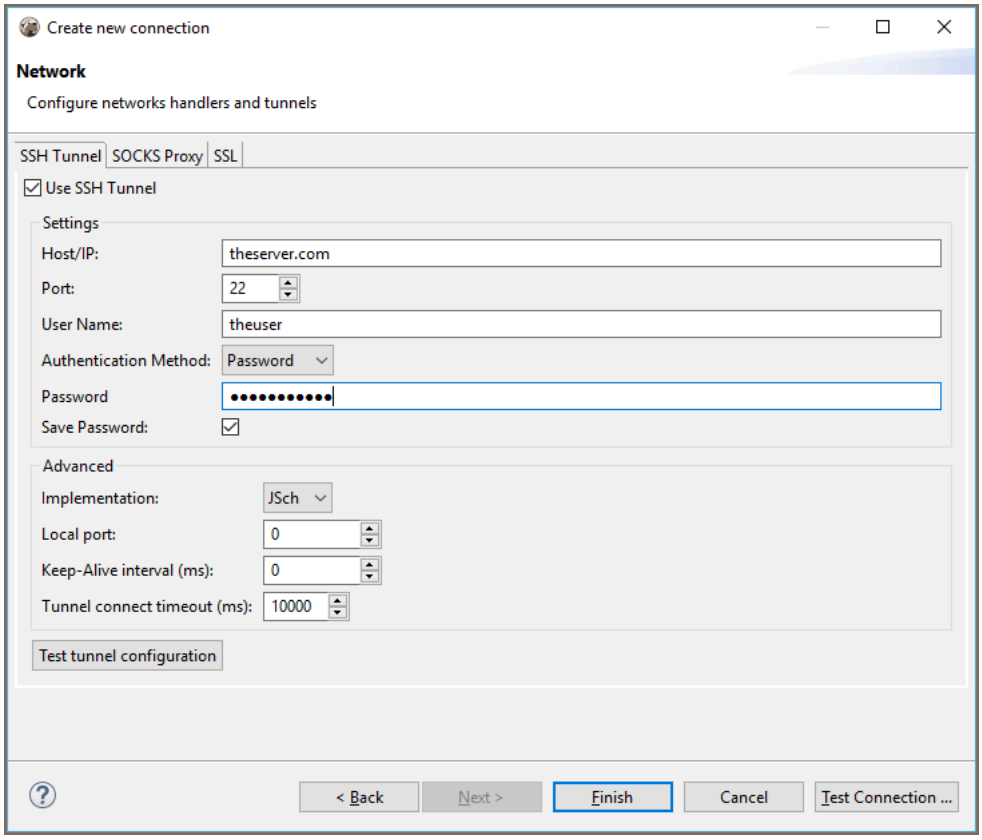
- Connecting to Cassandra cluster
- Browsing Cassandra tables
- Executing CQL
- ERD

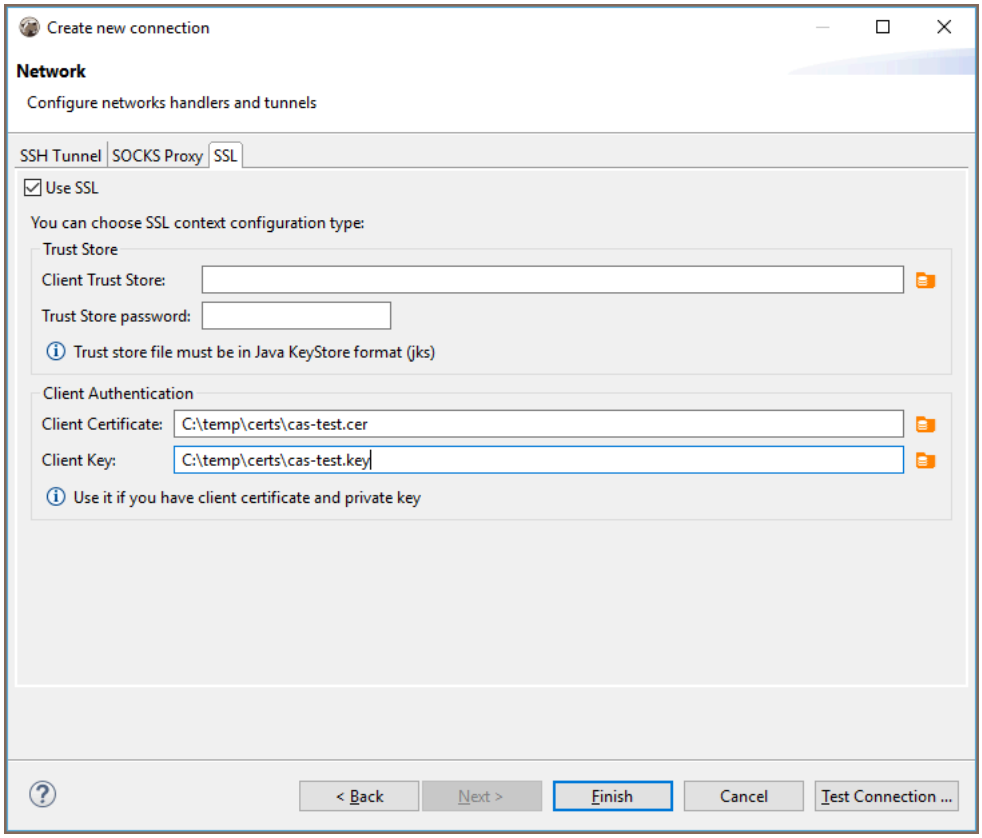
DBeaver supports Cassandra schema browser, data viewer and CQL queries execution. It also supports various administrative tools.

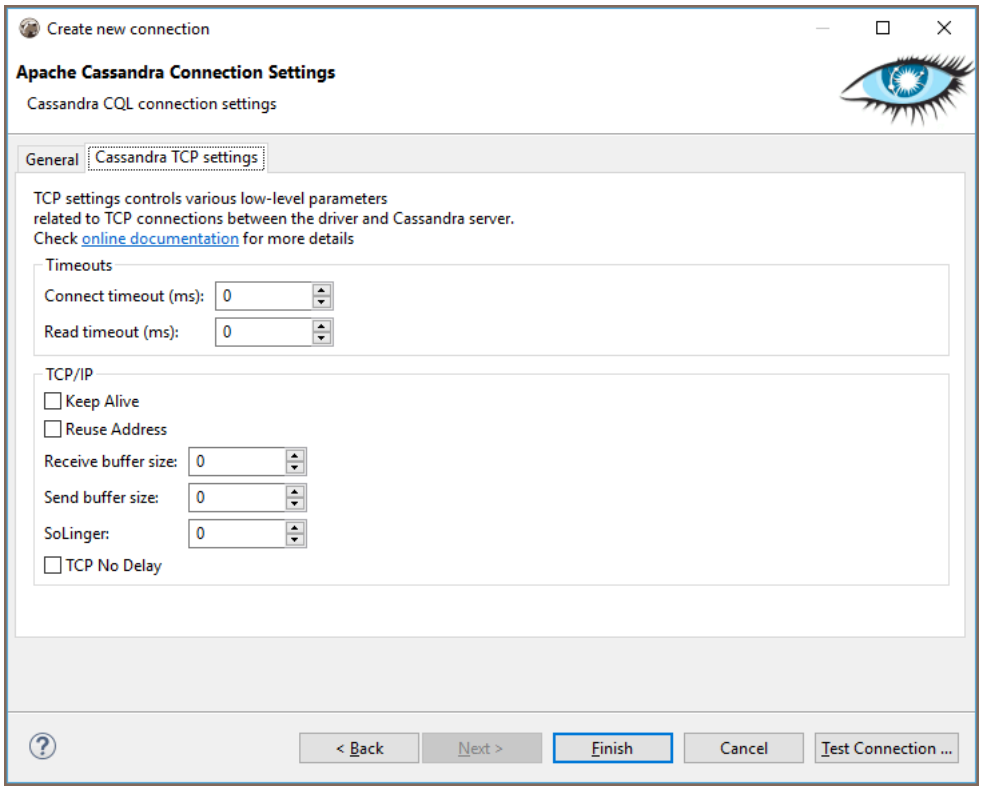
Connecting to Cassandra cluster

You can connect directly to a server or use SSH tunneling or SOCKS proxy. DBeaver uses the DataStax Java driver to operate with a server. It supports Cassandra servers 2.x, 3.x or higher.



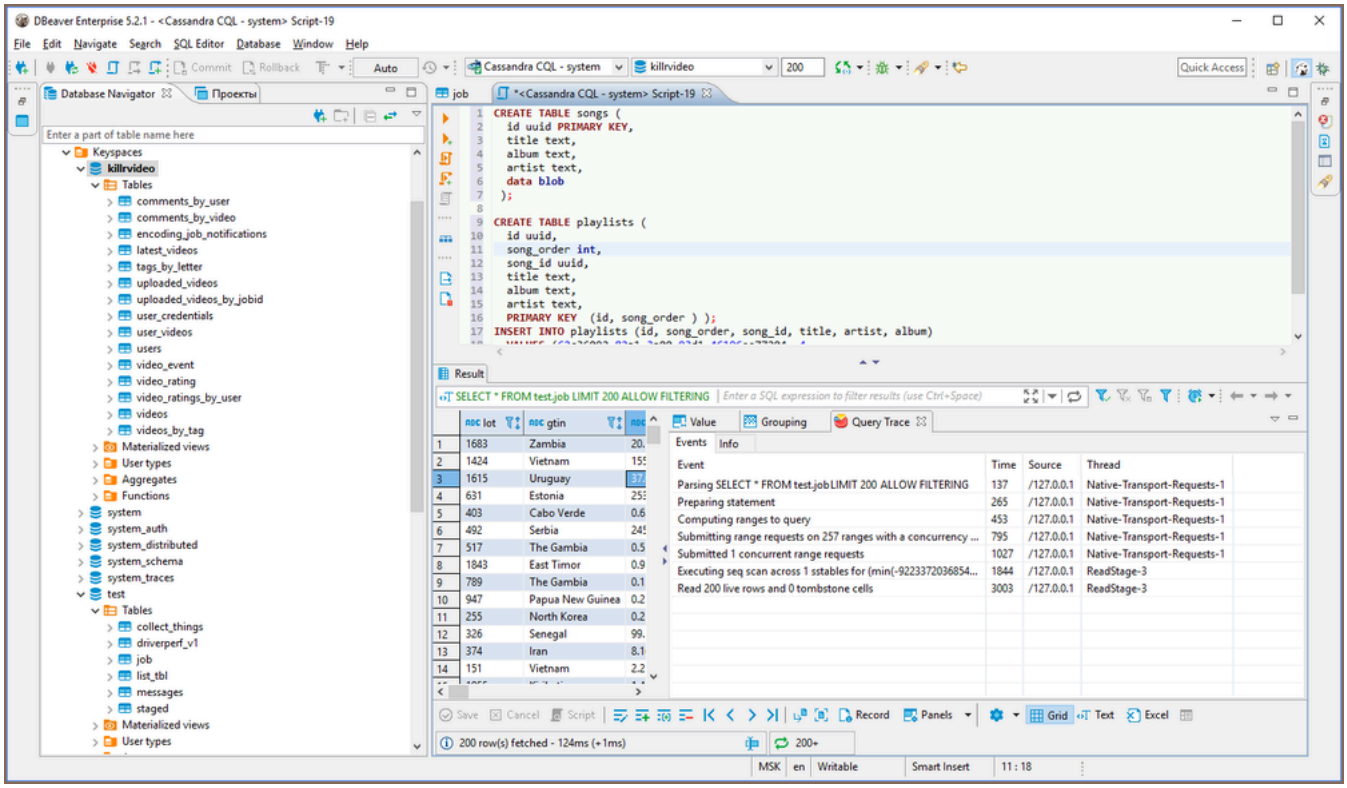


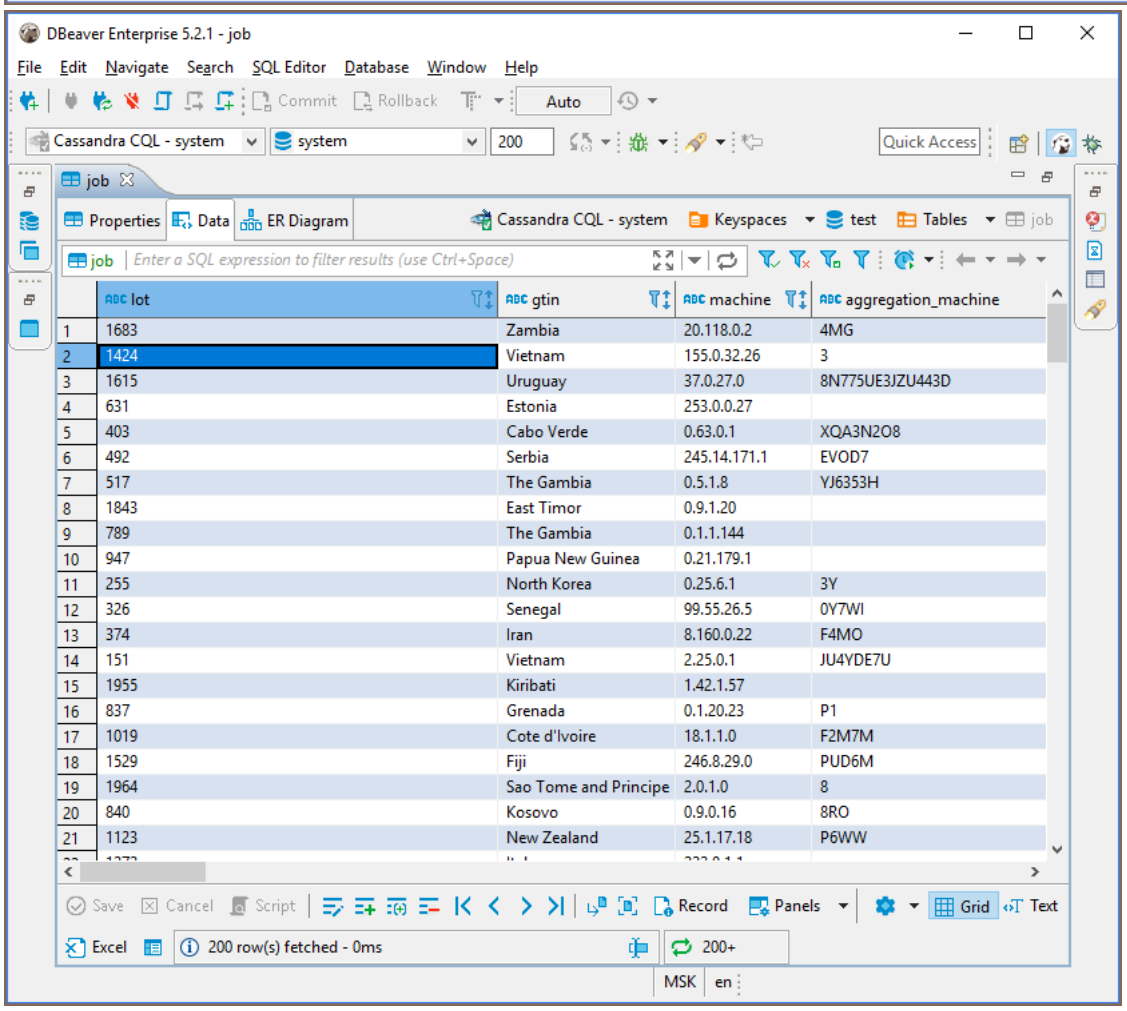




Browsing Cassandra tables

You can browse, view, edit and filter Cassandra tables the same way as with regular (relational) tables. However, being a distributed key-value database, Cassandra does not support any kind of referential integrity. There are no foreign keys, references, etc.
You should note that Cassandra has a very advanced (comparing to relational databases) data type system. Each column may be a collection, map, or set of values (with very big number of values). In some cases this makes browsing data in the "Grid" mode inconvenient.

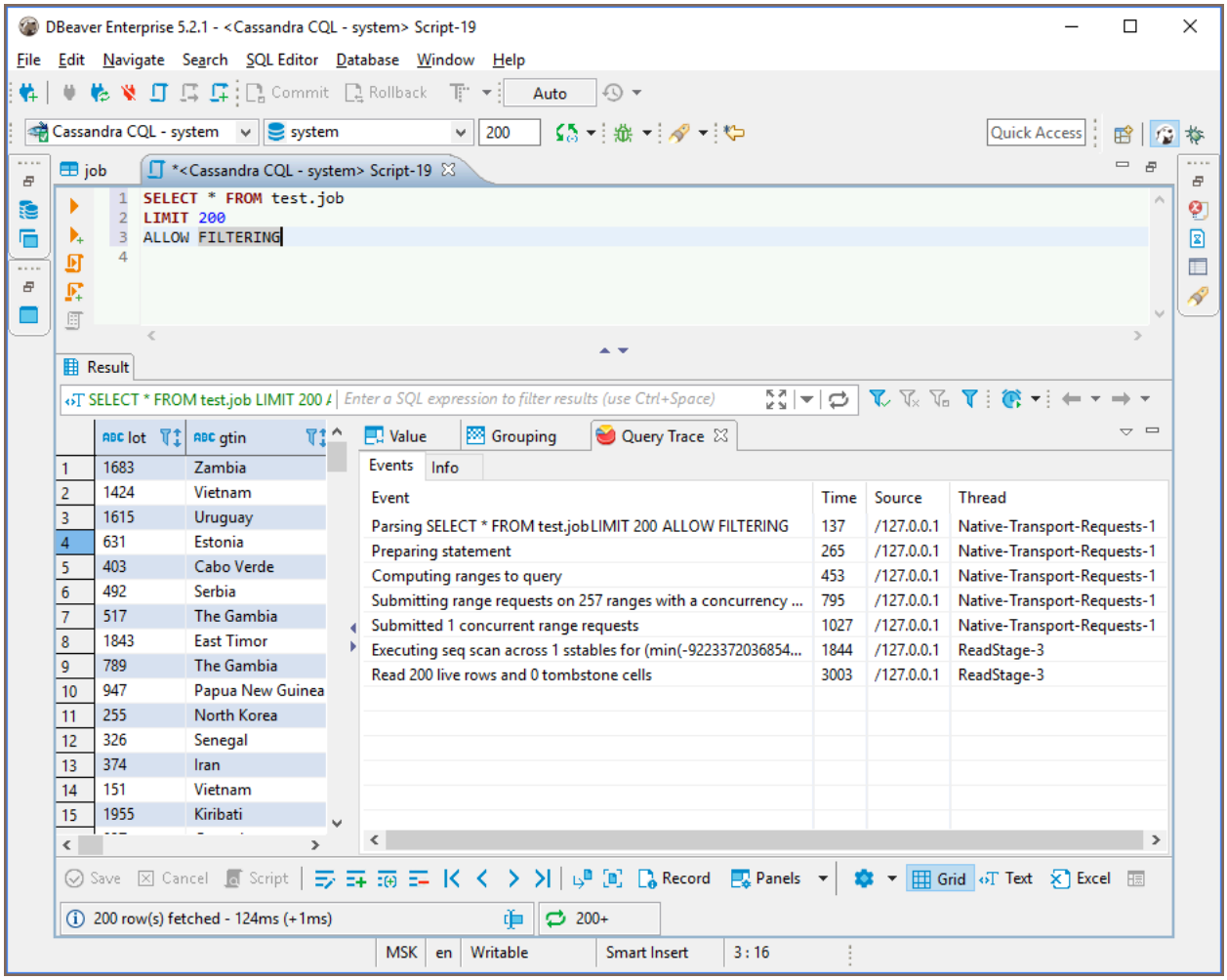




Executing CQL

CQL Cassandra Query Language is a very simple SQL language dialect.
It supports simple SELECT queries, DDL statements (like CREATE TABLE) and some other query types.

You can use the standard DBeaver SQL editor to execute CQL queries. DBeaver supports Cassandra query execution, results scrolling, data export/import, mock data generation and other features. Data viewer (of individual tables or custom CQL query results) query tracing is supported.



ERD

Physical ERD (Entity Relation Diagram) does not make much sense for Cassandra as there are no foreign keys. However, you can make you own custom ERD and connect an actual Cassandra table with each other using logical associations.

ClickHouse

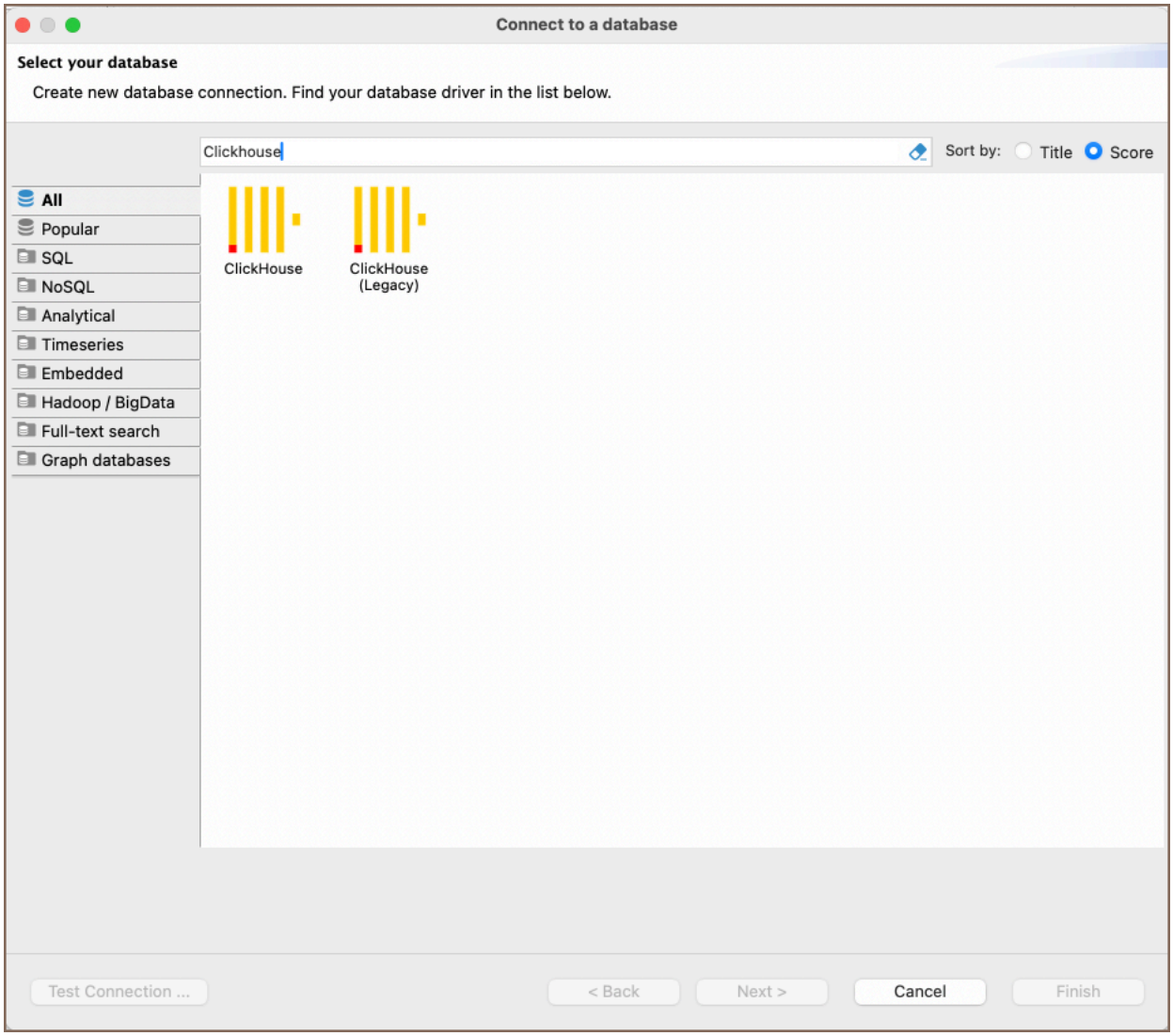
Table of contents

- Setting up
- ClickHouse connection settings
- Connection details
- ClickHouse driver properties
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering ClickHouse with DBeaver
- ClickHouse database objects
- ClickHouse features in DBeaver

This guide provides instructions on how to set up and use ClickHouse with DBeaver.

Before you start, you must create a connection in DBeaver and select ClickHouse. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the ClickHouse server using a specific driver. It supports all versions of ClickHouse, but the correct driver must be selected: use `ClickHouse (Legacy)` for driver version 21.3 and earlier, and `ClickHouse` for driver version 21.3 and later. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



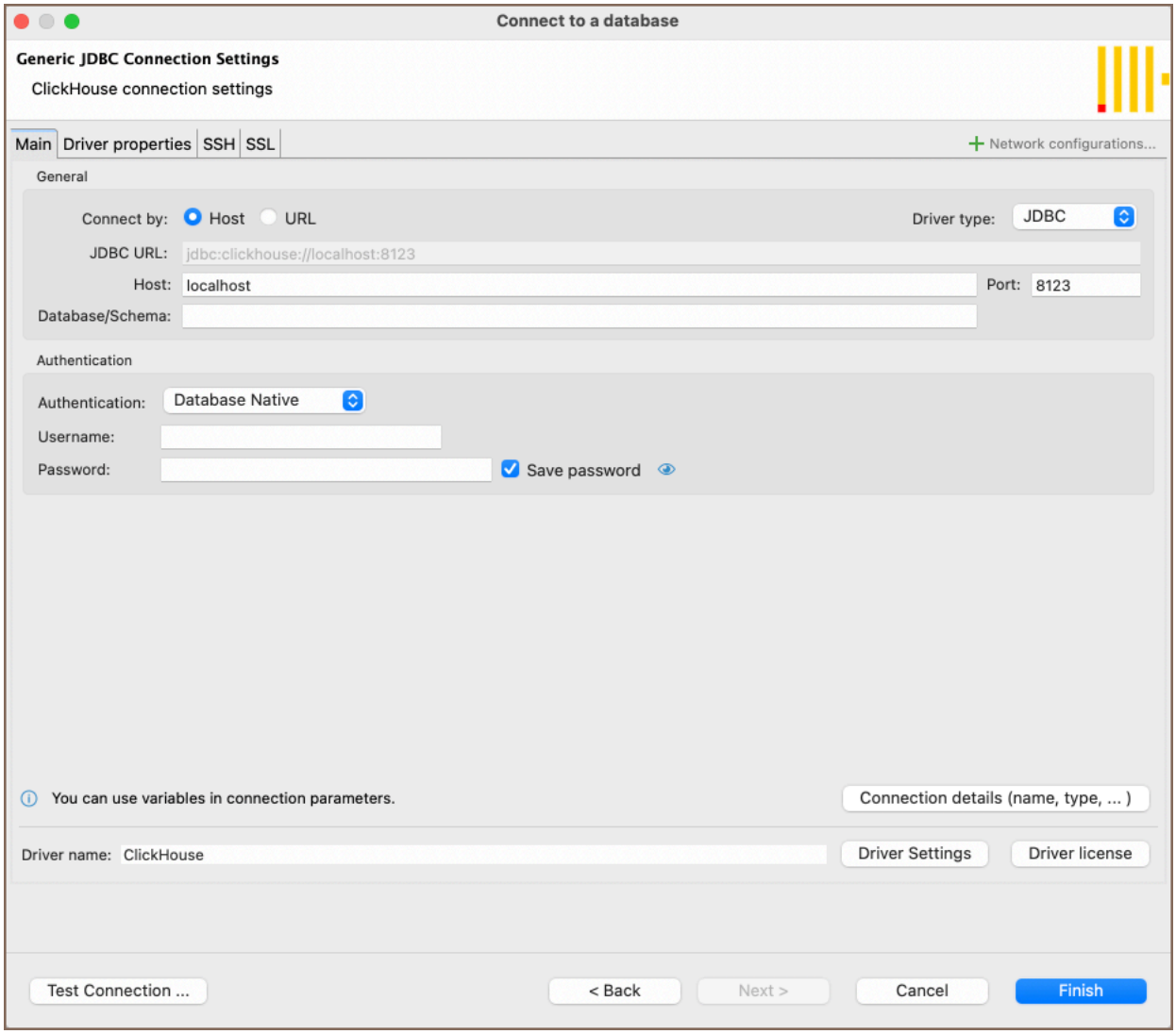
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for ClickHouse.

ClickHouse connection settings

In this subsection, we will outline the settings for establishing a direct connection to a ClickHouse database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your ClickHouse database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your ClickHouse database.
JDBC URL	If you are connecting via URL, enter the URL of your ClickHouse database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via Host, enter the host address of your ClickHouse database here.
Database/Schema	Enter the name of the ClickHouse database you want to connect to.
Port	Enter the port number for your ClickHouse database. The default ClickHouse port is 8123.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with ClickHouse database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide

- Connection Initialization Settings Guide
- Shell Commands Guide

ClickHouse driver properties

The settings for ClickHouse **Driver properties** enable you to adjust the performance of ClickHouse ClickHouse JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your ClickHouse database.

For a complete walkthrough on setting up ClickHouse JDBC and ODBC drivers properties, you can refer to the official ClickHouse JDBC documentation and ClickHouse ODBC documentation. These guides detail each driver's properties and how they can be used to optimize ClickHouse database connections.

You can customize the ClickHouse driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your ClickHouse database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure connection configurations

DBeaver supports secure connections to your ClickHouse database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering ClickHouse with DBeaver

DBeaver provides a host of features designed for ClickHouse databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

ClickHouse database objects

DBeaver lets you view and manipulate a wide range of ClickHouse database objects. DBeaver has extensive support for various ClickHouse metadata types, allowing you to interact with a wide variety of database objects, such as:

- Schemas
- Tables
  - Columns
- Views
- Procedures

ClickHouse features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for ClickHouse. Beyond regular SQL operations, DBeaver provides a range of ClickHouse-specific capabilities, such as:

Category	Feature
Data Types	ClickHouse data types such as Nested and Array
Database Management	ClickHouse Procedures

Additional features compatible with ClickHouse, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Schema Management	Schema Compare ★
Mock Data Generation	Mock data generation guide ★
Data Visualization	GIS Guide
	Query Execution plan ★

> **Note**: As a column-oriented database management system, ClickHouse does not include support for referential integrity, such as foreign keys. This particular limitation means that Entity-Relationship (ER) diagrams, which are typically valuable in visualizing and enforcing database relationships, become less applicable and effective in the context of ClickHouse databases.

Couchbase

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

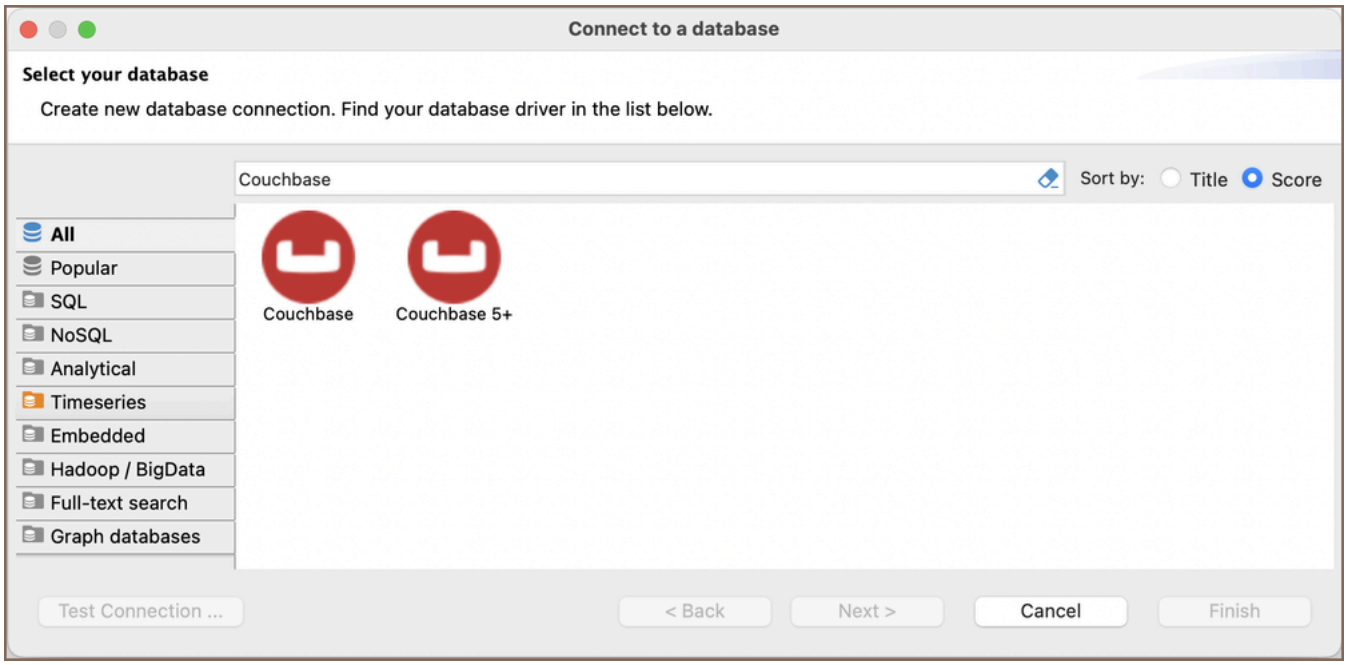
Table of contents

- Couchbase specialty
- Setting Up
- Couchbase connection configuration
  - Couchbase version 4 and below
  - Couchbase version 5 and above
- Connection details
- Couchbase driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Couchbase with DBeaver
- Couchbase database objects
- Couchbase features in DBeaver
- Executing Couchbase queries

This guide provides instructions on how to set up and use Couchbase with DBeaver.

Before you start, you must create a connection in DBeaver and select `Couchbase`. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the Couchbase server using a specific driver. It supports all versions of Couchbase, but the correct driver must be selected: use `Couchbase` for versions 4.x and earlier, and `Couchbase 5+` for version 5 and later.



Couchbase specialty

Couchbase is a NoSQL database that diverges from traditional relational databases by storing data in JSON documents instead of tables. It employs N1QL, an SQL-like language specifically designed for querying JSON. In Couchbase, the concept typically known as databases in traditional systems is represented as Buckets. Unlike relational databases, Couchbase does not support some common features, such as triggers, stored procedures, views, complex joins, and the constructs of primary and foreign keys. For a more comprehensive understanding of Couchbase, please refer to the official documentation.

DBeaver that are compatible with Couchbase.

Setting Up

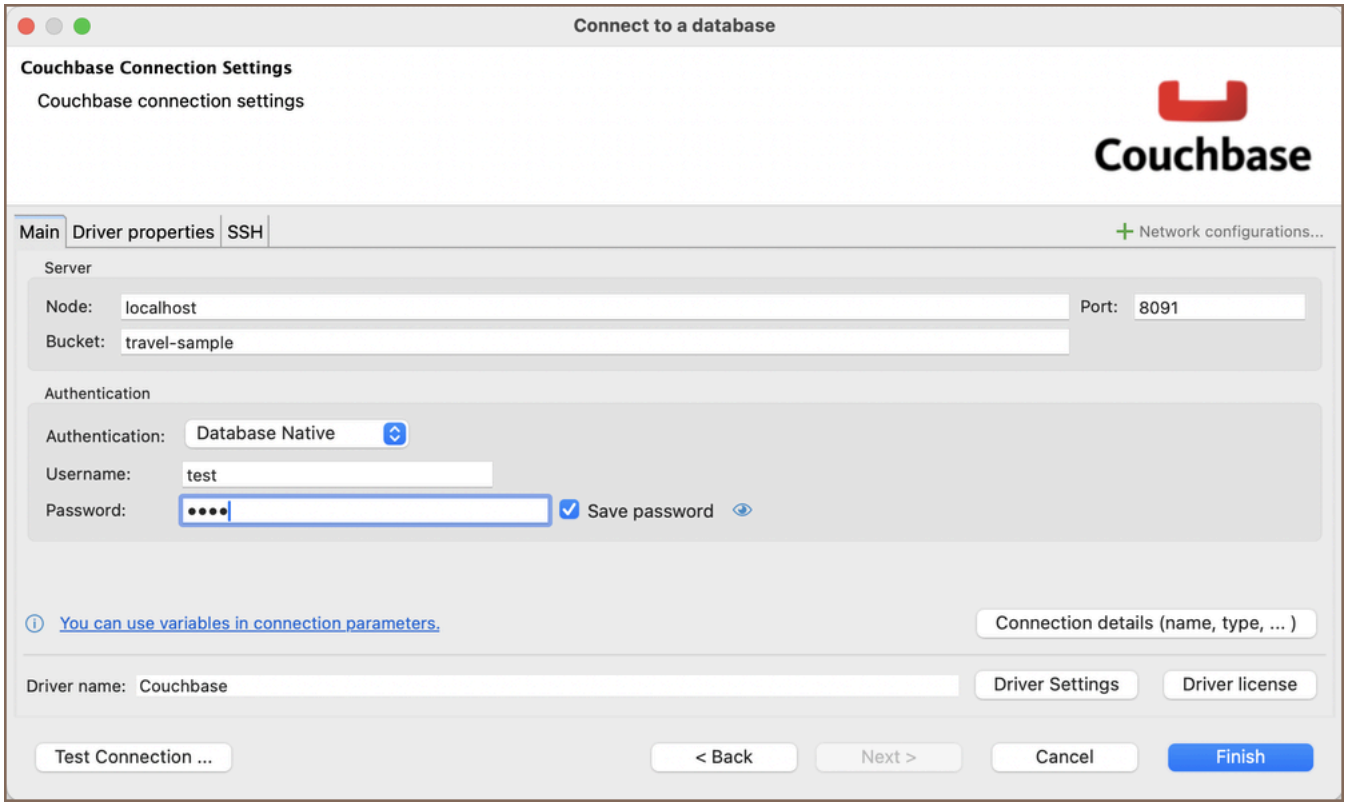
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, SSL, proxies, and Kubernetes.

Couchbase connection configuration

Couchbase version 4 and below

In this subsection, we will outline the settings for establishing a direct connection to a Couchbase version 4 and below database using DBeaver.

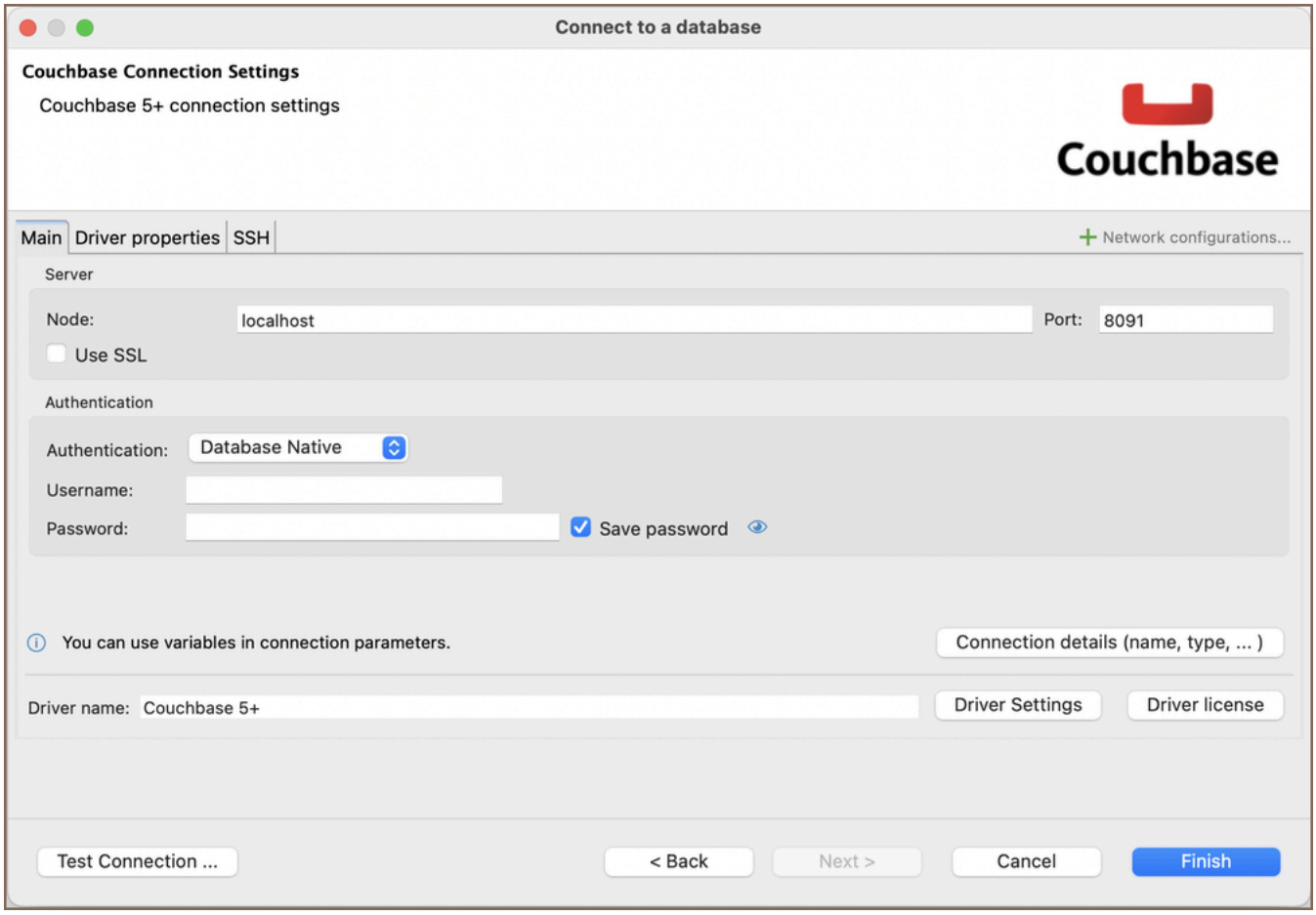
The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Node	Specify the node in the Couchbase cluster to connect to.
Bucket	Enter the name of the specific bucket within the Couchbase database to access.
Port	Enter the port number for your Couchbase database. The default Couchbase port is `8091`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	Specify additional driver settings, such as Timeouts and SSL properties.

Couchbase version 5 and above

The connection settings for these versions are similar to those for version 4 and below, with the notable exception that the **Bucket** setting is not used.



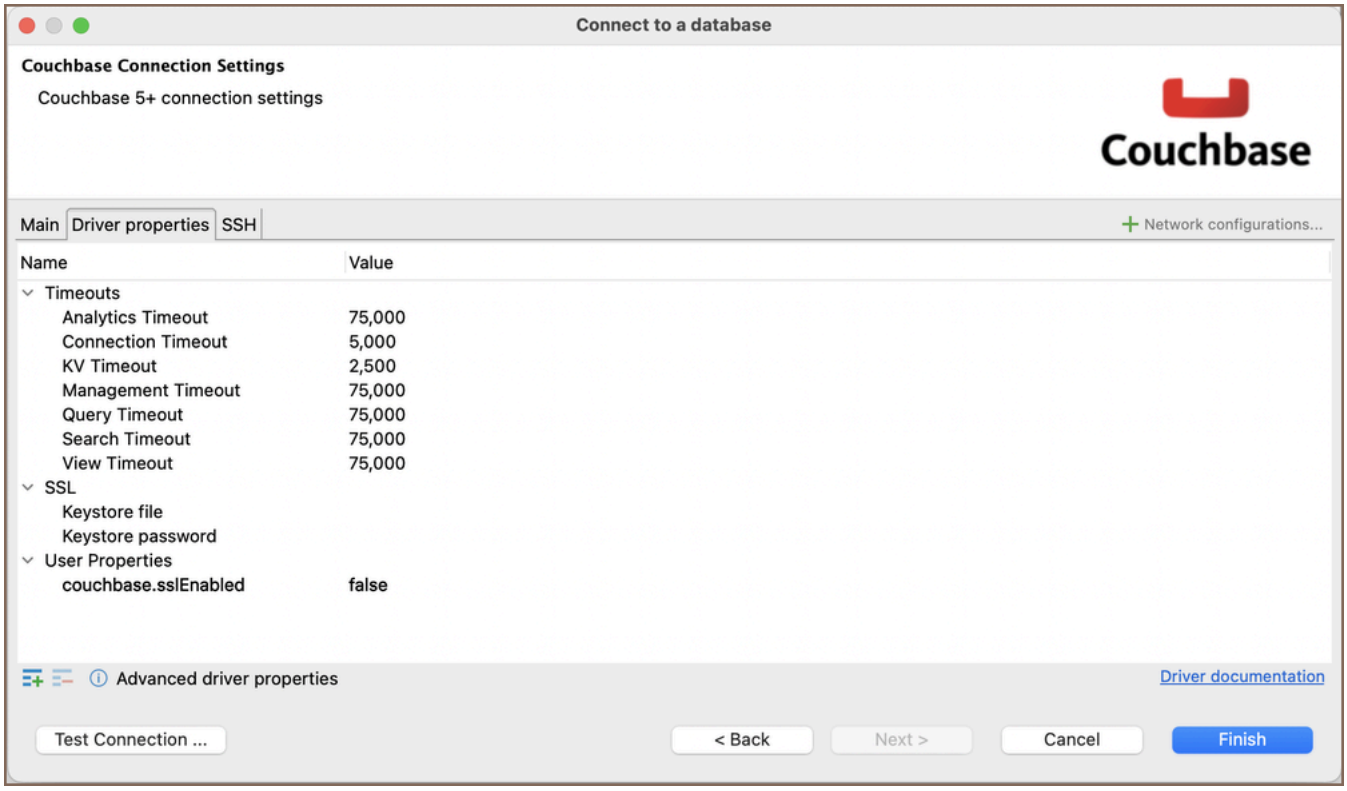
Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Couchbase database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Couchbase driver properties

Couchbase is a NoSQL database system integrated into DBeaver through its API. While configuring connections to Couchbase in DBeaver, there are specific custom properties available, such as Timeouts and SSL settings.



Property	Description
Analytics Timeout	The maximum time to wait for an analytics query before timing out.
Connection Timeout	The time to wait for a connection to be established before timing out.
KV Timeout	Default timeout for key-value operations.
Management Timeout	Time to wait for management operations to complete.
Query Timeout	The maximum time a query can run before it times out.
Search Timeout	Time to wait for a search query to complete before timing out.
View Timeout	Time to wait for a view query to complete before timing out.
Keystore file	Path to the keystore file used for SSL connections.
Keystore password	Password for the keystore file.
couchbase.sslEnabled	A Boolean property to enable or disable SSL. Set to `true` to enable SSL.

> **Tip**: All timeout settings are specified in milliseconds.

Secure Connection Configurations

DBeaver supports secure connections to your Couchbase database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

> **Note**: For SSL configuration, the required properties are set within the DBeaver Driver settings.

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Couchbase with DBeaver

DBeaver provides a host of features designed for Couchbase databases. Couchbase uses a document-oriented approach, which means each piece of data, or "document" can have a unique structure.

In DBeaver, you have the flexibility to view and edit the data within Couchbase buckets. This can be done similarly to how you would interact with tables in a traditional database, either through a grid view or as plain text. You can also work directly with JSON documents, making it easier to handle the different data formats that Couchbase supports.

Couchbase database objects

DBeaver lets you view and manipulate a few Couchbase database objects. DBeaver has extensive support for various Couchbase metadata types, allowing you to interact with:

- Buckets
  - Indexes
- Cluster
  - Users

Couchbase features in DBeaver

DBeaver also includes additional features compatible with Couchbase, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	ERD Guide
Data Management	Working with JSON

Executing Couchbase queries

Couchbase uses N1QL (Non-First Normal Form Query Language), a query language similar to SQL, it lets you search through JSON documents with complex queries. For further details on N1QL, refer to the official documentation.

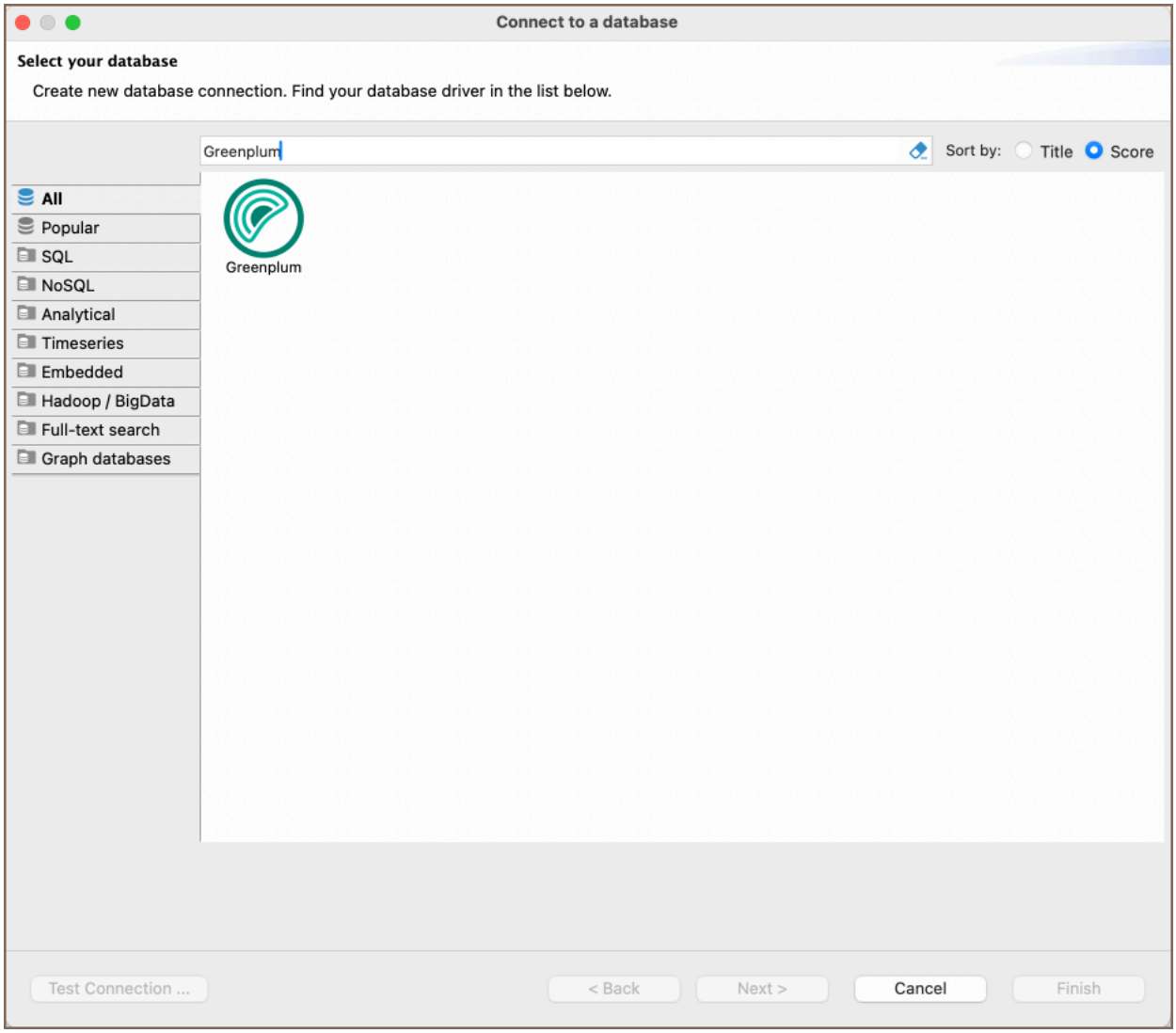
Greenplum

Table of contents

- Greenplum specialty
- Setting Up
- Greenplum connection settings
- Connection details
- Greenplum driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Greenplum with DBeaver
- Greenplum database objects
- Greenplum Features in DBeaver

This guide provides instructions on how to set up and use Greenplum with DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Greenplum` option to connect to the Greenplum database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Greenplum specialty

Greenplum is a highly scalable, multi-node relational database system, primarily used for data warehousing and analytics. A key characteristic of Greenplum is its columnar data storage capabilities, making it particularly effective for analytics and large-scale data processing. Built on PostgreSQL, it supports rich metadata structures, including tables, views, indexes, sequences, and various data types. Its setup in DBeaver mostly replicates the PostgreSQL driver configuration. Greenplum includes support for SQL triggers, stored procedures, and user-defined functions, enhancing its utility for complex data operations. It also provides robust data partitioning options and supports diverse storage models like row and column store, optimizing performance for a variety of workloads. For a more comprehensive understanding of Greenplum, please refer to the official documentation.

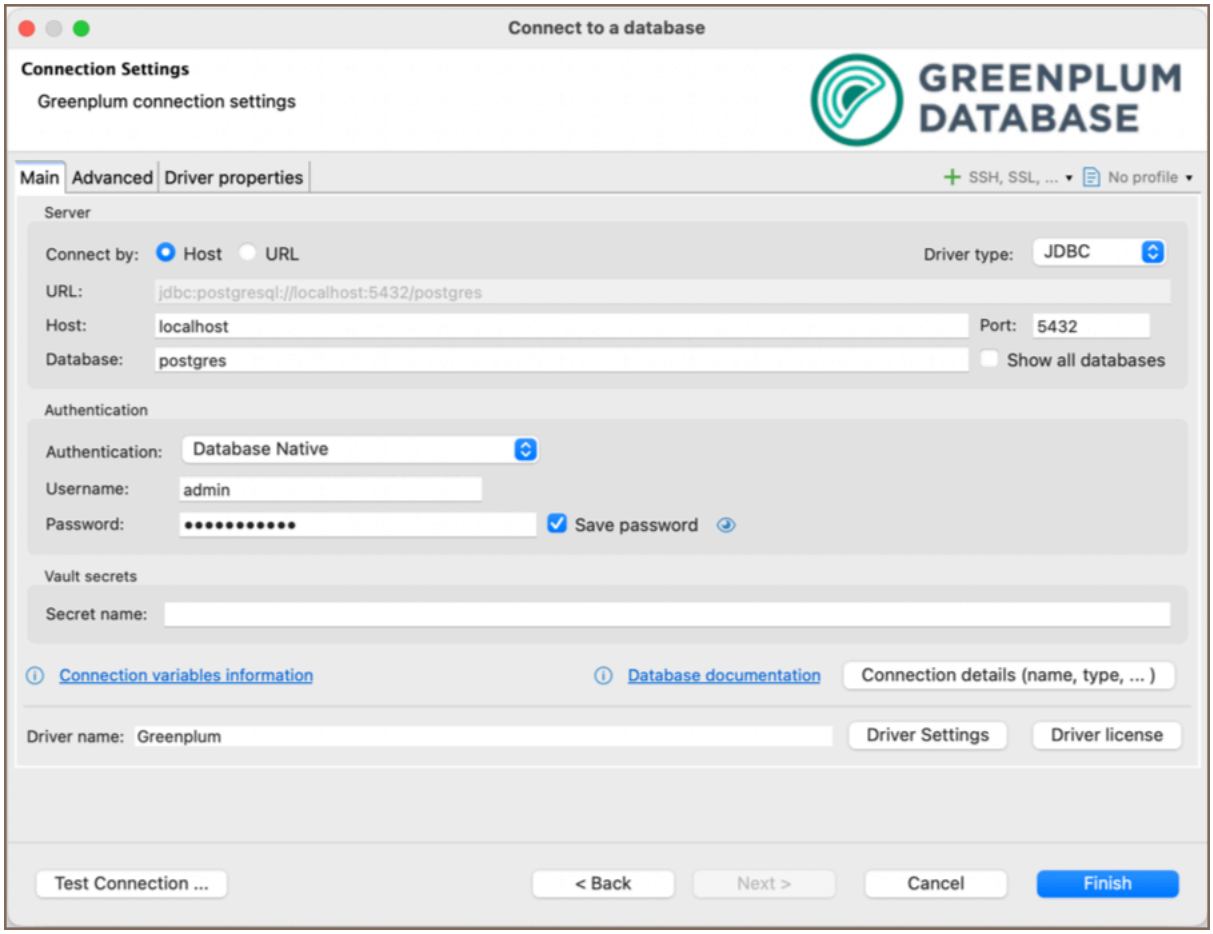
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, Kubernetes, SSL, and the setup of ODBC/JDBC drivers for Greenplum.

Greenplum connection settings

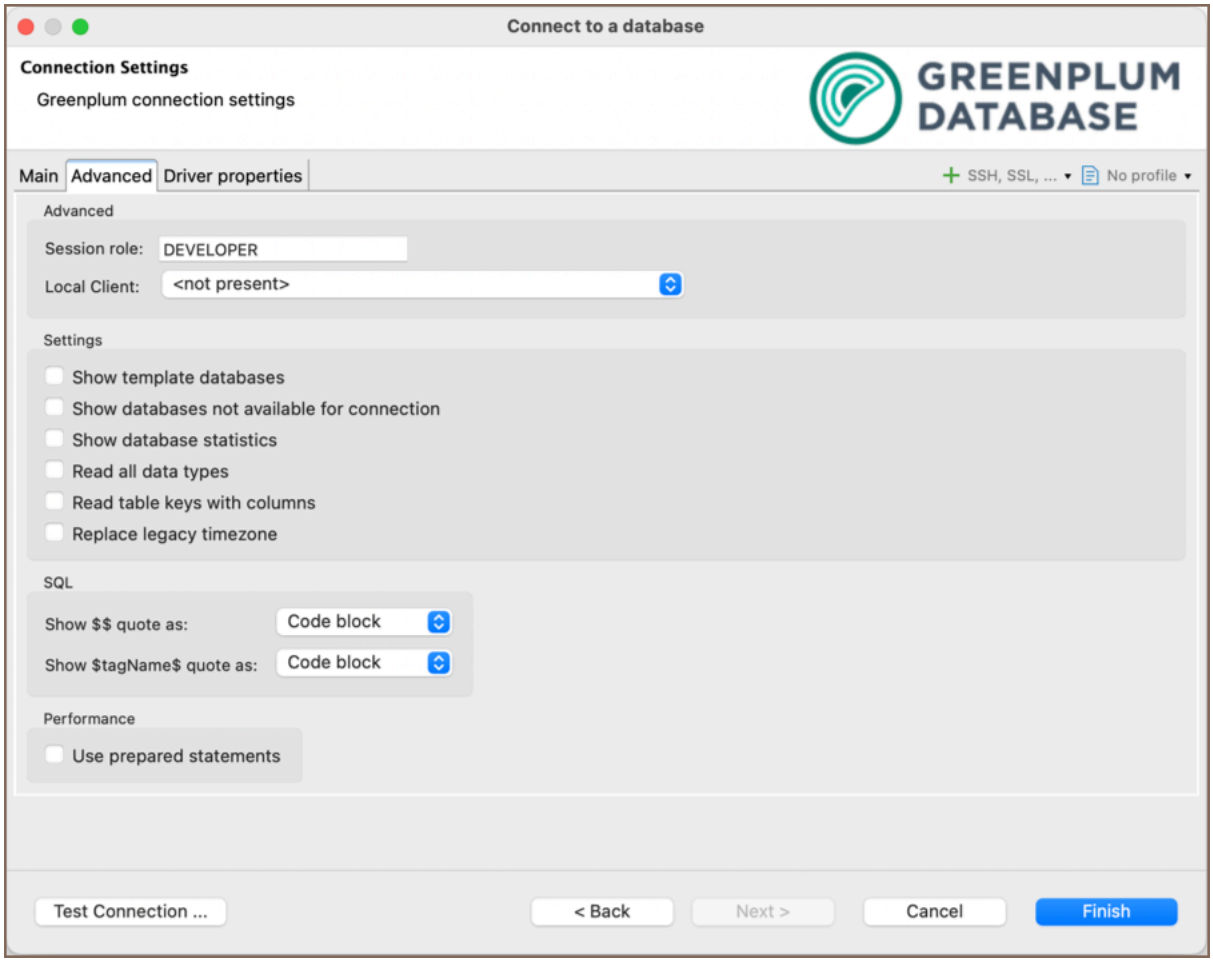
In this subsection, we will outline the settings for establishing a direct connection to a Greenplum database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Greenplum database.

1. The first page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Greenplum database.
URL	If you are connecting via URL, enter the URL of your Greenplum database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your Greenplum database here.
Database	Enter the name of the Greenplum database you want to connect to.
Show all databases	Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Port	Enter the port number for your Greenplum database. The default Greenplum port is 5432.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - PostgreSQL Kerberos Authentication ★ - PostgreSQL PgPass Authentication - PostgreSQL SSPI Authentication ★ You can also read about security in DBeaver PRO.
Local Client	Specify the local client, see our article about Local client.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The second page of the connection settings offers additional options that allow you to customize your further connection to the Greenplum database.



Field	Description
Show template databases	Displays template databases. This option is available when **Show all databases** is checked.
Show databases unavailable for connection	Show databases not available for connection in database list. This option is available when **Show all databases** is checked.
Show databases statistics	Displays statistics for databases.
Read all data types	Reads all data types from the server (including arrays).
Read table keys with columns	Enable reading table constraints during the column reading stage to ensure accurate recognition of keys. Note that this setting may decrease metadata loading performance.
Replace legacy timezone	Replaces legacy timezone settings. Enable only if you need to connect with legacy timezones like **Europe/Kyiv** or **Asia/Calcutta**. Only these two timezones are affected.
Show `$$` quote as	Displays `$$` quotes as code blocks or strings.
Show `$tagName$` quote as	Displays `$tagName$` quotes as code blocks or strings.
Use prepared statements	Uses prepared statements for SQL execution.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Greenplum connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Greenplum driver properties

The settings for Greenplum **Driver properties** enable you to adjust the performance of the Greenplum JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Greenplum database.

For a complete walkthrough on setting up Greenplum JDBC and ODBC drivers properties, you can refer to the official Greenplum JDBC documentation and Greenplum ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Greenplum database connections.

You can customize the Greenplum driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Greenplum database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Greenplum database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Greenplum with DBeaver

DBeaver provides a host of features designed for Greenplum databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Greenplum database objects

DBeaver lets you view and manipulate a wide range of Greenplum database objects. DBeaver has extensive support for various Greenplum metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Data types
    - Tables
      - Columns
      - Constraints
      - Indexes
      - Dependencies
      - References
      - Partitions
      - Triggers
      - Rules
      - Policies
    - External Tables
    - Views
    - Functions
    - Sequences
    - Data types
    - Aggregate functions
  - Event triggers
  - Storage
  - Tablespaces
  - Roles
- Administer
  - Jobs
  - Session Manager
  - Lock Manager

Greenplum Features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous additional features compatible with Greenplum, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Lock Management	Lock Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

Category	Feature
	Query Execution plan
Data Generation	Mock Data Generation

IBM Db2

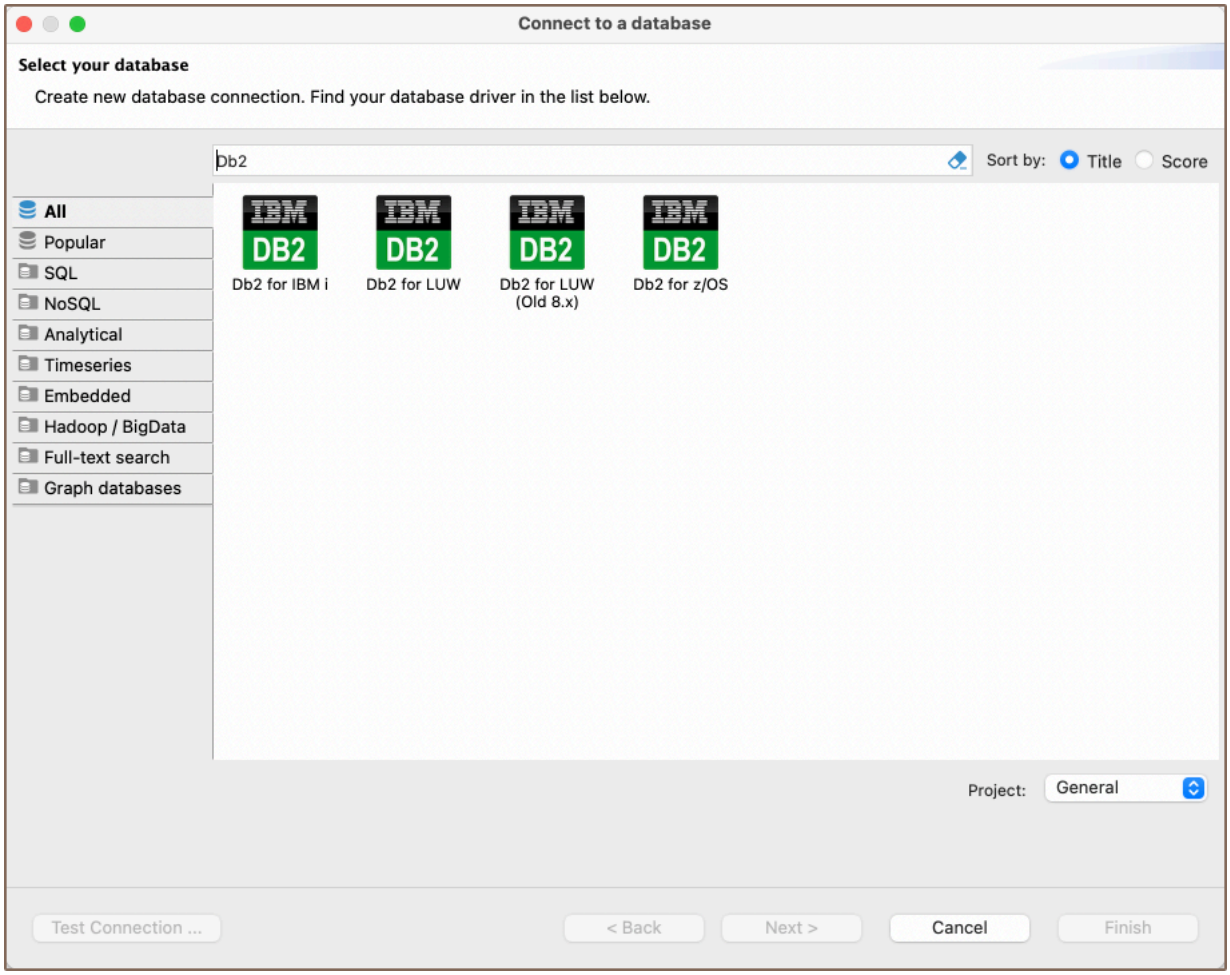
Table of contents

- Setting Up
- Db2 connection settings
  - Connection details
  - Trace Settings
- Db2 driver properties
- ODBC and JDBC Driver Configuration
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Db2 with DBeaver
- Db2 database objects
- Db2 features in DBeaver

This guide provides instructions on how to set up and use IBM DB2 with DBeaver.

Before you start, you must create a connection in DBeaver and select IBM DB2. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the IBM DB2 server using a specific driver, it supports all versions of IBM DB2, but the correct driver must be selected: use **Db2 for LUW (Old 8.x)** for versions below 8.x and **Db2 for LUW** for newer versions. DBeaver also supports various IBM DB2 databases such as **Db2 for IBM i**, and **Db2 for z/OS**. You must select the appropriate driver in the **Connect to a database** window for these databases.



Setting Up

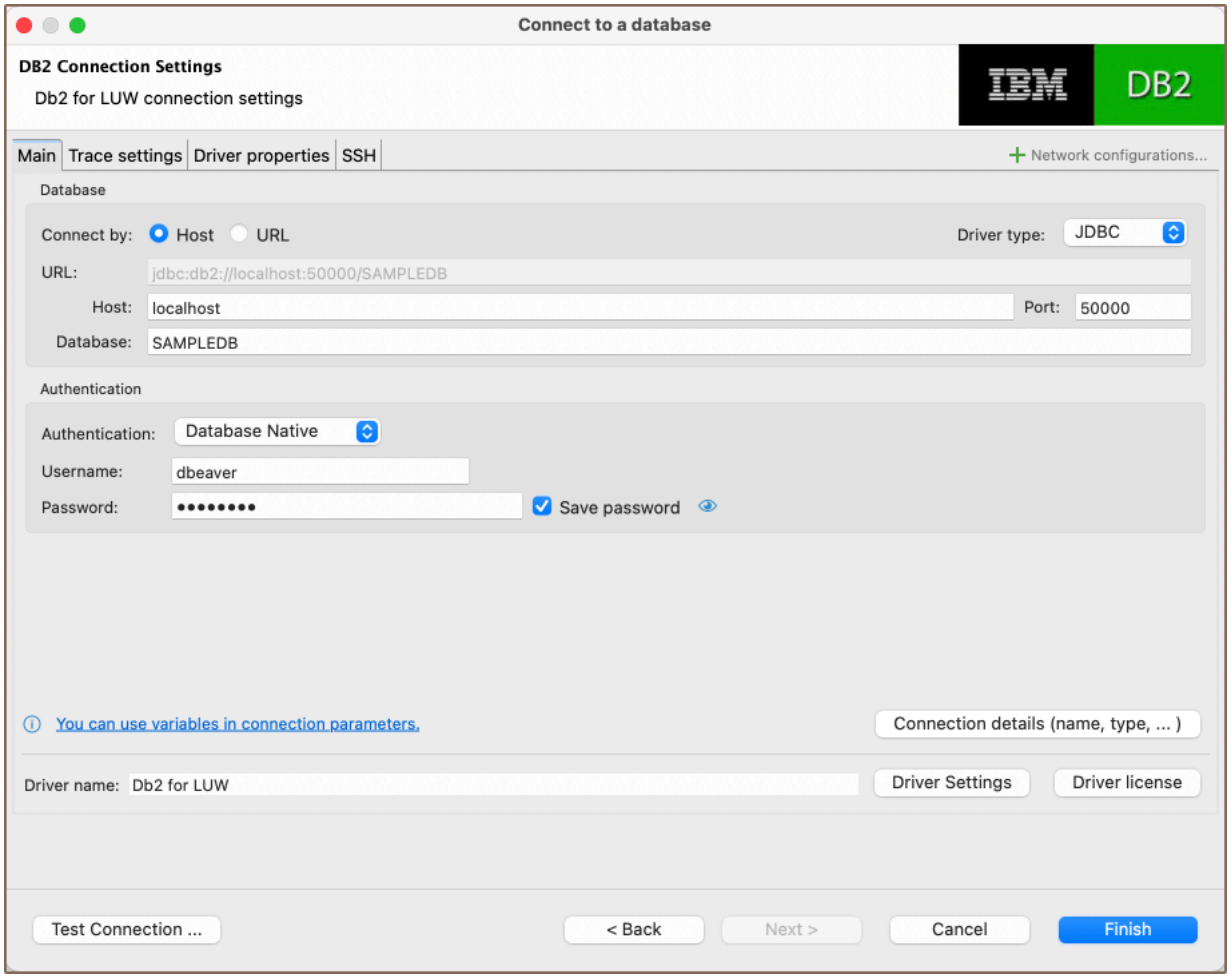
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for Db2.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

Db2 connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Db2 database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Db2 database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Db2 database.
URL	If you are connecting via URL, enter the URL of your Db2 database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your Db2 database here.
Database	Enter the name of the Db2 database you want to connect to.
Port	Enter the port number for your Db2 database.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - Kerberos Authentication (only supported with **Db2 for LUW** driver version 11) ★
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Db2 connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

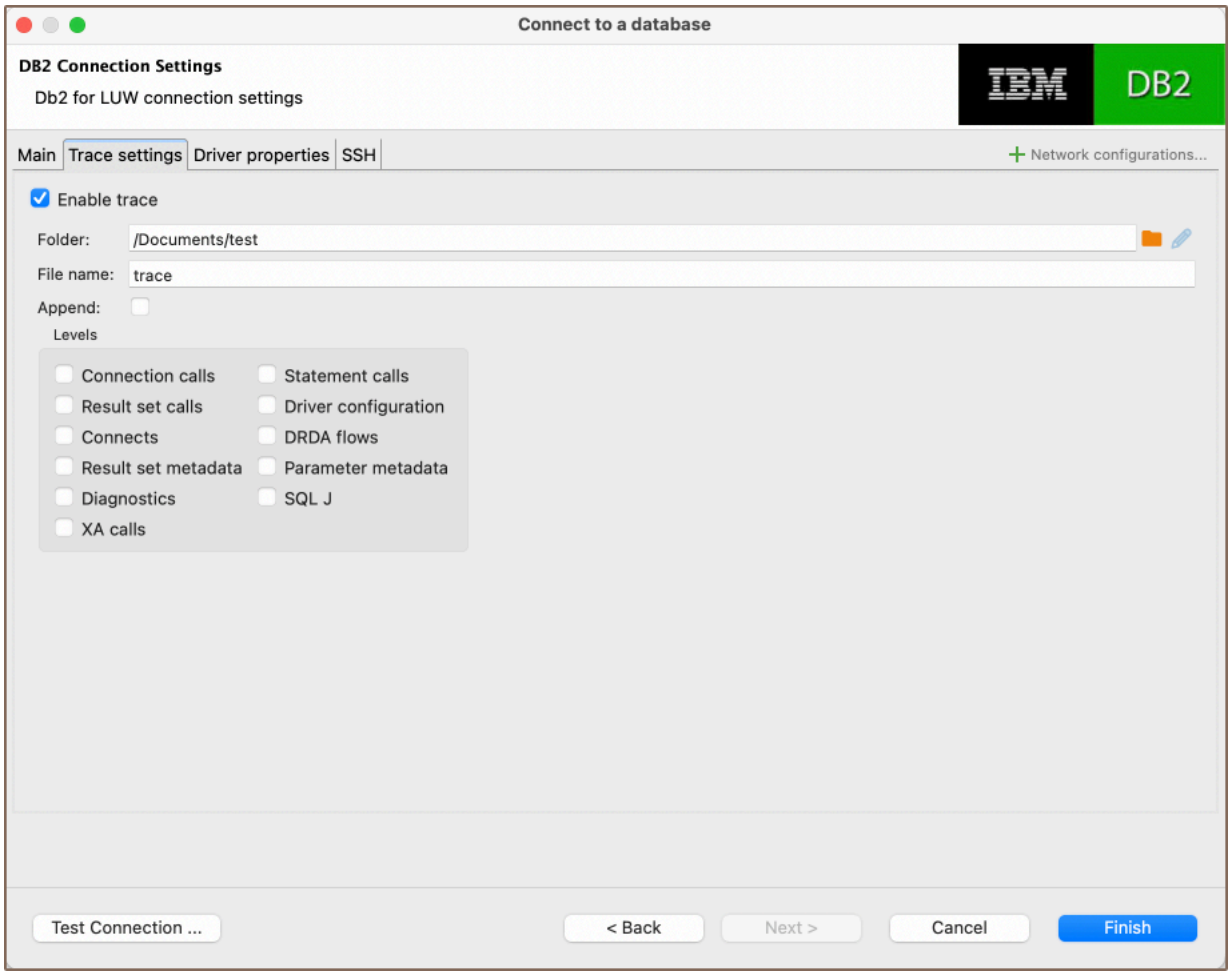
- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide

- Shell Commands Guide

Trace Settings

In DBeaver, the trace settings feature allows you to monitor and record database operations in a detailed manner. To enable this feature, check the **Enable trace** box. You will need to specify a file where the traced information will be stored and formatted into a readable format.

The trace settings page allows you to configure various levels and settings to tailor the tracing process according to your preferences.



Setting	Description
Folder/File name	Specify the directory and file name where the trace data will be stored. This file will contain detailed information about the operations performed in the database.
Append	This option allows you to add information to an existing trace file, instead of creating a new file each time the trace is enabled.
Connection Calls	Records calls related to database connections.
Result Set Calls	Records calls pertaining to the result sets.
Connects	Records information about database connects.
Result Set Metadata	Records metadata information from the result sets.
Diagnostics	Records diagnostic information about database operations.
XA Calls	Records information on XA distributed transactions.
Statement Calls	Records calls related to SQL statements.
Driver Configuration	Records configuration details of the database driver.
DRDA Flows	Records information on DRDA data flows.
Parameter Metadata	Records metadata about SQL parameters.
SQL J	Records information on SQL J operations.

For more detailed information on these settings, please refer to the IBM documentation.

Db2 driver properties

The settings for Db2 **Driver properties** enable you to adjust the performance of the Db2 JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Db2 database.

For a complete walkthrough on setting up Db2 JDBC and ODBC drivers properties, you can refer to the official Db2 JDBC documentation and Db2 ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Db2 database connections.

You can customize the Db2 driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Db2 database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

ODBC and JDBC Driver Configuration

DBeaver provides extensive capabilities for managing Db2 database connections via ODBC/JDBC drivers. This functionality enables you to connect to your Db2 database using native ODBC drivers, offering an alternative when specific DBeaver drivers are not available.

You can find a comprehensive, step-by-step guide on how to install the driver manager, set up drivers, configure data sources, and establish connections in DBeaver in our ODBC Driver Configuration article.

Secure Connection Configurations

DBeaver supports secure connections to your Db2 database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **SSL**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Db2 with DBeaver

DBeaver provides a host of features designed for Db2 databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Db2 database objects

DBeaver lets you view and manipulate a wide range of Db2 database objects. DBeaver has extensive support for various Db2 metadata types, allowing you to interact with a wide variety of database objects, such as:

- Schemas
  - Tables
    - Columns
    - Unique Constraints
    - Foreign Keys
    - Indexes
    - References
    - Check Constraints
    - Partitions
    - Periods
    - Triggers
    - Columns Masks
  - Views
  - MQTs
  - Indexes
  - Sequences
  - Aliases
  - Nicknames
  - Triggers
  - XML Schemas
  - Application Objects
    - Functions
    - Modules
    - Packages
    - Procedures
    - User Defined Types
- Federation
  - Wrappers
  - Remote Servers
  - User Mappings
- Global metadata
  - Types
  - Variables
  - XML Strings
- Storage
  - Bufferpools
  - Tablespaces

    - Storage Groups
- Administer
    - Session Manager
- Security
    - Roles
    - Groups
    - Users

Db2 features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for DB2. Beyond regular SQL operations, DBeaver provides a range of Db2-specific capabilities, such as:

Category	Feature
Schemas	Unique Constraints
	Periods
	Column Masks
Views	MQTs
Federation	Wrappers
	Remote Servers
Global Metadata	XML Strings
Storage	Bufferpools
	Tablespaces
Security	Roles
	Groups

Additional features compatible with Db2, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Lock Management	Lock Manager
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide
Query insights	Disable metadata queries

InfluxDB

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

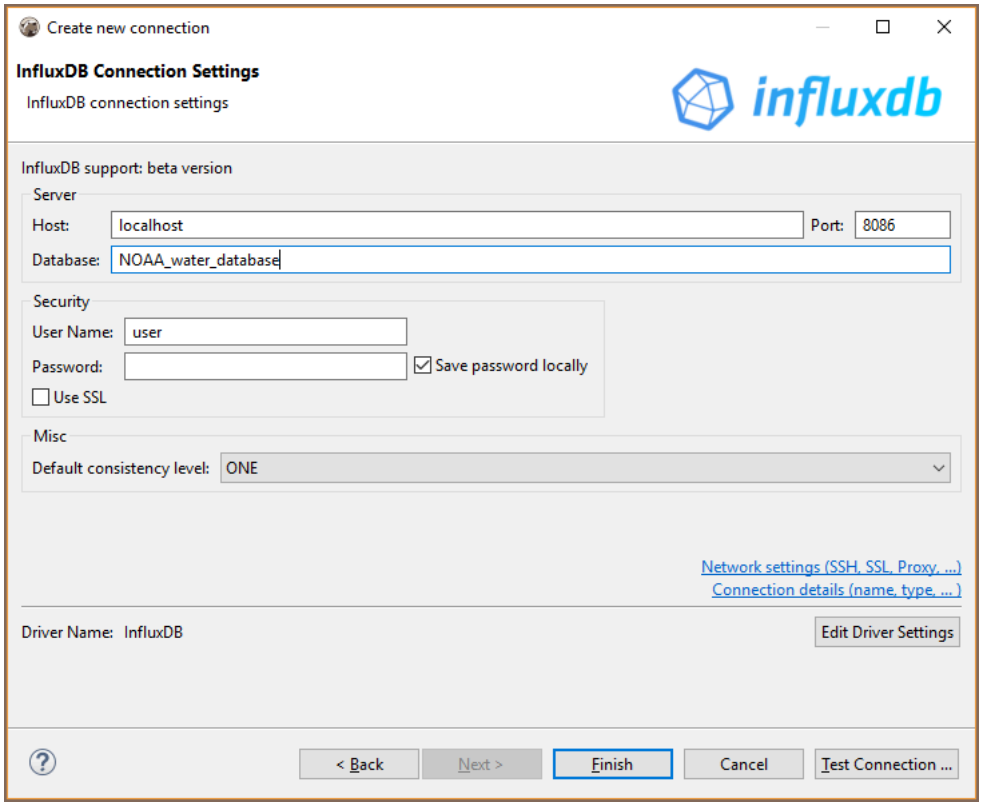
Table of contents

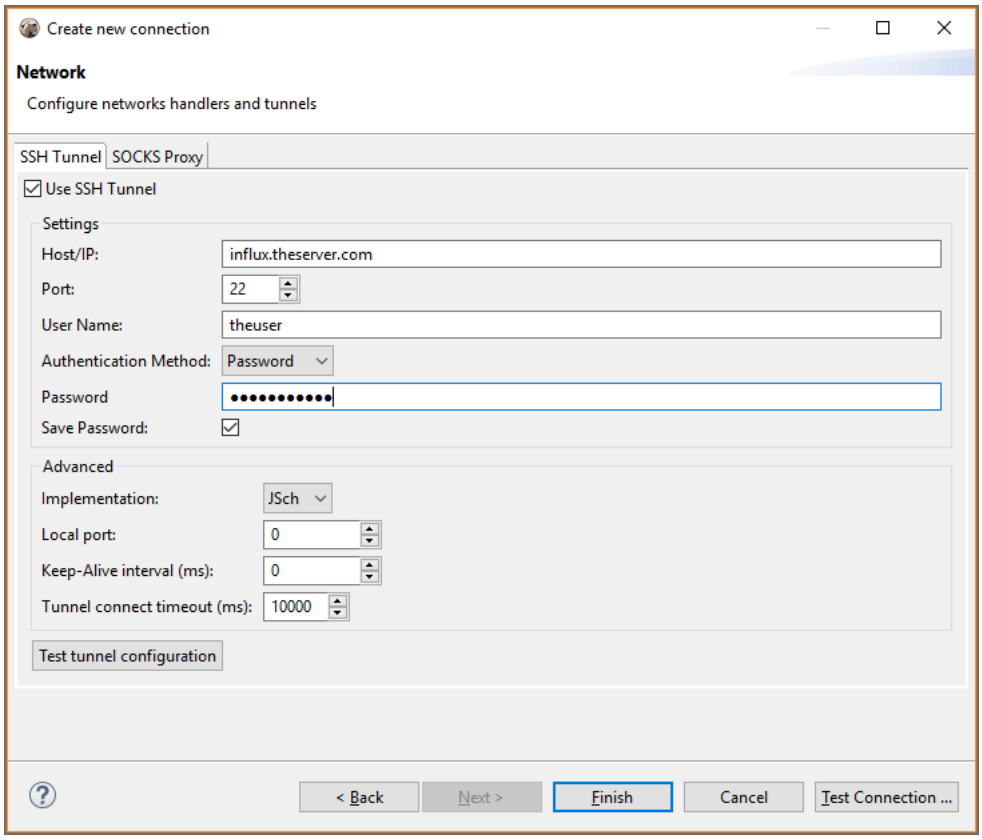
- Connecting to Influx Server
- Browsing InfluxDB schema
- Executing InfluxQL
- Executing Flux
- Query example

DBeaver supports InfluxDB schema browser, data viewer and InfluxQL queries execution.
DBeaver uses InfluxDB Java driver 2.12 to operate with the server over HTTP/HTTPS (standard InfluxDB protocol).
It supports InfluxDB servers of any version (in the moment of writing).

Connecting to Influx Server

You can connect directly to a server or use SSH tunneling or SOCKS proxy.

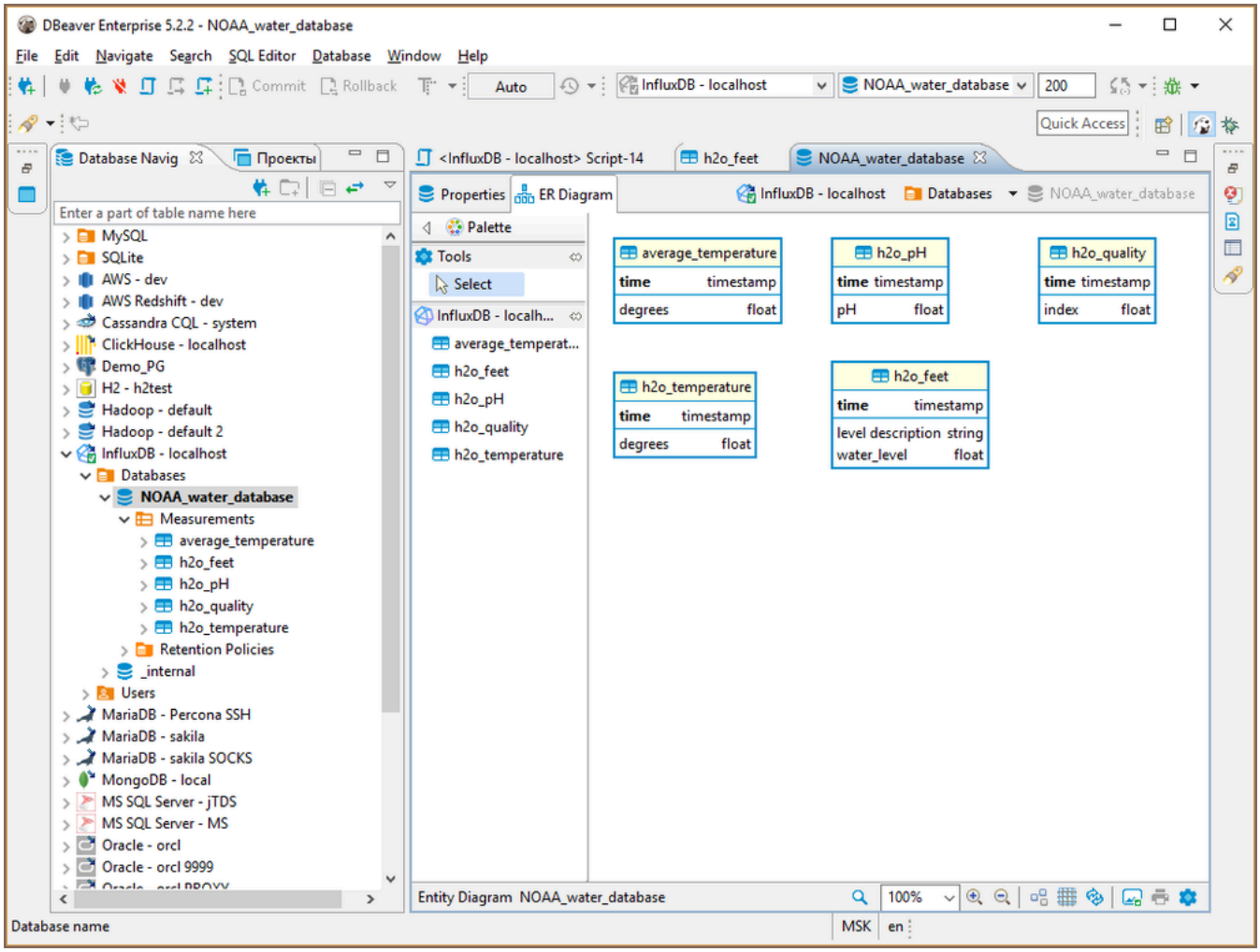




Browsing InfluxDB schema

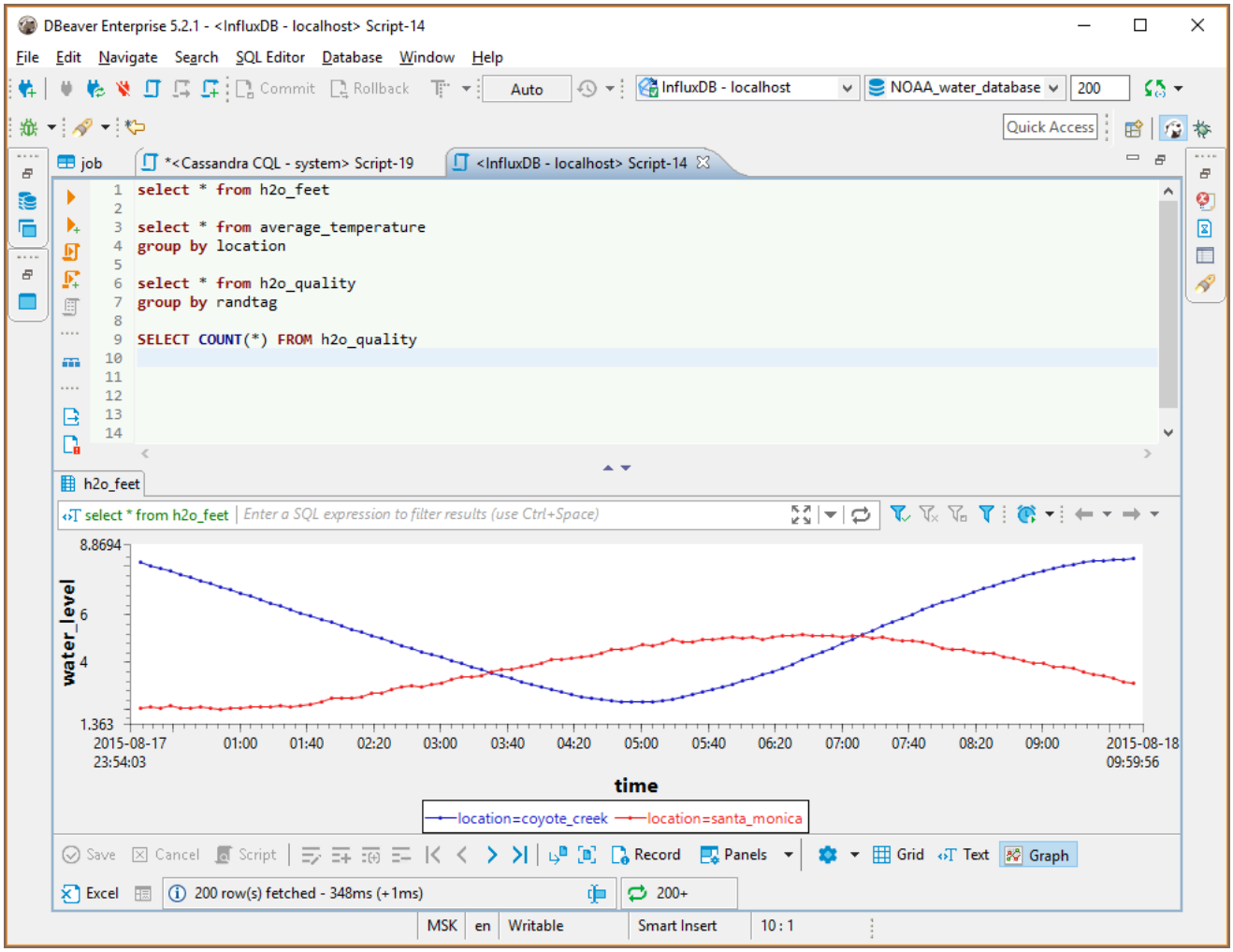
InfluxDB is TimeSeries database, it does not support tables, foreign keys and other relational entities.

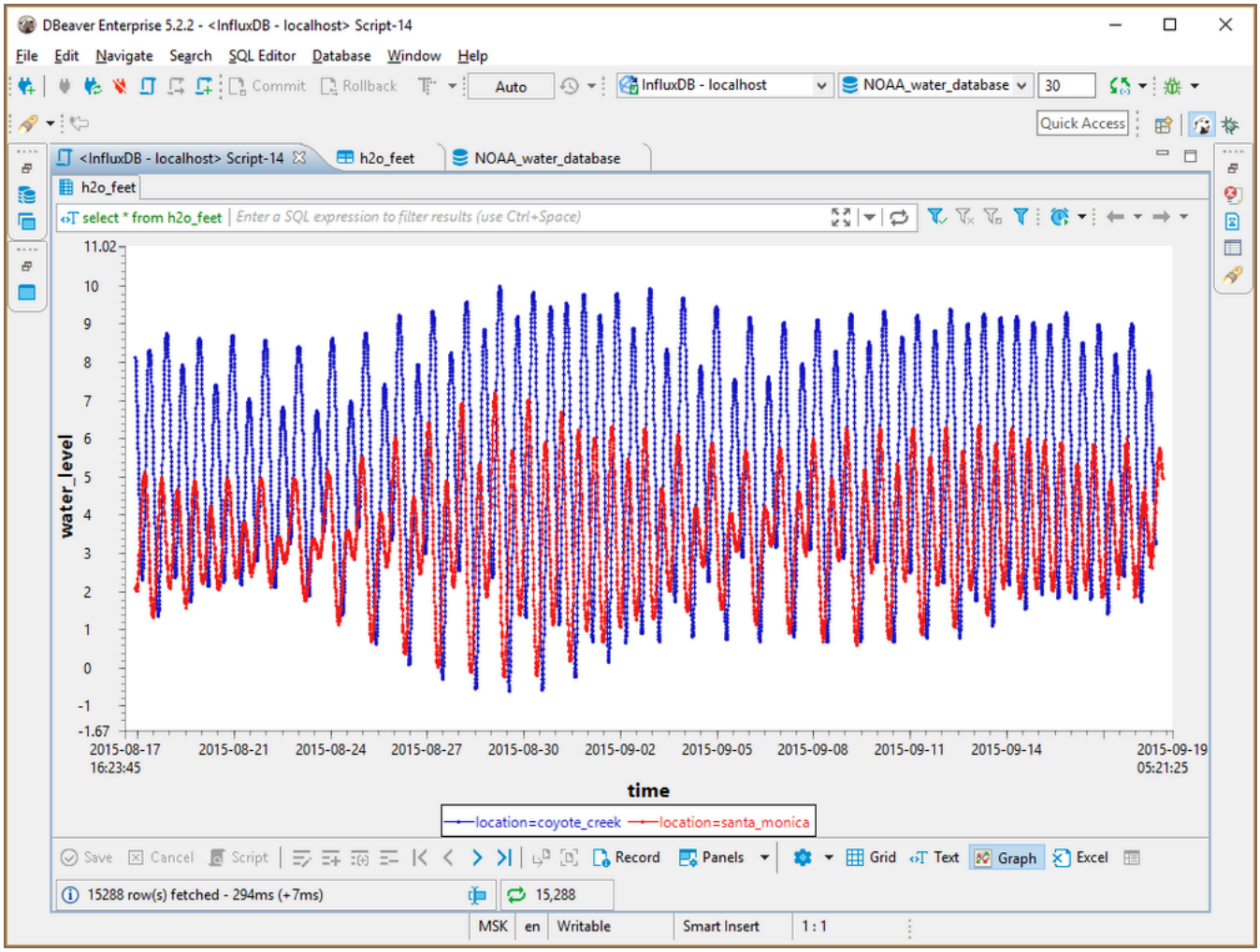
DBeaver does not support data insert/update in InfluxDB. Database is basically a in read-only state for DBeaver. You can browse schema and view/analyse data.
While data itself is loaded by various sensors/data collectors in real time.
Instead of tables InfluxDB has measurements. Instead of columns it has fields and tags.



Executing InfluxQL

versions lower than 2 InfluxQL is a query language similar to SQL.
DBeaver fully supports all InfluxQL statements. Query results are presented as grid or as graphs:


DBeaver Enterprise 5.2.1 - <InfluxDB - localhost> Script-14
File Edit Navigate Search SQL Editor Database Window Help
Commit Rollback Auto InfluxDB - localhost NOAA_water_database 200
Quick Access
job
*<Cassandra CQL - system> Script-19
<InfluxDB - localhost> Script-14
select * from h2o_feet
select * from average_temperature
group by location
select * from h2o_quality
group by randtag
SELECT COUNT(*) FROM h2o_quality
h2o_feet
select * from h2o_feet
Enter a SQL expression to filter results (use Ctrl+Space)
water_level
8.8694
6
4
1.363
2015-08-17 23:54:03
01:00
01:40
02:20
03:00
03:40
04:20
05:00
05:40
06:20
07:00
07:40
08:20
09:00
2015-08-18 09:59:56
time
location=coyote_creek
location=santa_monica
Save Cancel Script
Record
Panels
Grid
Text
Graph
Excel
200 row(s) fetched - 348ms (+1ms)
200+
MSK
en
Writable
Smart Insert
10 : 1



Executing Flux

versions higher than 2 Flux is a query language that is used in the new versions of Influx. It is not similar to InfluxQL and doesn't use SQL syntax.

Query example

```
from(bucket: "example-bucket")
    |> range(start: -1d)
    |> filter(fn: (r) => r._measurement == "example-measurement")
    |> mean()
    |> yield(name: "_results")
```

MariaDB

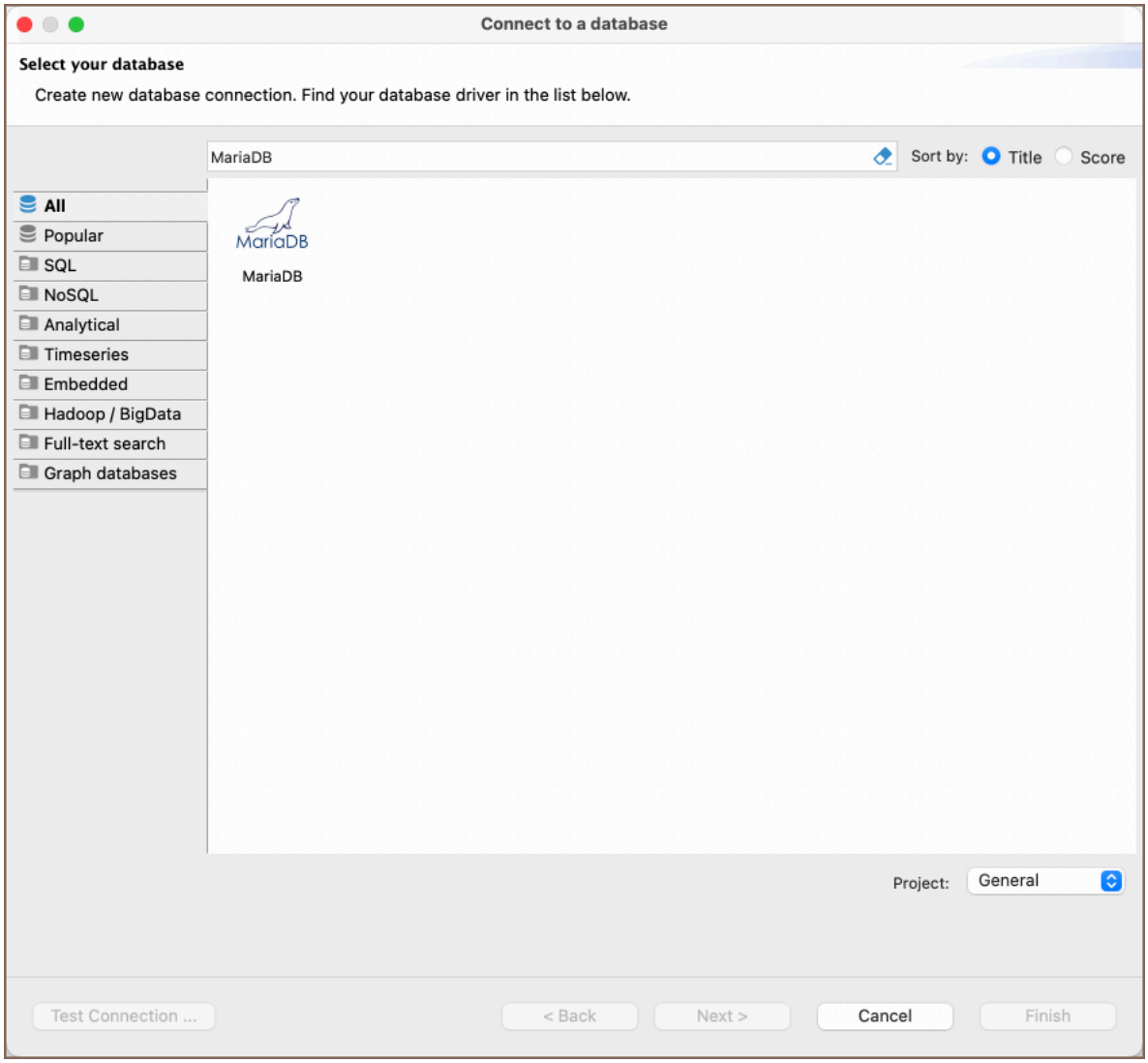
Table of contents

- Setting Up
- MariaDB connection settings
  - Connection details
- MariaDB driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering MariaDB with DBeaver
- MariaDB database objects
- MariaDB features in DBeaver

This guide provides instructions on how to set up and use MariaDB with DBeaver. DBeaver provides specialized features like support for unique data types including MariaDB Enumerations and Sets, as well as a variety of storage engines such as InnoDB and MyISAM.

Before you start, you must create a connection in DBeaver and select MariaDB. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the MariaDB server using a specific driver, it supports all versions of MariaDB.



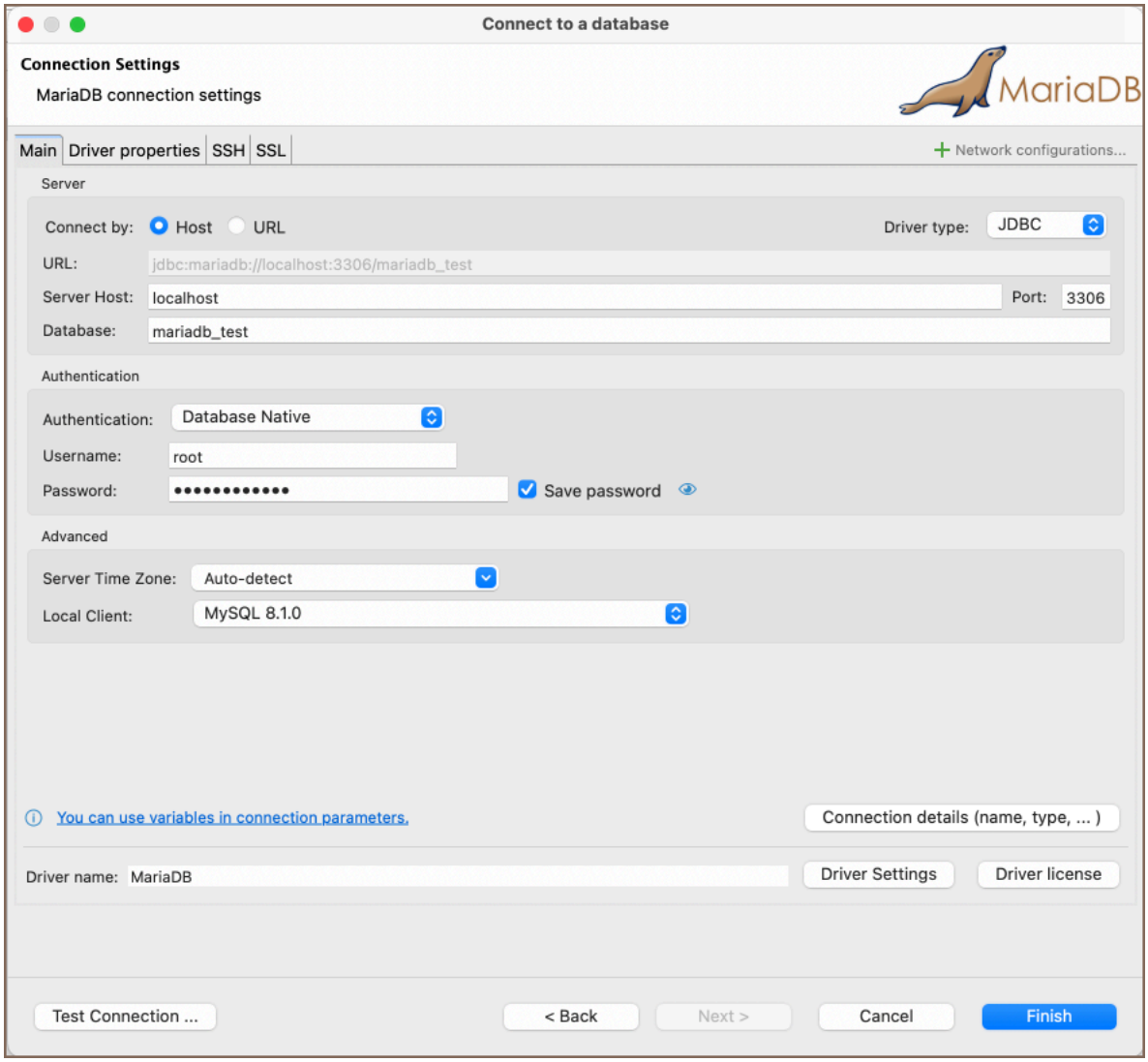
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for MariaDB.

MariaDB connection settings

In this subsection, we will outline the settings for establishing a direct connection to a MariaDB database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your MariaDB database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your MariaDB database.
URL	If you are connecting via URL, enter the URL of your MariaDB database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your MariaDB database here.
Database	Enter the name of the MariaDB database you want to connect to.
Show all databases	Displays all databases in the Connections. If not enabled, only the database from the connection is shown.
Port	Enter the port number for your MariaDB database. The default MariaDB port is `3306`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - AWS RDS IAM Authentication ★ - Azure AD Authentication for MySQL ★ - DBeaver Profile Authentication ★ - Kerberos Authentication ★ - Two-factor Authentication ★ You can also read about security in DBeaver PRO.
Local Client	Specify the local client.
Server Time Zone	This setting is responsible for determining the time zone used by the MariaDB server.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with MariaDB database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

MariaDB driver properties

The settings for MariaDB **Driver properties** enable you to adjust the performance of the MariaDB JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your MariaDB database.

For a complete walkthrough on setting up MariaDB JDBC and ODBC drivers properties, you can refer to the official MariaDB JDBC documentation and MariaDB ODBC documentation. These guides detail each driver's properties and how they can be used to optimize MariaDB database connections.

You can customize the MariaDB driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your MariaDB database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your MariaDB database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering MariaDB with DBeaver

DBeaver provides a host of features designed for MariaDB databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

MariaDB database objects

DBeaver lets you view and manipulate a wide range of MariaDB database objects. DBeaver has extensive support for various MariaDB metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Tables
    - Columns
    - Constraints
    - Foreign Keys
    - References
    - Triggers
    - Indexes
    - Partitions
  - Views
  - Procedures
  - Packages
  - Events
- Users
- Administer
  - Session Manager

MariaDB features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for MariaDB. Beyond regular SQL operations, DBeaver provides a range of MariaDB-specific capabilities, such as:

Category	Feature
Data Types	MariaDB Enumerations and Sets
Security	MariaDB User Management
Storage Engines	InnoDB, MyISAM, MRG_MyISAM, Archive, Blackhole, CSV, Memory, Performance_schema

Category	Feature
Database Management	MariaDB Events
	MariaDB Stored Procedures

Additional features compatible with MariaDB, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

Microsoft SQL Server

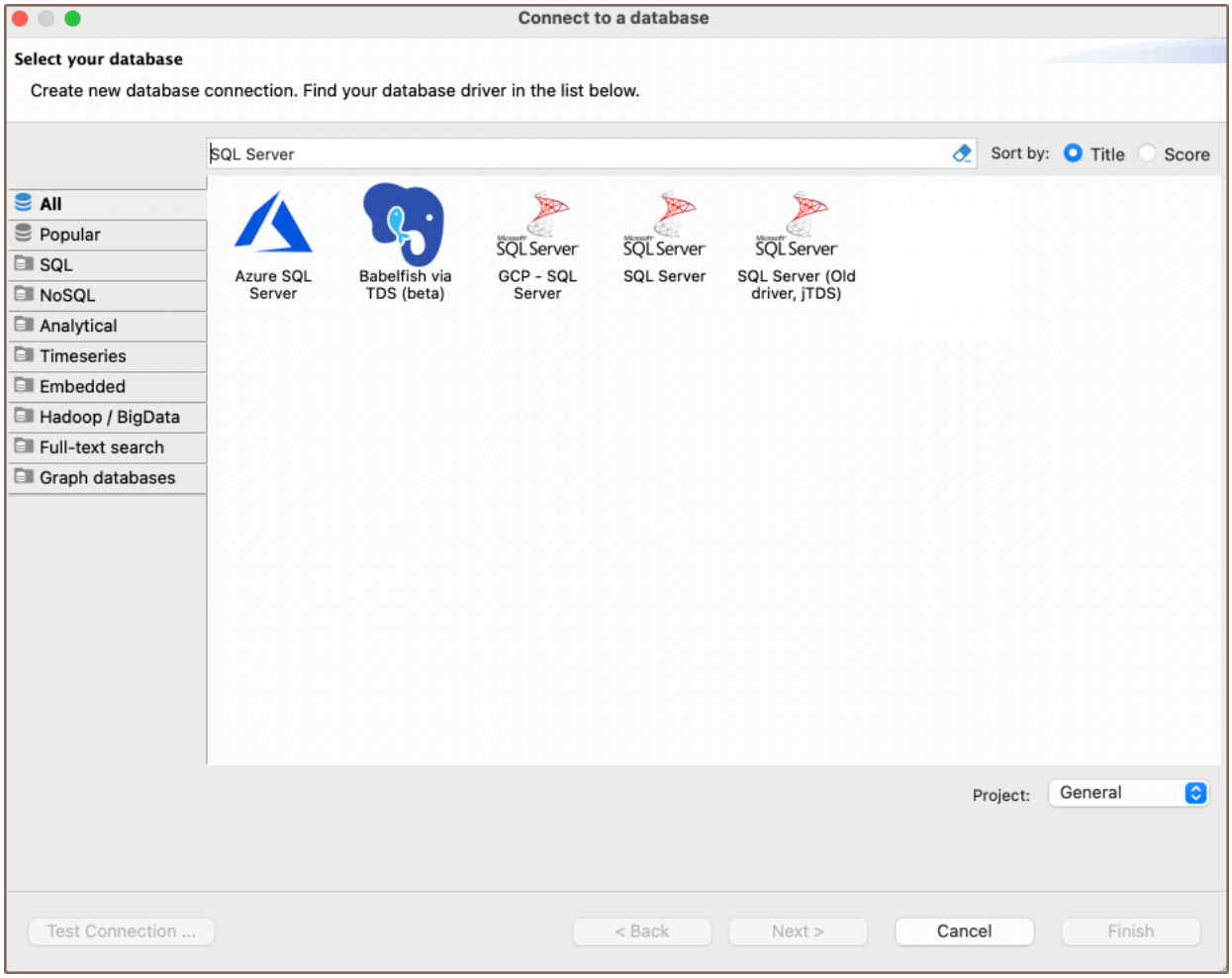
Table of contents

- Setting Up
- SQL Server connection settings
  - Connection details
- SQL Server driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering SQL Server with DBeaver
- SQL Server database objects
- SQL Server features in DBeaver

This guide provides instructions on how to set up and use SQL Server with DBeaver.

Before you start, you must create a connection in DBeaver and select SQL Server. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the SQL Server using a specific driver, it supports all versions of SQL Server, but the correct driver must be selected: use **SQL Server (Old driver, jTDS)** for versions below 2012 and **SQL Server** for newer versions. DBeaver also supports SQL Server extensions such as **Azure SQL Server**, **Babelfish via TDS** and **Google Cloud Platform SQL Server**. You must select the appropriate driver in the **Connect to a database** window for these extensions.



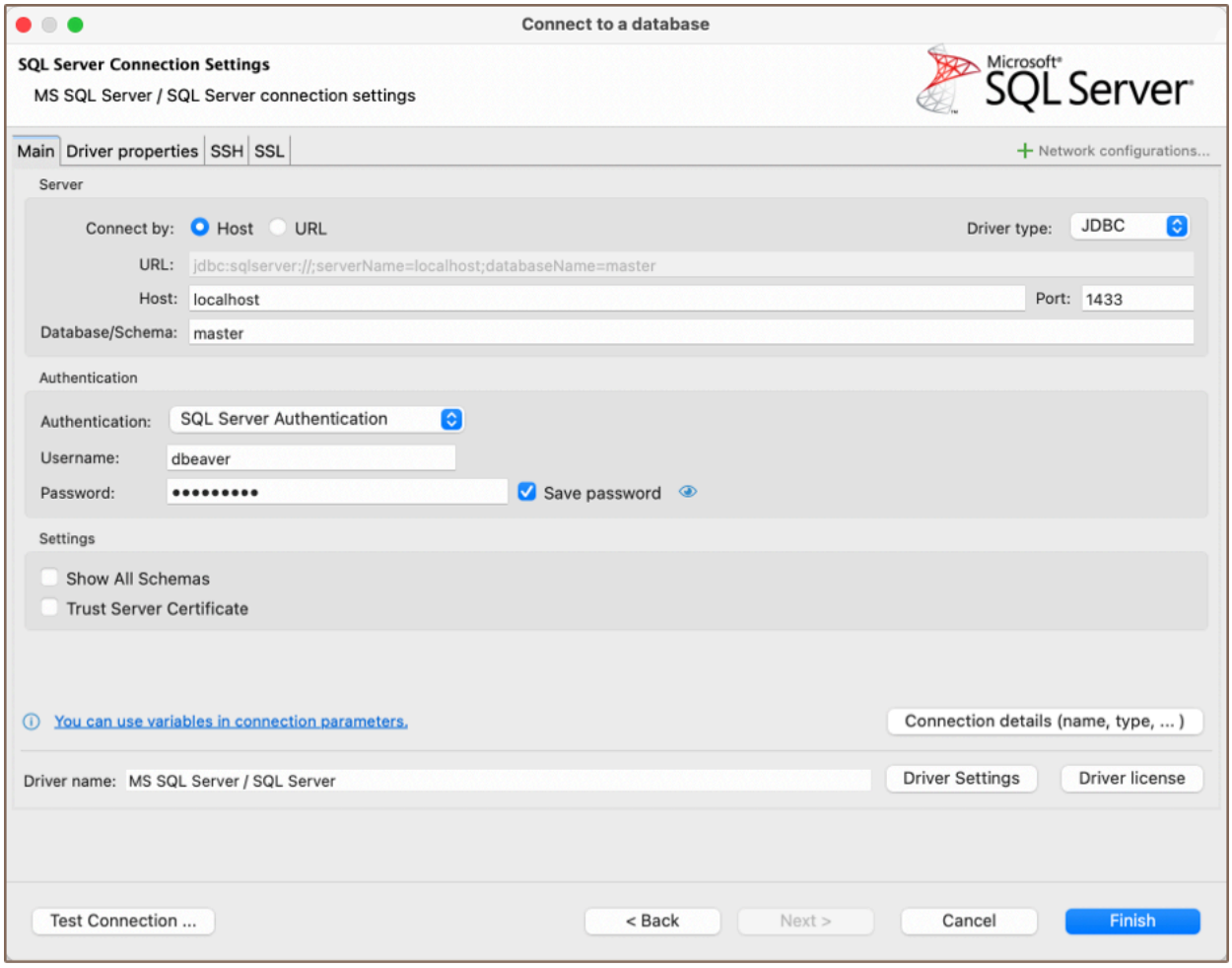
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection, the configuration of secure connections, and the setup of ODBC/JDBC drivers for SQL Server.

SQL Server connection settings

In this subsection, we will outline the settings for establishing a direct connection to a SQL Server database using DBeaver.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your SQL Server database.
URL	If you are connecting via URL, enter the URL of your SQL Server database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your SQL Server database here.
Database/Schema	Enter the name of the SQL Server database you want to connect to.
Port	Enter the port number for your SQL Server database. The default SQL Server port is `1433`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - SQL Server Authentication (coming soon) - Password from shell command - Active Directory - Integrated (coming soon) - Active Directory - MFA (coming soon) - Active Directory - MSI (coming soon) - Active Directory - Password (coming soon) - Azure AD for SQL Server ★ - Custom (coming soon) - DBeaver profile ★ - Kerberos ★ - NTLM (coming soon) - Window Authentication (coming soon) You can also read about security in DBeaver PRO.
Show All Schemas	Shows all database schemas. If disabled shows only non-empty schemas.
Trust Server Certificate	If `true`, the SQL Server SSL certificate is automatically trusted when the communication layer is encrypted using SSL. If `false`, the Microsoft JDBC Driver for SQL Server validates the server SSL certificate. If the server certificate validation fails, the driver raises an error and terminates the connection. To configure certificate settings in DBeaver, refer to our detailed article on Truststore.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with SQL Server database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

SQL Server driver properties

The settings for SQL Server **Driver properties** enable you to adjust the performance of the SQL Server JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your SQL Server database.

For a complete walkthrough on setting up SQL Server JDBC and ODBC drivers properties, you can refer to the official SQL Server JDBC documentation and SQL Server ODBC documentation. These guides detail each driver's properties and how they can be used to optimize SQL Server database connections.

You can customize the SQL Server driver in DBeaver via the **Edit Driver** window, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This window offers a range of settings that can influence your SQL Server database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your SQL Server database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering SQL Server with DBeaver

DBeaver provides a host of features designed for SQL Server databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

SQL Server database objects

DBeaver lets you view and manipulate a wide range of SQL Server database objects. DBeaver has extensive support for various SQL Server metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Tables
    - External Tables
    - Views
      - Columns Triggers
      - External Properties
    - Indexes
    - Procedures
    - Sequences
    - Synonyms
    - Triggers
    - Data Types
  - Database triggers
- Security
  - Logins
- Administer
  - Sessions
- Agent
  - Jobs

SQL Server features in DBeaver

DBeaver includes many features that go beyond basic SQL operations like querying, updating, inserting, and deleting data. It also includes numerous unique features specifically for SQL Server. Beyond regular SQL operations, DBeaver offers a range of SQL Server-specific capabilities, such as:

Category	Feature
Data Structure	SQL Server Tables & Views
	SQL Server Procedures
	SQL Server Sequences
	SQL Server Synonyms
	SQL Server External Tables
Job Management	Agent Jobs

Additional features compatible with SQL Server, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

MongoDB

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

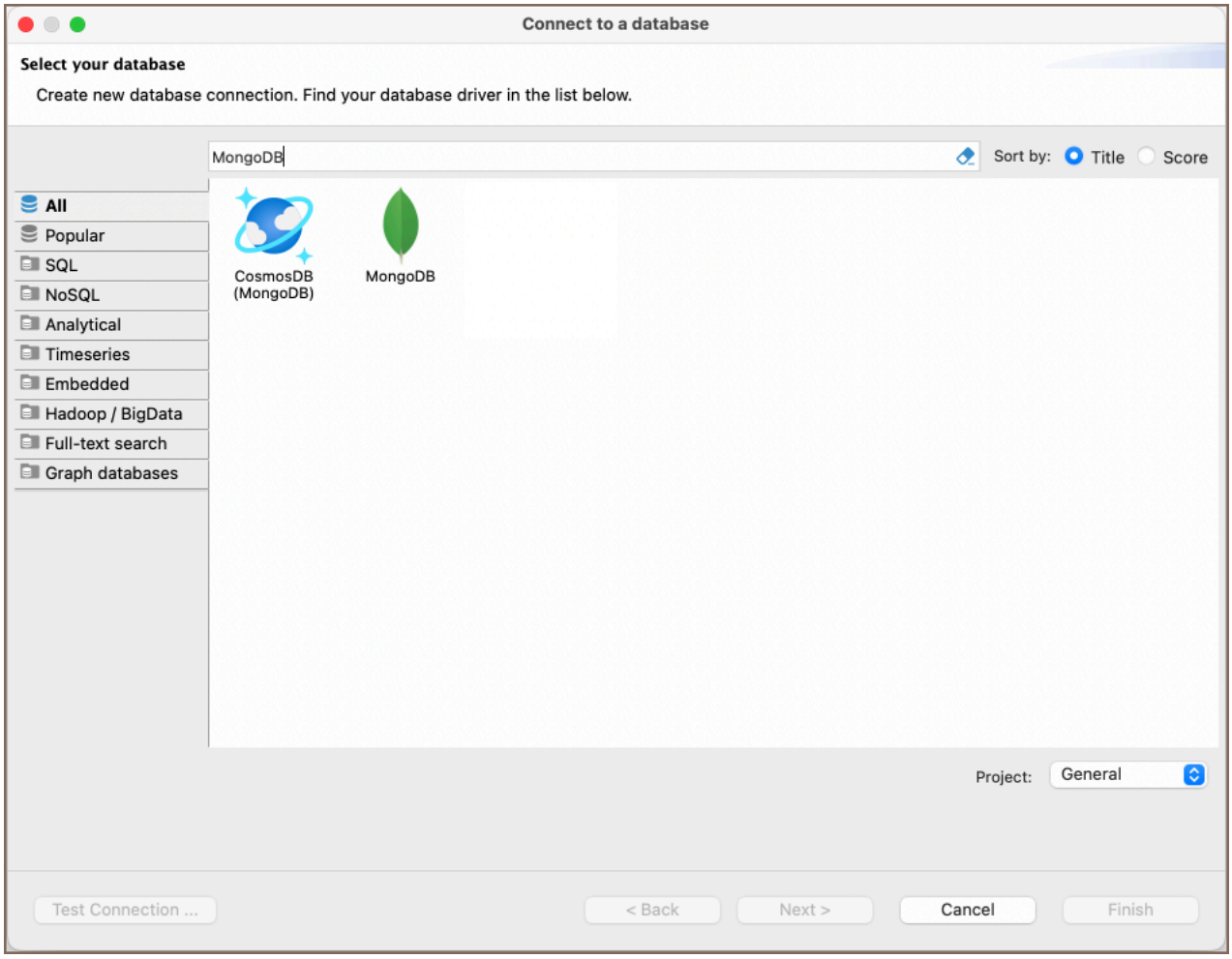
- Setting Up
- MongoDB connection settings
  - Connection details
- MongoDB driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering MongoDB with DBeaver
- MongoDB database objects
- MongoDB features in DBeaver
  - Browsing MongoDB collections
- Database operations
  - Executing JavaScript
  - Executing SQL
  - SELECT queries
    - Conditions
    - Nested fields
    - Working with object IDs
    - Working with JOINs
    - Aggregate functions
  - INSERT statement
  - UPDATE statement
  - DELETE statement
  - CREATE TABLE statement
  - DROP TABLE statement
  - Working with dates

This guide provides instructions on how to set up and use MongoDB with DBeaver.

One of the standout features of DBeaver's MongoDB support is its flexibility in presentation. You can view MongoDB collections as standard relational tables, JSON documents, or even in chart presentations.

DBeaver interacts with MongoDB servers using a specific driver, supporting versions from 2.x to the current version. DBeaver also supports MongoDB extension such as Cosmos DB, you can find more information about this driver in our article.

Before you start, you must create a connection in DBeaver and select MongoDB. If you have not done this, please refer to our Database Connection article.



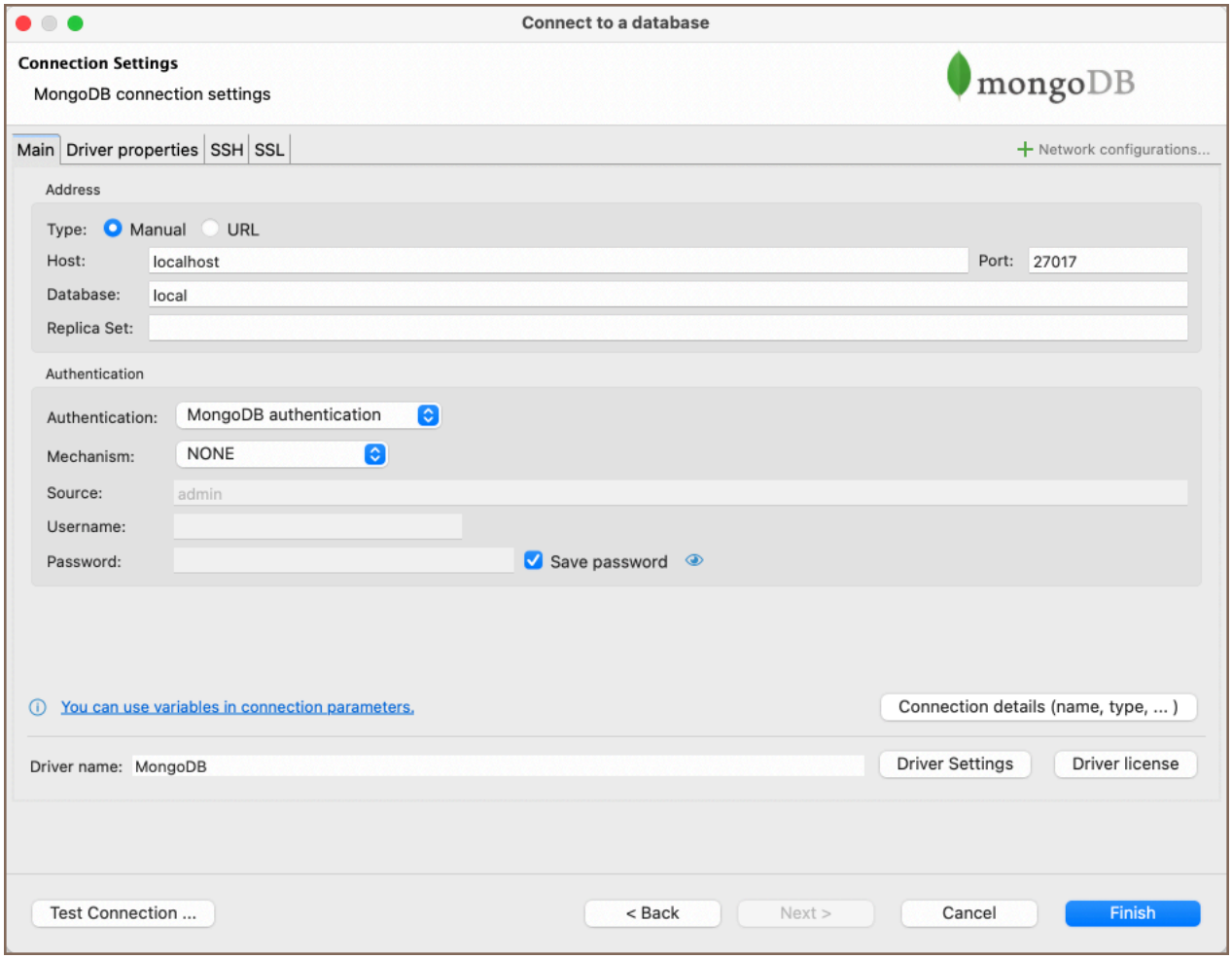
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies and SSL.

MongoDB connection settings

In this subsection, we will outline the settings for establishing a direct connection to a MongoDB database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your MongoDB database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
URL	If you are connecting via URL, enter the URL of your MongoDB database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your MongoDB database here.
Database	Enter the name of the MongoDB database you want to connect to.
Replica Set	Specify the name of the replica set if your MongoDB instance is a part of a replica set configuration.
Port	Enter the port number for your MongoDB database. The default MongoDB port is `27017`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - MongoDB Authentication - Password from shell command - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with MongoDB database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

MongoDB driver properties

You can customize the MongoDB driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your MongoDB database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your MongoDB database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering MongoDB with DBeaver

DBeaver provides a host of features designed for MongoDB databases. This includes the ability to view and manage collections, along with numerous unique capabilities aimed at optimizing database operations.

MongoDB database objects

DBeaver lets you view and manipulate a wide range of MongoDB database objects. DBeaver has extensive support for various MongoDB metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
- Collections
- Java Script
- Users
- Administration
- Active Operations

MongoDB features in DBeaver

DBeaver is not confined to handling typical SQL tasks. It also embraces the NoSQL database spectrum, offering numerous unique features specifically designed for MongoDB. Beyond standard SQL operations, DBeaver facilitates a plethora of MongoDB-specific capabilities, such as:

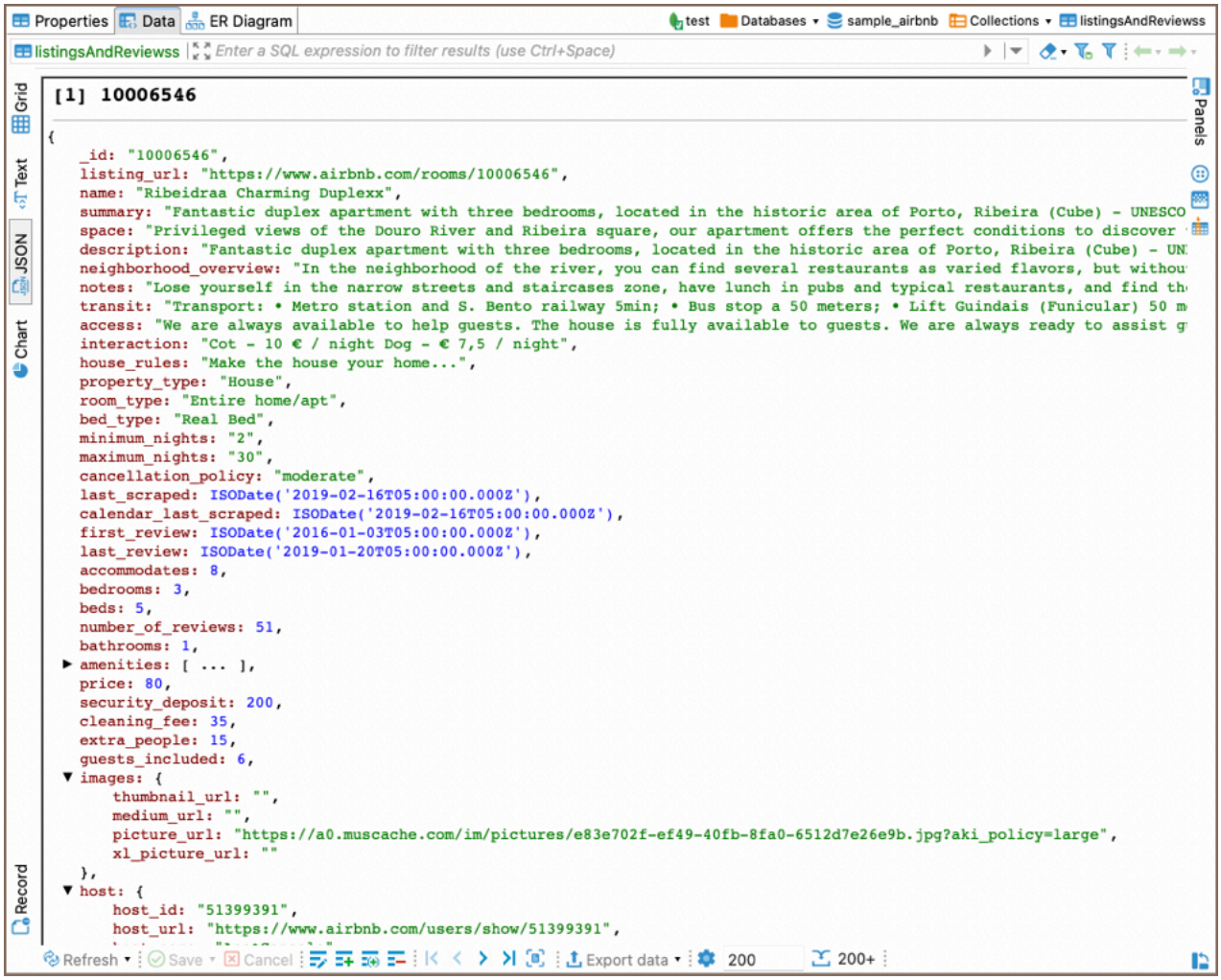
Category	Feature
Data Types	BSON Data Types (e.g., Object ID, ISODate)
Security	User-Based Access Control
Database Management	MongoDB Stored Procedures (JavaScript Functions)

Additional features compatible with MongoDB, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager

Browsing MongoDB collections

You can view or edit MongoDB collection content as standard relational tables (in grid/plain text presentations/chart) or as JSON documents. You can switch between these presentations using the toolbar of the Data Editor.



Database operations

Executing JavaScript

Execute JavaScript statements in the SQL editor as usual. DBeaver supports all JavaScript queries for MongoDB versions 2 and 3, as well as a subset of the `mongo` shell queries.

Here is an example that creates a user in the current database:

```
db.createUser({
    user: 'testuser',
    pwd: 'test',
    roles: []
})
```

This example returns all documents in the collection 'test_col':

```
db.test_col.find()
```

> **Note**: Scripts will be executed in the current database. You can not set an explicit database name in your query. The current database can be changed on the SQL Editor toolbar or on the Connections.

Executing SQL

You can use standard SQL statements (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) to manipulate data in MongoDB.

SELECT queries

SELECT queries can include `WHERE`, `ORDER BY`, `GROUP BY`, `JOIN` and `HAVING` clauses.

```
SELECT * FROM test_col
WHERE propName.subProp='value'

UPDATE FROM test_col
SET propsName.val1=123
WHERE propName.subProp='value'
```

> **Note**: The MongoDB dialect does not support SQL sub-queries.

Conditions

SELECT queries with `WHERE` clauses support `AND`, `OR`, `<`, `<=`, `>`, `>=`, `=` and `!=` operators:

```
SELECT * FROM Employees
WHERE (Country = 'CA' OR Country = 'RU') AND Age > 20;
```

> **Tip**: Be aware that `AND` has a higher precedence than `OR` and will be evaluated first; enclose it with parentheses to maintain the correct order.

Nested fields

You can differentiate nested JSON fields using a dot. Enclose fields containing special characters (like spaces or dashes) with double quotes, as demonstrated below:

```
SELECT title FROM movies WHERE info."imdb-details".rating > 6
```

Working with object IDs

To find a document by ID, use the `ObjectId` function:

```
SELECT * FROM documents
WHERE _id = ObjectId('5f9c458018e3c69d0adc0fbd')
ORDER BY value DESC
```

Working with JOINs

The SQL dialect for MongoDB supports `LEFT JOIN` and `INNER JOIN` currently:

```
SELECT
    ar.Name as Artist,
    al.Title as Album,
    SUM(tr.Milliseconds) as Duration
FROM Track tr
INNER JOIN Album al ON tr.AlbumId = al.AlbumId
INNER JOIN Artist ar ON al.ArtistId = ar.ArtistId
GROUP BY Artist, Album
ORDER BY Duration DESC
```

Remember to specify aliases for both the source and target tables in a defined order, as shown:

```
SELECT *
FROM <source> <source-alias>
INNER JOIN <target> <target-alias> ON <source-alias>.column = <target-alias>.column
```

Executing the script below does not yield a merged document but produces separate documents for `Track` and `Album`:

```
SELECT *
FROM Track tr
INNER JOIN Album al ON tr.AlbumId = al.AlbumId
```

Aggregate functions

In version 22.x only `COUNT` function is supported.

INSERT statement

You cannot use conditions in `INSERT` statements, only the basic form is supported:

```
INSERT INTO <collection-name> (field1, field2) VALUES (val1, val2);
```

UPDATE statement

While you can use various expressions in the `WHERE` clause, sub-selects or joins are not permissible.

```
UPDATE <collection-name> SET field2=val3 WHERE field1=val1;
```

DELETE statement

You may use any expression in the `WHERE` clause, but sub-selects or joins are not allowed.

```
DELETE FROM <collection-name> WHERE field1=val1;
```

CREATE TABLE statement

In the `CREATE TABLE` statement, only the collection name can be specified, column lists are not allowed.

```
CREATE TABLE  <collection-name>;
```

DROP TABLE statement

```
DROP TABLE  <collection-name>;
```

Working with dates

When working with dates, ensure to specify them in ISO format. This is applicable in both JavaScript and SQL dialects:

```
db.dates.insert([
    { value: new Date('2016-05-18T16:00:00Z') },
    { value: new Date('2017-05-18T16:00:00Z') },
    { value: new Date('2018-05-18T16:00:00Z') },
    { value: new Date('2019-05-18T16:00:00Z') },
    { value: new Date('2020-05-18T16:00:00Z') }
])
```

To query data in JavaScript, follow this example:

```
db.dates.find({
    value: { $gte: new Date('2018-05-18T16:00:00Z') }
})
```

When querying data in the SQL dialect, you can use either the ISO format or UNIX timestamp (in milliseconds):

```
SELECT value FROM dates
WHERE value > ISODate('2018-05-18T16:00:00.000Z')
ORDER BY value DESC

SELECT value FROM dates
WHERE value > ISODate(1526659200000)
ORDER BY value DESC
```

MySQL

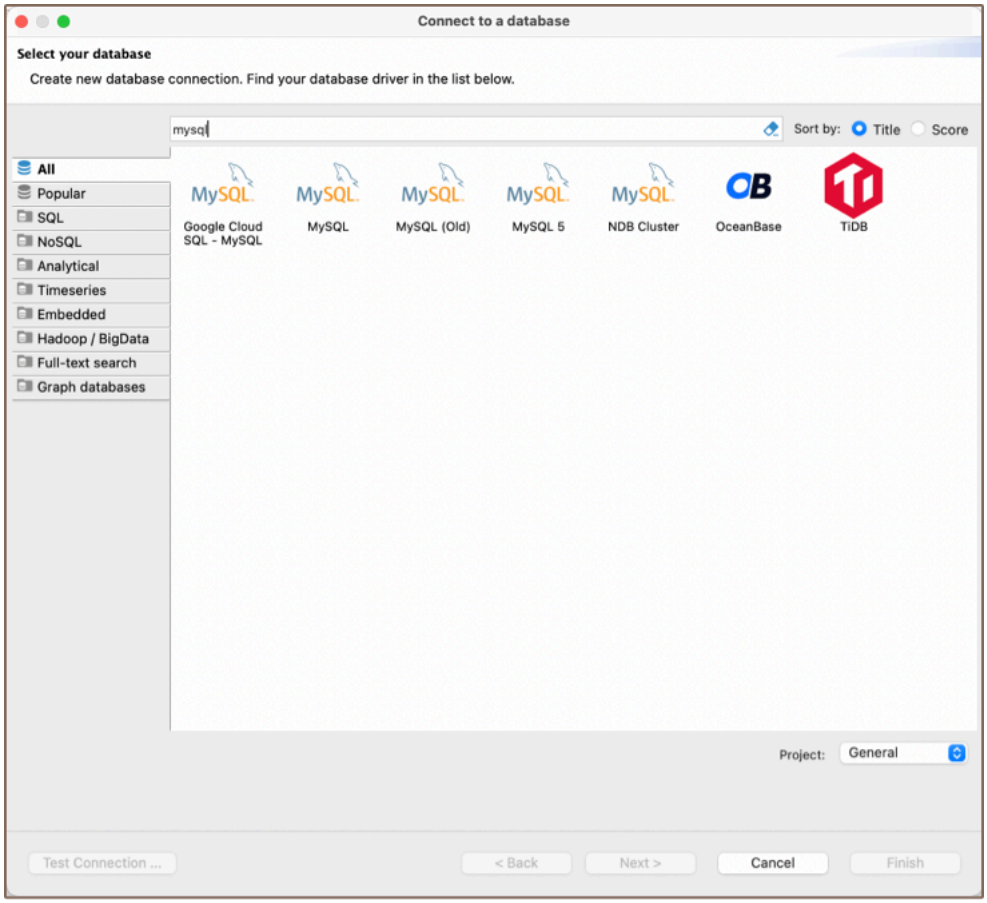
Table of contents

- Setting Up
- MySQL connection settings
    - Connection details
- MySQL driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering MySQL with DBeaver
- MySQL database objects
- MySQL Features in DBeaver

This guide provides instructions on how to set up and use MySQL with DBeaver.

Before you start, you must create a connection in DBeaver and select MySQL. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the MySQL server using a specific driver. It supports all versions of MySQL, but the correct driver must be selected: use MySQL (old) for versions 4.x and earlier, MySQL 5 for 5.x versions, and MySQL for version 8 and later. DBeaver also supports MySQL extensions such as NDB Cluster, OceanBase, and TiDB. For Google Cloud SQL, there is a dedicated Google Cloud SQL (MySQL) driver that supports MySQL version 8 and later. You must select the appropriate driver in the **Connect to a database** window for these extensions.



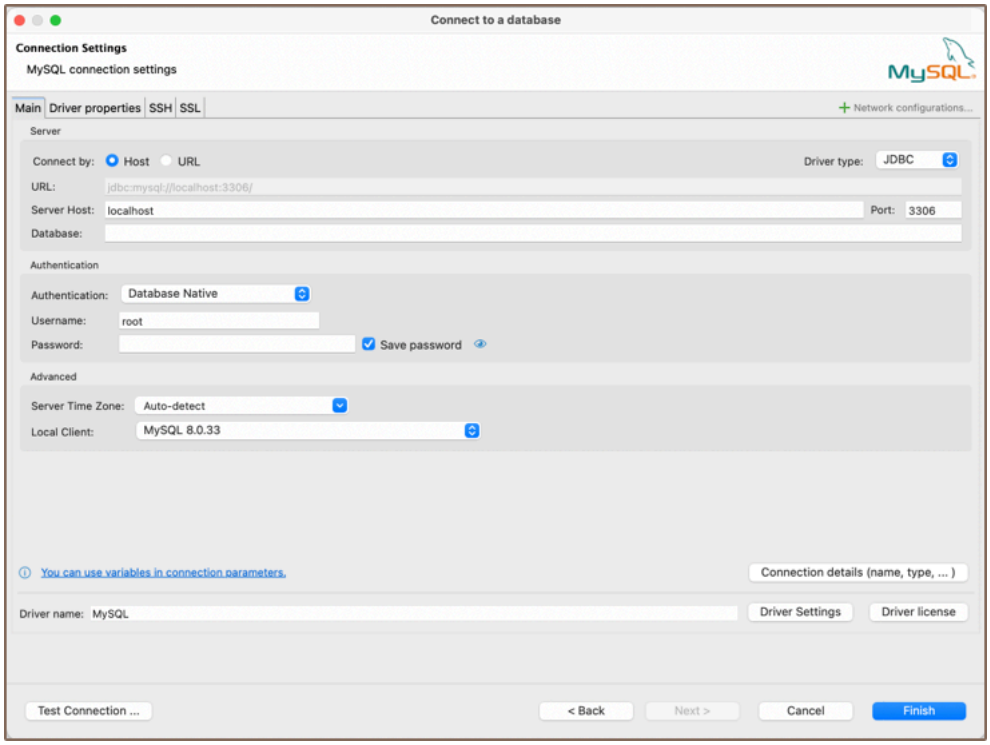
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for MySQL.

MySQL connection settings

In this subsection, we will outline the settings for establishing a direct connection to a MySQL database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your MySQL database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your MySQL database.
URL	If you are connecting via URL, enter the URL of your MySQL database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your MySQL database here.
Database	Enter the name of the MySQL database you want to connect to.
Show all databases	Displays all databases in the Connections. If not enabled, only the database from the connection is shown.
Port	Enter the port number for your MySQL database. The default MySQL port is `3306`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - AWS RDS IAM Authentication ★ - Azure AD Authentication for MySQL ★ - DBeaver Profile Authentication ★ - MySQL Kerberos Authentication ★ - Two-factor Authentication ★ You can also read about security in DBeaver PRO.
Local Client	Specify the local client, see our article about local client configuration.
Server Time Zone	This setting is responsible for determining the time zone used by the MySQL server.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with MySQL database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

MySQL driver properties

The settings for MySQL **Driver properties** enable you to adjust the performance of the MySQL JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your MySQL database.

For a complete walkthrough on setting up MySQL JDBC and ODBC drivers properties, you can refer to the official MySQL JDBC documentation and MySQL ODBC documentation. These guides detail each driver's properties and how they can be used to optimize MySQL database connections.

You can customize the MySQL driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your MySQL database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your MySQL database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering MySQL with DBeaver

DBeaver provides a host of features designed for MySQL databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

MySQL database objects

DBeaver lets you view and manipulate a wide range of MySQL database objects. DBeaver has extensive support for various MySQL metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Tables
    - Columns
    - Constraints
    - Indexes
    - Foreign Keys
    - References
    - Triggers
  - Views
  - Indexes
  - Procedures
  - Triggers
  - Events
- Users
- Administer
  - Session Manager

MySQL Features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for MySQL. Beyond regular SQL operations, DBeaver provides a range of MySQL-specific capabilities, such as:

Category	Feature
Data Types	MySQL Enumerations and Sets
Security	MySQL User Management
Storage Engines	InnoDB, MyISAM, MRG_MyISAM, Archive, Blackhole, CSV, Memory, Performance_schema
Database Management	MySQL Events
	MySQL Stored Procedures

Additional features compatible with MySQL, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

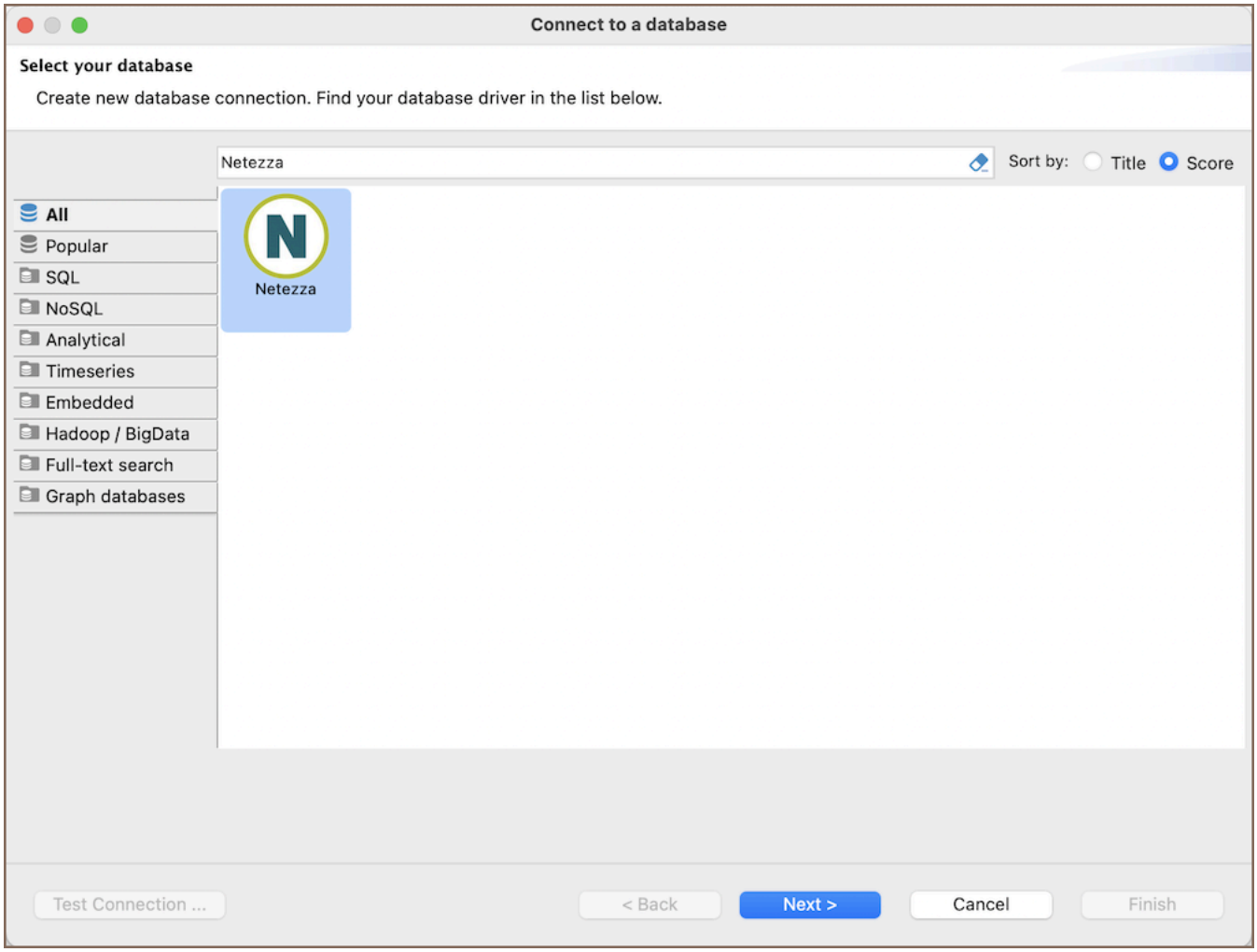
Netezza

Table of contents

- Netezza specialty
- Setting Up
- Netezza connection settings
- Connection details
- Netezza driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Netezza with DBeaver
- Netezza database objects
- Netezza features in DBeaver

This guide provides instructions on how to set up and use Netezza with DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Netezza` option to connect to the Netezza database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Netezza specialty

Netezza is a high-performance data warehousing appliance that excels in delivering fast data analytics and warehousing solutions through its massively parallel processing (MPP) architecture. This system is tailored for complex analytics on large datasets, often scaling to petabytes of data. Netezza simplifies analytics by allowing operations directly on compressed data, significantly reducing the storage requirements and enhancing query performance.

The integrated architecture of Netezza, which combines database, server, and storage into a single, streamlined system, facilitates the easy management and scaling of data warehousing capabilities.

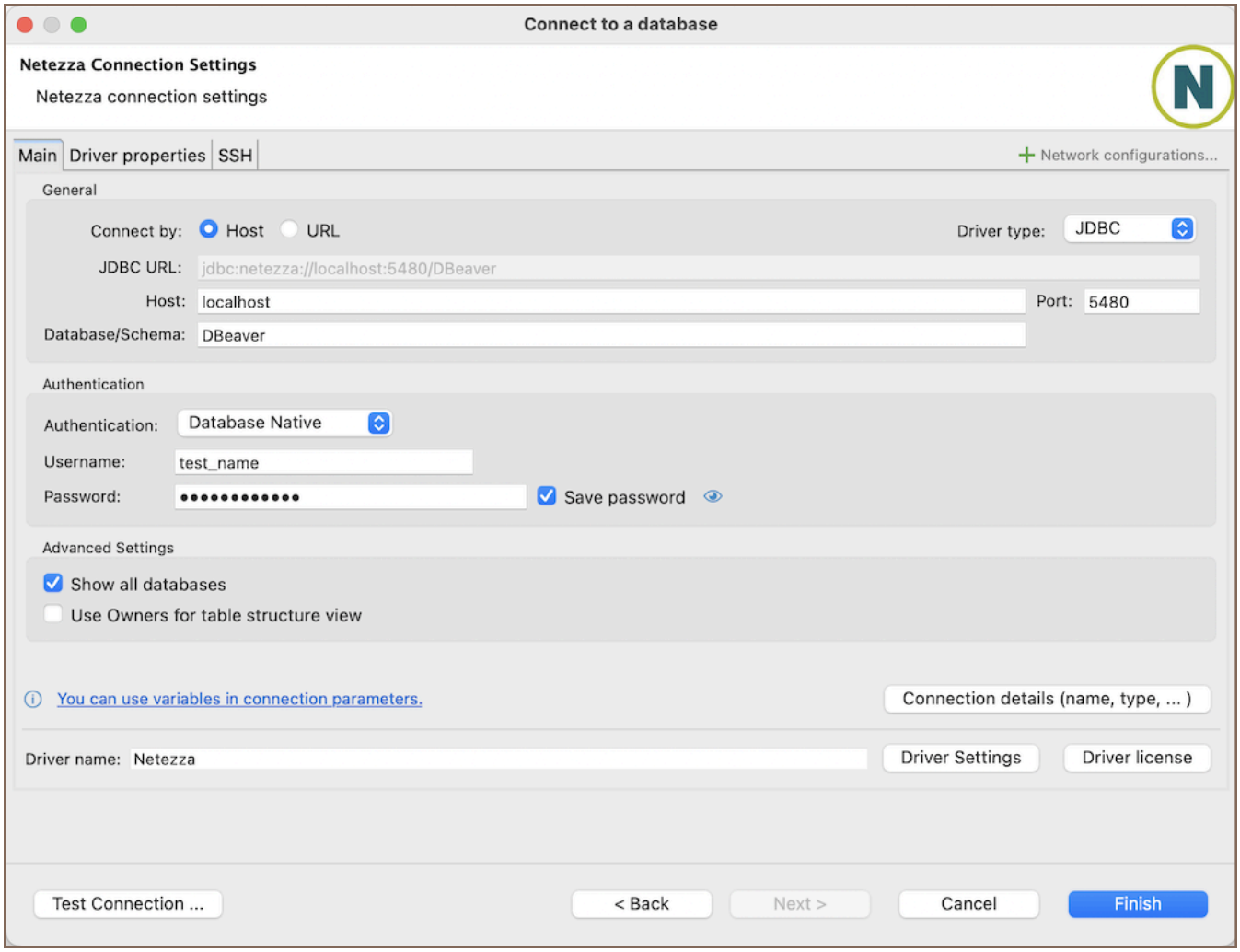
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, Kubernetes, SSL, and the setup of ODBC/JDBC drivers for Netezza.

Netezza connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Netezza database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Netezza database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Netezza database.
URL	If you are connecting via URL, enter the URL of your Netezza database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your Netezza database here.
Database/Schema	Enter the name of the Netezza database/schema you want to connect to.
Port	Enter the port number for your Netezza database. The default Netezza port is `5480`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ You can also read about security in DBeaver PRO.
Show all databases ★	Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Use Owners for table structure view ★	Show fully qualified table names in Connections with owners names instead schemas.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Netezza connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration

- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Netezza driver properties

The settings for Netezza **Driver properties** enable you to adjust the performance of the Netezza JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Netezza database.

For a complete walkthrough on setting up Netezza JDBC and ODBC drivers properties, you can refer to the official Netezza JDBC documentation. These guides detail each driver's properties and how they can be used to optimize Netezza database connections.

You can customize the Netezza driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Netezza database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Netezza database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Netezza with DBeaver

DBeaver provides a host of features designed for Netezza databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Netezza database objects

DBeaver lets you view and manipulate a wide range of Netezza database objects. DBeaver has extensive support for various Netezza metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Tables
      - Unique Keys
      - Primary Keys
      - Foreign Keys
    - Views
    - Procedures
    - Sequences ★
    - Synonyms ★
- Users ★
- Data types

Netezza features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous additional features compatible with Netezza, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management ★	Session Manager
Schema Management ★	Schema Compare
Data Visualization	ERD Guide
	Query Execution plan ★
Data Generation	Mock Data Generation ★

Oracle

Table of contents

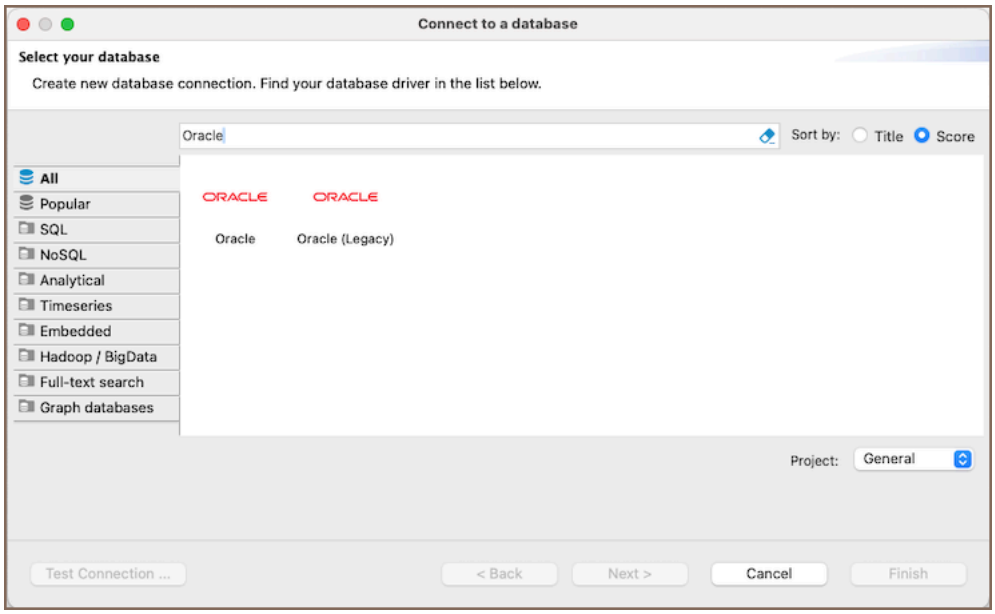
- Setting Up
- Oracle connection settings

    - Connection details
- Oracle driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Oracle with DBeaver
- Oracle database objects
- Oracle features in DBeaver

This guide provides instructions for setting up and using Oracle with DBeaver.

Before you start, you must create a connection in DBeaver and select Oracle. If you haven't done this, please refer to our Database Connection article.

DBeaver interacts with the Oracle server using a specific driver. It supports all versions of Oracle, but the correct driver must be selected: use 'Oracle (Legacy)' for versions `8.x`, `9.x`, `10.x` and 'Oracle' for version `11` and later. It also supports Oracle cloud database extensions such as Oracle Cloud JSON and Oracle NetSuite, both of which can be connected through Cloud Explorer.



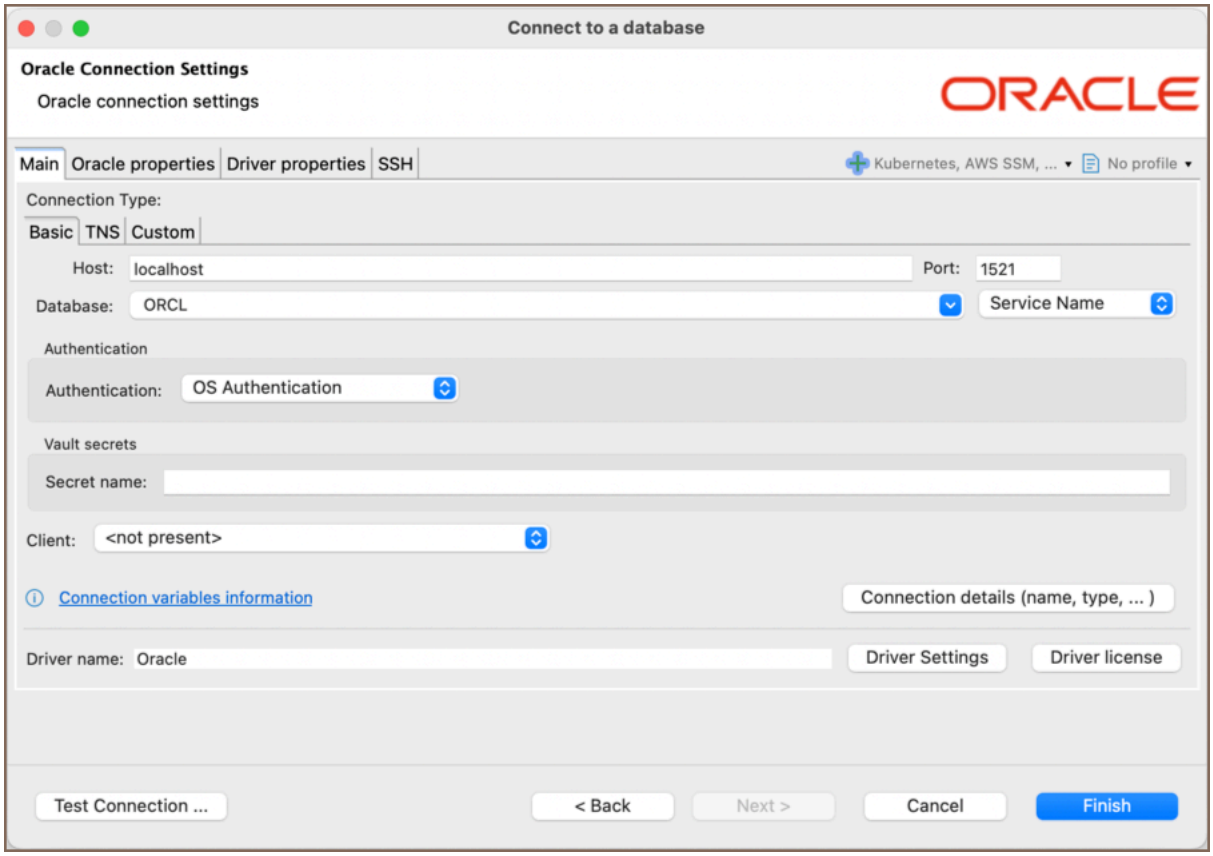
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, and the driver setup for Oracle.

Oracle connection settings

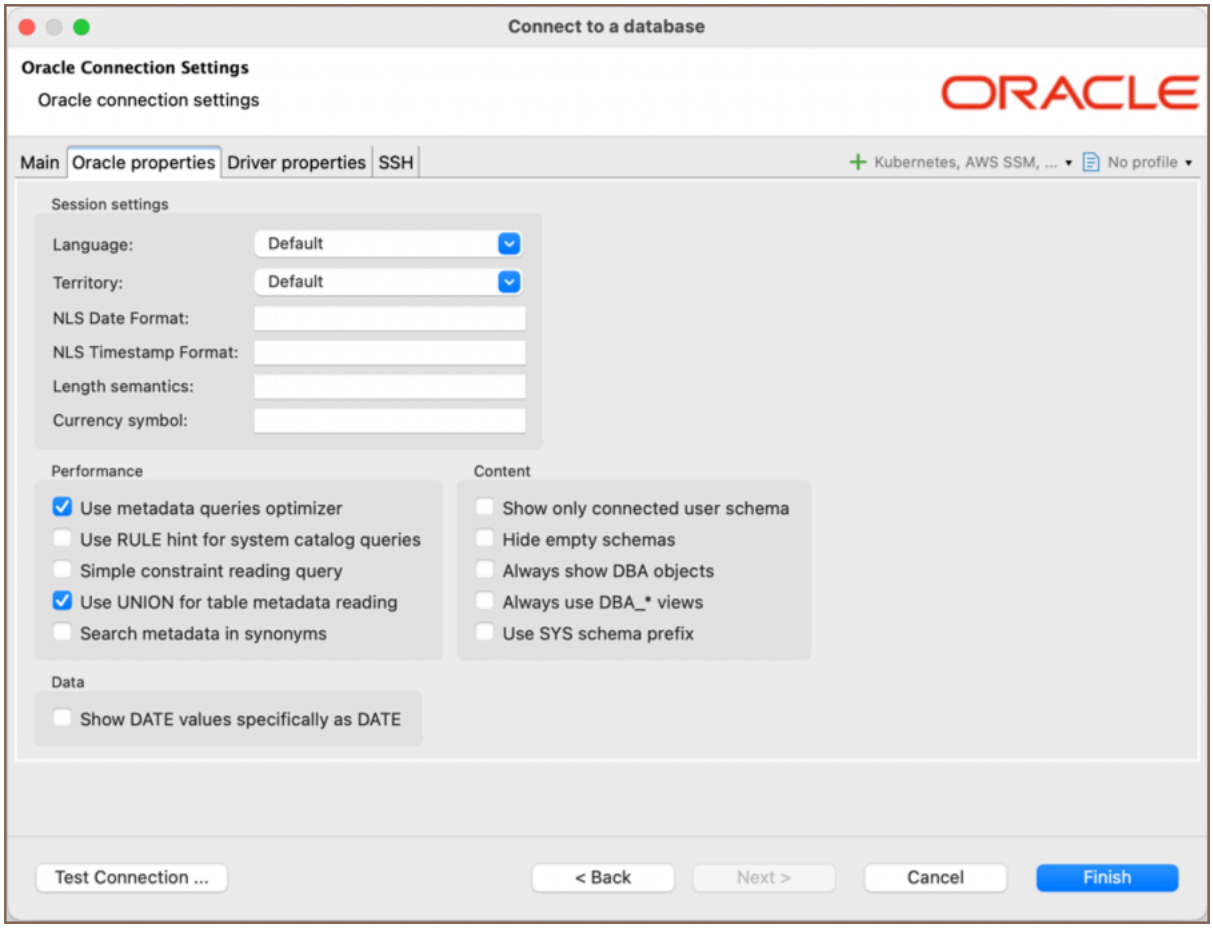
In this subsection, we'll outline the settings for establishing a direct connection to a Oracle database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Oracle database.

1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Host	If you're connecting via host, enter the host address of your Oracle database here.
Database	Enter the name of the Oracle database you want to connect to.
Port	Enter the port number for your Oracle database. The default Oracle port is `1521`.
Connection identifiers	Choose whether you want to connect using a `Service name` or a `SID` identifier.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/Password - Password from shell command - OS Authentication ★ - DBeaver Profile Authentication ★ - Oracle Kerberos Authentication ★ - Oracle Microsoft Entra ID ★ - Oracle Wallet ★ You can also read about security in DBeaver PRO.
Role	Choose whether you want to connect using a `Normal`, `SYSDABA` or a `SYSOPER` role. For more details, you can refer to the official Administering User Accounts and Security article.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The **Oracle Properties** page of the connection settings offers additional options that allow you to customize your further connection to the Oracle database.



Field	Description
Language	Specify the Session Language.
Territory	Specify the Session Territory.
NLS Date Format	Specify NLS (National Language Support) Date Format. The default value of the `NLS_DATE_FORMAT` parameter is determined by the `TERRITORY` parameter.
NLS Timestamp Format	Specify NLS Timestamp format. The default value of the `NLS_TIMESTAMP_FORMAT` parameter is determined by the `TERRITORY` parameter.
Length semantics	specify the length semantics for `VARCHAR2` and `CHAR` table columns, user-defined object attributes, and PL/SQL variables in database objects created in the session. The length semantics can be specified as either `BYTE` or `CHAR`.
Currency symbol	Specify the Currency symbol.You can find out which currency symbol your current session uses by querying the `V$NLS_PARAMETERS` view.
Show only connected user schema	Show only a scheme of a connected user in the Connections.
Hide empty schemas	Check existence of objects within schema and do not show empty schemas in tree. Enabled by default but it may cause performance problems on databases with very big number of objects.
Always show `DBA` objects	Always shows DBA-related metadata objects in tree even if user do not has `DBA` role.
Always use `DBA_*` views	Use `DBA_*` views instead of `ALL_*` views wherever it is possible
Use SYS schema prefix	Use SYS schema prefix in all metadata queries. Otherwise use view names without explicit schema.
Simple constraint reading query	Use simple metadata queries. May work slower but it is more stable for all Oracle versions.
Use UNION for table metadata reading	Use legacy table metadata query. With `UNION` instead `JOIN`. It helps in some cases speed up reading of table data.
Search metadata in synonyms	Search for metadata in synonyms so they're available in autocomplete, semantic validation, and Outline. May significantly slow down metadata search and autocompletion.
Search metadata in sequences	Search for metadata in sequences so they're available in autocomplete, semantic validation, and Outline. May significantly slow down metadata search and autocompletion.
Use RULE hint for system catalog queries	Adds `RULE` hint for some system catalog queries (like columns and constraints reading).It significantly increases performance on some Oracle databases (and decreases on others).
Show DATE values specifically as DATE	Show `DATE` data type values specifically as `DATE`, not as a `TIMESTAMP`.This setting will not work with disabled date/time formatting.
Use metadata queries optimizer	Use metadata queries optimizer. May significantly improve metadata reading performance on some systems.

Connection details

The **Connection Details** section in DBeaver allows to customize your experience while working with Oracle database. This includes options for adjusting the **Connections View**, setting up **Security** measures, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Oracle driver properties

The settings for Oracle **Driver properties** enable you to adjust the performance of the Oracle driver. These adjustments can influence the efficiency, compatibility, and features of your Oracle database.

For a complete walkthrough on setting up Oracle JDBC driver properties, you can refer to the official Oracle JDBC documentation. These guide detail each driver's properties and how they can be used to optimize Oracle database connections.

You can customize the Oracle driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Oracle database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Oracle database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Oracle with DBeaver

DBeaver provides a host of features designed for Oracle databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Oracle database objects

DBeaver lets you view and manipulate a wide range of Oracle database objects. DBeaver has extensive support for various Oracle metadata types, allowing you to interact with a wide variety of database objects, such as:

- Schemas
  - Tables
    - Columns
    - Constraints
    - Foreign Keys
    - References
    - Triggers
    - Indexes
    - Partitions
    - Dependencies
  - Views
  - Materialized Views
  - Indexes
  - Sequences
  - Queues
  - Types
  - Packages
  - Procedures
  - Functions
  - Synonyms
  - Schema Triggers
  - Table Triggers
  - Database Links
  - Java
  - Jobs
  - Scheduler

        - Jobs
        - Programs
    - Recycle bin
- Global metadata
    - Types
    - Public Synonyms
    - Public Database Links
    - User Recycle bin
- Storage
    - Tablespaces
        - Files
        - Objects
- Security
    - Users
    - Roles
    - Profiles
- Administer
    - Session Manager
    - Locks Manager

Oracle features in DBeaver

DBeaver isn't limited to typical SQL tasks. It also includes numerous unique features specifically for Oracle. Beyond regular SQL operations, DBeaver provides a range of Oracle-specific capabilities, such as:

Category	Feature
Data Types	Oracle Nested Tables
PL/SQL Support	PL/SQL Procedures
	PL/SQL Functions
Security	Oracle Permissions
	Oracle Profiles
	Oracle Roles
	Changing Password ★
User Management	Create/Clone Users ★
Data Organization	Oracle Partitions
Database Management	Oracle Dependencies
Performance Tuning	Oracle Performance Reports
	Oracle Execution Plans
Script execution	Execute scripts and commands with **SQL Plus** syntax

Additional features compatible with Oracle, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Backup and Restore	How to Backup/Restore data
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

PostgreSQL

Table of contents

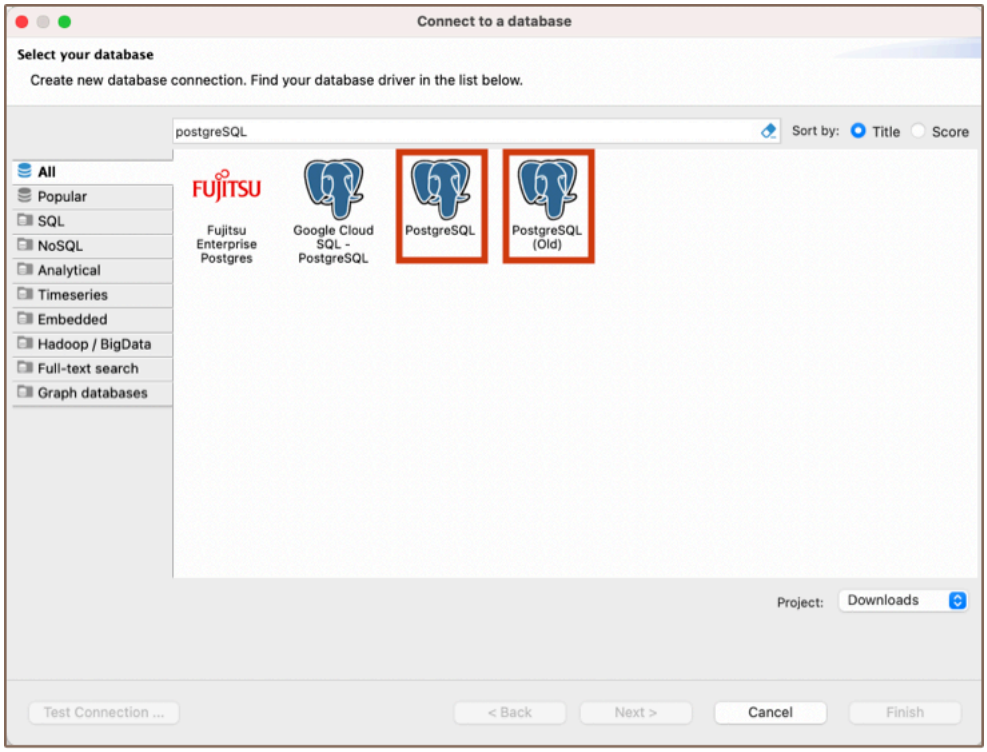
- Setting Up
- PostgreSQL connection settings
- Connection details
- PostgreSQL driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering PostgreSQL with DBeaver
- PostgreSQL database objects
- PostgreSQL features in DBeaver

This guide provides instructions on how to set up and use PostgreSQL with DBeaver.

DBeaver offers extensive support for PostgreSQL, but it might not cover all possible functionalities. If DBeaver is missing a feature you need, please submit a request through our GitHub page. If you are a developer, you have the option to enhance DBeaver's capabilities by contributing your own code.

Before you start, you must create a connection in DBeaver and select PostgreSQL. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the PostgreSQL server using a specific driver, it supports all versions of PostgreSQL, but the correct driver must be selected: use **PostgreSQL (old)** for versions below 8.x and **PostgreSQL** for newer versions. DBeaver also supports PostgreSQL extensions such as **EDB**, **Google Cloud SQL for PostgreSQL** and **Fujitsu Enterprise Postgres**. You must select the appropriate driver in the **Connect to a database** window for these extensions.



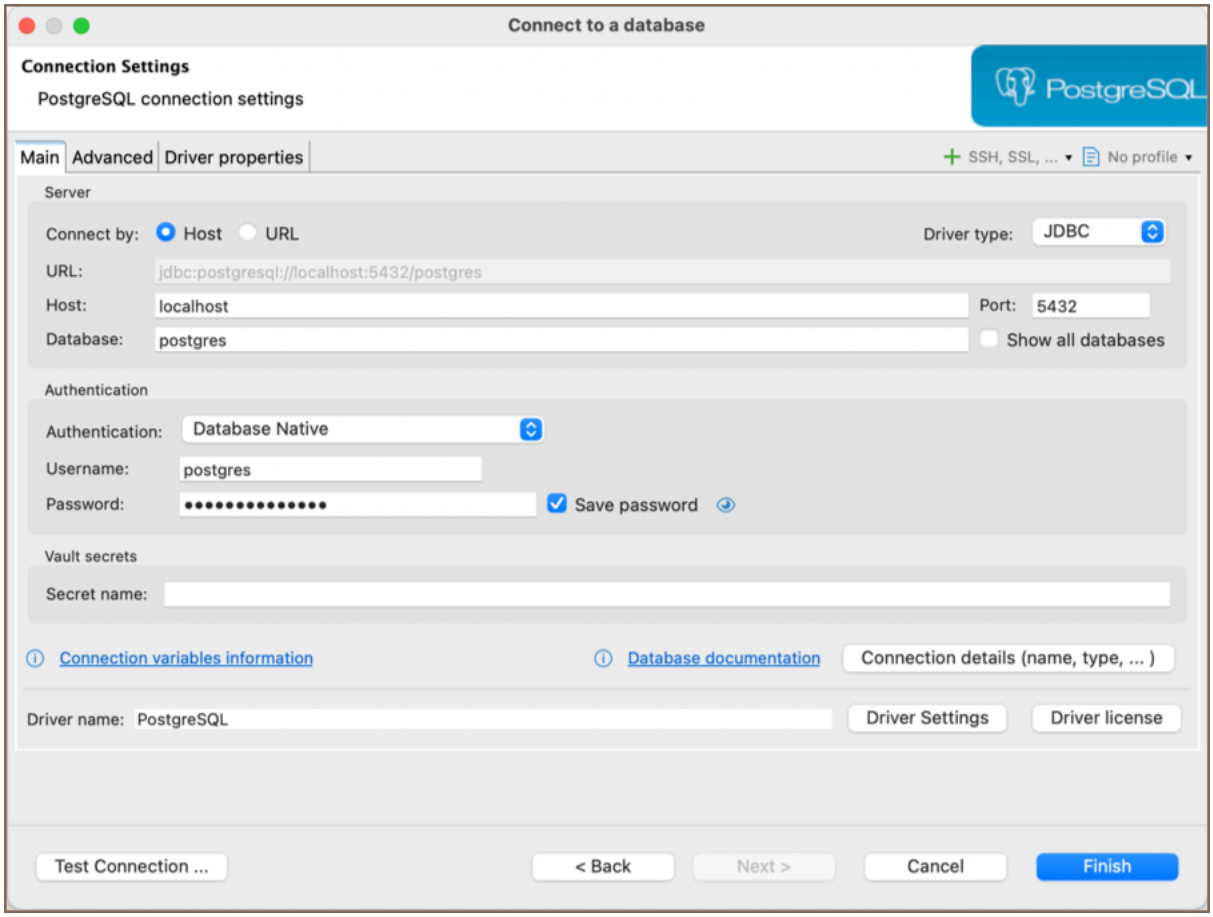
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for PostgreSQL.

PostgreSQL connection settings

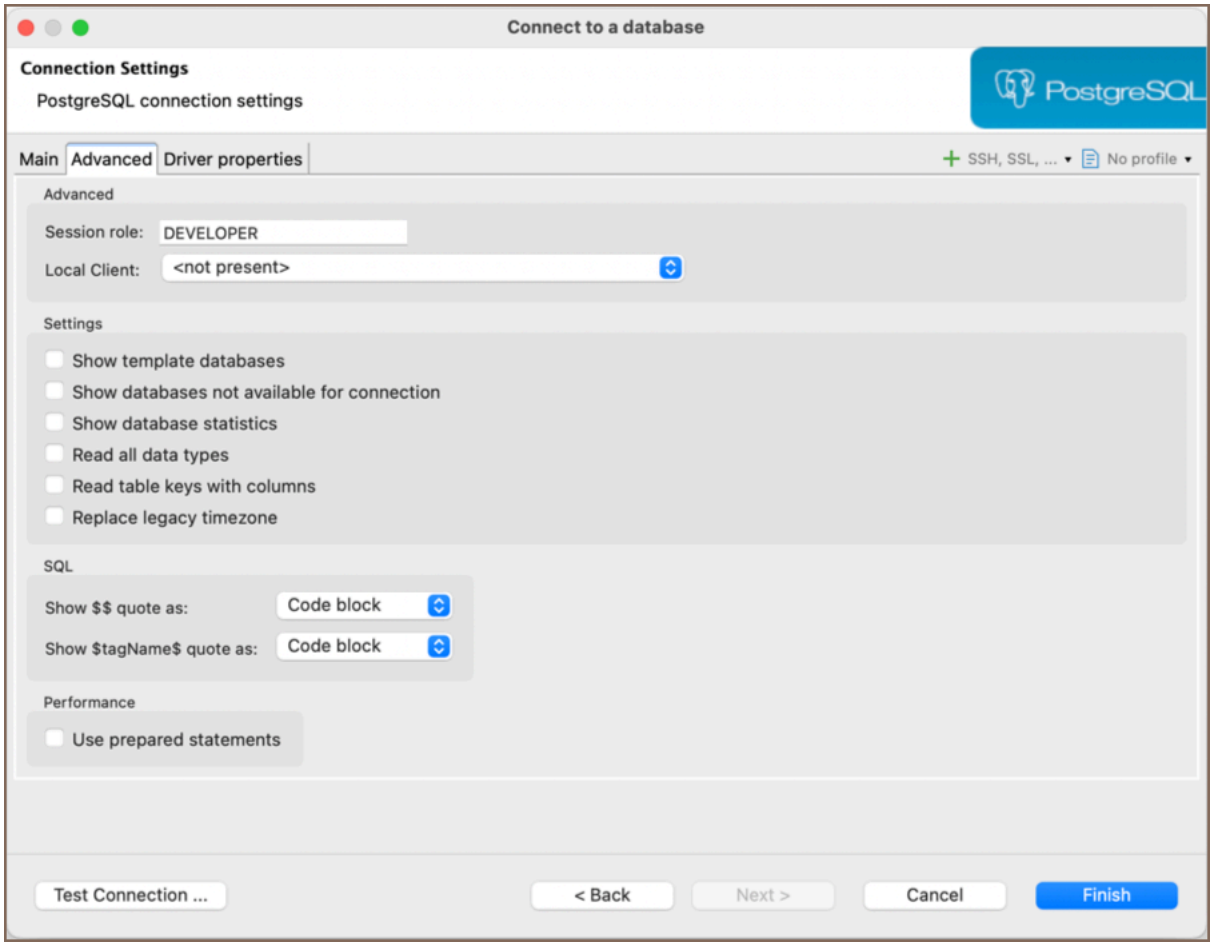
In this subsection, we will outline the settings for establishing a direct connection to a PostgreSQL database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your PostgreSQL database.

1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your PostgreSQL database.
URL	If you are connecting via URL, enter the URL of your PostgreSQL database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your PostgreSQL database here.
Database	Enter the name of the PostgreSQL database you want to connect to.
Show all databases	Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Port	Enter the port number for your PostgreSQL database. The default PostgreSQL port is 5432.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - AWS RDS IAM Authentication ★ - Azure AD Authentication for PostgreSQL ★ - DBeaver Profile Authentication ★ - PostgreSQL Kerberos Authentication ★ - PostgreSQL PgPass Authentication - PostgreSQL SSPI Authentication (Windows SSO) ★ You can also read about security in DBeaver PRO.
Local Client	Specify the local client, see our article about local client configuration.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The second page of the connection settings offers additional options that allow you to customize your further connection to the PostgreSQL database.



Field	Description
Show template databases	Displays template databases. This option is available when **Show all databases** is checked.
Show databases unavailable for connection	Show databases not available for connection in database list. This option is available when **Show all databases** is checked.
Show databases statistics	Displays statistics for databases.
Read all data types	Reads all data types from the server (including arrays).
Read table keys with columns	Enable reading table constraints during the column reading stage to ensure accurate recognition of keys. Note that this setting may decrease metadata loading performance.
Replace legacy timezone	Replaces legacy timezone settings. Enable only if you need to connect with legacy timezones like **Europe/Kyiv** or **Asia/Calcutta**. Only these two timezones are affected.
Show `$$` quote as	Displays `$$` quotes as code blocks or strings.
Show `$tagName$` quote as	Displays `$tagName$` quotes as code blocks or strings.
Use prepared statements	Uses prepared statements for SQL execution.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your PostgreSQL connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

PostgreSQL driver properties

The settings for PostgreSQL **Driver properties** enable you to adjust the performance of the PostgreSQL JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your PostgreSQL database.

For a complete walkthrough on setting up PostgreSQL JDBC and ODBC drivers properties, you can refer to the official PostgreSQL JDBC documentation and PostgreSQL ODBC documentation. These guides detail each driver's properties and how they can be used to optimize PostgreSQL database connections.

You can customize the PostgreSQL driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your PostgreSQL database connections. For a comprehensive guide on these settings, please refer to

our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your PostgreSQL database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering PostgreSQL with DBeaver

DBeaver provides a host of features designed for PostgreSQL databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

PostgreSQL database objects

DBeaver lets you view and manipulate a wide range of PostgreSQL database objects. DBeaver has extensive support for various PostgreSQL metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Data types
    - Tables
      - Columns
      - Constraints
      - Indexes
      - Foreign Keys
      - Dependencies
      - References
      - Partitions
      - Triggers
      - Rules
      - Policies
    - Foreign Tables
    - Views
    - Materialized Views
    - Functions
    - Sequences
    - Data types
    - Aggregate functions
  - Event Triggers
  - Extensions
  - Storage
    - Tablespaces
  - Roles
- Administer
  - Jobs
  - Session Manager
  - Lock Manager

For a practical guide on creating new tables and working with various database objects in PostgreSQL, check out our tutorial.

PostgreSQL features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for PostgreSQL. Beyond regular SQL operations, DBeaver provides a range of PostgreSQL-specific capabilities, such as:

Category	Feature
Data Types	PostgreSQL Arrays
	PostgreSQL Structures
Extensions	PostgreSQL Extensions
Security	PostgreSQL Permissions
	PostgreSQL Policies
	PostgreSQL Roles

Category	Feature
Data Organization	PostgreSQL Partitions
Database Management	PostgreSQL Dependencies
	PostgreSQL Tools
External Data	PostgreSQL Foreign Tables

Additional features compatible with PostgreSQL, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Lock Management	Lock Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

Redis

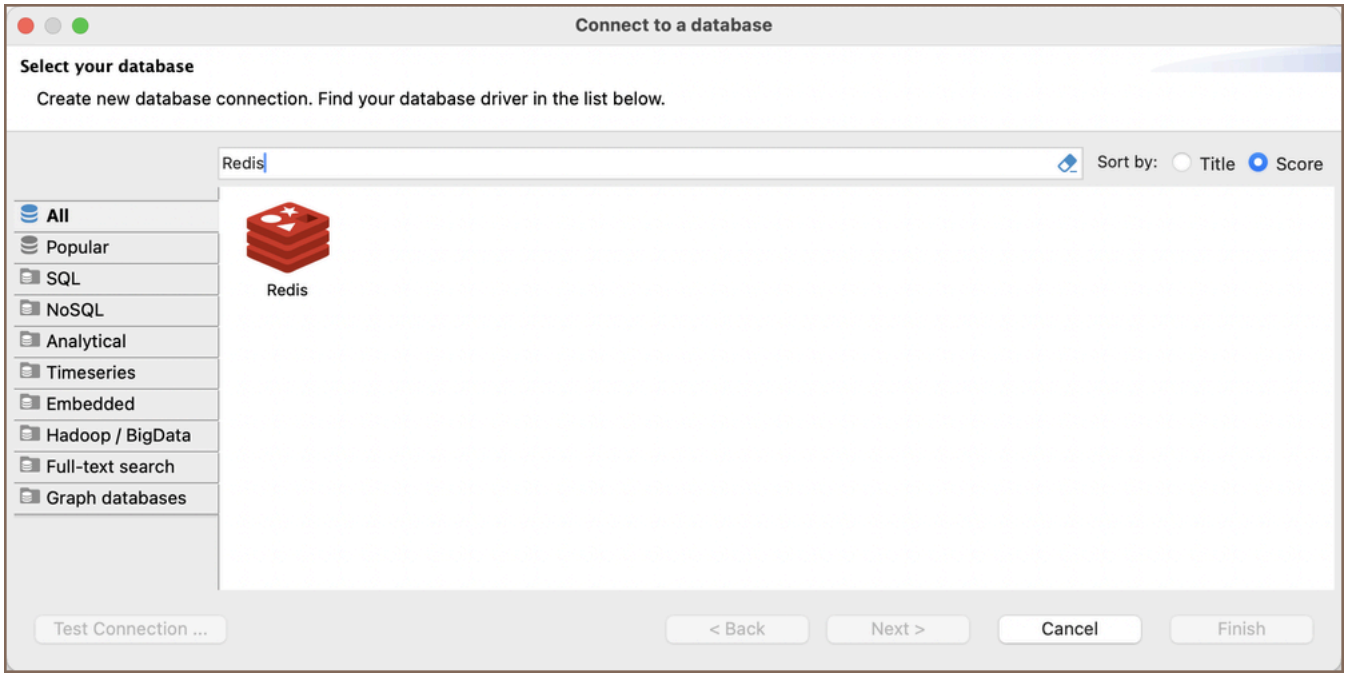
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Redis specialty
- Setting Up
- Redis connection settings
  - Connection details
  - Secure Connection Configurations
  - Secure Storage with Secret Providers
- Redis features in DBeaver
- Browsing keys
- Executing commands

This guide provides instructions on how to set up and use Redis with DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Redis` option to connect to the Redis. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Redis specialty

Redis is an open-source, in-memory data structure store, used as a database, cache, and message broker. It supports various data structures such as strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperloglogs, geospatial indexes, and streams. Redis has built-in replication, Lua scripting, LRU eviction,

transactions, and different levels of on-disk persistence. Additionally, Redis offers high availability via Redis Sentinel and automatic partitioning with Redis Cluster.

Redis is particularly well-suited for real-time applications like leaderboards, session stores, real-time analytics, and geospatial services due to its fast performance and support for complex data types and operations. For detailed information on Redis and its features, please refer to the official documentation.

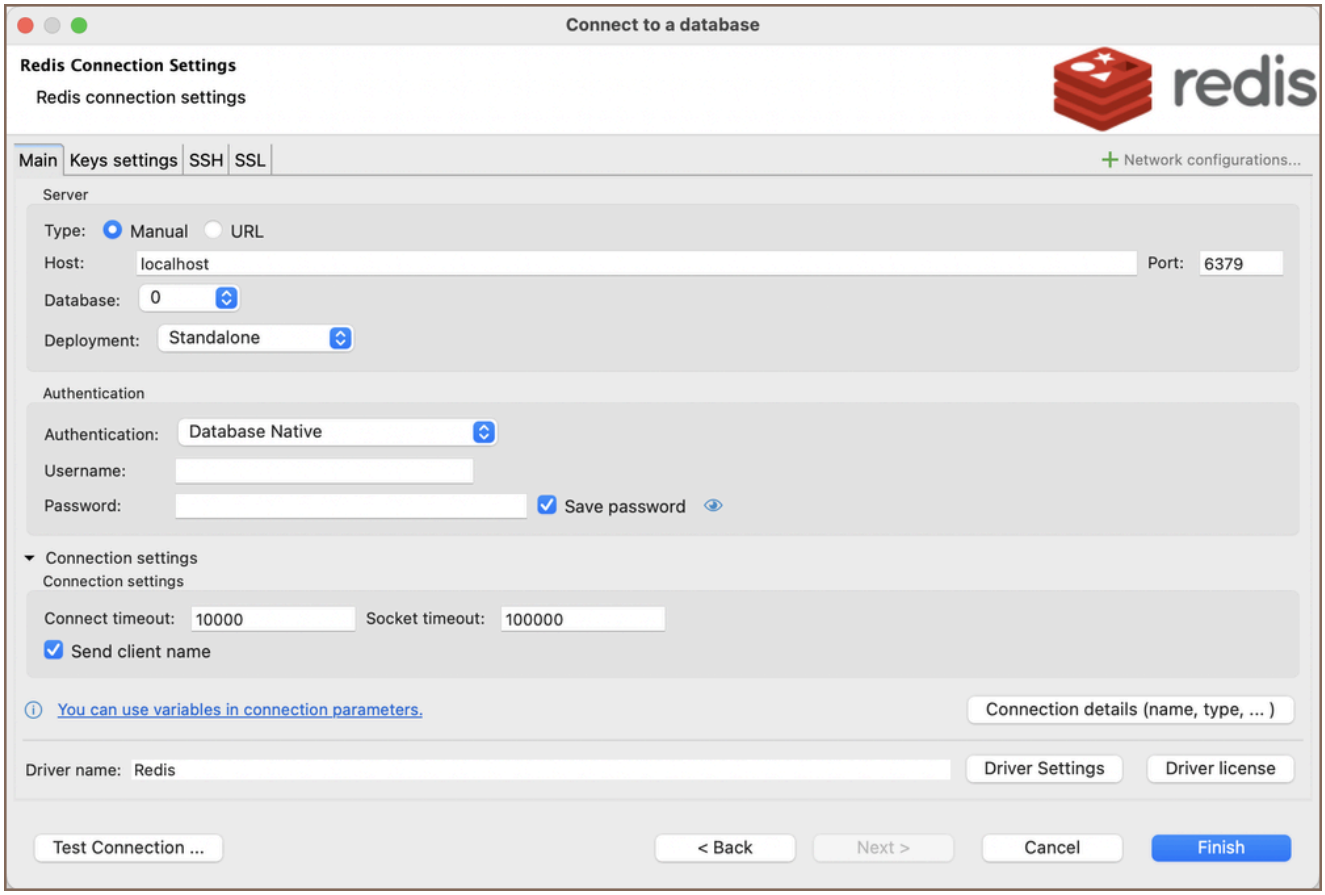
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, Kubernetes and SSL.

Redis connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Redis database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Redis database.

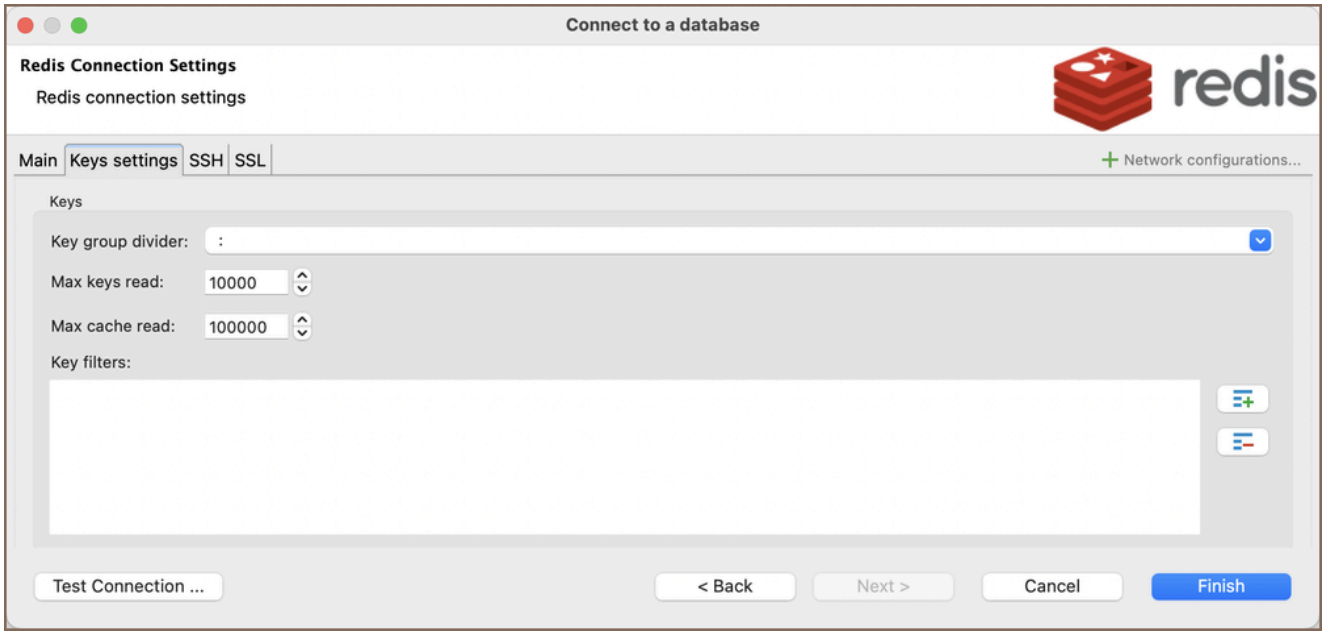
1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Manual/URL)	Choose whether you want to connect using a host or a URL.
URL	If you are connecting via URL, enter the URL of your Redis database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your Redis database here.
Database	Enter the database number you want to connect to.
Port	Enter the port number for your Redis database. The default Redis port is 6379.
Deployment	Select the type of Redis deployment for your connection. Options include:
	- **Standalone**: A single-node Redis deployment.
	- **Redis Cluster**: A multi-node Redis deployment for higher performance and fault tolerance.
	- **Redis Sentinel**: A system designed to help manage Redis instances, providing high availability and monitoring services.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication - Redis Sentinel (coming soon) You can also read about security in DBeaver PRO.
Connection settings	Customize connection settings:

Field	Description
	Connect timeout: Specify the duration, in milliseconds, the application waits to establish a connection before it times out. `10000` milliseconds is the default value.
	Socket timeout: Set the time, in milliseconds, the application waits for a server response to a query. The default setting here is `100000` milliseconds.
	Send client name: Check this option to transmit the client's name to the server when connecting. Some Redis servers do not support this feature, which may cause the connection attempt to fail.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The **Advanced** tab of the connection settings offers additional options that allow you to customize the keys settings.



Setting	Description	Default Value
Key group divider	Determines the delimiter used to group keys in the keyspace.	`:`
Max keys read	Limits the number of keys to read.	`10000`
Max cache read	Specifies the number of keys to read with pattern (cache).	`100000`
Key filters	Allows you to define patterns to include or exclude certain keys.	`*`

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Redis database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure Connection Configurations

DBeaver supports secure connections to your Redis database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Redis features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous additional features compatible with Redis, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Task Management	Task Management

Browsing keys

Redis keys are often viewed and edited as a simple list. However, when dealing with a large number of keys, which can run into millions or billions, a list view is not always practical or possible.

DBeaver offers a hierarchical view of Redis keys. Although Redis itself does not inherently support hierarchical data structures, it is common practice to organize keys into groups using specific delimiters like commas, dashes, or colons. DBeaver leverages these naming patterns to display keys in a hierarchical format. You can configure the group separator in the connection properties.

The key browser feature is useful, but for databases with a large key count, locating a specific key through the Connections might be challenging. In such cases, using the SQL editor with Redis commands is the most effective method to manage keys.

Executing commands

Redis operates differently from traditional SQL databases. It uses built-in commands and LUA scripting for operations.

To execute Redis commands, use the syntax similar to the Redis command line shell:

```
SET mykey "Hello"
```

This command will set the value of `mykey` to `Hello`. To run this command in DBeaver, you would type it into the SQL editor and execute it using either `CTRL+Enter` or `ALT+X`.

> To learn more about Redis commands, refer to the official Redis commands documentation.

For LUA script execution, enclose the script in curly braces {} and execute it as a single statement. If your LUA script includes empty lines or special characters, highlight the script text before executing it. For instance:

```
{
    -- Retrieve the value of a Redis key
    local key = 'key1038' -- Specify the key you want to retrieve
    local value = redis.call('GET', key)
    return value
}
```

> For LUA scripting in Redis, see LUA scripts in Redis.

Salesforce

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

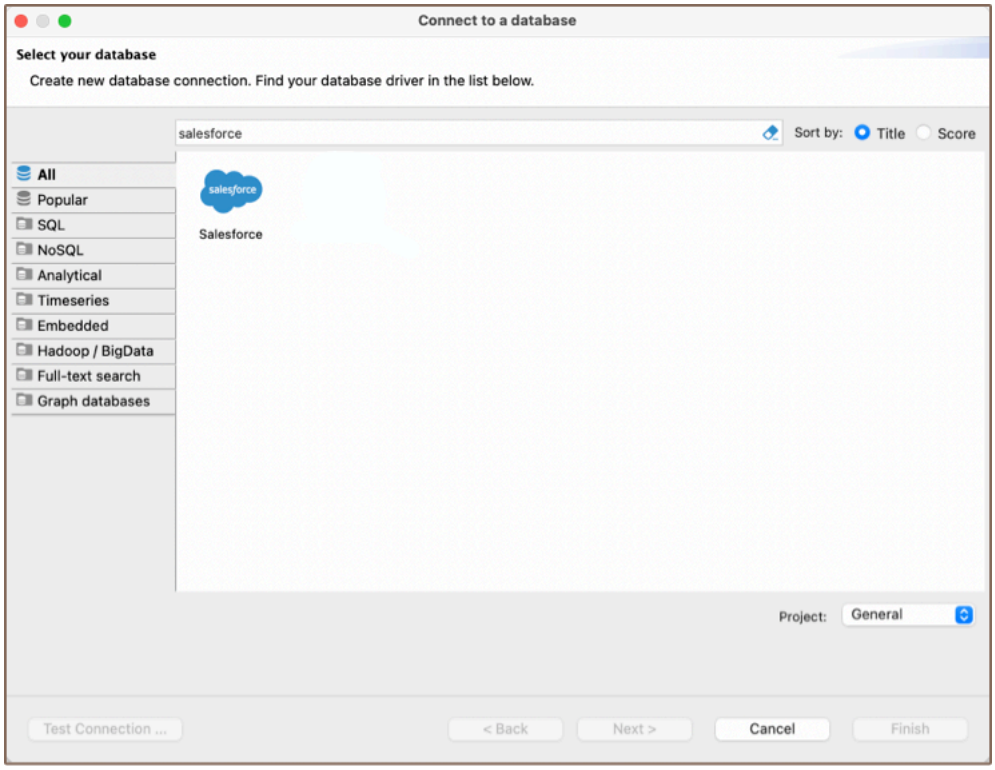
Table of contents

- Setting Up
- Salesforce connection settings
  - Connection details
- Salesforce driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Salesforce with DBeaver
- Salesforce database objects
- Salesforce Object Query Language
- SOQL vs SQL
- Examples
  - Simple Select
  - Select with Filtering
  - Select with Grouping

This guide provides instructions on how to set up and use Salesforce with DBeaver.

Before you start, you must create a connection in DBeaver and select Salesforce. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the Salesforce server using a specific driver. To use it, choose the right Salesforce driver in the Connect to a database window.



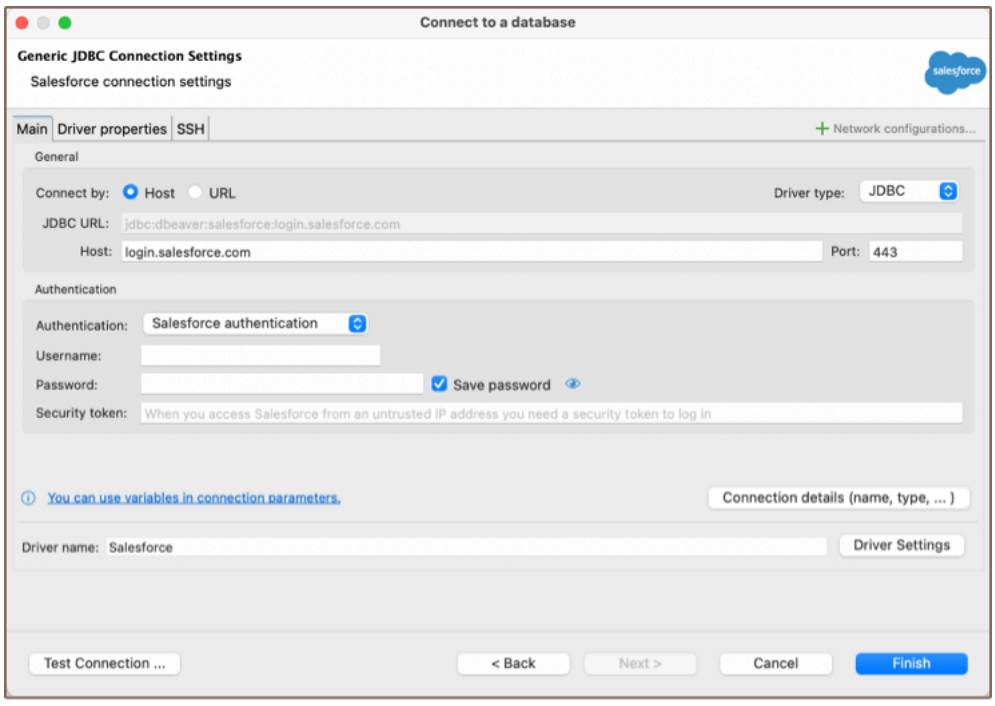
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH and proxies.

Salesforce connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Salesforce database using DBeaver.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Salesforce database.
URL	If you are connecting via URL, enter the URL of your Salesforce database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your Salesforce database here.
Port	Enter the port number for your Salesforce database. The default Salesforce port is 443.

Field	Description
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - DBeaver Profile Authentication - Salesforce authentication You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Salesforce database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Salesforce driver properties

Salesforce is not a traditional Relational Database Management System (RDBMS), and as such, it does not come with a standard JDBC driver. DBeaver uses REST API for connecting to the Salesforce server.

There are no particular properties for configuration at this time. The driver is designed with the current functionality of the Salesforce platform in mind, focusing solely on providing the ability to read data.

Secure Connection Configurations

DBeaver supports secure connections to your Salesforce database. Guidance on configuring such connections, specifically **SSH Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Salesforce with DBeaver

At the current version of DBeaver, the Salesforce database driver allows read-only access to its data, enabling you to execute `SELECT` queries to view data. This includes the ability to use filters, groupings, data export, Diagrams and other functionalities of DBeaver for viewing and analyzing the data.

Salesforce database objects

DBeaver lets you view Salesforce database objects and has support for a limited number of Salesforce metadata types, allowing you to interact with a select variety of database objects, such as:

- Tables
  - Columns
  - Keys
  - Foreign Keys
  - References

- Data Types

Salesforce Object Query Language

SOQL is a query language used to search Salesforce data. Within DBeaver, you can utilize SOQL to execute read-only `SELECT` queries on Salesforce data.

SOQL vs SQL

SOQL is specifically tailored for querying Salesforce data and has some distinct differences from traditional SQL:

- Read-Only Queries: SOQL is used primarily for querying data, not for data manipulation. In Salesforce, data insertion and manipulation are typically handled through the platform's user interface, APIs, or specialized data loading tools, rather than through SOQL.
- Limited `JOIN` Operations: Unlike SQL, SOQL does not support all types of `JOIN` operations.
- No Arbitrary Expressions: SOQL does not allow arbitrary expressions or calculations in the `SELECT` clause, unlike SQL.

Examples

Here are some brief examples, you can find more details in the official SOQL tutorial:

Simple Select

Retrieve the `Name` and `Id` of all accounts:

```
SELECT Id, Name
FROM Account
```

Select with Filtering

Retrieve accounts with a specific status:

```
SELECT Id, Name
FROM Account
WHERE Status = 'Active'
```

Select with Grouping

Group accounts by `status` and count them:

```
SELECT Status, COUNT(Id)
FROM Account
GROUP BY Status
```

Teradata

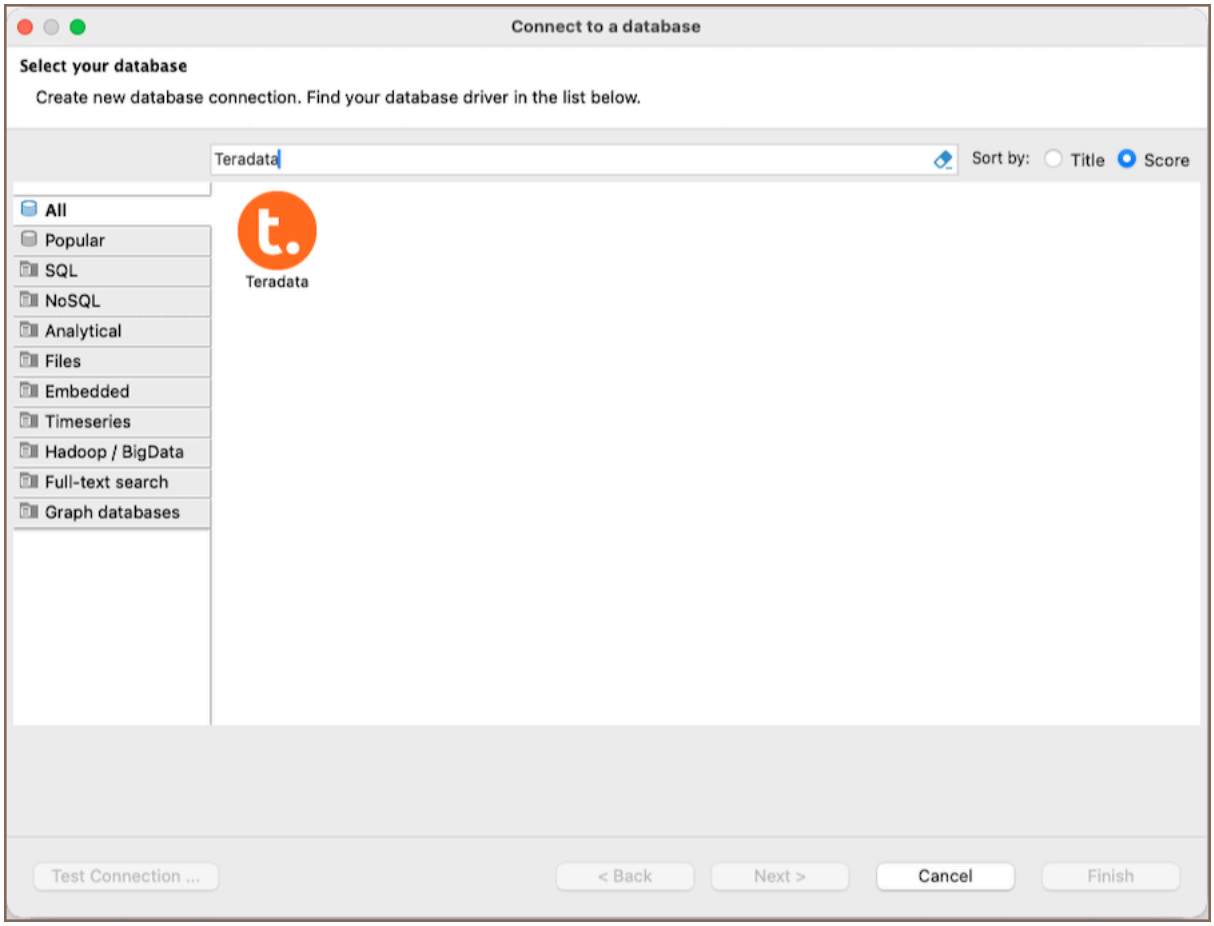
Table of contents

- Setting Up
- Teradata connection settings
  - Connection details
- Teradata driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Teradata with DBeaver
- Teradata database objects
- Teradata features in DBeaver

This guide provides instructions on how to set up and use Teradata with DBeaver. DBeaver enhances your experience with specialized support for Teradata features, including unique data types like Vargraphic and Long Vargraphic, as well as advanced security options through Roles and Profiles.

Before you start, you must create a connection in DBeaver and select Teradata. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the Teradata server using a specific driver, supporting all versions of Teradata.



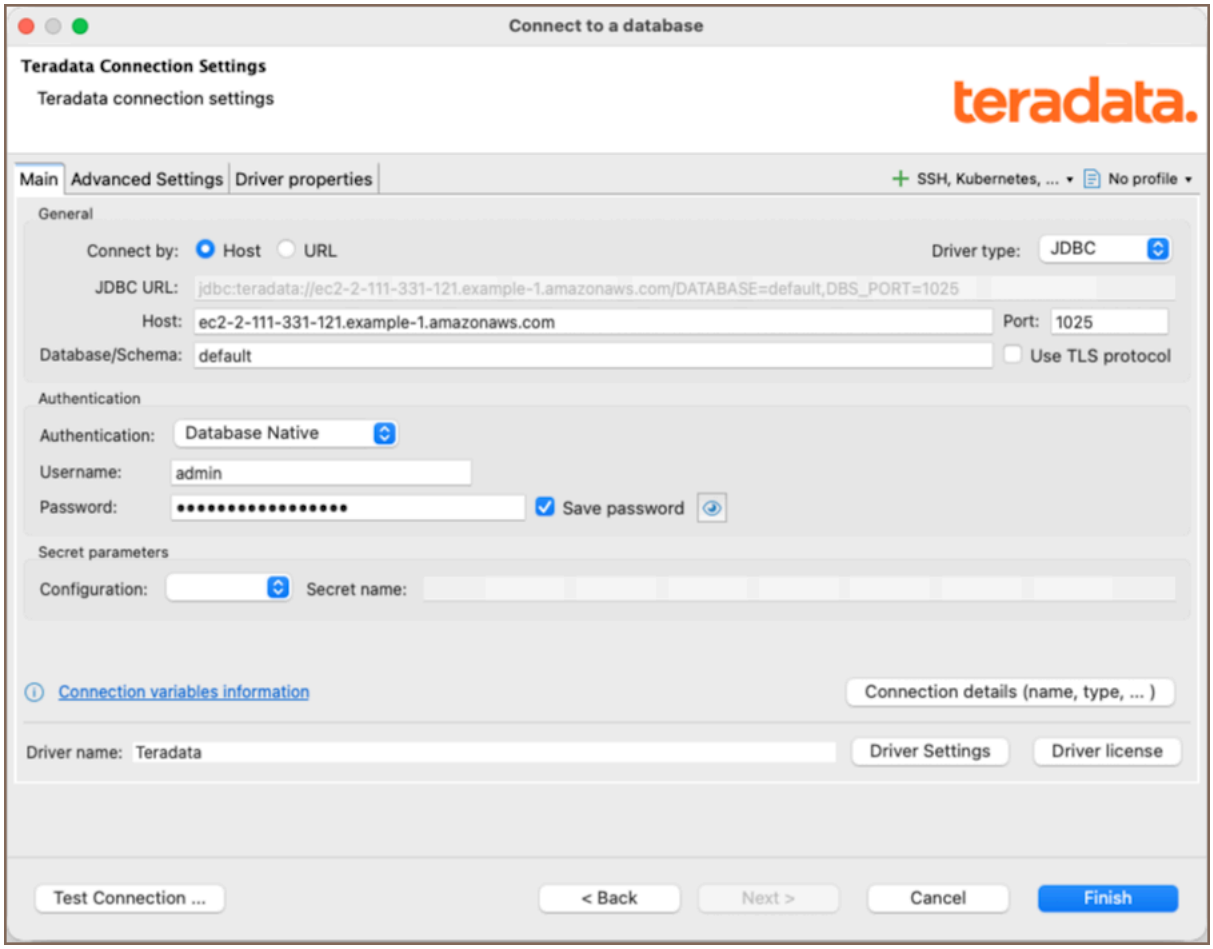
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, Kubernetes, and the setup of ODBC/JDBC drivers for Teradata.

Teradata connection settings

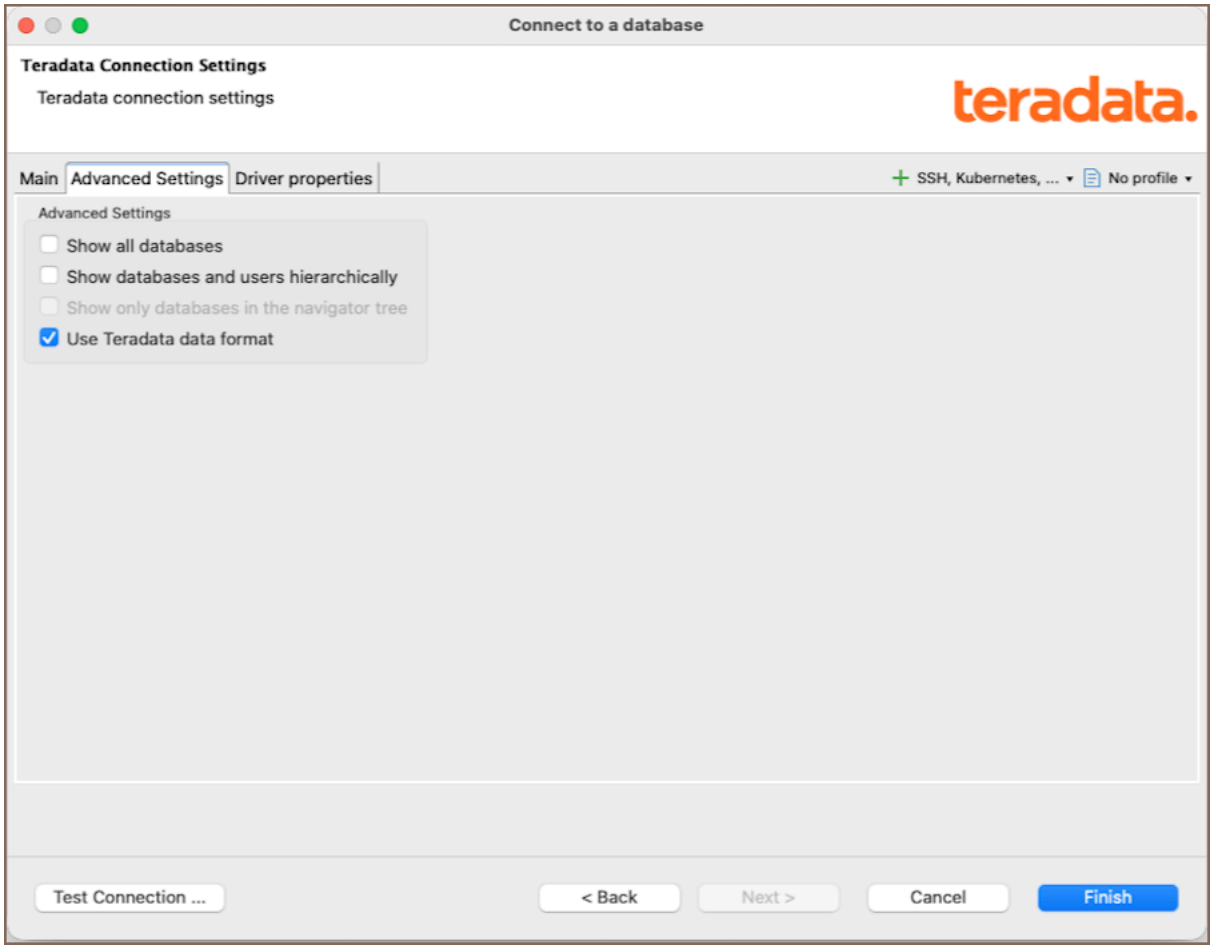
In this subsection, we will outline the settings for establishing a direct connection to a Teradata database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Teradata database.

1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Teradata database.
URL	If you are connecting via URL, enter the URL of your Teradata database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your Teradata database here.
Database/Schema	Enter the name of the Teradata database/schema you want to connect to.
Port	Enter the port number for your Teradata database. The default Teradata port is `1025`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - Kerberos ★ - LDAP Mechanism ★ - Single Sign-On ★ You can also read about security in DBeaver PRO.
Use TLS protocol	If you enable this checkbox, the connection will use TLS (Transport Layer Security) to encrypt the data that is transmitted between DBeaver and the Teradata database.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The second page of the connection settings offers additional options that allow you to customize your further connection to the Teradata database.



Field	Description
Show all databases	Displays all server databases in the Connections.
Show databases and users hierarchically	Enable this checkbox to view databases and associated user permissions in a hierarchical structure.
Show only databases in the Connections tree	Displays databases in the Connections tree and exclude other types of objects like schemas, tables, and users.
Use Teradata format	Use the `FORMAT` clause to control how values are displayed instead of default formatting. If no `FORMAT` is defined for a column, default formatting is used.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with MySQL database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Teradata driver properties

The settings for Teradata **Driver properties** enable you to adjust the performance of the Teradata JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Teradata database.

For a complete walkthrough on setting up Teradata JDBC and ODBC drivers properties, you can refer to the official Teradata JDBC documentation and Teradata ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Teradata database connections.

You can customize the Teradata driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Teradata database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Teradata database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Teradata with DBeaver

DBeaver provides a host of features designed for Teradata databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Teradata database objects

DBeaver lets you view and manipulate a wide range of Teradata database objects. DBeaver has extensive support for various Teradata metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Tables
    - Columns
    - Keys
    - Foreign keys
    - Indexes
    - References
    - Triggers
  - Views
  - Indexes
  - Procedures
  - Macros
  - Data Types
- Administration
  - Zones
  - Maps
  - Users
  - Roles
  - Profiles

Teradata features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for Teradata. Beyond regular SQL operations, DBeaver provides a range of Teradata-specific capabilities, such as:

Category	Feature
Data Types	Teradata Vargraphic and Long Vargraphic
Security	Teradata Roles and Profiles
Database Management	Teradata Procedures
	Teradata Macros

Additional features compatible with Teradata, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Schema Management	Schema Compare
Data Visualization	ERD Guide

Trino

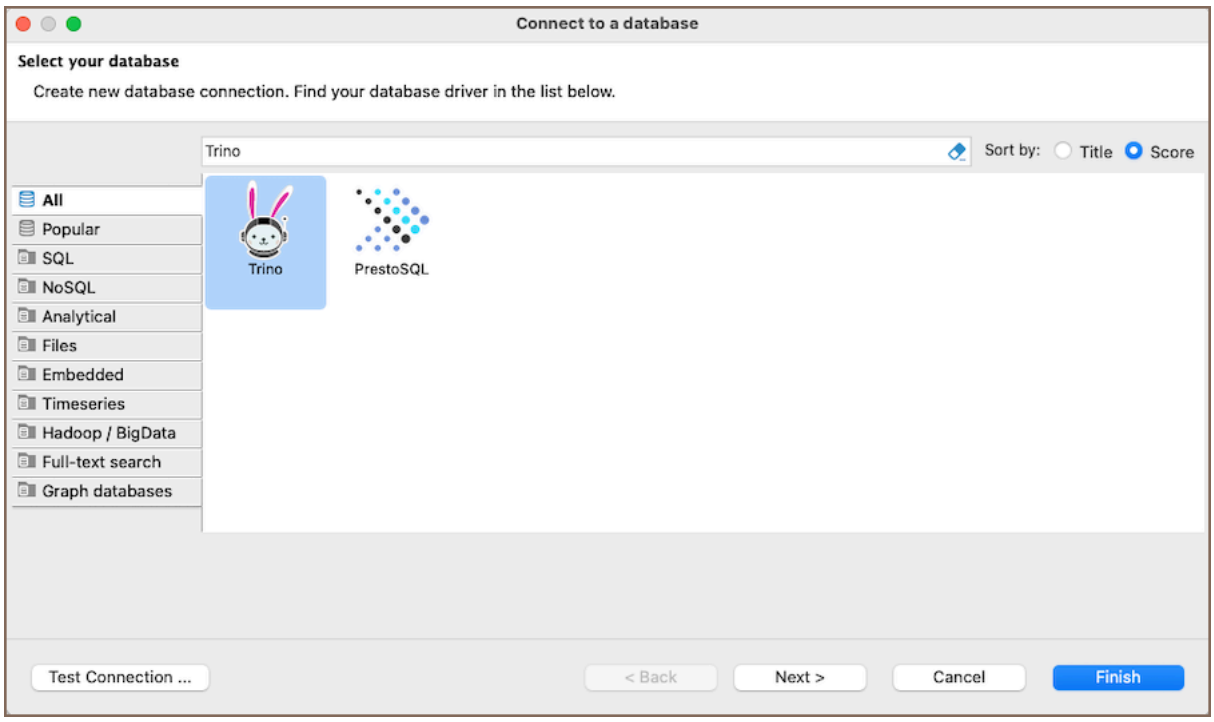
Table of contents

- Trino specialty
- Setting up
- Trino connection settings
- Connection details
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Trino with DBeaver
- Trino database objects
- Trino Features in DBeaver

This guide provides instructions on how to set up and use Trino with DBeaver.

Before you start, you must create a connection in DBeaver and select Trino. If you haven't done this yet, see our Database Connection article.

DBeaver connects to Trino using a dedicated JDBC driver. It supports all modern Trino versions. If you're working with older Presto (PrestoSQL) setups, you can also use the **Presto** driver. For newer deployments, use the **Trino** driver for better compatibility and feature support.



Trino specialty

Trino is a fast, distributed SQL engine designed for querying large datasets across different data sources. It lets you work with data where it lives - whether in databases, data lakes, or file systems - without copying or moving it. You can run SQL queries across multiple systems at once, joining data from sources like Hive, Iceberg, PostgreSQL, or S3.

Trino is built for scalability and performance. It uses a coordinator and worker architecture that supports high concurrency and horizontal scaling. Security and access control can be managed using external tools or plugins, with support for various authentication methods and permissions.

> For more, see the Trino documentation.

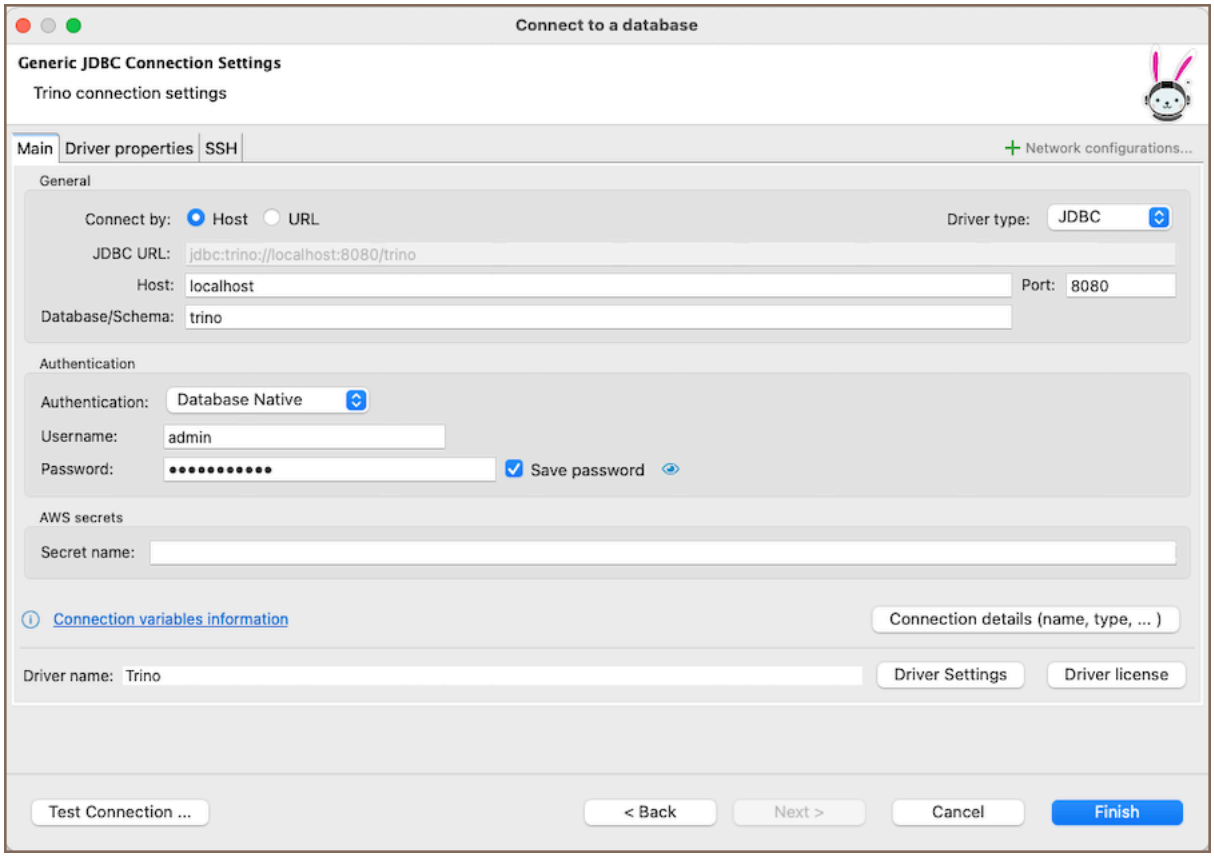
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, AWS SSM and Kubernetes.

Trino connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Trino database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Trino database.

The connection settings page requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
URL	If you are connecting via URL, enter the URL of your Trino database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your Trino database here.
Database/Schema	Enter the name of the Trino database you want to connect to.
Port	Enter the port number for your Trino database. The default Trino port is `8080`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - Trino Kerberos Authentication ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Trino database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure Connection Configurations

DBeaver supports secure connections to your Trino database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**

- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Trino with DBeaver

DBeaver provides a host of features designed for Trino databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Trino database objects

DBeaver lets you view and manipulate a wide range of Trino database objects. DBeaver has extensive support for various Trino metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases (schemas)
  - Tables
    - Columns
  - Views

Trino Features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for Trino. Beyond regular SQL operations, DBeaver provides a range of Trino-specific capabilities, such as:

Category	Feature
Data Types	Native support for complex and custom types: `map`, `row`, `json`, `hyperloglog`, `geometry`, `ipaddress`, `uuid`, `jsonpath`, etc.
Metadata Access	Explore `information_schema` and Trino's internal `system` schemas

Additional features compatible with Trino, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Data Compare	Data Compare ★
Data Generation	Mock Data Generation ★

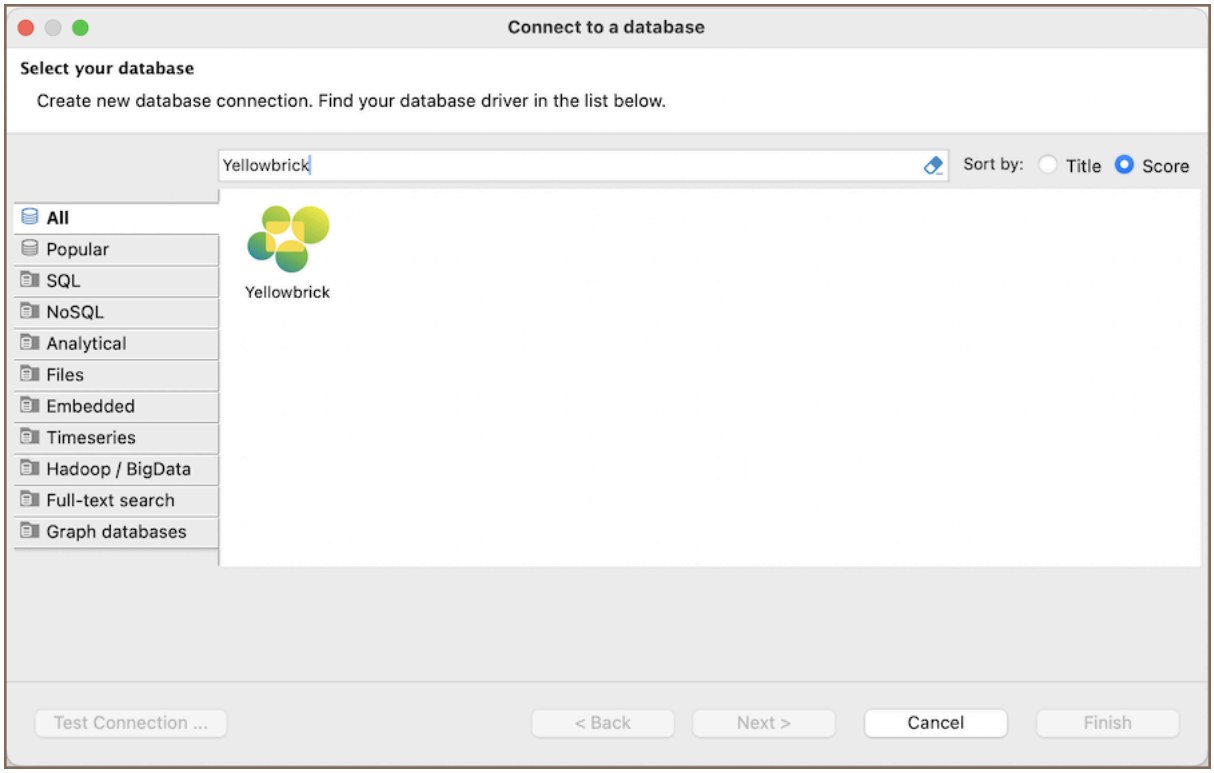
Yellowbrick

Table of contents

- Yellowbrick specialty
- Setting Up
- Yellowbrick connection settings
- Connection details
- Yellowbrick driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Yellowbrick with DBeaver
- Yellowbrick database objects
- Yellowbrick features in DBeaver

This guide provides instructions on how to set up and use Yellowbrick with DBeaver.

Before you start, you must start creating a connection in DBeaver and select Yellowbrick. If you haven't done this yet, see our Create Connection article.



Yellowbrick specialty

Yellowbrick is a high-performance, cloud-native data warehouse built for complex analytics at scale. It's optimized for fast SQL execution on large volumes of data, making it ideal for enterprise workloads that demand speed and concurrency.

Yellowbrick is based on PostgreSQL, so it supports standard SQL and offers compatibility with PostgreSQL clients, drivers, and tools. You can run complex queries using familiar syntax, with low-latency results even on billions of rows.

It integrates easily with BI tools, ETL pipelines, and existing data ecosystems. Yellowbrick supports hybrid deployments - on-premises, in the cloud, or both - and includes features for workload management, high availability, and enterprise-grade security.

> For more, see the Yellowbrick documentation.

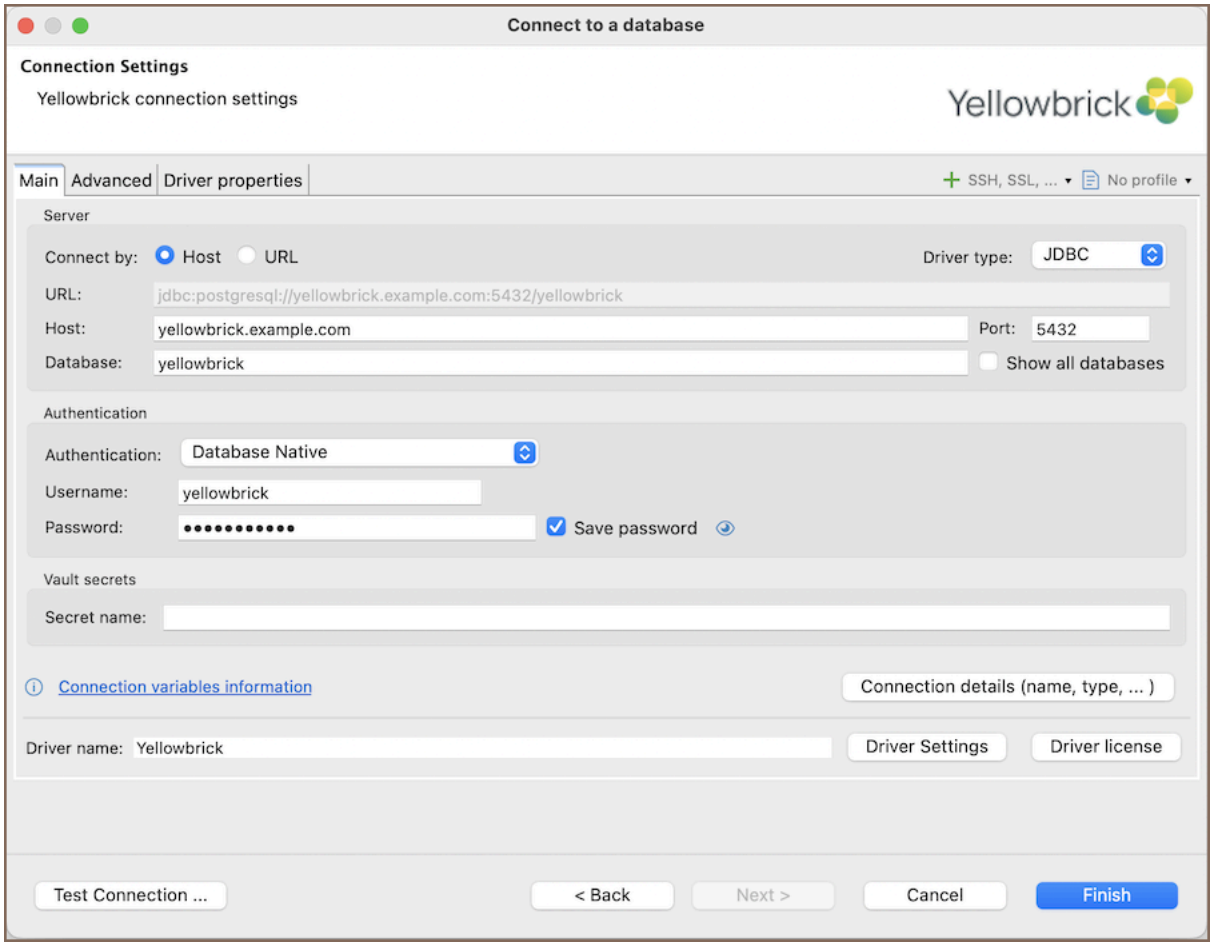
Setting Up

This section explains how to configure a direct connection in DBeaver and set up secure connections using SSH, proxies, SSL, and other available network configurations.

Yellowbrick connection settings

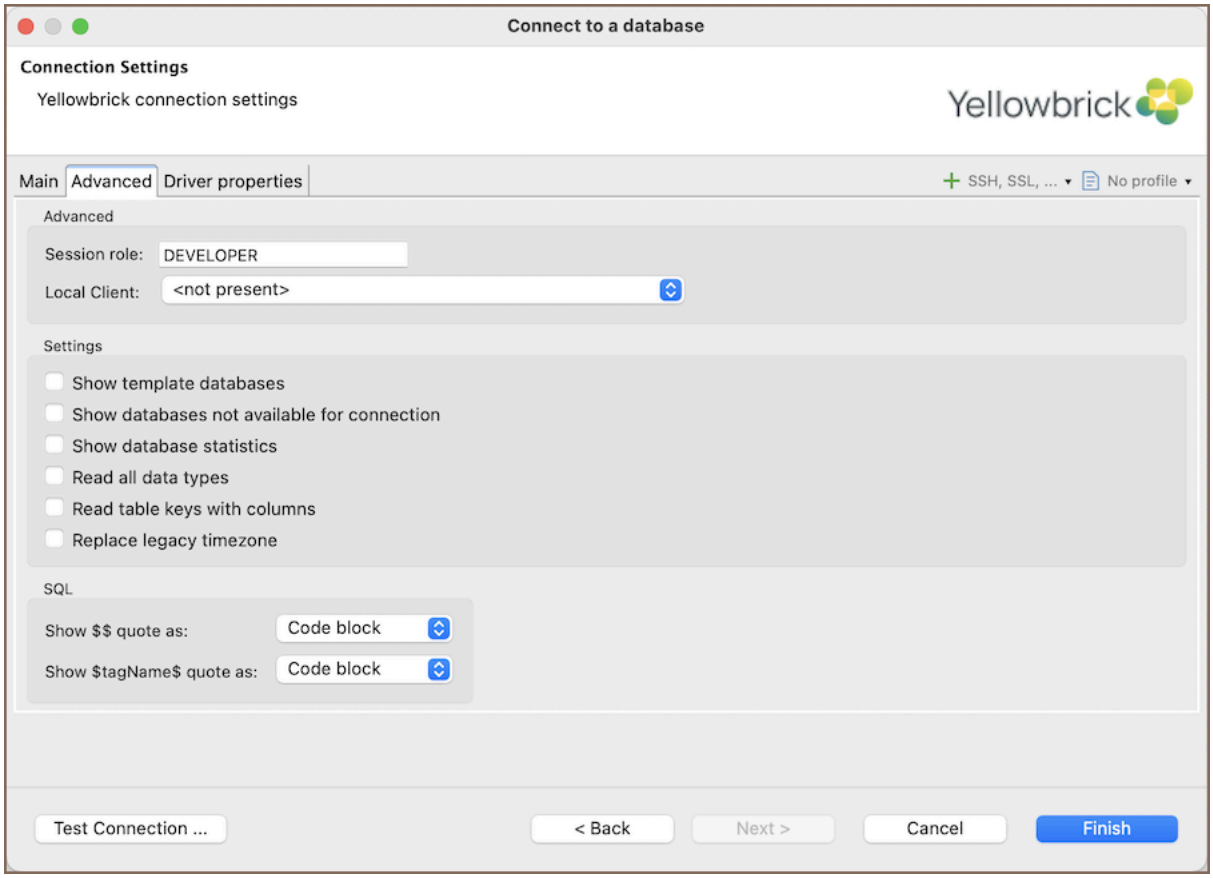
In this subsection, we will outline the settings for establishing a direct connection to a Yellowbrick database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Yellowbrick database.

1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
URL	If you are connecting via URL, enter the URL of your Yellowbrick database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your Yellowbrick database here.
Database	Enter the name of the Yellowbrick database you want to connect to.
Show all databases	(Enabled by default) Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Port	Enter the port number for your Yellowbrick database. The default Yellowbrick port is 5432.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - Kerberos Authentication ★ - PgPass Authentication - SSPI Authentication (Windows SSO) ★ - Yellowbrick OAuth2 ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The **Advanced** tab of the connection settings offers additional options that allow you to customize your further connection to the Yellowbrick database.



Field	Description
Session Role	This option allows you to define a specific role that your database session assumes upon connection. Setting a session role can be useful for managing permissions and access control within your Yellowbrick database.
Local Client	Specify the local client, see our article about local client configuration.
Show template databases	Displays template databases. This option is available when **Show all databases** is checked.
Show databases unavailable for connection	Show databases not available for connection in database list. This option is available when **Show all databases** is checked.
Show databases statistics	Displays statistics for databases.
Read all data types	Reads all data types from the server (including arrays).
Read table keys with columns	Enable reading table constraints during the column reading stage to ensure accurate recognition of keys. Note that this setting may decrease metadata loading performance.
Replace legacy timezone	Replaces legacy timezone settings. Enable only if you need to connect with legacy timezones like **Europe/Kyiv** or **Asia/Calcutta**. Only these two timezones are affected.
Show `$$` quote as	Displays `$$` quotes as code blocks or strings.
Show `$tagName$` quote as	Displays `$tagName$` quotes as code blocks or strings.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Yellowbrick connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Yellowbrick driver properties

The settings for Yellowbrick **Driver properties** enable you to adjust the performance of the Yellowbrick JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Yellowbrick database.

You can customize the Yellowbrick driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Yellowbrick database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Yellowbrick database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Yellowbrick with DBeaver

DBeaver provides a host of features designed for Yellowbrick databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Yellowbrick database objects

DBeaver lets you view and manipulate a wide range of Yellowbrick database objects. DBeaver has extensive support for various Yellowbrick metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Tables
      - Columns
      - Constraints
      - Foreign Keys
    - Foreign Tables
    - Views
    - Materialized Views
    - Sequences
    - Data types
    - Aggregate functions
  - Extensions
  - Storage
  - Tablespaces
  - Roles
- Administer
  - Jobs
  - Session Manager
  - Lock Manager

Yellowbrick features in DBeaver

DBeaver is not limited to typical SQL tasks. It also supports advanced features available in Yellowbrick, many of which extend PostgreSQL compatibility. These include:

Category	Feature
Data types	Support for arrays, ranges, network, JSON, and GIS-style types (`geography`, etc.)
Extensions	Built-in support for extensions like `pgcrypto`, `plpgsql`, `plpythonu`
Collations	Wide range of locale-based collations (`C`, `POSIX`, `utf8`, regional variants)
Storage	Tablespaces management (`pg_default`, `pg_global`)

Additional features compatible with Yellowbrick, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Lock Management	Lock Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide

Cloud

AWS

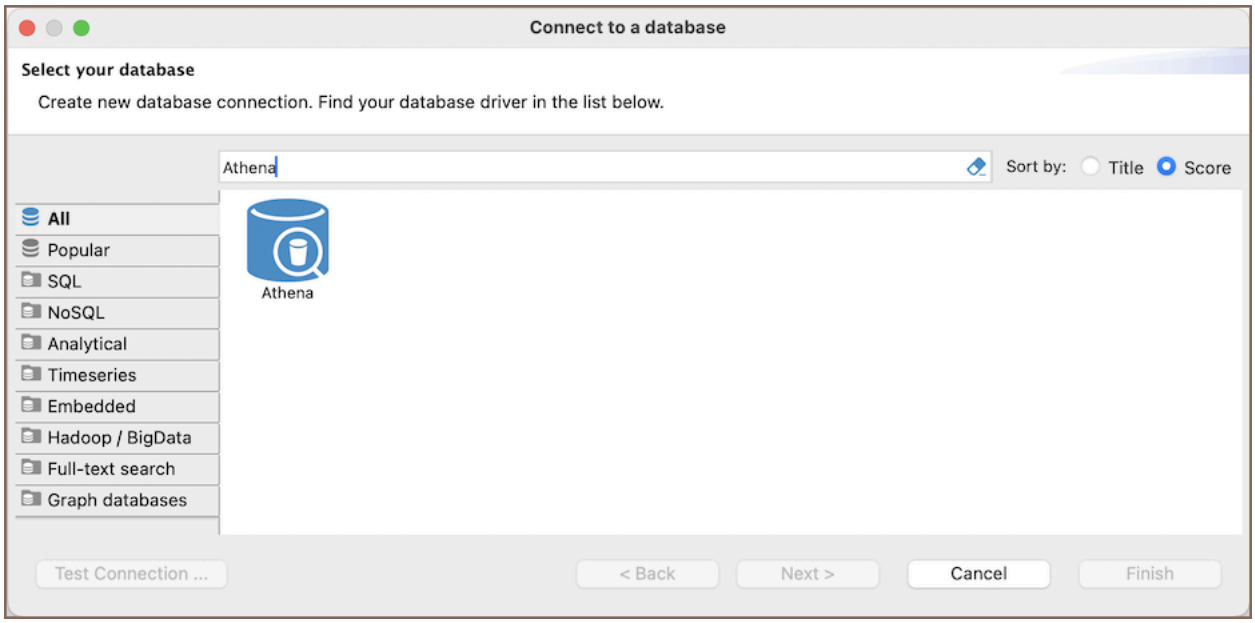
Athena

Table of contents

- Athena specialty
- Setting up
- Athena connection settings
- Connection details
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering Athena with DBeaver
- Athena database objects
- Athena additional features in DBeaver

This documentation outlines the process of configuring and using AWS Athena with DBeaver.

To manage databases using AWS Athena in DBeaver, establishing a connection is essential. This is done by selecting the `Athena` option in the database connection settings of DBeaver. For those new to this process, our Creating a Connection article provides detailed guidance.



Athena specialty

AWS Athena, as an analytical database service, has limitations compared to traditional relational databases. It does not support real-time data changes, such as adding or updating data. Also, Athena cannot modify data using `DELETE` or `UPDATE` commands. It lacks constraint enforcement like primary keys and foreign keys. For more detailed information, refer to the AWS Athena database documentation.

> **Important**: In DBeaver, AWS Athena is read-only.

For a detailed overview of the supported features and capabilities of AWS Athena within DBeaver, please see the section on Powering Athena with DBeaver.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

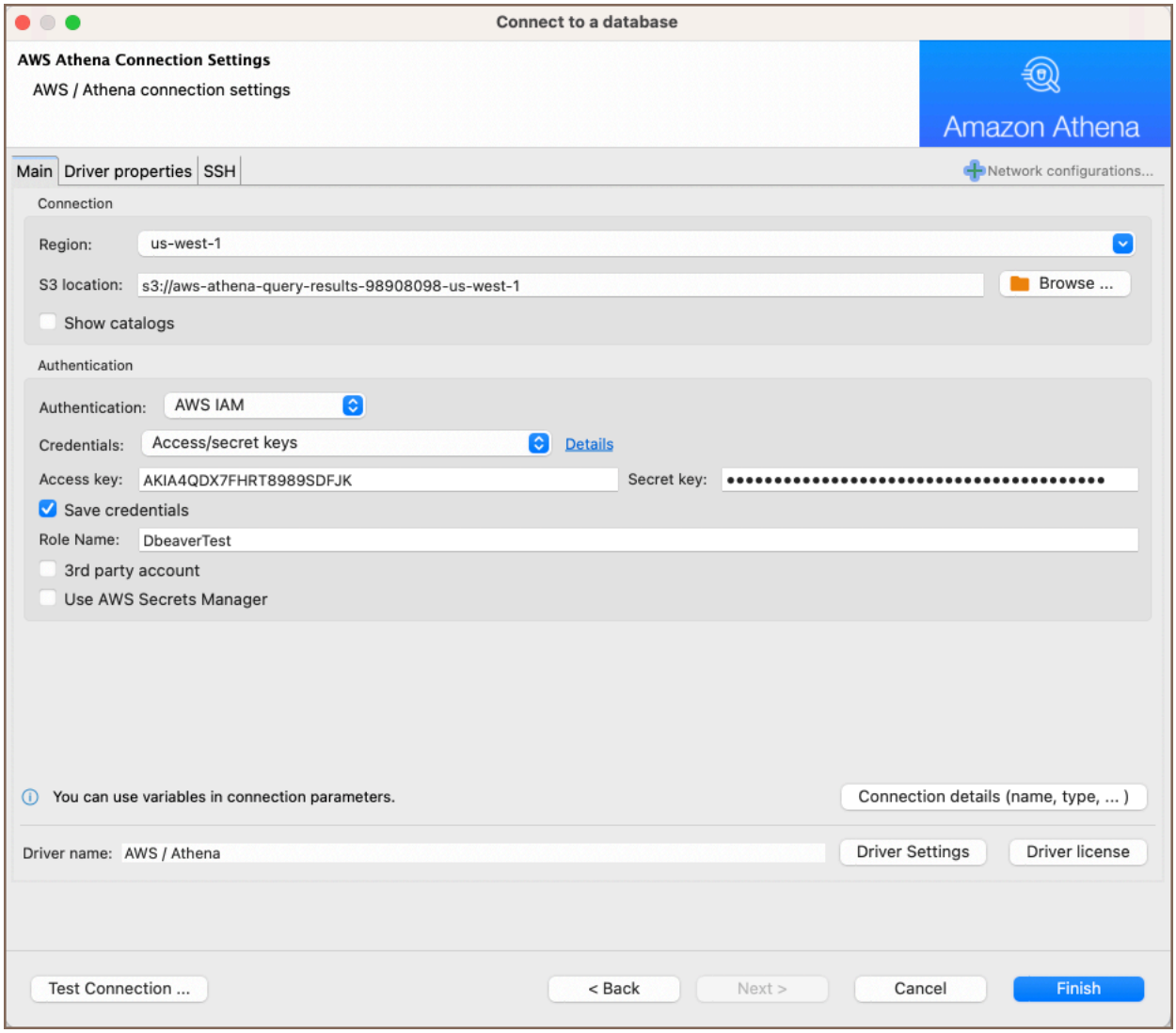
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, Proxies, and Kubernetes.

Athena connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Athena database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Athena database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Region	Specify the AWS region where your Athena database is located, such as `us-west-1`. This should match the region of the S3 bucket used for storing query results.
S3 location	Enter the output location for query results in the format `s3://aws-athena-query-results-[your-account-id]-[region]/`. Replace `[your-account-id]` with your actual AWS account ID and `[region]` with the region code, for example, `s3://aws-athena-query-results-80980980-us-east-1/`. Ensure you have the necessary permissions set for this S3 location.
Browse ★	Click to open a dialog that allows you to navigate and select an S3 bucket through DBeaver's interface.
Show catalogs	Check this option if you want to view the data catalog in the DBeaver connection. This will show the databases and tables defined in Athena's data catalog.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password - Password from shell command - AWS IAM - DBeaver Profile ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here. For detailed guide on driver properties please refer to the official AWS Athena documentation.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Athena connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration

- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure connection configurations

DBeaver supports secure connections to your Athena database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Athena with DBeaver

DBeaver provides a host of features designed for Athena databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Athena database objects

DBeaver lets you view a range of Athena database objects, such as:

- Schemas
  - Tables
    - Columns
  - Views
    - Columns
  - Procedures

Athena additional features in DBeaver

DBeaver provides additional features compatible with Athena, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Schema Management	Schema Compare ★

DocumentDB

Table of contents

- Connections
- Queries

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

AWS DocumentDB is based on the MongoDB engine.
It has several minor differences in the query processing and network configuration.
However, most features which work for MongoDB will work for DocumentDB as well. Please refer to the MongoDB article.

Connections

AWS restricts direct access to DocumentDB clusters from outside of the cloud (region). So you can connect to it directly (using a cluster host name) only when DBeaver is deployed on the EC2 instance.

In other cases you will need to use the SSH tunnel through a proxy machine to access DocumentDB instance. Please read the AWS Documentation about proxy configurations: https://docs.aws.amazon.com/documentdb/latest/developerguide/connect-from-outside-a-vpc.html

In DBeaver you can use the SSH tab on the connection settings page. Just enter proxy host, user name and specify a private key file (it is provided by AWS as a keypair).

Queries

DBeaver processes DocDB SQL queries exactly like in MongoDB. It supports SELECT, UPDATE, INSERT and DELETE queries.
SELECT queries support WHERE, ORDER BY, GROUP BY and HAVING clauses.

DocumentDB restricts the `eval` function so all JavaScript queries will be parsed on the client's side and then evaluated at a DocDB cluster one by one. Most JS functions work exactly like in Mongo Shell.

DynamoDB

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

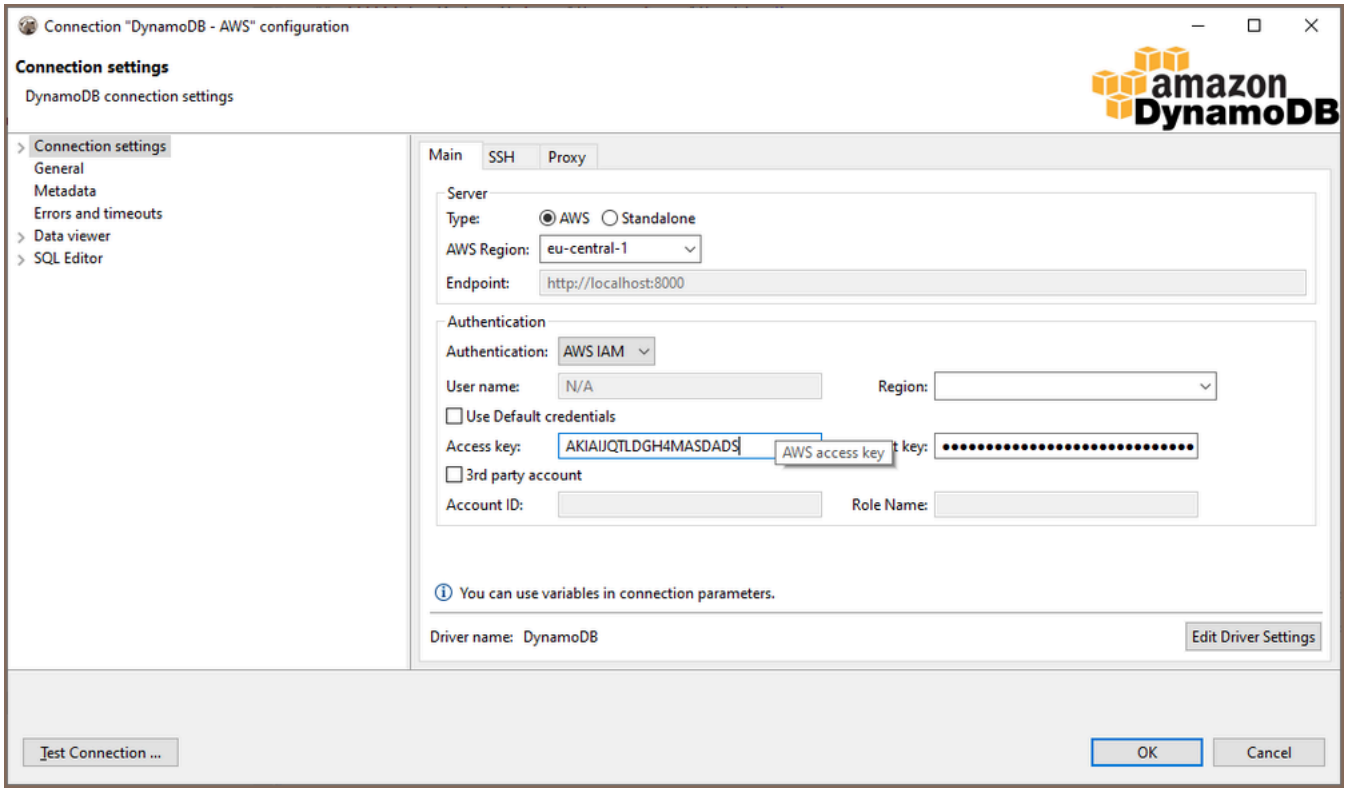
- Supported features
- DynamoDB connection
- Database navigation
- Viewing table data
- Viewing data in JSON document format
- Executing queries
- Exporting and importing data

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

Supported features

- Table data view
- Table data edit in document (json) mode
- Data filters
- SQL queries execution
- JSON queries execution
- Data export and import

DynamoDB connection



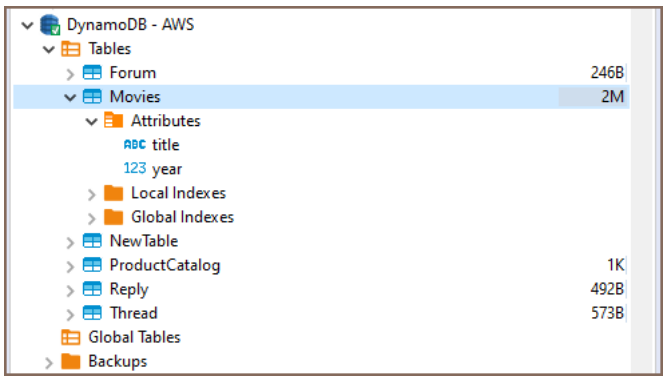
DBeaver supports AWS Cloud and Standalone versions of DynamoDB.
For standalone server you need to enter endpoint (http or https URL).
For cloud server you must enter the AWS region. DynamoDB exists in all available regions in your AWS account but the tables are different.

AWS Access Key and Secret Key are used for authentication.
For 3rd-party account access you must specify the 3rd party account ID (12-digits number) and the 3rd party role name. This role will be used for permission management. You account must be added to the whitelist in the 3rd party account.

Press "Test Connection" to validate your connection settings.

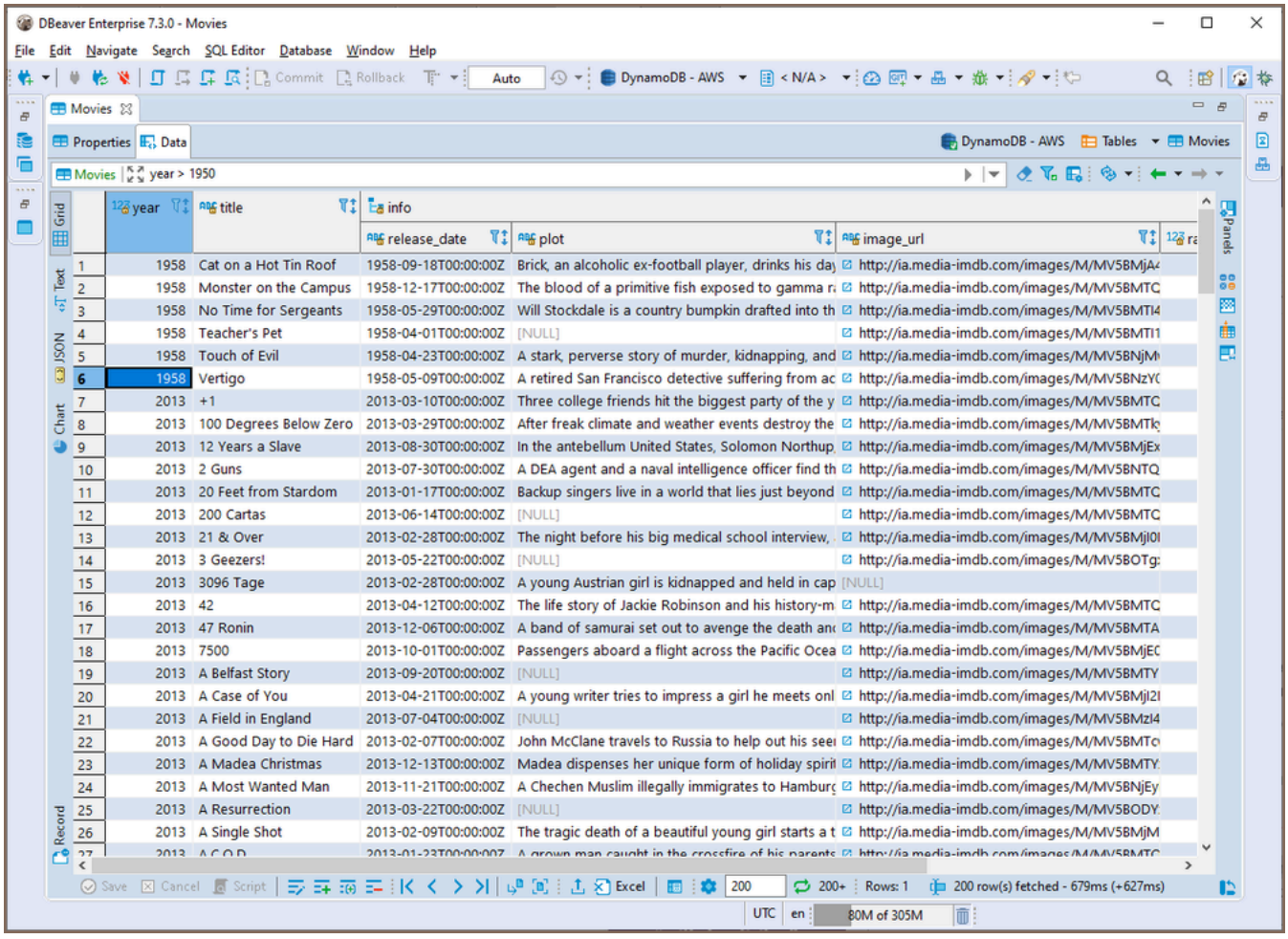
Database navigation

DynamoDB has a simple metadata structure. Basically, you can only access Table and Global tables.
Table has primary attributes (a kind of primary key) and indexes.
DynamoDB is a document-oriented database. Each table may have its own set of attributes and sub-attributes.



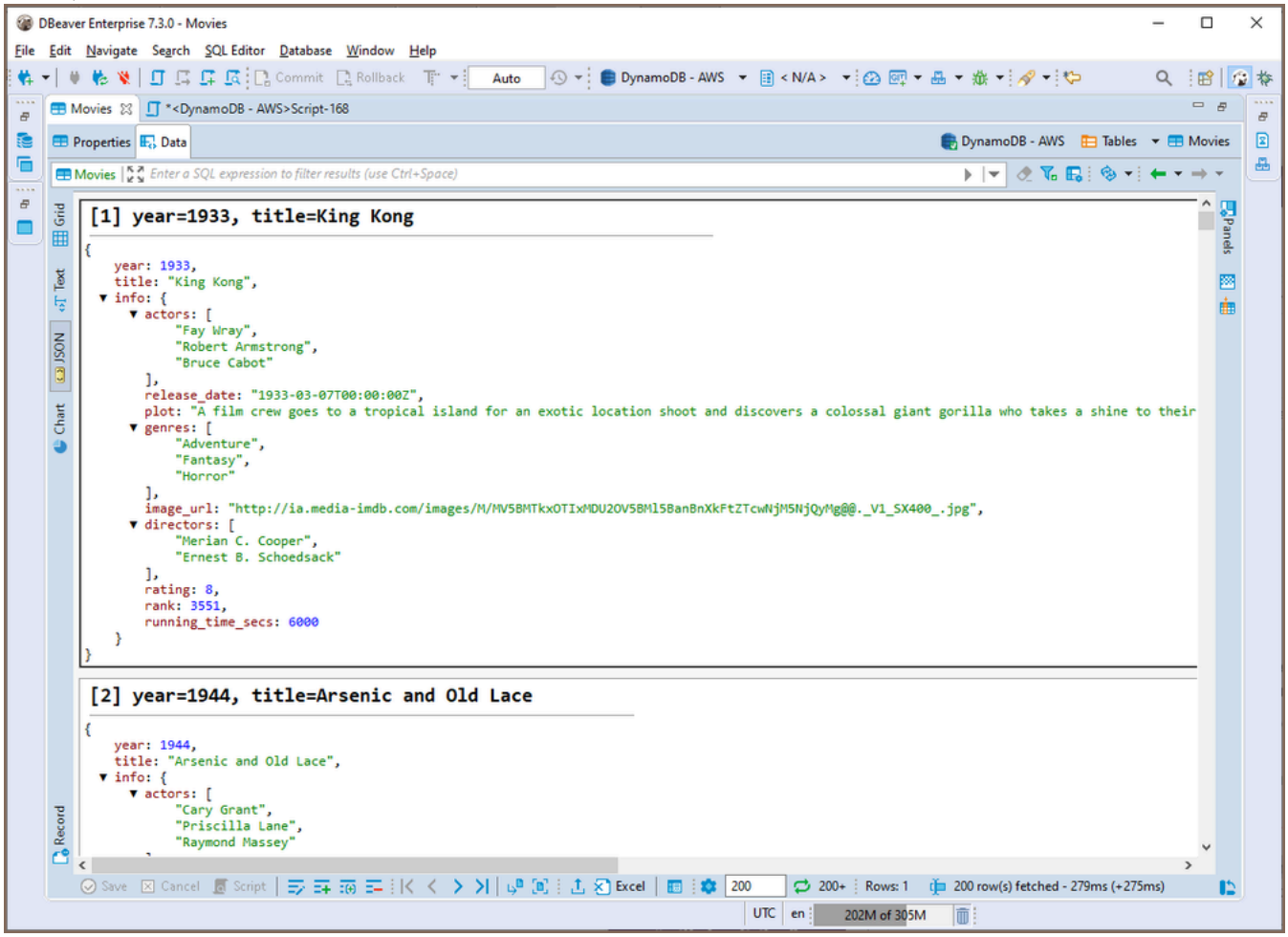
Viewing table data

You can open table editor and see the table data.
You may need to switch to the "Data" tab. DBeaver converts DynamoDB documents into a table format by default, but you can switch to another data representation.
You can use data filters in order to find documents.



Viewing data in JSON document format

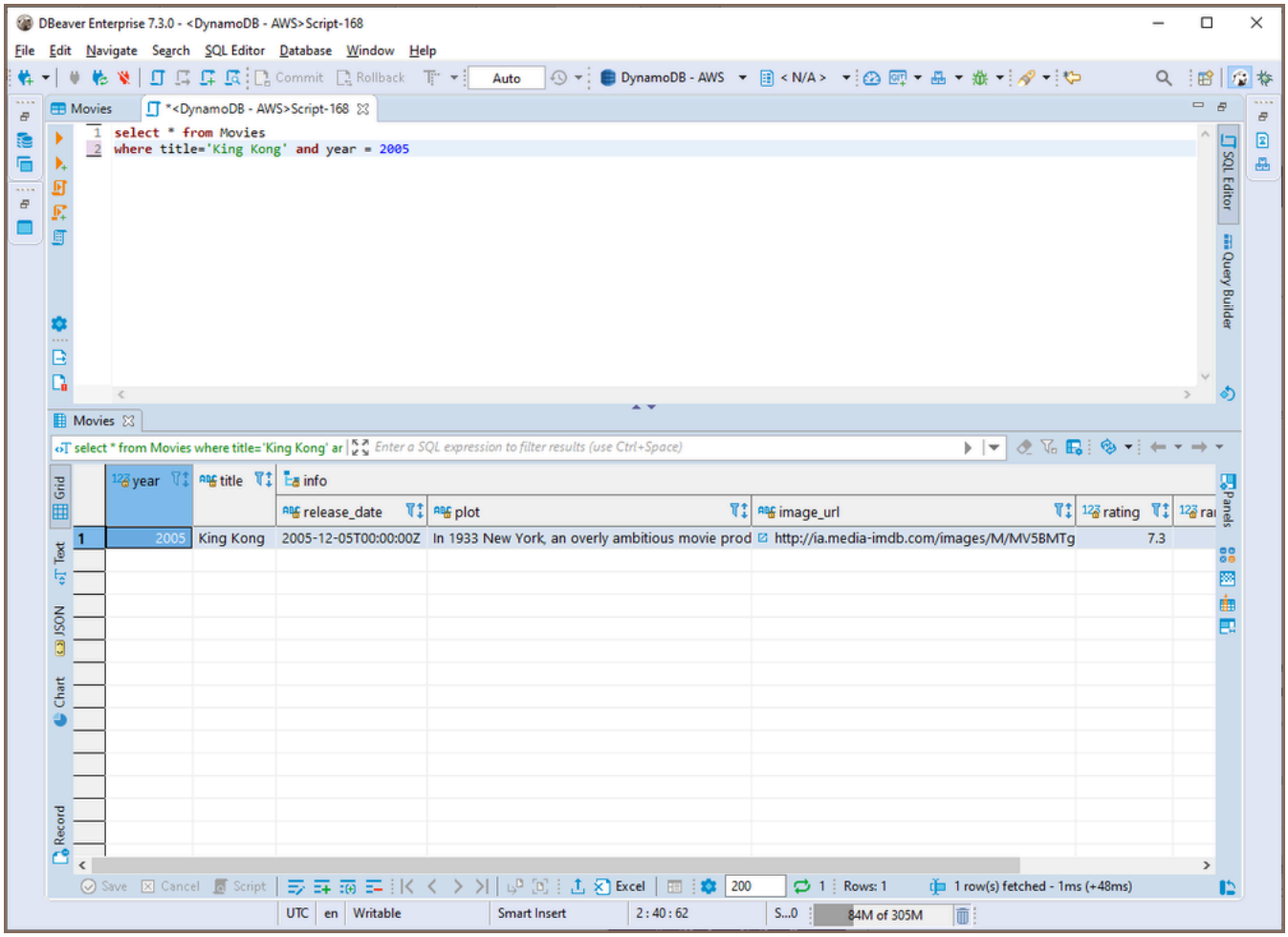
You view, search and edit JSON documents. Double-click on a document to activate the editor.



Executing queries

DBeaver supports simple SQL dialect for DynamoDB.
You can use the WHERE clause in the same fashion as in regular SQL in order to find or filter documents.

You can also use JSON requests syntax to query documents. See Amazon DynamoDB query reference.



Exporting and importing data

You can export data from a DynamoDB table in different file formats (CSV, XLSX, XML, JSON, etc,) or export data directly to another table.

Keyspaces

Table of contents

- Connecting to Keyspaces
- Browsing Keyspaces tables
- Executing CQL
- ERD

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

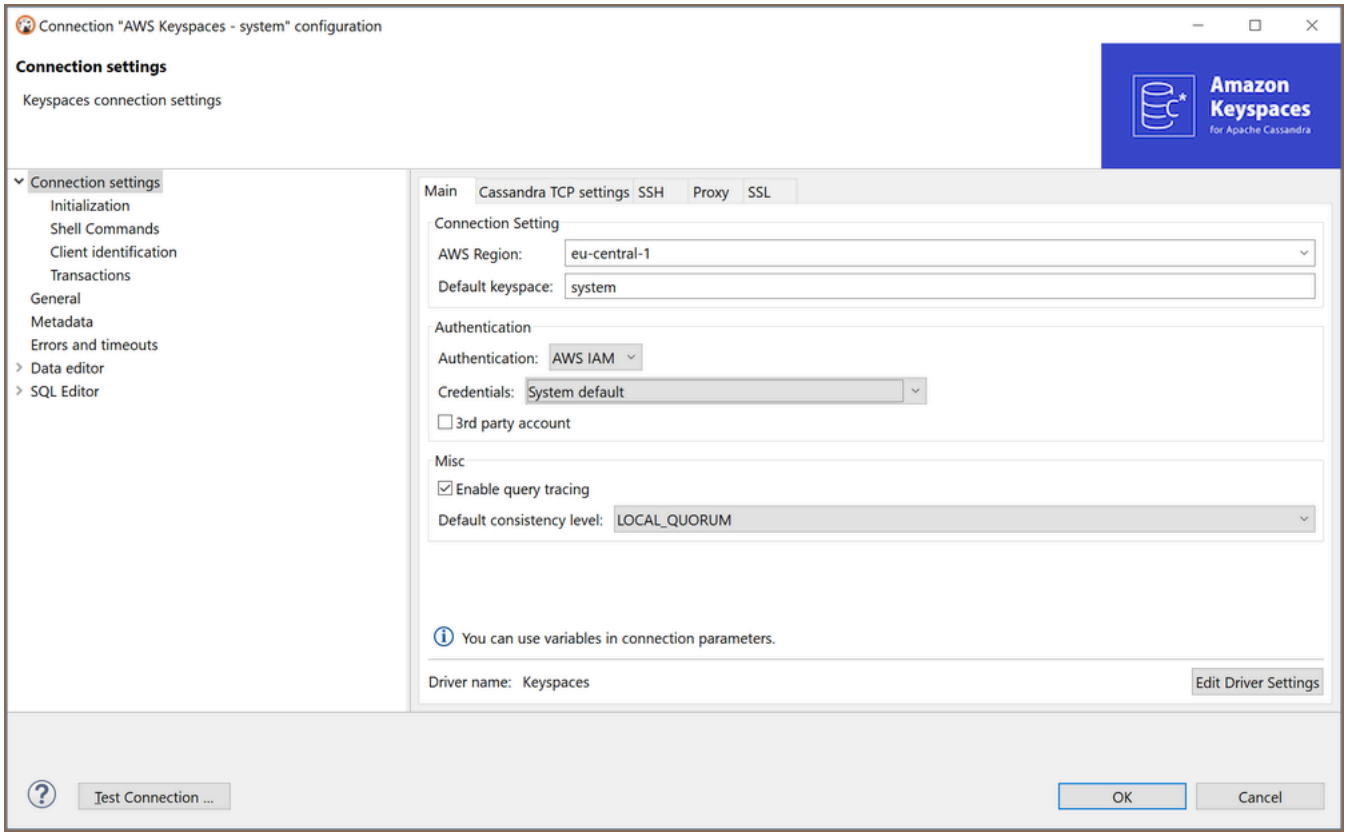
AWS Keyspaces is a key-value database based on Apache Cassandra.

DBeaver supports the Keyspaces schema browser, data viewer and CQL queries execution. It also supports various administrative tools.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

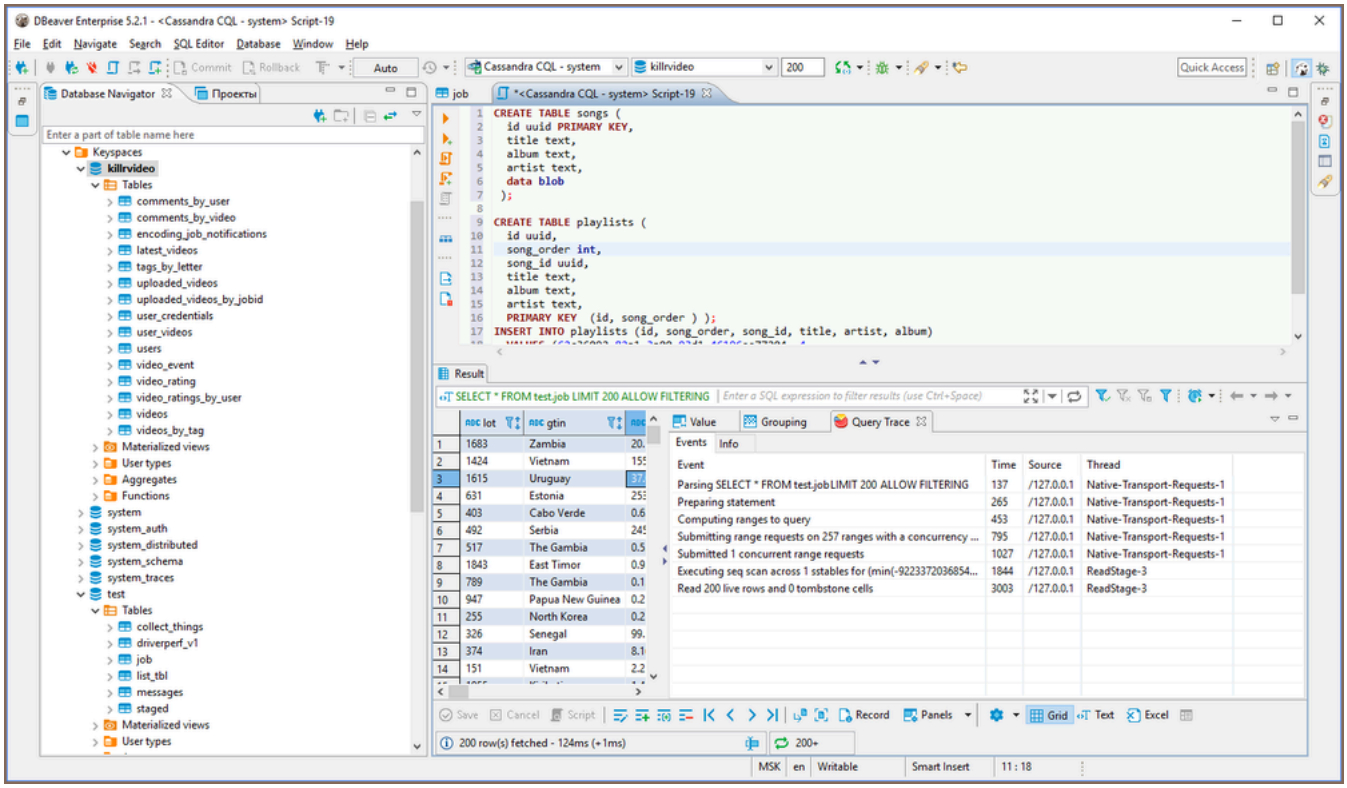
Connecting to Keyspaces

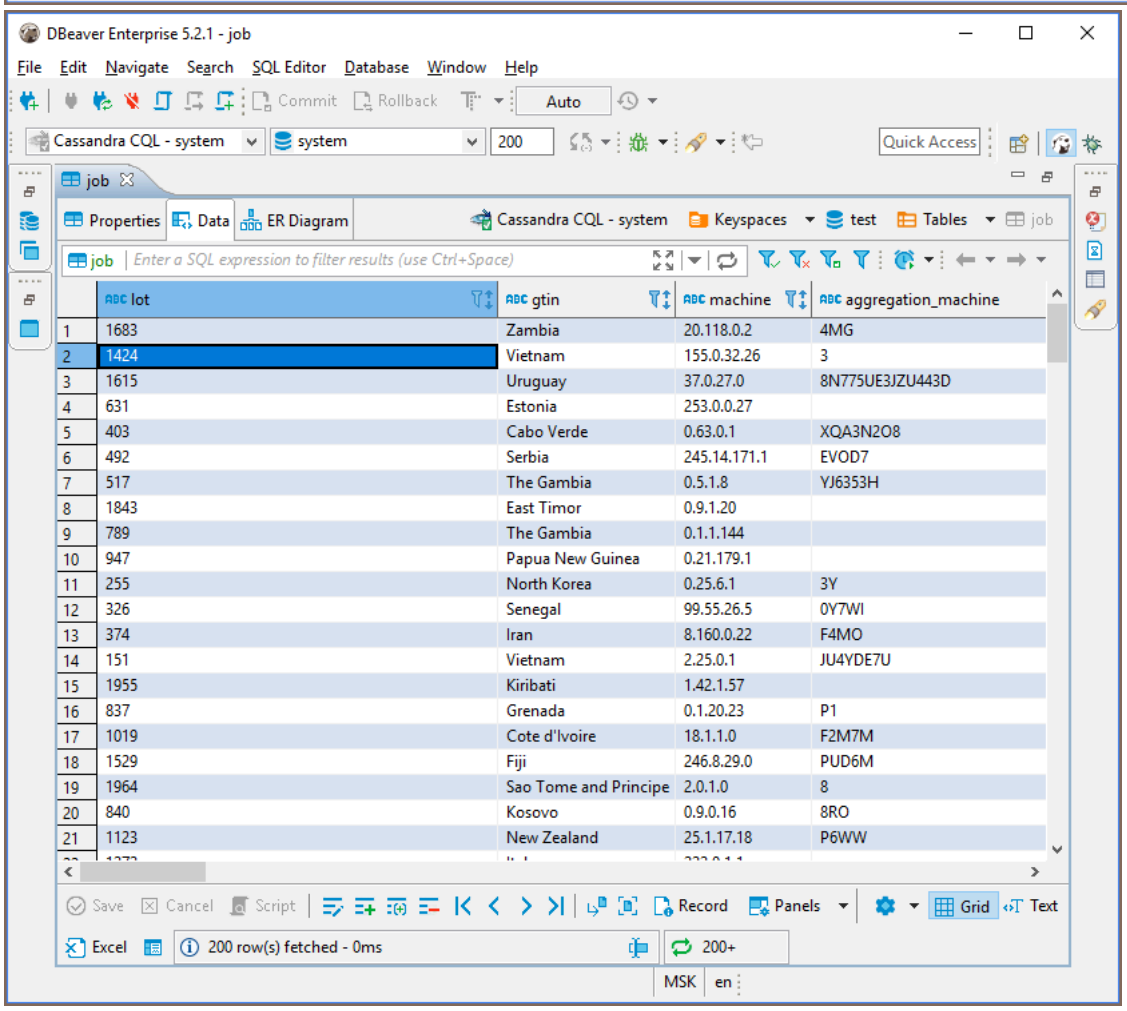
AWS Keyspaces uses AWS IAM authentication.
You need to specify your IAM credentials and AWS region. There is no specific endpoint - there is only one cluster per account per AWS region.
There is also no need to configure SSH or SSL - DBeaver uses default AWS settings to access the Keyspace cluster.



Browsing Keyspaces tables

You can browse, view, edit and filter Cassandra tables the same way as with regular (relational) tables. However, being a distributed key-value database, Keyspaces does not support any kind of referential integrity. There are no foreign keys, references, etc.
You should note that Cassandra has a very advanced (comparing to relational databases) data type system. Each column may be a collection, map, or set of values (with a very big number of values). In some cases this makes browsing data in the "Grid" mode inconvenient.





Executing CQL

CQL Cassandra Query Language is a very simple kind of SQL language dialect.
It supports simple SELECT queries, DDL statements (like CREATE TABLE) and some other query types.

You can use the standard DBeaver SQL editor to execute CQL queries. DBeaver supports Cassandra query execution, results scrolling, data export/import, mock data generation and other features.

ERD

Physical ERD (Entity Relation Diagram) does not make much sense for Keyspaces as there are no foreign keys. However, you can make you own custom ERD and connect Keyspaces tables with each other using logical associations.

Neptune

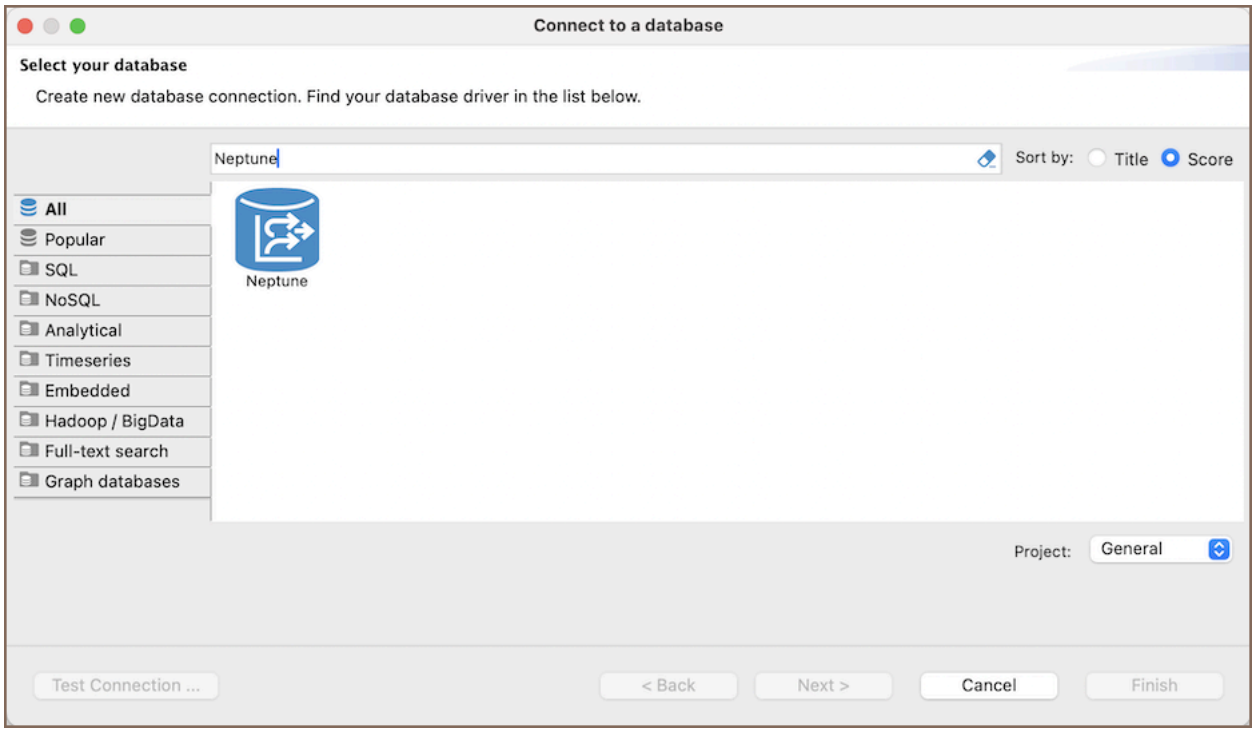
> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Neptune specialty
- Setting up
- Neptune connection settings
- Connection details
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering Neptune with DBeaver
- Neptune database objects
- Neptune additional features in DBeaver

This documentation outlines the process of configuring and using AWS Neptune with DBeaver.

To manage databases using AWS Neptune in DBeaver, establishing a connection is essential. This is done by selecting the `Neptune` option in the database connection settings of DBeaver. For those new to this process, our Creating a Connection article provides detailed guidance.



Neptune specialty

AWS Neptune, as a graph database service, is specialized for use cases involving highly connected data, such as social networks, recommendation engines, fraud detection, and knowledge graphs. Unlike traditional relational databases, Neptune is optimized for storing and querying graph data models, including property graphs and RDF (Resource Description Framework) data. For more detailed information, refer to the AWS Neptune database documentation.

> **Important**: In DBeaver, AWS Neptune is read-only.

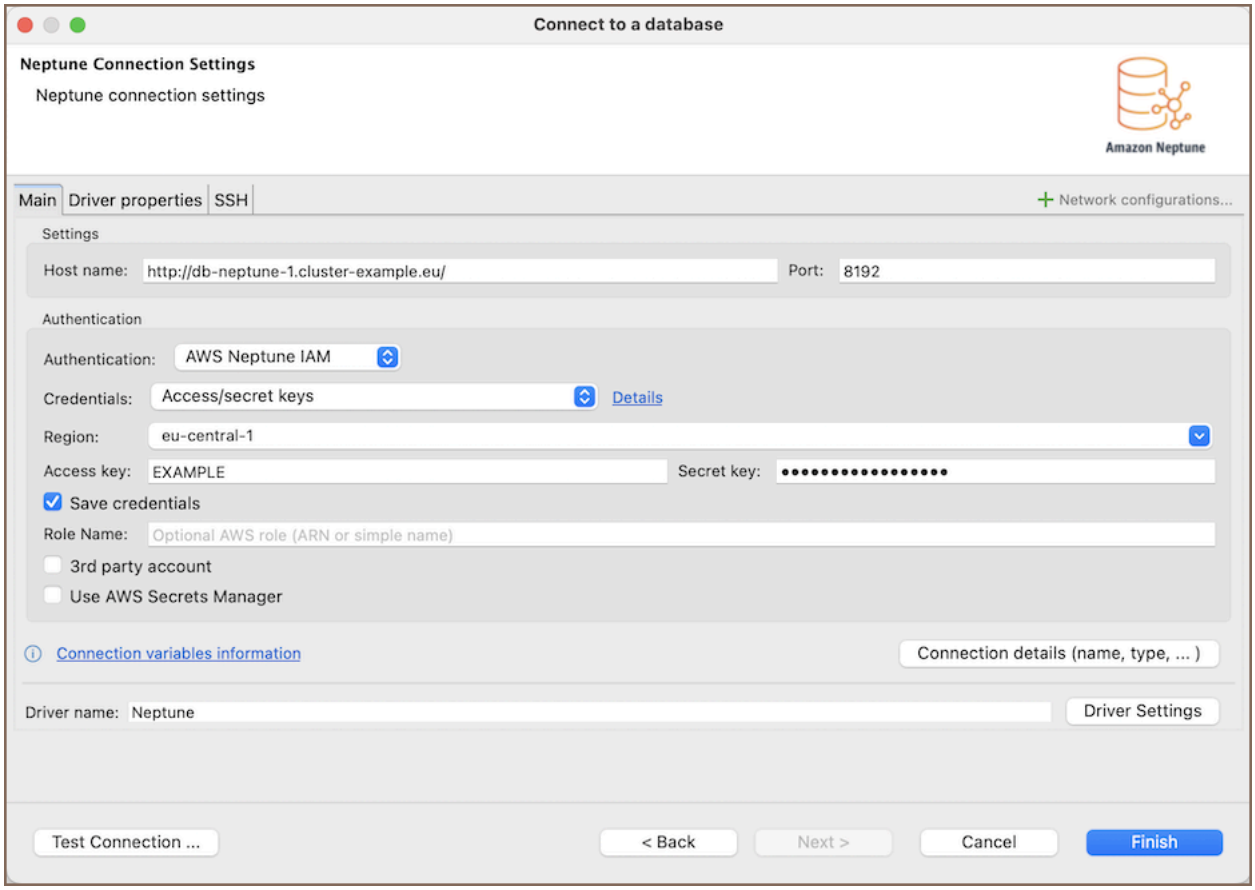
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, Proxies, AWS SSM and Kubernetes.

Neptune connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Neptune database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Neptune database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Host name	Enter the address of your AWS Neptune database here.
Port	Enter the port number for your Neptune database. The default Neptune port is 8192.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - AWS Neptune IAM - DBeaver Profile You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here. For detailed guide on driver properties please refer to the official AWS Neptune documentation.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Neptune connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure connection configurations

DBeaver supports secure connections to your Neptune database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Neptune with DBeaver

DBeaver provides a host of features designed for Neptune databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Neptune database objects

DBeaver lets you view a range of Neptune database objects, such as:

- Schemas
  - Tables
    - Columns

Neptune additional features in DBeaver

DBeaver provides additional features compatible with Neptune, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Schema Management	Schema Compare

Redshift

Table of contents

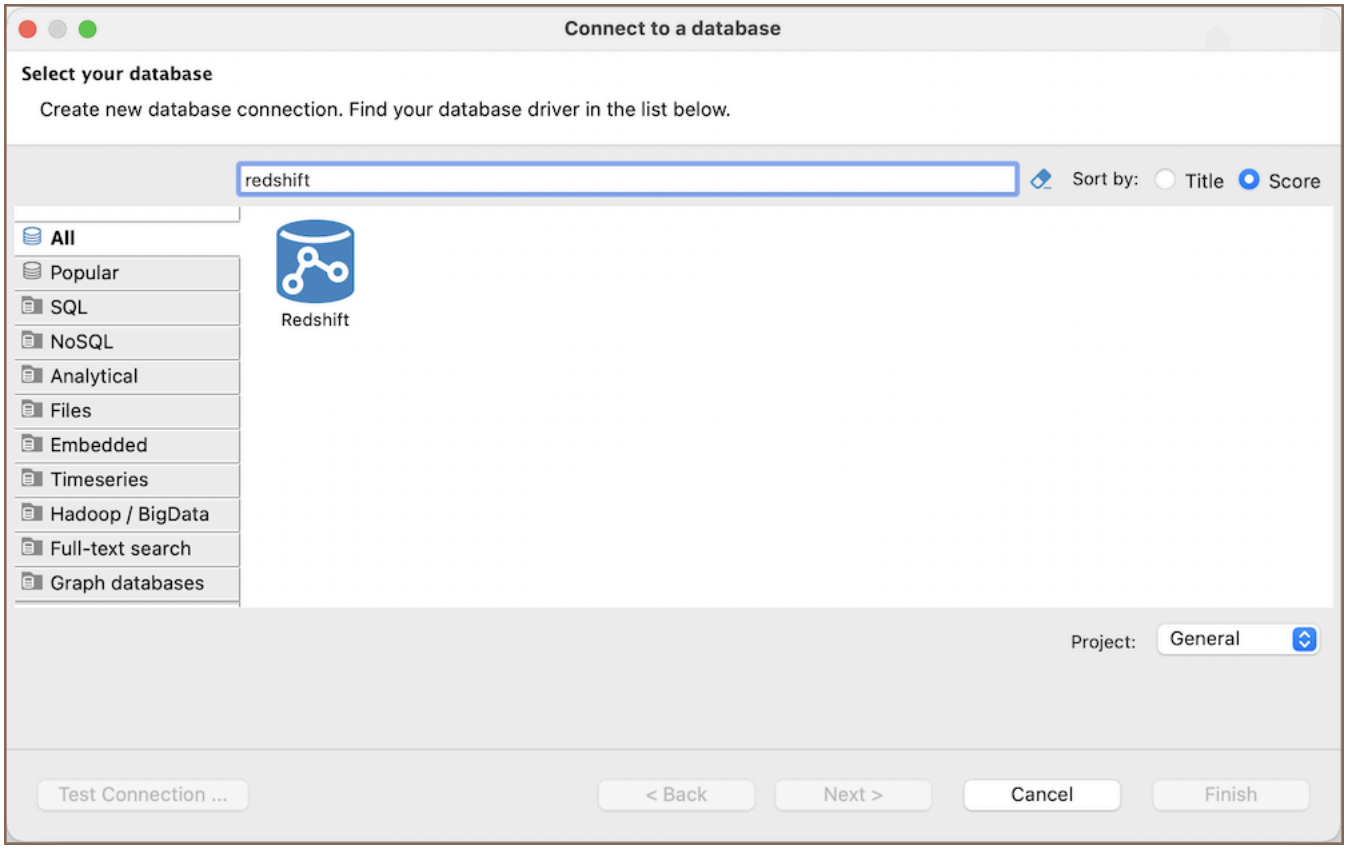
- Redshift specialty
- Setting up
- Redshift connection settings
- Connection details
- Redshift driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Redshift with DBeaver
- Redshift database objects
- Redshift additional features in DBeaver

This section provides guidance on configuring and using AWS Redshift with DBeaver.

AWS Redshift is a fully managed data warehouse service known for its high performance and scalability in handling large datasets. An important aspect of Redshift is that it is based on open standard PostgreSQL, which means it shares numerous settings and capabilities similar to PostgreSQL databases. For detailed information about AWS Redshift's features, visit the AWS Redshift documentation.

Before you start, you must create a connection in DBeaver and select Redshift. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the AWS Redshift server using a specific driver. It supports all Amazon Redshift versions, including both provisioned clusters and serverless.



Redshift specialty

AWS Redshift is an optimized database service designed for large-scale data warehousing and analytics, supporting a wide array of SQL operations. Its foundation in PostgreSQL, through a modified version tailored for its query processing, allows Redshift to support many of the standard SQL features of PostgreSQL. In addition to its PostgreSQL roots, a key feature of AWS Redshift is its columnar data storage. This approach to data storage is particularly efficient for analytics, as it allows for faster retrieval of specific columns, optimizing the performance for query-heavy workloads typically associated with data warehousing. However, it is important to note that AWS Redshift does not fully support referential integrity constraints, such as foreign keys. For more detailed information, refer to the AWS Redshift database documentation.

For a detailed overview of the supported features and capabilities of AWS Redshift within DBeaver, please see the section on Powering Redshift with DBeaver.

> **Warning**: The Redshift Serverless may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

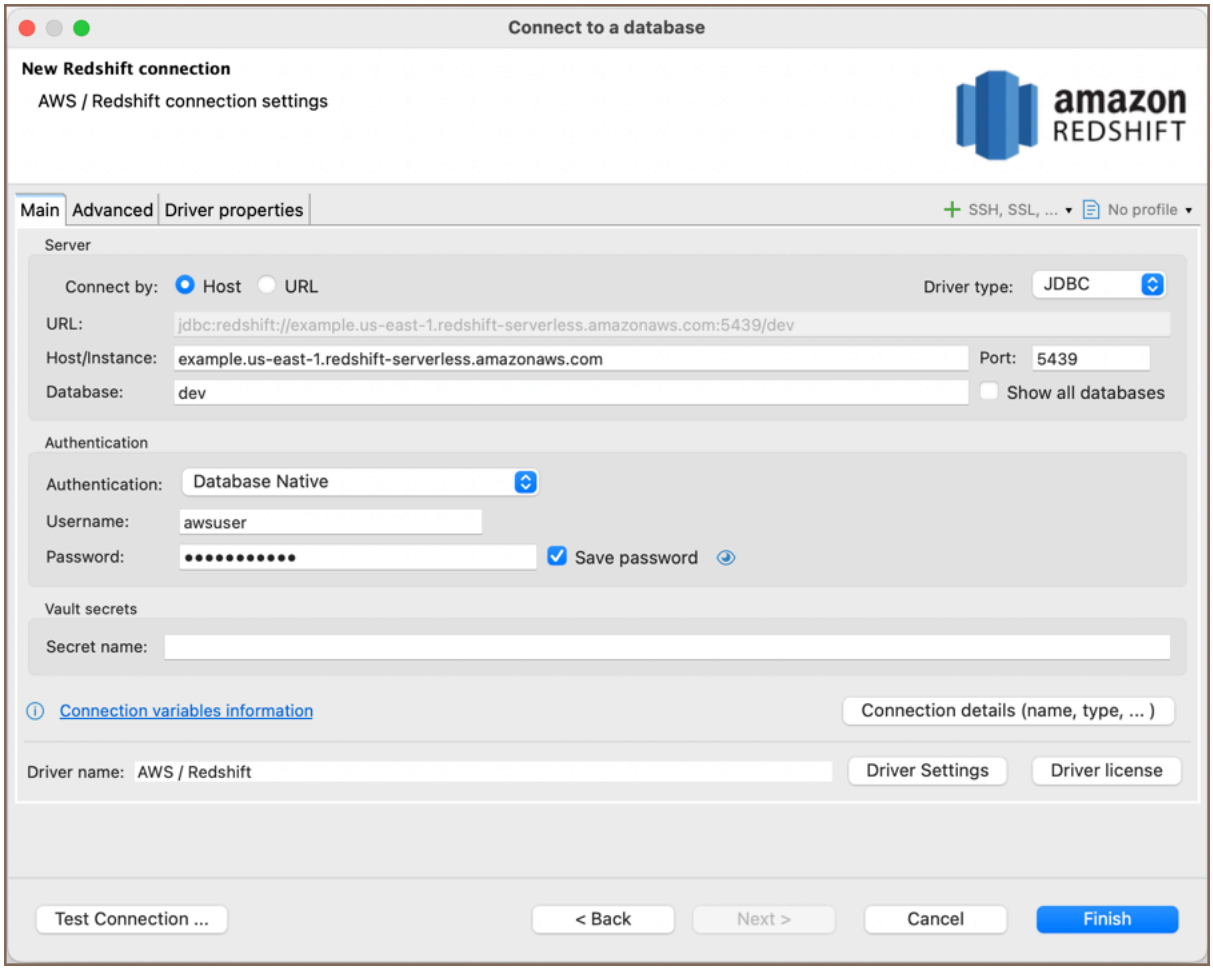
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, SSL, Proxies, Kubernetes, and the setup of ODBC/JDBC drivers for Redshift.

Redshift connection settings

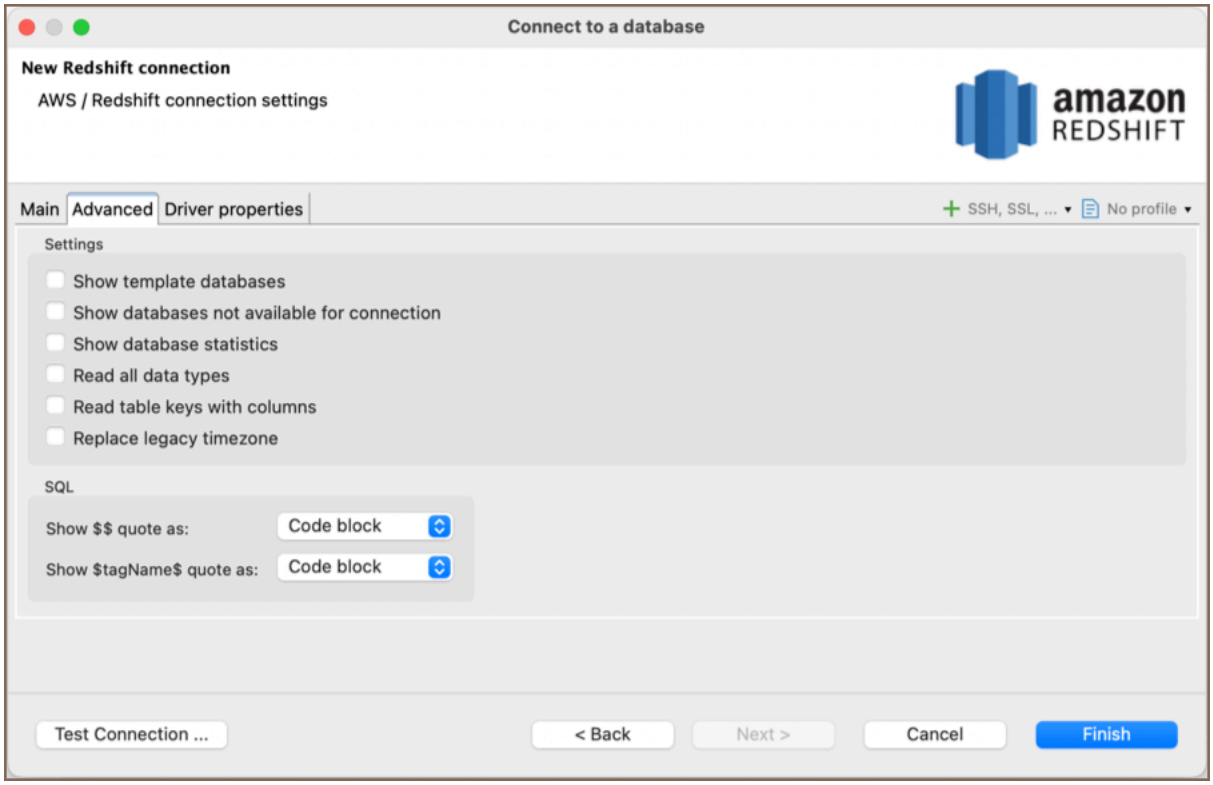
In this subsection, we will outline the settings for establishing a direct connection to a Redshift database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Redshift database.

1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Redshift database.
URL	If you are connecting via URL, enter the URL of your Redshift database here. This field is hidden if you are connecting via the host.
Host/Instance	If you are connecting via host or instance, enter the address of your AWS Redshift database here.
Database	Enter the name of the Redshift database you want to connect to.
Show all databases	Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Port	Enter the port number for your Redshift database. The default Redshift port is 5439.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - PostgreSQL PgPass - AWS Redshift IAM ★ - DBeaver Profile Authentication ★ - PostgreSQL Kerberos ★ - PostgreSQL SSPI (Windows SSO) ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The **Advanced** tab of the connection settings offers additional options that allow you to customize your further connection to the Redshift database.



Field	Description
Show template databases	Displays template databases. This option is available when **Show all databases** is checked.
Show databases not available for connection	Show databases not available for connection in database list. This option is available when **Show all databases** is checked.
Show databases statistics	Displays statistics for databases.
Read all data types	Reads all data types from the server (including arrays).
Read table keys with columns	Enable reading table constraints during the column reading stage to ensure accurate recognition of keys. Note that this setting may decrease metadata loading performance.
Replace legacy timezone	Replaces legacy timezone settings. Enable only if you need to connect with legacy timezones like **Europe/Kyiv** or **Asia/Calcutta**. Only these two timezones are affected.
Show `$$` quote as	Displays `$$` quotes as code blocks or strings.
Show `$tagName$` quote as	Displays `$tagName$` quotes as code blocks or strings.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Redshift connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Redshift driver properties

The settings for Redshift **Driver properties** enable you to adjust the performance of the Redshift JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Redshift database.

For a complete walkthrough on setting up Redshift JDBC and ODBC drivers properties, you can refer to the official AWS Redshift JDBC documentation and AWS Redshift ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Redshift database connections.

You can customize the Redshift driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Redshift database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Redshift database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**

- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Redshift with DBeaver

DBeaver provides a host of features designed for Redshift databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Redshift database objects

DBeaver lets you view and manipulate a wide range of Redshift database objects. DBeaver has extensive support for various Redshift metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Tables
      - Columns
      - Constraints
      - Foreign Keys
      - Dependencies
      - References
    - Views
    - Functions
    - Data types
    - Aggregate functions
  - Users ★
  - Groups ★
  - Roles ★
- Administer
  - Jobs
  - Session Manager
- Datashares ★
  - Objects ★
  - Consumers ★

Redshift additional features in DBeaver

DBeaver provides additional features compatible with Redshift, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Data Visualization	ERD Guide
	Query Execution plan
Session Management	Session Manager
Schema Management	Schema Compare ★
Data Generation	Mock Data Generation ★

Timestream

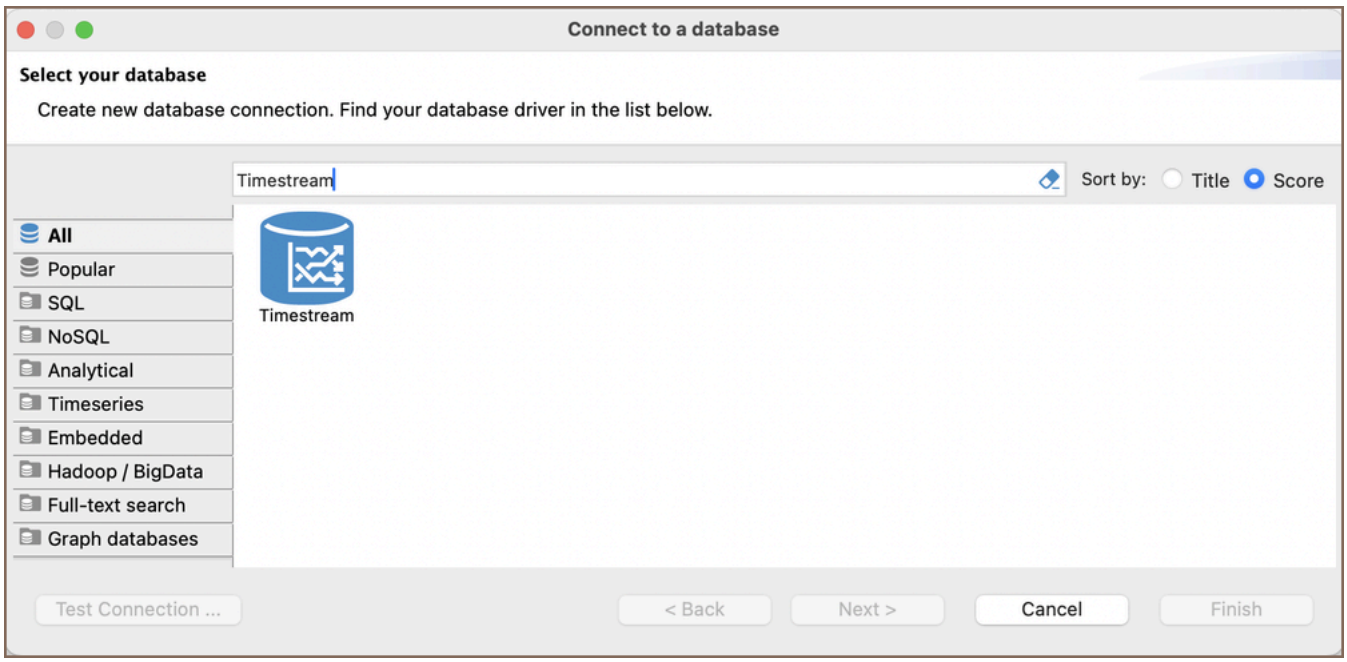
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Timestream specialty
- Setting up
- Timestream connection settings
- Connection details
- Timestream driver properties
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering Timestream with DBeaver
- Timestream database objects
- Timestream features in DBeaver

This documentation outlines the process of configuring and using AWS Timestream with DBeaver. For more information, visit the official AWS Timestream website.

To manage databases using AWS Timestream in DBeaver, establishing a connection is essential. This is done by selecting the `Timestream` option in the database connection settings of DBeaver. For those new to this process, our Creating a Connection article provides detailed guidance.



Timestream specialty

AWS Timestream, while optimized for time-series data, lacks some features common in traditional relational databases. It does not have features like constraints, foreign keys, triggers, views, or complex joins, and sub-queries limit DBeaver's capabilities when working with Timestream. For more detailed information, refer to the AWS Timestream database documentation. Due to the absence of certain features in AWS Timestream, there is an inability to link tables, automate tasks based on database changes, create virtual tables from queries, or conduct an in-depth analysis of how data is related within Timestream. For more detailed information, refer to the AWS Timestream database documentation.

> **Important**: In DBeaver, AWS Timestream is read-only.

For a detailed overview of the supported features and capabilities of AWS Timestream within DBeaver, please see the section on Powering Timestream with DBeaver.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

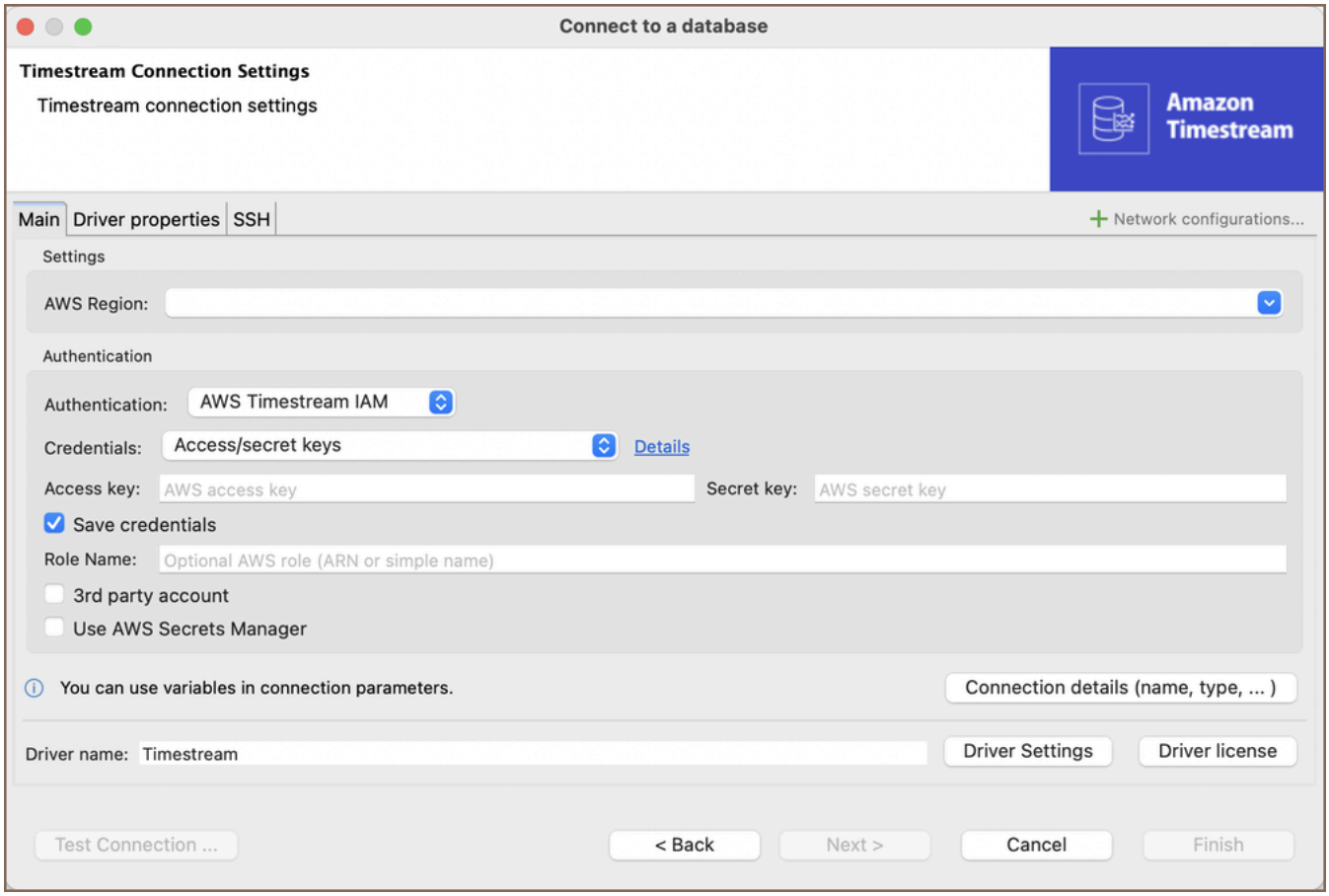
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, Proxies, and Kubernetes.

Timestream connection settings

This subsection will outline the settings for establishing a direct connection to a Timestream database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Timestream database.

The connection settings page requires you to fill in specific fields to establish the initial connection.



Field	Description
Region	Specify the AWS region where your Timestream database is located, such as `us-west-1`.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - AWS Timestream IAM - DBeaver Profile You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here. For a detailed guide on driver properties, please refer to the official AWS Timestream documentation.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Timestream connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Timestream driver properties

The settings for Timestream **Driver properties** enable you to adjust the performance of the Timestream JDBC driver.

For a complete walkthrough on setting up Timestream JDBC driver properties, you can refer to the official Timestream JDBC documentation. These guides detail each driver's properties and how they can be used to optimize Timestream database connections.

You can customize the Timestream driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Timestream database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure connection configurations

DBeaver supports secure connections to your Timestream database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Timestream with DBeaver

DBeaver provides a host of features designed for Timestream databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Timestream database objects

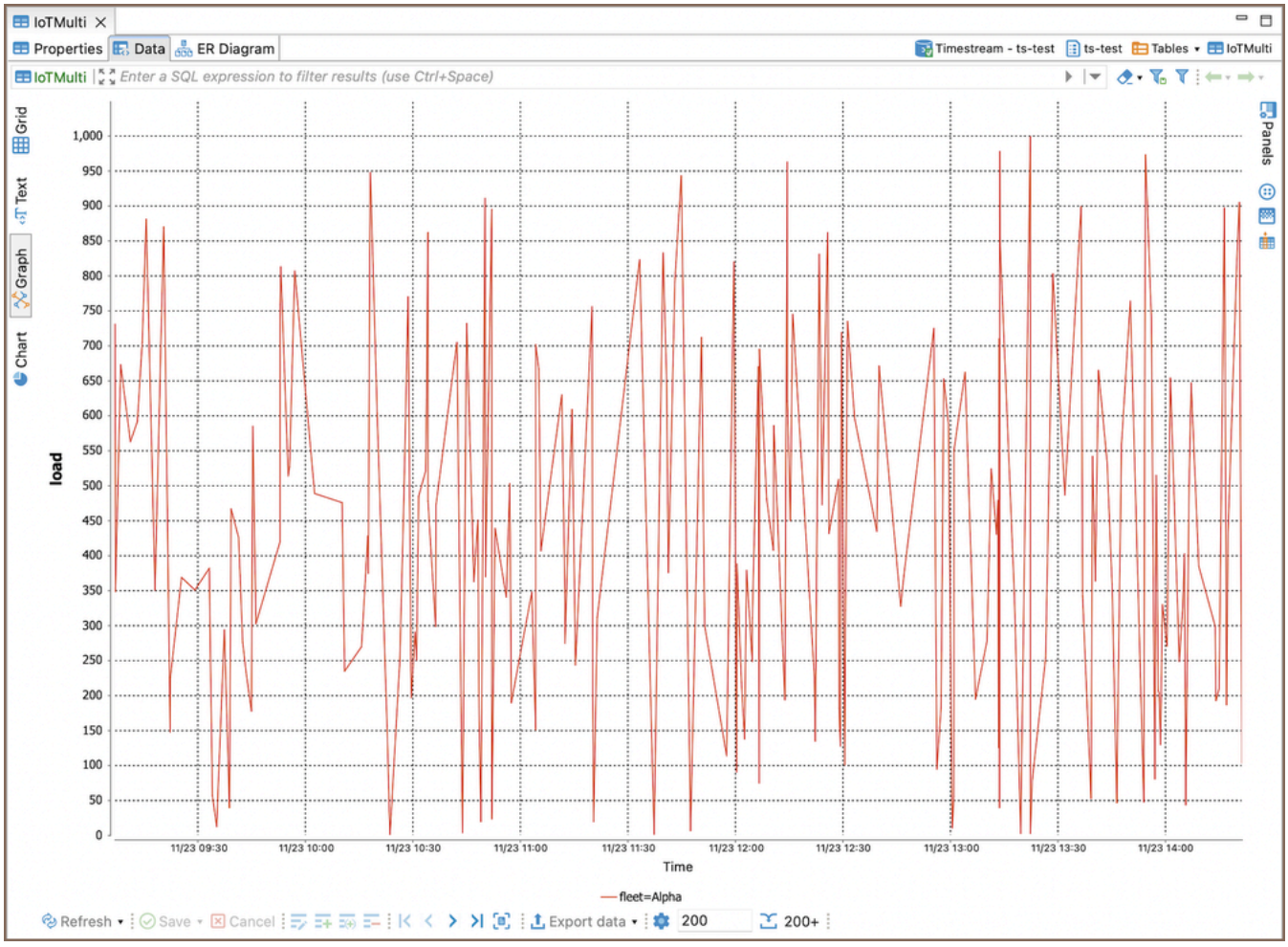
DBeaver lets you view a few of the Timestream database objects, such as:

- Schemas
  - Tables
    - Columns

Timestream features in DBeaver

DBeaver provides the following features compatible with Timestream, but not exclusive to it:

- **Data Export**: Enables the exportation of data. See Data Export for more details.
- **Graph Construction**: Allows the building of detailed and informative graphs.



Azure

Cosmos DB

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Setting Up
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Differences between Cosmos DB and MongoDB in DBeaver
- Session Manager

DBeaver offers extensive support for Cosmos DB, a globally distributed, multimodel database service provided by Microsoft Azure.

Before you start, you must create a connection in DBeaver and select Cosmos DB. If you have not done this, please refer to our Database Connection article.

Cosmos DB is similar to MongoDB in many ways, and DBeaver's support for Cosmos DB includes collections browsing, data viewing, and SQL queries execution. For a detailed guide, refer to the MongoDB article.

Setting Up

Setting up Cosmos DB in DBeaver is also similar to setting up MongoDB.

Secure Connection Configurations

DBeaver supports secure connections to your Cosmos DB database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Differences between Cosmos DB and MongoDB in DBeaver

Session Manager

Unlike MongoDB, Cosmos DB does not have a Session Manager in DBeaver. This means that you would not be able to manage server sessions directly within DBeaver when using Cosmos DB.

Cosmos DB for NoSQL

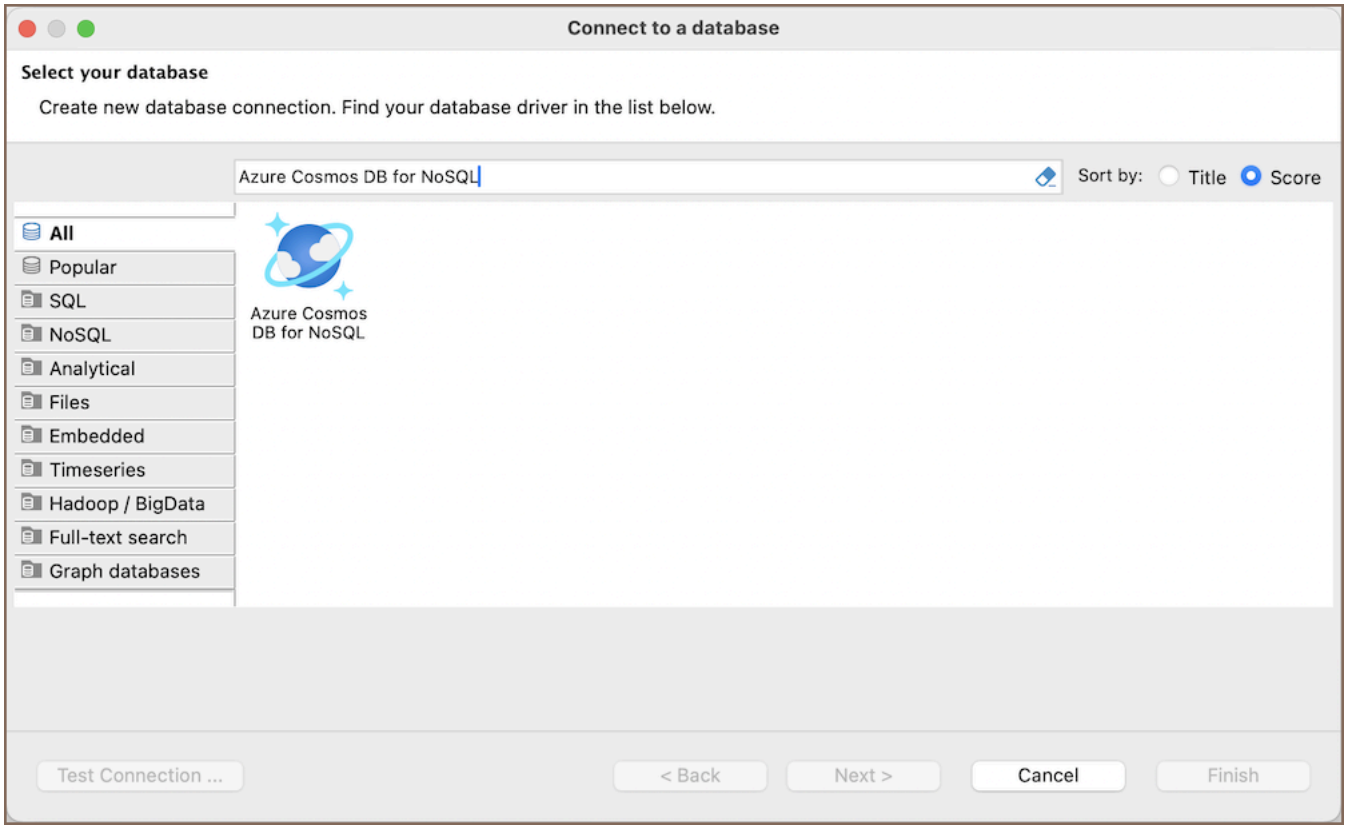
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Azure Cosmos DB for NoSQL specialty
- Setting Up
- Connection settings
- Connection details
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Azure Cosmos DB for NoSQL with DBeaver
- Supported queries
- Azure Cosmos DB for NoSQL database objects
- Azure Cosmos DB for NoSQL features in DBeaver

This guide shows how to connect Azure Cosmos DB for NoSQL in DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Azure Cosmos DB for NoSQL` option to connect to the Azure Cosmos DB for NoSQL database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Azure Cosmos DB for NoSQL specialty

Azure Cosmos DB for NoSQL is a cloud database from Microsoft. It stores data as `JSON` documents and doesn't require a fixed schema. It's fast, can scale to handle large workloads, and lets you run SQL-like queries. For details, see Supported queries.

It's designed for global apps, with automatic replication and high availability. You can also set different consistency levels to balance speed and accuracy. For details, see the official documentation.

> **Important**: In DBeaver, Azure Cosmos DB for NoSQL is read-only.

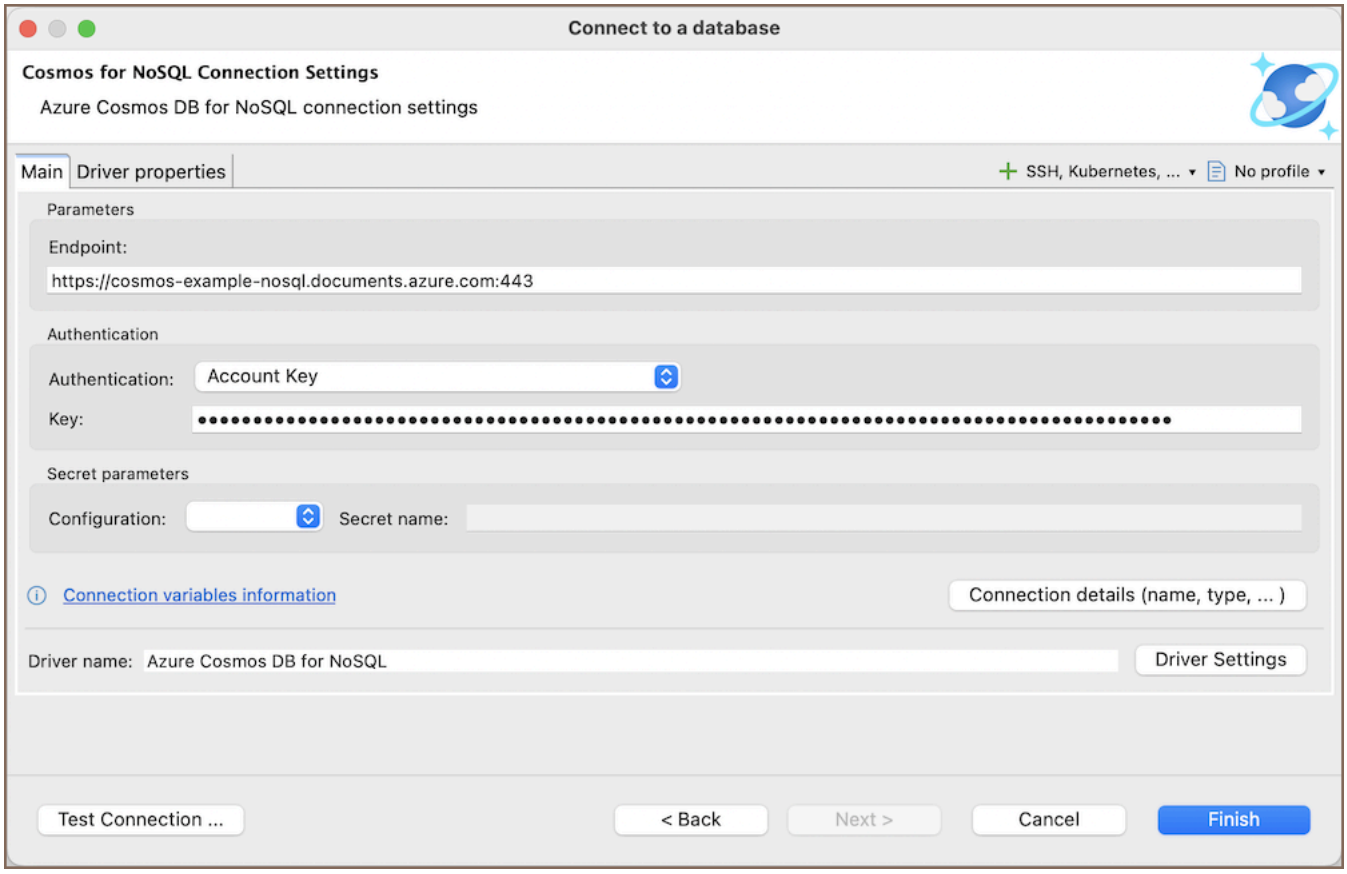
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies and Kubernetes for Azure Cosmos DB for NoSQL.

Connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Azure Cosmos DB for NoSQL database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Azure Cosmos DB for NoSQL database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Endpoint	Enter the connection endpoint (URI) of your Azure Cosmos DB for NoSQL account.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - DBeaver Profile Authentication - Account key (article coming soon) - Microsoft Entra ID for Cosmos DB for NoSQL
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Azure Cosmos DB for NoSQL connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure Connection Configurations

DBeaver supports secure connections to your Azure Cosmos DB for NoSQL database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Azure Cosmos DB for NoSQL with DBeaver

DBeaver provides a host of features designed for Azure Cosmos DB for NoSQL databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Supported queries

Azure Cosmos DB for NoSQL in DBeaver supports a subset of SQL-like syntax:

- `SELECT`, `FROM`, `WHERE`
- aliases (`SELECT * FROM c`)
- `FROM` can use a fully qualified name, a container name, or an alias

Azure Cosmos DB for NoSQL database objects

DBeaver lets you view and work with the main Azure Cosmos DB for NoSQL database objects:

- Databases
  - Containers
    - Tables

Azure Cosmos DB for NoSQL features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous features compatible with Azure CosmosDB, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Dashboards	Dashboards
AI Assistant	AI Assistant

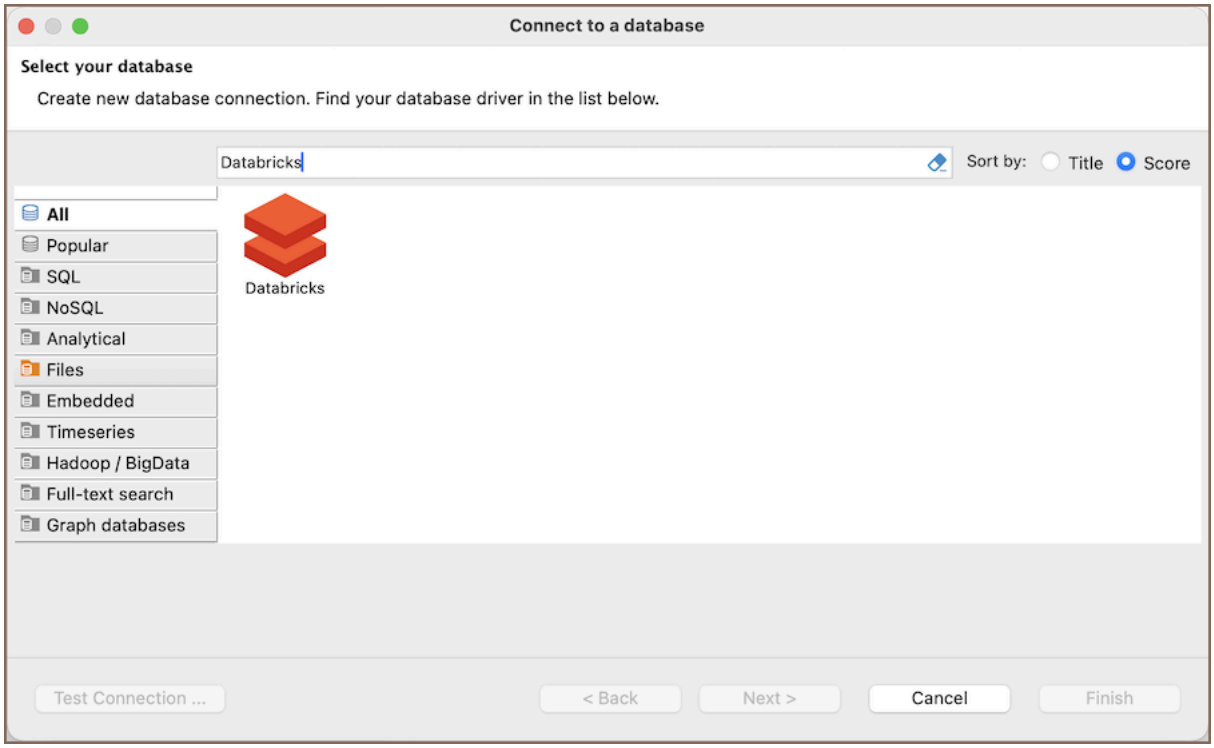
Databricks

Table of contents

- Databricks specialty
- Setting up
- Databricks connection settings
- Connection details
- Databricks driver properties
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering Databricks with DBeaver
- Databricks database objects
- Databricks features in DBeaver

This guide provides instructions on how to set up and use Databricks with DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Databricks` option to connect to the Databricks database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Databricks specialty

Databricks, based on Apache Spark, is a distributed computing system primarily used for machine learning and big data analytics. Its architecture is specially optimized for handling large-scale data processing and advanced analytics tasks. Databricks integrates with MLlib in Apache Spark, offering scalable algorithms and utilities for machine learning tasks. It also facilitates collaborative work through its notebooks, allowing teams to share code, visualizations, and insights effectively. For more detailed information and a comprehensive understanding of Databricks, please refer to the official documentation.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

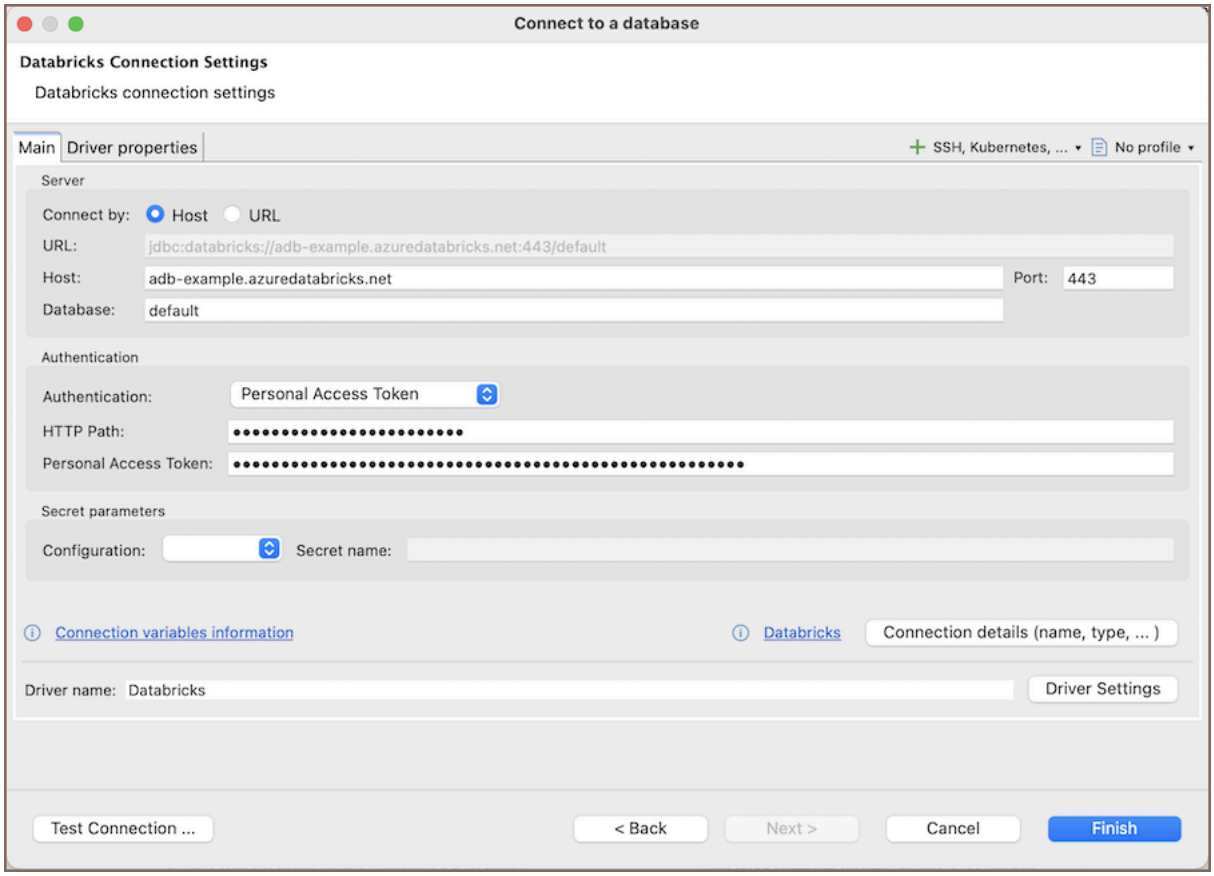
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH and proxies for Databricks.

Databricks connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Databricks database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Databricks database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Host/Instance	If you are connecting via host or instance, enter the address of your Databricks database here.
Database	Enter the name of the Databricks database you want to connect to.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - OAuth (Machine to Machine) ★ - OAuth (User to Machine) ★ - Personal Access Token ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with a Databricks database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Databricks driver properties

The settings for Databricks **Driver properties** enable you to adjust the performance of Databricks JDBC driver. These adjustments can influence the efficiency, compatibility, and features of your Databricks database.

For a complete walkthrough on setting up Databricks JDBC driver properties, you can refer to the official Databricks JDBC documentation. These guides detail each driver's properties and how they can be used to optimize Databricks database connections.

You can customize the Databricks driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Databricks database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure connection configurations

DBeaver supports secure connections to your Databricks database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Databricks with DBeaver

DBeaver provides a host of features designed for Databricks databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Databricks database objects

DBeaver lets you view and manipulate a wide range of Databricks database objects. DBeaver has extensive support for various Databricks metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Tables
    - Columns
    - Keys
    - Foreign keys
    - References
  - Views
  - Procedures

Databricks features in DBeaver

DBeaver is not limited to typical SQL tasks. It also includes numerous additional features compatible with Databricks, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Data Visualization	ERD Guide
	GIS Guide
Data Generation	Mock Data Generation

Google

AlloyDB for PostgreSQL

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

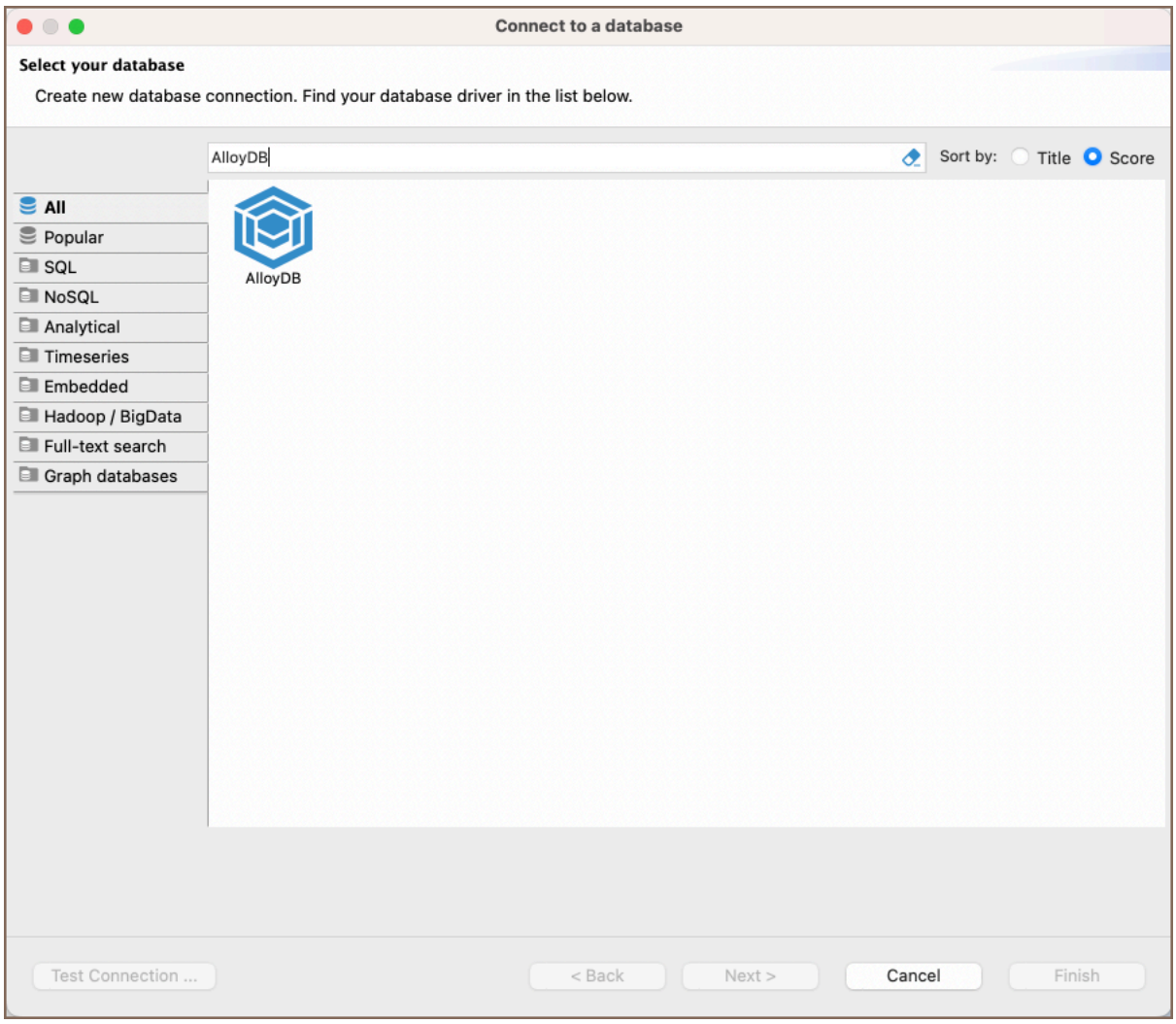
Table of contents

- AlloyDB specialty
- Setting Up
- AlloyDB connection settings
- Connection details
- Driver properties
- ODBC and JDBC driver configuration
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering AlloyDB with DBeaver
- AlloyDB database objects
- AlloyDB additional features in DBeaver

AlloyDB for PostgreSQL is a fully managed database service, tailored for demanding enterprise workloads. It blends the reliability and familiarity of PostgreSQL with Google's advanced technology. This service delivers enhanced performance, scalability, and high availability, making it an ideal choice for critical business applications.

For more detailed information, visit Google AlloyDB webpage.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `AlloyDB` option to connect to the AlloyDB database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



AlloyDB specialty

AlloyDB for PostgreSQL is a fully managed database service, tailored for demanding enterprise workloads. It blends the reliability and familiarity of PostgreSQL with Google's advanced technology and supports rich metadata structures, including tables, views, indexes, sequences, and various data types. Its setup in DBeaver mostly replicates the PostgreSQL driver configuration. AlloyDB includes support for SQL triggers, stored procedures, and user-defined functions, enhancing its utility for complex data operations. For a more comprehensive understanding of AlloyDB, please refer to the official documentation.

Setting Up

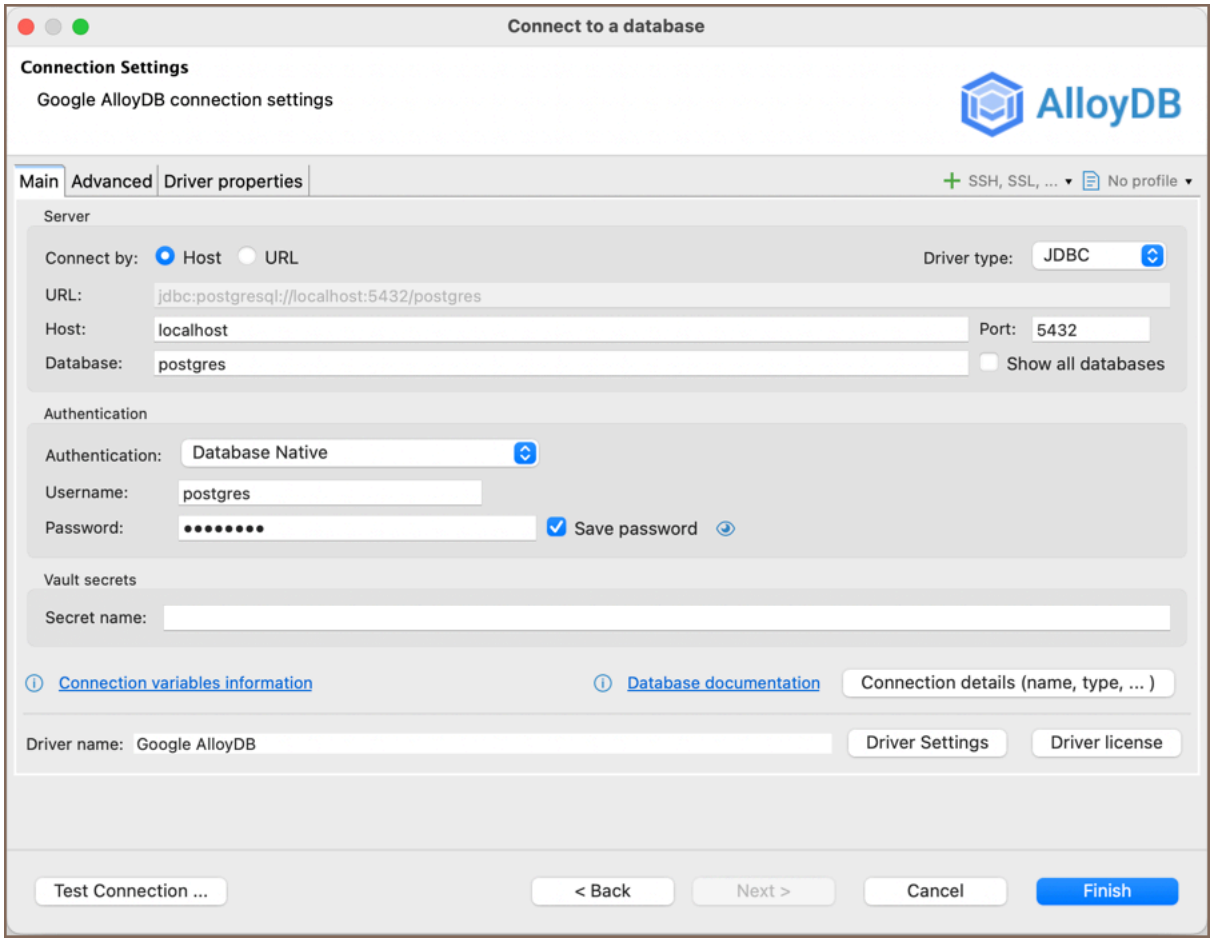
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for AlloyDB.

> **Note**: The connection process in DBeaver utilizes the standard PostgreSQL driver, tailored to interact with the AlloyDB service, highlighting its cloud-based characteristics.

AlloyDB connection settings

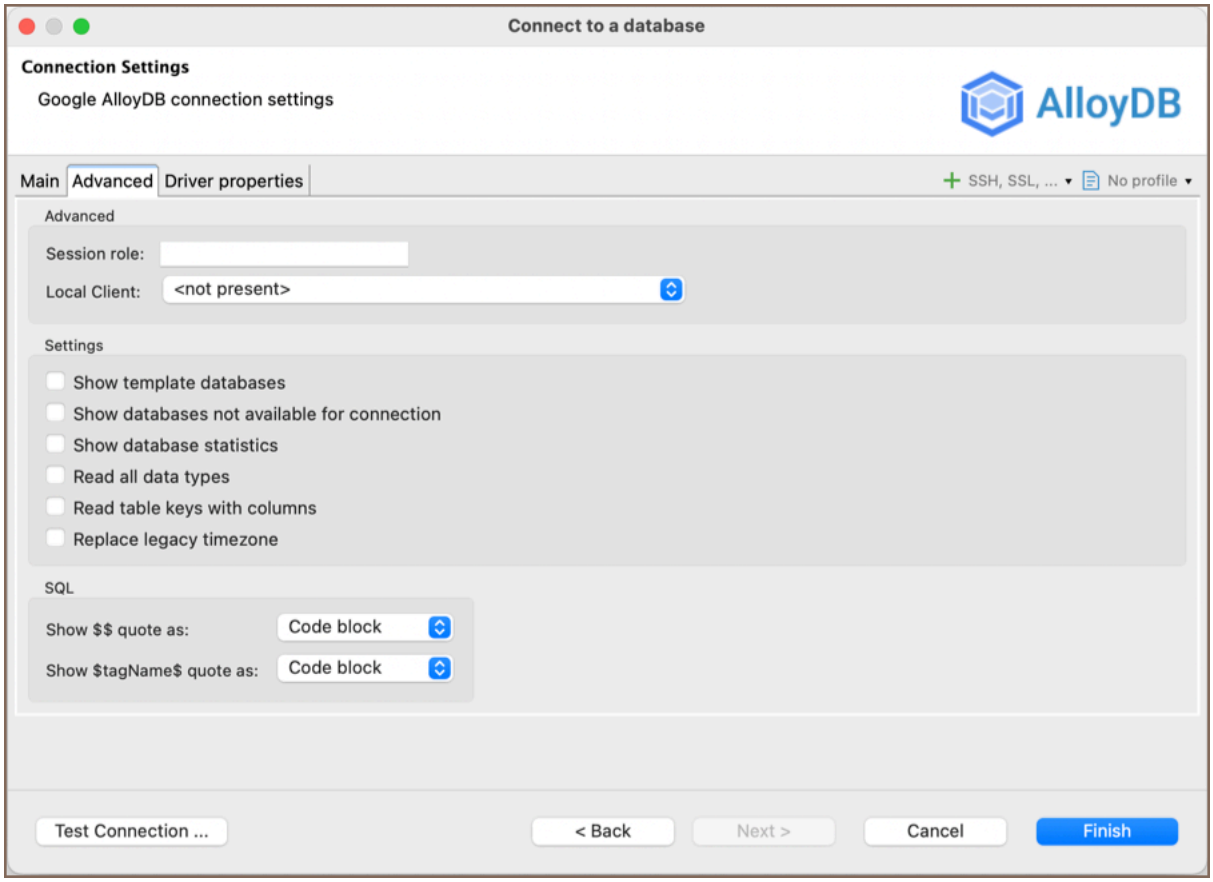
In this subsection, we will outline the settings for establishing a direct connection to an AlloyDB database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your AlloyDB database.

1. The **Main** tab of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your AlloyDB database.
URL	If you are connecting via URL, enter the URL of your AlloyDB database here. This field is hidden if you are connecting via the host.
Host	If you are connecting via host, enter the host address of your AlloyDB database here.
Database	Enter the name of the AlloyDB database you want to connect to.
Show all databases	Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - PostgreSQL Kerberos Authentication ★ - PostgreSQL PgPass Authentication - PostgreSQL SSPI Authentication (Windows SSO) ★ You can also read about security in DBeaver PRO.
Local Client	Specify the local client, see our article about local client configuration.
Session role	This option allows you to define a specific role that your database session assumes upon connection. Setting a session role can be useful for managing permissions and access control within your AlloyDB database.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The **Advanced** tab of the connection settings offers additional options that allow you to customize your further connection to the AlloyDB database.



Field	Description
Show template databases	Displays template databases. This option is available when **Show all databases** is checked.
Show databases unavailable for connection	Show databases not available for connection in database list. This option is available when **Show all databases** is checked.
Show databases statistics	Displays statistics for databases.
Read all data types	Reads all data types from the server (including arrays).
Read table keys with columns	Enable reading table constraints during the column reading stage to ensure accurate recognition of keys. Note that this setting may decrease metadata loading performance.
Replace legacy timezone	Replaces legacy timezone settings. Enable only if you need to connect with legacy timezones like **Europe/Kyiv** or **Asia/Calcutta**. Only these two timezones are affected.
Show `$$` quote as	Displays `$$` quotes as code blocks or strings.
Show `$tagName$` quote as	Displays `$tagName$` quotes as code blocks or strings.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your AlloyDB connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Driver properties

While managing AlloyDB with DBeaver, it's important to understand that the underlying driver is the standard PostgreSQL JDBC and ODBC driver.

The settings for AlloyDB **Driver properties** enable you to adjust the performance of the JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your AlloyDB database.

For a complete walkthrough on setting up JDBC and ODBC drivers properties, you can refer to the official JDBC documentation and ODBC documentation. These guides detail each driver's properties and how they can be used to optimize AlloyDB database connections.

You can customize the driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your AlloyDB database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

ODBC and JDBC driver configuration

DBeaver provides extensive capabilities for managing AlloyDB database connections via ODBC/JDBC drivers. This functionality enables you to connect to your AlloyDB database using native ODBC drivers, offering an alternative when specific DBeaver drivers are not available.

You can find a comprehensive, step-by-step guide on how to install the driver manager, set up drivers, configure data sources, and establish connections in DBeaver in our ODBC Driver Configuration article.

Secure connection configurations

DBeaver supports secure connections to your AlloyDB database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering AlloyDB with DBeaver

DBeaver provides a host of features designed for AlloyDB databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

AlloyDB database objects

DBeaver lets you view and manipulate a wide range of AlloyDB database objects. DBeaver has extensive support for various AlloyDB metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Data types
    - Tables
      - Columns
      - Constraints
      - Indexes
      - Foreign Keys
      - Dependencies
      - References
      - Partitions
      - Triggers
      - Rules
      - Policies
    - Foreign Tables
    - Views
    - Materialized Views
    - Functions
    - Sequences
    - Data types
    - Aggregate functions
  - Event Triggers
  - Extensions
  - Storage
    - Tablespaces
  - Roles
- Administer
  - Jobs
  - Session Manager
  - Lock Manager

AlloyDB additional features in DBeaver

DBeaver provides additional features compatible with AlloyDB, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Lock Management	Lock Manager

Category	Feature
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide
	Query Execution plan

BigQuery

Table of contents

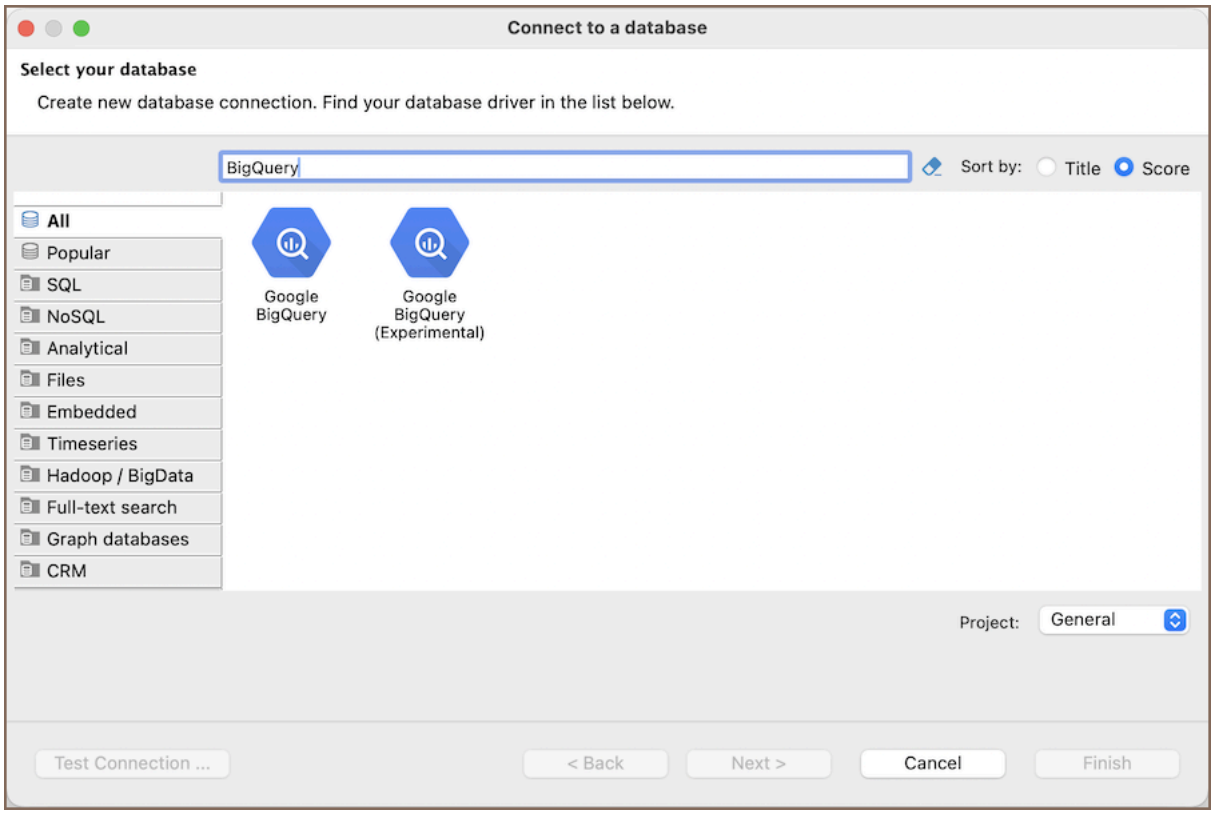
- BigQuery specialty
- Setting up
- BigQuery connection settings
- Connection details
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering BigQuery with DBeaver
- BigQuery database objects
- BigQuery additional features in DBeaver

This documentation details the steps for configuring and utilizing BigQuery with DBeaver. The integration in DBeaver supports distinctive BigQuery functionalities, accommodating its serverless infrastructure, robust scalability, and compatibility with multi-cloud data analytics.

Before you can start managing your database, it's essential to establish a connection in DBeaver. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.

DBeaver supports two BigQuery drivers. Select the appropriate one in the **Connect to a database** window:

- **BigQuery** - the official Google JDBC driver for BigQuery and the default choice for new connections.
- **BigQuery (Legacy)** - the previous Simba-based driver. Use it when you need compatibility with an existing connection.



BigQuery specialty

BigQuery is a fully managed, serverless data warehouse built for large-scale analytics. It's optimized for fast SQL queries across massive datasets and supports both structured and semi-structured data. BigQuery separates storage and compute, and uses a distributed architecture to execute queries efficiently. Data is stored in a columnar format, and the system can auto-scale to handle multiple workloads in parallel. BigQuery also supports advanced data types like `GEOGRAPHY` for spatial analysis, `BIGNUMERIC` for high-precision calculations, and `INTERVAL` for time-based logic. You can customize schemas using views and automate workflows using stored procedures. For more on BigQuery features, see the BigQuery docs overview.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-

> request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

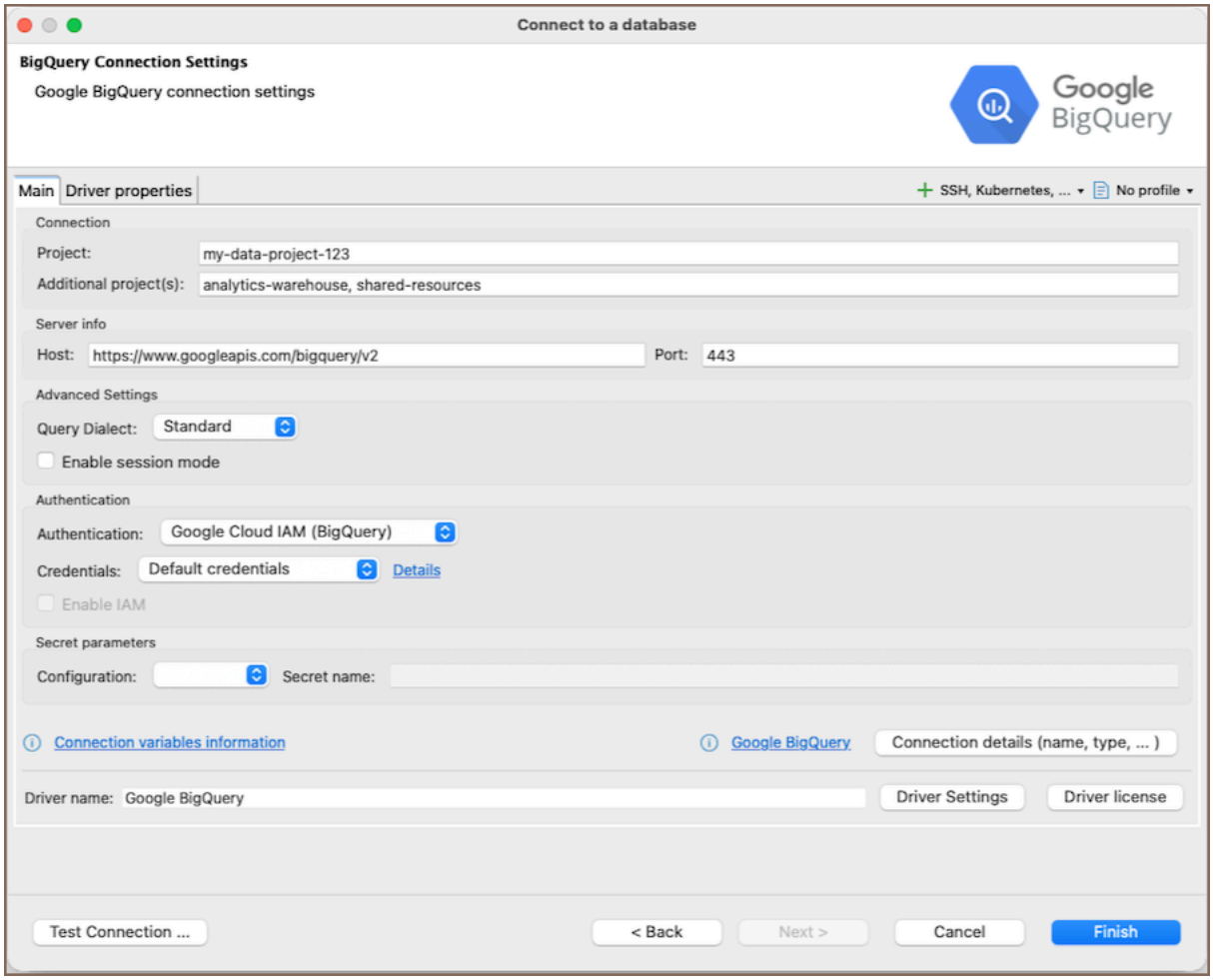
Setting up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, Proxies, and Kubernetes.

BigQuery connection settings

In this subsection, we will outline the settings for establishing a direct connection to a BigQuery database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your BigQuery database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Project	Enter the project ID for the Google Cloud Project where the BigQuery service is located.
Additional project(s)	Specify any additional project IDs if applicable.
Host	Set the host URL to the BigQuery API endpoint.
Port	Enter the port number for your BigQuery database. The default BigQuery port is 443.
Query dialect	Choose the SQL dialect used for query execution: **Standard** (recommended) or **Legacy** BigQuery SQL.
Enable session mode	Enable this option to maintain state across queries. This is useful for temporary tables or variables. Requires a valid location to be set for the connection to support session mode.
Location	Choose a regional location where the session should run (appears when **Enable session mode** is checked).
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Default credentials - Key file - Web browser / SSO - OAuth (Pregenerated) - DBeaver profile authentication You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.

Field	Description
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your BigQuery connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure connection configurations

DBeaver supports secure connections to your BigQuery database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering BigQuery with DBeaver

DBeaver provides a host of features designed for BigQuery databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

BigQuery database objects

DBeaver lets you view and manipulate a range of BigQuery database objects, such as:

- Schemas
  - Tables
    - Columns
    - Keys
    - Foreign keys
    - Indexes
    - References
  - Views
  - Procedures
  - ML Models

BigQuery additional features in DBeaver

DBeaver provides additional features compatible with BigQuery, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Schema Management	Schema Compare
	ERD Guide
Data Generation	Mock Data Generation
Query insights	Result details
	Disable metadata queries
Dashboards	Bytes Processed for Project

Bigtable

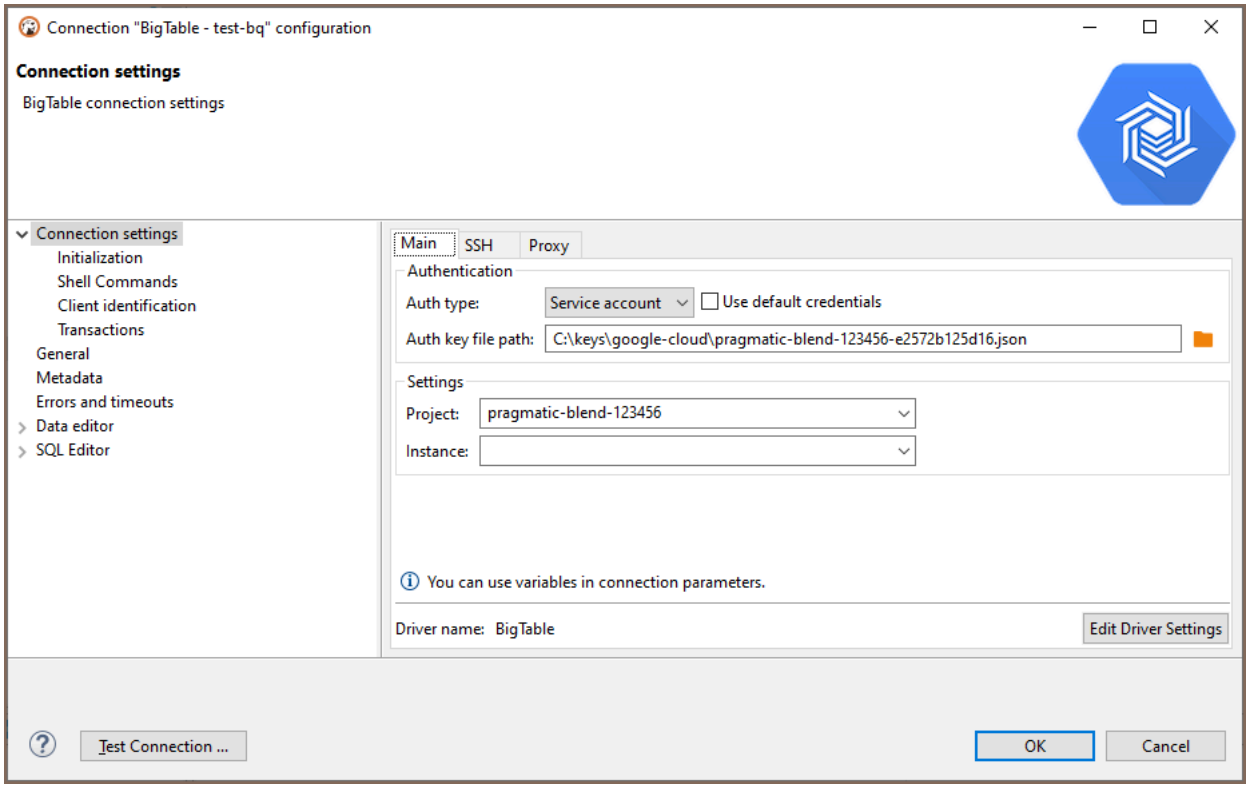
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Bigtable connection

Supported features: - Table data view - Table data edit in document (json) mode - Data filters - SQL queries execution - Data export and import

Bigtable connection



Cloud SQL for MySQL

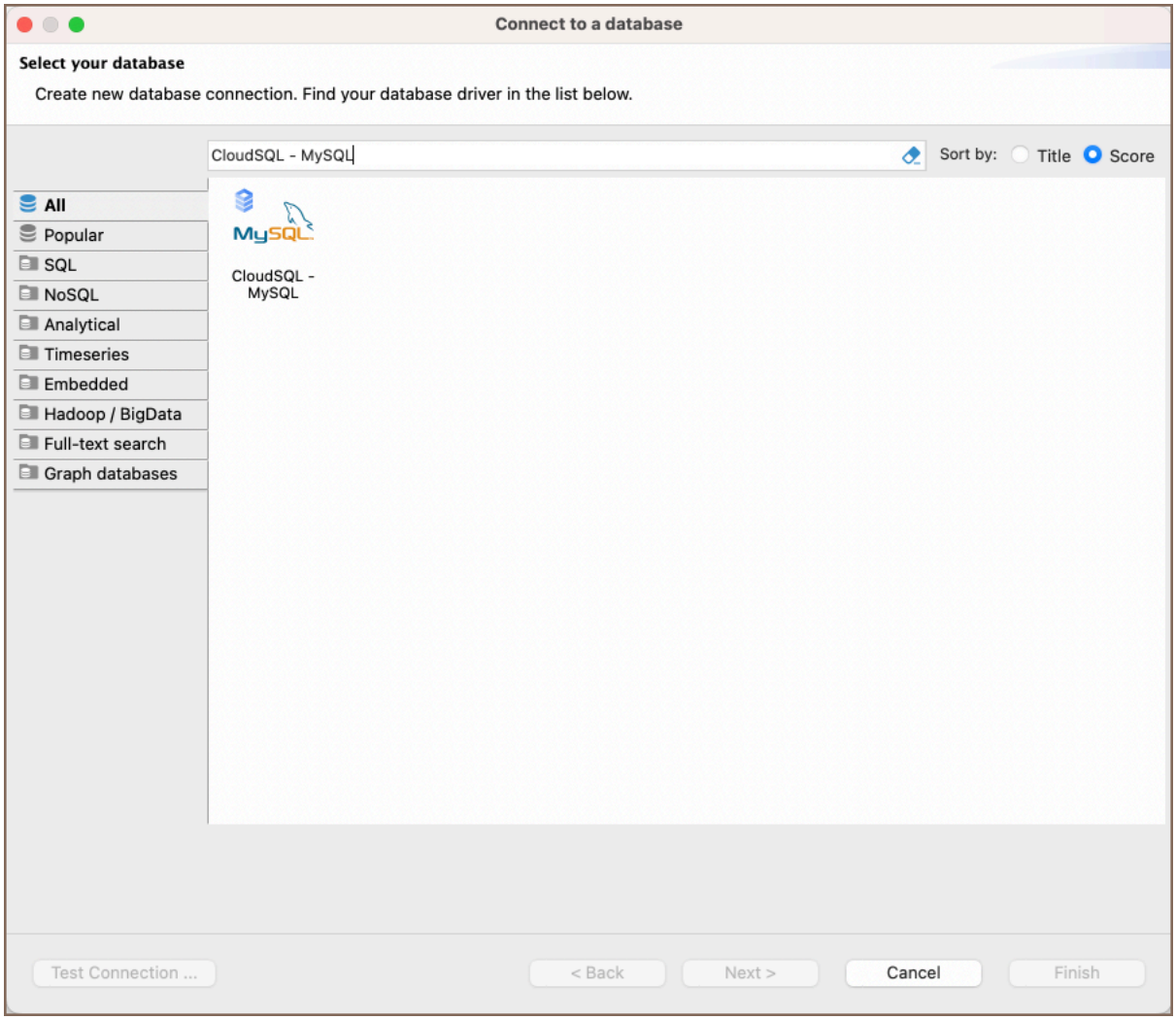
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Feature highlights
- Setting Up
- Google Cloud MySQL connection settings
  - Connection details
- Driver properties
- ODBC and JDBC Driver Configuration
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Google Cloud MySQL with DBeaver
- Google Cloud MySQL database objects
- MySQL additional features in DBeaver

Google Cloud SQL for MySQL is a managed database service provided by Google Cloud, offering the robust capabilities of MySQL with the added benefits of cloud hosting. This guide is designed to assist in the setup and management of Google Cloud SQL for MySQL databases using DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `CloudSQL - MySQL` option to connect to the Google Cloud SQL service. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Feature highlights

DBeaver extends beyond standard SQL tasks, offering a suite of MySQL-specific features. These capabilities enhance the functionality and management of Google Cloud MySQL databases, catering to specific needs and advanced use cases.

- **Data Types**: DBeaver fully supports MySQL-specific data types such as Enumerations and Sets. This enables precise data structuring and validation directly within the interface.
- **Security Management**: An integral part of managing databases is user security. DBeaver includes comprehensive tools for MySQL User Management, allowing for detailed control over user permissions and roles.
- **Storage Engines**: Understanding and choosing the right storage engine is crucial for database performance. DBeaver supports various MySQL storage engines, including InnoDB, MyISAM, and specialized engines like MRG_MyISAM, Archive, Blackhole, CSV, Memory, and Performance_schema.
- **Database Management Tools**: DBeaver provides advanced database management features specific to MySQL. This includes the management of MySQL Events and Stored Procedures, allowing for automation and efficient management of routine database tasks.

Setting Up

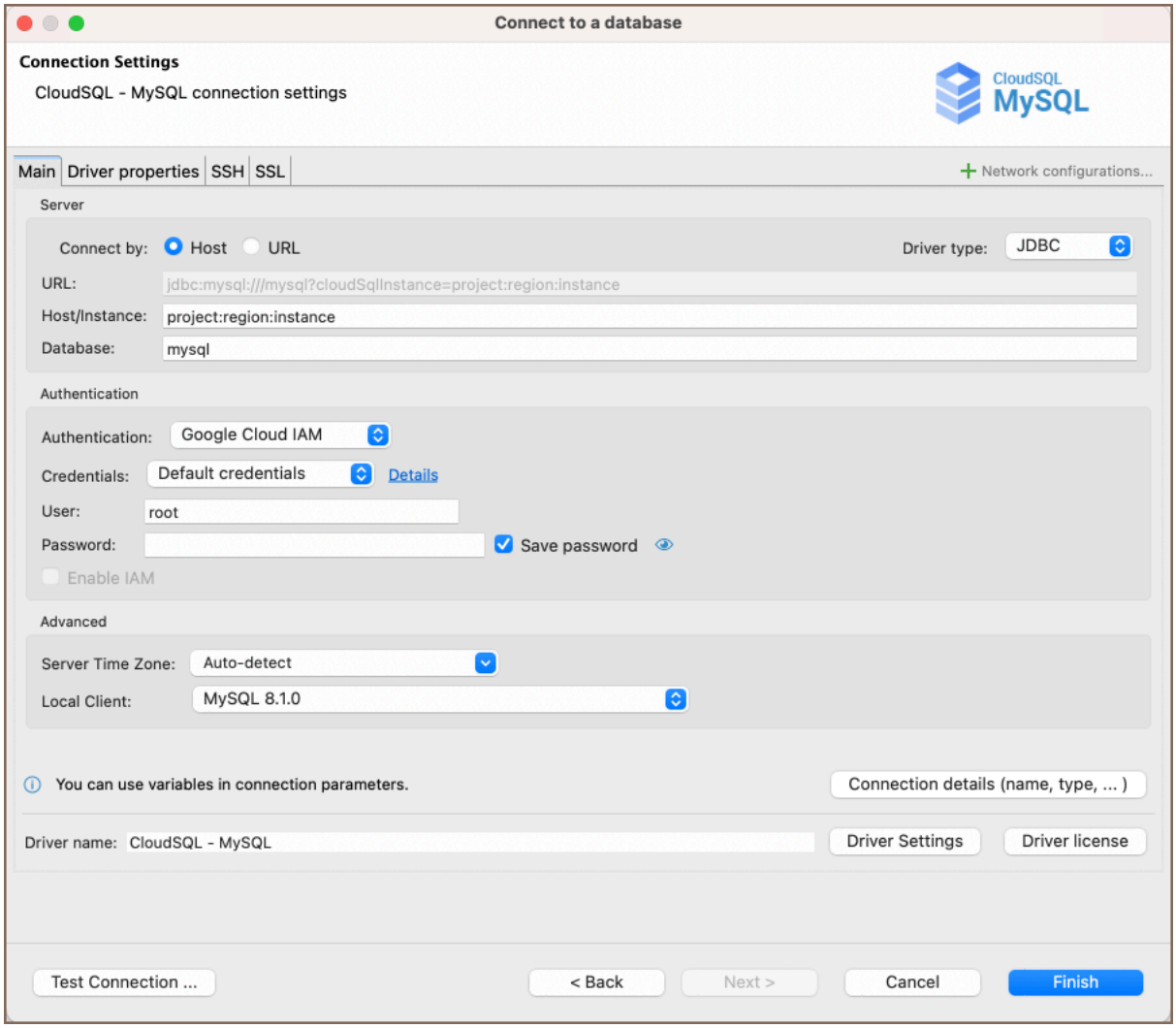
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for Google Cloud MySQL.

> **Note**: The connection process in DBeaver utilizes the standard MySQL driver, tailored to interact with the Google Cloud SQL for MySQL service, highlighting its cloud-based characteristics.

Google Cloud MySQL connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Google Cloud MySQL database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Google Cloud MySQL database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Google Cloud MySQL database.
URL	If you are connecting via URL, enter the URL of your Google Cloud MySQL database here. This field is disabled if you're connecting via the host.
Host/Instance	For a direct connection to a Google Cloud SQL instance, use the format `project:region:instance`. Here, `project` refers to your Google Cloud project ID, `region` denotes the geographical location of your instance, and `instance` is the unique ID of your Google Cloud SQL instance.
Database	Enter the name of the Google Cloud MySQL database you want to connect to.
Show all databases	Displays all databases in the Connections. If not enabled, only the database from the connection is shown.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Google Cloud IAM - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Local Client	Specify the local client, see our article about local client configuration.
Server Time Zone	This setting is responsible for determining the time zone used by the Google Cloud MySQL server.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Google Cloud MySQL database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Driver properties

While managing MySQL databases hosted on Google Cloud with DBeaver, it's important to understand that the underlying driver is the standard MySQL JDBC and ODBC driver.

The settings for Google Cloud MySQL **Driver properties** enable you to adjust the performance of the MySQL JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Google Cloud MySQL database.

For a complete walkthrough on setting up MySQL JDBC and ODBC drivers properties, you can refer to the official MySQL JDBC documentation and MySQL ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Google Cloud MySQL database connections.

You can customize the MySQL driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Google Cloud MySQL database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

ODBC and JDBC Driver Configuration

DBeaver provides extensive capabilities for managing Google Cloud MySQL database connections via ODBC/JDBC drivers. This functionality enables you to connect to your Google Cloud MySQL database using native ODBC drivers, offering an alternative when specific DBeaver drivers are not available.

This process is similar across most databases that support ODBC/JDBC drivers. For a comprehensive, step-by-step guide on how to install the driver manager, set up drivers, configure data sources, and establish connections in DBeaver, you can refer to our ODBC Driver Configuration article.

Secure Connection Configurations

DBeaver supports secure connections to your Google Cloud MySQL database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Google Cloud MySQL with DBeaver

DBeaver offers an array of features specifically tailored for MySQL databases, including those hosted on Google Cloud. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Google Cloud MySQL database objects

DBeaver lets you view and manipulate a wide range of database objects. DBeaver has extensive support for various MySQL metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Tables
    - Columns
    - Constraints
    - Indexes
    - Foreign Keys
    - References
    - Triggers
  - Views
  - Indexes
  - Procedures
  - Triggers
  - Events
- Users
- Administer
  - Session Manager

MySQL additional features in DBeaver

DBeaver provides additional features compatible with Google Cloud MySQL, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide
	Query Execution plan

Cloud SQL for PostgreSQL

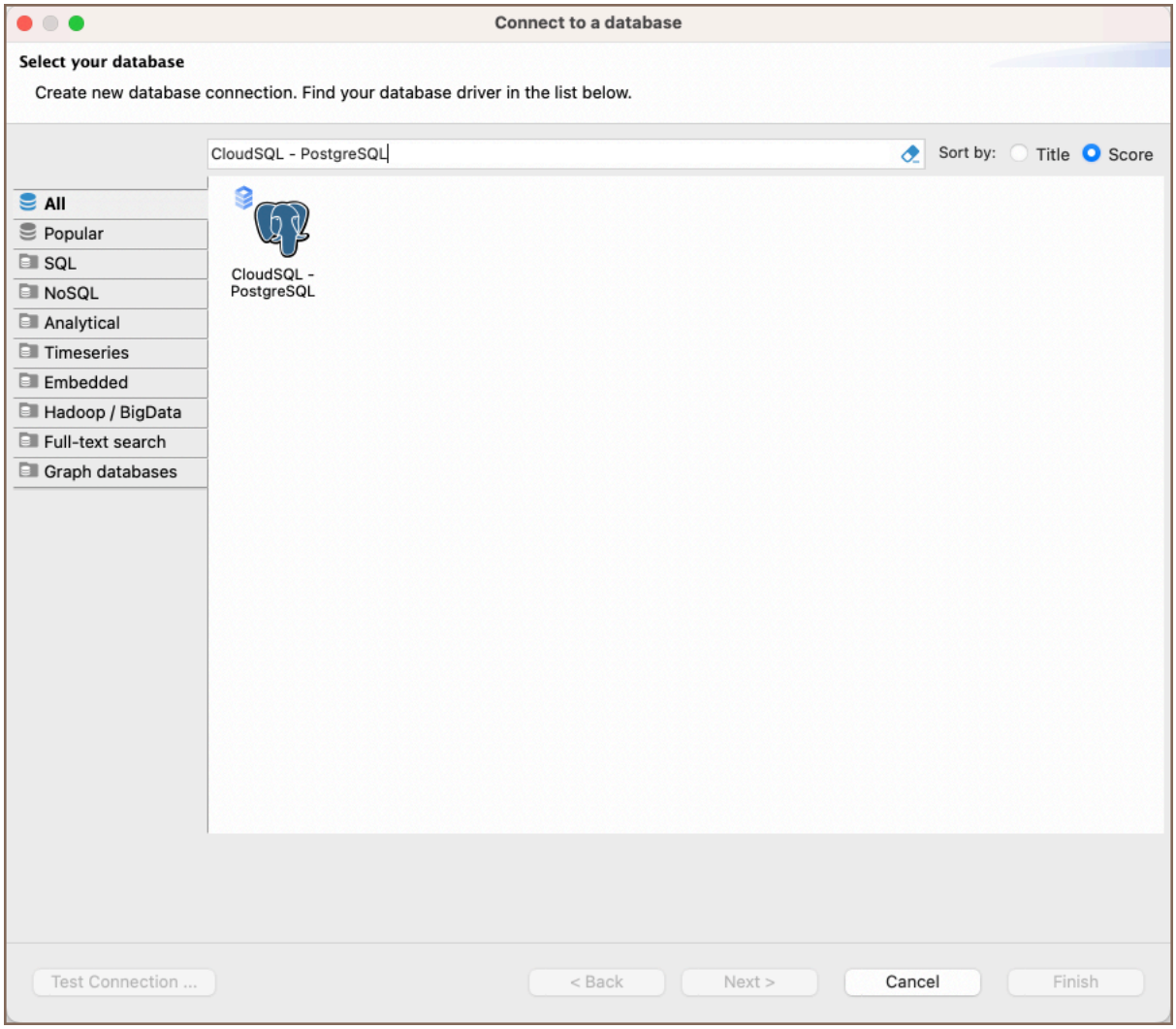
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Feature Highlights
- Setting Up
- PostgreSQL connection settings
- Connection details
- Driver properties
- ODBC and JDBC driver configuration
- Secure connection configurations
- Secure Storage with Secret Providers
- Powering Google Cloud PostgreSQL with DBeaver
- Google Cloud PostgreSQL database objects
- Google Cloud PostgreSQL additional features in DBeaver

Google Cloud SQL for PostgreSQL is a managed database service provided by Google Cloud, offering the robust capabilities of Google Cloud PostgreSQL with the added benefits of cloud hosting. This guide is designed to assist in the setup and management of Google Cloud SQL for PostgreSQL databases using DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `CloudSQL - PostgreSQL` option to connect to the Google Cloud SQL service. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Feature Highlights

DBeaver extends beyond standard SQL tasks, offering a suite of PostgreSQL-specific features. These capabilities enhance the functionality and management of Google Cloud PostgreSQL databases, catering to specific needs and advanced use cases.

- **Data Types**: DBeaver supports complex data types unique to this system. This includes Arrays and various structures, enabling intricate data organization and manipulation.
- **Extensions and Security**: DBeaver includes robust support for Extensions, broadening the database's functionality. It also provides comprehensive security features, encompassing Permissions, Policies, and Roles management, ensuring thorough control over database access and operations.
- **Data Organization**: For effective data handling, DBeaver supports Partitions. This feature assists in efficient data segmentation and organization, enhancing database performance and management.
- **External Data Integration**: The platform also facilitates the integration of Foreign Tables. This feature enables seamless access and manipulation of external data sources, providing a more flexible and expansive data management environment.

Setting Up

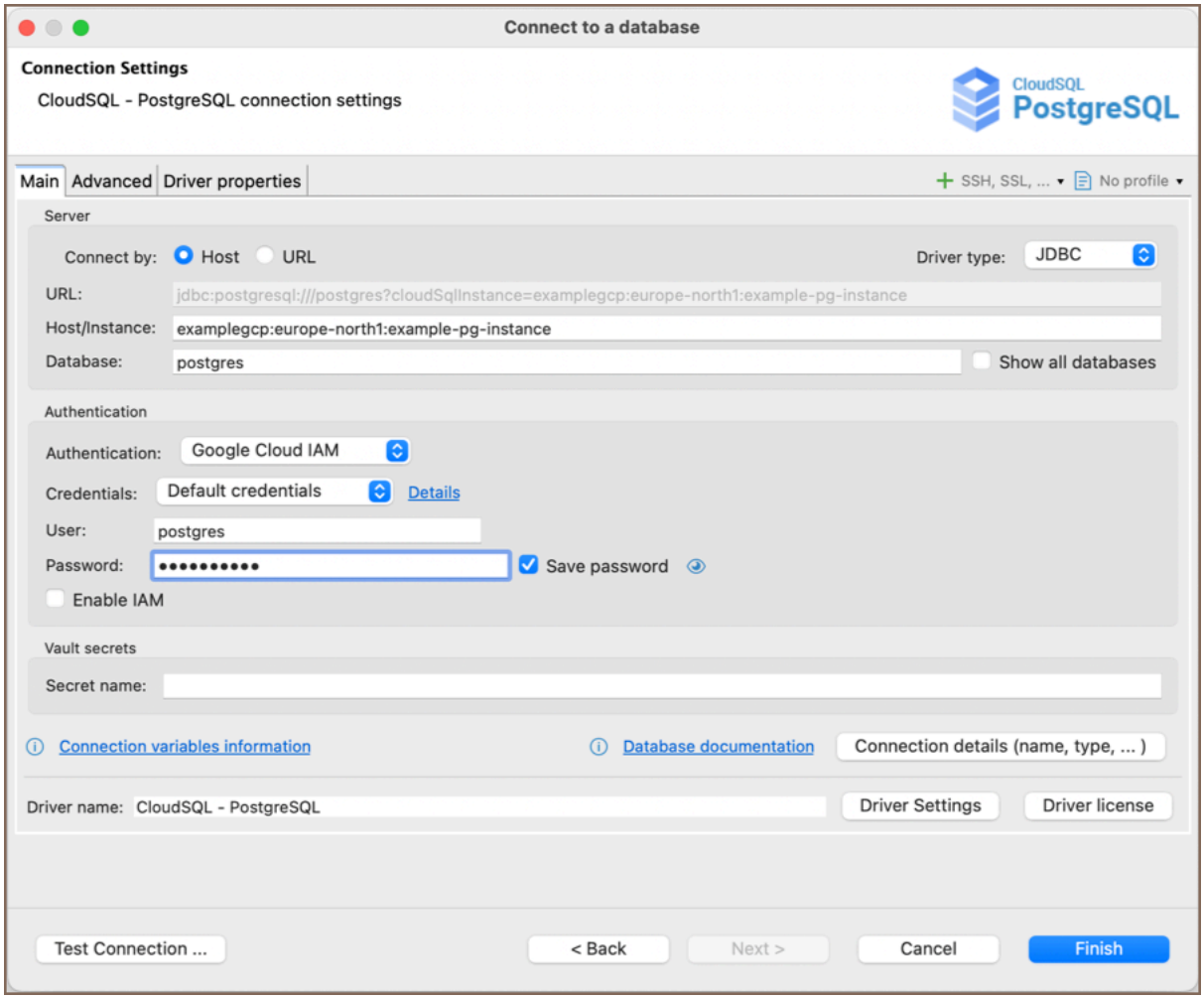
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for Google Cloud PostgreSQL.

> **Note**: The connection process in DBeaver utilizes the standard PostgreSQL driver, tailored to interact with the Google Cloud SQL for PostgreSQL service, highlighting its cloud-based characteristics.

PostgreSQL connection settings

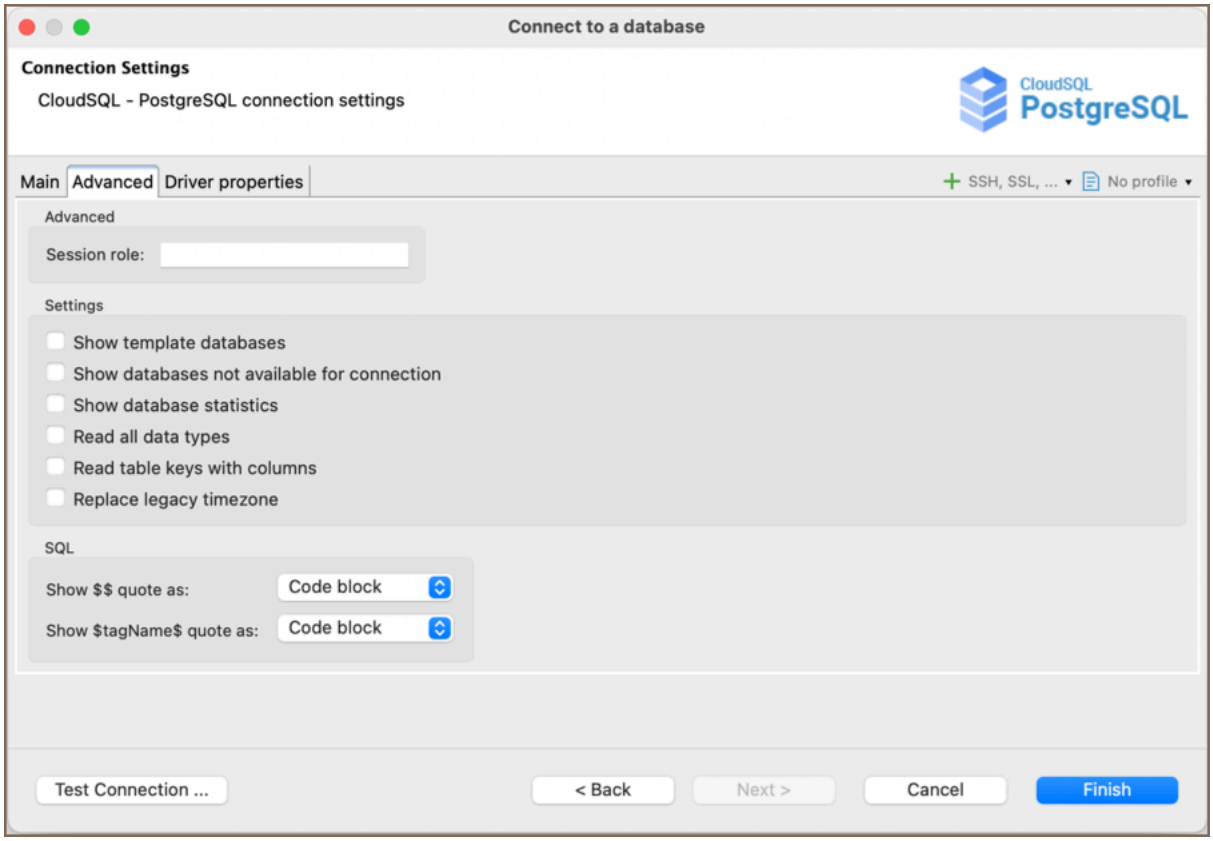
In this subsection, we will outline the settings for establishing a direct connection to a Google Cloud PostgreSQL database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Google Cloud PostgreSQL database.

1. The first page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Google Cloud PostgreSQL database.
URL	If you are connecting via URL, enter the URL of your Google Cloud PostgreSQL database here. This field is hidden if you are connecting via the host.
Host/Instance	For a direct connection to a Google Cloud SQL instance, use the format `project:region:instance`. Here, `project` refers to your Google Cloud project ID, `region` denotes the geographical location of your instance, and `instance` is the unique ID of your Google Cloud SQL instance.
Database	Enter the name of the Google Cloud PostgreSQL database you want to connect to.
Show all databases	Displays all databases in the Connections. **Note**: This option works only when the **Host** connection type is selected. It does not work when connecting via **URL**.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Google Cloud IAM - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Local Client	Specify the local client, see our article about local client configuration.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

2. The second page of the connection settings offers additional options that allow you to customize your further connection to the Google Cloud PostgreSQL database.



Field	Description
Show template databases	Displays template databases. This option is available when **Show all databases** is checked.
Show databases unavailable for connection	Displays databases unavailable. This option is available when **Show all databases** is checked.
Show databases statistics	Displays statistics for databases.
Read all data types	Reads all data types from the server.
Read table keys with columns	Reads table keys with columns.
Replace legacy timezone	Replaces legacy timezone settings. Enable only if you need to connect with legacy timezones like **Europe/Kyiv** or **Asia/Calcutta**. Only these two timezones are affected.
Show `$$` quote as	Displays `$$` quotes as code blocks or strings.
Show `$tagName$` quote as	Displays `$tagName$` quotes as code blocks or strings.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Google Cloud PostgreSQL connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Driver properties

While managing PostgreSQL databases hosted on Google Cloud with DBeaver, it's important to understand that the underlying driver is the standard PostgreSQL JDBC and ODBC driver.

The settings for Google Cloud PostgreSQL **Driver properties** enable you to adjust the performance of the PostgreSQL JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Google Cloud PostgreSQL database.

For a complete walkthrough on setting up PostgreSQL JDBC and ODBC drivers properties, you can refer to the official PostgreSQL JDBC documentation and PostgreSQL ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Google Cloud PostgreSQL database connections.

You can customize the PostgreSQL driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Google Cloud PostgreSQL database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

ODBC and JDBC driver configuration

DBeaver provides extensive capabilities for managing Google Cloud PostgreSQL database connections via ODBC/JDBC drivers. This functionality enables you to connect to your Google Cloud PostgreSQL database using native ODBC drivers, offering an alternative when specific DBeaver drivers are not available.

You can find a comprehensive, step-by-step guide on how to install the driver manager, set up drivers, configure data sources, and establish connections in DBeaver in our ODBC Driver Configuration article.

Secure connection configurations

DBeaver supports secure connections to your Google Cloud PostgreSQL database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Google Cloud PostgreSQL with DBeaver

DBeaver provides a host of features designed for Google Cloud PostgreSQL databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Google Cloud PostgreSQL database objects

DBeaver lets you view and manipulate a wide range of Google Cloud PostgreSQL database objects. DBeaver has extensive support for various PostgreSQL metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Schemas
    - Data types
    - Tables
      - Columns
      - Constraints
      - Indexes
      - Foreign Keys
      - Dependencies
      - References
      - Partitions
      - Triggers
      - Rules
      - Policies
    - Foreign Tables
    - Views
    - Materialized Views
    - Functions
    - Sequences
    - Data types
    - Aggregate functions
  - Event Triggers
  - Extensions
  - Storage
    - Tablespaces
  - Roles
- Administer
  - Jobs
  - Session Manager
  - Lock Manager

Google Cloud PostgreSQL additional features in DBeaver

DBeaver provides additional features compatible with Google Cloud PostgreSQL, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Lock Management	Lock Manager
Backup and Restore	How to Backup/Restore data
Script execution	Execute script with local client

Category	Feature
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide
	Query Execution plan

Cloud SQL for SQL Server

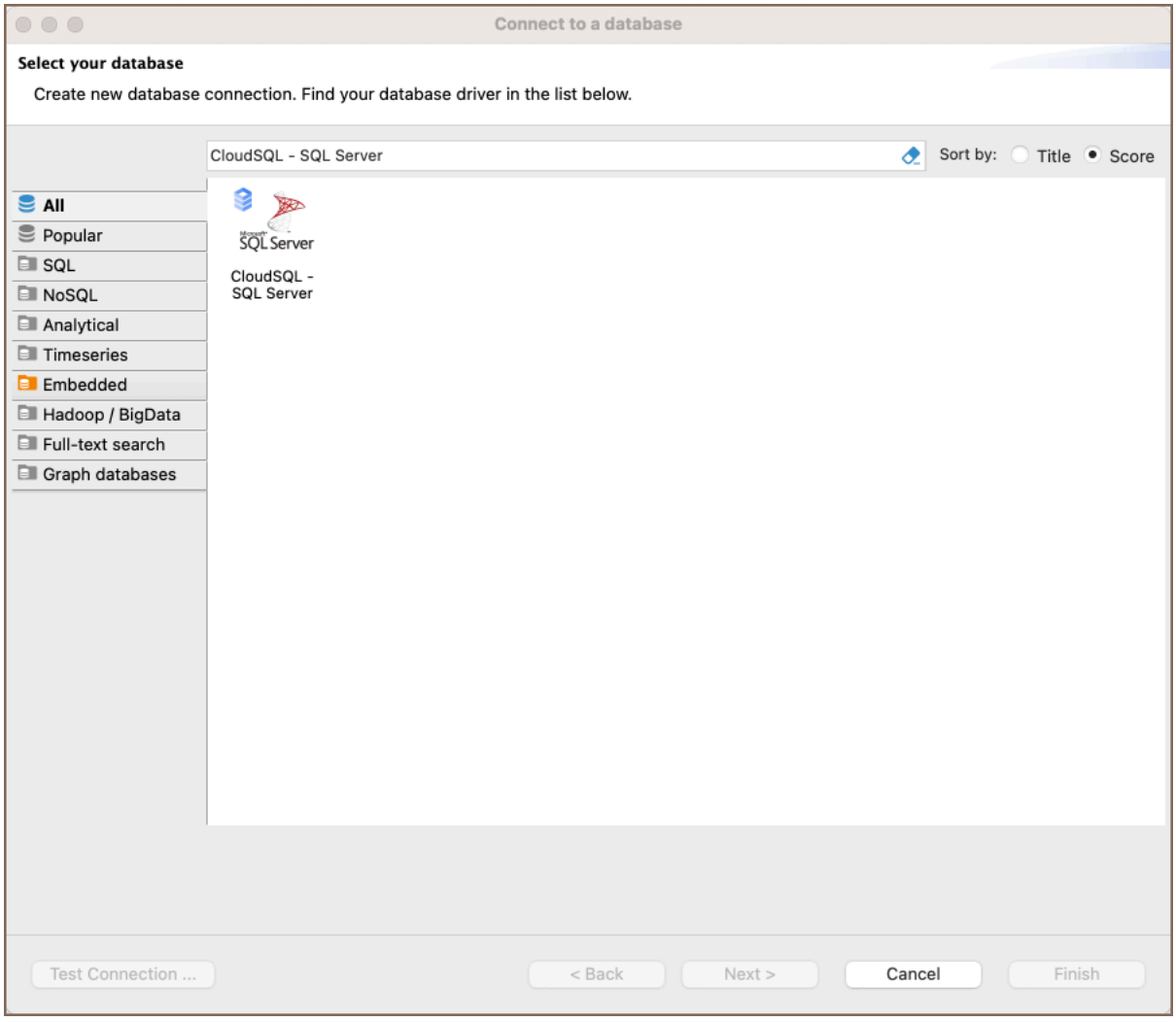
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Feature Highlights
- Setting Up
- Google Cloud SQL Server connection settings
- Connection details
- Driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Google Cloud SQL Server with DBeaver
- Google Cloud SQL Server database objects
- Google Cloud SQL Server additional features in DBeaver

Google Cloud SQL for SQL Server is a managed database service provided by Google Cloud, offering the robust capabilities of Google Cloud SQL Server with the added benefits of cloud hosting. This guide is designed to assist in the setup and management of Google Cloud SQL for SQL Server databases using DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `CloudSQL - SQL Server` option to connect to the Google Cloud SQL service. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Feature Highlights

DBeaver extends beyond standard SQL tasks, offering a suite of SQL Server-specific features. These capabilities enhance the functionality and management of Google Cloud SQL Server databases, catering to specific needs and advanced use cases.

- **Data Structure**: DBeaver provides extensive support for SQL Server's data structures, ensuring comprehensive management and interaction capabilities. This includes:
- **SQL Server Tables & Views**: Facilitating advanced data representation and querying capabilities.
- **SQL Server Procedures**: Supporting the creation, modification, and management of stored procedures for efficient data handling.
- **SQL Server Sequences**: Enabling the creation and management of sequences for generating unique numbers.
- **SQL Server Synonyms**: Offering the ability to use synonyms for database objects, simplifying SQL queries and enhancing data accessibility.
- **SQL Server External Tables**: Supporting the integration of external data sources, allowing for extended data management beyond traditional database boundaries.
- **Job Management**: DBeaver extends its functionalities to job management in SQL Server databases. This includes support for Agent Jobs, empowering users to schedule and automate various tasks, enhancing the efficiency of database operations and maintenance.

Setting Up

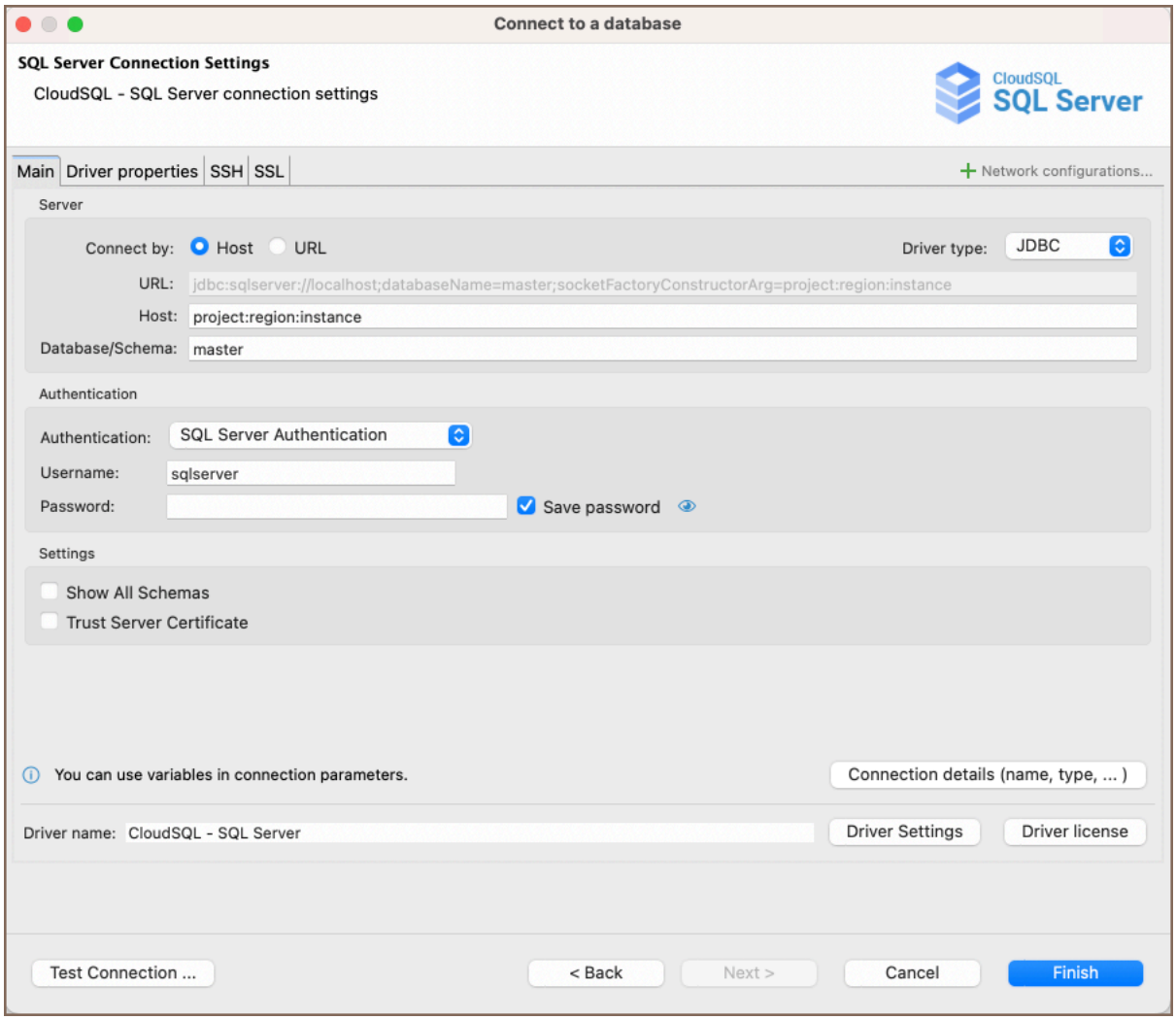
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, SSL, and the setup of ODBC/JDBC drivers for Google Cloud SQL Server.

> **Note**: The connection process in DBeaver utilizes the standard SQL Server driver, tailored to interact with the Google Cloud SQL for SQL Server service, highlighting its cloud-based characteristics.

Google Cloud SQL Server connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Google Cloud SQL Server database using DBeaver.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your Google Cloud SQL Server database.
URL	If you are connecting via URL, enter the URL of your Google Cloud SQL Server database here. This field is hidden if you are connecting via the host.

Field	Description
Host/Instance	For a direct connection to a Google Cloud SQL instance, use the format `project:region:instance`. Here, `project` refers to your Google Cloud project ID, `region` denotes the geographical location of your instance, and `instance` is the unique ID of your Google Cloud SQL instance.
Database/Schema	Enter the name of the Google Cloud SQL Server database you want to connect to.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Google Cloud IAM - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Show All Schemas	Shows all database schemas. If disabled shows only non-empty schemas.
Trust Server Certificate	If `true`, the Google Cloud SQL Server SSL certificate is automatically trusted when the communication layer is encrypted using SSL. If `false`, the Microsoft JDBC Driver for SQL Server validates the server SSL certificate. If the server certificate validation fails, the driver raises an error and terminates the connection. To configure certificate settings in DBeaver, refer to our detailed article on Truststore.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Google Cloud SQL Server database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Driver properties

While managing Google Cloud SQL Server databases hosted on Google Cloud with DBeaver, it's important to understand that the underlying driver is the standard SQL Server JDBC and ODBC driver.

The settings for Google Cloud SQL Server **Driver properties** enable you to adjust the performance of the SQL Server JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your Google Cloud SQL Server database.

For a complete walkthrough on setting up SQL Server JDBC and ODBC drivers properties, you can refer to the official SQL Server JDBC documentation and SQL Server ODBC documentation. These guides detail each driver's properties and how they can be used to optimize Google Cloud SQL Server database connections.

You can customize the SQL Server driver in DBeaver via the **Edit Driver** window, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This window offers a range of settings that can influence your Google Cloud SQL Server database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Google Cloud SQL Server database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, **AWS SSM** and **SSL** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **SSL Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Google Cloud SQL Server with DBeaver

DBeaver provides a host of features designed for Google Cloud SQL Server databases. This includes the ability to view schemas, along with numerous unique capabilities aimed at optimizing database operations.

Google Cloud SQL Server database objects

DBeaver lets you view and manipulate a wide range of Google Cloud SQL Server database objects. DBeaver has extensive support for various SQL Server metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases

        - Schemas
            - Tables
            - External Tables
            - Views
                - Columns
            - Triggers
            - External Properties
            - Indexes
            - Procedures
            - Sequences
            - Synonyms
            - Triggers
            - Data Types
        - Database triggers
- Security
    - Logins
- Administer
    - Sessions
- Agent
    - Jobs

Google Cloud SQL Server additional features in DBeaver

DBeaver provides additional features compatible with Google Cloud SQL Server, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Session Management	Session Manager
Schema Management	Schema Compare
Data Visualization	GIS Guide
	ERD Guide
	Query Execution plan

Firestore

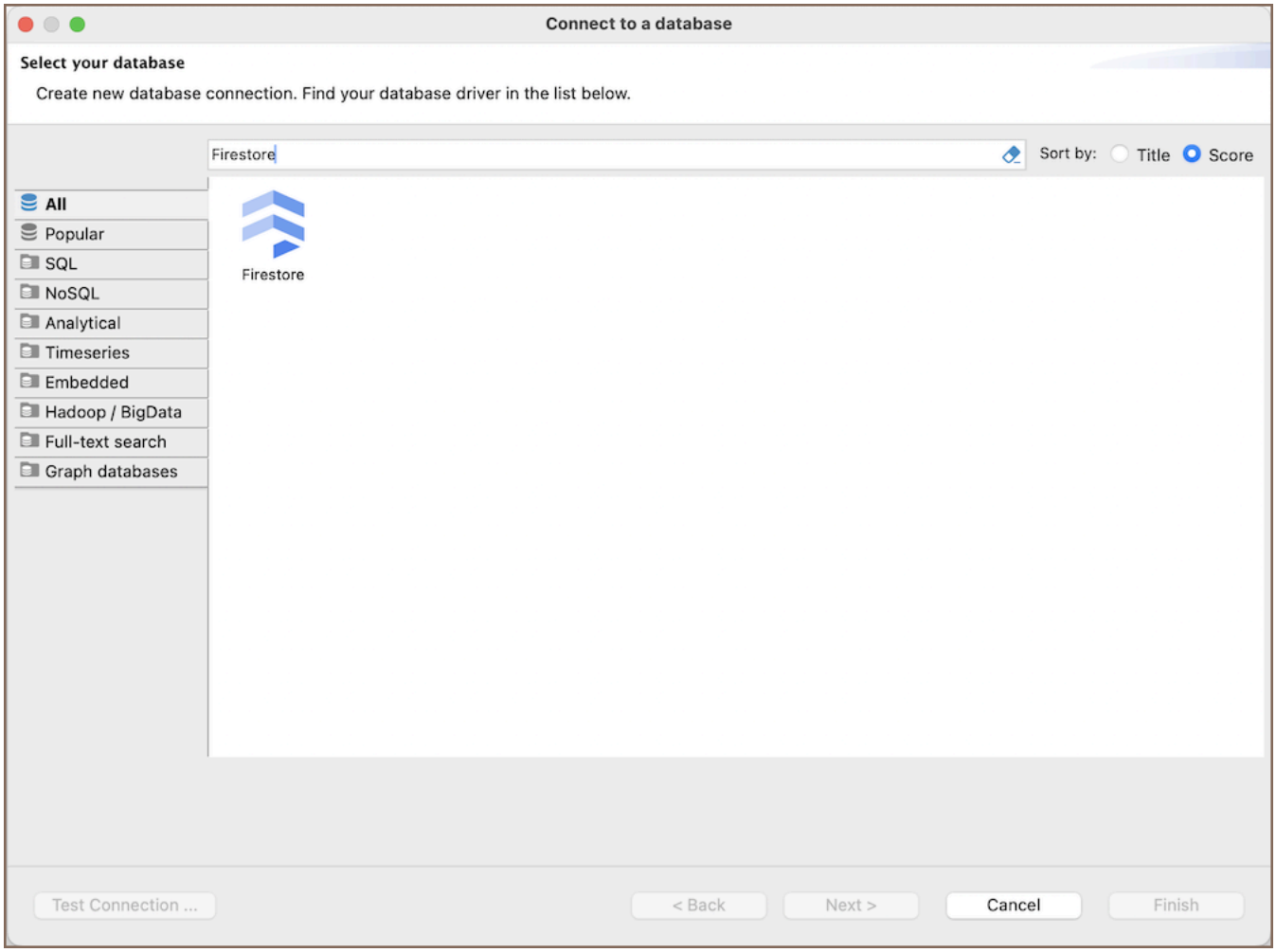
> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Firestore specialty
- Setting Up
- Firestore connection settings
- Connection details
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Firestore with DBeaver
- Working with Collections and Subcollections
- Creating a Subcollection
- Viewing Subcollections
- Firestore features in DBeaver

This guide provides instructions on how to set up and use Firestore with DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Firestore` option to connect to the Firestore database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Firestore specialty

Firestore is a flexible, scalable database for mobile, web, and server development from Firebase and Google Cloud Platform. Firestore is a NoSQL database that stores data in documents, which are organized into collections. Each document contains a set of key-value pairs and is identified by a unique identifier. Firestore's real-time data synchronization capabilities allow developers to build collaborative applications by enabling real-time updates to data across all connected clients. It supports offline data access, enabling applications to function seamlessly without an internet connection. Security and data validation are enforced through Firestore Security Rules, allowing developers to control access to documents and collections. Firestore is designed to scale automatically, handling large datasets with ease. It supports powerful query capabilities, allowing efficient data retrieval and filtering. For more details, refer to the official Firestore documentation.

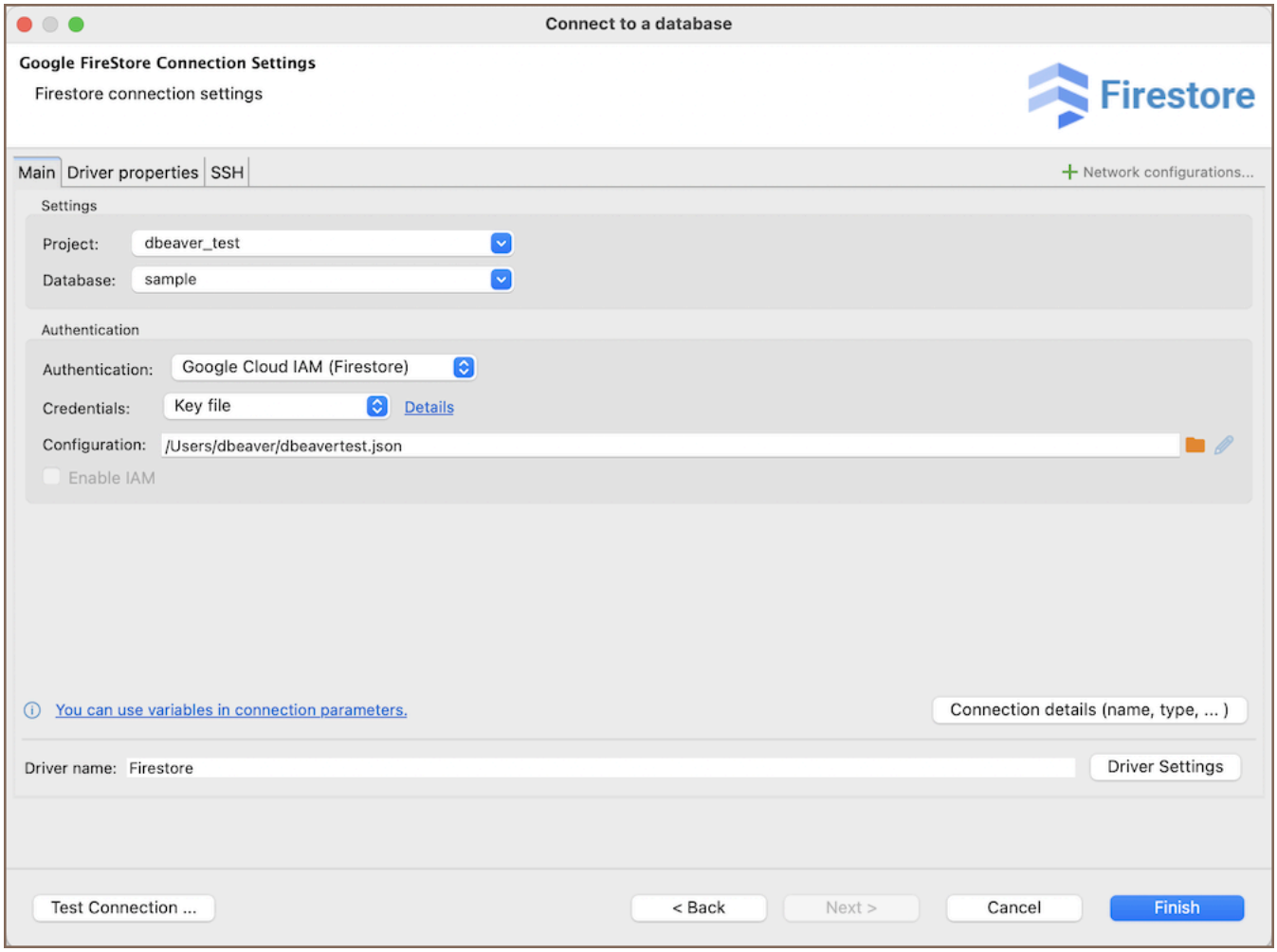
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, and Kubernetes.

Firestore connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Firestore database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Firestore database.

The connection settings page requires you to fill in specific fields to establish the initial connection.



Field	Description
Project	Choose the name of the Project you want to connect to.
Database	Enter the name of the Firestore database you want to connect to.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - DBeaver Profile Authentication - Google Cloud IAM (coming soon) You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Firestore connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure Connection Configurations

DBeaver supports secure connections to your Firestore database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**

- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Firestore with DBeaver

DBeaver provides support for Firestore, focusing on facilitating the management of collections and subcollections.

Working with Collections and Subcollections

Firestore organizes data into collections, which are containers for documents. Additionally, collections can contain subcollections, enabling a hierarchical structure for data organization.

The process for managing Firestore collections mirrors the familiar experience of creating and handling objects in the Connections.

Creating a Subcollection

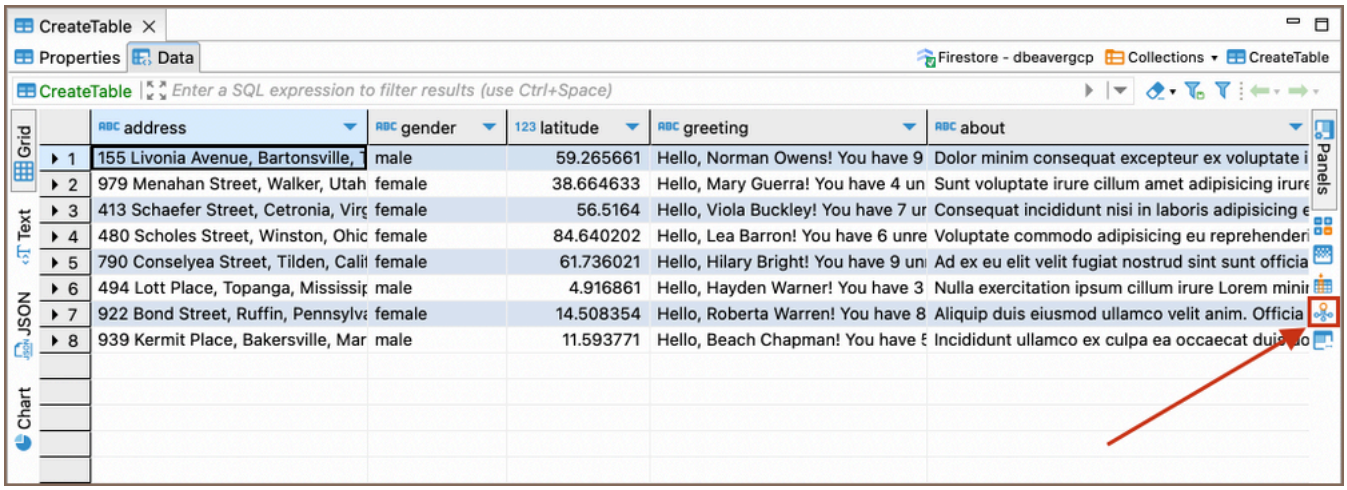
To work with Firestore subcollections, follow these instructions:

1. Open the collection in Data Editor and click the Subcollections button located in the right sidebar to access the management interface for Firestore's subcollections.
2. In the Subcollections window, you will find options to create new subcollections, delete the existing ones, or refresh the list to view the latest data.
3. To add a Subcollection, click the **Create collection** button . You will be prompted to enter a name for the new collection.
4. After setting up the subcollection's name, it is ready to be filled with data, structured according to your application's requirements.

Viewing Subcollections

To view subcollections:

1. Navigate to the collection in Data Editor.
2. Select the object (collection) for which you wish to view the subcollections.
3. Click the Subcollections button .



> **Tip**: To view data in JSON format, click the corresponding button in the left sidebar.

Firestore features in DBeaver

In addition to facilitating the management of collections and subcollections, DBeaver's integration with Firestore includes a range of features designed to enhance the user experience and efficiency when working with Firestore databases. Here are some of the key features:

Category	Feature
Data Transfer	Data Export

Spanner

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

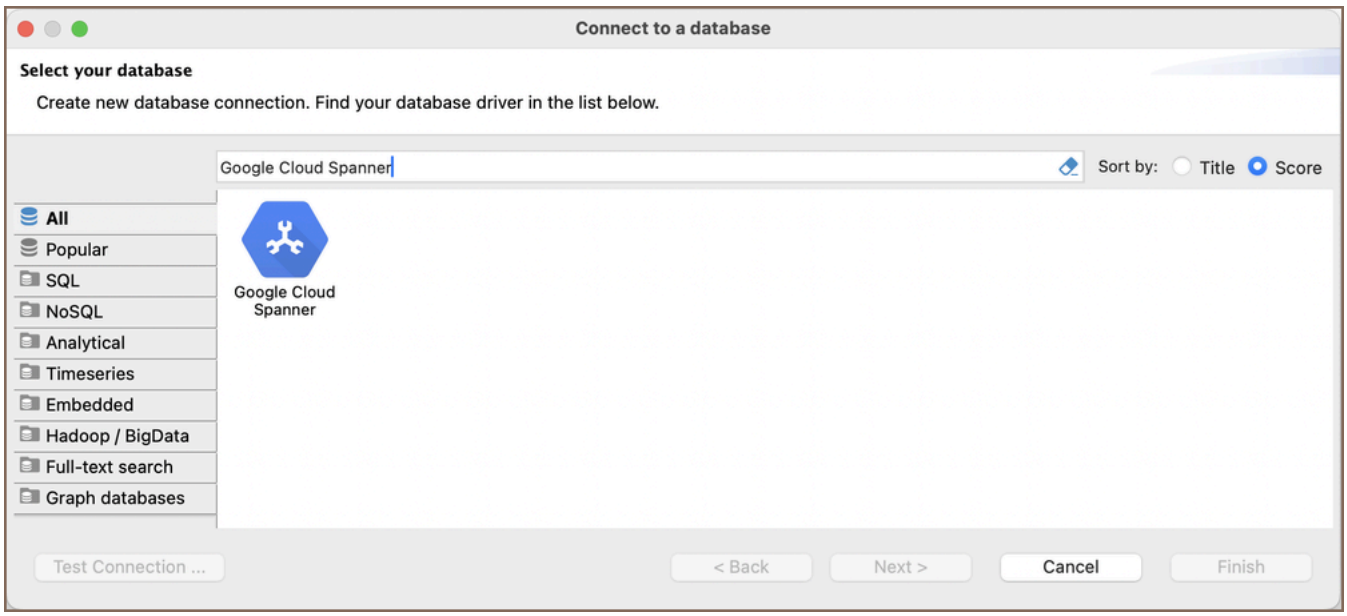
Table of contents

- Google Spanner specialty
- Setting Up
- Google Spanner connection settings
- Connection details

- Google Spanner driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Google Spanner with DBeaver
- Google Spanner database objects
- Google Spanner additional features in DBeaver

This guide provides step-by-step instructions on how to set up and manage Google Cloud Spanner databases using DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Google Cloud Spanner` option to connect to your database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Google Spanner specialty

Google Spanner is a distributed database system that combines features from both relational and NoSQL databases. It stands out for its large-scale capabilities, designed to manage extensive data across numerous nodes while maintaining high availability and global consistency.

In Google Spanner, the metadata types resemble those in traditional relational databases, with tables, columns, schemas, and key constraints, including primary keys, foreign keys, and unique constraints. A significant aspect of Google Spanner is its synchronization mechanism for nodes. It uses atomic clocks and GPS receivers to power its TrueTime system, ensuring precise synchronization across all nodes. This advanced timekeeping infrastructure is crucial for maintaining external consistency in distributed transactions and offers a robust solution for strong consistency on a global scale. For a more comprehensive understanding of Google Spanner, please refer to the official documentation.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

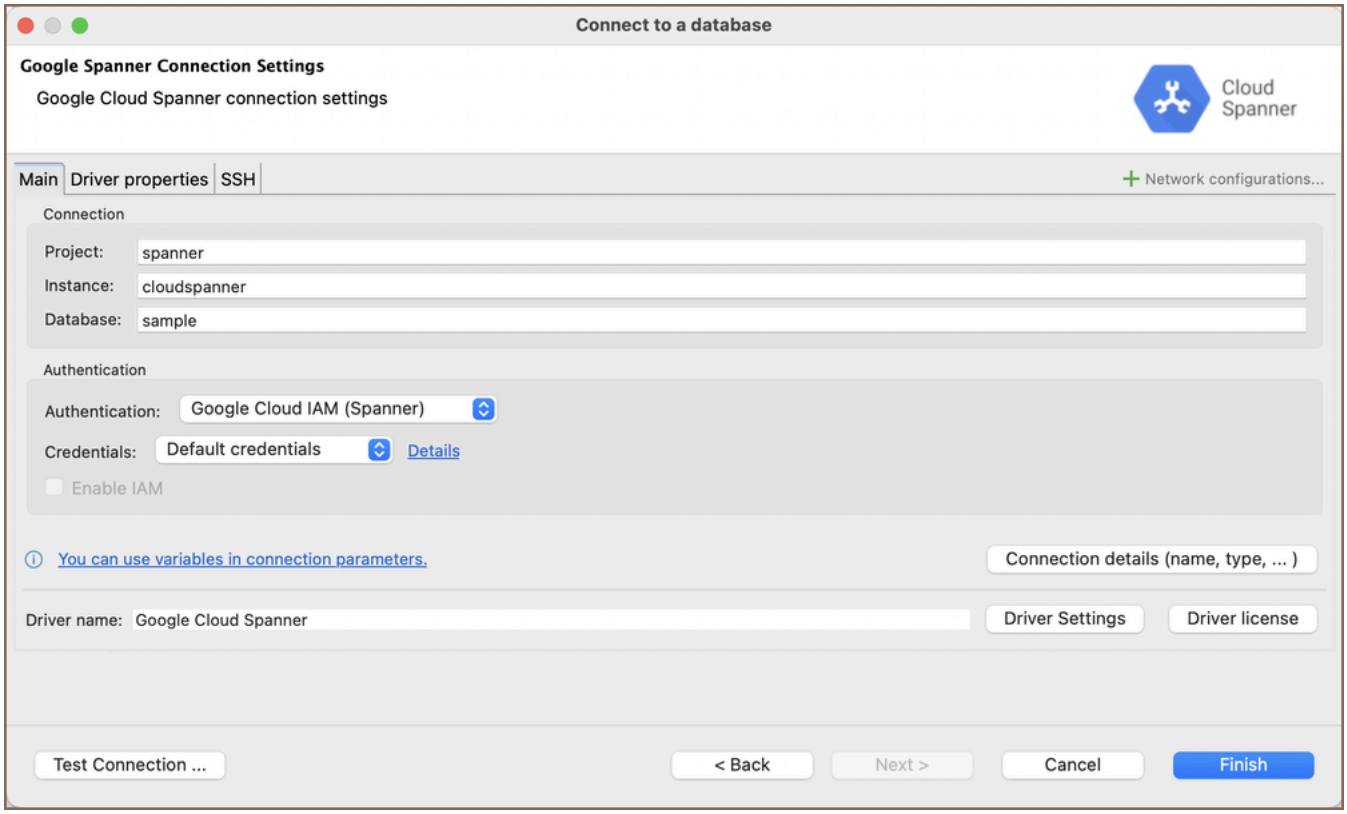
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, and Kubernetes.

Google Spanner connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Google Spanner database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Google Spanner database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Project	Enter the project name of your Google Spanner database here.
Instance	For a direct connection to a Google Spanner instance, use the format `project:region:instance`. Here, `project` refers to your Google Cloud project ID, `region` denotes the geographical location of your instance, and `instance` is the unique ID of your Google Cloud SQL instance.
Database	Enter the name of the Google Spanner database you want to connect to.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Google Cloud IAM (Spanner) - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows for further customization of your Google Spanner connection. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Google Spanner driver properties

The settings for Google Spanner **Driver properties** enable you to adjust the performance of the Google Spanner JDBC driver. These adjustments can influence the efficiency, compatibility, and features of your Google Spanner database.

For a complete walkthrough on setting up Google Spanner JDBC driver properties, you can refer to the official Google Spanner JDBC documentation.

You can customize the Google Spanner driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your Google Spanner database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your Google Spanner database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Google Spanner with DBeaver

DBeaver provides a host of features designed for Google Spanner databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Google Spanner database objects

DBeaver lets you view and manipulate a wide range of Google Spanner database objects. DBeaver has extensive support for various Google Spanner metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
  - Tables
    - Columns
    - Keys
    - Foreign Keys
    - Indexes
    - References
  - Views
  - Procedures

Google Spanner additional features in DBeaver

DBeaver provides additional features compatible with Google Spanner, but not exclusive to it:

Category	Feature
Data Transfer	Data Import
	Data Export
Data Visualization	ERD Guide
Data Generation	Mock Data Generation

Snowflake

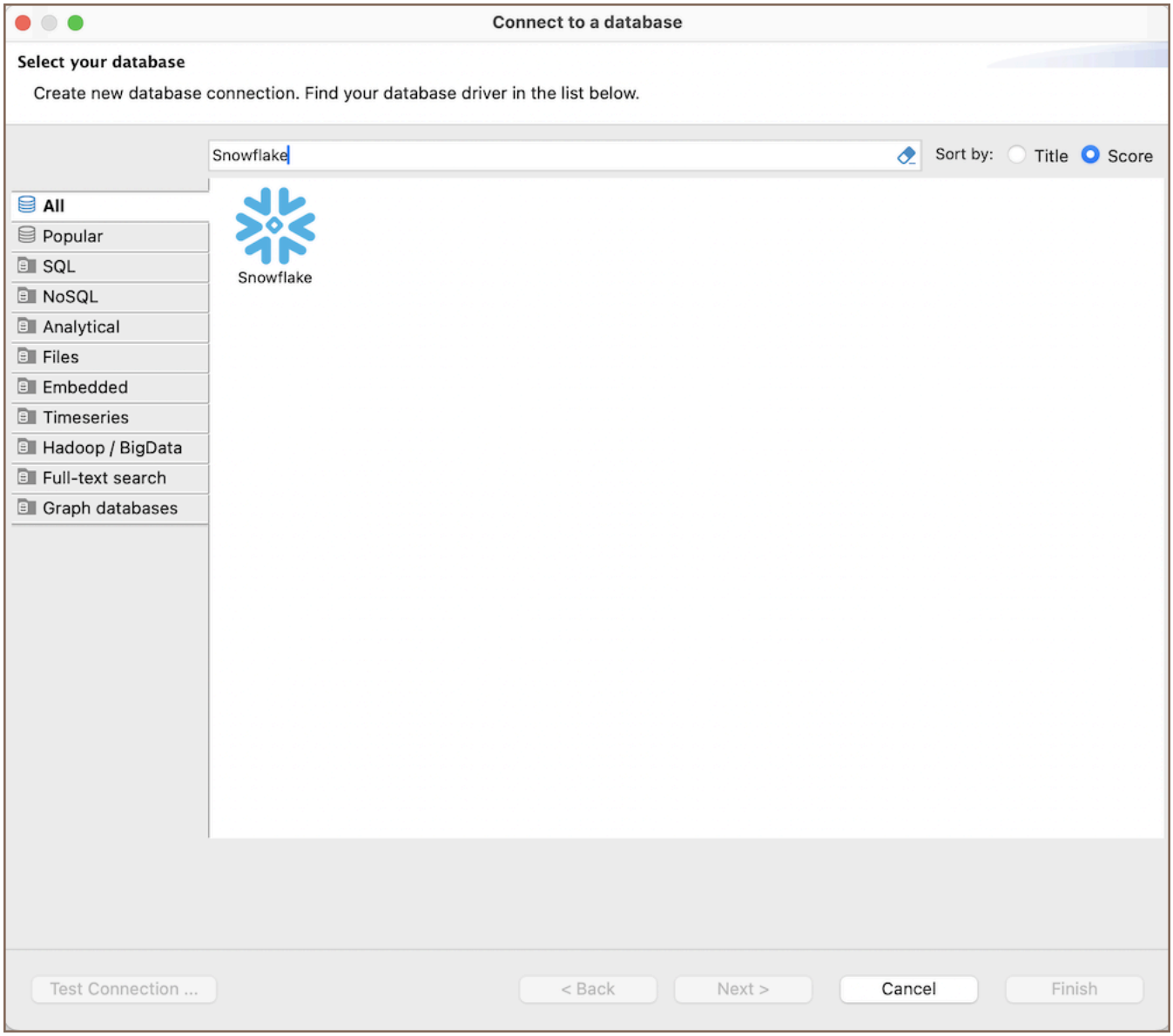
Table of contents

- Snowflake specialty
- Setting Up
- Snowflake connection settings
- Connection details
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Snowflake
- Snowflake database objects
- Snowflake features

> **Important**: If your Snowflake connection stopped working after updating to version `25.0`, check the driver version.
> If it's `3.13.30` or older, please update it to the latest stable release manually.

This guide provides instructions on how to set up and use Snowflake with DBeaver.

Before you can start managing your database, it's essential to establish a connection in DBeaver. This involves selecting the `Snowflake` option to connect to the Snowflake database. If you have not yet created a connection in DBeaver, please refer to our Creating a Connection article for guidance.



Snowflake specialty

Snowflake is a cloud-native data platform designed for fast, scalable analytics on large datasets. Its architecture separates storage and compute, so you can scale each independently. Snowflake handles structured and semi-structured data, supports concurrent workloads, and automatically manages performance, availability, and maintenance. Snowflake stores data in a compressed columnar format and runs queries using MPP (massively parallel processing) for high performance. For details, see the Snowflake documentation.

> **Warning**: This database may charge for metadata queries, such as listing tables or reading schema information. These queries can trigger compute usage or per-request billing. You can turn off metadata queries to avoid extra costs. For details, see Disable metadata queries.

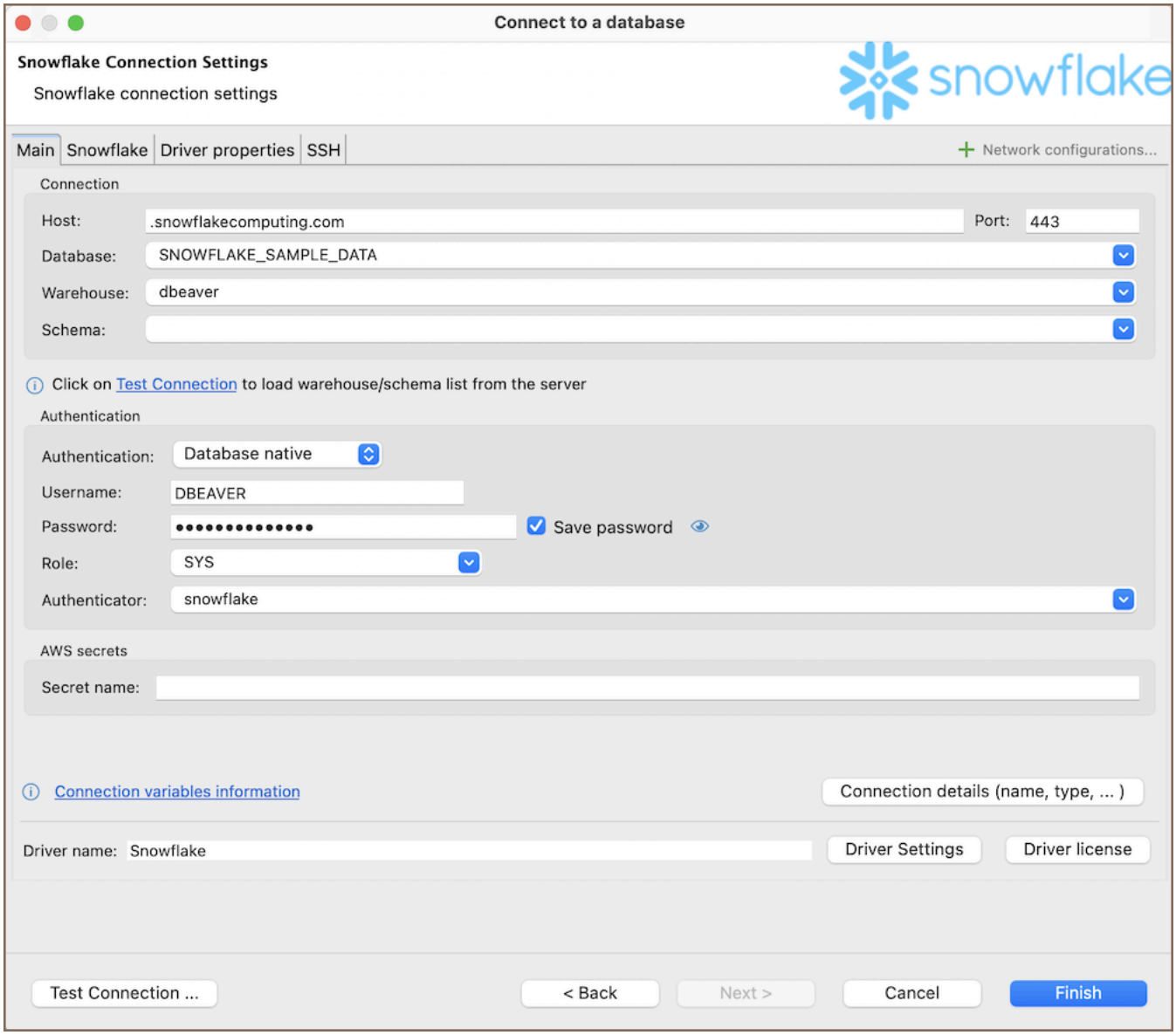
Setting Up

This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, AWS SSM and Kubernetes.

Snowflake connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Snowflake database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Snowflake database.

The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Host	Enter the Snowflake account URL (e.g., `abcde12345.snowflakecomputing.com`).
Port	Use port `443` for HTTPS connections.
Database	Enter the name of the database you want to connect to.
Warehouse	Enter the name of the virtual warehouse to use for query execution.
Schema	Enter the name of the schema that contains your tables and views.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication ★ - Private key ★ - SSO (Browser) ★ You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be autofilled based on your selected driver type.

> **Tip**: Click the **Test Connection** button to load the list of available warehouses/schemas from the server.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Snowflake database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these

settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure Connection Configurations

DBeaver supports secure connections to your Snowflake database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Snowflake

DBeaver provides a host of features designed for Snowflake databases. This includes the ability to view and manage databases, along with numerous unique capabilities aimed at optimizing database operations.

Snowflake database objects

DBeaver lets you view and manipulate a wide range of Snowflake database objects. DBeaver has extensive support for various Snowflake metadata types, allowing you to interact with a wide variety of database objects, such as:

- Databases
- Schemas
  - Tables
  - Columns
  - Primary Keys
  - Unique Keys
  - Foreign Keys
  - References
  - Views
  - Materialized Views ★
  - Procedures
  - Sequences ★
  - Stages ★
  - Pipes ★
  - Streams ★
  - Tasks ★
- Data types

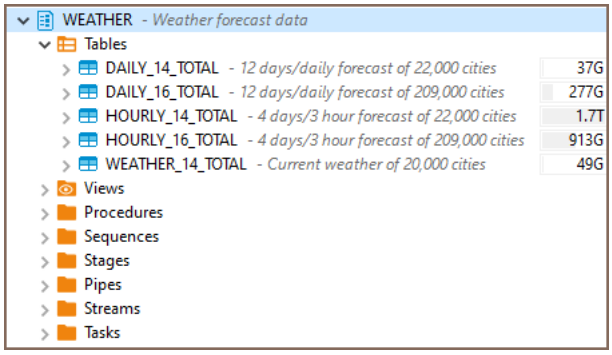
Snowflake features

DBeaver is not limited to typical SQL tasks. It also includes numerous unique features specifically for Snowflake. Beyond regular SQL operations, DBeaver provides a range of Snowflake-specific capabilities, such as:

Category	Feature
Task management	Automate workflows with **Tasks** and **Streams**.
Access control	Role-based Access Control (RBAC).
Authentication	Support for **SSO** and **key pair authentication**.

Additional features compatible with Snowflake, but not exclusive to it:

Category	Feature
Data Transfer	Data Import.
	Data Export.
Data Visualization	ERD Guide.
Query insights	Disable metadata queries



Embedded

SQLite

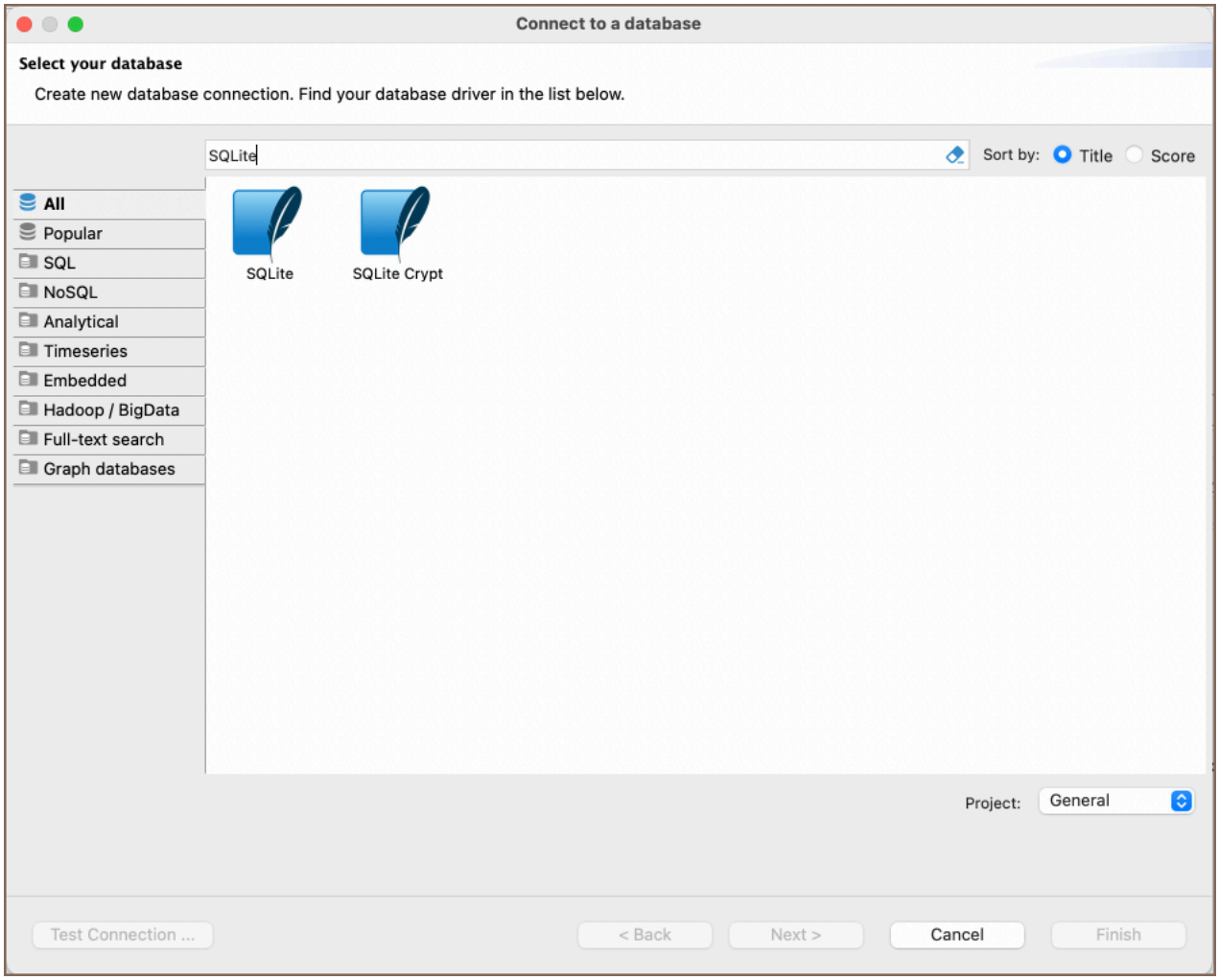
Table of contents

- Setting Up
- SQLite connection configuration
  - General SQLite connection settings
  - SQLite Crypt connection settings
  - Extensions in Community Edition
  - Remote database connection
  - Synchronization
- Connection details
- SQLite driver properties
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering SQLite with DBeaver
- SQLite database objects

This guide provides instructions on how to set up and use SQLite with DBeaver.

Before you start, you must create a connection in DBeaver and select SQLite. If you have not done this, please refer to our Database Connection article.

DBeaver interacts with the SQLite server using a specific driver, supporting all versions of SQLite. DBeaver also supports SQLite extensions such as SQLite Crypt (Cipher). You can also create a sample database in DBeaver. See our Creating a Sample Database article for more information.



Setting Up

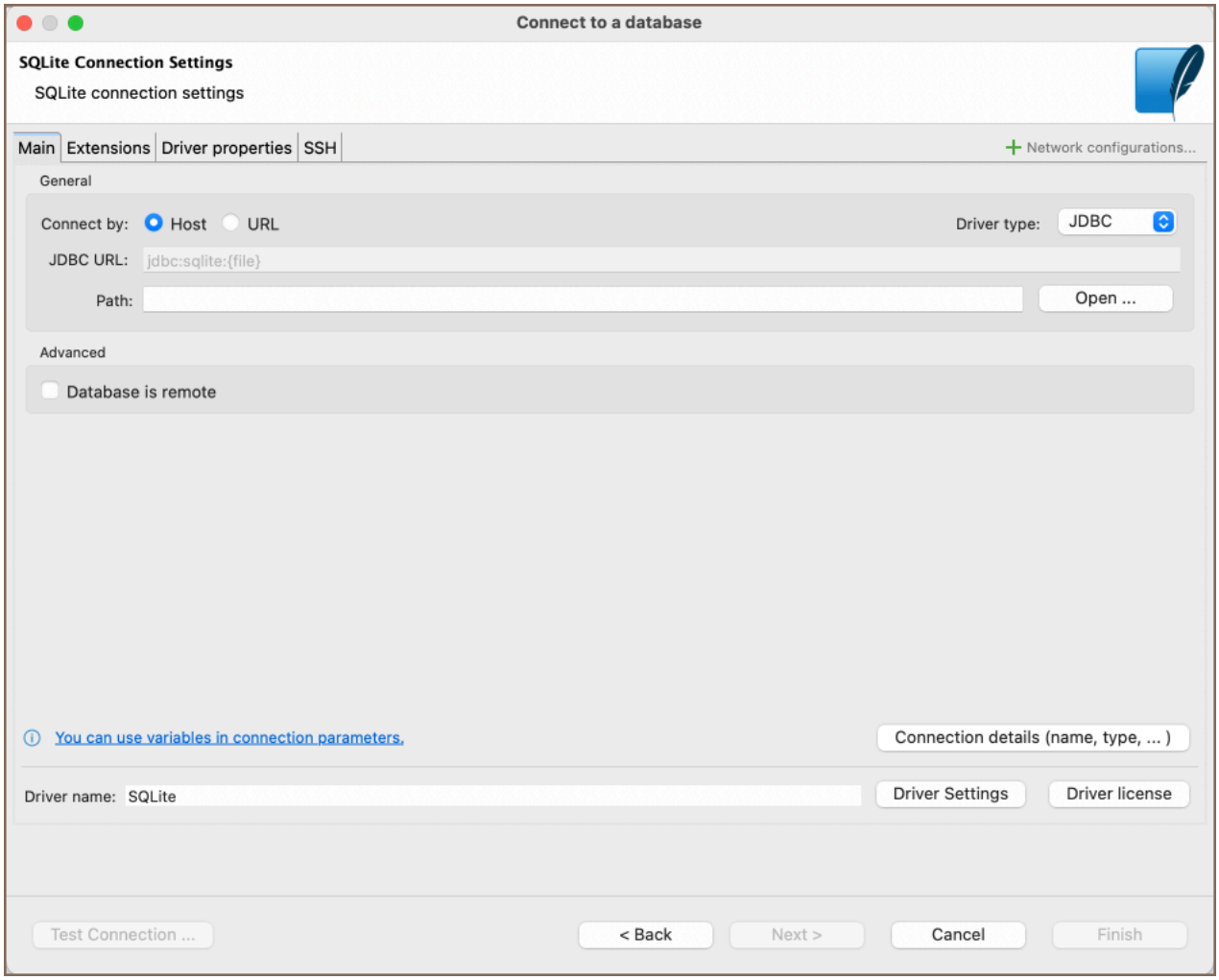
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, Kubernetes, and the setup of ODBC/JDBC drivers for SQLite.

SQLite connection configuration

General SQLite connection settings

In this subsection, we will outline the settings for establishing a direct connection to a SQLite database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your SQLite database.

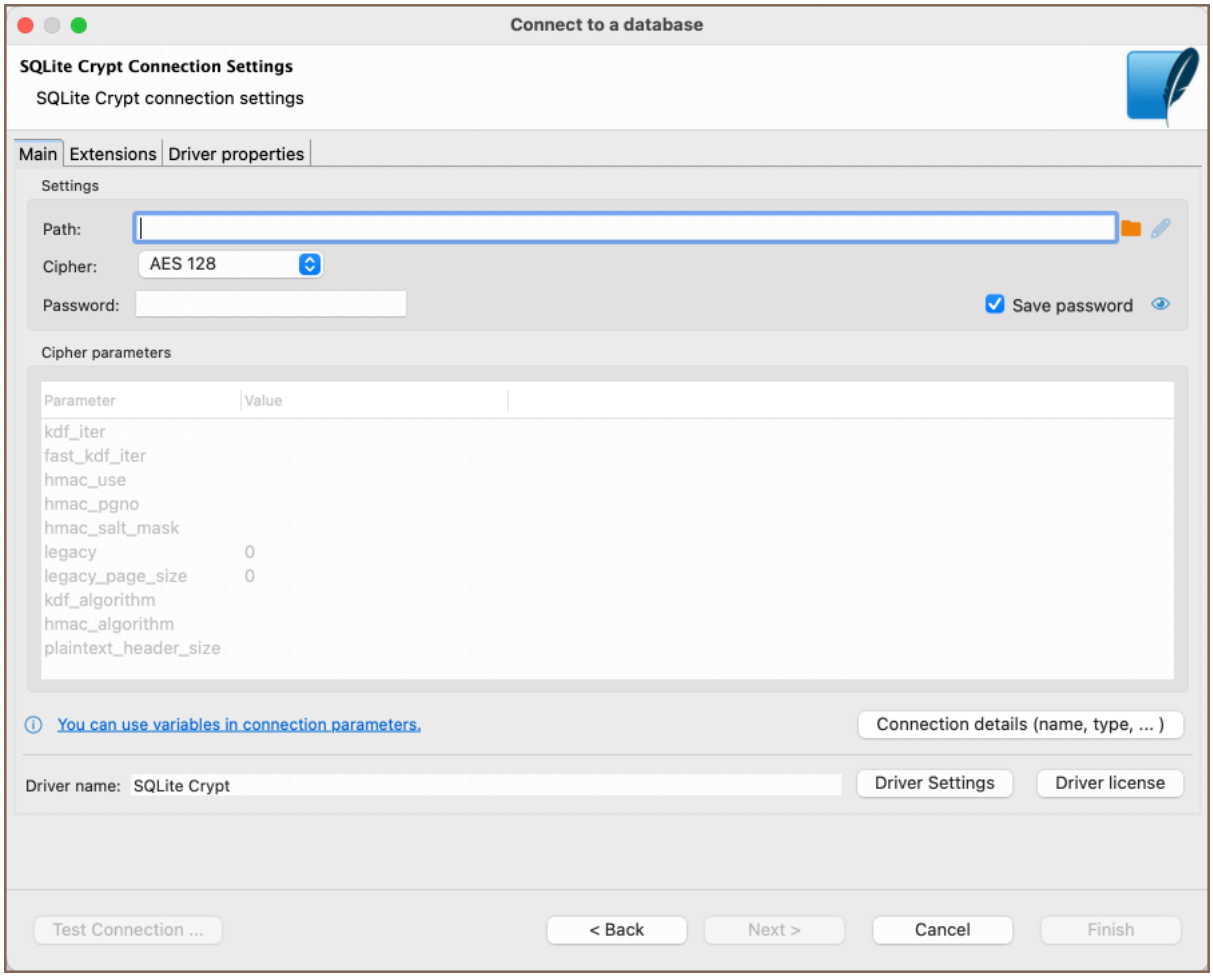
The page of the connection settings requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
Driver Type	Select the appropriate driver for your SQLite database.
URL	If you are connecting via URL, enter the URL of your SQLite database here. This field is disabled if you're connecting via the host.
Path	Enter the path to the database file.
Database is remote	If the SQLite database resides on a remote server, check this box. Reed more.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

SQLite Crypt connection settings

This subsection outlines the settings to establish a connection to encrypted SQLite databases.



Field	Description
Path	Enter the path to the database file.
Cipher	Choose the encryption algorithm for your database. Available options include `AES 128`, `AES 256`, `ChaCha20`, `SQLCipher`, `SQLCipher v1`, `SQLCipher v2`, `SQLCipher v3`, `SQLCipher v4`, `RC4`, and `Custom` option. For more details, refer to the SQLite Ciphers documentations.
Password	This field will be auto-filled based on your selected driver type.
Cipher parameters	These parameters are read-only and change according to the chosen Cipher. You can customize them only if you select **Custom** in the Cipher field.

Extensions in Community Edition

To enable and load SQLite extensions in the Community Edition, follow these steps:

1. Set the `enable_load_extension` property to `true` in Driver properties tab.
2. Execute `SELECT load_extension('extension-name.so')` either as a bootstrap query in the Connection Initialization Settings or in the SQL editor.

> **Note**: You need to have a 64-bit version of the extension library.

Remote database connection

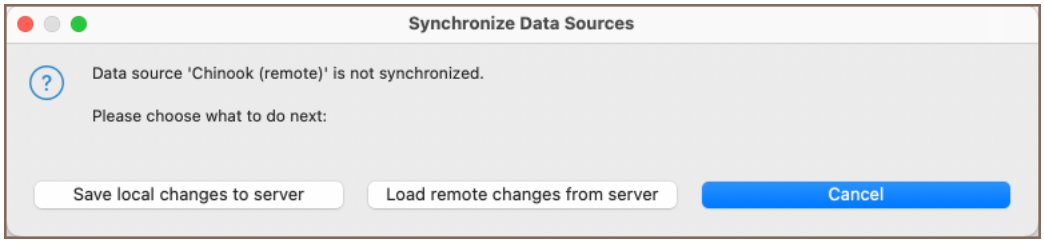
When connecting to a remote SQLite database in DBeaver:

1. Specify a local path in the connection settings. This path should mirror the path on the remote server.
2. Set up an SSH tunnel.

Synchronization

To maintain consistency between your local SQLite database and the remote server, follow these steps:

1. Make local changes to your database.
2. Save changes.
3. Click the **Synchronize** button in the main toolbar.
4. A dialog will prompt you with two options:



- **Save local changes to server**: This will push your local changes to the server.
- **Load remote changes from server**: This will pull the server's data to your local environment. After selecting this, refresh the result set to view updated data.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with SQLite database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

SQLite driver properties

The settings for SQLite **Driver properties** enable you to adjust the performance of the SQLite JDBC and ODBC driver. These adjustments can influence the efficiency, compatibility, and features of your SQLite database.

For a complete walkthrough on setting up SQLite JDBC and ODBC drivers properties, you can refer to the official SQLite JDBC documentation and SQLite ODBC documentation. These guides detail each driver's properties and how they can be used to optimize SQLite database connections.

You can customize the SQLite driver in DBeaver via the **Edit Driver** page, accessible by clicking on the **Driver Settings** button on the first page of the driver settings. This page offers a range of settings that can influence your SQLite database connections. For a comprehensive guide on these settings, please refer to our Driver manager article.

Secure Connection Configurations

DBeaver supports secure connections to your SQLite database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes** and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration** ★
- **Proxy Configuration** ★
- **Kubernetes Configuration** ★
- **AWS SSM** ★

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering SQLite with DBeaver

DBeaver provides a host of features designed for SQLite databases. This includes the ability to view and manage tables, along with numerous unique capabilities aimed at optimizing database operations.

SQLite database objects

DBeaver lets you view and manipulate a few SQLite database objects. DBeaver has extensive support for various SQLite metadata types, allowing you to interact with:

- Tables
- Views

File drivers

Multi format

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

- Multi Format driver connection settings
- Features and capabilities
- Advanced SQL query capabilities
- Folder structure

- Internal database
- Additional features

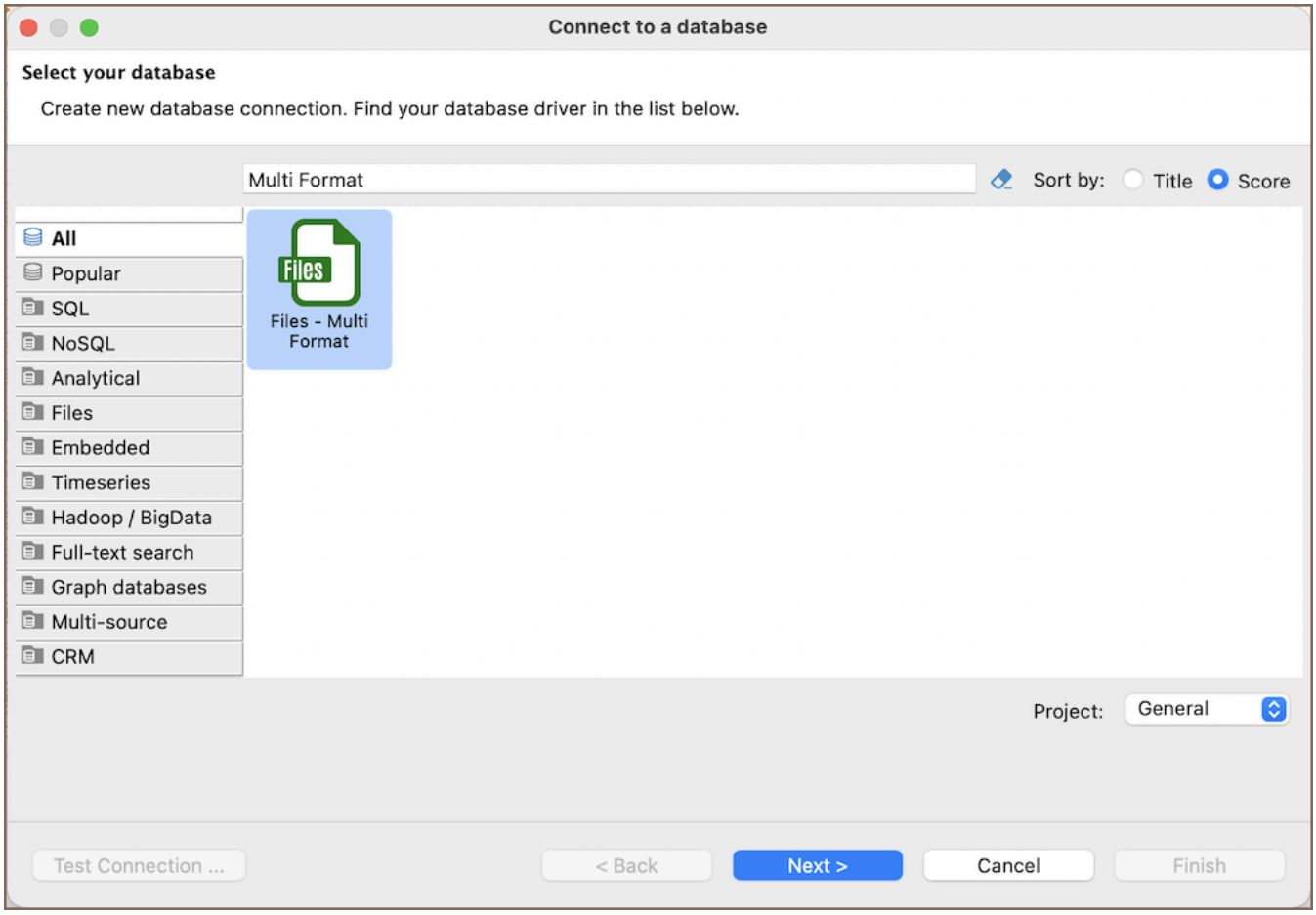
This guide explains how to set up and use the **Multi Format** driver in DBeaver. This driver allows you to work with multiple files of different formats within a single connection, including:

- CSV
- JSON
- Parquet
- XML
- XLSX

You can query data from different sources simultaneously, apply filters and sorting, and even join data across files.

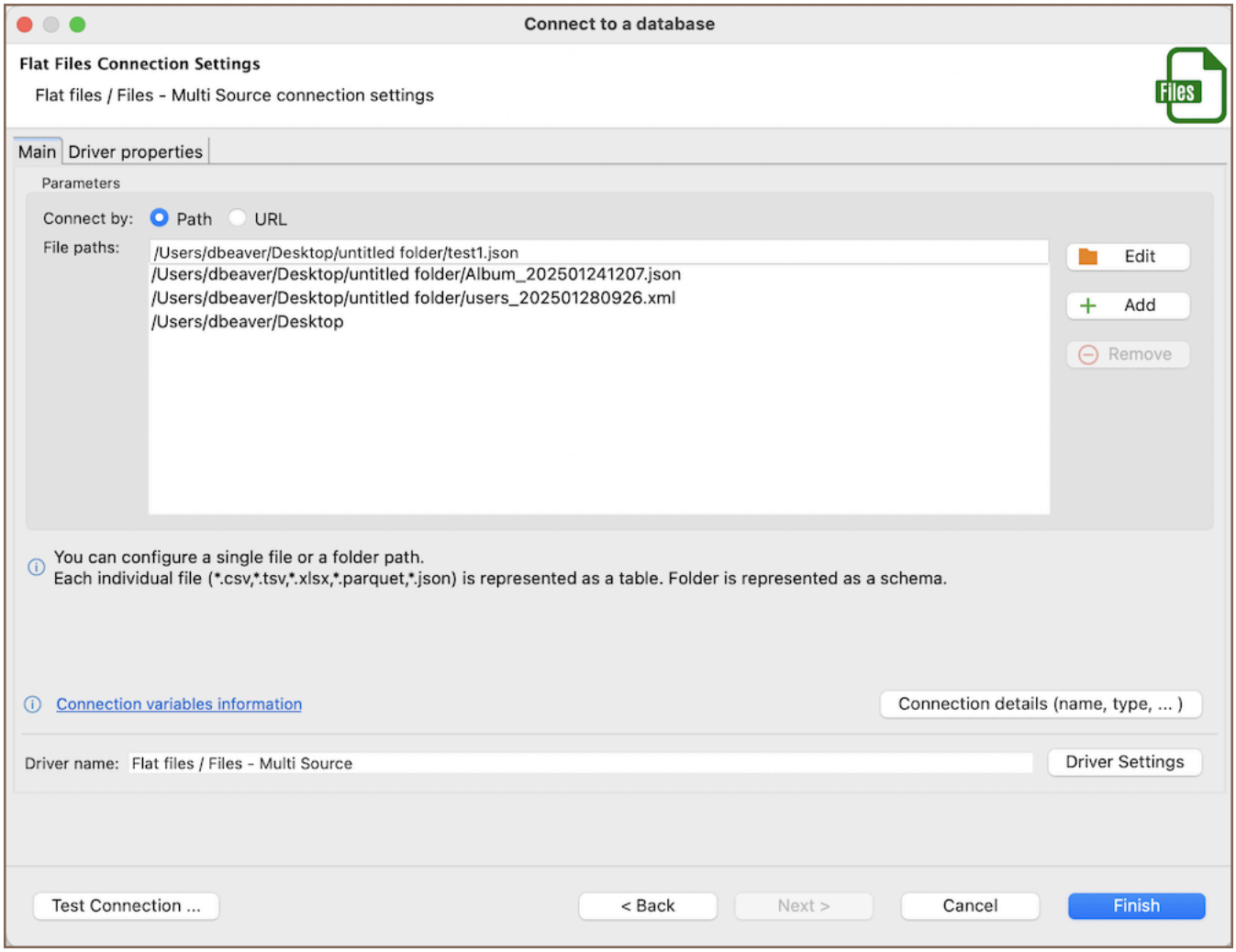
> **Important**: When using the **Multi Format** driver, all connected files are read-only. To make changes, update the original files outside DBeaver.

Before you start, create a connection in DBeaver and select the **Multi Format** driver. If you haven't done this, see our Database Connection article.



Multi Format driver connection settings

This section describes how to set up a connection using the **Multi Format** driver. The connection settings page requires the following fields:



Field	Description
Connect by (Path/URL)	Choose whether to connect using a local host path or a URL.
File paths	Specify the location of the Multi Source file(s). Choose an action: - **Edit**: Modify an existing file or folder selection. - **Add**: Add a new file or folder. - **Remove**: Delete a selected file or folder. When **Editing** or **Adding**, choose from: - **File**: Select a single file. - **Folder**: Choose a directory containing multiple files. For more details, see Folder structure section. - **Remote**: Access a remote folder via Cloud Storage. This feature is available only in Ultimate Edition.
Driver name	This field will be auto-filled based on your selected driver type.
Driver settings	If there are any specific driver settings, configure them here.

> **Tip**: When using the **Folder** option, DBeaver scans the directory up to two levels deep for supported files. If the folder contains multiple files, DBeaver organizes them into schemas based on their directory structure. For more information, see folder structure.

Features and capabilities

Advanced SQL query capabilities

The **Multi Format** driver supports the full range of SQL queries:

- **Simple queries** (e.g., `SELECT * FROM table`): Data is read directly from the files.
- **Complex queries** (e.g., using `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`): When a complex query is executed for the first time, the driver imports the data from all selected files into an internal database to enable advanced SQL functions. Subsequent queries run faster because the data is already imported into an internal database.

> **Note**: If you join data from different files, they must be opened in the same connection.

Folder structure

When working with multiple files and folders, DBeaver organizes them as follows:

Location	Schema in DBeaver
Root files	`Default` schema
Subfolder files	Schema named after the subfolder
Files in deeper folders	Ignored

If your folder structure looks like this:

```
Data/
├── employees.csv
├── sales.json
└── Reports/
    ├── monthly.xml
    ├── yearly.parquet
    └── summary.xlsx
```

DBeaver will create:

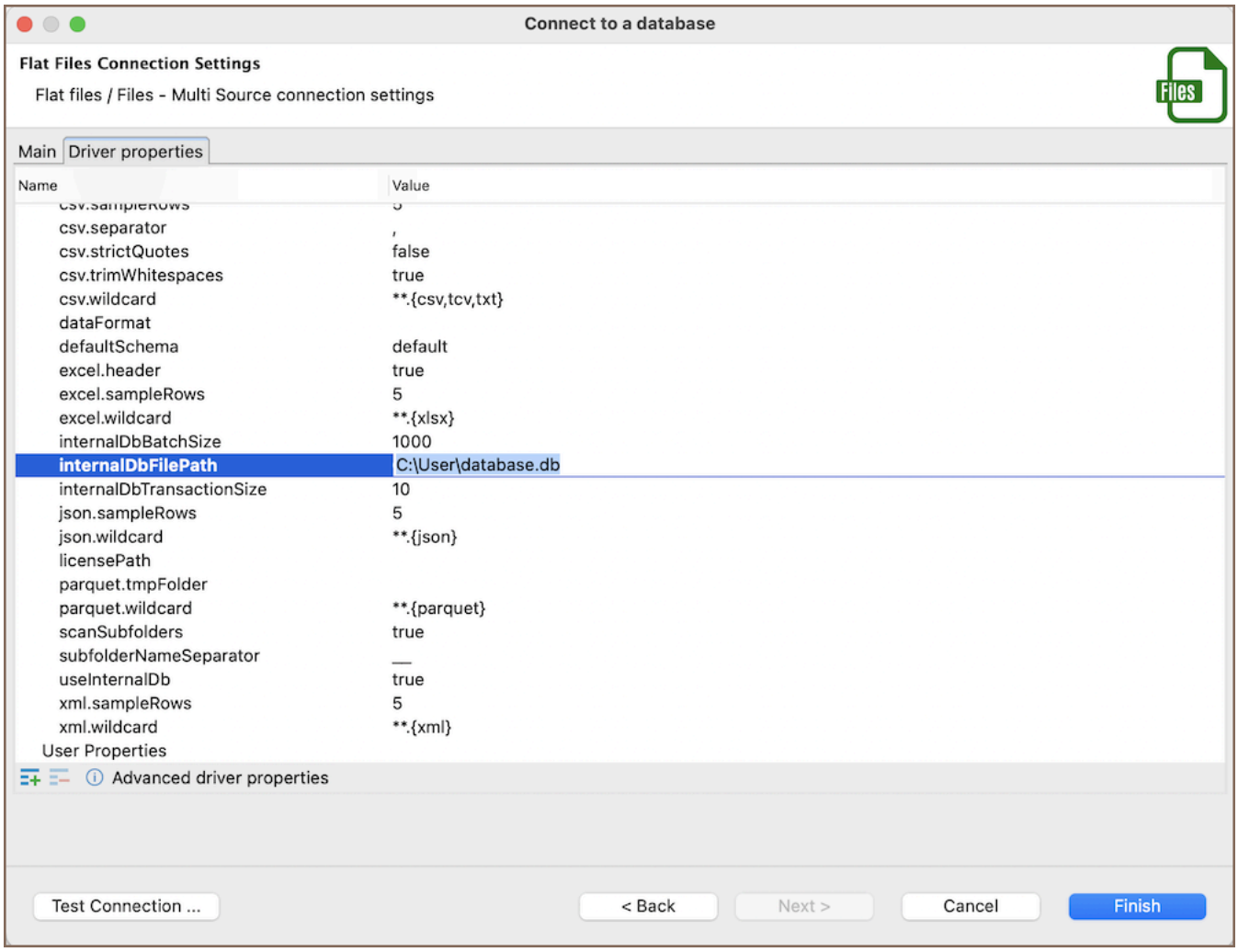
- `Default` schema: `employees`, `sales`
- `Reports` schema: `monthly`, `yearly`, `summary`

> **Tip**: To focus on specific files, consider selecting individual files or folders when configuring the connection.

Internal database

When you execute a complex query (such as `WHERE`, `JOIN`, `GROUP BY`, or `ORDER BY`.) on multiple files for the first time, the **Multi Format** driver processes the data by importing it into a temporary internal SQLite database.

By default, this internal database stores data temporarily on the disk during your session and is cleared when DBeaver restarts. To speed up queries on the same files in future sessions, you can specify the `internalDbFilePath` option in the **Driver properties** tab (e.g., `C:\User\database.db`) to reuse the processed data.



> For more details on driver properties, see File-based driver properties.

Additional features

DBeaver provides additional features compatible with the **Multi Format** driver, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	Visual Query Builder
	Charts

CSV

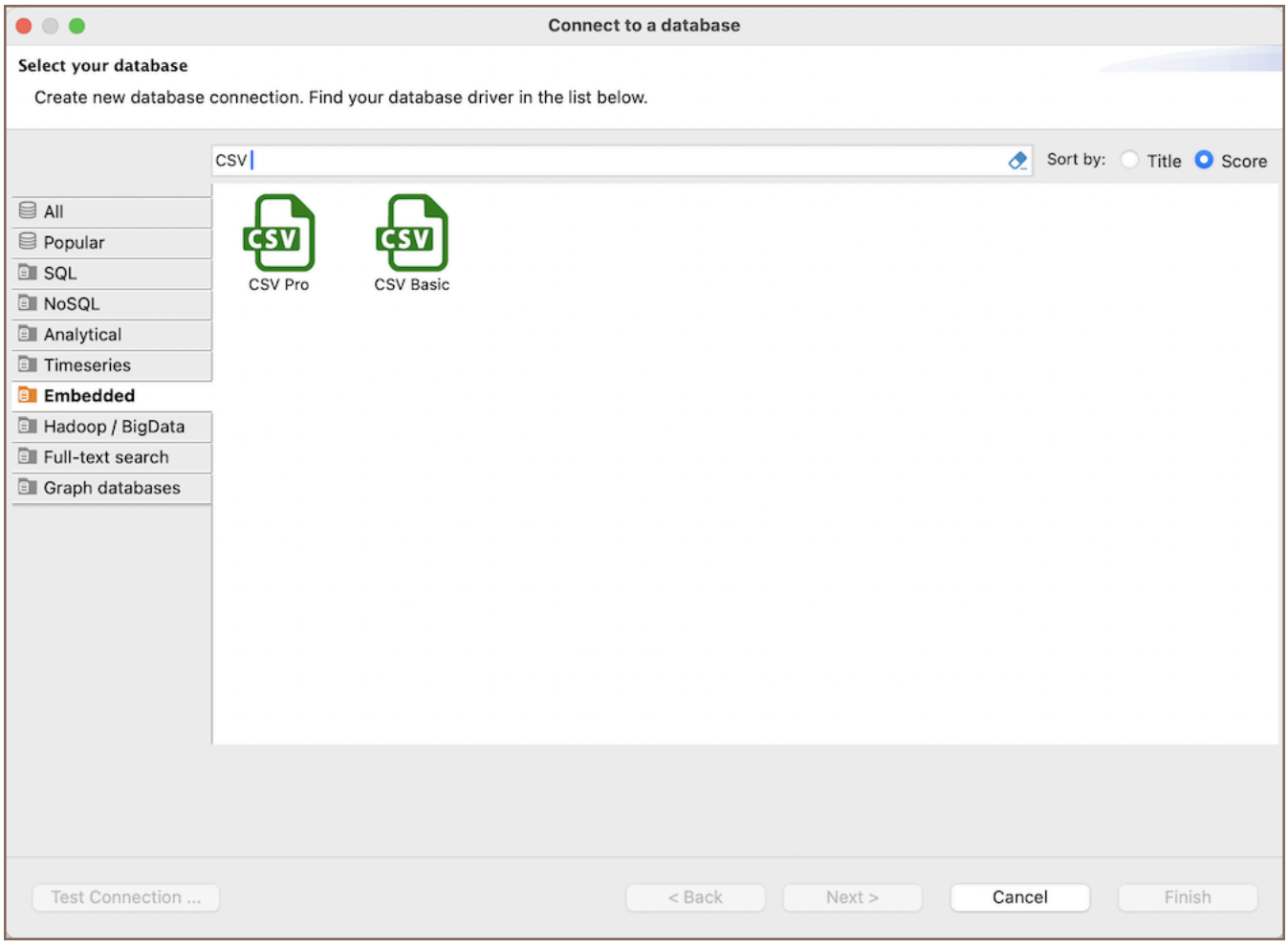
Table of contents

- Legacy CSV driver
- CSV Files driver connection settings
- Features and capabilities
- Advanced SQL query capabilities
- Structuring CSV files with a schema
  - Why use a DDL file
  - How to create a DDL file
- Folder structure
- Internal database
- Additional features

This guide provides instructions on how to set up and use CSV files with DBeaver. The **CSV Pro** driver allows you to work with CSV data as if it were in a database. You can retrieve data and apply filters, sorting, and other operations, even combining data from multiple files.

> **Important**: When using the **CSV Pro** driver, all connected CSV files are read-only. To make changes, you need to update the original files outside DBeaver.

Before you start, you need to create a connection in DBeaver and select the **CSV Pro** driver. If you haven't done this, see our Database Connection article.

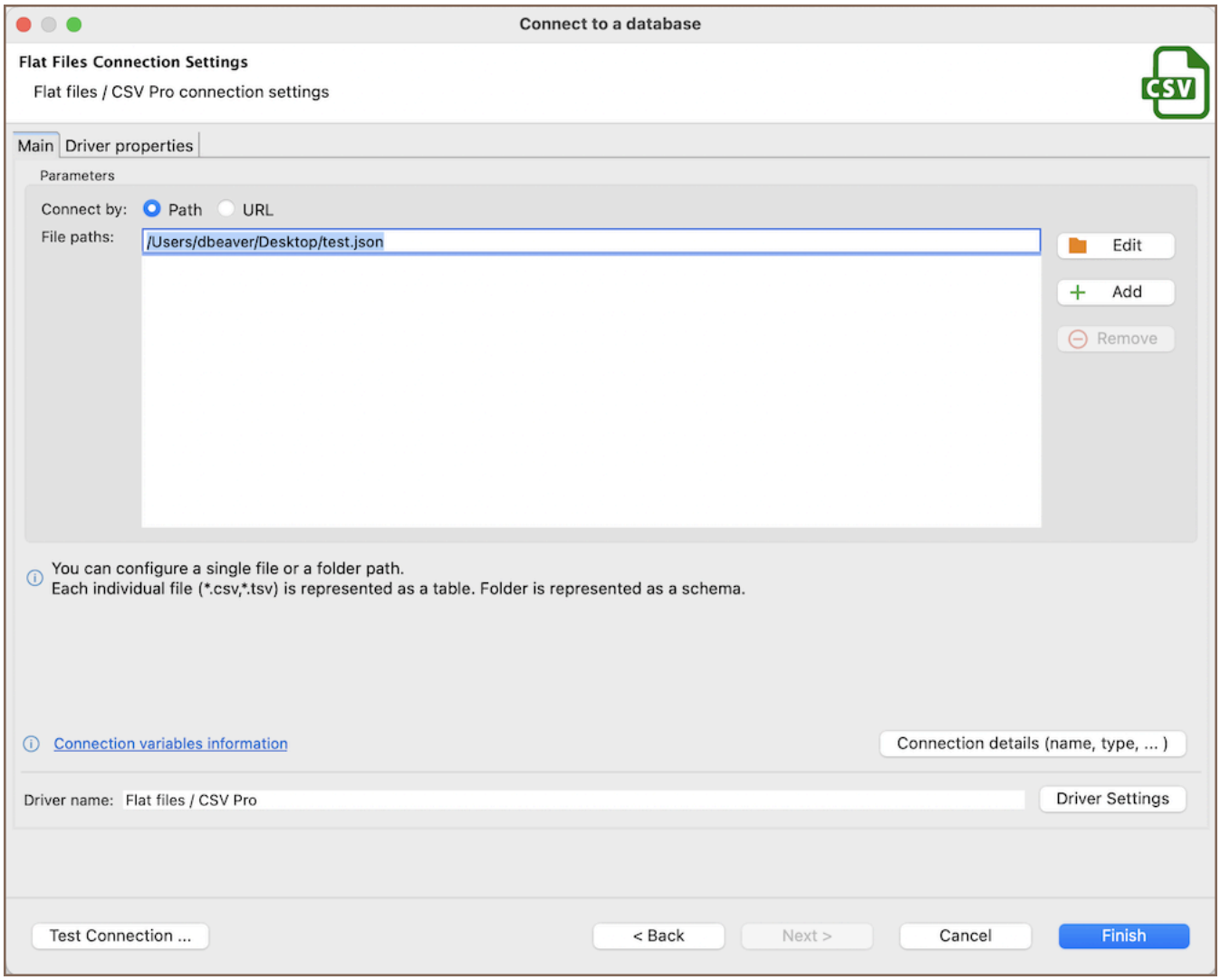


Legacy CSV driver

In the **Community Edition**, a legacy **CSV Basic** driver is available. It supports only simple operations.

CSV Files driver connection settings

This section describes how to set up a connection using the **CSV Pro** driver. The connection settings page requires the following fields:



Field	Description
Connect by (Path/URL)	Choose whether to connect using a local host path or a URL.
File paths	Specify the location of the CSV file(s). Choose an action: - **Edit**: Modify an existing file or folder selection. - **Add**: Add a new file or folder. - **Remove**: Delete a selected file or folder. When **Editing** or **Adding**, choose from: - **File**: Select a single file. - **Folder**: Choose a directory containing multiple CSV files. For more details, see Folder structure section. - **Remote**: Access a remote folder via Cloud Storage. This feature is available only in Ultimate Edition.
Driver name	This field will be auto-filled based on your selected driver type.
Driver settings	If there are any specific driver settings, configure them here.

For details on driver properties, see File-based driver properties.

> **Tip**: When using the **Folder** option, DBeaver scans the directory up to two levels deep for CSV files. For more information, see folder structure. If you select a folder, DBeaver organizes files in schemas based on their directory structure.

Features and capabilities

Advanced SQL query capabilities

The **CSV Pro** driver supports the full range of SQL queries:

- **Simple queries** (e.g., `SELECT * FROM table`): Data is read directly from the CSV file.
- **Complex queries** (e.g., using `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`): When a complex query is executed for the first time, the driver imports the entire CSV file into an internal database to enable advanced SQL functions. Subsequent queries run faster because the data is already imported into internal database.

Structuring CSV files with a schema

CSV files don't include metadata about their structure, such as column names or data types. To enhance how DBeaver interprets these files, you can define a schema using a DDL (Data Definition Language) file.

> For information on creating a virtual database structure from SQL files, see DDL driver.

Why use a DDL file

A DDL file helps DBeaver interpret your data more accurately by defining:

- Column names
- Data types
- Optional indexes for better performance

How to create a DDL file

1. Create a `.ddl` file with the same name as your CSV file, placing it in the same directory (e.g., `employees.csv` and `employees.csv.ddl`).
2. Write a schema using the `CREATE TABLE` statement:

```
CREATE TABLE employees (
  id INTEGER,
  name TEXT NOT NULL,
  age INTEGER,
  department TEXT
  );
  CREATE INDEX idx_employees_id ON employees (id);
```

You can also use the `WITH` clause to set a data range - add `firstRow` and `rowCount` to your `CREATE TABLE` statement:

```
WITH (firstRow = 2, rowCount = 100)
```

- `firstRow` - row number to start reading from (default: 1)
- `rowCount` - maximum number of rows to read

> **Tip**: You can also set `firstRow` and `rowCount` in the connection properties. DDL file settings take priority.

> **Important**: If the DDL file contains errors, DBeaver will ignore it. You can still query the file, but column types will be inferred from the data.

Folder structure

When working with a folder containing multiple CSV files, DBeaver organizes them as follows:

Folder structure	Schema in DBeaver
Root files	`Default` schema
Subfolder files	Schema named after the subfolder
Files in deeper folders	Ignored

If your folder looks like this:

```
Data/
├── employees.csv
├── sales.csv
└── Reports/
    └── monthly.csv
    └── yearly.csv
```

DBeaver will create:

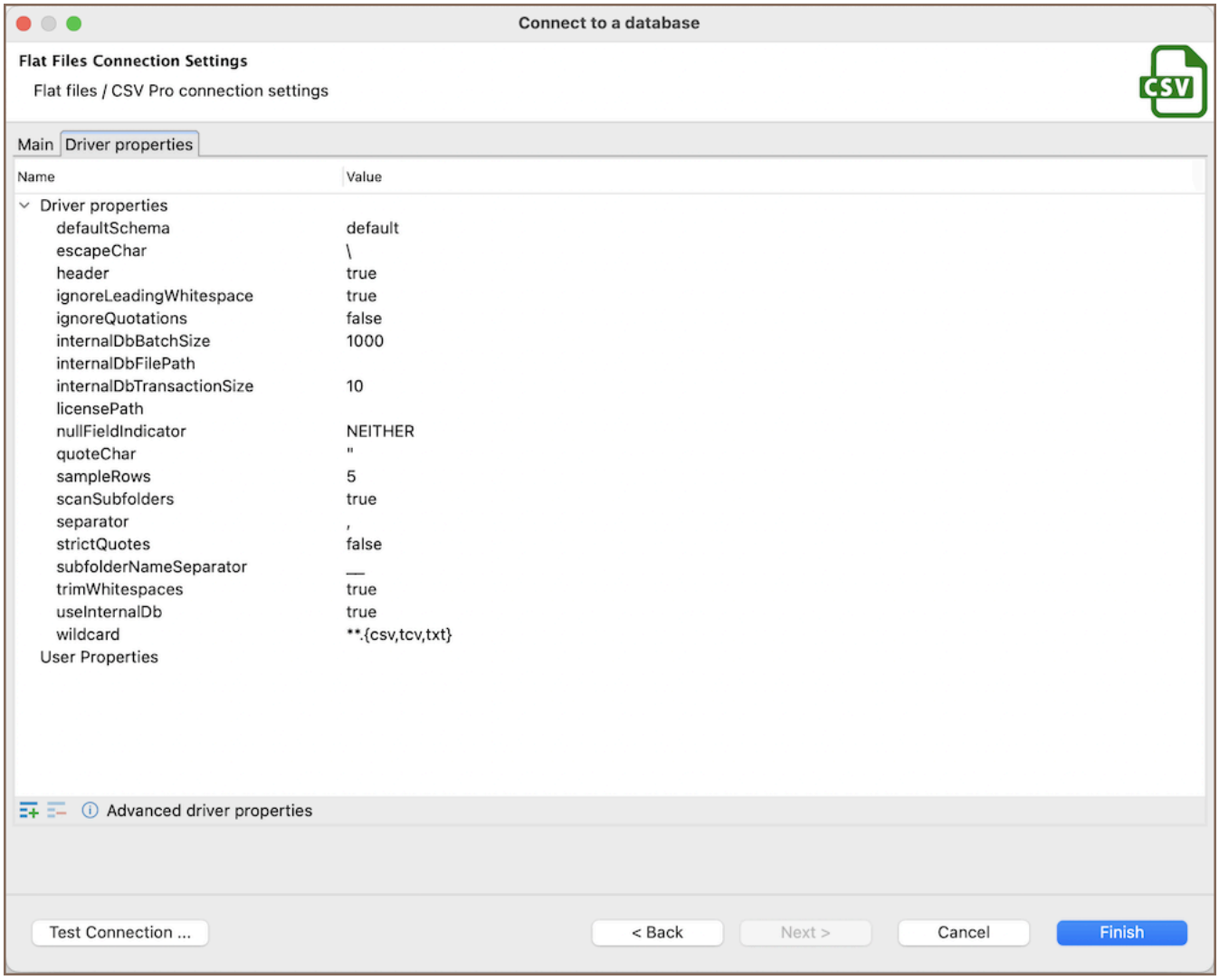
- `Default` schema: `employees`, `sales`
- `Reports` schema: `monthly`, `yearly`

> **Tip**: To focus on specific files, consider selecting individual files or folders when configuring the connection.

Internal database

When you execute a complex query (such as `WHERE`, `JOIN`, `GROUP BY`, or `ORDER BY`.), on an **CSV** file for the first time, the **CSV Pro** driver processes the data by importing it into a temporary internal SQLite database.

By default, this internal database stores data temporarily on the disk during your session and is cleared when DBeaver restarts. To speed up queries on the same file in future sessions, you can specify the `internalDbFilePath` option in the **Driver properties** tab (e.g., `C:\User\database.db`) to reuse the processed data.



Additional features

DBeaver provides additional features compatible with **CSV Pro** driver, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	Visual Query Builder
	Charts

JSON

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

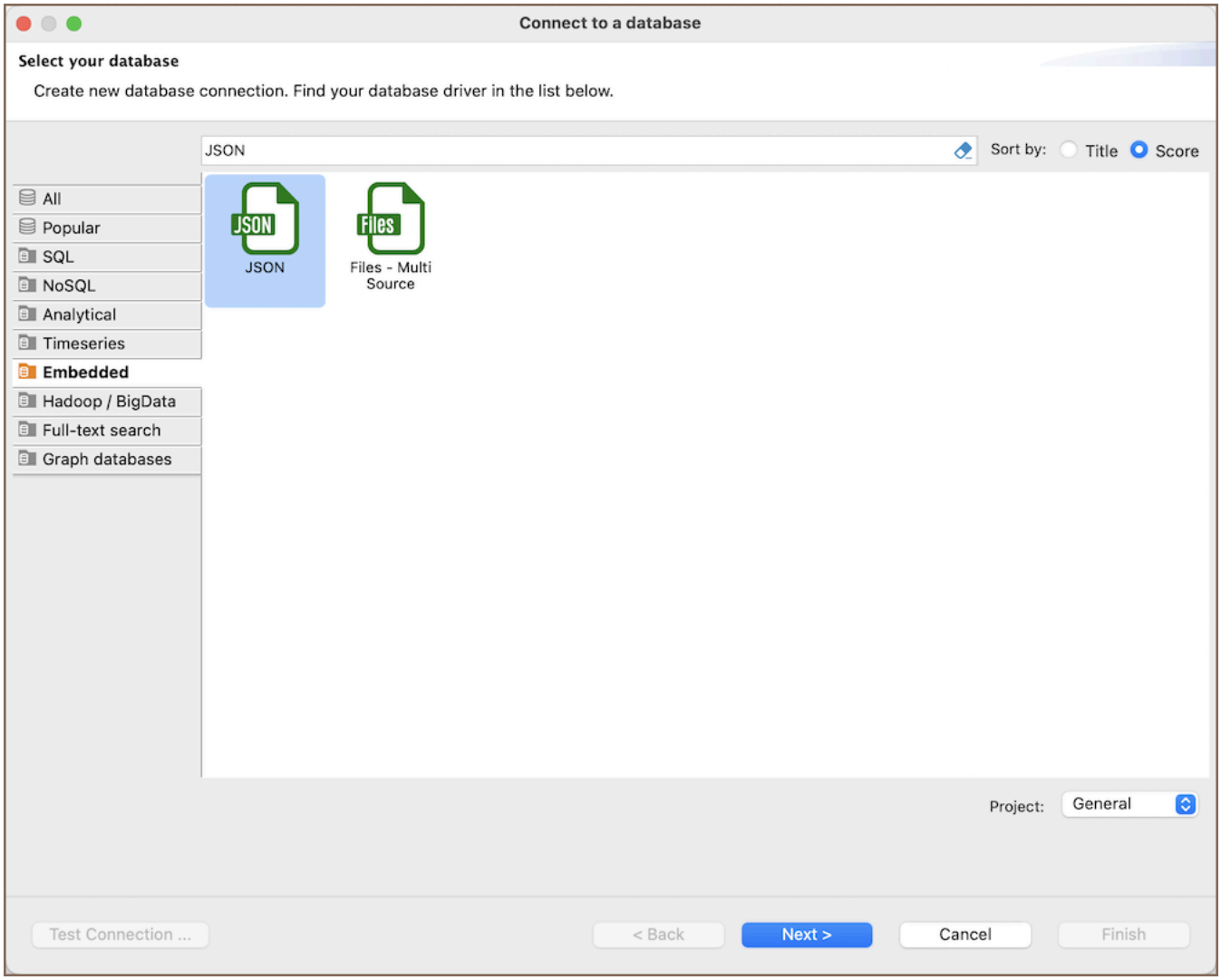
Table of contents

- JSON Files driver connection settings
- Features and capabilities
- Advanced SQL query capabilities
- Structuring JSON files with a schema
  - How to create a DDL file
- Folder structure
- Internal database
- Additional features

This guide provides instructions on how to set up and use JSON files with DBeaver. The **JSON** driver supports standard JSON files and allows you to work with JSON data as if it were in a database. You can retrieve data and apply filters, sorting, and other operations, even combining data from multiple files.

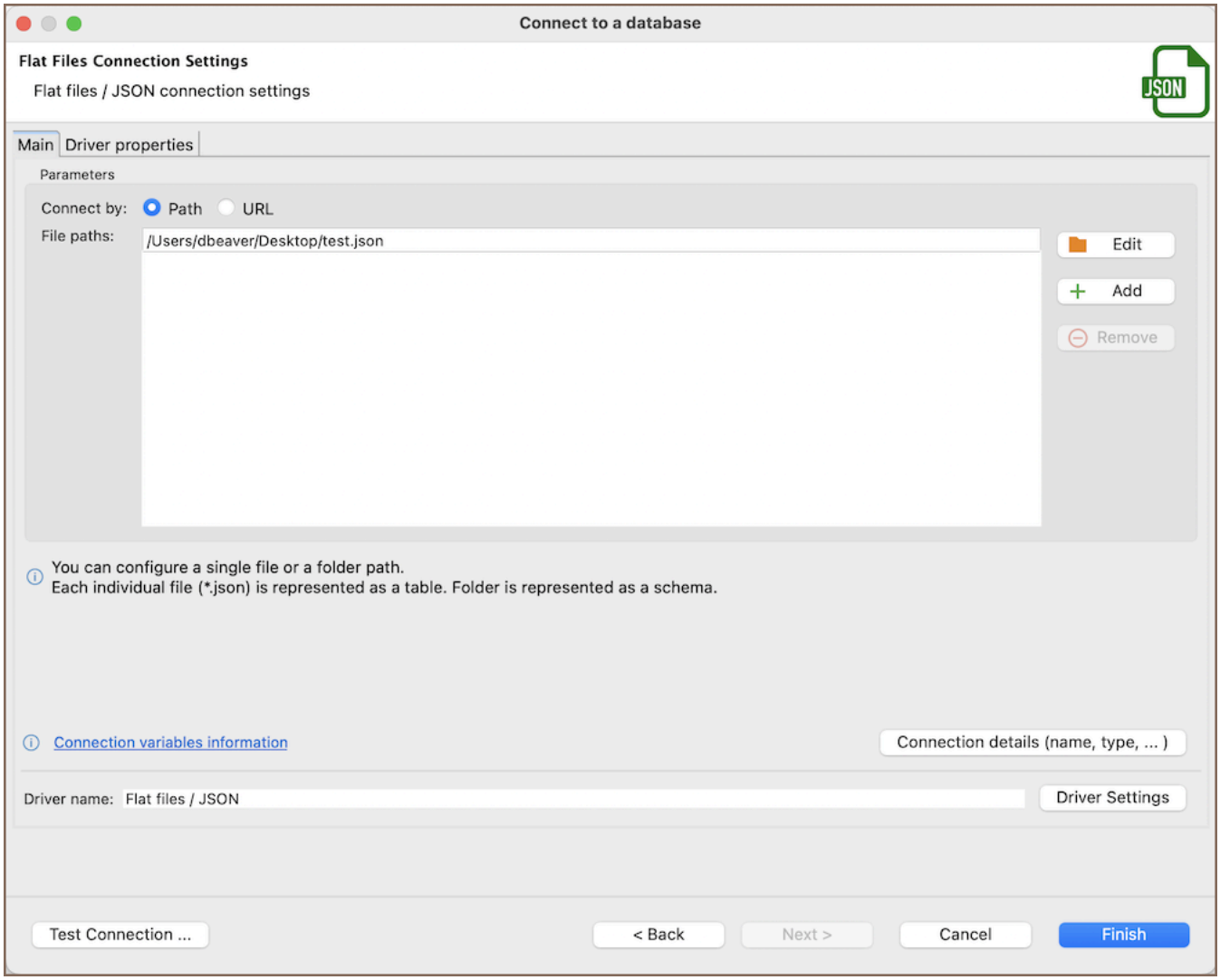
> **Important**: When using the **JSON** driver, all connected JSON files are read-only. To make changes, you need to update the original files outside DBeaver.

Before you start, you need to create a connection in DBeaver and select the appropriate **JSON** driver. If you haven't done this, see our Database Connection article.



JSON Files driver connection settings

This section describes how to set up a connection using the **JSON** driver. The connection settings page requires the following fields:



Field	Description
Connect by (Path/URL)	Choose whether to connect using a local host path or a URL.
File paths	Specify the location of the JSON file(s). Choose an action: - **Edit**: Modify an existing file or folder selection. - **Add**: Add a new file or folder. - **Remove**: Delete a selected file or folder. When **Editing** or **Adding**, choose from: - **File**: Select a single file. - **Folder**: Choose a directory containing multiple JSON files. For more details, see Folder structure section. - **Remote**: Access a remote folder via Cloud Storage. This feature is available only in Ultimate Edition.
Driver name	This field will be auto-filled based on your selected driver type.
Driver settings	If there are any specific driver settings, configure them here.

For details on driver properties, see File-based driver properties.

> **Tip**: When using the **Folder** option, DBeaver scans the directory up to two levels deep for JSON files. For more information, see folder structure. If you select a folder, DBeaver organizes files in schemas based on their directory structure.

Features and capabilities

Advanced SQL query capabilities

The **JSON** driver supports the full range of SQL queries:

- **Simple queries** (e.g., `SELECT * FROM table`): Data is read directly from the JSON file.
- **Complex queries** (e.g., using `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`): When a complex query is executed for the first time, the driver imports the entire JSON file into an internal database to enable advanced SQL functions. Subsequent queries run faster because the data is already imported into an internal database.

Structuring JSON files with a schema

To enhance how DBeaver interprets JSON files, you can define a schema using a DDL (Data Definition Language) file.

> For information on creating a virtual database structure from SQL files, see DDL driver.

How to create a DDL file

1. Create a `.ddl` file with the same name as your JSON file, placing it in the same directory (e.g., `employees.json` and `employees.json.ddl`).
2. Write a schema using the `CREATE TABLE` statement:

```
CREATE TABLE employees
(
    id         INTEGER,
    name       TEXT NOT NULL,
    age        INTEGER,
    department TEXT
);
```

You can also use the `WITH` clause to set a data range - add `firstRow` and `rowCount` to your `CREATE TABLE` statement:

```
WITH (firstRow = 2, rowCount = 100)
```

- `firstRow` - row number to start reading from (default: 1)
- `rowCount` - maximum number of rows to read

> **Tip**: You can also set `firstRow` and `rowCount` in the connection properties. DDL file settings take priority.

> **Important**: If the DDL file contains errors, DBeaver will ignore it.

Folder structure

When working with a folder containing multiple JSON files, DBeaver organizes them as follows:

Folder structure	Schema in DBeaver
Root files	`Default` schema
Subfolder files	Schema named after the subfolder
Files in deeper folders	Ignored

If your folder looks like this:

```
Data/
├── employees.json
├── sales.json
└── Reports/
    └── monthly.json
    └── yearly.json
```

DBeaver will create:

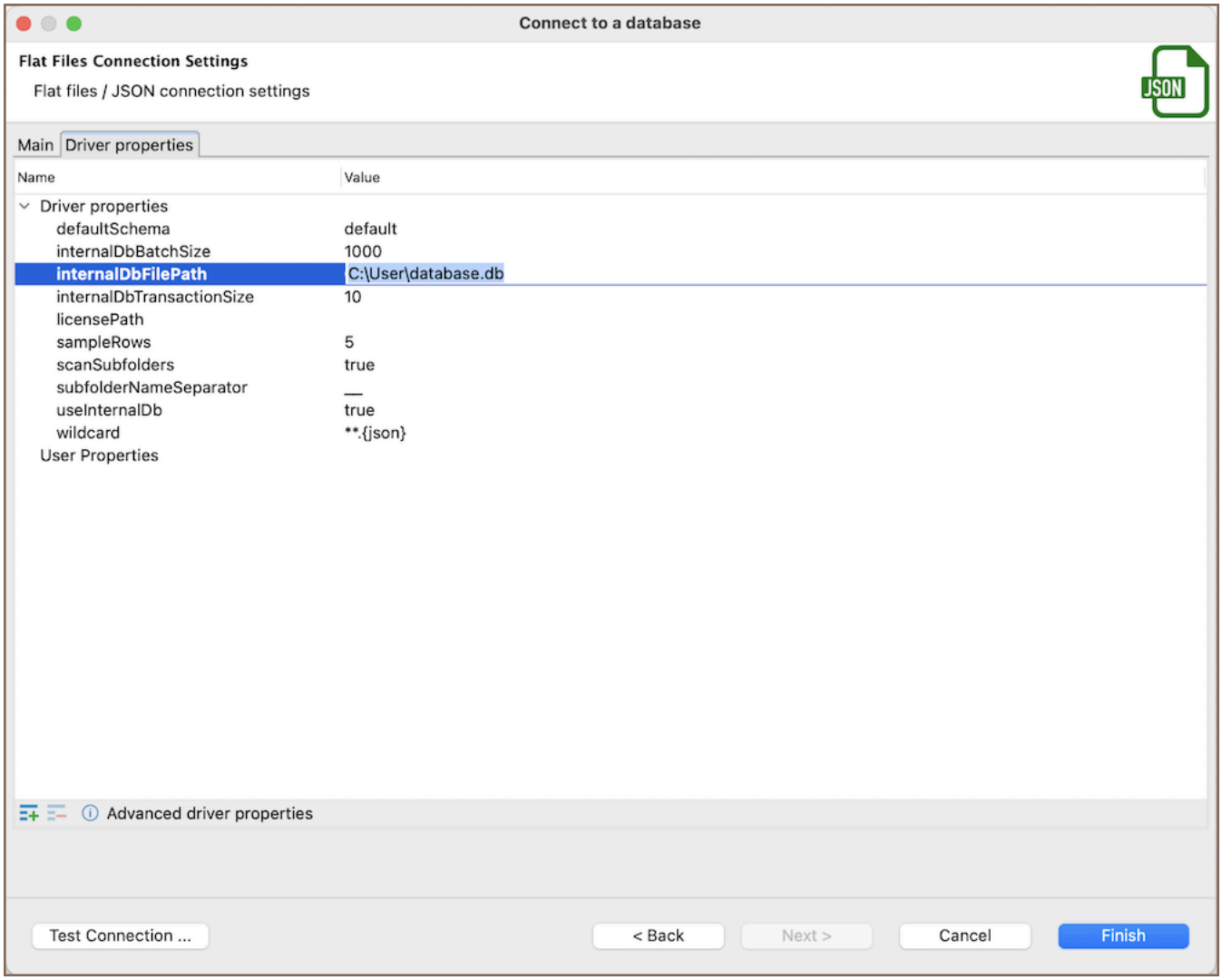
- `Default` schema: `employees`, `sales`
- `Reports` schema: `monthly`, `yearly`

> **Tip**: To focus on specific files, consider selecting individual files or folders when configuring the connection.

Internal database

When you execute a complex query (such as `WHERE`, `JOIN`, `GROUP BY`, or `ORDER BY`.), on a **JSON** file for the first time, the **JSON** driver processes the data by importing it into a temporary internal SQLite database.

By default, this internal database stores data temporarily on the disk during your session and is cleared when DBeaver restarts. To speed up queries on the same file in future sessions, you can specify the `internalDbFilePath` option in the **Driver properties** tab (e.g., `C:\User\database.db`) to reuse the processed data.



Additional features

DBeaver provides additional features compatible with **JSON** driver, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	Visual Query Builder
	Charts

Parquet

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

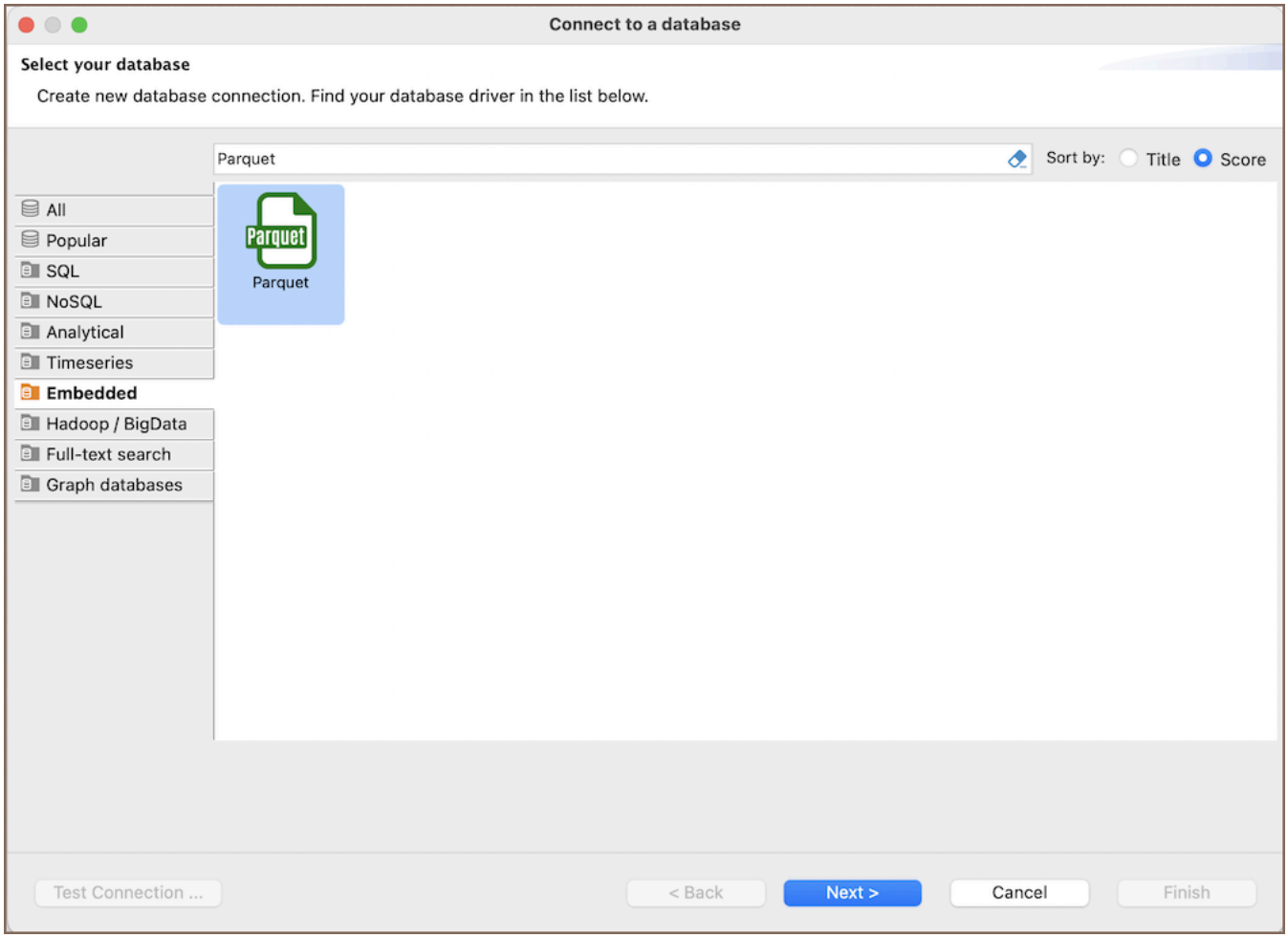
Table of contents

- Parquet Files driver connection settings
- Features and capabilities
- Advanced SQL query capabilities
- Structuring Parquet files with a schema
  - How to create a DDL file
- Folder structure
- Internal database
- Additional features
- Supported compression formats

This guide provides instructions on how to set up and use Parquet files with DBeaver. The **Parquet** driver allows you to work with Parquet data as if it were in a database. You can retrieve data and apply filters, sorting, and other operations, even combining data from multiple files.

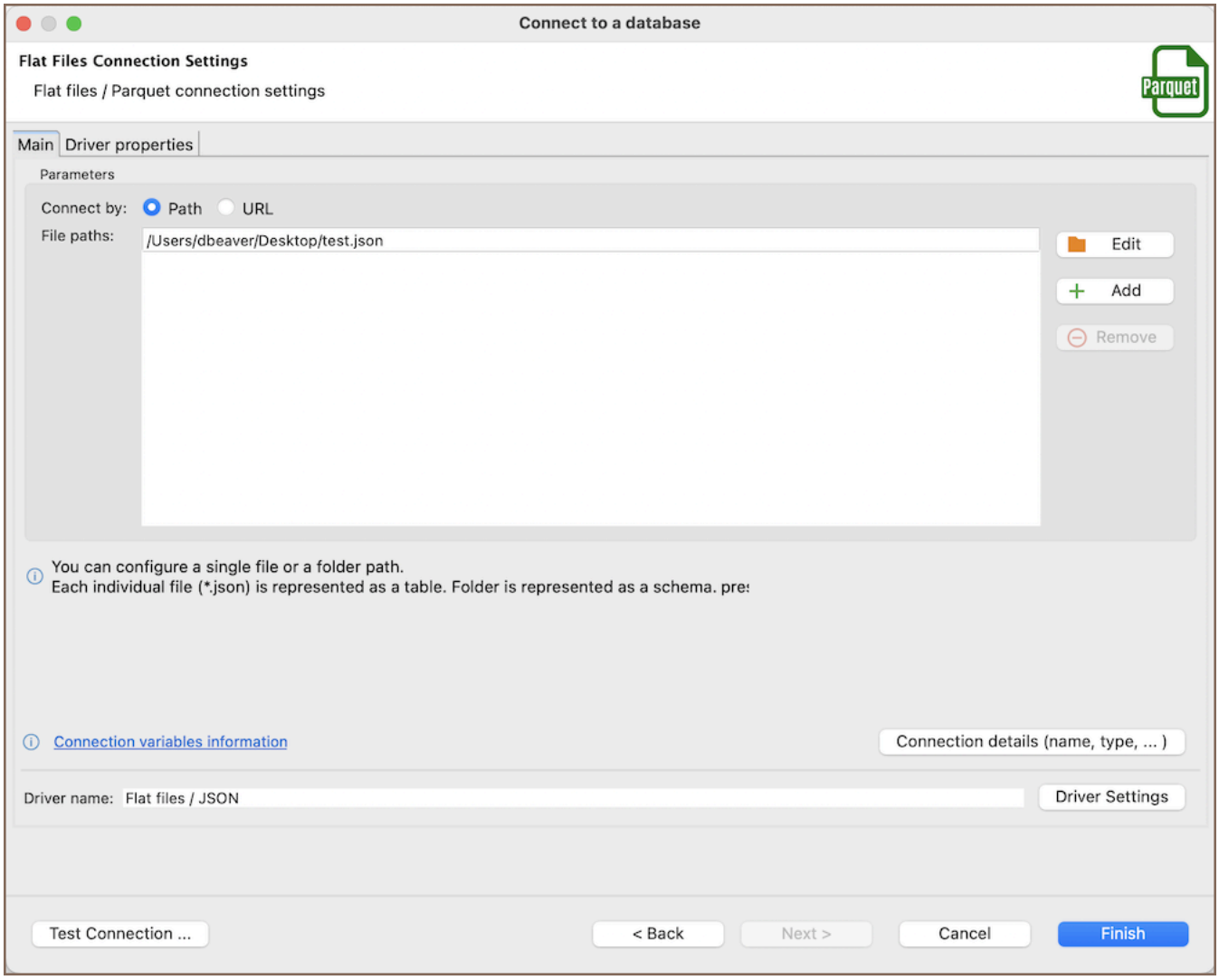
> **Important**: When using the **Parquet** driver, all connected Parquet files are read-only. To make changes, you need to update the original files outside DBeaver.

Before you start, you need to create a connection in DBeaver and select the appropriate **Parquet** driver. If you haven't done this, see our Database Connection article.



Parquet Files driver connection settings

This section describes how to set up a connection using the **Parquet** driver. The connection settings page requires the following fields:



Field	Description
Connect by (Path/URL)	Choose whether to connect using a local host path or a URL.
File paths	Specify the location of the Parquet file(s). Choose an action: - **Edit**: Modify an existing file or folder selection. - **Add**: Add a new file or folder. - **Remove**: Delete a selected file or folder. When **Editing** or **Adding**, choose from: - **File**: Select a single file. - **Folder**: Choose a directory containing multiple Parquet files. For more details, see Folder structure section. - **Remote**: Access a remote folder via Cloud Storage. This feature is available only in Ultimate Edition.
Driver name	This field will be auto-filled based on your selected driver type.
Driver settings	If there are any specific driver settings, configure them here.

For details on driver properties, see File-based driver properties.

> **Tip**: When using the **Folder** option, DBeaver scans the directory up to two levels deep for Parquet files. For more information, see folder structure. If you select a folder, DBeaver organizes files in schemas based on their directory structure.

Features and capabilities

Advanced SQL query capabilities

The **Parquet** driver supports the full range of SQL queries:

- **Simple queries** (e.g., `SELECT * FROM table`): Data is read directly from the Parquet file.
- **Complex queries** (e.g., using `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`): When a complex query is executed for the first time, the driver imports the entire Parquet file into an internal database to enable advanced SQL functions. Subsequent queries run faster because the data is already imported into internal database.

Structuring Parquet files with a schema

To control how DBeaver reads Parquet files, you can define a schema using a DDL (Data Definition Language) file.

> For information on creating a virtual database structure from SQL files, see DDL driver.

How to create a DDL file

1. Create a `.ddl` file with the same name as your Parquet file, placing it in the same directory (e.g., `employees.parquet` and `employees.parquet.ddl`).
2. Write a schema using the `CREATE TABLE` statement:

```
CREATE TABLE employees
(
    id         INTEGER,
    name       TEXT NOT NULL,
    age        INTEGER,
    department TEXT
);
```

You can also use the `WITH` clause to set a data range - add `firstRow` and `rowCount` to your `CREATE TABLE` statement:

```
WITH (firstRow = 2, rowCount = 100)
```

- `firstRow` - row number to start reading from (default: 1)
- `rowCount` - maximum number of rows to read

> **Tip**: You can also set `firstRow` and `rowCount` in the connection properties. DDL file settings take priority.

> **Important**: If the DDL file contains errors, DBeaver will ignore it.

Folder structure

When working with a folder containing multiple Parquet files, DBeaver organizes them as follows:

Folder structure	Schema in DBeaver
Root files	`Default` schema
Subfolder files	Schema named after the subfolder
Files in deeper folders	Ignored

If your folder looks like this:

```
Data/
├── employees.parquet
├── sales.parquet
└── Reports/
    └── monthly.parquet
    └── yearly.parquet
```

DBeaver will create:

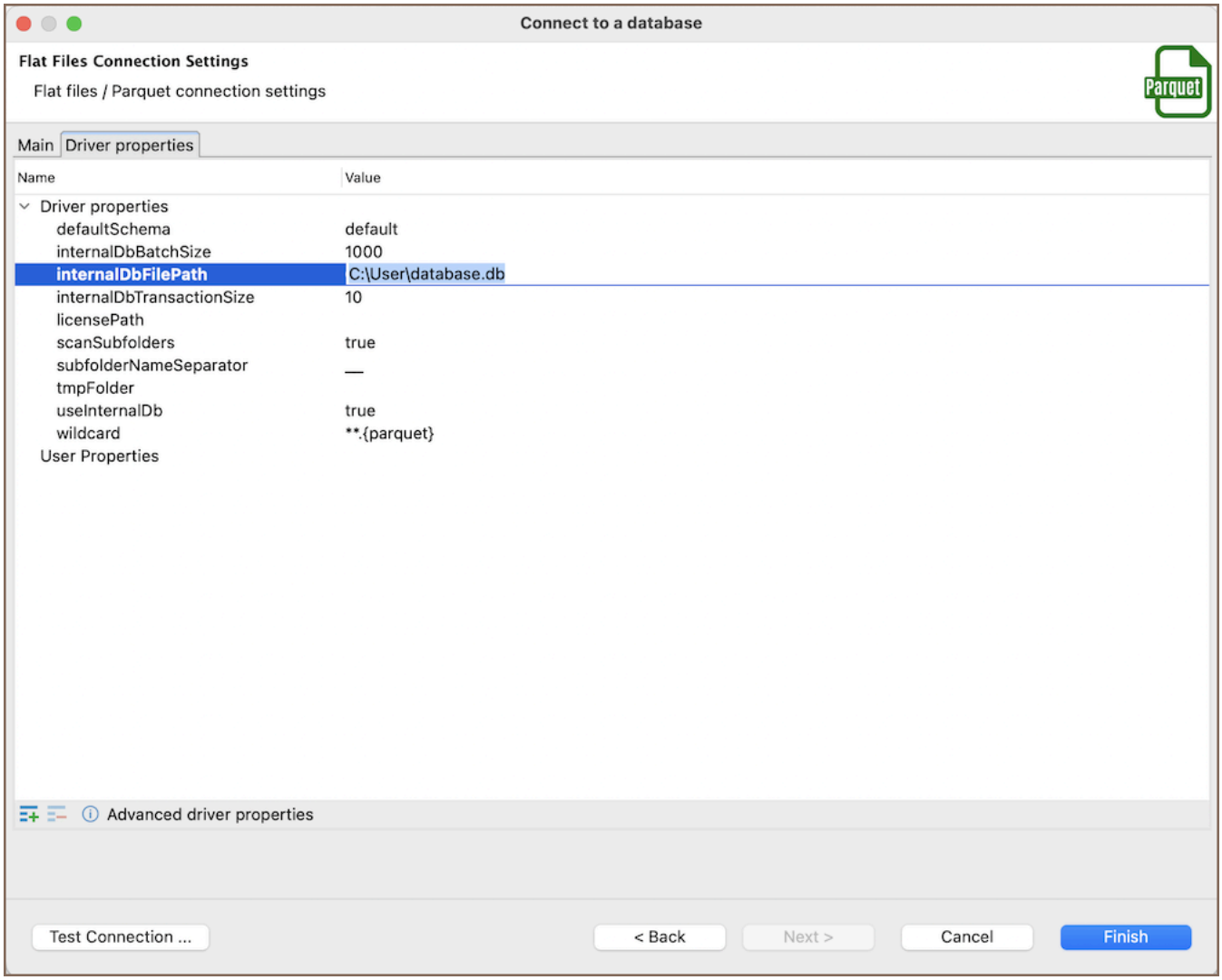
- `Default` schema: `employees`, `sales`
- `Reports` schema: `monthly`, `yearly`

> **Tip**: To focus on specific files, consider selecting individual files or folders when configuring the connection.

Internal database

When you execute a complex query (such as `WHERE`, `JOIN`, `GROUP BY`, or `ORDER BY`.), on a **Parquet** file for the first time, the **Parquet** driver processes the data by importing it into a temporary internal SQLite database.

By default, this internal database stores data temporarily on the disk during your session and is cleared when DBeaver restarts. To speed up queries on the same file in future sessions, you can specify the `internalDbFilePath` option in the **Driver properties** tab (e.g., `C:\User\database.db`) to reuse the processed data.



Additional features

DBeaver provides additional features compatible with **Parquet** driver, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	Visual Query Builder
	Charts

Supported compression formats

DBeaver supports the following compression formats for Parquet files:

Format	Supported out of the box	Notes
Zstandard (`zstd`)	Yes	Works without additional setup
Snappy	No	Add the `snappy-java` `JAR` to the driver library
`LZ4_RAW`	No	Add the `aircompressor` `JAR` to the driver library

> For details on adding a `JAR` files to the driver library, see Driver Manager.

XLSX

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

Table of contents

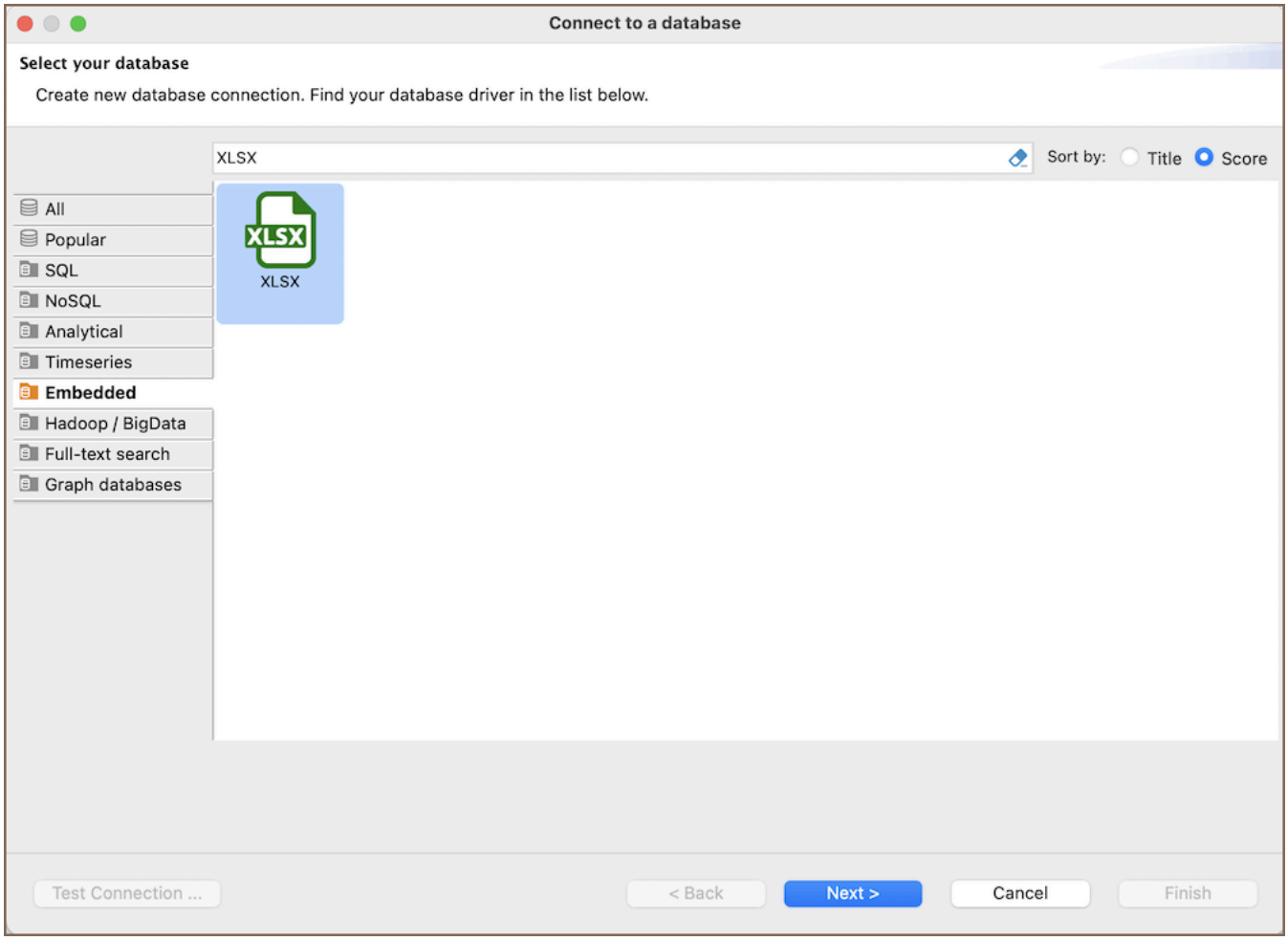
- XLSX Files driver connection settings
- Features and capabilities
- Advanced SQL query capabilities
- Structuring XLSX files with a schema

    - Why use a DDL file
    - How to create a DDL file
- Folder structure
- Internal database
- Additional features

This guide provides instructions on how to set up and use XLSX files with DBeaver. The **XLSX** driver allows you to work with XLSX data as if it were in a database. You can retrieve data and apply filters, sorting, and other operations, even combining data from multiple files.

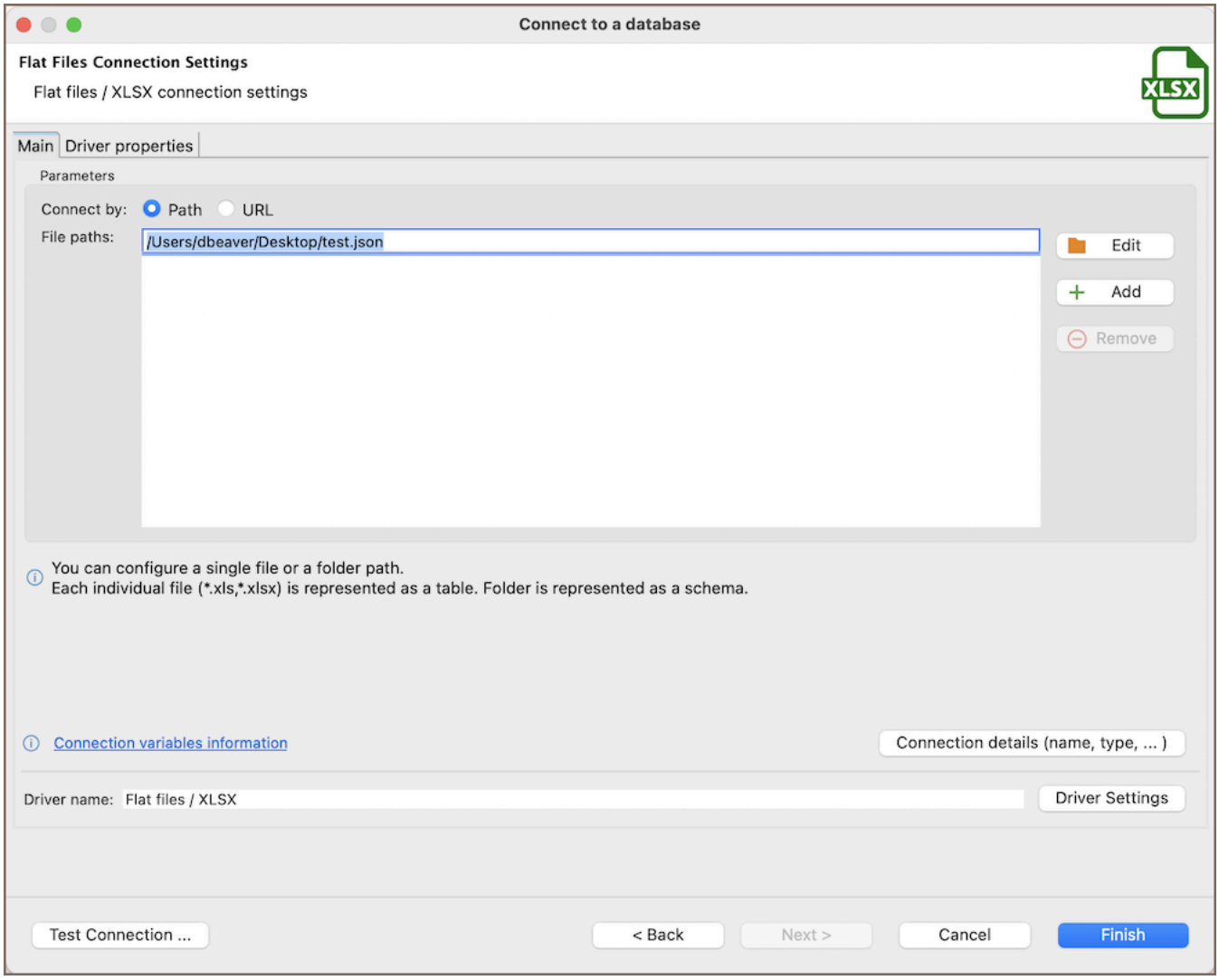
> **Important**: When using the **XLSX** driver, all connected XLSX files are read-only. To make changes, you need to update the original files outside DBeaver.

Before you start, you need to create a connection in DBeaver and select the **XLSX** driver. If you haven't done this, see our Database Connection article.



XLSX Files driver connection settings

This section describes how to set up a connection using the **XLSX** driver. The connection settings page requires the following fields:



Field	Description
Connect by (Path/URL)	Choose whether to connect using a local host path or a URL.
File paths	Specify the location of the XLSX file(s). Choose an action: - **Edit**: Modify an existing file or folder selection. - **Add**: Add a new file or folder. - **Remove**: Delete a selected file or folder. When **Editing** or **Adding**, choose from: - **File**: Select a single file. - **Folder**: Choose a directory containing multiple XLSX files. For more details, see Folder structure section. - **Remote**: Access a remote folder via Cloud Storage. This feature is available only in Ultimate Edition.
Driver name	This field will be auto-filled based on your selected driver type.
Driver settings	If there are any specific driver settings, configure them here.

For details on driver properties, see File-based driver properties.

> **Tip**: When using the **Folder** option, DBeaver scans the directory up to two levels deep for XLSX files. For more information, see folder structure. If you select a folder, DBeaver organizes files in schemas based on their directory structure.

Features and capabilities

Advanced SQL query capabilities

The **XLSX** driver supports the full range of SQL queries:

- **Simple queries** (e.g., `SELECT * FROM table`): Data is read directly from the XLSX file.
- **Complex queries** (e.g., using `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`): When a complex query is executed for the first time, the driver imports the entire XLSX file into an internal database to enable advanced SQL functions. Subsequent queries run faster because the data is already imported into internal database.

Structuring XLSX files with a schema

XLSX files don't include metadata about their structure, such as column names or data types. To enhance how DBeaver interprets these files, you can define a schema using a DDL (Data Definition Language) file.

> For information on creating a virtual database structure from SQL files, see DDL driver.

Why use a DDL file

A DDL file helps DBeaver interpret your data more accurately by defining:

- Column names
- Data types
- Optional indexes for better performance

How to create a DDL file

1. Create a `.ddl` file with the same name as your XLSX file, placing it in the same directory (e.g., `employees.xlsx` and `employees.xlsx.ddl`).
2. Write a schema using the `CREATE TABLE` statement:

```
CREATE TABLE employees (
  id INTEGER,
  name TEXT NOT NULL,
  age INTEGER,
  department TEXT
  );
  CREATE INDEX idx_employees_id ON employees (id);
```

If your XLSX file contains multiple sheets, you can define multiple `CREATE TABLE` statements in a single DDL file, one for each worksheet. The table names in the DDL file should match the worksheet names:

```
CREATE TABLE employees
(
    id         INTEGER,
    name       TEXT NOT NULL,
    age        INTEGER DEFAULT 30,
    department TEXT
);

CREATE TABLE sales
(
    sale_id  INTEGER,
    product  TEXT,
    quantity INTEGER,
    price    DECIMAL(10, 2)
);
```

You can also use the `WITH` clause to set a data range - add `sheet`, `firstRow`, and `rowCount` to your `CREATE TABLE` statement:

```
WITH (sheet = Sheet1, firstRow = 2, rowCount = 100)
```

- `sheet` - worksheet name to read from
- `firstRow` - row number to start reading from (default: 1)
- `rowCount` - maximum number of rows to read

> **Tip**: You can also set `firstRow` and `rowCount` in the connection properties. DDL file settings take priority.

> **Important**: If the DDL file contains errors, DBeaver will ignore it. You can still query the file, but column types will be inferred from the data.

Folder structure

When working with a folder containing multiple XLSX files, DBeaver organizes them as follows:

Folder structure	Schema in DBeaver
Root files	`Default` schema
Subfolder files	Schema named after the subfolder
Files in deeper folders	Ignored

> **Note**: Each file becomes a schema, and each sheet in a file becomes a table in that schema.

If your folder looks like this:

```
Data/
├── employees.xlsx
│   ├── Sheet1
```

```
│   ├── Sheet2
├── sales.xlsx
│   ├── SalesData
│   ├── Summary
└── Reports/
    ├── monthly.xlsx
    │   ├── January
    │   ├── February
    ├── yearly.xlsx
        ├── 2023
        ├── 2024
```

DBeaver will create:

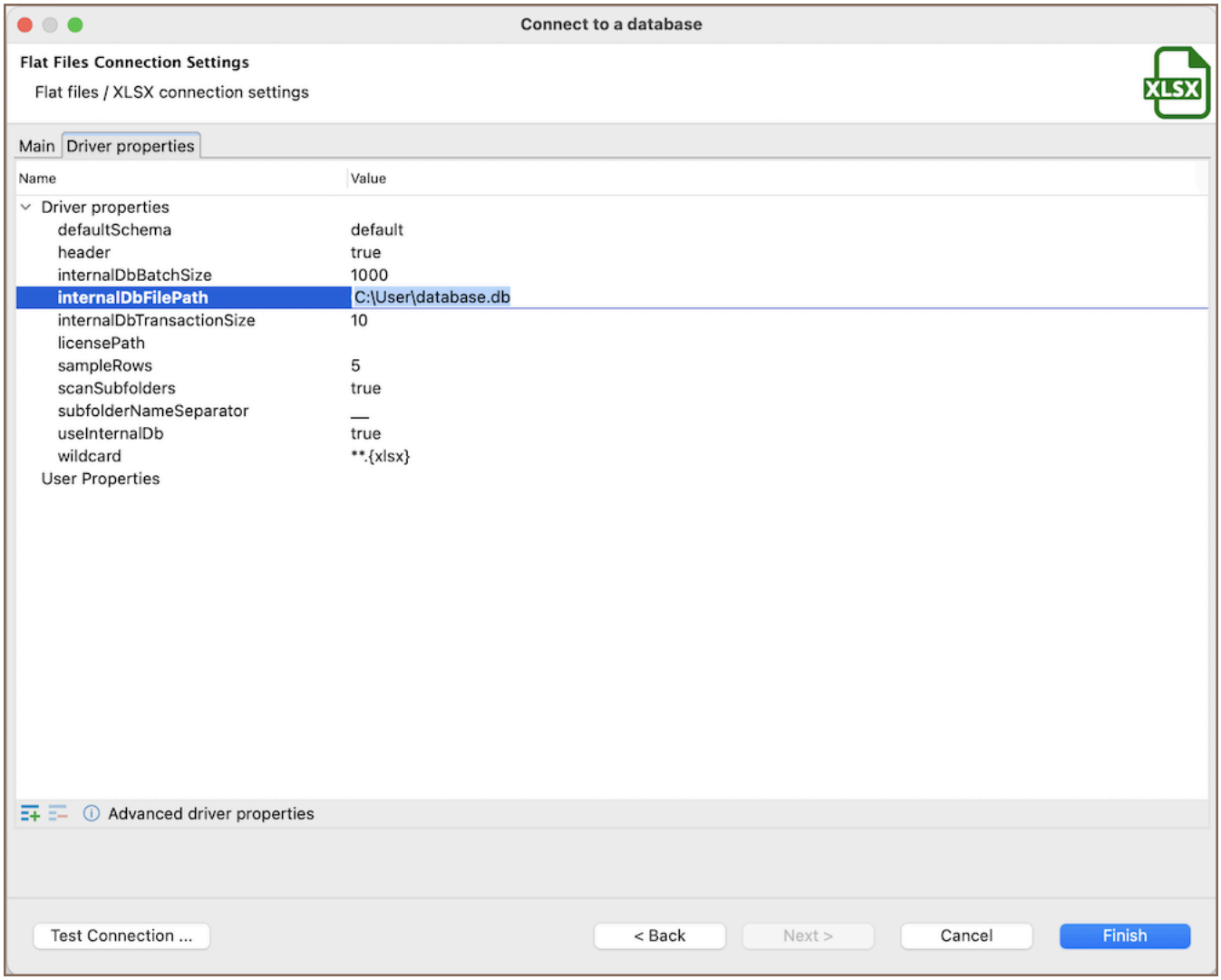
- **Schema**: `Default`
  Tables:
- `employees.Sheet1`
- `employees.Sheet2`
- `sales.SalesData`
- `sales.Summary`
- **Schema**: `Reports`
  Tables:
- `monthly.January`
- `monthly.February`
- `yearly.2023`
- `yearly.2024`

> **Tip**: To focus on specific files, consider selecting individual files or folders when configuring the connection.

Internal database

When you execute a complex query (such as `WHERE`, `JOIN`, `GROUP BY`, or `ORDER BY`.), on an **XLSX** file for the first time, the **XLSX** driver processes the data by importing it into a temporary internal SQLite database.

By default, this internal database stores data temporarily on the disk during your session and is cleared when DBeaver restarts. To speed up queries on the same file in future sessions, you can specify the `internalDbFilePath` option in the **Driver properties** tab (e.g., `C:\User\database.db`) to reuse the processed data.



Additional features

DBeaver provides additional features compatible with **XLSX** driver, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	Visual Query Builder
	Charts

XML

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

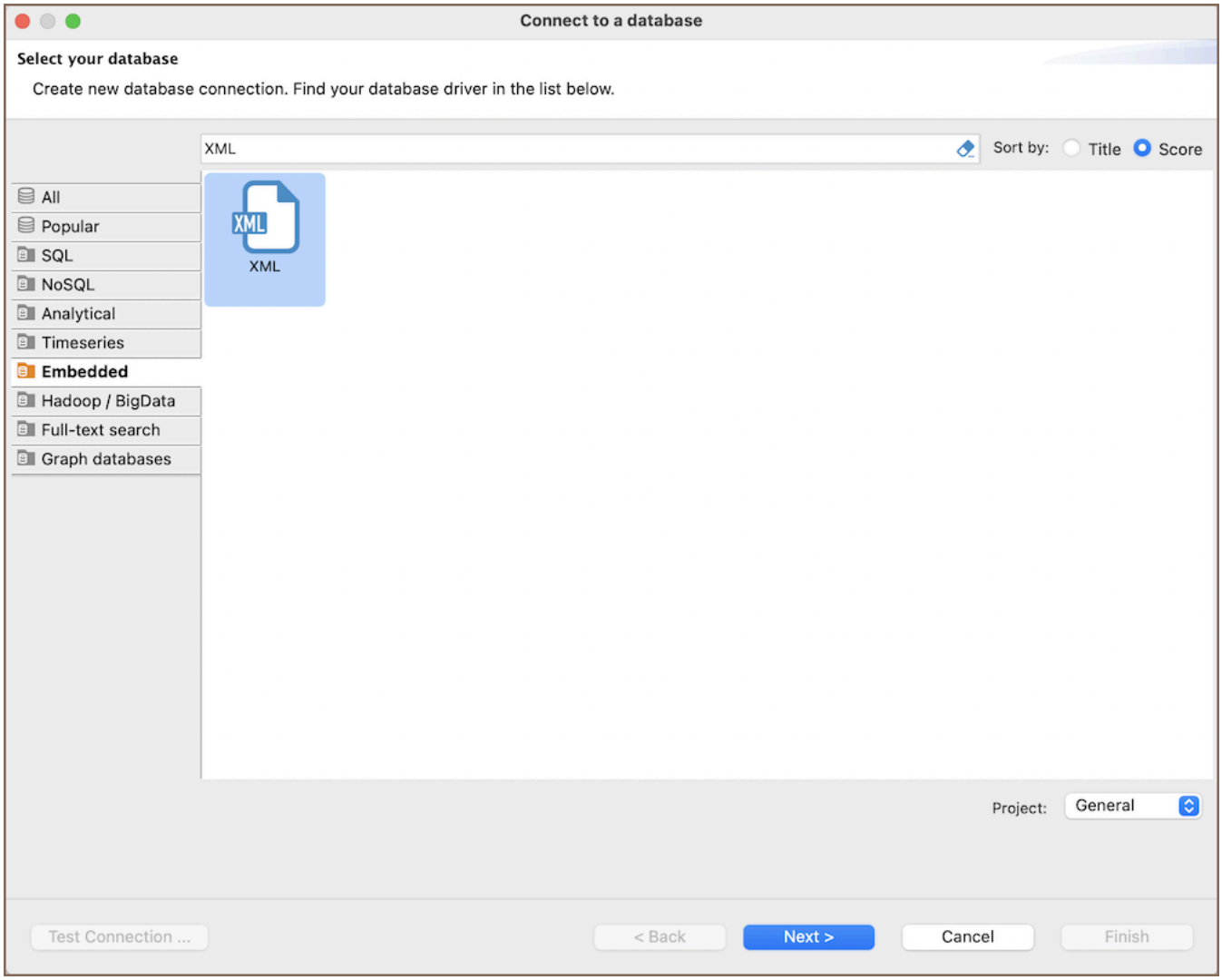
Table of contents

- XML Files driver connection settings
- Features and capabilities
- Advanced SQL query capabilities
- Structuring XML files with a schema
  - How to create a DDL file
- Folder structure
- Internal database
- Additional features

This guide provides instructions on how to set up and use XML files with DBeaver. The **XML** driver supports standard XML files, including both attribute-based and nested element-based structures. You can retrieve data and apply filters, sorting, and other operations, even combining data from multiple files.

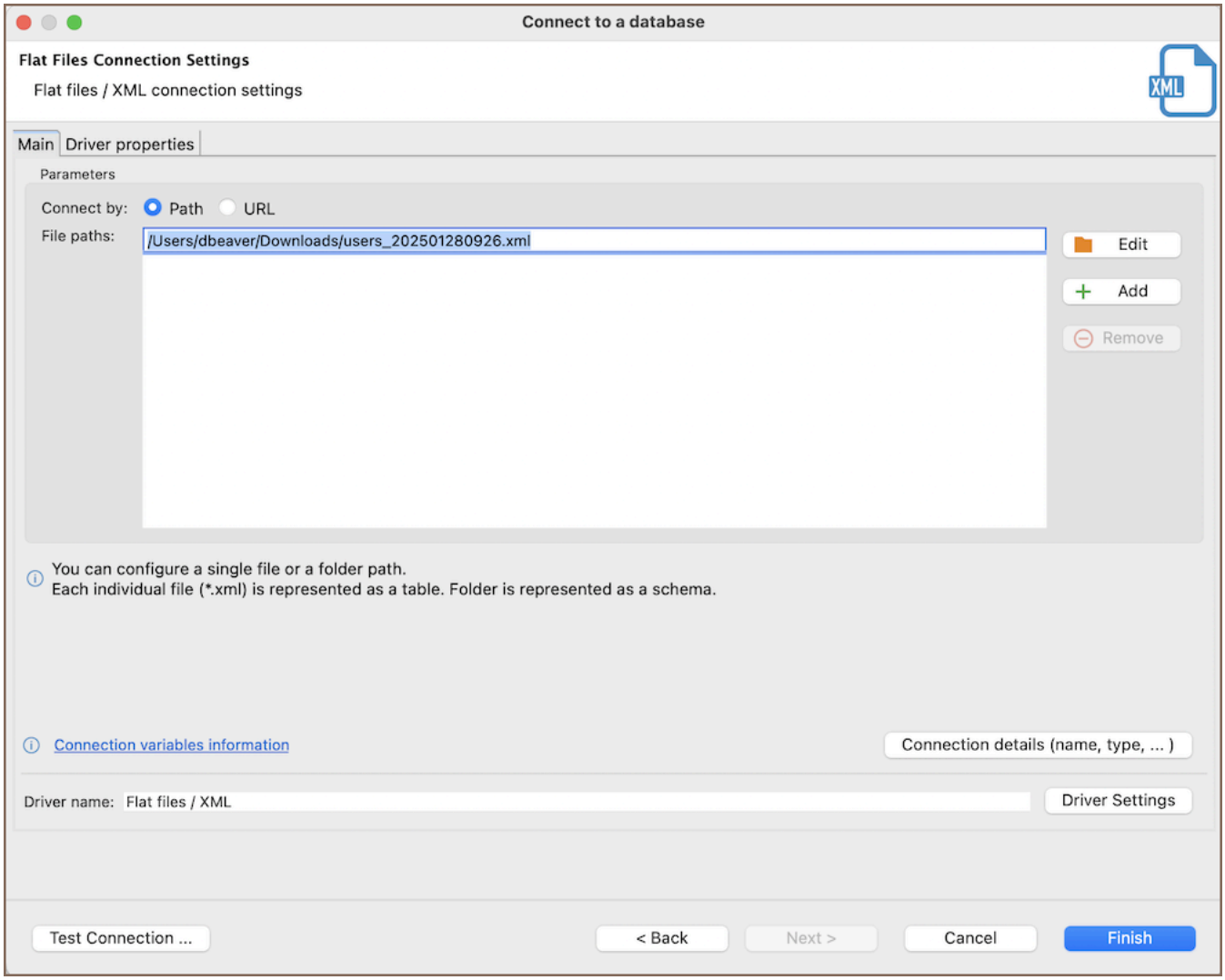
> **Important**: When using the **XML** driver, all connected **XML** files are read-only. To make changes, you need to update the original files outside DBeaver.

Before you start, you need to create a connection in DBeaver and select the appropriate **XML** driver. If you haven't done this, see our Database Connection article.



XML Files driver connection settings

This section describes how to set up a connection using the **XML** driver. The connection settings page requires the following fields:



Field	Description
Connect by (Path/URL)	Choose whether to connect using a local host path or a URL.
File paths	Specify the location of the XML file(s). Choose an action: - **Edit**: Modify an existing file or folder selection. - **Add**: Add a new file or folder. - **Remove**: Delete a selected file or folder. When **Editing** or **Adding**, choose from: - **File**: Select a single file. - **Folder**: Choose a directory containing multiple XML files. For more details, see Folder structure section. - **Remote**: Access a remote folder via Cloud Storage. This feature is available only in Ultimate Edition.
Driver name	This field will be auto-filled based on your selected driver type.
Driver settings	If there are any specific driver settings, configure them here.

For details on driver properties, see File-based driver properties.

> **Tip**: When using the **Folder** option, DBeaver scans the directory up to two levels deep for XML files. For more information, see folder structure. If you select a folder, DBeaver organizes files in schemas based on their directory structure.

Features and capabilities

Advanced SQL query capabilities

The **XML** driver supports the full range of SQL queries:

- **Simple queries** (e.g., `SELECT * FROM table`): Data is read directly from the XML file.
- **Complex queries** (e.g., using `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`): When a complex query is executed for the first time, the driver imports the entire XML file into an internal database to enable advanced SQL functions. Subsequent queries run faster because the data is already imported into an internal database.

Structuring XML files with a schema

To enhance how DBeaver interprets XML files, you can define a schema using a DDL (Data Definition Language) file.

> For information on creating a virtual database structure from SQL files, see DDL driver.

How to create a DDL file

1. Create a `.ddl` file with the same name as your XML file, placing it in the same directory (e.g., `employees.xml` and `employees.xml.ddl`).
2. Write a schema using the `CREATE TABLE` statement:

```
CREATE TABLE employees
(
    id         INTEGER,
    name       TEXT NOT NULL,
    age        INTEGER,
    department TEXT
);
```

You can also use the `WITH` clause to set a data range - add `firstRow` and `rowCount` to your `CREATE TABLE` statement:

```
WITH (firstRow = 2, rowCount = 100)
```

- `firstRow` - row number to start reading from (default: 1)
- `rowCount` - maximum number of rows to read

> **Tip**: You can also set `firstRow` and `rowCount` in the connection properties. DDL file settings take priority.

> **Important**: If the DDL file contains errors, DBeaver will ignore it.

Folder structure

When working with a folder containing multiple XML files, DBeaver organizes them as follows:

Folder structure	Schema in DBeaver
Root files	`Default` schema
Subfolder files	Schema named after the subfolder
Files in deeper folders	Ignored

If your folder looks like this:

```
Data/
├── employees.xml
├── sales.xml
└── Reports/
    └── monthly.xml
    └── yearly.xml
```

DBeaver will create:

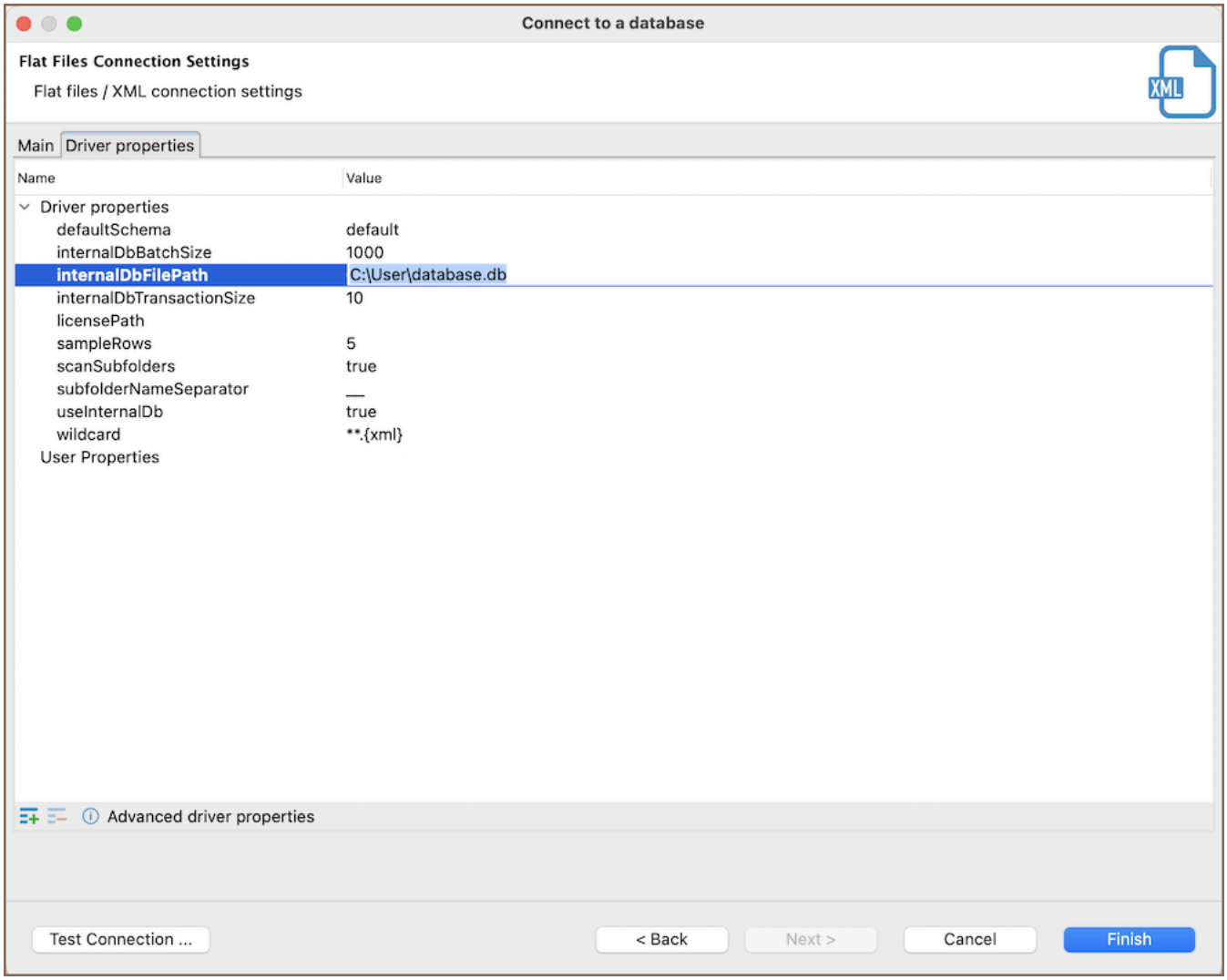
- `Default` schema: `employees`, `sales`
- `Reports` schema: `monthly`, `yearly`

> **Tip**: To focus on specific files, consider selecting individual files or folders when configuring the connection.

Internal database

When you execute a complex query (such as `WHERE`, `JOIN`, `GROUP BY`, or `ORDER BY`.), on an **XML** file for the first time, the **XML** driver processes the data by importing it into a temporary internal SQLite database.

By default, this internal database stores data temporarily on the disk during your session and is cleared when DBeaver restarts. To speed up queries on the same file in future sessions, you can specify the `internalDbFilePath` option in the **Driver properties** tab (e.g., `C:\User\database.db`) to reuse the processed data.



Additional features

DBeaver provides additional features compatible with **XML** driver, but not exclusive to it:

Category	Feature
Data Transfer	Data Export
Data Visualization	Visual Query Builder
	Charts

Graph

Neo4j

> **Note**: This driver is available in Lite, Enterprise, and Ultimate editions only.

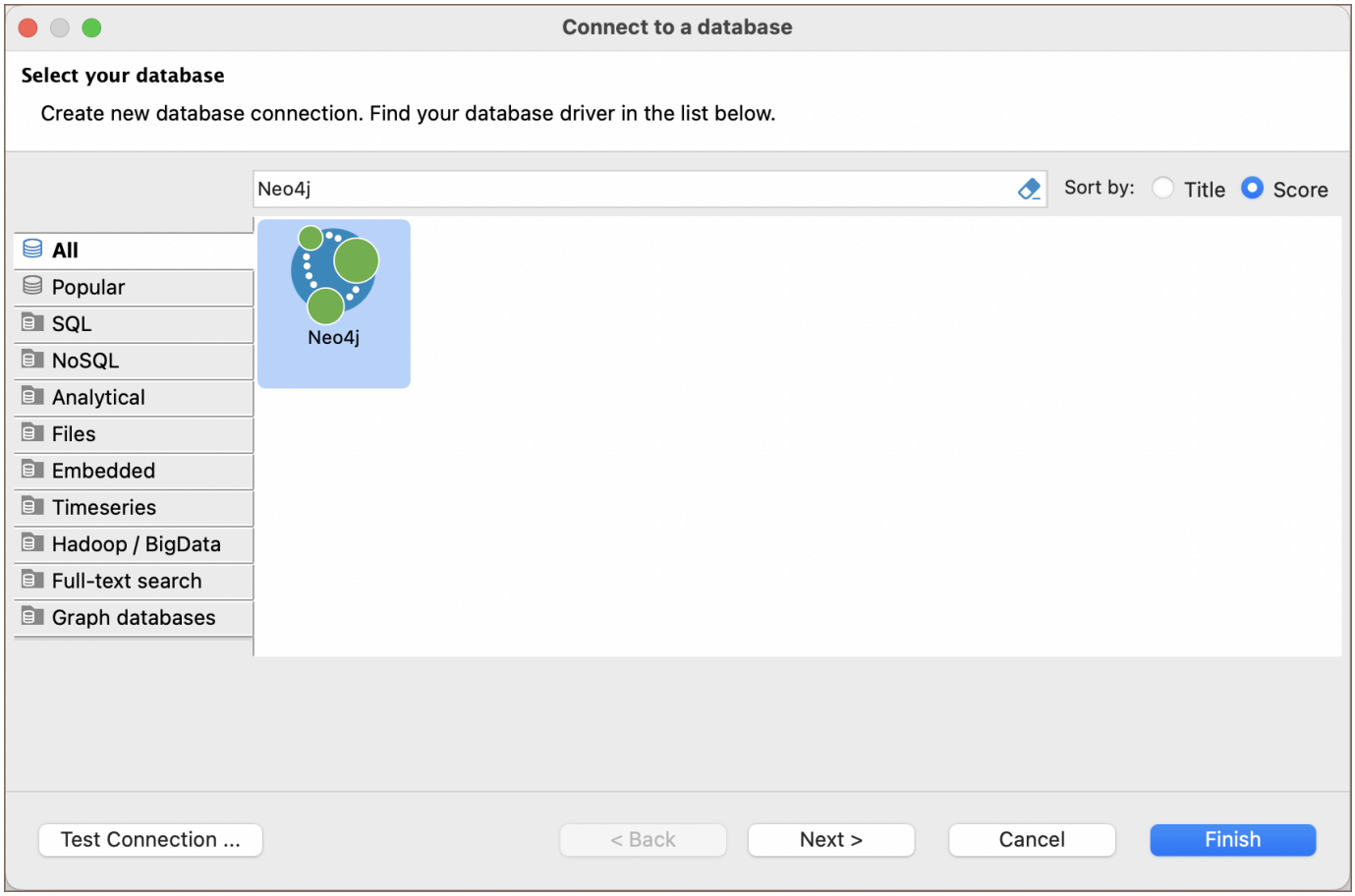
Table of contents

- Neo4j specialty
- Setting up
- Neo4j connection settings
- Connection details
- Secure Connection Configurations
- Secure Storage with Secret Providers
- Powering Neo4j with DBeaver
- Neo4j database objects
- Neo4j features in DBeaver

This guide provides instructions on how to set up and use Neo4j with DBeaver.

First, you must start creating a connection in DBeaver and select Neo4j. If you haven't done this yet, see our Database Connection article.

DBeaver connects to Neo4j using a dedicated JDBC driver. It supports all modern Neo4j versions.



Neo4j specialty

Neo4j is a graph database built for working with connected data. It stores data as **nodes** and **relationships**, not tables. This format is great for things like networks, hierarchies, and recommendations.

Neo4j uses **Cypher** – a SQL-like language designed for graph patterns. It's optimized for exploring relationships, not just rows.

> For more, see the Neo4j documentation.

Setting up

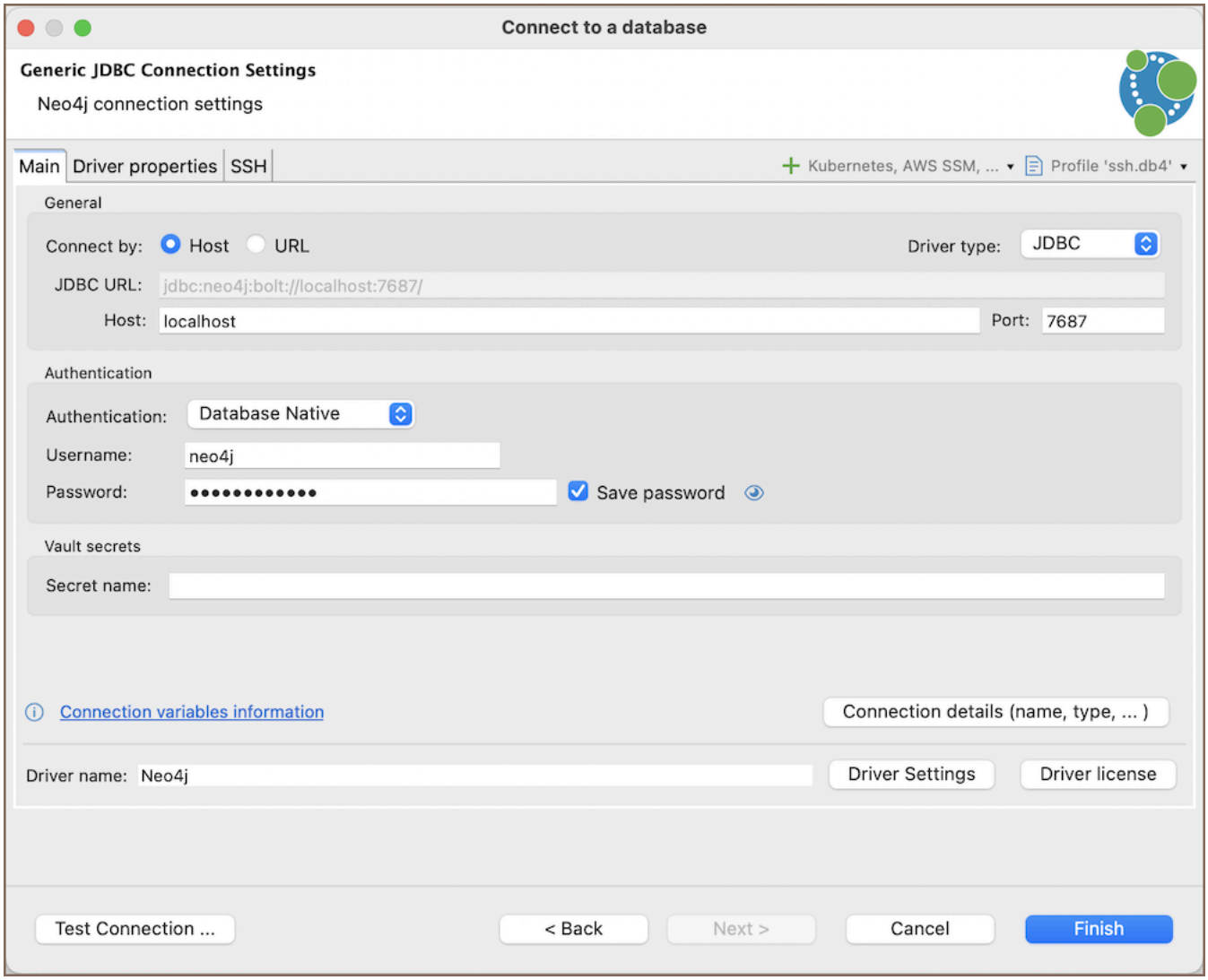
This section provides an overview of DBeaver's settings for establishing a direct connection and the configuration of secure connections using SSH, proxies, AWS SSM and Kubernetes.

> **Tip**: When you connect to Neo4j for the first time, DBeaver will prompt you to download the driver. Click the value under the **Version** column to open a dropdown menu and choose the version you need. For more on changing or updating the version, see how to update driver version.

Neo4j connection settings

In this subsection, we will outline the settings for establishing a direct connection to a Neo4j database using DBeaver. Correctly configuring your connection ensures seamless interaction between DBeaver and your Neo4j database.

The connection settings page requires you to fill in specific fields to establish the initial connection.



Field	Description
Connect by (Host/URL)	Choose whether you want to connect using a host or a URL.
URL	If you are connecting via URL, enter the URL of your Neo4j database here. This field is disabled if you're connecting via the host.
Host	If you are connecting via host, enter the host address of your Neo4j database here.
Port	Enter the port number for your Neo4j database. The default Neo4j port is 7687.
Authentication	Choose the type of authentication you want to use for the connection. For detailed guides on authentication types, please refer to the following articles: - Username/password Authentication - Password from shell command - DBeaver Profile Authentication You can also read about security in DBeaver PRO.
Connection Details	Provide additional connection details if necessary.
Driver Name	This field will be auto-filled based on your selected driver type.
Driver Settings	If there are any specific driver settings, configure them here.

Connection details

The **Connection Details** section in DBeaver allows you to customize your experience while working with Neo4j database. This includes options for adjusting the **Connections View**, setting up **Security measures**, applying **Filters**, configuring **Connection Initialization** settings, and setting up **Shell Commands**. Each of these settings can significantly impact your database operations and workflow. For detailed guides on these settings, please refer to the following articles:

- Connection Details Configuration
- Connections
- Security Settings Guide
- Filters Settings Guide
- Connection Initialization Settings Guide
- Shell Commands Guide

Secure Connection Configurations

DBeaver supports secure connections to your Neo4j database. Guidance on configuring such connections, specifically **SSH**, **Proxy**, **Kubernetes**, and **AWS SSM** connections, can be found in various referenced articles. For a comprehensive understanding, please refer to these articles:

- **SSH Configuration**
- **Proxy Configuration**
- **Kubernetes Configuration**
- **AWS SSM**

Secure Storage with Secret Providers

DBeaver supports various cloud-based secret providers to retrieve database credentials. For detailed setup instructions, see Secret Providers.

Powering Neo4j with DBeaver

DBeaver provides the ability to view and manage databases.

Neo4j database objects

DBeaver lets you view and work with Neo4j database objects. It supports key metadata types, so you can explore and manage:

- Databases
- Tables (Nodes)
  - Columns (Properties)
  - Constraints
  - Indexes
  - Foreign Keys (Relationships)
- Data types

Neo4j features in DBeaver

In addition, Neo4j includes a range of features designed to enhance the user experience and efficiency when working with Neo4j databases. Here are some of the key features:

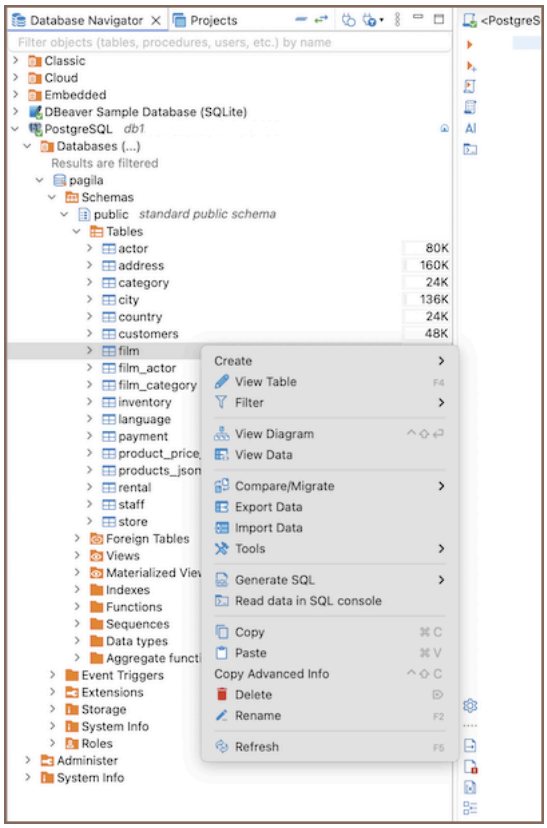
Category	Feature
Data Transfer	Data Export
Browsing collections	JSON documents

Connections

Table of contents

- Navigation
- Browse database objects
- Filter database objects
- Group database objects in folders
- Simple and advanced view
- Highlight object from editor
- Show all databases
- Toolbar
- View menu
- Context menu
- Filtering options
- Connections preferences
- Additional capabilities
- Drag and drop from Connections
- Shortcuts

Connections is the main tool in DBeaver for working with database structure and data. It shows a tree of objects - like connections, schemas, tables, and views - based on your database type.



> **Tip**: If you don't see Connections, go to **Window -> Connections** to open it.

Navigation

Browse database objects

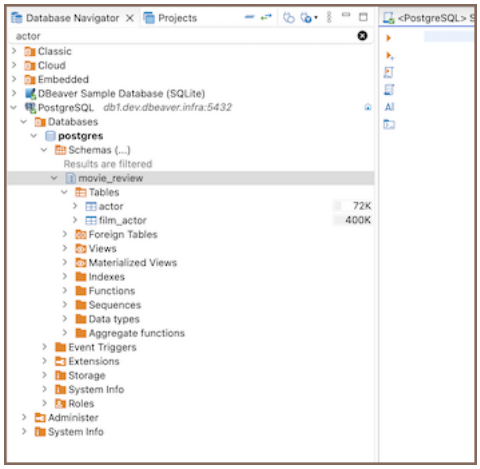
Use the tree in **Connections** to view and explore databases. Each connection shows its own set of objects - schemas, tables, views, procedures, and more. Expand a connection to access its contents. Click any object to open it in the appropriate editor or viewer.

Filter database objects

Use the filter bar above the Connections tree to search and filter database objects.

Start typing to highlight matching items. The filter supports partial matches and updates results instantly.

You can also filter using the view options or context menu.

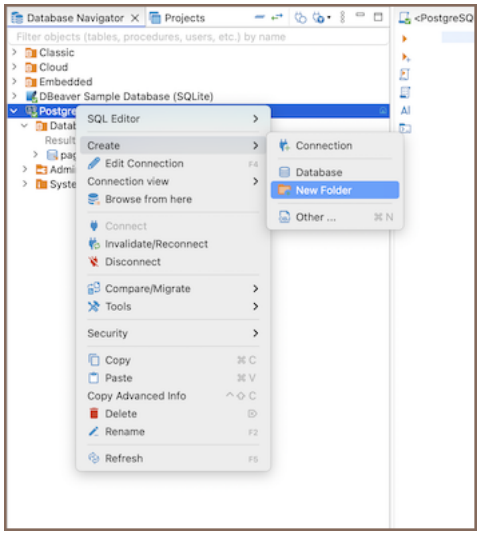


> **Tip**: The filter applies only to expanded nodes. For details, see Filter database objects.

Group database objects in folders

Use folders to organize connections:

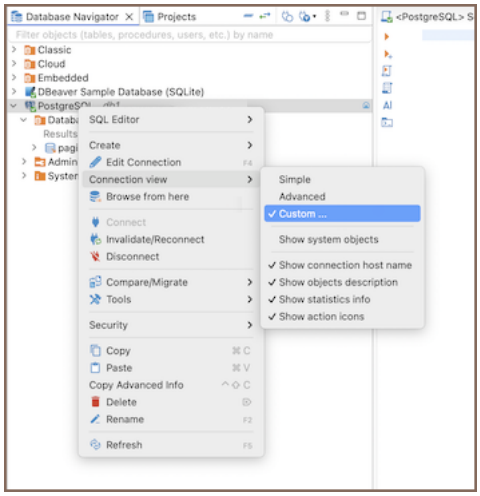
- Right-click in the tree and select **Create -> New Folder** to add a new folder.
- Drag and drop connections or objects into folders to group them.



> **Tip**: You can also create subfolders within existing folders to further organize your database objects.

Simple and advanced view

Switch between **Simple**, **Advanced**, or **Custom** view to control how much detail is shown in **Connections**.



Use this option to simplify the tree or show all available objects, depending on your needs.

> **Tip**: Each connection shows its current view mode as an indicator to the right of the connection name:
>
> - Simple
> - Advanced
> - Custom

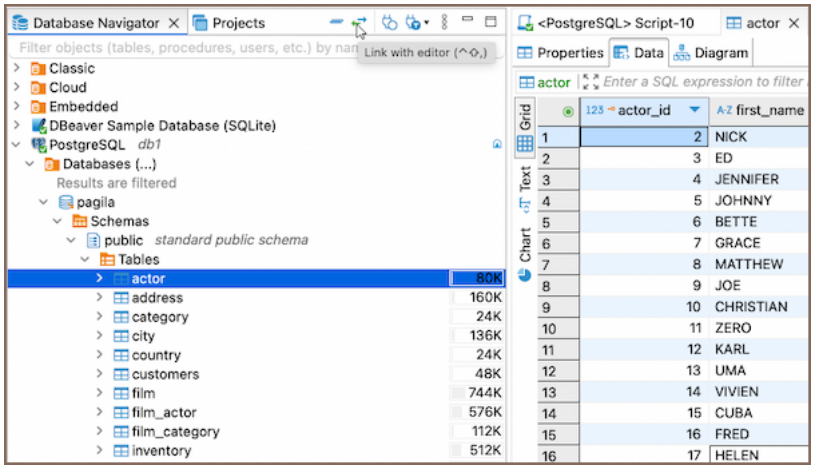
> For details, see Simple and advanced view.

Highlight object from editor

Use **Link with editor** button to highlight the object currently opened in the SQL Editor or Data Editor in **Connections**.

When enabled, **Connections** selects and scrolls to the schema that contains the active object.

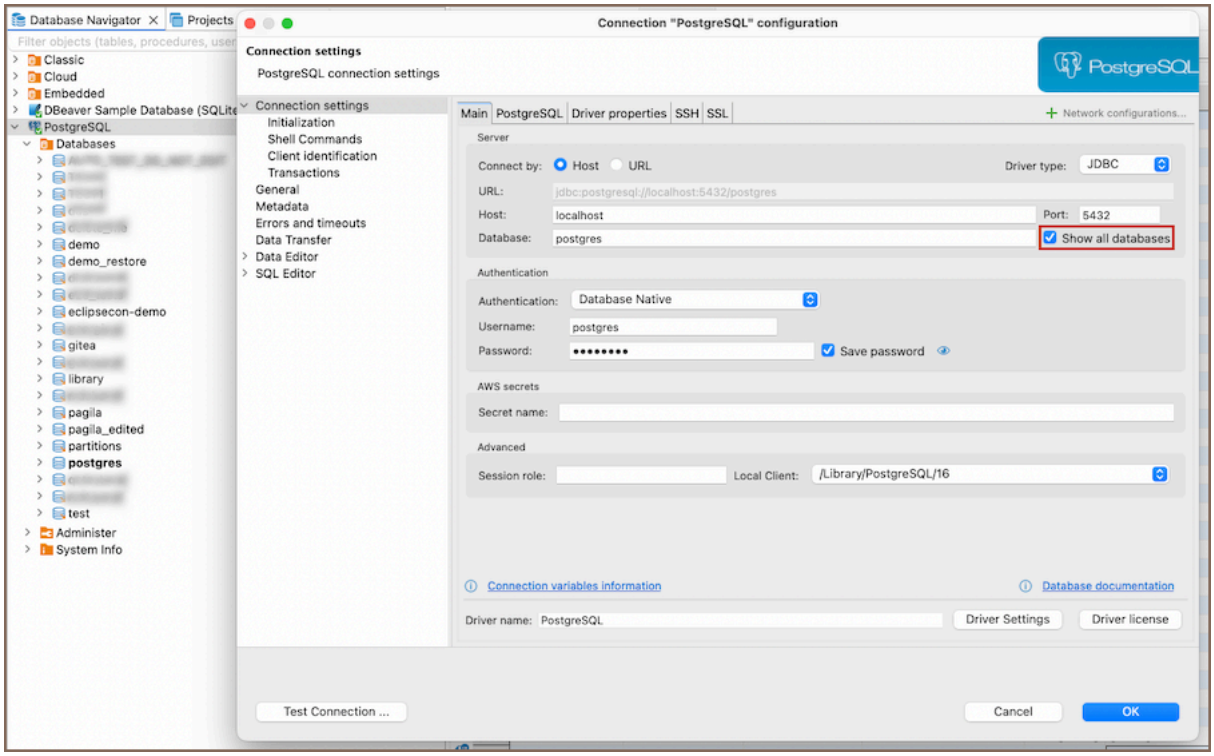
Toggle this option from the top toolbar of the **Connections**.



Show all databases

Enable **Show all databases** in the connection settings to display all databases in **Connections**.

Some databases (such as PostgreSQL) may show only the active database or a limited set of schemas by default.



Toolbar

The toolbar contains buttons for the most basic and frequently used commands:

Icon	Menu item	Description
	Collapse All	Collapses all tree items to the root level.
	Link with editor	Syncs the selected item in active editor with schema in the Connections.
	Show all connections/Show connected only	Filters the tree to show either all connections or only connected ones.
	Filter mode	Cycles through object filter levels: connections, catalogs and schemas, or individual objects. Learn more
	View menu	Opens additional options. Learn more

View menu

Click the **View Menu** button ( ) in the upper-right corner of the **Connections** panel to open additional options.

Icon	Menu item	Description
	New Database Connection	Opens the connection wizard.
	Driver Manager	Opens the driver manager for configuring drivers.
	Active Project	Switches between projects.
	New Folder	Creates a new folder in the current project.
	Collapse All	Collapses all nodes in the tree.
	Link with editor	Syncs the selected object with the active editor tab.
	Preferences	Opens DBeaver settings.
	Refresh Projects	Reloads all projects and folders.

> **Tip**: Use the **View Menu** for project-level actions and layout options - it's different from the context menu, which changes based on the selected object.

Context menu

Right-click an object in the **Connections** tree to open its context menu. The menu shows different items depending on the database type and selected object.

Menu item	Description
Create	Creates objects like connection, folder, and other.
Open folder	Opens the selected folder in a new view.
Copy	Copies the selected object.
Paste	Pastes the copied object into the selected folder or node.
Delete	Deletes the object. **Warning**: This action removes the object from the database or file system. It can't be undone.
Rename	Opens a dialog to rename the object.
Refresh	Refreshes the selected node and its children.
Connect	Connects to the database.
Invalidate/Reconnect	Reconnects if the connection is broken.
Disconnect	Disconnects from the database.
SQL Editor	Opens new script, last edited script for this connection.
Edit connection	Opens the connection settings.
Customize view	Lets you switch between **Simple**, **Advanced**, or **Custom** views. Read more
View `[object]`	Opens the object in a viewer.
Edit `[object]`	Opens the object in an editor.
Create new `[object]`	Starts creating a new object.
Filter	Opens filtering options for the selected object. Learn more
Copy Advanced Info	Copies the object's full name.
Read data in SQL console	Opens a console to preview data from the object.
Compare/Migrate	Opens the export wizard.

Generate SQL	Opens options to generate SQL statements:
	`- SELECT`
	`- INSERT`
	`- UPDATE`
	`- DELETE`
	`- MERGE`
	`- DDL`

Export Data	Opens the export wizard.
Import Data	Opens the import wizard.

Menu item	Description
Tools	Opens tools like Dashboards, Tasks, Backup and restore.

> **Tip**: The available menu items change depending on the selected object and database.

Filtering options

Use filtering options to control which objects are visible in the Connections.

To open filtering options, right-click an object and select **Filter**.

Available actions:

- **Hide** `[object]` - Exclude the selected object from the view.
- **Show only** `[object]` - Show only the selected object.
- **Configure** `[object]` **filter** - Set custom filtering rules.
- **Toggle filter** - Enable or disable the filter.
- **Clear filter** - Remove all filters.

> For information on filter configuration, see Configure filters.

Connections preferences

You can control how **Connections** behaves and displays objects. These settings are available in **Window -> Preferences -> User Interface -> Connections**.

Setting name	Description
Show connection host name	Show target database host name or tunnel host name next to the connection name
Show objects description	Shows object (table, column, etc.) description next to it
Show statistics info	Show statistics (e.g. table size) on the right side of navigator
Show action icons	Show action icons for database elements
Show placeholders for special folders	Show special folders (e.g. Scripts) even if they were not created yet
Folders first	Show folders before regular elements
Show child count in Connections	Display the number of children for expanded folders in the Connections tree
Order elements alphabetically	Order navigator nodes (not folders) alphabetically
Case-insensitive alphabetical sorting	Order navigator nodes without case sensitivity, for example, `A,a,B,b`
Set connection color for all elements	Color all elements according to connection type, otherwise set color only for connection node itself
Show object modifiers in tree	Show object modifiers (e.g. column data type) in the tree right after the name
Show tooltips	Show tooltips in Connections views
Show file contents in tooltips	Show file (e.g. SQL script) contents in tooltips
Show grid lines in lists	Displays grid lines between rows and columns in navigator and editor lists for better readability
Double-click on node	- **Open Properties** - opens the object properties tab - **Expand / Collapse** - expands or collapses the selected node in the tree
Double-click on connection	- **Open Properties** - opens the connection properties - **Connect / Disconnect** - connects or disconnects from the database - **Open SQL Editor** - opens the SQL Editor - **Open new SQL Editor** - opens a new SQL Editor tab - **Expand / Collapse** - expands or collapses the connection node
Default editor page	- **Last opened** - opens the editor on the page that was active last time - **Properties** - opens the Properties page - **Data** - opens the Data tab - **Diagram** - opens the Diagram tab
Expand Connections tree on connect	Expand the Connections tree for active elements on connect
Save Connections filter	Saves object filter specified in the Connections view between application restarts
Elements fetch size	Children elements fetch size for long lists. Rest of elements can be read by double clicking on the last element
Restore Connections state up to depth	Restore Connections state on startup up to depth

> **Tip**: You can also configure these settings through preference files without opening the app. For details, see Manage application preferences.

Additional capabilities

Drag and drop from Connections

You can drag objects from **Connections** into the **SQL Editor** to quickly add their names to a query.

> **Note**: After you drop an object, it stays selected in the editor. If you run the query right away, only the selected part runs - which often causes an error because it's not a complete query. Click in the editor to deselect the object before running the query.

Shortcuts

You can navigate, filter, and edit objects faster with keyboard shortcuts.

> For a full list, see Shortcuts.

Data Editor

Table of contents

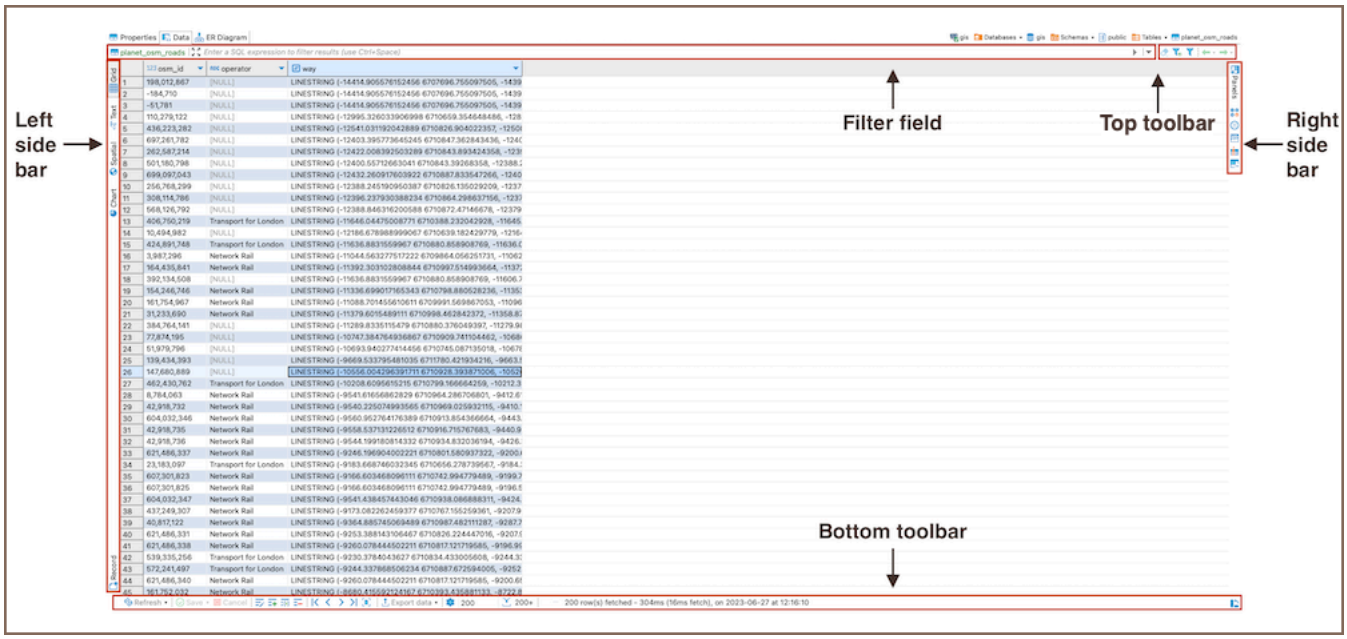
- Top toolbar
- Left sidebar
- Right sidebar
- Bottom toolbar
- Table status indicators
- Column context menu
- Cell context menu
- Additional features
- Disable metadata queries

The Data editor allows to view and edit data of a database table or view. The Data editor appears:

- as the **Data** tab of the Database Object Editor, which is only available for tables and views;
- as the **Results** tab when you run a custom SQL query in SQL Editor.

The central part of the Data editor is the data grid. The editor also provides top toolbar, bottom toolbar, left sidebar, right sidebar and a filter field:

> To configure how data is displayed and edited, see Data Editor preferences.



> **Tip**: To learn about ways to navigate data in the table data, see Navigation article.

Top toolbar

The top toolbar contains the following buttons:

Button	Name	Description
	Remove all filters/orderings	Removes all filters and orderings applied to the data in the filter field.
	Custom Filters	Opens the Result Set Order/Filter Settings window. See Data Filters article for more information.
	Forward and backward - history navigation buttons	Navigate forward and backward in the Data Editor history, see the history section of our article for more information.

Left sidebar

The left sidebar contains the following tabs:

Button	Name	Description
Grid	**Grid**	Switches to the grid view of data.
Text	**Text**	Switches to the plain text view of data.
Spatial	**Spatial**	Switches to the spatial view. For more details, see the GIS data article.

Button	Name	Description
Chart	**Chart**	Switches to the chart view. For more details, see Charts.
Record	**Record**	- Same as pressing `Tab` - Switches the positions of rows and columns so that the columns appear as rows, and the rows hide in one **Value** column. See details in the Table vs. Record Views section of our article.
JSON	**JSON** ★	Switches to the JSON editor view. Supports structured editing and navigation.
XML	**XML** ★	Switches to the XML editor view. Available for supported databases and drivers.

Right sidebar

The right sidebar contains the following tabs (see the Panels for more information):

Button	Name	Description
Panels	**Panels**	Opens panels on the right side of the Data Editor.
	Calc	Opens the result cells calculation panel (`SUM`, `MAX`, `AVG`, etc.).
	Grouping	Opens grouping panel window tools.
	Metadata	Opens Metadata panel.
	References	Opens References panel.
	Value	Opens Value Viewer.
	Result Details/Query Trace	Opens the Result Details or Query Trace panel depending on the database.

> **Note**: The **Result Details** panel is compatible with specific databases such as BigQuery, while the **Query Trace** panel is available for Cassandra and Yugabyte CQL. For guidance on enabling **Query Trace**, see Query Trace Panel article.

Bottom toolbar

The bottom toolbar provides the following buttons:

Button	Name	Description
	Refresh	Refreshes the whole results set, including all items that are not visible on the screen, while its dropdown option allows to customize the refresh frequency over a specific period.
Save	**Save**	Saves all unsaved changes to the data such as adding, duplicating, deleting rows, inline editing of values. See the Data Viewing and Editing article for information.
Cancel	**Cancel**	Discards all unsaved changes to the data.
	Edit cell value in separate dialog/editor	Opens the cell in focus for editing in a separate editor or dialog box. See details in the Cell Editor section of our article.
	Add new row	Adds a new empty row below the current row, see details in the Adding, Copying and Deleting Rows section of our article.
	Duplicate current row	Copies the current rows and pastes the copy below the current row, see details in Adding, Copying and Deleting Rows section of our article.
	Delete current row	Colors the rows in focus in red to mark them for deletion, see details in the Adding, Copying and Deleting Rows section of our article.
	Move to first row	Moves the focus (highlighting) from the current to the first row of the table.
	Move to previous row	Moves the focus (highlighting) from the current to the previous row of the table.
	Move to next row	Moves the focus (highlighting) from the current to the next row of the table.
	Move to last row	Moves the focus (highlighting) from the current to the last row of the table.
	Fetch all data	Fetches the whole result set making it ready for display. See the Scrolling Results Page section of our article.
	Configure	Opens a menu with settings.
200	**Result-set fetch size**	Displays the initial fetch number of rows in the result set on loading or refreshing.
	Calculate total row count	Calculates the total number of rows in the table.

To learn how many rows the table data contains, click the **Calculate total row count** button. The number of rows appears in a status field next to the button: 203

> **Note**: Some of these buttons may be disabled and may not work if you are using a read-only connection, connecting to a read-only database or if you see the result of a complex query, such as joining two or more tables.

Table status indicators

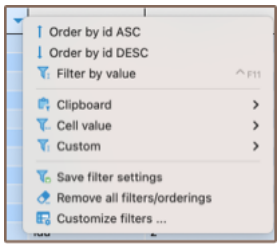
When working in the Data Editor grid, you might notice icons in the top left corner of each table. These icons show why you may not be able to edit data in specific tables:

Icon	Indicator	Description
	No unique key was found. Data modification is not possible.	The table lacks a unique key, restricting editing.
	PRIMARY KEY `column_name`	A unique key enables editing of the table.
	VIRTUAL PRIMARY KEY `column_name`.	A virtual key enables editing of the table. For more information about virtual keys, see Virtual keys.
	Table metadata not found. Data edit is not possible.	Unavailable metadata prevents editing of the table.
	Read-only connection.	A read-only connection prevents any edits.

> **Tip**: In addition to table status indicators, you can highlight column headers. See Hints for details.

Column context menu

Each column has a context menu, accessed by clicking the downward arrow button , providing different filter options.



Cell context menu

Every cell in the table data also has a context menu - right-click the cell to open the menu. The context menu provides the following items:

Menu Item	Description
Copy	Copies the content of the current cell or column to the clipboard.
Advanced Copy	Opens advanced copy submenu that allows copying data with preset formatting parameters.
Paste	Pastes the copied content to the cells in focus.
Advanced Paste	Pastes several values delimited with a tabulation or line break starting from the selected cell.
Delete	Deletes the row that has the cell in focus. **NOTE**: In fact, when users click **Delete**, the system only highlights the red row while the actual deletion happens when users click **Save**.
Edit	Opens a submenu enabling inline editing. See the Data Viewing and Editing article.
Order	Displays a submenu that allows to specify ordering criteria for the data. The submenu contains the most common ordering options that can be applied to the column in focus. See details in Data Filters article. By default, DBeaver orders data by sending a request to the server (the Server-side results ordering checkbox selected). To order data on the client side using DBeaver's internal algorithm, clear the checkbox.
Filter	Displays a submenu that allows to specify filter criteria for the data. The submenu contains the most common filters that can be applied to the cell in focus. See details in Data Filters article. By default, DBeaver filters data by sending a request to the server (the Server-side results ordering checkbox selected). To filter data on the client side using DBeaver's internal algorithm, clear the checkbox.
View/Format	Opens a submenu that provides options for formatting and modifying the view of data. See the Data View and Format article.
Navigate	Opens a submenu that helps users to navigate through the table data. See the Navigation article.
Layout	Changes the layout of data, see the Data View and Format article.
Export data	Opens the Data Transfer wizard that guides you through the steps to export data to a selected format. See the Data Transfer article. **Note**: The system exports the whole result set including records that are not visible on the screen and preserves all applied data filters and ordering.
Generate SQL	Opens a submenu where you can select the type of SQL query to generate. See the SQL Generation article.
Generate Mock Data	Opens Mock Data Generator. See the Mock Data Generation in DBeaver article.
Logical structure	Opens a submenu allowing you to write virtual column expressions, including use of functions, namespaces, and special variables like `row` and `table`. See the article about Virtual column expressions.
Open with	Opens the data in external applications like Excel or a web browser.
Toggle result panel	Opens Value panel on the right side of the Data Editor.
Refresh	Refreshes the whole results set including all items that are not visible on the screen.

Additional features

Disable metadata queries

If you want to improve performance and save money in databases where you pay for each query, you can turn off metadata queries in the Data Editor. This stops the editor from automatically asking for information about tables and columns.

To enable, navigate to **Window -> Preferences -> Connections -> Metadata** and check the option **Do not read tables information in SQL and data editors**.

- **Benefits**:
    - Makes the Editor work faster because it runs fewer queries.
    - Saves money in databases where each query costs you, by only running queries that you start yourself.
- **Disadvantages**:
    - You won't have auto-completion or other tools that need metadata.
    - Results from queries are read-only. To check if modifications are possible, look for an indicator in the top left corner of the table grid. For more information on indicators, see Table status indicators.

> **Tip**: This setting helps most with databases such as:
>
> - Amazon Athena
> - Amazon Redshift Serverless
> - Amazon Timestream
> - AWS DynamoDB
> - AWS Keyspaces
> - Databricks
> - Google BigQuery
> - Google Cloud Spanner
> - Snowflake
>
> In general, any cloud database may charge for metadata queries, even when you're just browsing schema or table lists.

SQL Editor

Table of contents

- Getting started
- SQL Editor overview
- Script panel
  - Spelling
  - Hyperlinks
  - Highlighting
- Toolbar
- Results Panel
  - Multiple results in one tab
- Additional features
- Active Database
- Layout Adjustment
- Outline
- Error indication
  - Explain errors with AI
- Features summary

With the SQL Editor in DBeaver, you can write and execute multiple SQL scripts within a single database connection, save them as files, and reuse them later.

Getting started

To start working with the SQL Editor, you have several options:

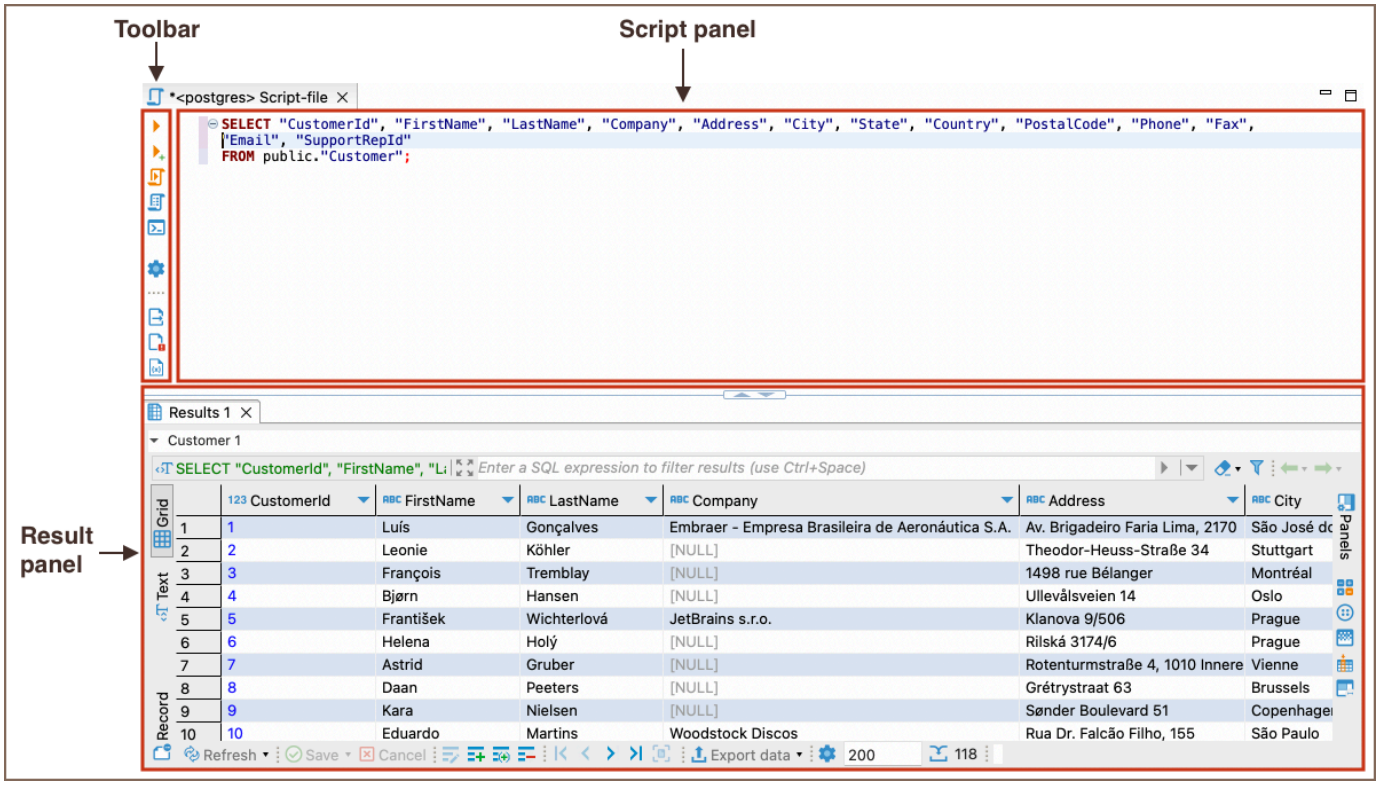
1. **Via Connections**:
   - Navigate to your desired database connection in the Connections view.
   - Press `F4` or go to **SQL Editor** -> **Open SQL script** from the main menu, or right-click on the connection and choose **Open SQL script** from the context menu.
   - A **Choose SQL Script** window appears. Click any script to open it in a new tab.
2. **Open Recent SQL script**:
   - Right-click on your database connection and select **SQL Editor** -> **Recent SQL script** from the context menu or go to **SQL Editor** -> **Recent SQL script** from the main menu. Alternatively use the `Ctrl+Enter` shortcut in the **Connections** view.
   - A **Choose SQL Script** window appears. Click any script to open it in a new tab.
3. **Create a New SQL Script**:
   - Navigate to **SQL Editor -> New SQL Editor** on the main menu.
   - Press `F3` and click **New Script** in the **Choose SQL Script** window.

> **Note**: SQL Editor for a connection is different from SQL console for a table or view. Unlike the console, it can save scripts and changes made to them.

You can see all your saved SQL scripts in the Project Explorer view in the **Scripts** folder.

SQL Editor overview

The SQL Editor contains the Script panel, the Toolbar, and the Result panel.



> **Tip**: You can open the SQL editor preferences by pressing `Alt+Enter` or right-click and navigate to **Preferences**.

Script panel

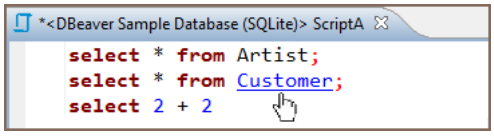
The Script Panel is the primary area where you can write, edit, and manage your SQL scripts. It provides basic text editing features with the added benefits of specialized functionalities tailored for SQL development:

Spelling

The panel includes a spelling checker to identify and highlight misspelled words, assisting you in maintaining the quality of your scripts. For more information check out our article.

Hyperlinks

You can press and hold `Ctrl` and at the same time move the mouse over the SQL text. If DBeaver recognizes some identifier as a table/view name, it presents it as a hyperlink. You can click the hyperlink to open this object`s editor:



Highlighting

DBeaver uses SQL syntax highlighting which depends on the database associated with the script. Different databases have different sets of reserved keywords and system functions. For more information, see the relevant section on highlighting settings in our article on SQL Code Editor.

Toolbar

The toolbar is customizable and contains buttons for commonly used commands. For more information on customization, see our article on toolbar customization.

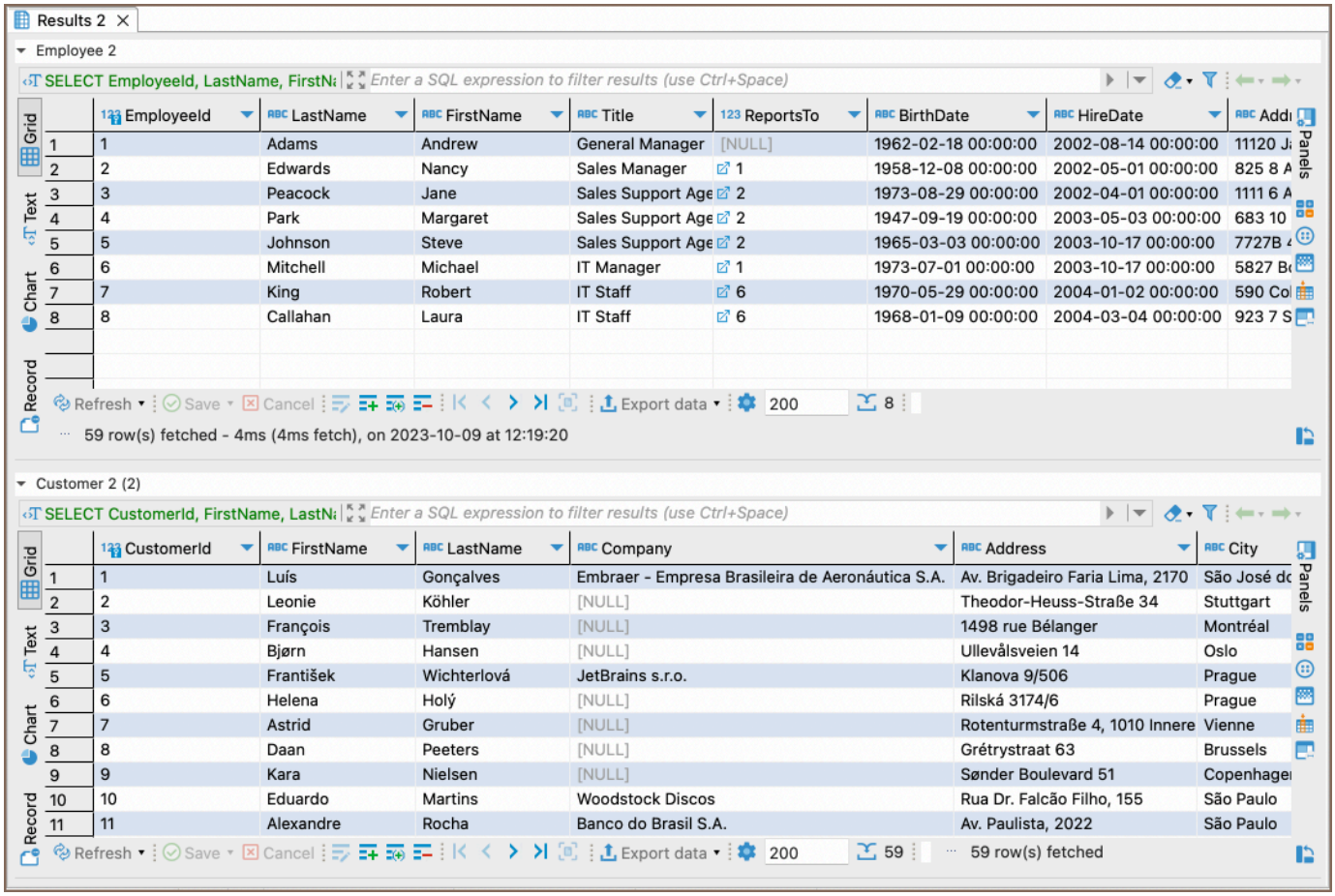
Results Panel

The results panel displays tabs with results in various formats. The tabs resulting from script execution represent instances of the Data Editor. You can create, edit and execute SQL scripts in the script panel and then see the results in the result tabs.

> **Tip**: You can also visualize query results as charts to explore data patterns. For details, see Charts.

Multiple results in one tab

You can view and manage multiple query results within a single tab.



To use the Multiple results feature in the SQL Editor, follow these steps:

1. **Add Toolbar Item**: Add the **Show multiple results in a single tab** to the SQL Editor toolbar. Read more on Toolbar customization.

   Alternatively:

   - Right-click in the SQL Editor window and navigate to **Execute -> Layout -> Show multiple results in a single tab**.
   - Navigate to **SQL Editor** -> **Show multiple results in a single tab**.

2. **Toggle**: Click the toolbar item to activate multiple query results view.

3. **View results**:

   - To view multiple results in a single tab, execute queries using the **Execute SQL Script** button .
   - You can collapse some result sets according to your preference by clicking the arrow button .

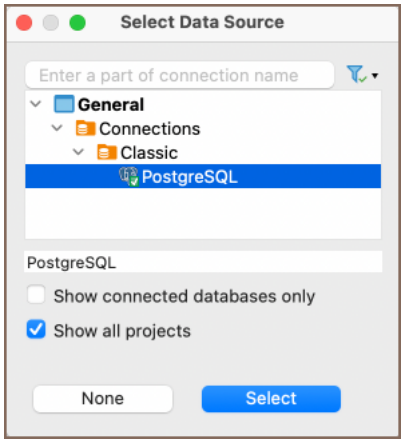
> **Important**: When using two or more queries in parallel, exercise caution as this may lead to client UI freeze, high database server load, or transaction deadlock.

Additional features

Active Database

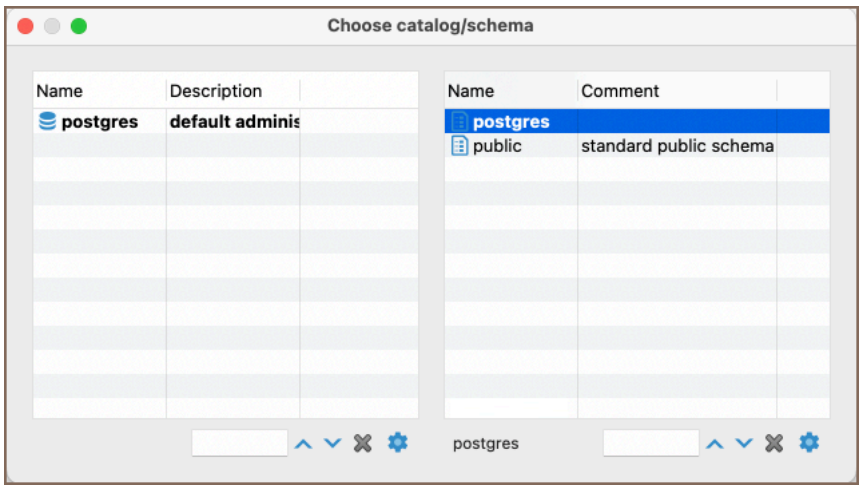
You can change the connection associated with the current SQL editor or change the active database/schema, at the same time retaining the SQL text.
To change the connection, press `Ctrl+9` or click the **Active datasource** box on DBeaver`s main toolbar:

The Select Data Source dialog box opens. In the tree of connections, click the required connection and then click **Select**. To disassociate the SQL Editor with any connection, click **None**:



To change the active schema, press `Ctrl+0` or click the **Active Catalog/Schema** box in DBeaver`s main toolbar:

The Choose catalog/schema dialog box opens. In the list of schemas, double-click the required schema:



If there are many schemas, and they do not fit in the dialog box use the search field to find the schema.

To configure the set of columns to be visible for each schema in the dialog box, click the **Configure columns** button .

> **Tip**: You can easily associate the SQL Editor with the connection that is currently in focus in the Connections (the focus can be on any object of the connection - a table, a folder, etc.) - click the **Link with editor** or use the shortcut `Ctrl+Shift+,`.

Layout Adjustment

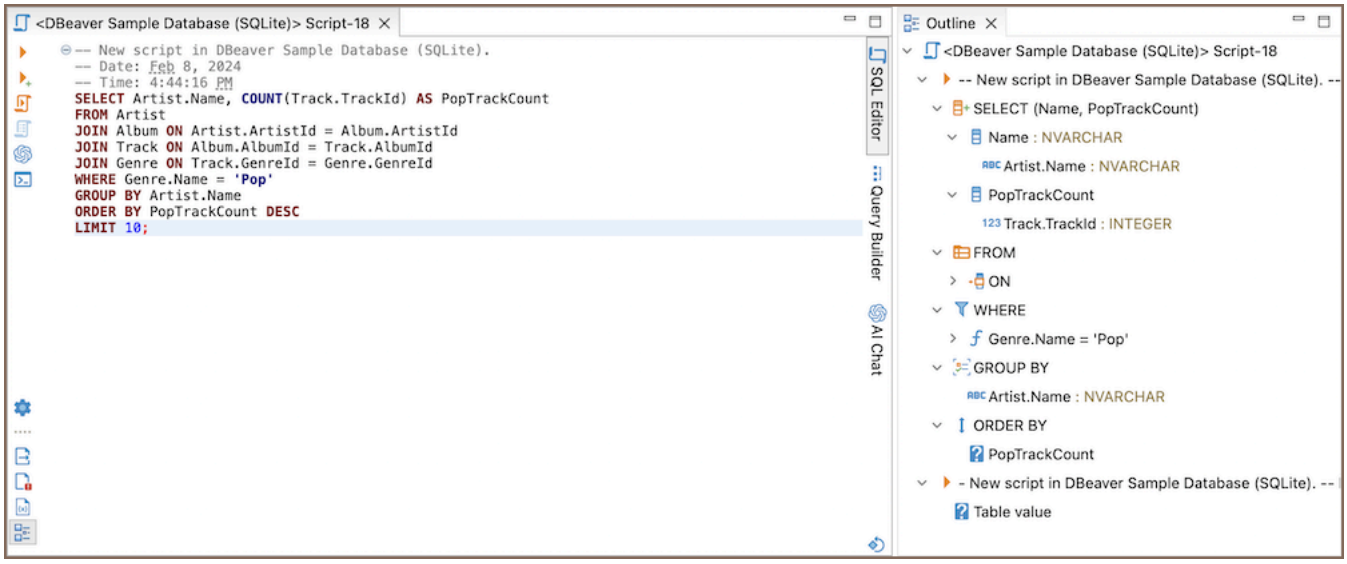
You can modify the layout of the SQL Editor by showing/hiding the results panel and changing the horizontal/vertical position of the panes.

- To toggle (hide/show) the results panel, press `CTRL+6` or right-click anywhere in the script pane and, on the context menu, click **Layout** -> **Toggle results panel**.
- To maximize the results panel, press `CTRL+Shift+T`, or double-click the results tab name, or right-click anywhere in the script panel and, on the context menu, click **Layout** -> **Maximize results panel**.
- To switch between the script panel and the results pane, press `Ctrl+Alt+T` or right-click anywhere in the script panel and, on the context menu, click **Layout** -> **Switch active panel**.

To position both panels horizontally, right-click anywhere in the script panel and, on the context menu, click **Layout** -> **Horizontal**.
To position both panels vertically, right-click anywhere in the script panel and, on the context menu, click **Layout** -> **Vertical**.

Outline

You can use the Outline feature in the SQL Editor to get a structured view of your SQL query.



Here's how you can access the Outline:

- Use the shortcut: `Alt+Shift+Q, O` (on macOS use `⌥⌘Q O`).
- Click on the **Toggle Outline** button in the SQL Editor bottom toolbar.
- Select **Panels -> Toggle outline** from the Script panel.
- Select **SQL Editor -> Panels -> Toggle outline** from Menu bar.

When you open the Outline, it shows a tree structure of your SQL query. This representation reflects the components of your SQL query and is beneficial for analyzing and moving through large queries. The relationship between the Outline and SQL Editor is interactive:

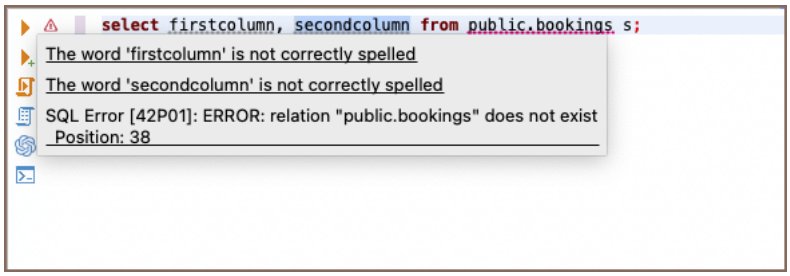
- The tree selection in the Outline automatically follows the cursor movement in the SQL Editor. This synchronization helps you identify the specific section of the query you are editing within its overall structure.
- Alternatively, clicking an item in the Outline tree will highlight the corresponding fragment in the SQL Editor, facilitating swift navigation to different parts of your SQL query.

Error indication

The SQL Editor indicates errors during the execution of an incorrect query by displaying an error icon to the left of the query text. Hovering over this icon reveals a tooltip.

- **Tooltip**: A tooltip will show a detailed list of errors.
- **Error list**: Moving the mouse over an error in the list will highlight it directly in the query.
- **Error types**: The SQL Editor detects and displays server issues, semantic errors, and spelling mistakes.

> **Note**: Semantic errors are displayed only when the **Enable semantic analysis** setting is activated and the database in use is relational. For more information, see Query analysis settings.



Explain errors with AI

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

You can use AI to explain and fix errors after query execution.

> For details, see AI error explanation.

Features summary

The SQL Editor offers a variety of advanced features to enhance your scripting experience.

- **SQL templates**: Insert pre-defined SQL code snippets for quick access.

- **SQL assist and auto-complete**: Complete SQL queries with suggested options as you type.
- **AI SQL assistance**: Get AI-powered suggestions for optimizations and best practices.
- **SQL formatting**: Automatically format SQL queries to enhance readability.
- **SQL execution**: Choose from various options to execute SQL queries.
- **SQL terminal**: Use a terminal interface for executing SQL commands directly.
- **Variables**: Manage variables within SQL queries.
  For information on this feature, see also Pre-configured Variables.
- **Query execution plan**: Visualize the execution plan of SQL queries for better insight.
- **Visual query builder**: Use a graphical interface to build SQL queries.
- **Script management**: Manage multiple SQL scripts within a single editor interface.
- **Client-side commands**: Execute client-side commands directly within the editor.

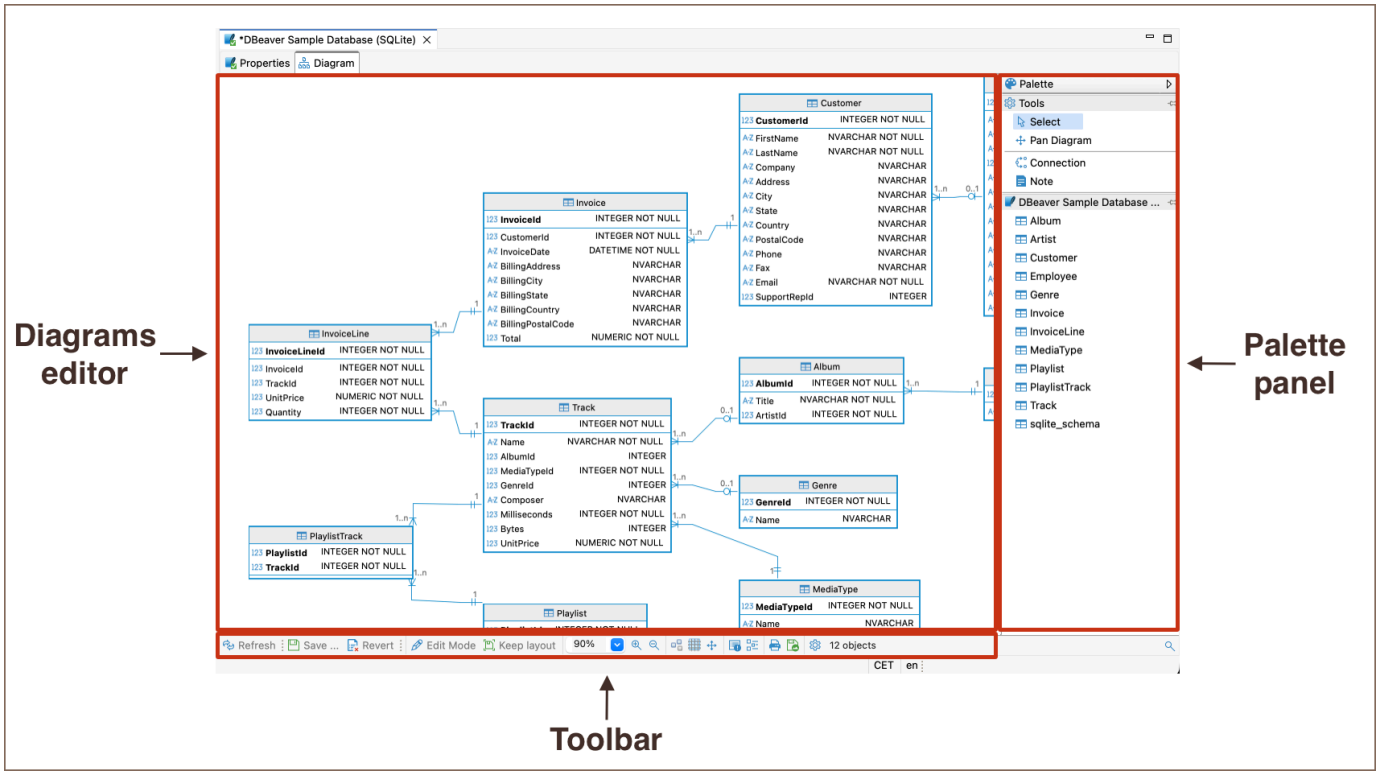
Entity relation diagrams (ERD)

Table of contents

- Open diagram
- Open a table or view diagram
- Open a schema diagram
- Diagram editor
- Custom diagrams
- Palette panel
- Toolbar and contex menu
- Toolbar
- Context menu
- Diagram preferences
- Diagram contents
- Color settings
- Attributes visibility
- Attribute styles
  - Grid
- Print
- Advanced

In DBeaver, you can use diagrams to explore database structure, analyze relationships between tables, and navigate complex schemas visually.

DBeaver supports diagrams for existing tables, schemas, and custom visualizations. You can select elements, adjust the layout, and export diagrams for further use. The interface includes:



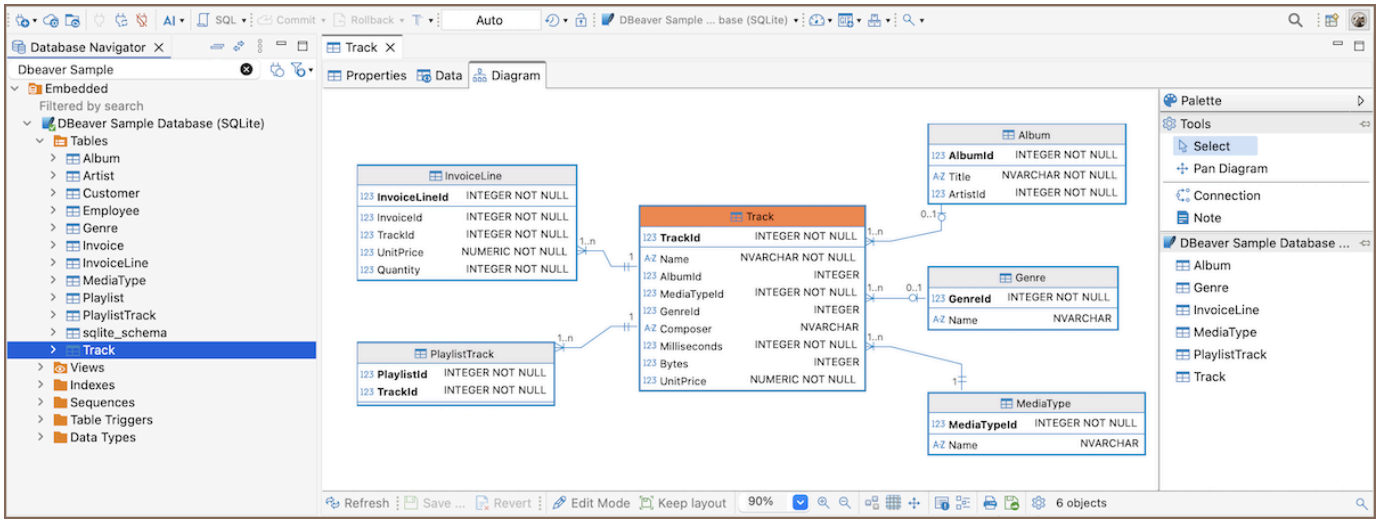
- **Diagrams editor**: View and rearrange entities and their relationships.
- **Palette panel**: Access tools for adding connections, notes, and more.
- **Toolbar**: Manage diagrams with options for saving, exporting, zooming, and configuration.

Open diagram

You can open diagram for individual tables and views, or for full database schemas.

Open a table or view diagram

To view the Diagram for a table or view, double-click the table or view in the Connections and then, in the Database Object Editor, switch to the **Diagram** tab:



Open a schema diagram

To view the Diagram for a full database schema, double-click the schema name in the Connections or the previous node in the path (usually - **Tables**):

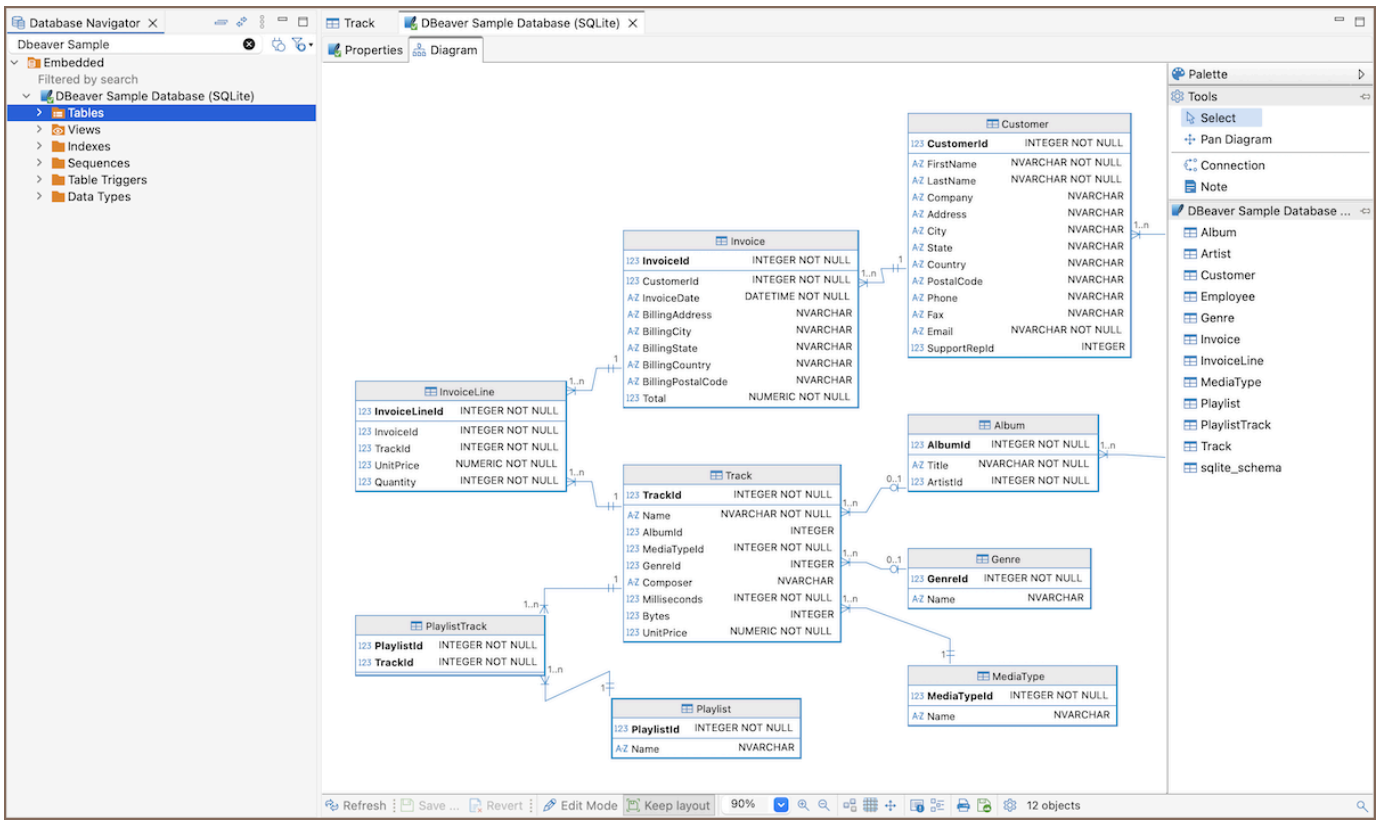


Diagram editor

The Diagram editor lets you explore and work with database structure visually. You can generate SQL, rearrange the layout, change view settings, use different notations and routing types, search within diagrams, and export or print them.

> For detailed instructions, see How to use Diagrams.

Custom diagrams

You can combine tables from different connections in a single workspace using custom diagrams.

> For details, see Custom Diagrams.

Palette panel

The **Palette** panel provides tools for interacting with diagram elements. It includes selection tools, navigation tools, connection creation, notes, and a list of available tables that you can drag onto the diagram.

Tool	Description
Select	Selects objects on the diagram.
Pan Diagram	Moves the diagram without selecting elements.
Connection	Creates a new connection between tables.
Note	Adds a new note element to the diagram.

Toolbar and contex menu

Use the diagram toolbar to access common actions, and the context menu to control the diagram view.

Toolbar

Button	Name	Description
Refresh	**Refresh**	Reloads the diagram to show external changes in the database schema.
Save ...	**Save**	Saves changes and generates an SQL script for execution.
Revert	**Revert**	Cancels all unsaved changes made in edit mode.
Edit Mode	**Edit Mode**	Enables or disables edit mode for creating and modifying database objects. Learn more
Keep layout	**Keep layout**	Saves the diagram layout locally. Otherwise, the layout is reset when you reopen the editor.

| 100% | **Zoom level dropdown** | Sets the zoom level for the diagram (e.g., 100%). | | | **Zoom In / Zoom Out** | Adjusts the diagram scale. | | | **Auto-arrange layout** | Automatically rearranges tables into a compact and readable layout. | | | **Toggle Grid** | Shows or hides the diagram grid. | | | **Toggle Hand Tool** | Enables pan mode to move the diagram view. | | | **Properties** | Selected object properties. | | | **Outline** | Opens a diagram overview mini-map for easier navigation. | | | **Print** | Prints the current diagram. | | | **Save diagram in external format** | Exports the diagram as `SVG` ★, `PNG`, `GIF`, `BMP`, or `GraphML`. | | | **Configuration** | Opens diagram preferences. | | | **Search** | Opens the search field and highlights matching tables and columns on the diagram. |

Context menu

> **Note**: Available actions in the context menu depend on what you select in the diagram:
>
> - the diagram canvas
> - a table
> - a column

Item	Description
Create	Opens a submenu for creating new objects in a diagram (Table, Column, Foreign Key, Index).
Create New Column	Creates a new column in the selected table.
New Index from Selection	Creates a new index based on the selected column.
New Constraint from Selection	Creates a new constraint using the selected colum.
Rename	Renames the selected column.
Edit Mode	Enables or disables edit mode for creating and modifying database objects. Learn more
Bring to front	Moves the selected element above others.
Send to back	Moves the selected element behind others.
Set color	Assigns a custom color to the selected object.
Remove color	Removes the custom color from the selected object.
Customize...	Opens object-specific customization options.
View Styles	Opens display style options. Learn more
Show Attributes	Controls attribute visibility. Learn more
Notation	Switches relationship notation.
Router	Switches routing type.
Arrange Diagram	Automatically rearranges tables into a compact and readable layout.
Toggle Grid	Shows or hides the diagram grid.
Keep layout	Saves the diagram layout locally. Otherwise, the layout is reset when reopening the editor.
Copy	Copies the selected diagram object to the clipboard.
Delete	Removes the selected element from the diagram.
Save diagram as...	Exports the diagram as `SVG` ★, `PNG`, `GIF`, `BMP`, or `GraphML`.
View Diagram	Opens a new diagram focused on the selected object and its related objects.
Tools	Opens additional tools available for the selected object.
Generate SQL	Generates SQL statements for the selected tables or relationships. Learn more
Undo	Reverts the last action.
Redo	Re-applies the reverted action.

Item	Description
Select All	Selects all diagram elements.

Diagram preferences

To customize how diagrams are displayed using global Diagram preferences, navigate to **Window -> Preferences -> Editors -> Diagram Editor**.

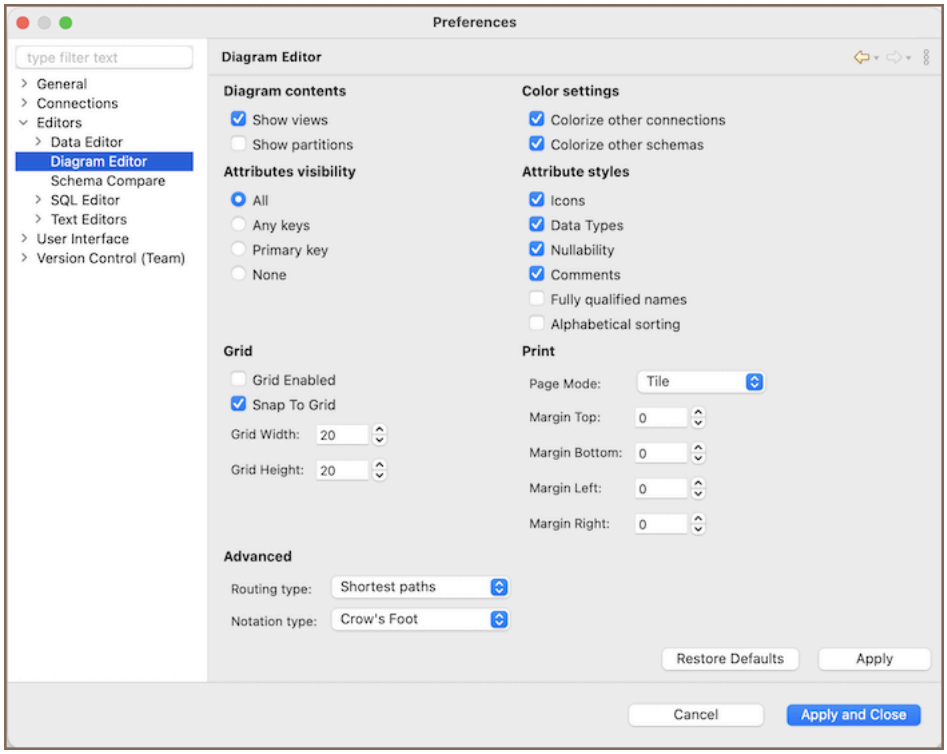


Diagram contents

Option	Description
Show views	Display database views on the diagram.
Show partitions	Display table partitions on the diagram.

Color settings

Option	Description
Colorize other connections	Apply color highlighting to non-selected connections.
Colorize other schemas	Colorize tables that belong to other schemas.

Attributes visibility

Option	Description
All	Show all attributes in tables.
Any keys	Show only primary and foreign keys.
Primary key	Show only primary keys.
None	Hide all attributes.

Attribute styles

Option	Description
Icons	Show attribute icons.
Data Types	Show attribute data types.
Nullability	Show NULL/NOT NULL markers.
Comments	Show column comments.
Fully qualified names	Show full table names with schema.
Alphabetical sorting	Sort attributes alphabetically.

Grid

Option	Description
Grid Enabled	Display grid lines on the diagram.
Snap To Grid	Snap elements to the grid when moving.

Option	Description
Grid Width / Grid Height	Set grid cell size.

Print

Option	Description
Page Mode	Select a layout mode for printing (e.g., Tile).
Margin Top / Bottom / Left / Right	Set print margins in pixels.

Advanced

Option	Description
Routing type	Configure connection routing (Shortest paths, etc.). Learn more
Notation type	Choose diagram notation (Crow's Foot, IDEF1X, Bachman, etc.). Learn more

Cloud services

Cloud Explorer

> **Note**: This feature is available in Ultimate edition only.

Table of contents

- Supported Databases
- Cloud configuration
- First-time setup
- Add a new configuration
  - Amazon Web Services
  - Google Cloud Platform
  - Microsoft Azure
- Configuring SSH Tunnels
- Explorer
- Database cloud information

Cloud Explorer offers deep integration with popular cloud service providers. As of the latest version, it supports Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure.

With **Cloud Explorer**, you can set up your cloud access once and then easily browse, connect, and manage your cloud databases. It saves you from manually configuring each database connection, as it reads all database endpoint information directly from the cloud provider.

Authentication is centralized. You use your cloud account to access your cloud databases. Before you start using **Cloud Explorer**, you need to set up your cloud provider access. This setup includes access credentials, availability zones for database search, and other cloud-specific settings.

Supported Databases

The **Cloud Explorer** allows you to easily connect with and manage various databases. Here is the list of databases it supports:

Providers	Databases
Amazon Web Services	PostgreSQL
	MySQL
	Oracle
	Amazon Redshift
	Amazon Athena
	Amazon DocumentDB
	Amazon DynamoDB
	Amazon Keyspaces
	Amazon ElastiCache (Redis)
	Amazon Timestream
	Amazon Neptune
Google Cloud Platform	AlloyDB for PostgreSQL
	Microsoft SQL Server
	PostgreSQL
	MySQL
	Spanner
	Firestore
	BigQuery
	Bigtable
Microsoft Azure	Microsoft SQL Server
	PostgreSQL
	MySQL
	Azure Cosmos DB for MongoDB
	Azure Cosmos DB for Cassandra
	Azure Cosmos DB for NoSQL

Cloud configuration

To start configuring, click the **Cloud Explorer** button in the Toolbar.

First-time setup

If this is your first time, the **Create Cloud Configuration** dialog opens automatically. Use it to:

- Enter your access credentials.
- Specify availability zones for database search.
- Adjust cloud-specific settings.

Once completed, **Cloud Explorer** will display your cloud databases.

> **Note**: Cloud configuration steps differ for each cloud provider.

Add a new configuration

To create an additional setup, click the **New** button in the **Cloud Explorer** dialog.

Amazon Web Services

Amazon Web Services supports the following authentication methods for cloud connections:

Name	Description
Default Credentials	Allows AWS to determine credentials using the standard credential providers.
AWS Profile	Allows you to choose which credentials profile you want to use.
Access/Secret Key	Allows you to choose which credentials profile you want to use.
Single Sign-On (SSO)	Allows access to AWS resources without explicitly specifying user credentials in DBeaver connections configuration. Learn more

> For step-by-step connection instructions, see AWS database connection via Cloud Explorer.

> About permissions, read AWS managed policies article.

Google Cloud Platform

Google Cloud Platform supports the following authentication methods for cloud connections:

Name	Description
Default Credentials	Allows GCP to determine credentials using the standard credential providers.
Web browser or **SSO**	Allows you to authorize your connection through your Google account without needing a service or user-based key file. Learn more
Access key file	Allows you to provide the path to your service or user credentials files.

> For step-by-step connection instructions, see Google database connection via Cloud Explorer.

Microsoft Azure

There are several environment variables available for Microsoft Entra ID. For detailed information on Entra ID authentication, see Authentication Microsoft Entra ID.

> For step-by-step connection instructions, see Microsoft Azure database via Cloud Explorer.

Configuring SSH Tunnels

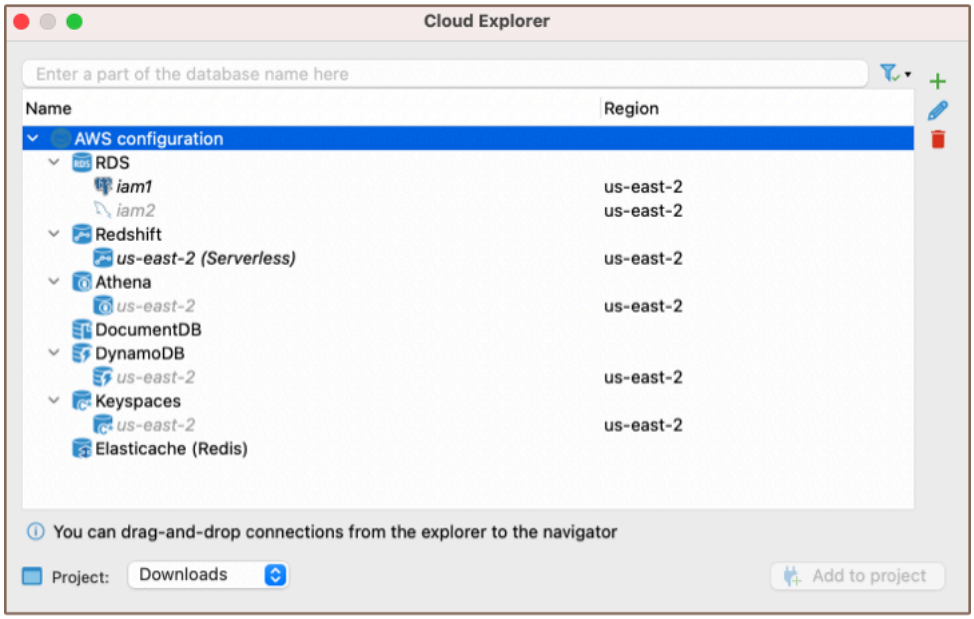
DBeaver supports SSH tunnel connections for enhanced security when accessing cloud databases.

> To set up SSH tunnels for cloud databases, see SSH Tunnel Configuration.

Explorer

Once you've configured your cloud provider access, open the **Cloud Explorer** dialog and add database connections. In the top drop-down of the explorer dialog, you can select the active cloud configuration or click **Edit** button to change the cloud configuration.

You can see cloud databases in the center of the dialog in a hierarchical view. All databases are grouped by database/service type. When you expand one of the top elements, DBeaver will search cloud databases in configured availability zones/regions.



If you have a large number of databases in your cloud, you can search or filter them using filter text above the cloud navigator.

You can drag-and-drop cloud databases directly to Connections view or projects view. You can also add any number of databases to your project directly from the **Cloud Explorer**. To do this, click on the database you want to add. Once selected, click on the **Add to Project** button located in the bottom right corner of the **Cloud Explorer** dialog, or right-click on the database and select **Add to Project** from the context menu. This will add the selected database to your project, making it easier to manage and navigate your databases within specific projects.

Database cloud information

You can access your cloud database configuration directly from the **Connections**. To do this, right-click on the database you are interested in and select **Edit Connection**. This action opens a special tab in the connection settings dialog. The information displayed in this tab is specific to the cloud and database type of the selected database.

Cloud Storage

> **Note**: This feature is available in Ultimate edition only.

Table of contents

- Cloud Storage
- Capabilities in Cloud Storage
- Direct capabilities in Cloud Storage
- Additional interactions via other features
- Trust cloud provider certificates

Cloud Storage lets you manage files in your cloud environment. This feature simplifies working with cloud-stored files - upload, download, and organize files just like in a local file system. It supports:

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage

To use **Cloud Storage**, set up a cloud access configuration in the Cloud Explorer.

Cloud Storage

After configuring your cloud provider access, you can start using **Cloud Storage**. To access it:

- Click the **Cloud Storage** button on the toolbar.
- Alternatively, go to the menu and select **Window -> Cloud Storage**.

This action will launch a window displaying the cloud services for which credentials were provided during setup. In this window, use the dropdown menu to navigate through folders and view files stored in the cloud.

You can filter and sort your folders and files by their name or region using the interface's `Name` and `Region` columns. Click on the respective column header to organize the displayed folders, according to these criteria, especially useful when managing a large number of folders.

Capabilities in Cloud Storage

The **Cloud Storage** offers a range of functionalities for managing files within the cloud. Here are the actions you can perform:

Direct capabilities in Cloud Storage

Some functionalities are directly accessible within the **Cloud Storage** interface:

Capability	Description
Open Resource	Open a folder in a new window to view details like `Name`, `Path`, `Size`, `Location`, and `Last Modified`. Double-click on the folder, or right-click on the folder and select **Open resource** `folder name` from the context menu. Use the **Configure columns** button to customize which columns are displayed in the window, tailoring the view to your specific needs. In the opened window, use the **Search item** button to conduct searches within the folder.
Create new folder	Create new folders within the cloud environment. To do this, right-click where you want the new folder and select **Create New Folder** button.
Save resource(s) to local disk	Save selected resource(s) from the cloud to the local disk. Select the file(s) or folder(s) in **Cloud Storage** you wish to download, then choose the **Save resource(s) to local disk** option available both in the top toolbar of **Cloud Storage** and the context menu.
Load resource(s) from local disk	Upload resource(s) from the local disk to the cloud. Select the target folder in **Cloud Storage** where you want to upload files, then choose the **Load resource(s) from local disk** option which can be found in the top toolbar of **Cloud Storage** as well as the context menu.

> **Tip**: In addition to the **Save to local disk** and **Load from local disk** features, you can also copy files using the drag-and-drop mechanism. You can drag files between **Cloud Storage** and your **local disk**, or between **Cloud Storage** and Project Explorer.

Additional interactions via other features

Some functionalities of **Cloud Storage** are accessible through the integration with other features of DBeaver:

Capability	Description
Export/Import files	Export to or import files from folders within the **Cloud Storage**. Details are available in Data transfer article.
Backup/Restore	Perform backup and restore operations using files from or saving outputs to the **Cloud Storage**, as detailed in Backup and Restore.
Data compare	After comparing files or folders, save the results directly to **Cloud Storage**. Details are available in Data Comparison.
Schema Compare	Compare database schemas and optionally save comparison results to **Cloud Storage**. Further information is found in Schema Comparison.

> **Tip**: To ensure you see the most updated information after performing file-related actions in **Cloud Storage**, it's important to refresh the folder manually. Right-click on the folder and select the refresh option from the context menu.

Trust cloud provider certificates

If you run into connection errors or SSL trust issues when connecting to cloud databases or services, the problem may be caused by missing certificate authorities (CAs). For details, see Managing truststore settings.

AI Smart Assistance

Table of contents

- Supported providers
- AI Assistant features
- Chat
- Work with SQL
- Customize prompts
- Best practices for prompt formulation
- Disable AI Assistant
- Data privacy

DBeaver provides AI powered features for working with databases in natural language, using popular providers. It can create and edit queries ★, answer questions, troubleshoot errors ★, or explore database objects ★.

Supported providers

AI Assistant in DBeaver works with several AI providers.

- OpenAI
- GitHub Copilot
- Azure OpenAI ★
- Google Gemini ★
- Ollama ★
- Anthropic Claude ★
- Amazon Bedrock ★
- Grok ★

DBeaver isn't affiliated with these providers. Integration with AI assistance is achieved through the public APIs. To use AI Assistant, sign up with a provider and activate it in DBeaver.

> **Important**: Some paid AI provider plans don't allow access for use in third-party applications. Make sure your subscription includes this capability before connecting.

AI Assistant features

Chat

- AI Chat. Chat naturally with AI to ask questions and get queries.

Work with SQL

- AI command. Run AI command directly in the SQL Editor. Type command like `@ai show films in which Grace Mostel starred`, and let AI generate the SQL for you.

Customize prompts

- AI prompts. Tune the instructions used for chat, SQL generation, and other AI features.

Best practices for prompt formulation

Follow these guidelines to get the most accurate results from AI:

- **Use English**: While AI supports multiple languages, English works best.
- **Know your database**: The more you understand your database structure, the more accurate the generated queries.
- **Refine iteratively**: Start broad, then add details step by step to improve results.
- **Be explicit**: Mention specific tables or columns if you know them - this helps AI generate precise queries.

Disable AI Assistant

You can turn off AI Assistant entirely or disable specific features.

> Learn more in Disable AI Assistant.

Data privacy

DBeaver protect your data and privacy with strict safeguards. AI Assistant works with the minimum information required, and all processing follows provider privacy policies.

> Learn more in Data privacy.

AI Chat

Table of contents

- Work with AI Chat
- AI Chat configuration
- Connection field
- Multiple conversations
- Resetting query context
- Defining the scope
- Active configuration
- Import files
- Clarifying questions
- Voice input
- AI Chat settings
- Customize the AI Chat display
- Message statistics and token usage

DBeaver provides AI Chat for working with databases in natural language. You can enter requests in plain text to generate SQL queries, get explanations, or ask questions about your databases.

Work with AI Chat

1. Click on the **AI Chat** icon located in the DBeaver toolbar.

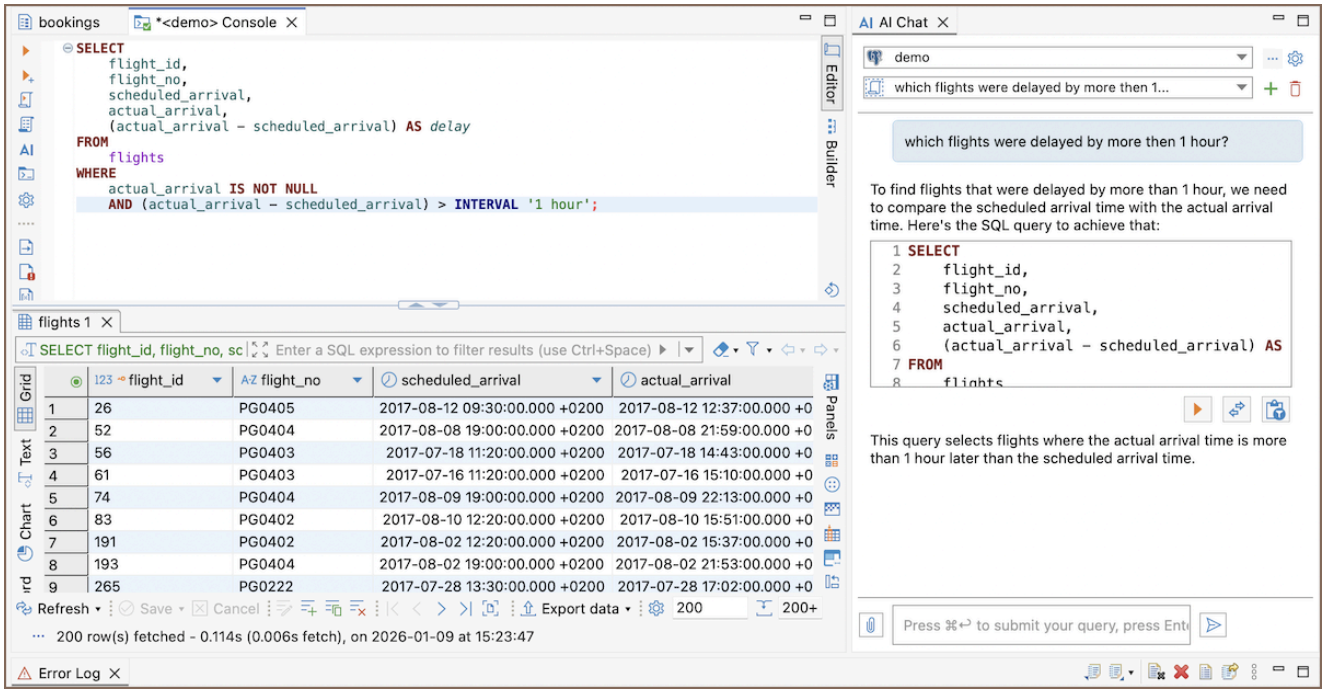
   - Alternatively, open the SQL Editor and click on the **AI Assistant** icon AI located in the left toolbar.

> **Note**: The toolbar is customizable. For further information, refer to Toolbar Customization article.

2. In the chat window that opens, select a connection and, optionally, a scope.
3. Enter your request in natural language.
4. Press the **Send** button to get the SQL translation. You can then choose from the following options:
   - **Execute** to run the generated SQL query
   - **Editor** to open the query in the SQL Editor and continue working with it directly

> **Tip**: If the cursor is on a query, clicking **Editor** will replace it. Move to a new line to keep the old query.

   - **Copy** to copy the query to your clipboard



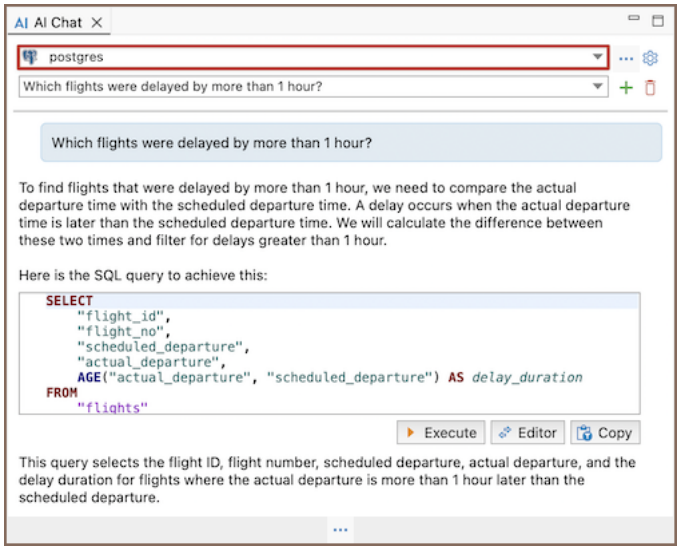
> **Tip**: To stop response generation, select the **Stop** button next to the input field.

> For information on AI Chat keyboard shortcuts, see Shortcuts.

AI Chat configuration

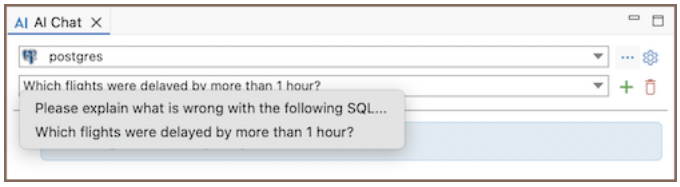
Connection field

To use a specific connection in **AI Chat**, select the connection from the toolbar at the top of the chat panel. The selected connection defines which database the AI will use when generating SQL queries.



Multiple conversations

You can create multiple chats for different connections.



The first chat is created automatically when you open **AI Chat**. Each chat is named based on your first prompt. Messages related to the same query or object in the same connection are kept together in one conversation.

Action	Button/icon	How to do it
Add a new chat	+	Click the button to start a new conversation
Switch chats	*(Dropdown menu)*	Use the dropdown menu to switch between existing chats
Delete a chat	🗑	Select the chat from the dropdown, then click the delete button

> **Important**: Deleted conversations can't be restored.

Resetting query context

AI Chat keeps the history of your questions so you can build on them. Sometimes you may want to start fresh. To reset the context or clear history up to a certain message, click the reset button (🧹) next to that message.

> **Example**: You asked about the "orders" table and then want to switch to "customers". Reset before your new question so AI doesn't mix both topics.

Defining the scope

To improve accuracy, especially in databases with large schemas, you can limit what database context AI sees.

1. In the **AI Chat** toolbar, click the **Change scope** button ⋯ to open the context menu.
2. Under **Applies to**, choose whether the scope below is remembered for the connection or just for this chat:

Option	Description
This connection	The scope is saved with the connection and reused across conversations
This conversation	The scope only applies to the current chat and resets when you start a new one

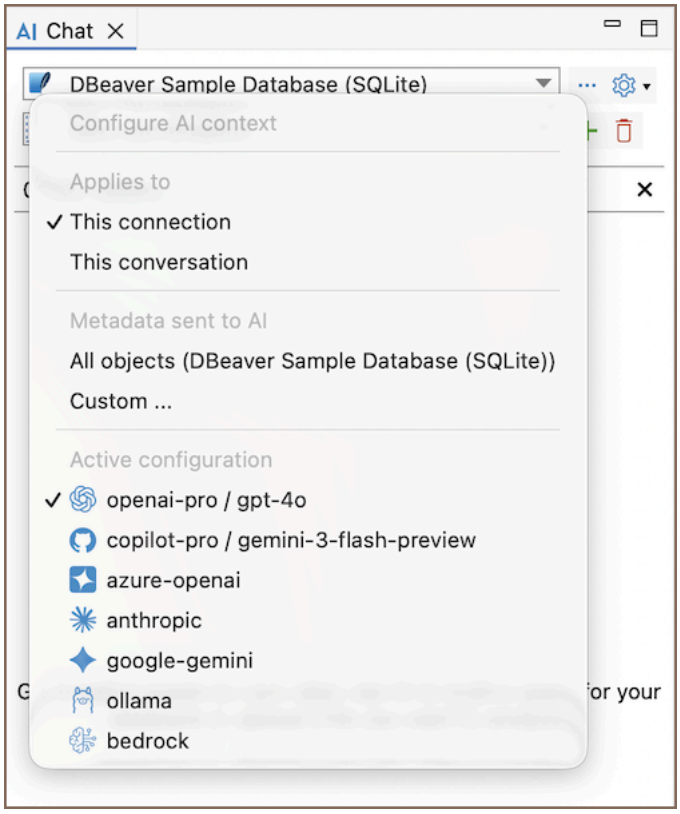
3. Under **Metadata sent to AI**, choose how much database context AI sees:

Scope option	Description
Current schema	AI focuses only on the schema you're currently using
Current database	AI is limited to the currently selected database
All objects	AI can access metadata for all schemas and objects in the current connection
Custom	Lets you define a more specific scope, such as a particular table or set of objects

Active configuration

Each conversation starts with the default AI profile. To change the active profile or model, use the selector row below **AI Chat**:

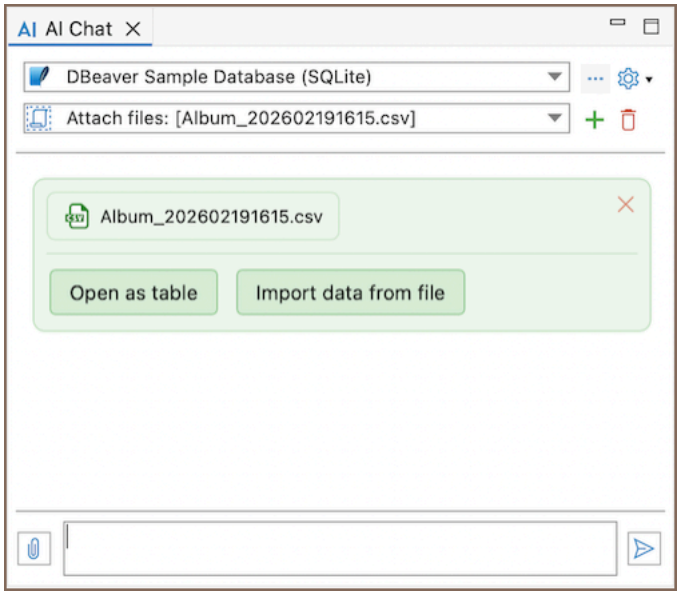
1. Select **Profile**, then choose a profile. This selection applies only to the current conversation and doesn't change the default profile in **Preferences**.
2. Select **Model**, then choose a model. The first time you open **Model** for a profile, the available models load automatically. Later openings reuse the loaded list.



> The switch only applies to the current conversation - it doesn't change the default profile in **Preferences**.

Import files

You can upload files directly in **AI Chat** and use AI to import data or open files as tables.



To add files, do one of the following:

- drag and drop file(s) into the AI Chat input area
- click the **Attach a file** button ( ) in the chat input and select file(s) manually

File handling in AI Chat is limited to the formats listed below:

Workflow	Supported formats / drivers
Open as table	- CSV - JSON - Parquet - XML - XLSX - SQLite
Import data from file	Supports all file formats available for data import.

> When importing data, you can upload multiple files of the same format and import them together.

Clarifying questions

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

When your request can be read in more than one way, AI asks you to clarify it right in the chat instead of guessing. The question appears as a card with the options to pick from.

> For more details, see User interaction.

Voice input

You can dictate your AI Chat requests using your operating system's voice input tools.

OS	Tool
Windows	Voice Typing (**Win + H**) or Voice Access
macOS	Dictation

To use voice input in AI Chat:

1. Click in the AI chat input field.
2. Start your OS's voice input tool and speak your request.
3. Review and edit the transcribed text, then press **Send**.

AI Chat settings

Adjust chat options to make **AI Chat** work the way you prefer.

Go to **Window -> Preferences -> AI -> Chat Settings**.

> For the full list of AI Chat options, see Chat Settings.

Customize the AI Chat display

1. In the **AI Chat** toolbar, click the **Change scope** button ⋯ to open the dropdown menu.
2. Select what to show:
    - **Show profile and model** shows or hides both selectors together. You can't hide only **Model**.
    - **Show total tokens spent** shows or hides total token-usage statistics.

Message statistics and token usage

You can show per-message statistics in **AI Chat**, including message time, time spent, and token usage.

1. In the **AI Chat** toolbar, click the **Change scope** button ⋯ to open the dropdown menu.
2. Enable the stats you want to see:
    - **Show message time**
    - **Show time spent**
    - **Show tokens spent**

AI Assistant settings

Table of contents

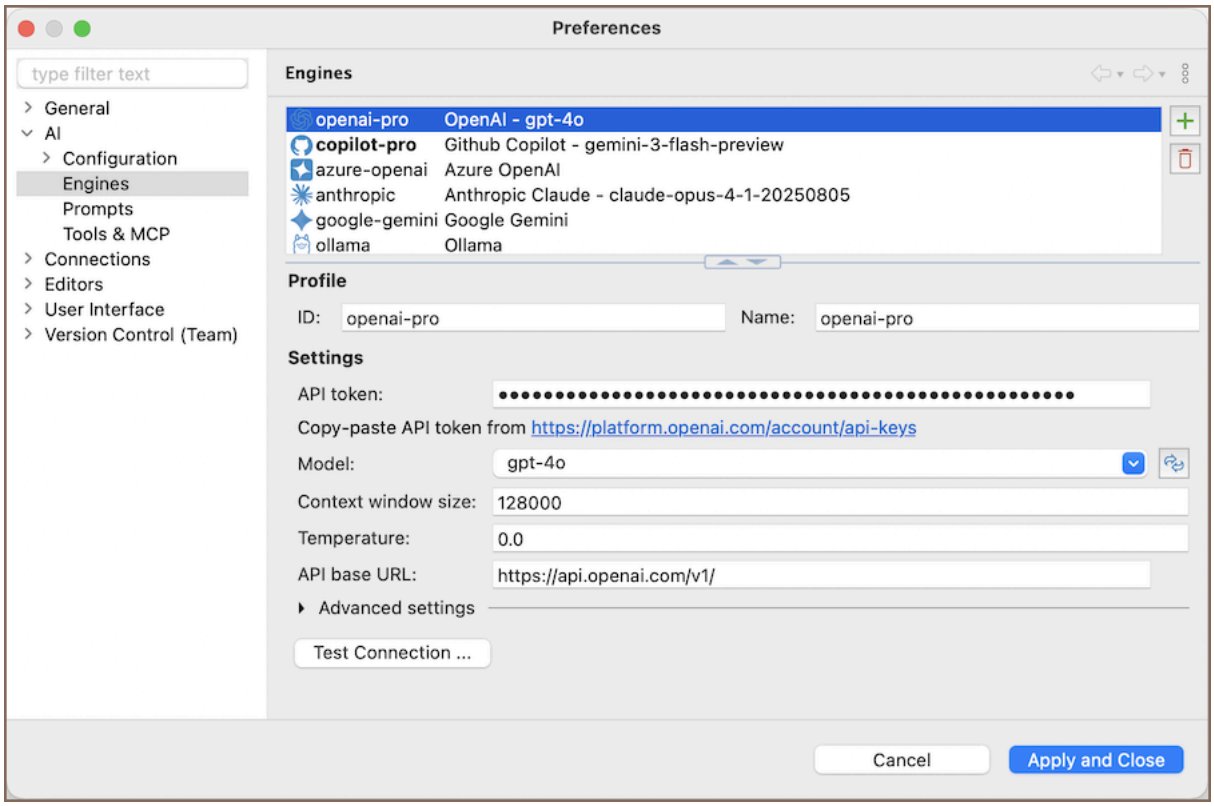
- AI Assistant setup
- Profiles
- Chat Settings
- AI Configuration
- General
- AI query execution
- Send database structure
    - Automatic prompt reduction
- Prompts
- Troubleshooting
- AI chat shows N/A on Windows Server

Use this page to configure the general behavior of AI in DBeaver.

> **Note**: AI Assistant is turned on by default. If it was disabled earlier, you can re-enable it in **Window -> Preferences -> AI** by selecting **Enable AI integration**.

AI Assistant setup

To get started, open **Window -> Preferences -> AI-> Model configurations**, choose a provider, and paste your API token.



Profiles

The Model configurations page manages AI configurations as **profiles**. Each profile pairs a name with one provider and its own settings, so you can keep several configurations side by side - for example, two OpenAI profiles with different models or keys.

- Click Add icon ( ) to create a profile: pick a provider, then set its **ID** (fixed after creation) and **Name** (you can rename it anytime).
- Click the trash icon ( ) to delete the selected profile.

> The profile selected when you apply changes becomes the **default** and appears in bold in the list. AI features use the default profile unless you pick another one explicitly.

Chat Settings

For configuring AI Chat settings, navigate to **Window -> Preferences -> AI ->Configuration -> Chat Settings**.

Setting	Description	Default
Use stream mode	Displays the AI's reply as it's generated instead of waiting for the full response. Gives a faster perceived response and lower memory use. Available only if the selected AI provider supports it.	On
Execute in a new SQL console	Opens a new SQL console when you run a query from the AI chat and keeps the executed query there. When you turn this off, DBeaver reuses the currently active console instead.	On
Include source in query comment	Shows your original request above the AI-generated query in the SQL Editor.	Off
Format SQL query ★	Formats the generated SQL query using your database's default keyword case.	Off
Table join rule ★	Controls how the AI joins tables in generated queries: the AI's default judgment, explicit `JOIN` clauses, or sub-queries.	Default
Days to keep chats ★	Sets how many days chat history is stored locally. Older chats will be deleted automatically.	7

> **Important**: If you add filters in the Connections, only the filtered tables are sent to the AI. The same applies in the Data Editor — if you filter tables, they won't appear in completion or be validated correctly in queries, because if they're not in Connections, there's no metadata about them.

AI Configuration

For configuring specific settings, navigate to **Window -> Preferences -> AI -> Configuration**.

General

Setting	Description	Default
Language	Choose the language for AI responses. This affects the language used in answers, explanations, and code comments.	English

> Besides the listed options, you can also type in a language that's not included in the dropdown menu.

AI query execution

These settings control how AI-generated SQL queries are executed. You can choose to run them immediately, show a confirmation prompt, or disable autocommit so changes must be committed manually.

Setting	Description	Options	Default
Select	Controls execution of `SELECT` queries.	- **Execute immediately** - run `SELECT` queries without confirmation. - **Show confirmation** - ask for confirmation before executing.	Execute immediately
Modify	Controls execution of data modification queries (`INSERT`, `DELETE`, `UPDATE`).	- **Execute immediately** - run modification queries without confirmation. - **Show confirmation** - ask for confirmation before executing. - **Disable autocommit** - turn off autocommit so changes require manual commit.	Show confirmation
Schema	Controls execution of schema modification queries (`CREATE`, `DROP`, `ALTER`, etc.).	- **Execute immediately** - run schema modification queries without confirmation. - **Show confirmation** - ask for confirmation before executing.	Show confirmation
Other	Controls execution of queries DBeaver cannot classify as valid SQL (including NoSQL queries).	- **Execute immediately** - run unknown or unclassified queries without confirmation. - **Show confirmation** - ask before executing queries not recognized as SQL.	Show confirmation

> **Tip**: For the AI command, using **Show confirmation** is strongly recommended.
> It lets you preview the generated SQL in a dialog before execution, so you can review or cancel risky changes.

Send database structure

Setting	Description	Default
Send column data type information	Sends column data types (like `INT`, `VARCHAR`, etc.) to the AI provider. Improves completion quality, but consumes more tokens.	On
Send object description	Sends object description to the AI provider. Improves completion, but may consume significant amount of tokens.	Off
Send foreign keys information ★	Sends foreign key information to the AI provider. Improves completion quality, but consumes more tokens.	Off
Send unique and primary keys information ★	Sends unique and primary key information to the AI provider. Improves completion quality, but consumes more tokens.	Off

Automatic prompt reduction

When sending database metadata and sample data to the AI provider, the amount of information is limited by the selected context size. If the content exceeds this limit, DBeaver automatically shortens the prompt in two steps:

1. Removes extra details (data types, keys, constraints, sample data) if all are enabled and the size is too large.
2. If it's still too large, trims the snapshot until it fits the context limit.

Prompts

Use this page to customize the built-in instructions that DBeaver sends to the AI provider for specific features. These instructions are applied automatically when you use the related AI feature.

Open it in **Window -> Preferences -> AI -> Prompts**.

> **Note**: User-defined instructions are added on top of default instructions. Default instructions can't be changed.

Prompt	What it affects
Analyze Data Compare results ★	Explains differences found in Data Compare results.
Describe database objects ★	Returns detailed database object description and its role in the database schema.
Explain SQL queries ★	Explains the structure and business value of the SQL query, including involved entities and relationships.
Explain SQL query execution plan ★	Explains the SQL query execution plan and helps understand how the database runs the query.
Fix SQL query errors ★	Fixes SQL query errors using the query text, the error message, and environment information.
AI Auto Complete ★	Performs AI Auto Complete based on the query, comments, and surrounding queries.
SQL query generation	Generates SQL queries from a natural language description.
Smart mapping ★	Finds the closest matching data types when converting columns between source and target databases.

Troubleshooting

AI chat shows N/A on Windows Server

If AI chat displays **N/A** and the error log contains unhandled event loop exceptions, the cause is likely a missing WebView2 component. For information on how to fix it, see Troubleshooting WebView2 issues.

Data transfer and schema compare

Table of contents

- Supported formats
- Export data
- Import data
- Transfer data between database tables

Data transfer is a crucial feature that enables you to export and import data in various formats as well as moving data between tables in the same or to different databases.

The data transfer operation runs in the background, allowing you to continue working with your database during the export or import process. However, try to avoid changing data in the tables that have been selected for export or import until the process is complete.

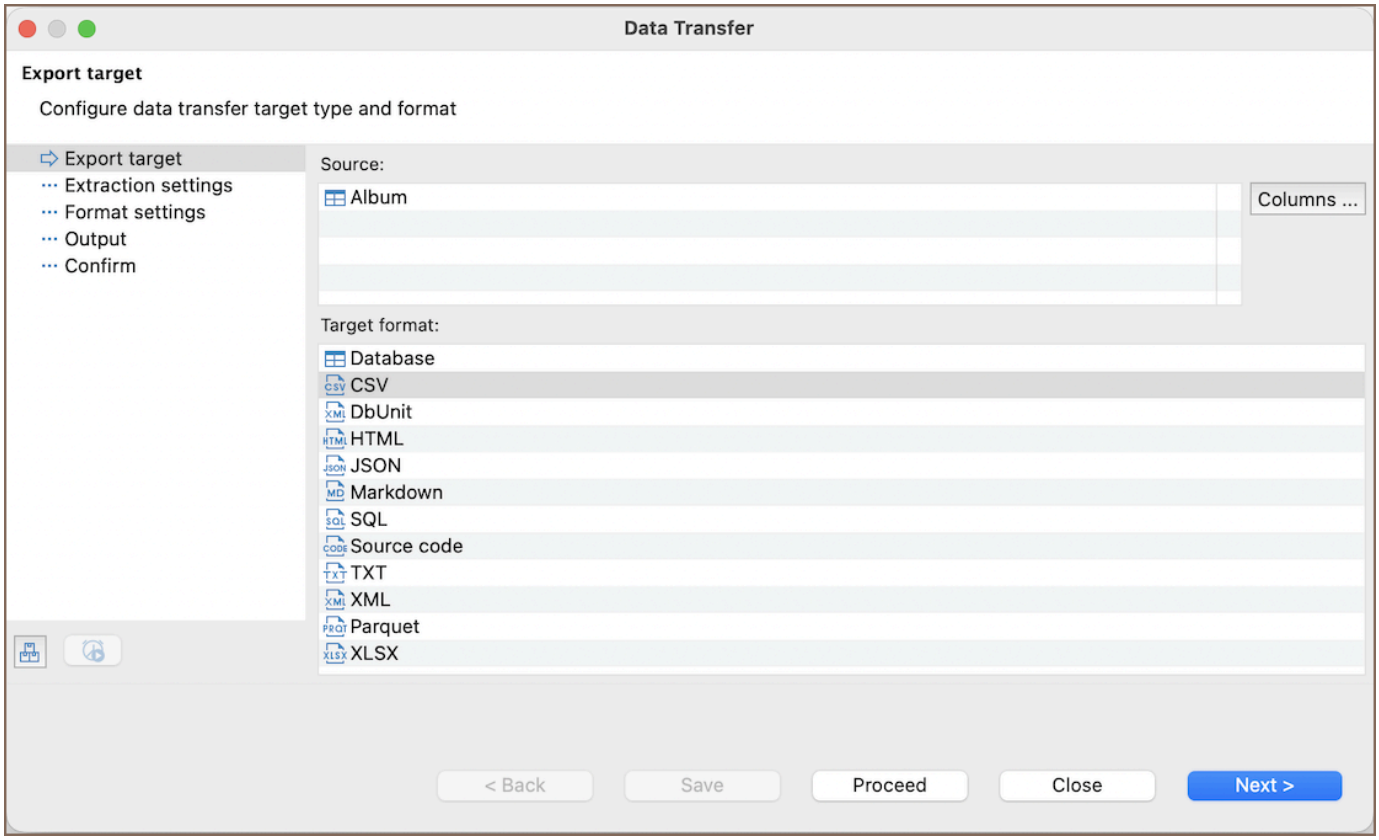
Supported formats

Export	Import
Database (table-to-table transfer)	CSV
CSV	XLSX ★
DbUnit	XML ★
HTML	Table
JSON	
Markdown	
SQL	
Source code	
TXT	
XML	
Parquet ★	
XLSX ★	

Export data

DBeaver allows you to export data from your database to various formats for use in other systems or applications.

> For more details, see Data Export.



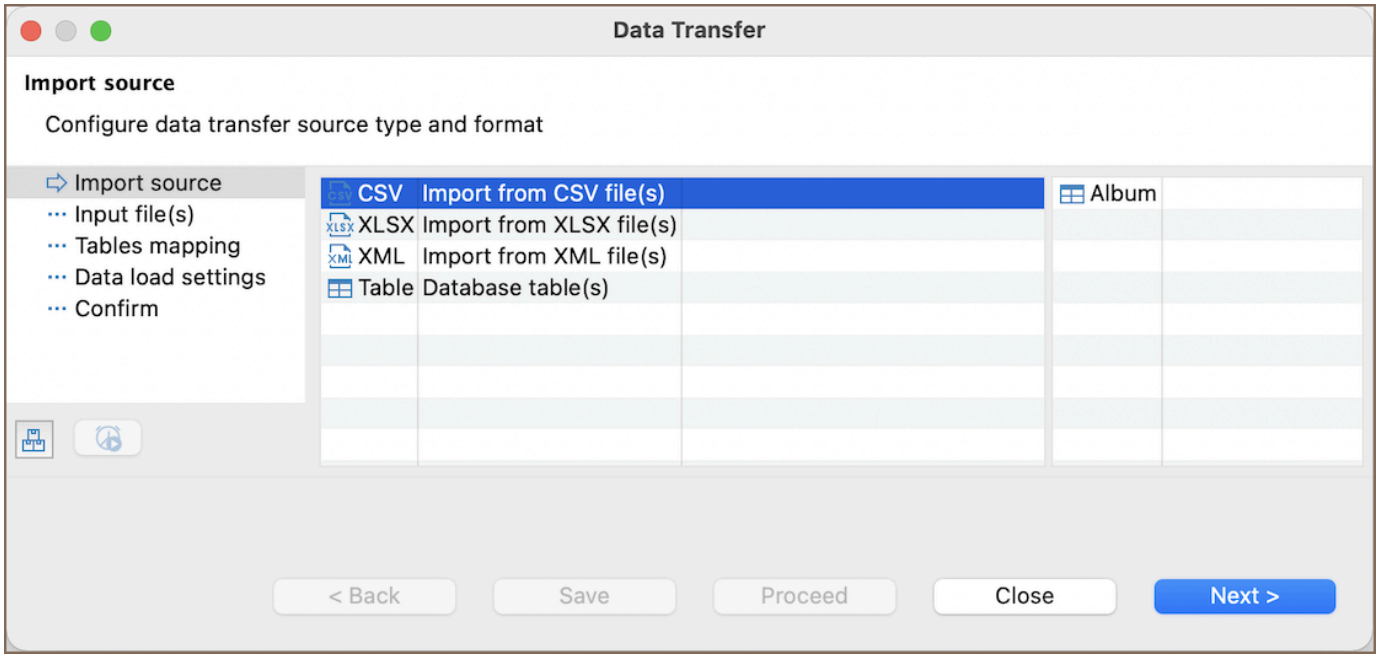
Depending on the selected format, exporting allows you to:

- export query results, fetched rows, or entire tables.
- adjust data extraction, formatting, and encoding options.
- set options for delimiters, character encoding, or `NULL` values.
- use advanced features like multi-threading for large exports or adding data to existing files (e.g., Excel).
- save export configurations as tasks for repeated use.

Import data

DBeaver enables you to import data from external sources into your database, making it easy to fill tables with new or updated information.

> For more details, see Data Import.

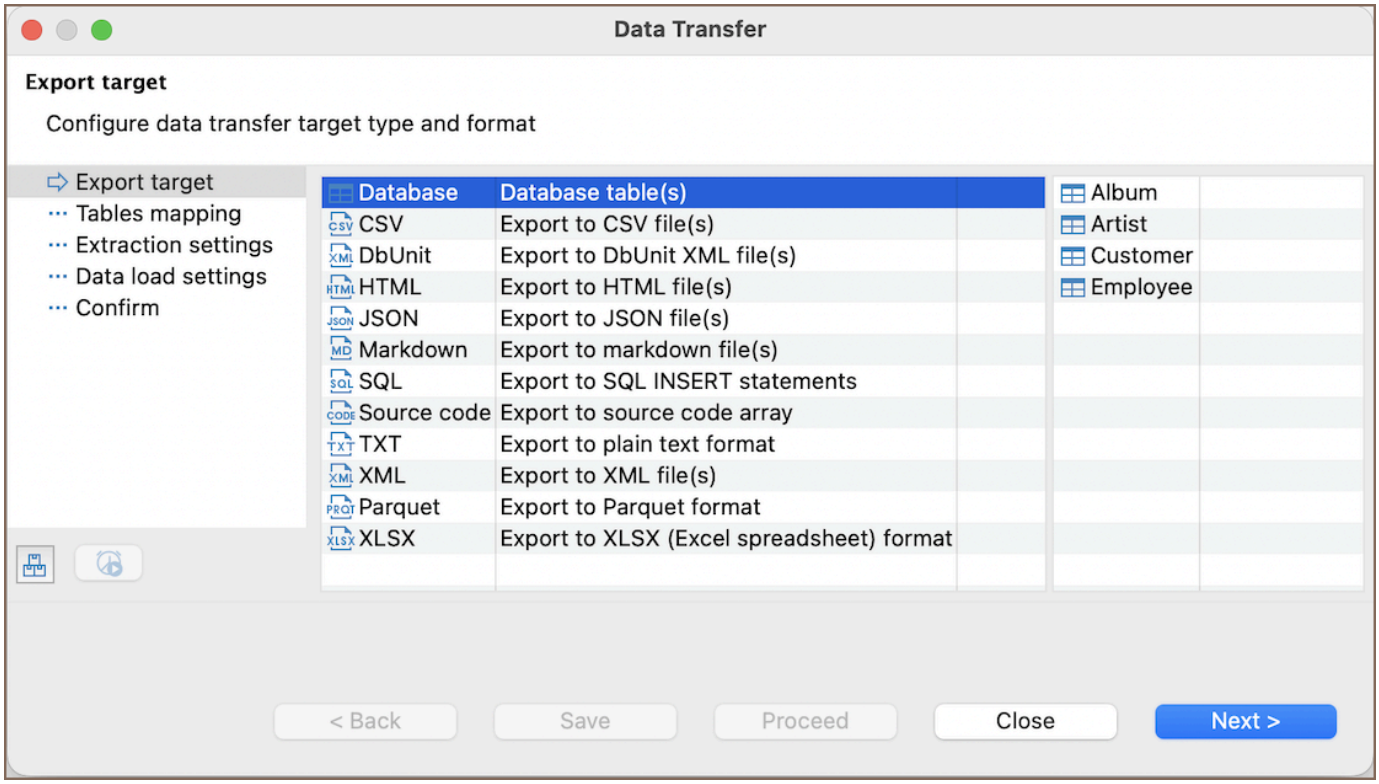


Depending on the selected format, importing allows you to:

- import data into tables from external files or sources.
- map columns from source files to database tables.
- adjust file-specific settings like headers, date formats, and `NULL` handling.
- use advanced options like multi-sheet imports or specifying a timezone.
- save import configurations as tasks for repeated use.

Transfer data between database tables

DBeaver provides data migration capabilities, allowing you to both export and import data between database tables. You can transfer data from one table to another within the same database or between different databases. For more detailed instructions, see Data Migration.



Data import

Table of contents

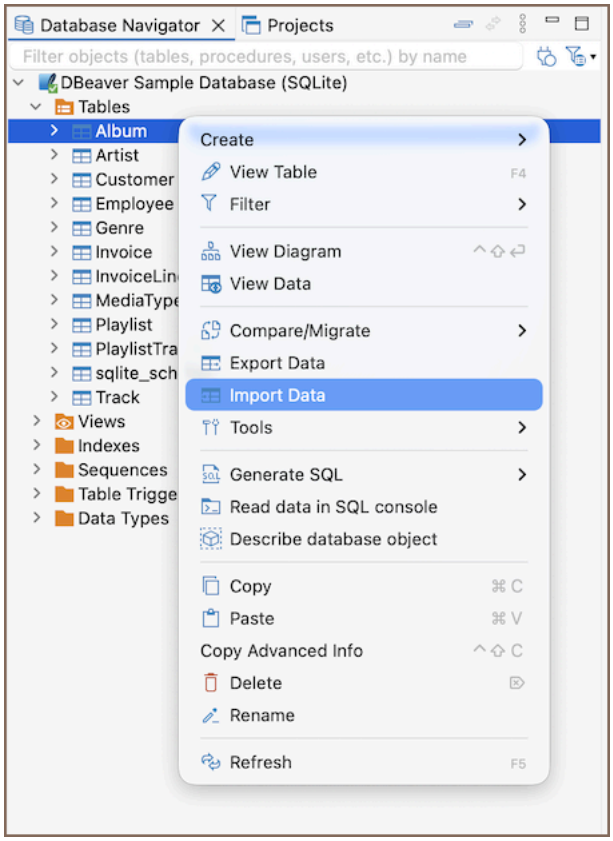
- Steps to import data
- Import parameters
- CSV
- XLSX
- XML
- Patterns for data and time format

DBeaver enables you to import data into tables. This guide explains the import process using CSV as an example. For details on supported formats, see Data Transfer.

> **Note**: Each format may have specific configuration settings that can be adjusted to meet your needs.

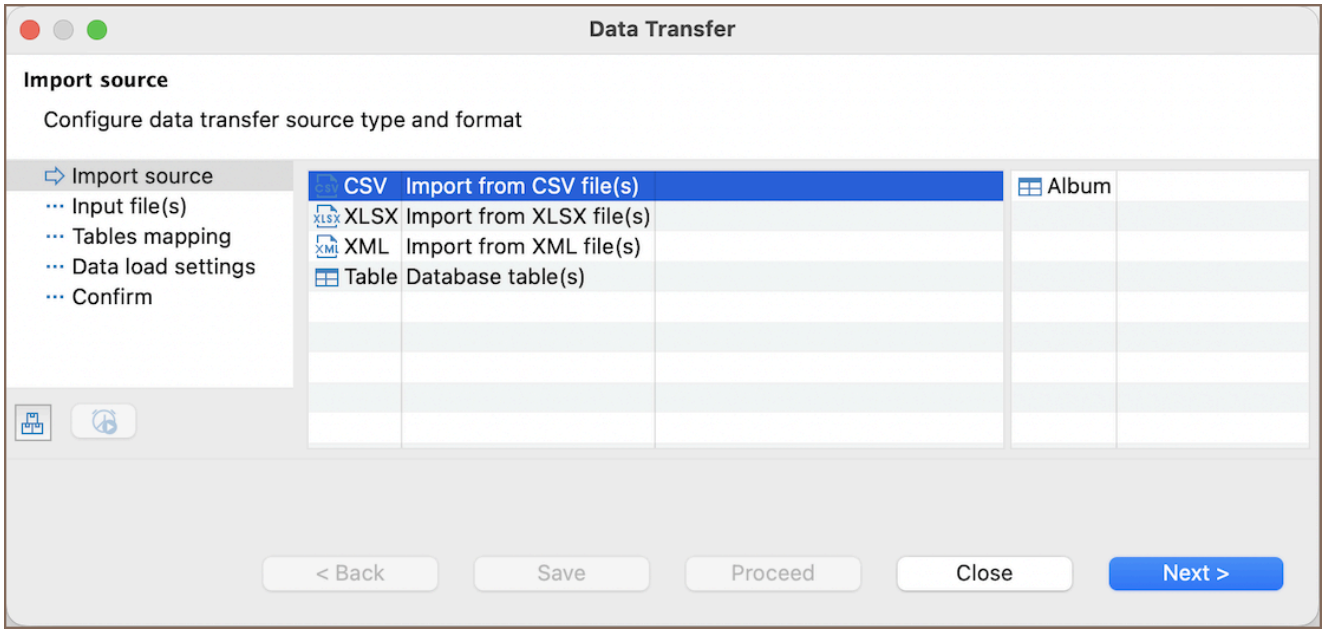
Steps to import data

1. Choose the database table (or tables) you want to import data into. Do this by right-clicking on the table name in the **Connections** section and then clicking on **Import Data**.

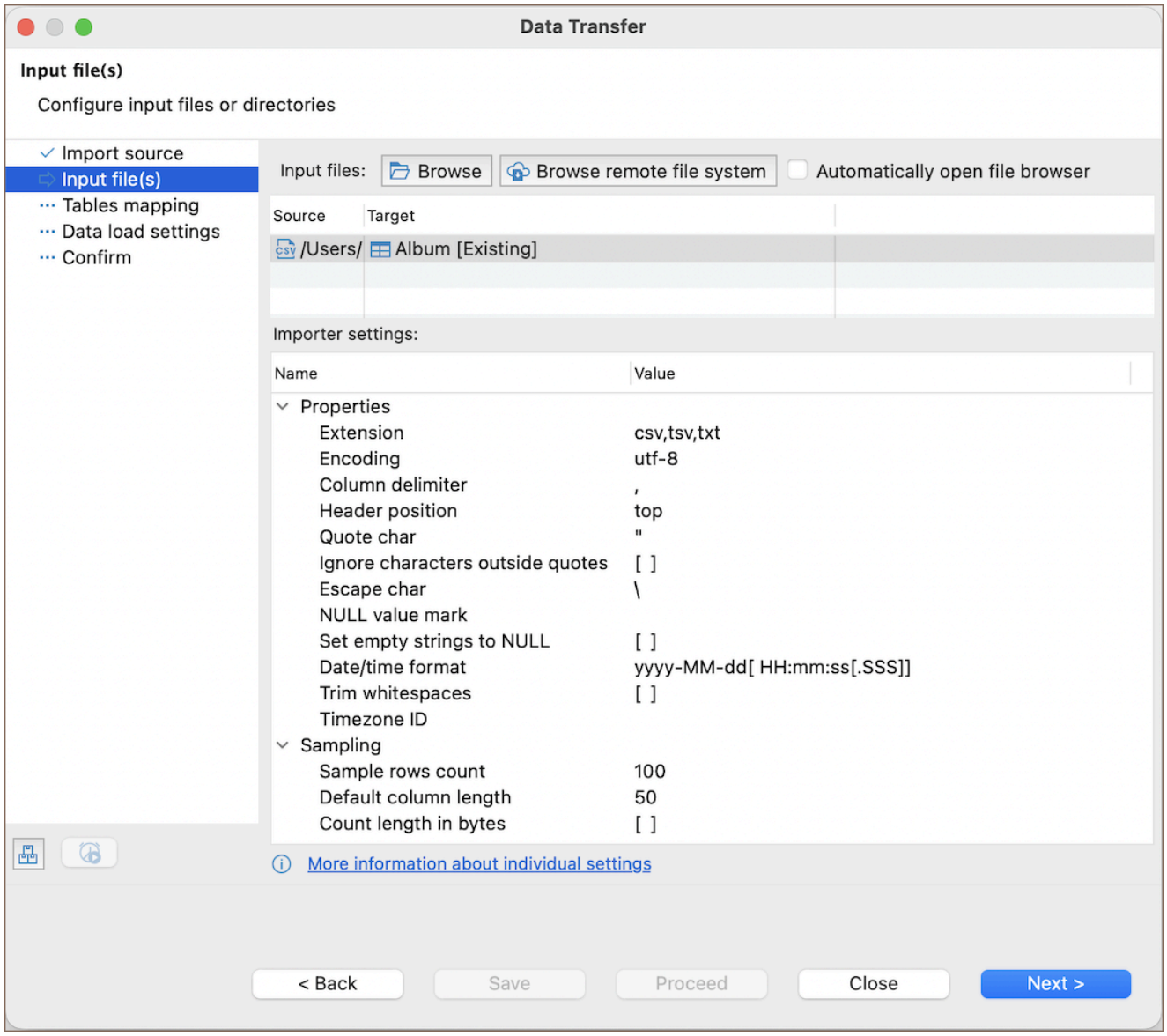


> **Tip**: You can also start the import through AI:
>
> - by uploading files in AI Chat
> - by using Data transfer actions commands

2. In the window that appears, choose csv and click **Next**.



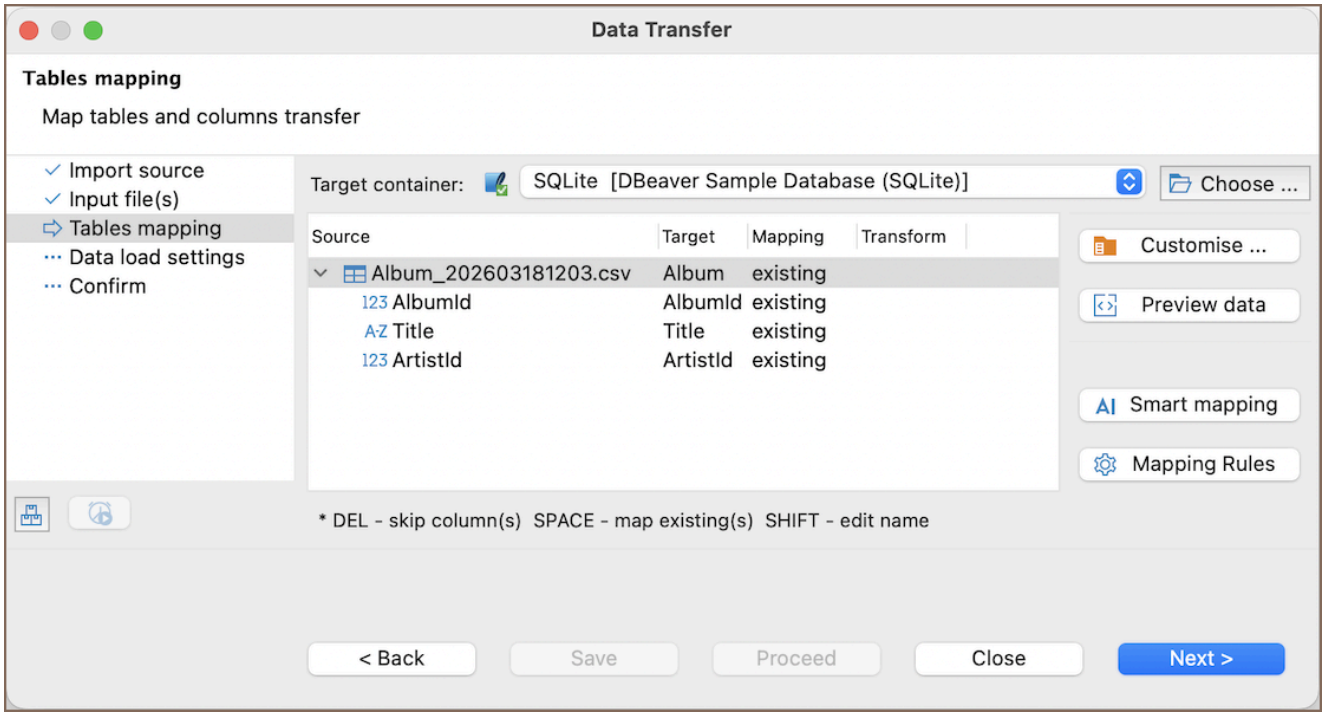
3. In the following window, choose the file that contains the data you wish to import into the table, select the appropriate settings, then click **Next**. For more details on importer settings, see Import parameters.



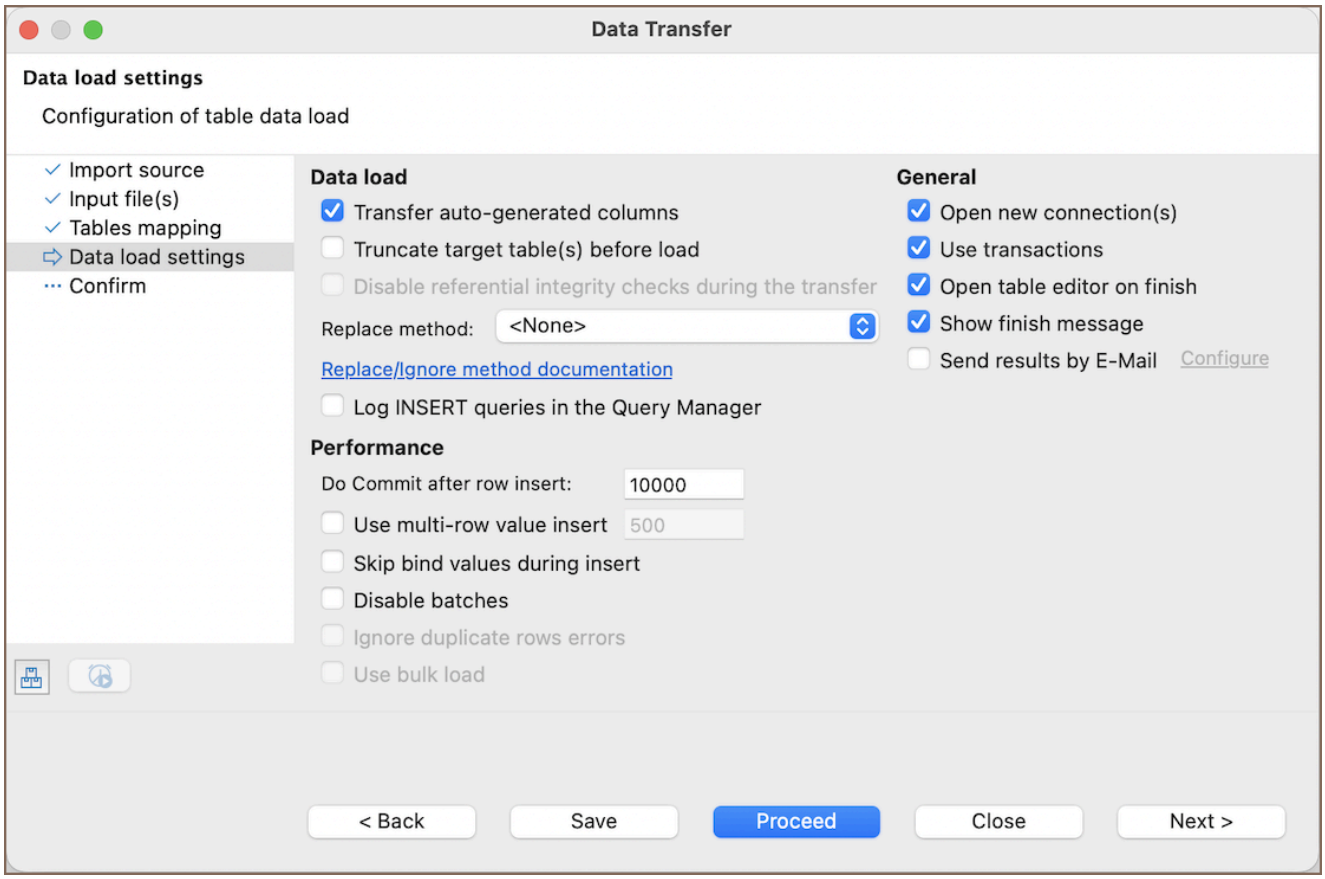
> **Tip**: You can import files from a remote file system using the Cloud Storage by clicking the **Browser remote file system** button . This feature is exclusively available to users of the Ultimate Edition, Team Edition, and CloudBeaver PRO versions.

4. In the next window, set CSV-to-table mappings.

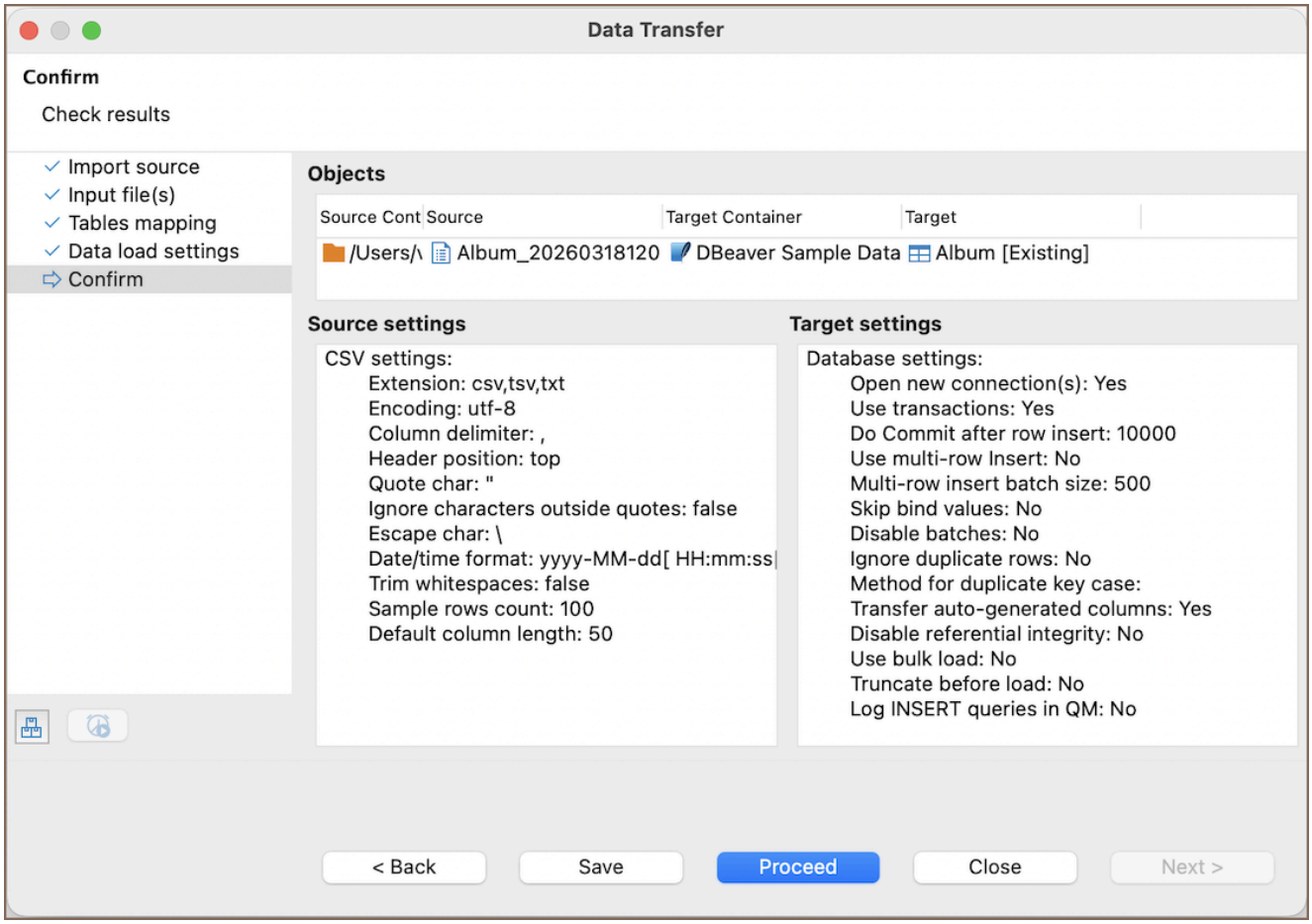
> Refer to our mapping process guide for more detailed information.



5. Select your data load settings in the subsequent window, and then click **Next**. For more information, please refer to our article's section **Data load settings** article.

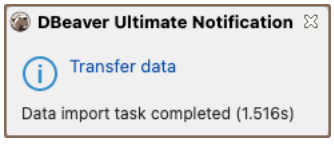


6. In the final window, you can review all the settings you selected earlier. If you missed something, you could go **Back** and fix it. When you are ready, finish the import by clicking **Proceed**.



> **Tip**: Use **Save** button to open the task window and quickly create or configure a data transfer task.

7. If the csv file is valid and there are no errors, you will see a notification window with information about the completion of the task. You can keep working with your database during export, as the data loading will be performed in the background.



Import parameters

CSV

Setting name	Description
Encoding	Character encoding of the file (for example, `UTF-8`).
Column delimiter	Character that separates columns. You can also use special symbols like `\t` (tab), `\n` (newline), or `\r` (carriage return).
Header position	Defines if the first row is used as column names (`top`) or if the file has no headers.
Quote char	Character used to wrap values (for example, `"`).
Ignore characters outside quotes	Any characters outside quotes that should be ignored during parsing.
Escape char	Character used to escape special characters (for example, `\`).
NULL value mark	String used as a marker for `NULL`. These values are imported as `NULL`.
Set empty strings to NULL	If enabled, empty strings are converted to `NULL`. Otherwise, they are kept as empty values.
Date/time format	Pattern that defines how to read date and time values. Doesn't change the output format. See *java DateTimeFormatter* for details.
Trim whitespaces	Removes spaces before and after values to prevent parsing errors.
Timezone ID	Timezone for date/time values. By default, the local machine timezone is used. You can set: - A local offset (e.g., `+3`, `-04:30`) - A GMT/UTC offset (e.g., `GMT+2`, `UTC+01:00`) - A region ID (e.g., `UTC`, `ECT`, `PST`).
Sample rows count	Number of rows to analyze for guessing column length and data types.
Default column length	Default length for string columns when creating new ones.

Setting name	Description
Count length in bytes	Counts column length in bytes instead of characters. Useful for databases that measure string length in bytes.

XLSX

Setting name	Description
Header position	Determines the location of the column names in the Excel table, either at the top or none. This setting specifies whether the column names are located in the first row of the Excel table or if there are no column names present.
Skip empty rows	If this setting is enabled, any open string values encountered during the data processing will be ignored and not inserted into the corresponding cells in the row. If the setting is disabled, all cells in the row will be filled with a `NULL` value if an empty string is encountered.
Import all sheets	Specifies that all sheets in the file should be imported during the data import process.
Specific sheet name	Enables you to choose a particular sheet from the Excel file for importing during the data transfer process.
Date/time format	Use this setting to specify the date format used in the `XLSX` file. This is used to clarify the date format during the import process and does not affect the output data. You can refer to the section Patterns for data import for details on the format pattern syntax.
Timezone ID	The local machine timezone is used by default. There are three ways to specify the timezone: 1) Local zone offset: Specify the offset from UTC in the format of either a positive or negative number (e.g., +3, -04:30). 2) Specific zone offset: Specify the offset from GMT or UTC in the format of GMT+/-X or UTC+/-X (e.g., GMT+2, UTC+01:00). 3) Region-based: Specify the timezone using a region-based identifier such as UTC, ECT, PST, etc.
Sample rows count	Determines the number of rows that will be used as a sample to estimate the length and data types of the imported data.
Minimum column length	This value is used when creating a new column and, if necessary, specifying its type. It indicates the minimum number of characters or digits expected in the column. This information helps determine the appropriate data type and size for the column during the creation process.

XML

Setting name	Description
Column read method	Defines how to extract values from XML. Options: - **elements** - use nested elements inside item elements - **attributes** - use attributes of item elements - **both** - use both elements and attributes
Item element	Name of the element treated as a row (item). If empty, any second-level elements are treated as items.

Patterns for data and time format

When specifying the date/time format for data import, ensure that you use the correct format specifiers. A common issue involves confusing format codes, such as using `MM` (for months) when `mm` (for minutes) is intended.

Refer to the table below for a list of common format symbols and their meanings.

Symbol	Meaning	Presentation	Examples
`G`	era	text	AD; Anno Domini; A
`u`	year	year	2004; 04
`y`	year-of-era	year	2004; 04
`D`	day-of-year	number	189
`M/L`	month-of-year	number/text	7; 07; Jul; July; J
`d`	day-of-month	number	10
`Q/q`	quarter-of-year	number/text	3; 03; Q3; 3rd quarter
`Y`	week-based-year	year	1996; 96
`w`	week-of-week-based-year	number	27
`W`	week-of-month	number	4
`E`	day-of-week	text	Tue; Tuesday; T
`e/c`	localized day-of-week	number/text	2; 02; Tue; Tuesday; T
`F`	week-of-month	number	3
`a`	am-pm-of-day	text	PM
`h`	clock-hour-of-am-pm (1-12)	number	12
`K`	hour-of-am-pm (0-11)	number	0
`k`	clock-hour-of-am-pm (1-24)	number	0
`H`	hour-of-day (0-23)	number	0
`m`	minute-of-hour	number	30
`s`	second-of-minute	number	55
`S`	fraction-of-second	fraction	978
`A`	milli-of-day	number	1234
`n`	nano-of-second	number	987654321

Symbol	Meaning	Presentation	Examples
N	nano-of-day	number	1234000000
V	time-zone ID	zone-id	America/Los_Angeles; Z; -08:30
z	time-zone name	zone-name	Pacific Standard Time; PST
O	localized zone-offset	offset-O	GMT+8; GMT+08:00; UTC-08:00;
X	zone-offset 'Z' for zero	offset-X	Z; -08; -0830; -08:30; -083015; -08:30:15;
x	zone-offset	offset-x	+0000; -08; -0830; -08:30; -083015; -08:30:15;
Z	zone-offset	offset-Z	+0000; -0800; -08:00;

> For more information on valid patterns, see the Java DateTimeFormatter Patterns.

Data export

Table of contents

- Steps to export data
- Export parameters
- CSV
- DBUnit
- JSON
- Markdown
- Source code
- SQL
- TXT
- XML
- XLSX
- HTML

DBeaver allows you to export query results, tables, or data in the Data Editor. This guide explains the export process using csv as an example. For details on supported formats, see Data Transfer.

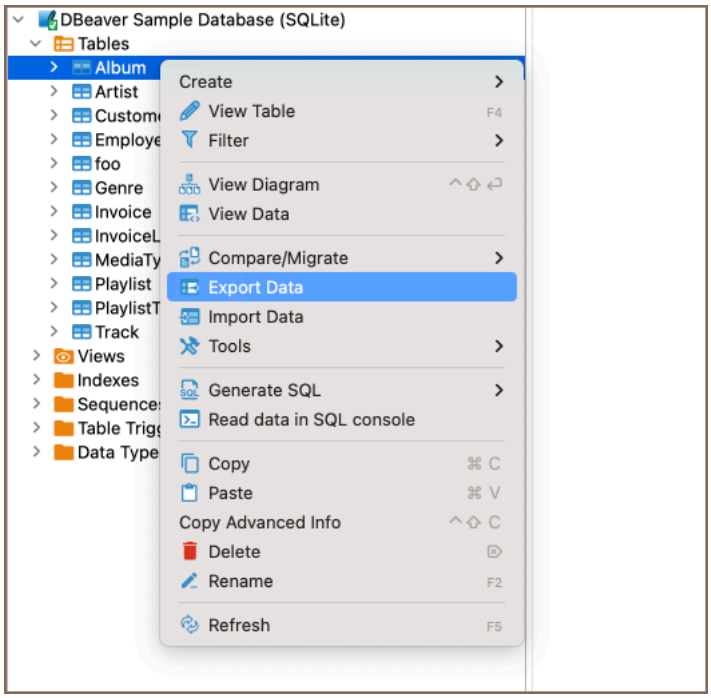
For information on generating and saving DDL for database objects, see SQL Generation.

> **Note**: Each format may have specific configuration settings that can be adjusted to meet your needs.

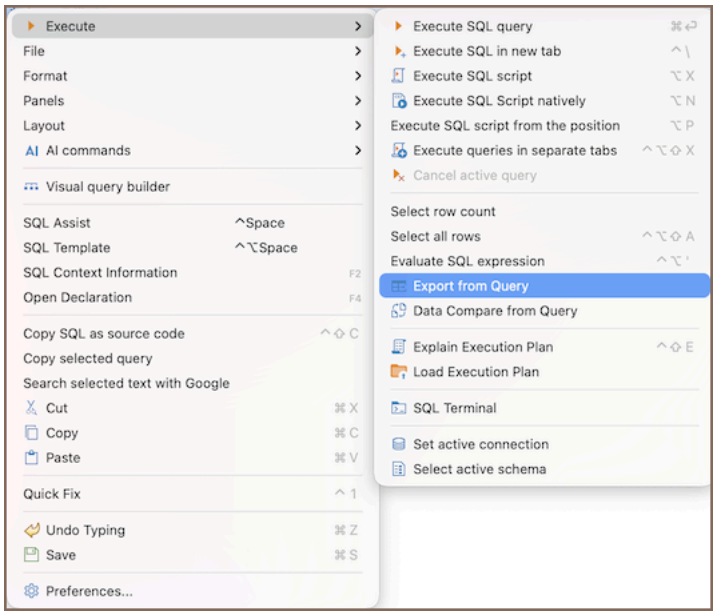
> For database-to-database transfers, see Data migration.

Steps to export data

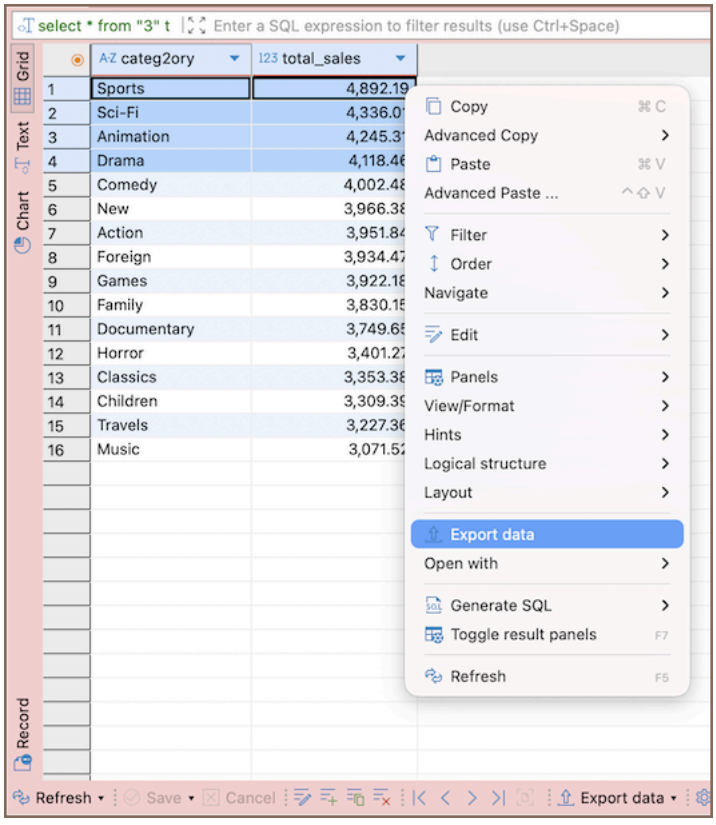
1. Start the export:
    - Connections - right-click a table and choose **Export Data**



- SQL script - use **Execute -> Export from Query**

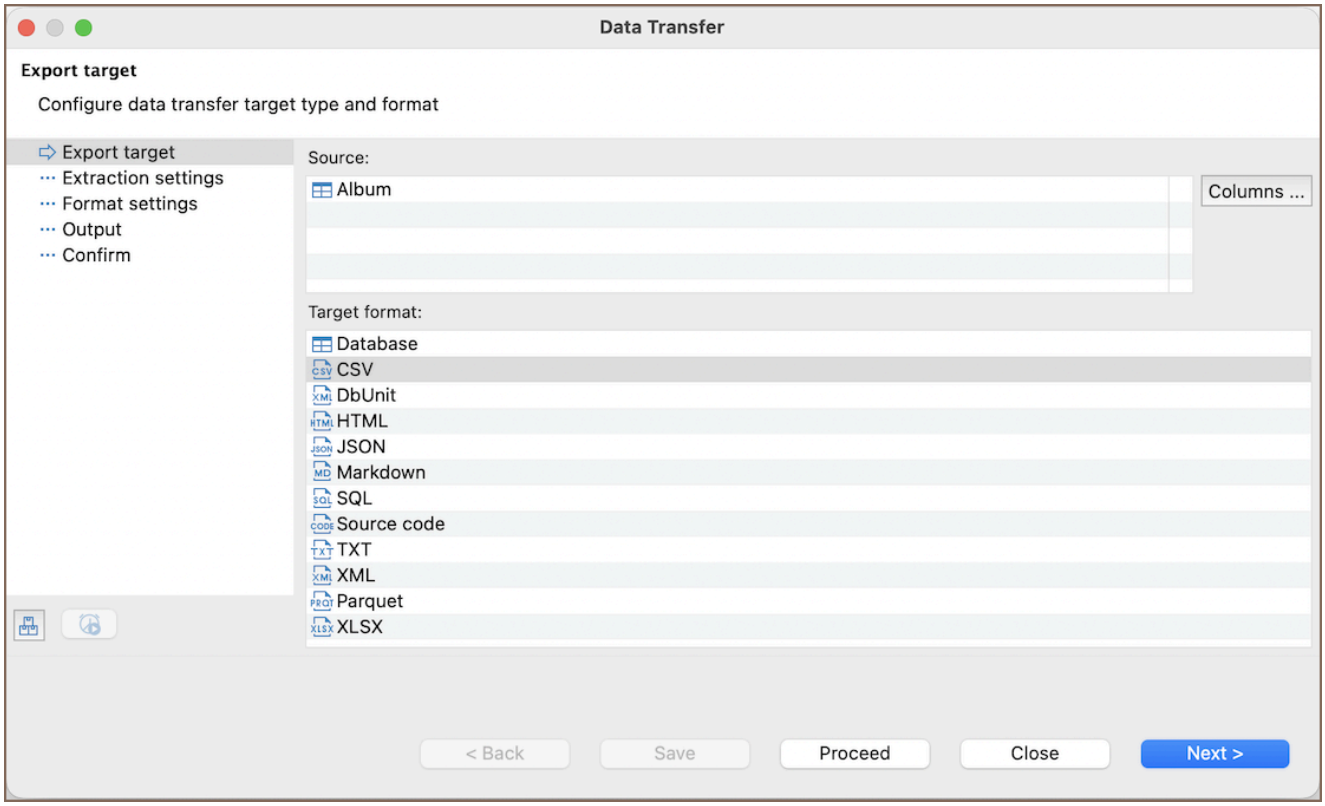


- query results or Data Editor - right-click the grid and choose **Export data**



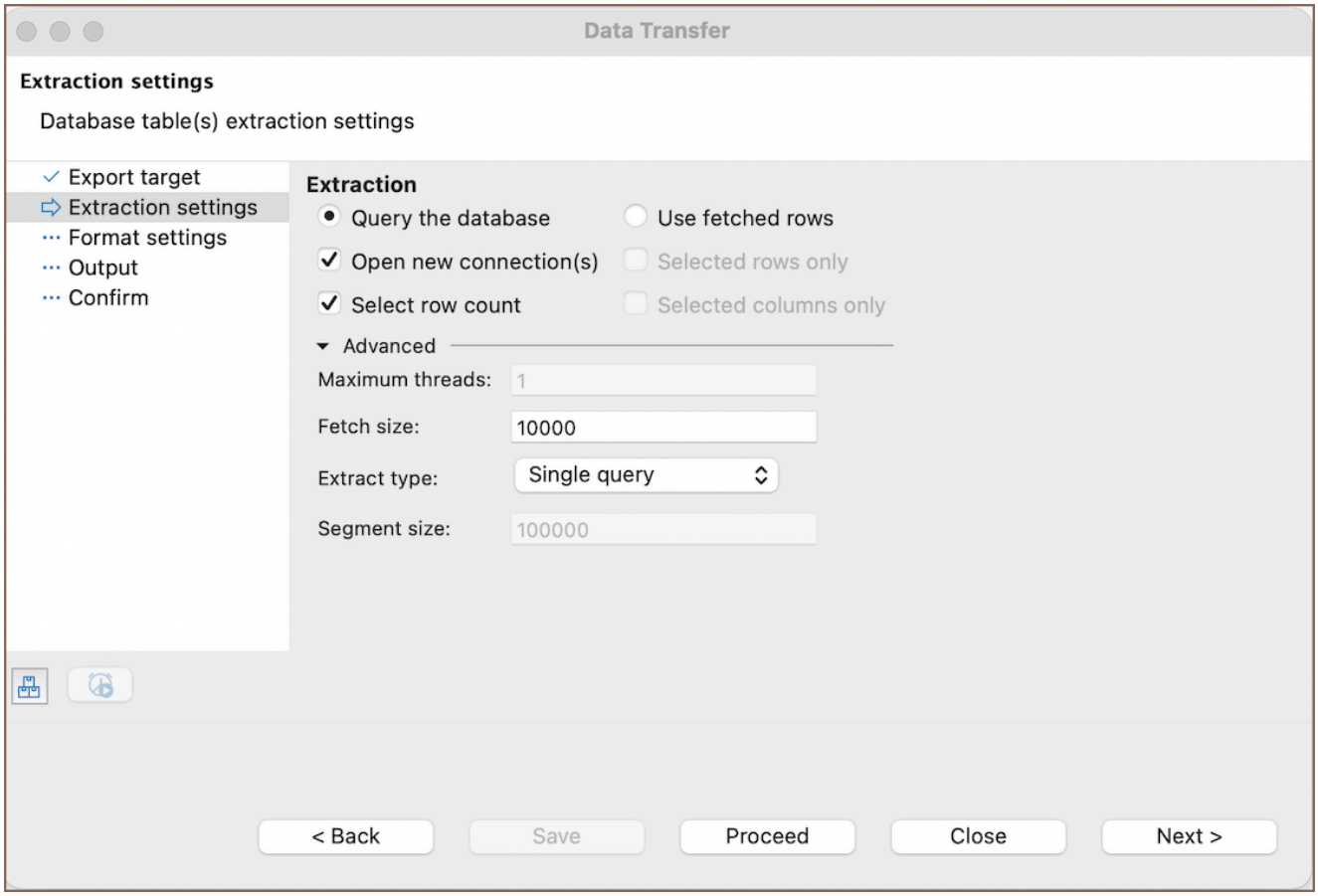
> **Tip**: You can start the export through AI by sending a short request in AI chat. See Data transfer actions for supported commands.

2. In the window that appears, choose csv and click **Next**.



Set which columns to export. Click **Columns** to open the column list. Select the checkbox next to the table to export all columns, or select individual columns to export only specific data.

3. Set the data extraction options (how data will be read during export) and the format options for your chosen format. Extraction options may affect performance:



Setting name	Description
Query the database	Executes a query during export and reads data directly from the database.
Open new connection(s)	Opens a new physical connection for data reading, recommended if you plan to continue working with the database during the export process.

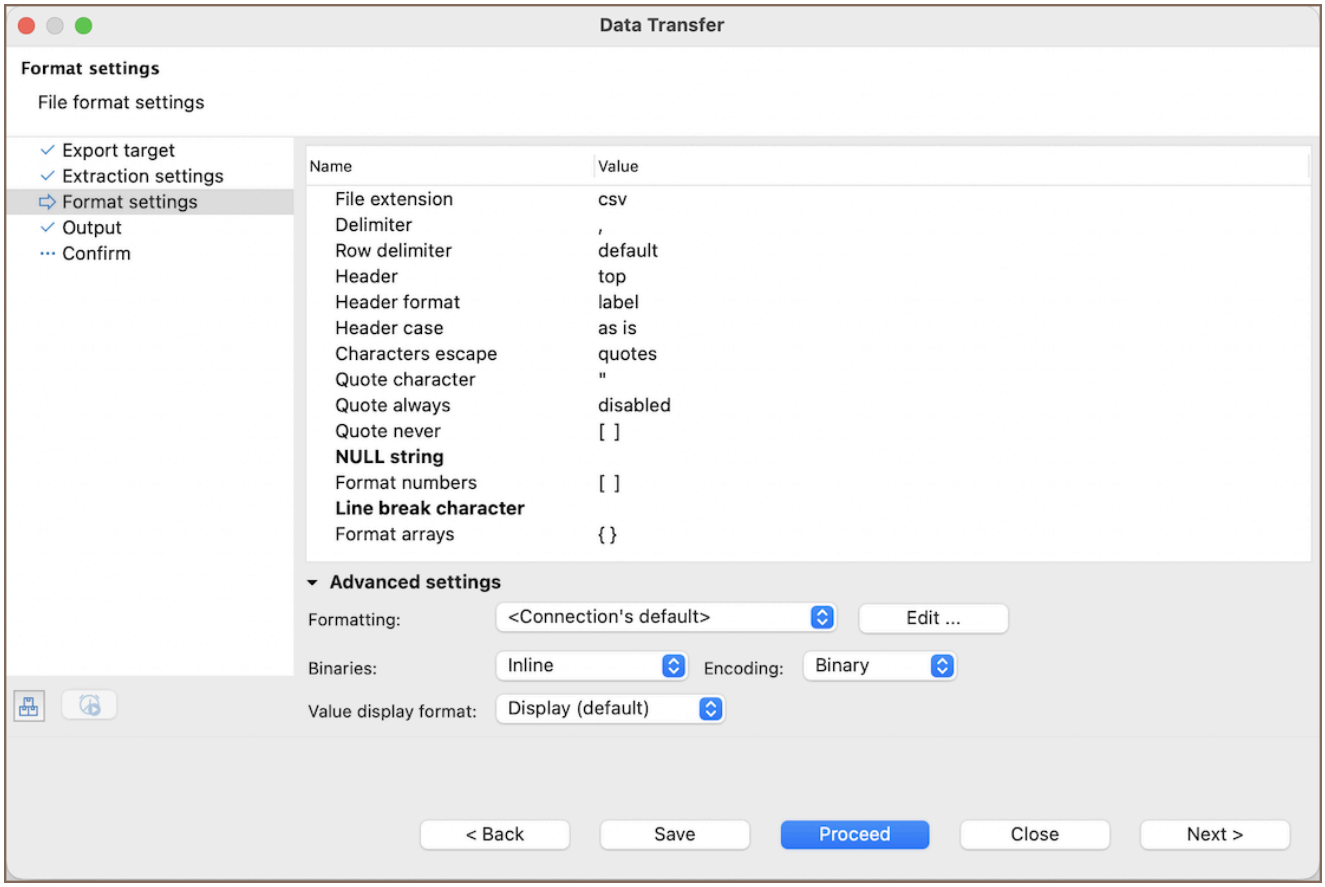
Setting name	Description
Select row count	Queries row count before performing export to track progress, but may cause performance issues in some cases.
Use fetched rows	Exports data already loaded in the result set without executing a new query.
Selected rows only	Exports only selected rows from the result set.
Selected columns only	Exports only selected columns from the result set.
Maximum threads	Number of threads used for data extraction.
Fetch size	Number of rows the driver retrieves in one batch from the database.
Extract type	Defines the method of data extraction (`Single query` or `Multiple queries`).
Segment size	The size of data segments to be extracted at a time (active while Multiple queries are selected as an **Extract type**).

> **Tip**: The **Maximum threads** option is active when exporting data from multiple tables. It is recommended to set the number of threads to match the number of cores in your computer for optimal performance.

> **Note**: **Use fetched rows**, **Selected rows only**, and **Selected columns only** are available only when exporting from query results or the Data Editor. When exporting from the Connections, or SQL script, data is always read directly from the database.

4. Configure the format settings.

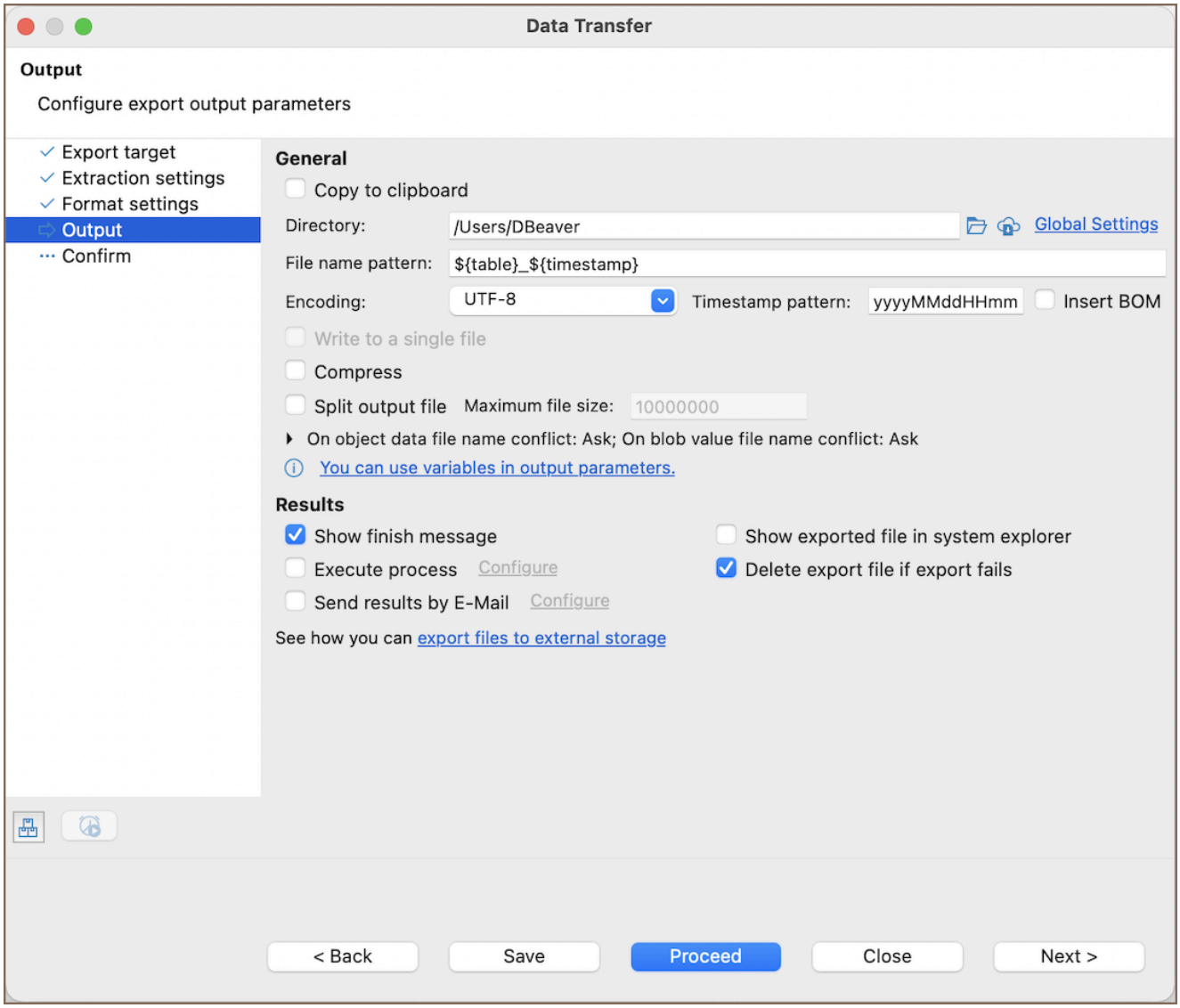
   At this stage, you configure the format settings for the output file. These settings determine how the data will be presented and encoded in the exported file. For more details on exporter settings, see Export parameters.



Setting name	Description
Formatting	Selects the formatting settings for the exported data. Reed more in the related article
Binaries	Specifies how binary data is represented in the export.
Encoding	Sets the character encoding for the exported file.
Value display format	Chooses the format for displaying values in the export.

> **Note**: The options within the Exporter settings section will vary and provide specific controls tailored to the format selected for export.

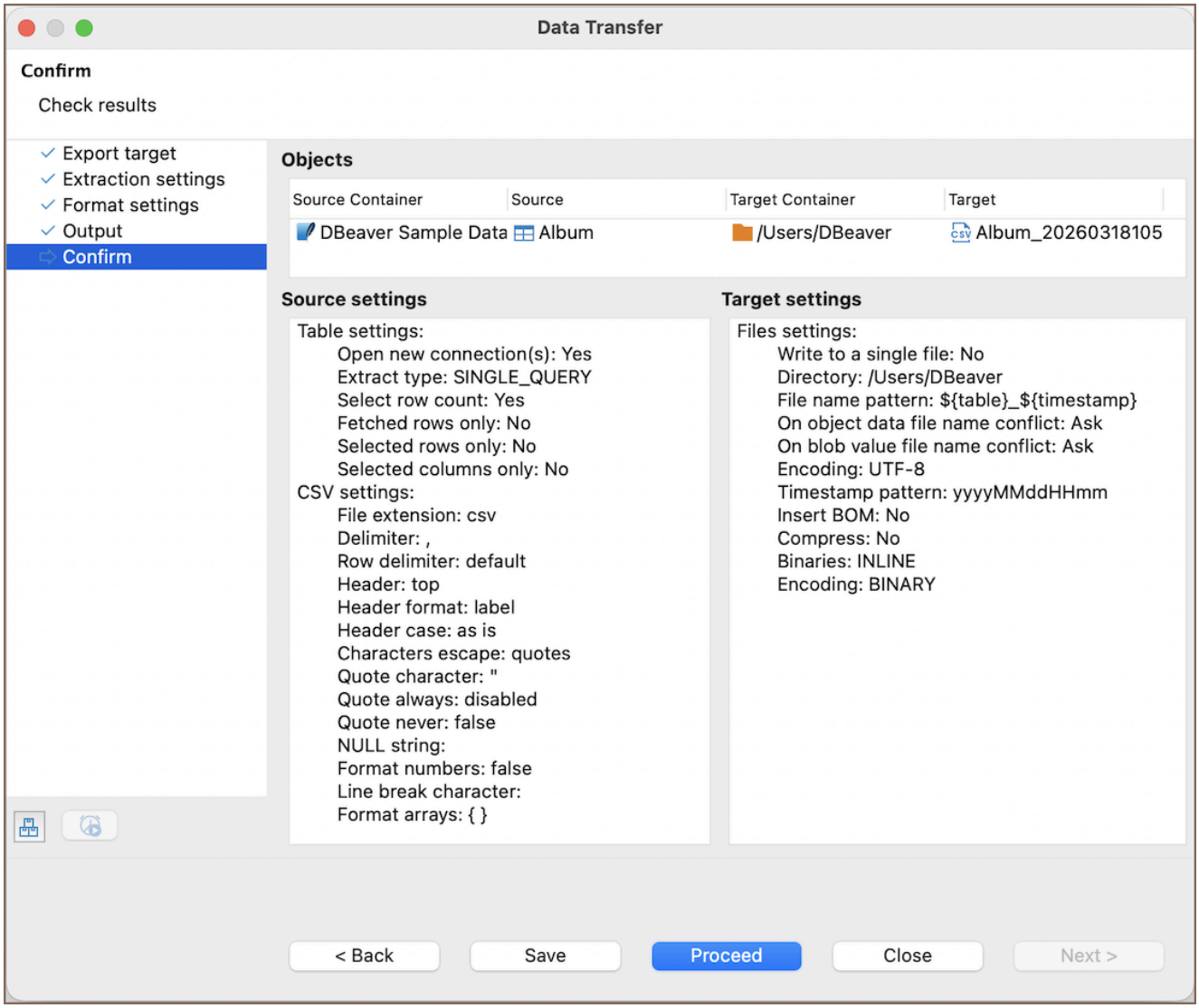
5. Set options for output files.



Choose where the exported data will be saved - local files ( ), or external storage.

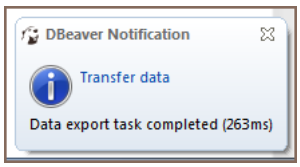
> For details on exporting data to external storage, see Data transfer to external storage.

6. Review what you want to format and which format you will export. You can also save all your settings as a task in this step or change the task variables:



> **Tip**: Use **Save** button to open the task window and quickly create or configure a data transfer task.

7. Click **Finish** to start the export. If the wizard shows **Proceed**, click it to continue through the remaining configuration pages, then click **Finish** on the final page. You can keep working with your database while the export runs in the background.



Export parameters

In this section, you will find specific configuration settings for each supported export format. These settings allow you to customize the export process according to the requirements of the data format you choose.

CSV

Setting name	Description	Available options
Characters escape	Bad characters escaping model (surrounded with quotes or escaped with '\' character).	`quotes/escape`
Delimiter	Column delimiter.	You can use special characters like `\t`, `\n`, and `\r`.
File extension	The default file extension for the exported file.	`csv`
Format numbers	Format numeric values using locale settings.	`true/false`
Header	CSV header settings.	`none/top/bottom`
Header case	You can choose lower or upper case for column names or descriptions in the header.	`as is/upper/lower`
Header format	Defines the formatting of the header.	`label/description/both`

Setting name	Description	Available options
NULL string	String which will be used instead of NULL values.	
Quote always	Quote all cell values. Cannot be used with "Quote Never".	`disabled/all/strings/all but numbers/all including nulls`
Quote character	Character which will be used to quote strings (space means no quote).	
Quote never	Do not quote cell values. Cannot be used with "Quote Always".	`true/false`
Row delimiter	Row delimiter. Default is system-specific line feed delimiter.	`default/\n/\r/\r\n/\n\r`

DBUnit

Setting name	Description	Available options
File extension	The default file extension for the exported file.	`xml`
Force upper case column names	Convert all column names to upper case.	`true/false`
Force upper case table name	Convert the table name to upper case.	`true/false`
Include NULL values in export	Include NULL values in the exported data.	`true/false`
Replace NULL values with	Specify a string to replace NULL values in the export.	

JSON

Setting name	Description	Available options
File extension	The default file extension for the exported file.	`json`
Format dates in ISO 8601	Convert all date values to ISO 8601 format.	`true/false`
Print table name	Include the table name in the exported file.	`true/false`
Export JSON values as	Export values as text or as JSON objects.	`string/JSON`

Markdown

Setting name	Description	Available Options
Confluence format	Enable Confluence-specific Markdown formatting.	`true/false`
File extension	The default file extension for the exported file.	`md`
Format numbers	Format numeric values using locale settings.	`true/false`
NULL string	String to represent NULL values in the export.	
Show header separator	Include a separator line below the header.	`true/false`

Source code

Setting name	Description	Available options
File extension	The default file extension for the exported file.	`php`
Format dates in ISO 8601	Convert all date values to ISO 8601 format.	`true/false`
Language	Specifies the PHP version compatibility for the generated code.	`PHP < 5.4`, `PHP 5.4+`
Quote character	Character which will be used to quote strings.	`"`, `'`
Row delimiter	Row delimiter. Default is system-specific line feed delimiter.	`default`, `\n`, `\r`, `\r\n`, `\n\r`

SQL

Setting name	Description	Available options
Data rows per statement	Specifies the number of data rows in a single insert statement.	`integer`
File extension	The default file extension for the exported file.	`sql`
Identifier case	Allows selection of lower or upper keyword case for table and column names.	`as is/upper/lower`
Include generated columns	Specifies whether to include auto-generated columns (e.g., auto-increment) in SQL INSERT statements.	`true/false`
Insert line before rows	Specifies inserting a line feed before values in multi-row inserts.	`true/false`
Keyword case	Allows selection of lower or upper keyword case.	`upper/lower`
Native date/time format	Specifies using native date/time format in INSERT statements.	`true/false`
Omit schema name	Specifies omitting schema/catalog name in INSERT statements.	`true/false`
On conflict expression	Provides an expression for the end of the statement. This setting is specific to the database.	
Target table name	Allows specification of the target table name to generate an INSERT statement.	

Setting name	Description	Available options
Upsert keyword	Allows selection of different upsert keywords.	`INSERT/INSERT ALL/UPDATE OR/UPSERT INTO/REPLACE INTO/ON DUPLICATE KEY UPDATE/ON CONFLICT`

TXT

Setting name	Description	Available options
Batch size	Specifies the number of records per batch.	`integer`
File extension	Specifies the file type for output.	`txt`
In-between delimiter	Adds a custom character between data values.	
Max column length	Specifies the maximum length of data in a column; longer values will be cropped.	`integer`
Min column length	Specifies the minimum length of data in a column; shorter values will be padded with spaces.	`integer`
Print header	Specifies whether to print column names at the top of the file.	`true/false`
Show header delimiter	Adds hyphen characters either in the first row without a header or between the header and data.	`true/false`
Show leading delimiter	Adds a pipe character at the start of the row.	`true/false`
Show NULLs	Controls the display of NULL values in the output.	`true/false`
Show trailing delimiter	Adds a pipe character at the end of the row.	`true/false`

XML

Setting name	Description	Available options
File extension	Specifies the file type for output.	`xml`
Include DOCTYPE declaration	Specifies whether to include the DOCTYPE declaration in the XML file export.	`true/false`

XLSX

Setting name	Description	Available options
Append strategy	Strategy used when appending data to an existing file.	`create new sheets/use existing sheets`
Boolean string FALSE	String that replaces FALSE boolean values in the exported file.	`true/false`
Boolean string TRUE	String that replaces TRUE boolean values in the exported file.	`true/false`
Border style	Style of cell borders in the exported file.	`NONE/THIN/THICK`
Excel date format	Date and time format used in the Excel file (modifiable in Excel application).	`m/d/yy` / `d-mmm-yy` / `d-mmm` / `mmm-yy` / `h:mm AM/PM` / `h:mm:ss AM/PM` / `h:mm` / `h:mm:ss` / `m/d/yy h:mm`
Column group	Column number used for grouping rows on a sheet by column value.	`integer`
Export SQL	Specifies whether to export SQL to a second sheet in the Excel file.	`true/false`
File extension	File extension for the exported document.	`xlsx`
Header format	Format of the header in the exported file.	`label/description/both/none`
Header row font	Font styling for the first row in the exported file.	`NONE/BOLD/ITALIC/STRIKEOUT/UNDERLINE`
Max row on sheet	Maximum number of rows allowed on a single sheet, after which data will split into another sheet.	`integer`
NULL string	String used instead of displaying NULL values in the exported file.	
Row number(s)	Includes the row index as the first column in the exported file.	`true/false`
Split SQL Text	Splits exported SQL into rows by CR (Carriage Return) in the exported file.	`true/false`
Trim strings	Removes extra leading and trailing spaces from all string values in the exported file.	`true/false`

HTML

Setting name	Description	Available options
File extension	File extension for the exported document.	`html`
Images	Extracts images to graphic files.	`true/false`
Output column headers	Outputs column names as an additional row in the generated table.	`true/false`
Output table header	Outputs the query or table name as the first row in the generated table.	`true/false`

Data compare

> **Note**: This feature is available in Enterprise, and Ultimate editions only.

Table of contents

- How it works
- Start a comparison
- Compare two tables
- Compare fresh query results
- Compare results you already ran
- Manage compared objects
- Add a source and assign a target
- Remove a pair
- Change source or target
- Column mapping
- Keys configuration
- Composite keys
- Compare settings
- Execution log
- Viewing the results
- Troubleshooting
- No key columns are chosen
- No target specified

DBeaver lets you compare tables and query results to spot differences, track changes, and generate SQL scripts to sync your data.

You can compare:

- **Two tables** - select a pair and compare them directly.
- **Multiple tables** - select several tables at once and assign each a target to compare against.
- **Query results** - compare the output of a SELECT query, run fresh or reuse the rows already loaded in a result set.

> You can compare objects from the same or different databases, or even between different Relational Database Management Systems (RDBMS), such as PostgreSQL and MySQL.

How it works

1. Start a comparison - select your tables or queries
2. Manage compared objects - add pairs, assign source and target
3. Map columns - match columns between source and target
4. Configure keys - set which columns identify each row
5. Adjust settings - set limits, output format, and export options
6. View results - explore differences and export the SQL diff

Start a comparison

How you start Data Compare decides what you compare and whether DBeaver reads the data fresh or reuses rows you already have. Pick the scenario that matches your goal:

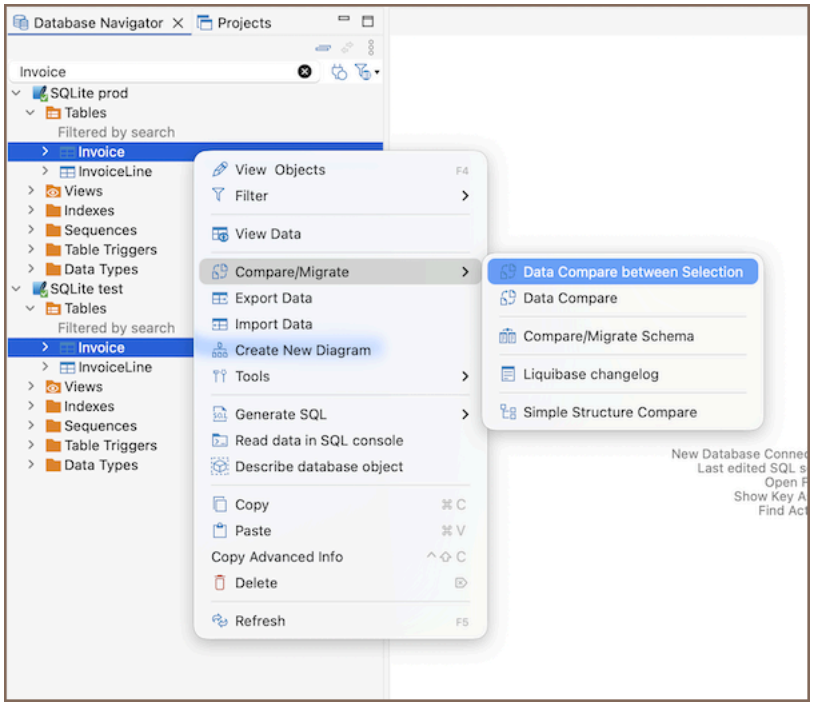
Goal	Where to start	What gets compared
Compare two tables	Connections	Data is read fresh when the compare runs
Compare fresh query results	SQL Editor	The query runs when the compare starts
Compare results you already ran	Data Editor result set	The rows already loaded in the grid, no re-run

Compare two tables

Start from the Connections.

> **Tip**: You can select more than two objects.

1. Select the tables, open the context menu, and go to Compare/Migrate.



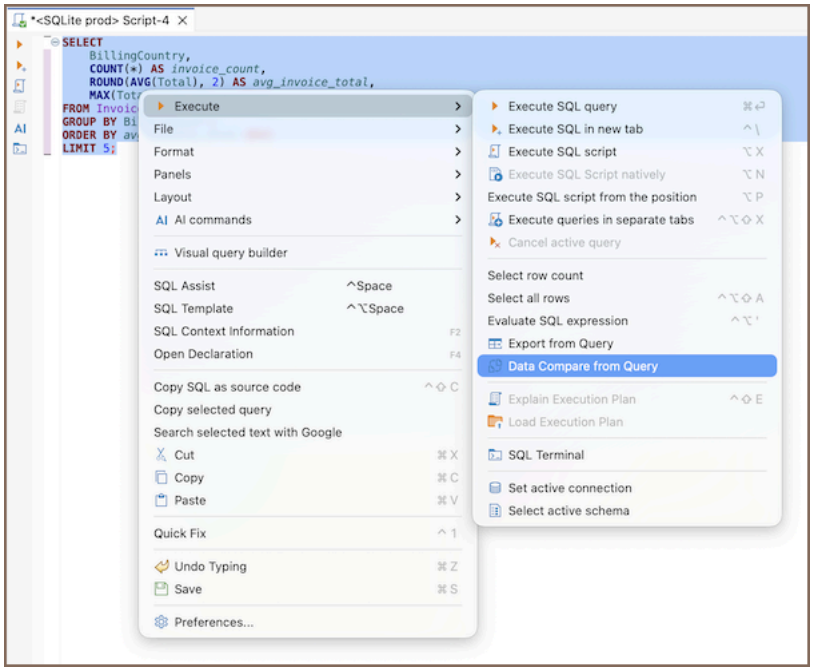
2. Choose an option:
   - **Data Compare** - opens the wizard with the selected tables as **source**. You then assign a **target** table or query for each source.
   - **Data Compare between Selection** - opens the wizard with the two selected tables prefilled as a source-target pair.

> **Note**: **Data Compare between Selection** is available only when you select exactly two objects.

Compare fresh query results

Start from the SQL Editor when you want the query to run as part of the comparison.

1. Write a SELECT query.



2. Select the query, open the context menu, and choose **Execute -> Data Compare from query**.

The query runs when the compare starts, so you always compare current data. You can select up to two queries at once - each one comes in as a separate source, and you assign a target for each.

> **Tip**: You can also start the compare through AI by sending a short request in AI chat. See Data transfer actions for supported commands.

Compare results you already ran

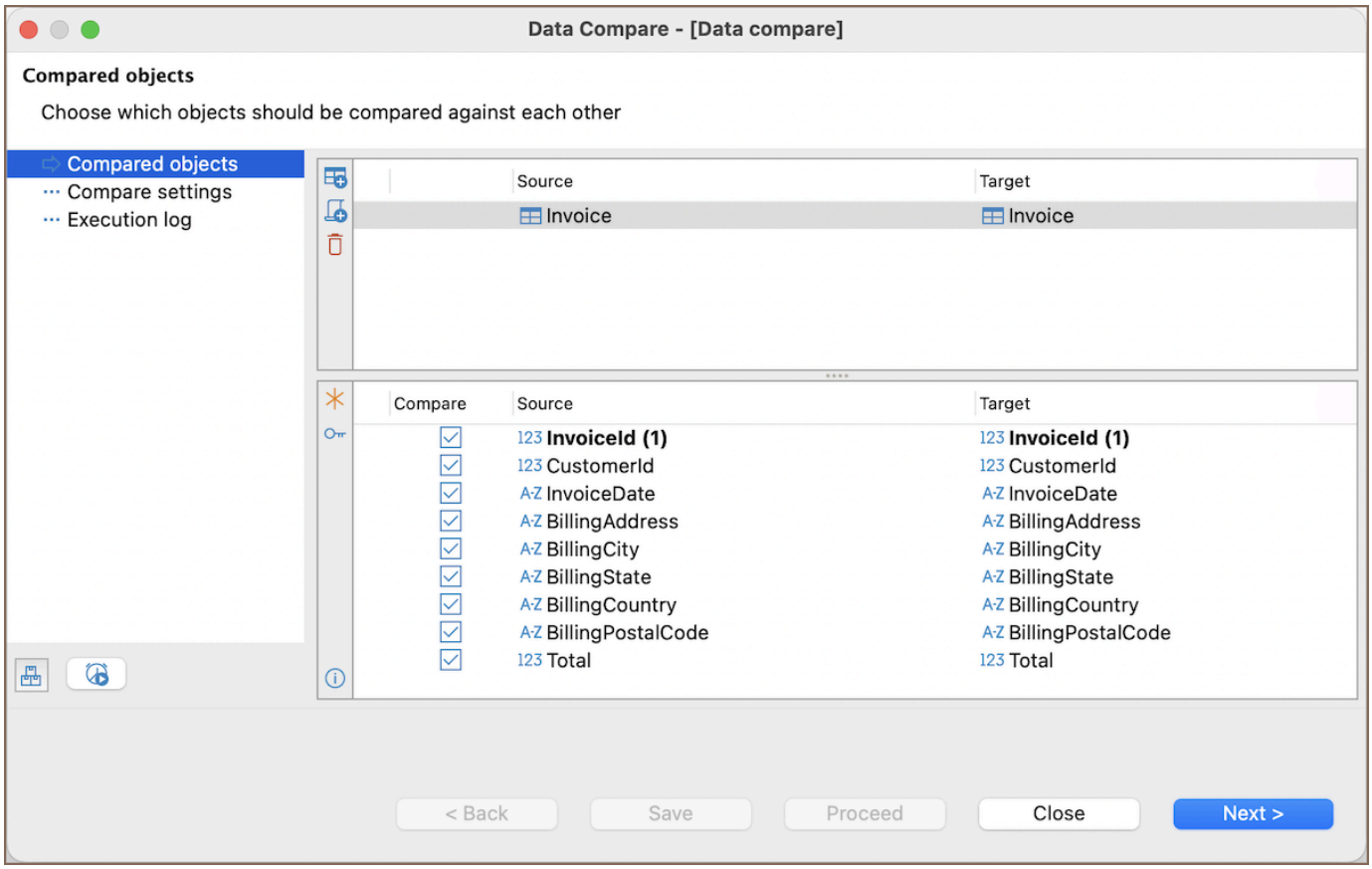
Start from a result set in the Data Editor when you've already run a query and want to compare those exact rows without running anything again. This is handy for checking data before and after a change, or for lining up two queries you've already executed.

1. Right-click anywhere in the result grid.
2. Point to **Data Compare to...**.
3. Do one of the following:
    - Select **Choose...** to open the wizard with the result set as **source**. You then pick a **target** table or query.
    - If another result tab is open in the same SQL Editor, select it from the submenu to compare the two result sets directly. Both sides use their already-loaded rows, and the wizard opens with source and target set.

> **Warning**: The comparison uses only the rows already fetched into the grid. If the result set loads lazily and you haven't scrolled to the end, the unfetched rows are left out. Load all rows first when you need a complete comparison.

> **Note**: Data Compare started this way uses the rows already loaded in the grid, not a fresh run of the query. If you save the comparison as a task, the task stores the query itself, so the next run executes it fresh instead of reusing the cached rows.

Once you start the comparison, the Data Compare wizard opens.



Manage compared objects

Use the **Compared objects** page to set up what you compare. Here you add and remove pairs, assign **source** and **target**, and configure column mapping and key columns for the selected pair.

> **Tip**: The wizard is prefilled based on how you start it. **Data Compare between Selection** prefills the **source** and **target**. **Data Compare** prefills only **source**. You then choose a **target** for each source.

Add a source and assign a target

1. On the **Compared objects** page, click one of the toolbar buttons:
    - click **Add table(s) as a source** button ( ) to add one or more objects as **source**
    - click **Add query as a source** button ( ) to add an SQL query result as **source**
2. For each added source, assign a target:
    - Click **Browse...** in the **target** column, then pick a table, or choose a query (depending on what you want to compare)

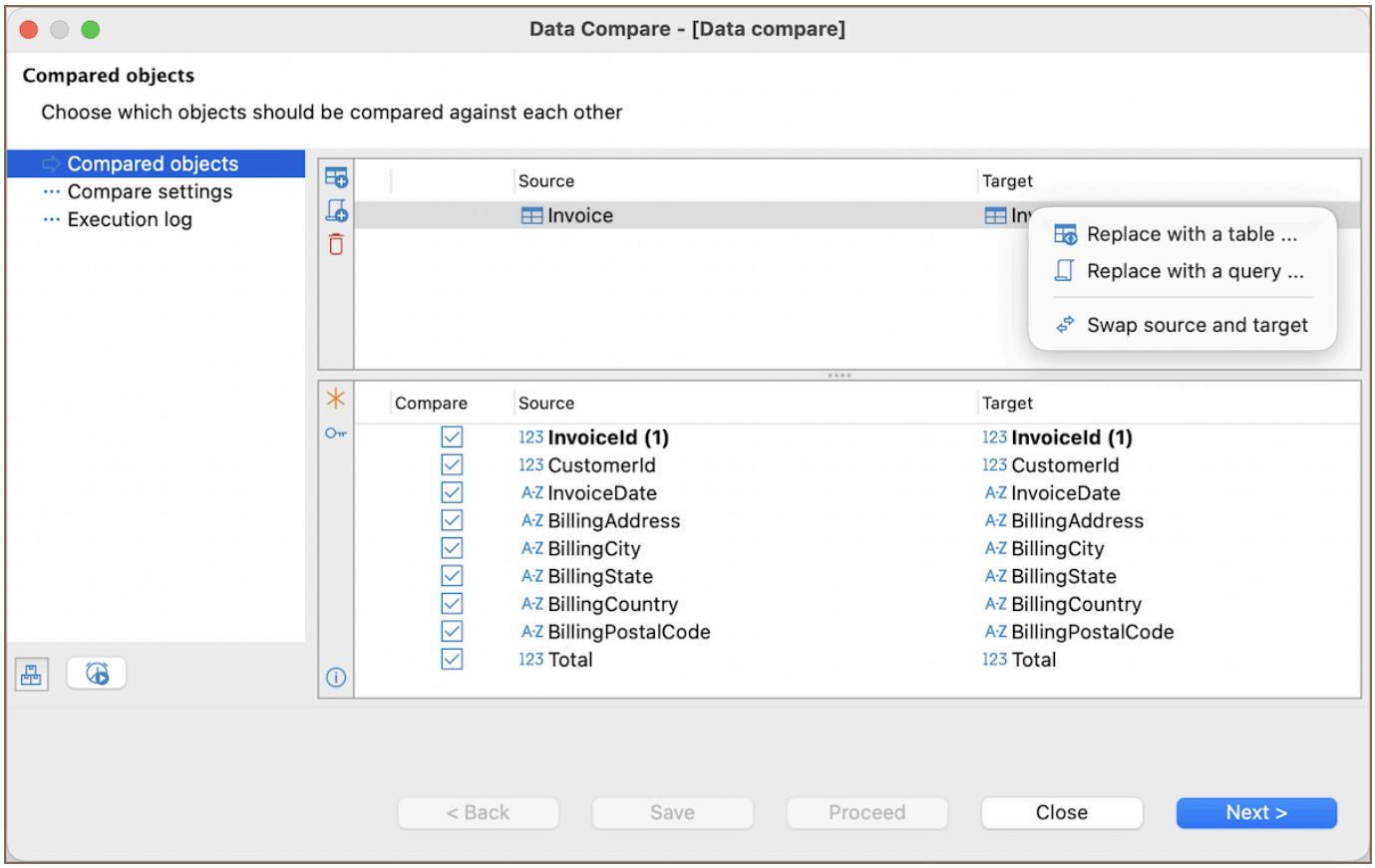
> **Tip**: If you add multiple source objects, the wizard creates multiple pairs. Assign a target for each source row.

Remove a pair

Select a row in the list, then click **Remove selected pair** button ( ).

Change source or target

Use the context menu on a cell in the **source** or **target** column:

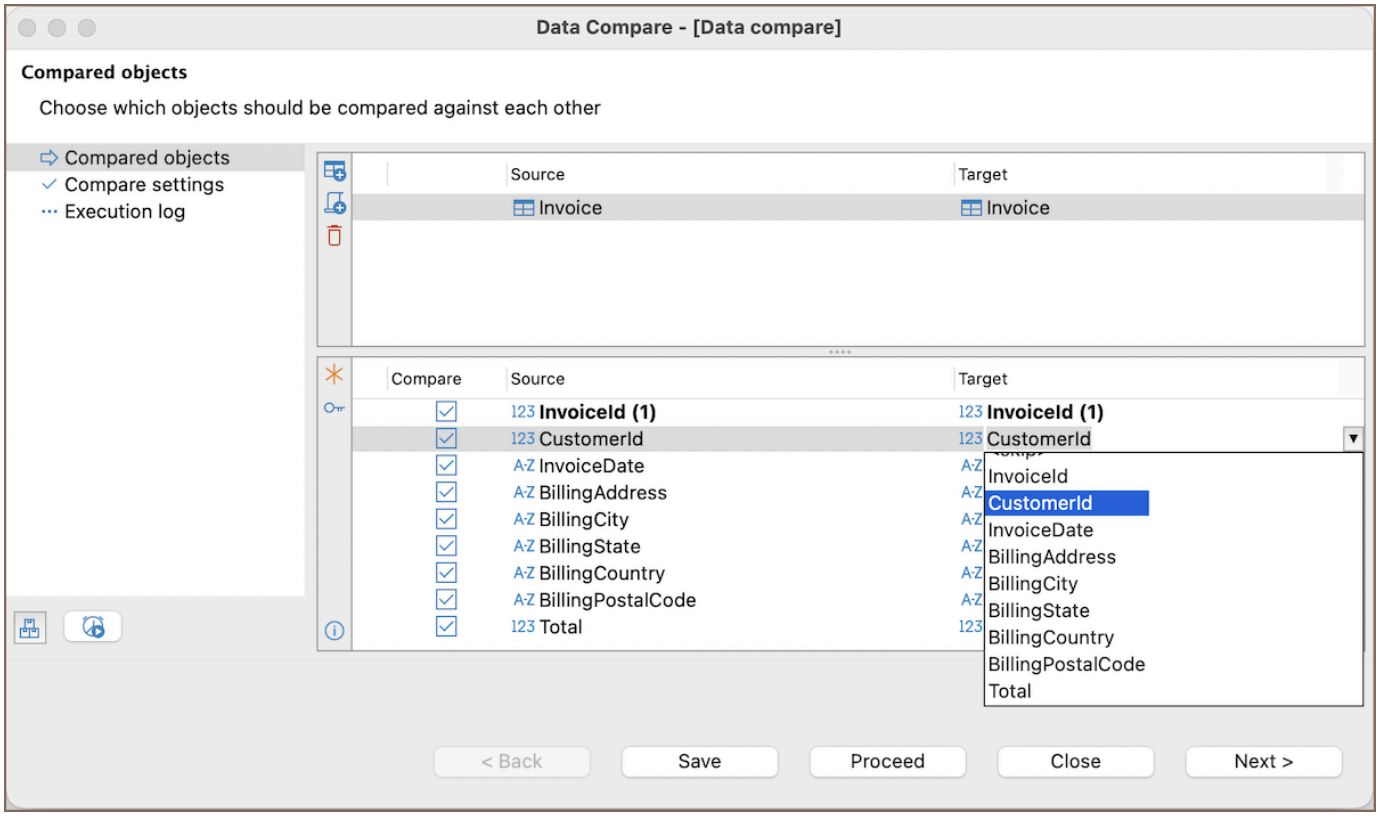


- **Replace with a table**: replace the current side with another table
- **Replace with a query**: replace the current side with an SQL query result
- **Swap source and target**: swap the pair

> **Tip**: You can also replace a table or query by double-clicking the **source** or **target** cell.

Column mapping

Use column mapping to control which columns are compared, and how source columns match target columns. This is useful when objects have different column names, order, or structure.



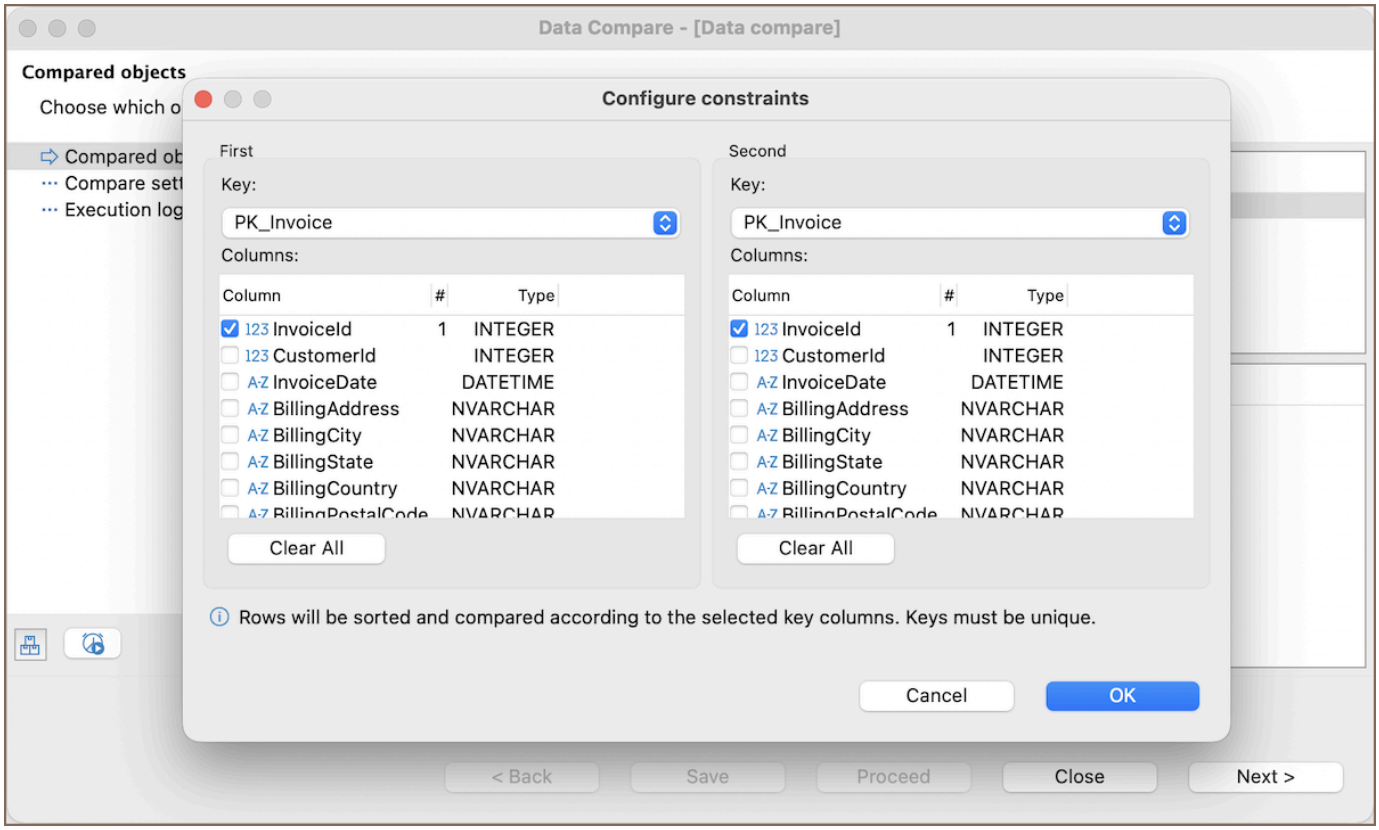
1. In the top list, select the source-target pair you want to configure. The column list below updates for that pair.
2. In the bottom list, review the mappings:
    - use the **Compare** checkbox to include or exclude a column from comparison
    - open the **Target** cell dropdown to map the source column to a different target column
    - if a column has no match on the other side, it's shown as `<unspecified>`
3. (Optional) click **Auto assign columns** button ( ) to match columns automatically.

> **Note**: Key columns are handled separately. They are used to match rows, not as regular mapped columns.

Keys configuration

Keys are used to match rows between **source** and **target**. Rows are sorted and compared by the selected key columns, so keys must uniquely identify each row.

1. In the top list, select the source-target pair you want to configure.
2. Click **Choose constraints** button ( ). The **Configure constraints** dialog opens.
3. In the dialog, select one or more key columns for **source** and **target**, then click **OK**.



> **Note**: The number of selected key columns must be the same on both sides.

> **Important**: Make sure the selected key columns are unique, or the comparison results may be incorrect.

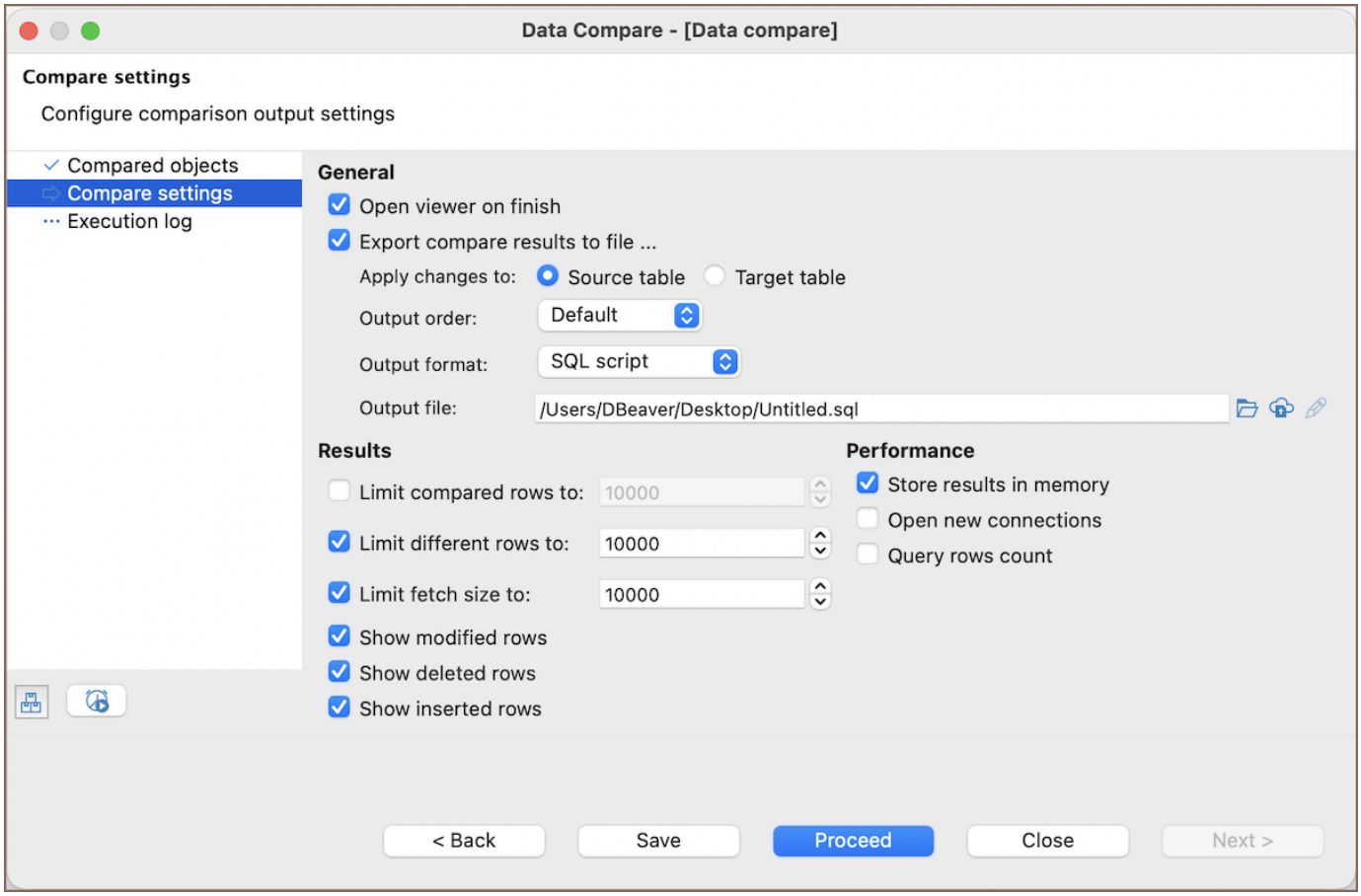
Composite keys

When you select multiple columns, they form a **composite key**. Composite keys uniquely identify rows based on a combination of values across the selected columns.

> **Tip**: Select columns for composite keys that, together, uniquely identify each row. Avoid columns with duplicate or null values, as they may impact comparison accuracy.

Compare settings

On the final page of the **Data Compare** wizard, configure settings to tailor the data comparison to your needs:

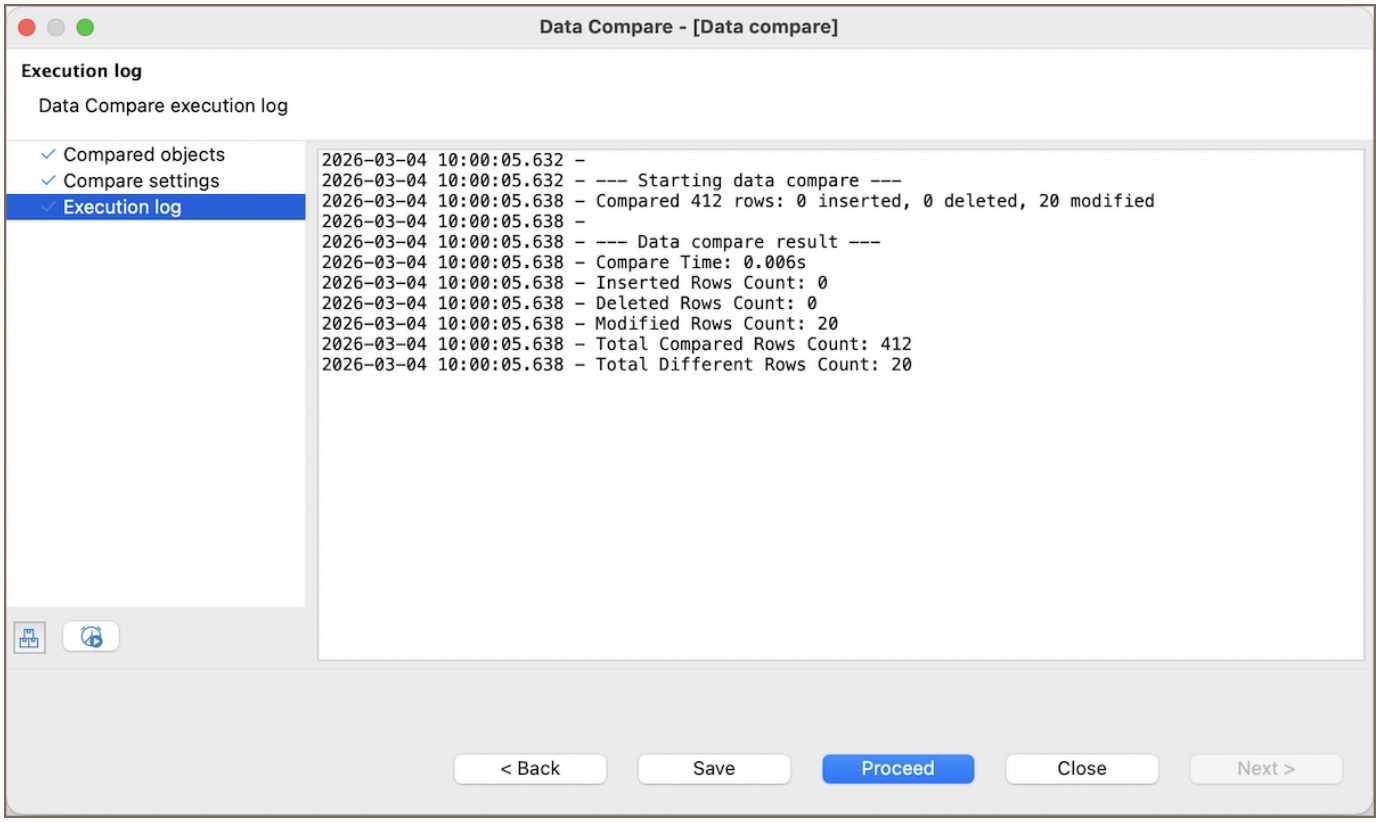


Setting	Description
Open viewer on finish	Opens the compare viewer for a visual overview of the differences once the comparison is complete.
Export compare results to file	Exports the SQL script diff to a file. You can choose the output format and specify the output file path.
Apply changes to	Chooses which table the exported script updates. Pick **Source table** or **Target table** to define where the `diff` is applied.
Output order	Controls how changes are sorted in the exported result. **Default** keeps the comparison order. **By type** groups inserts, updates, then deletes.
Output format	Chooses how the results are written to the file. You can export an **SQL script**, **HTML table**, or **Daff (Data Diff)**.
Output file	Sets the path and file name where DBeaver will save the exported diff.
Limit compared rows to	Limits how many rows must be compared against each other before ending the comparison.
Limit different rows to	Limits how many rows must be differ between each other before ending the comparison.
Limit fetch size to	Determines how many rows should be fetched at once, which helps manage memory and performance during comparison.
Show modified rows	Determines whether modified rows should be included in the diff or not, allowing you to track changes in the data.
Show deleted rows	Determines whether deleted rows should be included in the diff or not, providing insight into data that has been removed from the database.
Show inserted rows	Determines whether newly inserted rows should be included in the diff or not, allowing you to see additions to the database.
Store results in memory	Stores results in memory for further processing. Note that DBeaver might run out of memory when comparing large tables if this option is enabled. When disabled, results will be stored in a temporary database.
Open new connections	Opens new physical connections for data reading, which can help manage resources and ensure the stability of data comparison operations.
Query rows count	Queries row count before performing compare. This lets you track the progress of the comparison but may cause performance faults in some cases.
Save	Opens the **Create Task window** to assist in creating a task during the compare process. This window provides options and settings for creating and configuring a task related to the compare operation.

Execution log

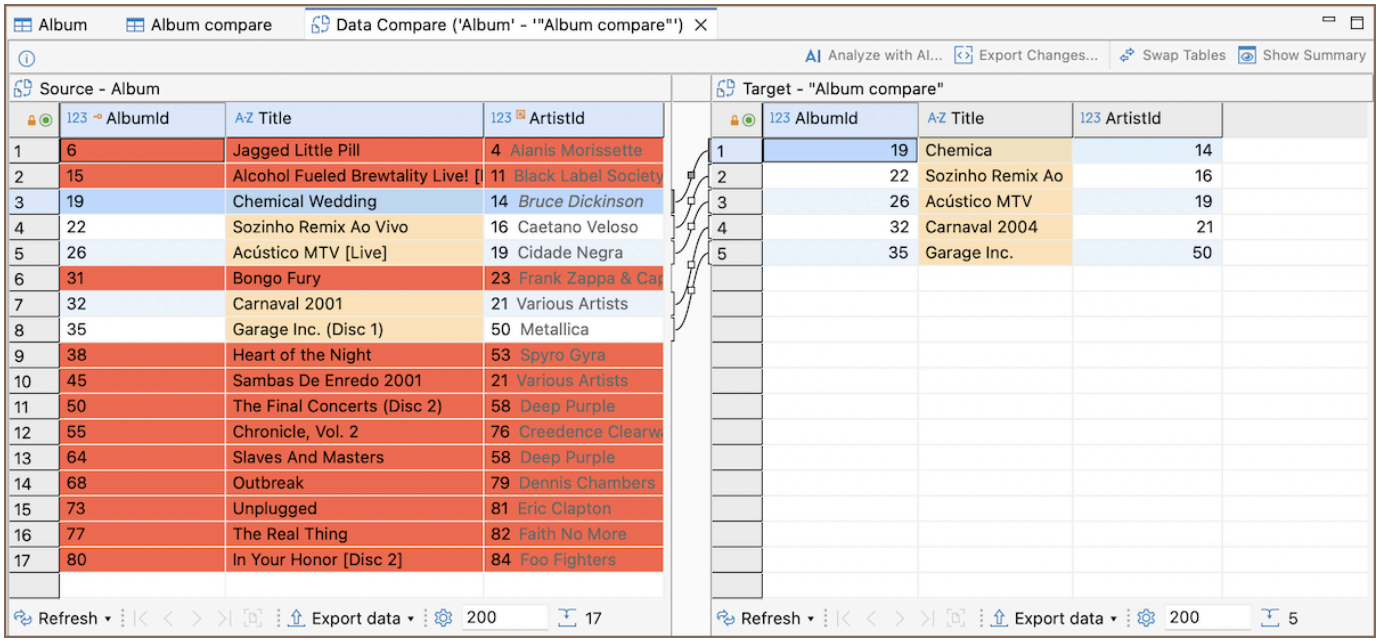
Once you have configured all settings and clicked **Proceed**, the Data Compare wizard will perform the comparison operation:

- an **Execution Log** window will display the progress and outcome of the comparison process.
- simultaneously, a separate window will open showing the comparison results. This window provides a detailed view of differences between the datasets based on the specified settings



Viewing the results

After the comparison process is complete, you can view the results in the viewer. It highlights the differences between the datasets in detail, with each row that shows a discrepancy being clearly marked.



Note: A complete visual preview of the results is available only when **Data Compare** is launched through the wizard. If launched as part of a task, the visual editor will not open. Instead, only statistics will be recorded in the execution log.

The viewer not only displays the differences but also provides tools to manage and interpret these differences effectively.

Feature	Button image	Description
Analyze with AI	AI	Analyzes changes and provides recommendations using AI.
Export changes		The results of the **Data Compare** can be exported as an SQL script, which includes `DELETE`, `INSERT`, and `UPDATE` statements to make the data in the second table match the first. Alternatively, you can export the results in an `HTML` format as a table for easy viewing and sharing.

Feature	Button image	Description
Swap tables		The **Swap tables** button allows you to switch the positions of the source and target data panels within the viewer.
Show summary		By clicking the **Show Summary** button, you can access a statistical summary and analysis of the differences found between the datasets. This summary provides a quick overview of the total changes, including the number of modified, deleted, and inserted rows.

> **Tip**: You can also export the comparison results table through Export data button Export data in the bottom toolbar or copy rows or individual values that differ.

Troubleshooting

If the **Next** button is greyed out, there's a configuration error in the **Compared objects** page. Look for a warning icon ( ) next to a pair in the source-target list and hover over it to see the error details.

No key columns are chosen

No key columns are configured for the pair. Without keys, DBeaver can't match rows between source and target.

1. Select the affected pair in the top list
2. Click **Choose constraints** button ( ) and select at least one key
3. Click **OK** - the warning disappears and **Next** becomes active.

No target specified

The source was added but no target is assigned yet.

1. Find the row with the warning icon.
2. Click **Browse...** in the **Target** column and pick a table or query.

> **Tip**: If you added multiple sources at once, each one needs its own target.

Backup and restore

> **Note**: This feature is available in Community, Enterprise, and Ultimate editions only.

Table of contents

- Supported databases
- Database specific settings
- MySQL
  - Backup MySQL database
  - Restore MySQL database
- PostgreSQL
  - Backup PostgreSQL database
  - Global PostgreSQL database backup
  - Restore PostgreSQL database
- Extra command arguments
- Dump and restore operations as tasks
- Troubleshooting
- Native client not found

DBeaver backs up and restores databases with each database's own native client tools, instead of moving data through DBeaver like the standard Data Transfer feature.

You can back up a single table, a schema, or a whole database. Select the object in the Database Navigator, right-click, and choose the backup command under **Tools** (or the main toolbar menu under **Database -> Tools**). To restore, pick the matching **Restore** command and point it at a backup file.

> **Tip**: To run a SQL script directly through the database's native client, outside of restore, use the **Execute script** command. See Execute script with local client.

Supported databases

Classic databases	Cloud databases
CrateDB	**AlloyDB**
Greenplum	**Google Cloud SQL for MySQL**
MariaDB	**Google Cloud SQL for PostgreSQL**
Materialize	
MySQL	
PostgreSQL	

Classic databases	Cloud databases
SingleStore	
TiDB	
TimescaleDB	
Yellowbrick	
YugabyteDB	
EnterpriseDB	

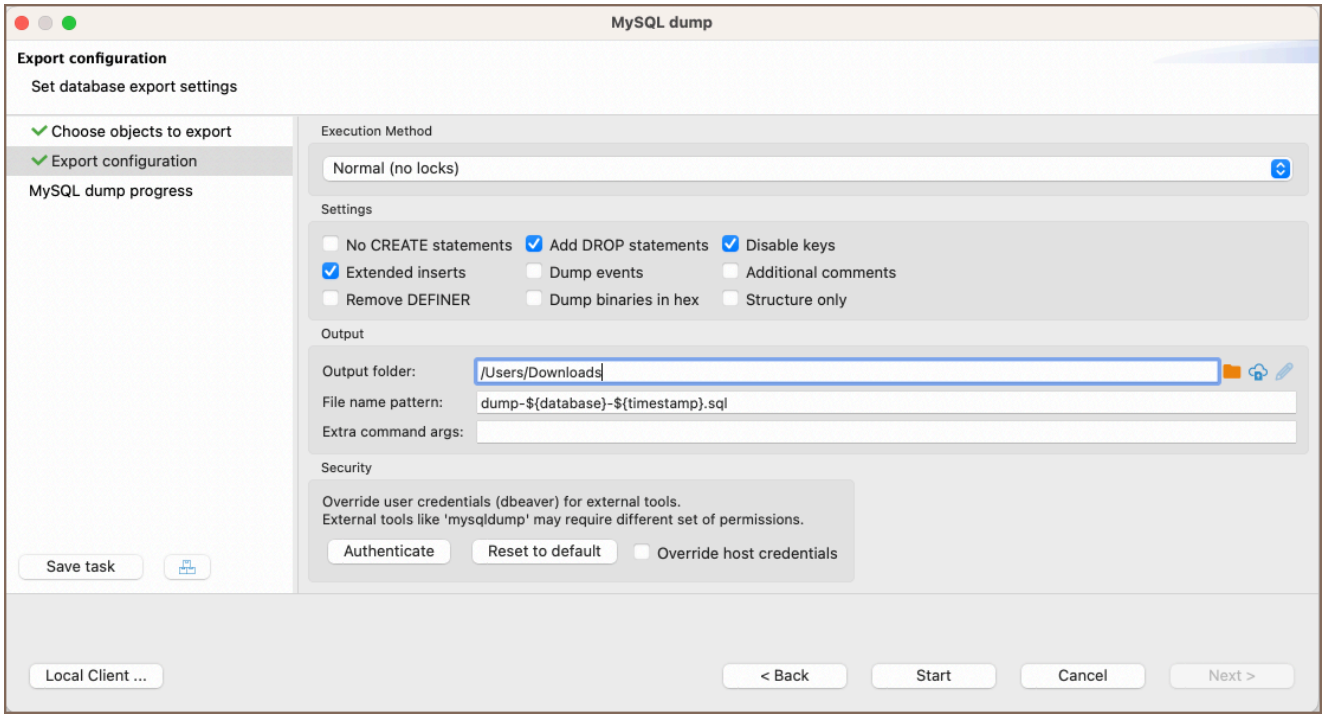
Database specific settings

Each database system has its own backup and restore settings. This section covers the settings for MySQL and PostgreSQL.

MySQL

Backup MySQL database

1. Select the desired database.
2. Right-click on the database and choose **Tools -> Dump database**.
3. The **Dump** window opens. Select the objects you need and click **Next**.
4. The **Export configuration** tab opens with these checkboxes and fields:



Option/Field	Description
Execution Method	Method for execution. The default is **Normal (no locks)**.
Execution Method options:	**Normal (no locks)**: Default method. Dumps without locking any tables, so it causes minimal disruption.
	Online backup in single transaction: Dumps within a single transaction for a consistent snapshot. Best for transactional engines like InnoDB. Doesn't interrupt other operations but needs a higher isolation level.
	Lock all tables: Locks every table during the dump. Guarantees a consistent snapshot but can interrupt other operations.
No CREATE statements	Do not include `CREATE` statements in the dump.
Compressed	Include the `--compress` flag to enable network compression during dump.
Add DROP statements	Include `DROP` statements before CREATE statements.
Disable keys	Disable keys during the dump process.
Extended inserts	Use extended `INSERT` statements in the dump.
Dump events	Include events in the dump.
Additional comments	Include comments in the dump.
Remove DEFINER	Remove `DEFINER` clause from the dump.
Dump binaries in hex	Display binary data as `HEX` in the dump.

Option/Field	Description
Structure only	Only dump table structures, not data.
Output folder	Specify the directory where the dump file will be saved. For Ultimate Edition, Team Edition, and CloudBeaver versions, users have the option to save to a remote file system using the **Browser remote file system** button .
Extra command args	Provide additional command-line arguments. See details in the Extra command arguments section.
Authenticate	Click to open the **Authentication** window where you can fill in the Username/Password for override.
Reset to default	Reset credentials to their default values.
Override host credentials	Use different host credentials, if necessary.
Local Client	Open a window to specify the path to the Local client. Learn more.

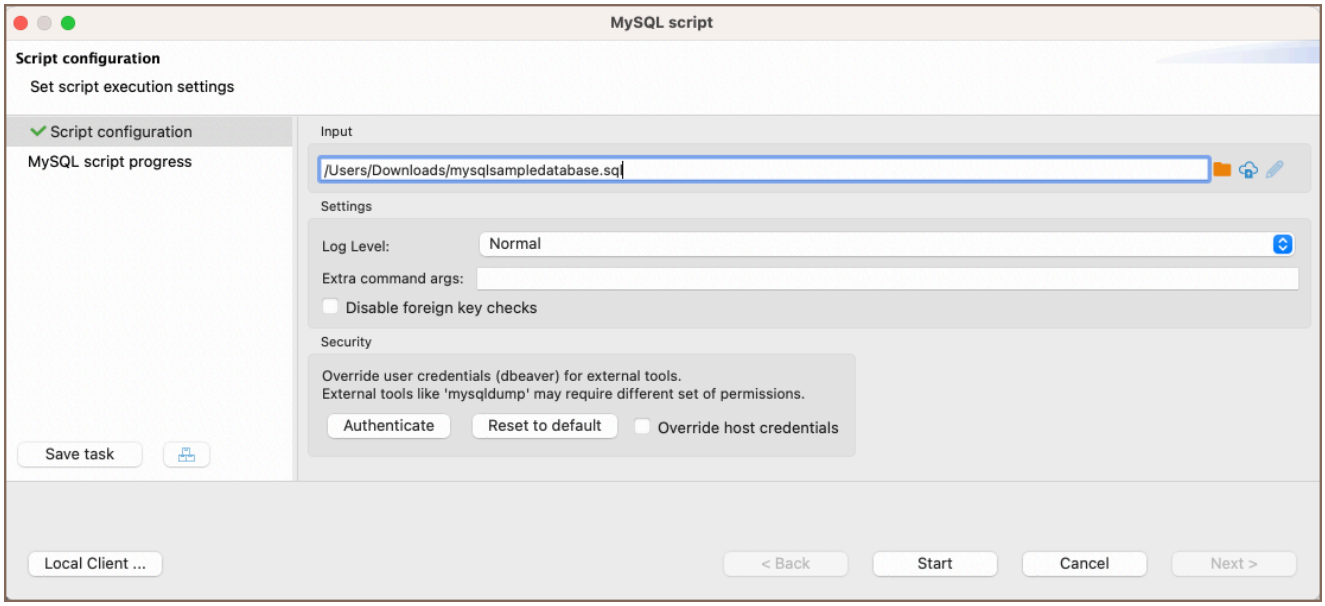
> **Note**: The particular set of configuration options depends on the database type.

5. After configuring the settings, click **Start**.
6. When the process finishes, a notification appears with the details.
7. You'll find the backup file in the folder you set in the **Export configuration** step.

Restore MySQL database

1. Select the desired database.
2. Right-click on the database and choose **Tools -> Restore database**.
3. The **Restore** window opens. Enter the path to the SQL file you want to restore from in the designated field.

> **Example**: path: `/path/to/file.sql`



Below are the settings for the restore process:

Setting Name	Description
Log Level	Choose the level of logging for the restore process.
Log Level options:	**Normal**: Standard logging. Captures the essential details of the restore and keeps logs short.
	Verbose: More detail than Normal. Adds context when you want to follow the process closely, without full debug output.
	Debug: The most detailed logging. Captures every event, including low-level operations. Best for troubleshooting.
Extra command args	Provide additional command-line arguments. See details in the Extra command arguments section.
Disable foreign key checks	Disable foreign key checks during the restore process.
Authenticate	Click to open the **Authentication** window where you can fill in the Username/Password for override.
Reset to default	Reset credentials to their default values.
Override host credentials	Use different host credentials, if necessary.
Local Client	Specify the path to the Local client.

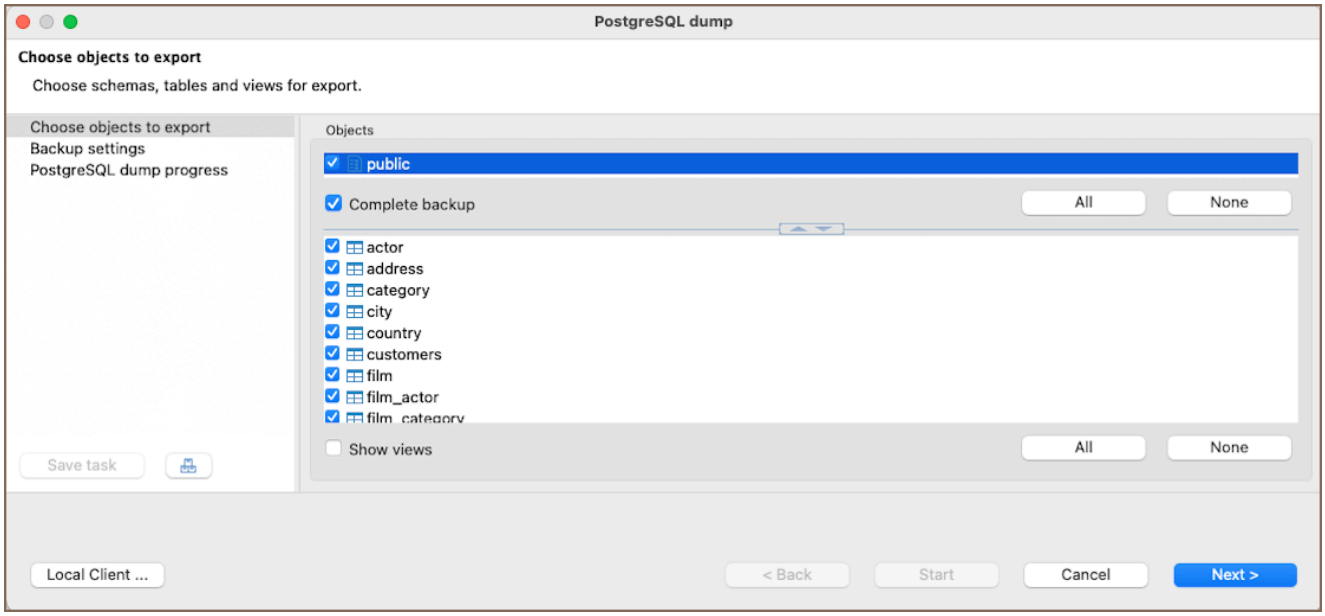
4. After configuring the settings, click **Start**.

5. When the process finishes, a notification appears with the details.

PostgreSQL

Backup PostgreSQL database

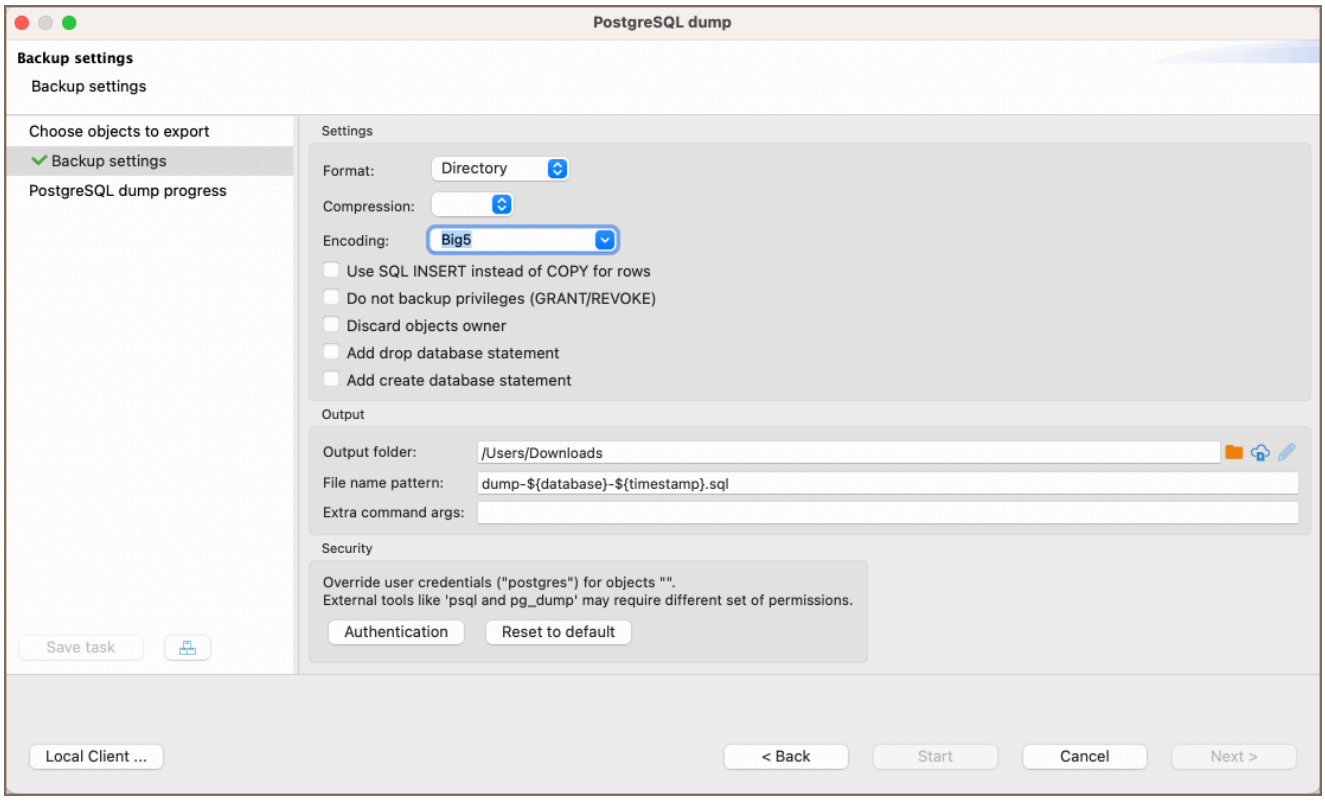
1. Select the desired database.
2. Right-click on the database and choose **Tools -> Backup**.
3. The **Dump** window opens. Select the objects you need and click **Next**.



> **Note**: By default, when all schemas are selected, DBeaver enables the **Complete backup** checkbox and skips `-n` flags in the `pg_dump` command to include all database objects. Clear the checkbox to add `-n` for each selected schema and back up only those. For details, see the `pg_dump` --schema documentation.

> **Warning**: When you select more than one schema, DBeaver ignores individual table selections and backs up each selected schema in full. To back up specific tables, select a single schema.

4. The **Backup settings** tab opens with these checkboxes and fields:



Setting	Description
Format	Choose the format for the dump. Options include Directory, Tar, Custom, and Plain.
Format options:	**Directory**: Splits the output into one file per table. Suitable for larger databases as it facilitates parallel restoration.
	Tar: Produces archive in tar format. Useful for backups since it can be read by standard tools.
	Custom: A flexible format that allows selective restore and other operations.
	Plain: Produces plain-text SQL script file. It can be used with psql for restoring.
Compression	Set the compression level for the dump, ranging from 0 (no compression) to 9 (maximum compression).
Encoding	Select the character encoding for the dump. The available encodings depend on database configuration and locale.
Use SQL INSERT instead of COPY for rows	Use the `INSERT` command instead of the `COPY` command for row data.
Do not backup privileges (GRANT/REVOKE)	Exclude privilege commands (`GRANT/REVOKE`) from the dump.
Discard objects owner	Exclude the ownership information from the dump.
Add drop database statement	Include a statement in the dump to drop the database before restoring.
Add create database statement	Include a statement in the dump to create the database when restoring.
Output folder	Specify the directory where the dump file will be saved. For Ultimate Edition, Team Edition, and CloudBeaver versions, users have the option to save to a remote file system using the **Browser remote file system** button.
File name pattern	Define the naming pattern for the dump file, with variables like `${database}` and `${timestamp}` to customize the filename.
Extra command args	Provide additional command-line arguments. See details in the Extra command arguments section.
Authentication	Click to open the **Authentication** window where you can fill in the Username/Password for override.
Reset to default	Reset credentials to their default values.
Override host credentials	Use different host credentials, if necessary.
Local Client	Specify the path to the Local client.

> **Note**: The particular set of configuration options depends on the database type.

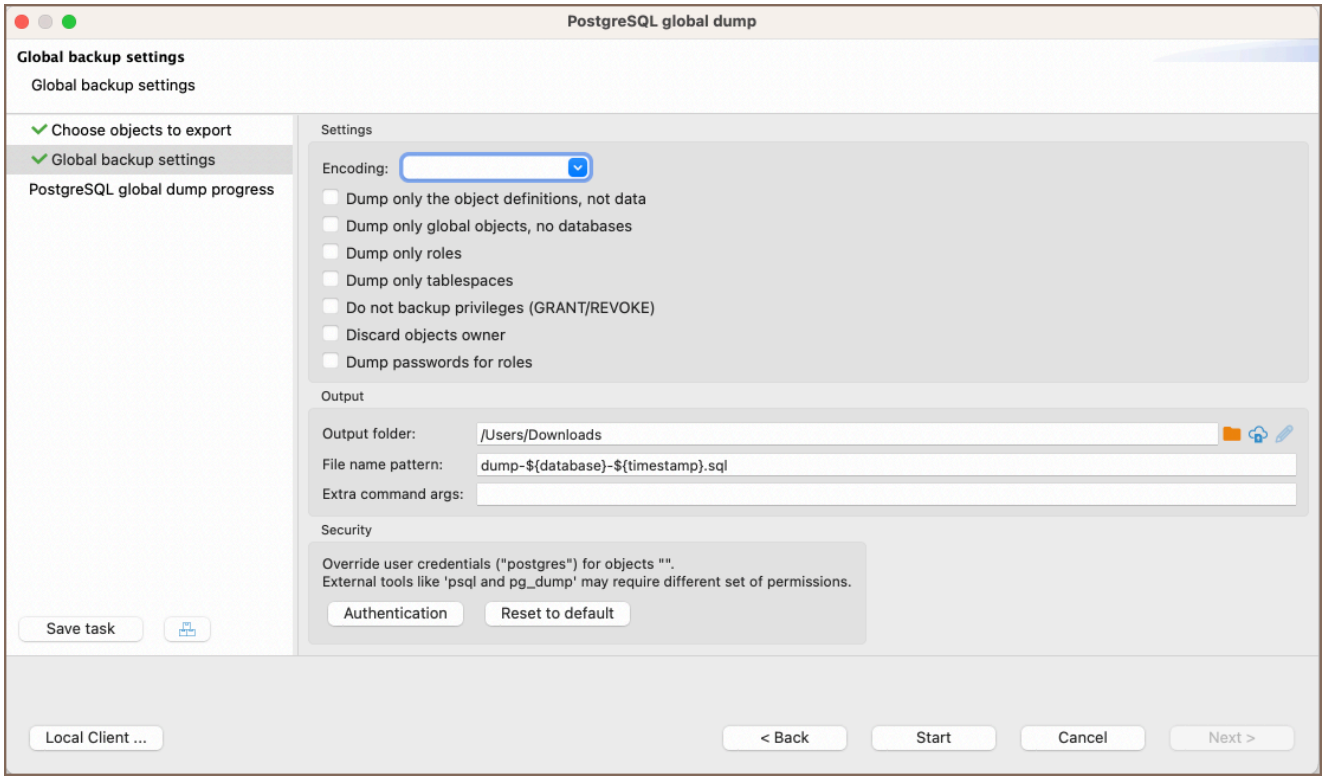
5. After configuring the settings, click **Start**.
6. When the process finishes, a notification appears with the details.
7. You'll find the backup file in the folder you set in the **Backup settings** step.

Global PostgreSQL database backup

When performing a Global PostgreSQL database Backup, the entire database is dumped, including roles and tablespaces. This differs from standard backup procedures where only specific schemas and their contents can be selected. Additionally, multiple databases can be chosen for backup at once in the global method.

To run a global backup:

1. Select the desired database.
2. Right-click on the database and choose **Tools -> Global Backup**.
3. The **Global Dump** window opens. Select the objects you need and click **Next**.
4. The **Backup settings** tab opens with these checkboxes and fields:

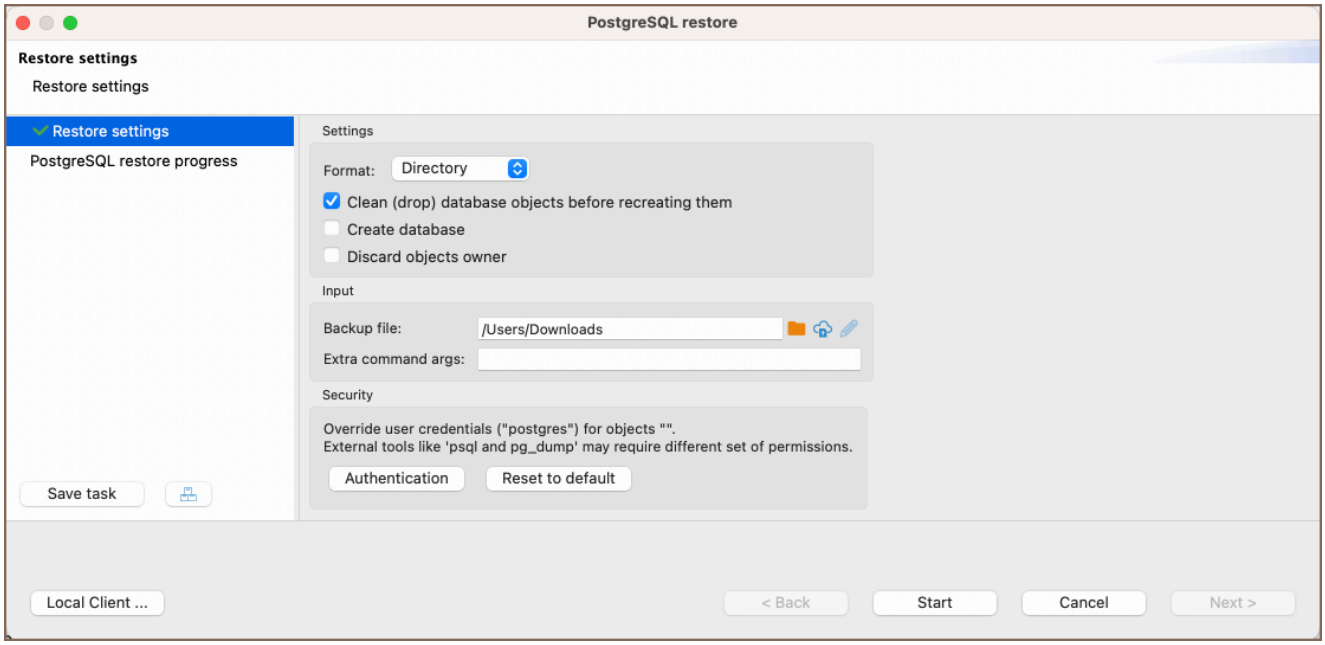


Setting	Description
Encoding	Set the character encoding for the backup.
Dump only the object definitions, not data	Includes only the structure of the database objects, excluding the data.
Dump only global objects, no databases	Includes only global objects and excludes individual databases.
Dump only roles	Includes only user roles.
Dump only tablespaces	Captures only tablespaces without the databases.
Do not backup privileges (GRANT/REVOKE)	Excludes privilege statements like `GRANT` and `REVOKE`.
Discard objects owner	Excludes the ownership information of the database objects.
Dump passwords for roles	Includes passwords associated with the user roles.
Output folder	Specify the directory where the dump file will be saved. For Ultimate Edition, Team Edition, and CloudBeaver versions, users have the option to save to a remote file system using the **Browser remote file system** button .
File name pattern	Define the naming pattern for the dump file, with variables like `${database}` and `${timestamp}` to customize the filename.
Extra command args	Provide additional command-line arguments. See details in the Extra command arguments section.
Authenticate	Click to open the **Authentication** window where you can fill in the Username/Password for override.
Reset to default	Reset credentials to their default values.
Override host credentials	Use different host credentials, if necessary.
Local Client	Specify the path to the Local client.

5. After configuring the settings, click **Start**.
6. When the process finishes, a notification appears with the details.
7. You'll find the backup file in the folder you set in the **Global backup settings** step.

Restore PostgreSQL database

1. Select the desired database.
2. Right-click on the database and choose **Tools -> Restore**.



Setting	Description
Format	Choose the format for the restore operation. Options include **Directory**, **Tar**, **Custom**, and **Plain**.
Format options:	**Directory**: Assumes the input consists of one file per table. Suitable for larger databases as it supports parallel restore.
	Tar: Assumes the input is an archive in tar format. This is useful when restoring backups made in tar format.
	Custom: A flexible input format that allows selective restore and other operations.
	Plain: Assumes the input is a plain-text SQL script file. Suitable for scripts that can be executed with psql.
Clean (drop) database objects before recreating them	Drop database objects before recreating them from the backup.
Create database	Create a new database from the backup.
Discard objects owner	Exclude ownership information during the restore.
Backup file	Specify the path to the backup file that you wish to restore. For Ultimate Edition, Team Edition, and CloudBeaver versions, use the **Browser remote file system** button to select a backup file from a remote file system via Cloud Storage.
Extra command args	Provide additional command-line arguments. See details in the Extra command arguments section.
Authentication	Click to open the **Authentication** window where you can fill in the Username/Password for override.
Reset to default	Reset credentials to their default values.
Local Client	Specify the path to the Local client.

3. After configuring the settings, click **Start**.
4. When the process finishes, a notification appears with the details.

Extra command arguments

The **Extra command args** field takes extra command-line arguments, giving you finer control over the dump.

For example, when configuring backup/restore for PostgreSQL, you may want to exclude specific tables. To exclude a table named `employee_data`, use the `--exclude-table` option. Enter `--exclude-table=employee_data` in the **Extra command args** field during the setup process.

By doing this, the resulting process won't contain any data or structure from the `employee_data` table.

> **Note**: Always consult the database's official documentation for a comprehensive list and explanation of available command-line arguments.

Dump and restore operations as tasks

You can set up both restore and dump operations as tasks. To save an operation as a task, click the **Save task** button Save task during the dump/restore process.

> For detailed instructions and features related to task management in DBeaver, refer to Task management article.

Troubleshooting

Backup, restore run through the database's native client tools. DBeaver doesn't bundle these tools, so you point it at a local installation.

Native client not found

DBeaver reports that the client binaries location isn't set, or that the client home can't be found. It couldn't locate the native client on your machine.

Install the client tools, then set the folder with the **Local Client** button in the tool window, or in the connection settings.

> For details, see Local Client Configuration.

Schema compare

> **Note**: This feature is available in Enterprise, and Ultimate editions only.

Table of contents

- Supported databases
- Schema Compare process
- Select two entities to compare
- Specify objects for comparison
- Schema Compare settings
- Results view
  - Save operation as a task
- Schema Compare global preferences
- Troubleshooting

DBeaver lets you compare two database structures and see how they differ. You can compare schemas or full databases, view the changes in a clean diff, and generate scripts to bring one side in sync with the other. Under the hood, DBeaver uses Liquibase snapshots to read metadata from both sides and highlight the differences between them.

Supported databases

- Amazon RDS MariaDB
- Amazon RDS MySQL
- Amazon RDS Oracle
- BigQuery
- CockroachDB
- DB2 for LUW
- DB2i
- EnterpriseDB
- Google Cloud MSSQL
- Google Cloud MySQL
- Google Cloud PostgreSQL
- Greenplum
- HSQL
- Informix
- MariaDB
- MySQL
- Netezza
- Oracle
- PostgreSQL
- SAP HANA
- Snowflake
- SQL Server
- SQLite
- Teradata
- Vertica
- YugabyteDB

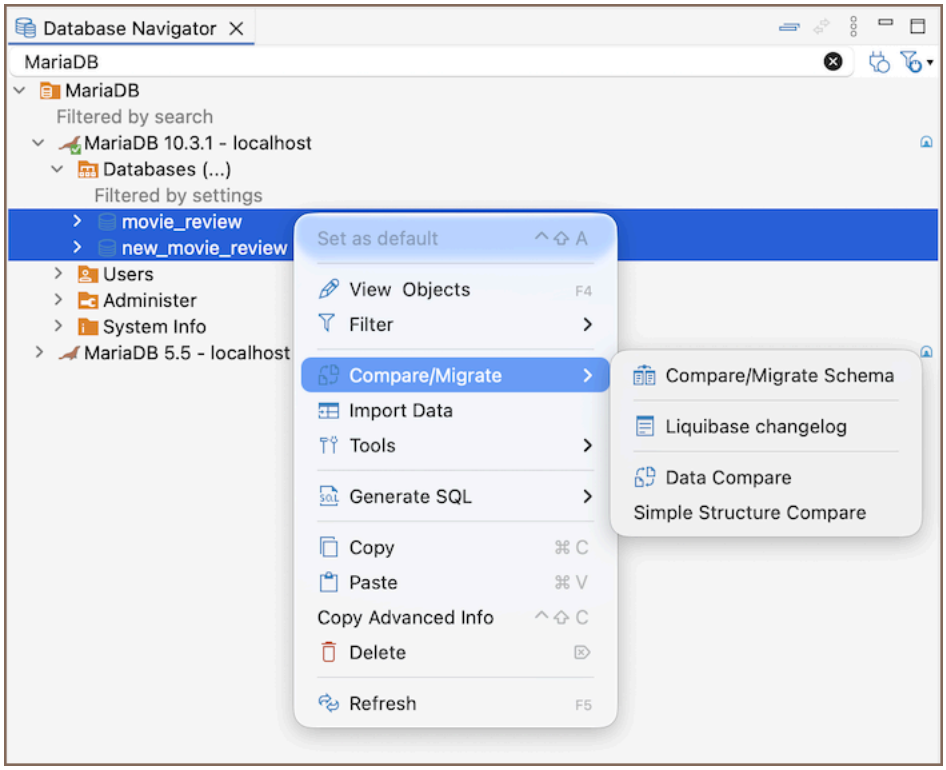
Schema Compare process

Select two entities to compare

1. In the **Connections**, select two objects you want to compare - schemas or databases, depending on what your database supports

   > **Tip**: You can also start a compare through AI by sending a short request in AI chat. For supported commands, see Data transfer actions.

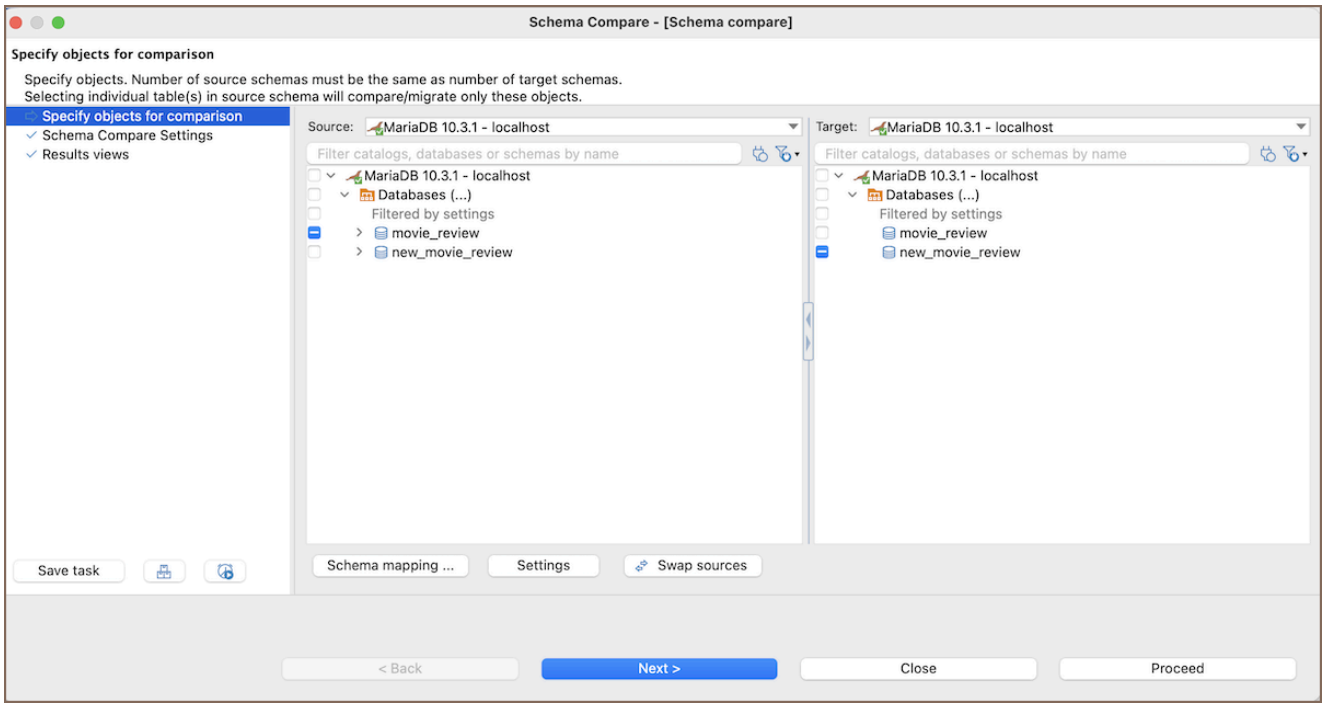
2. Open the context menu
3. Choose **Compare/Migrate** -> **Compare/Migrate Schema** to open the comparison window



Specify objects for comparison

1. Make sure you selected the correct objects

> **Tip**: If needed, switch source and target using **Swap sources**.



Select only containers that actually hold tables:

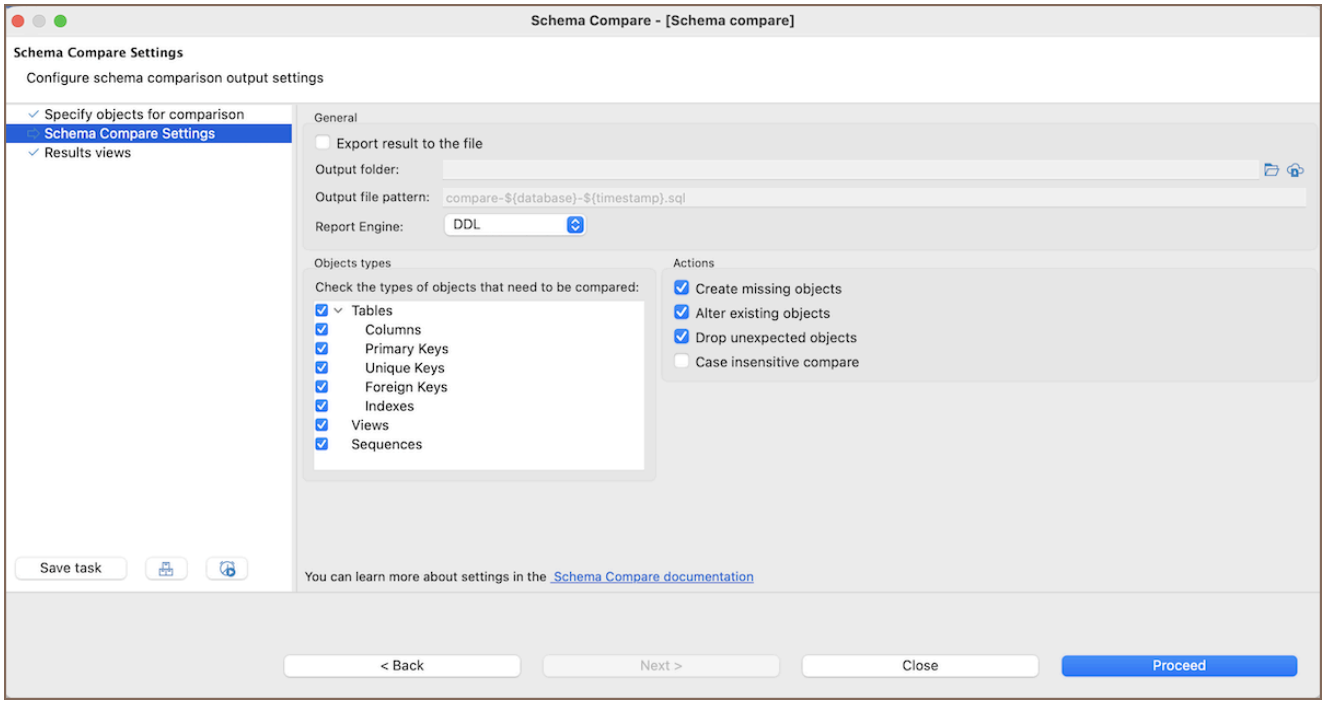
- **Schemas** - when the database supports schemas
- **Databases** - when the database uses catalogs but not schemas
- **Individual objects** - such as tables or views, when supported by the driver
- **Datasources** - when the database doesn't support schemas or catalogs

> **Tip**: Use Preferences to adjust additional Schema Compare options.

2. If everything looks right, click **Next** to continue to comparison settings

Schema Compare settings

1. Review and adjust the settings you want to use

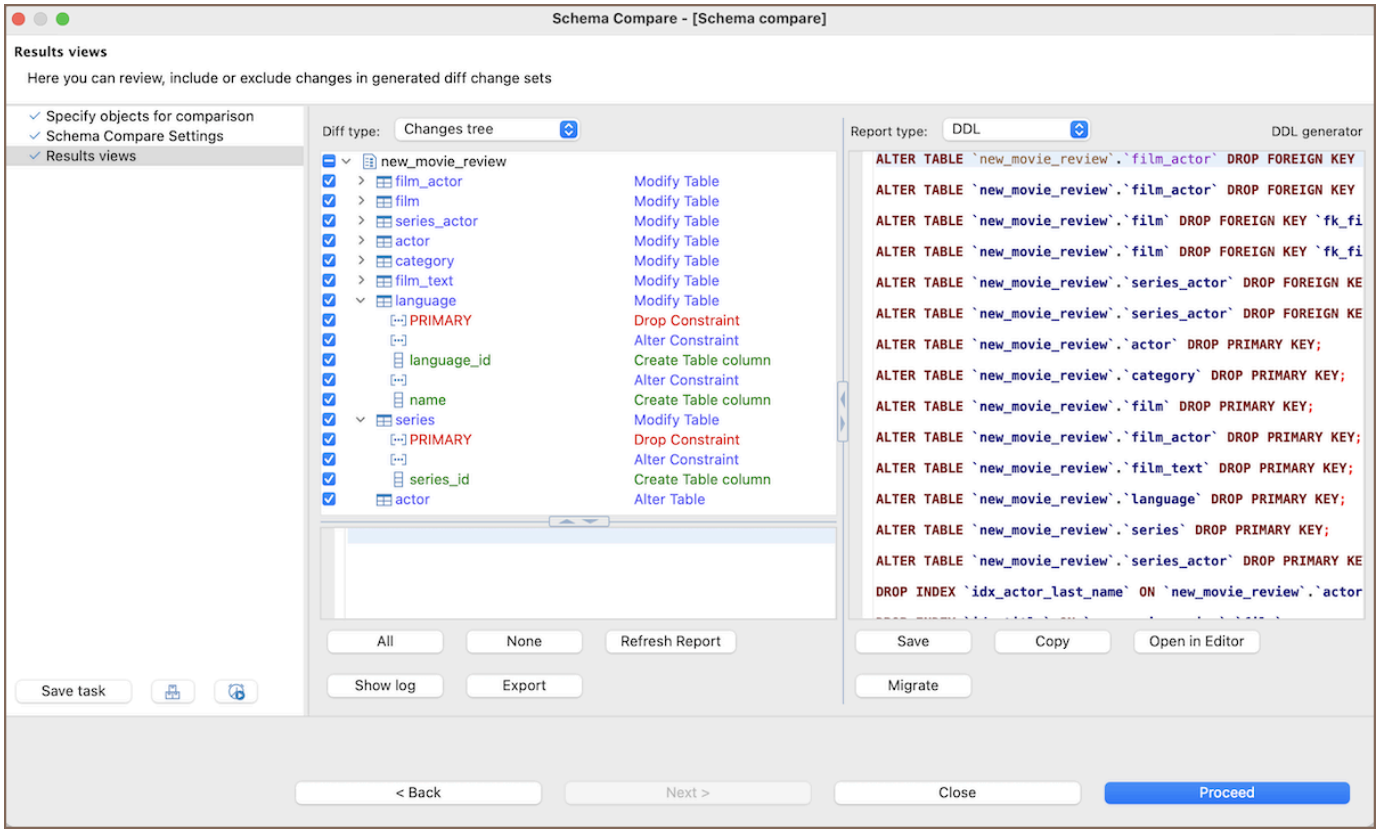


Setting	Description
Export result to the file	Additionally saves the generated diff to a file.
Output folder	Folder where the generated file will be saved. Enabled only when **Export result to the file** is turned on.
Output file pattern	Lets you use variables such as database name or timestamp to build the file name. Enabled only when **Export result to the file** is turned on. See more details about Pre-configured variables.
Report Engine	Selects the output format: **DDL**, **Change Log**, **JSON**, **YAML**, or **Plain text**. Enabled only when **Export result to the file** is turned on.
Object types	Chooses which structural objects to include in comparison (tables, columns, keys, views, etc.). With Liquibase PRO, you can work with extended metadata. For details, see the Liquibase PRO guide.
Create missing objects	Enables generation of `CREATE` statements for objects missing on target.
Alter existing objects	Enables generation of `ALTER` statements for changed objects.
Drop unexpected objects	Enables generation of `DROP` statements for objects missing on source.
Case insensitive compare	Ignores case differences in object names (`test` vs `TesT`).

2. Click **Proceed** to generate the diff report

Results view

By default, you get a **DDL** diff. It shows the `CREATE`, `ALTER`, and `DROP` statements you need to run to make the target schema match the source schema.



Use the action panel to manage the results:

Button/Option	Description
All	Enables all changes in the changes tree.
None	Disables all changes in the changes tree.
Refresh Report	Updates the comparison report to reflect any changes made to the selection or schema.
Show log	Opens the Schema Compare log. You can control the log detail level in Preferences.
Export	Exports the current diff report to a `.txt` file.
Save	Saves the current state of the report.
Copy	Copies the generated SQL or report text to the clipboard.
Open in Editor	Opens the current diff report in the SQL Editor.
Migrate	Executes the generated SQL statements on the target database, applying all selected changes.

> **Warning**: Migration can't be undone automatically. Make sure you have a backup or can restore the target database before applying changes.

Save operation as a task

If you run Schema Compare regularly, save it as a task. You can execute the task at any time or add it to a schedule.

You can also include Schema Compare in a composite task. For example, if you export the result as a script, the next step in the composite task can run that script automatically.

Schema Compare global preferences

Go to **Preferences -> Editors -> Schema Compare** to configure global Schema Compare settings.

Setting	Description
Create missing objects	Include `CREATE` statements for objects that exist only on the source side.
Alter existing objects	Include `ALTER` statements for objects that differ between sides.
Drop unexpected objects	Include `DROP` statements for objects that exist only on the target side.
Case insensitive compare	Ignore case differences in object names when comparing structures. The exact behavior depends on the database.
Quote all objects names in the DDL log	Define how object names are quoted in the generated DDL log. When enabled, all object names are quoted. Otherwise, only reserved words or names with special characters are quoted.
Log Level	Set how much detail appears in the log: **OFF**, **SEVERE**, **WARNING**, **INFO**, **DEBUG**.
Import Liquibase Pro Key	Import a Liquibase PRO license key. For details, see Liquibase PRO.

Troubleshooting

If a comparison fails or produces unexpected results, try reducing the set of objects you compare. Some databases supported by Liquibase may expose metadata differently through JDBC.

Turn off specific object types - such as sequences or views - and run the comparison again. This often helps when a driver returns incomplete or unusual metadata.

Task management

> **Note**: This feature is available in Community, Enterprise, and Ultimate editions only.

Table of contents

- Database Tasks view
- Toolbar actions
- Context menu actions
- Create tasks
- Task setup
- Available tasks
- Manage tasks
- Add, edit, or delete tasks
- Run a task
  - Run tasks from the command line
- Adjusting task configurations
- Scheduling tasks
- Set task timeout
- Customize the Database Tasks view
- Group tasks
- Configure columns
- Folders
  - Move tasks between folders
  - Delete a folder

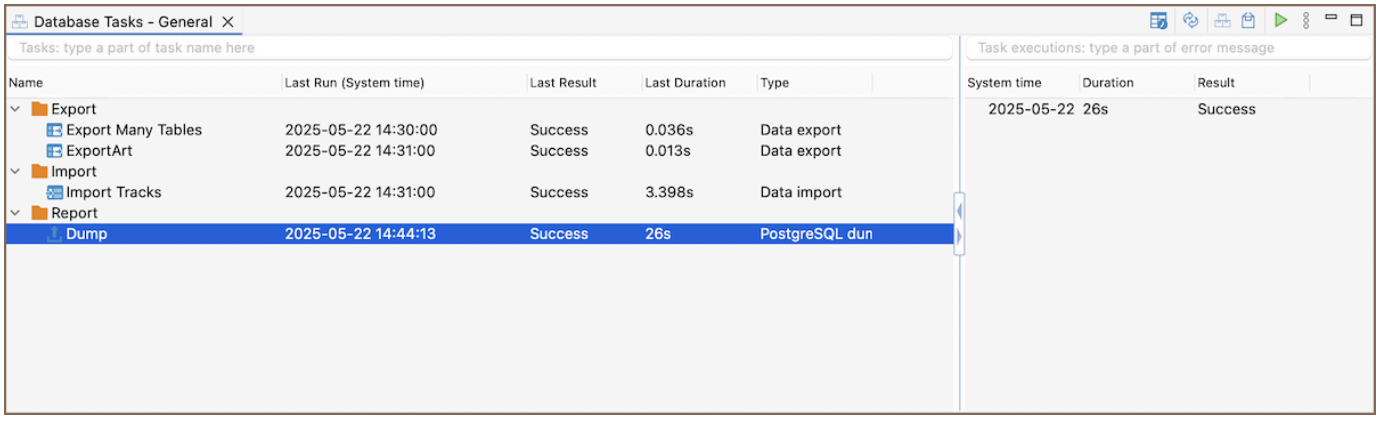
Use tasks to save and reuse configurations for database tools like data transfer or import/export. Tasks help you automate routine actions and run them with one click. You can create tasks from tool wizards or from the main menu, group them in folders, and manage them in a dedicated view.

Database Tasks view

All your tasks are stored in the **Database Tasks** view. Use it to review, organize, and run tasks when you need them.

From this view, you can:

- Add, edit, or delete tasks.
- See task information.
- Run tasks manually.
- Schedule tasks.



To open it:

- Go to **Database -> Tasks -> Database Tasks**,
- Or click **Open Tasks view** ( ) from the task creation wizard.

Toolbar actions

The toolbar includes quick-access buttons:

Icon	Name	Description
	Open scheduler	Configure time-based task runs.
	Refresh	Reload the task list.
	Group by category	Toggle grouping by category (Common/Database-specific).
	Group by type	Toggle grouping by type (Export, Import, etc.).

Icon	Name	Description
	Run task	Start the selected task manually.
	View menu	Open options to create a new task, add a folder, or delete items.

> **Tip**: You can also right-click a task in the list to access these actions.

Context menu actions

Right-click any task in the **Database Tasks** view to access task actions:

Action	Description
Run task	Execute the selected task immediately.
Edit task	Open the task editor.
Create new task...	Open the task creation wizard.
Copy	Copy task name to clipboard.
Delete	Remove the selected task.
Create new task folder	Create a folder to organize tasks.
Rename folder	Rename a selected folder.
Scheduler	Access task scheduling settings.
Group tasks by category	Enable grouping by user-defined categories.
Group tasks by type	Enable grouping by task type.
Copy	Copy task info to clipboard.
Configure columns	Choose which columns to display in the view.
Auto-size columns	Adjust column widths to fit content.

Create tasks

You can create a new task from:

- Database Tasks view: Go to **Database -> Tasks -> Database Tasks**
- Menu bar: Go to **Database -> Tasks -> Create New Task**
- Toolbar: click arrow next to **Open Tasks view** ( ) and choose **Create new task**.

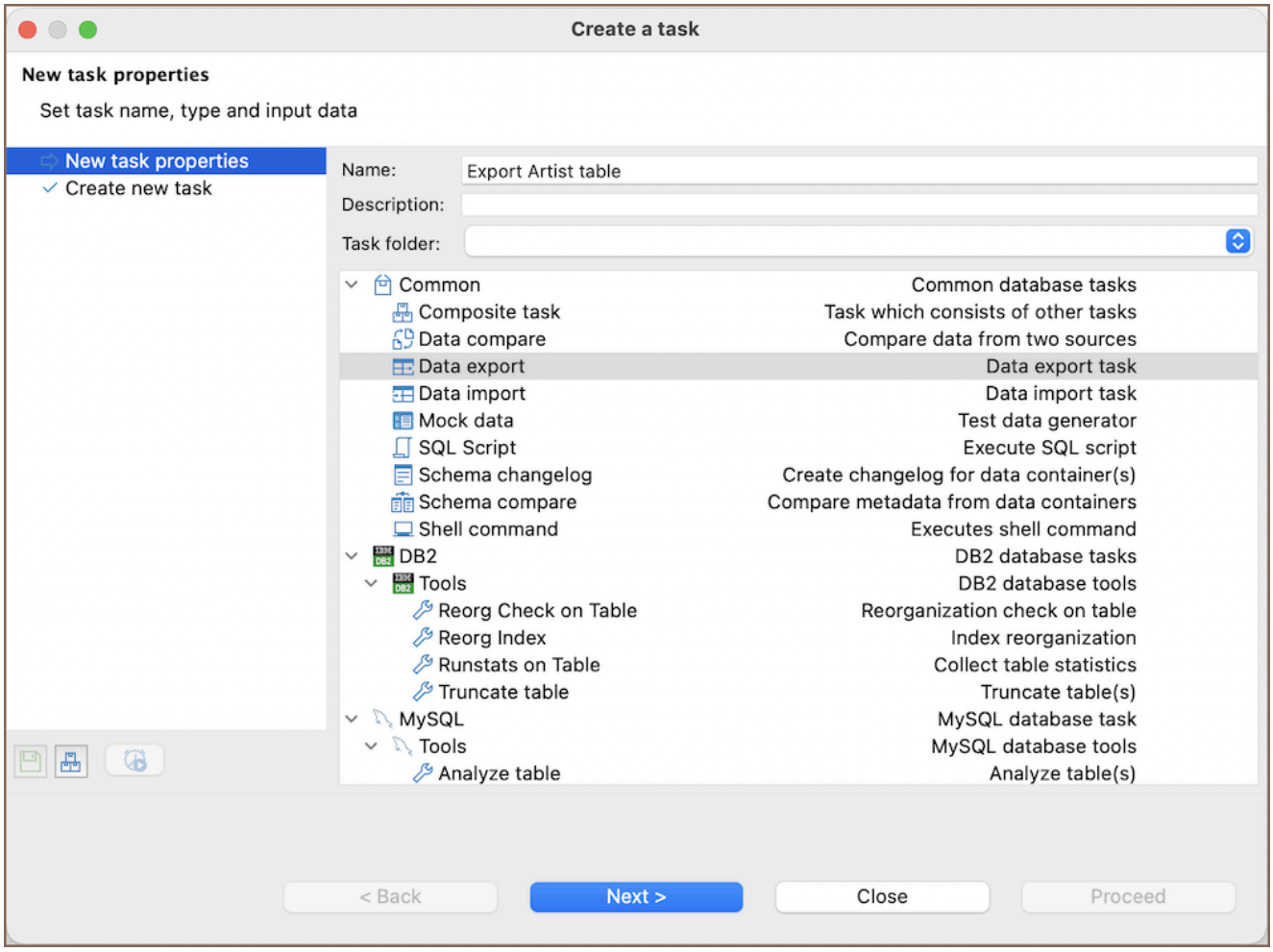
  > **Note**: The toolbar is customizable. For further information, refer to Toolbar Customization article.

- Connections: Right-click a database object, then select **Tools -> Create new task**
- Properties editor: Right-click a database object, then choose **Tools -> Create new task**

Alternatively, you can create tasks directly from tools like Data Transfer, Data Compare, Back up and Restore, Mock Data generator. Just configure the tool, click **Save** button, fill in the task details, and save.

Task setup

1. Open the task creation wizard.



2. Fill in the task properties and choose the tool you want to automate:

Field	Description
Name	Enter a unique name for the task.
Description	(Optional) Add a short description to help identify the task later.
Task folder	(Optional) Select a folder to organize tasks.
Task type	Choose the tool you want to automate. Select from the list of task types:
	Common: Available for any database type.
	Database-specific: Available only for supported database types.

> **Note**: The list of available task types depends on the databases listed in **Connections**.

3. Click **Next**.

4. Complete the task-specific configuration wizard.
5. Save the task when you're done.

Available tasks

Some databases provide their own task sets. For example, MySQL, Oracle, or Exasol add tools like table maintenance, statistics collection, or trigger control. These tasks appear under the corresponding database node and are available only for that database type.

> **Note**: Database-specific tasks appear only if at least one connection of that database type exists.

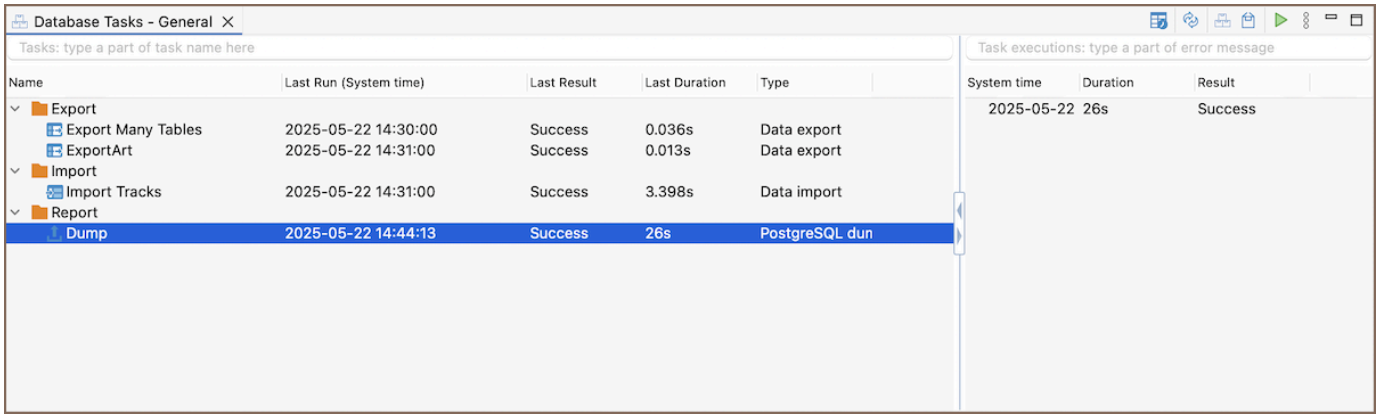
Alongside that, DBeaver provides **Common** tasks. They work with any supported database and cover typical cross-database workflows:

Task	Description	Reference
Composite task	Run multiple tasks as a single workflow.	Composite tasks
Data compare	Compare data between sources and review differences.	Data compare
Data export	Export data to files or external targets.	Data export
Data import	Import data from files.	Data import
Mock data	Generate test data.	Mock data generation

Task	Description	Reference
SQL Script	Execute one or more SQL scripts automatically.	SQL Script task
Schema changelog	Create a changelog for selected data containers.	Schema changelog
Schema compare	Compare database metadata between schemas or databases.	Schema compare
Shell command	Run a shell command as part of a task.	Shell command task

Manage tasks

To manage tasks open the Database Tasks view.



Add, edit, or delete tasks

You can manage tasks using the toolbar or by right-clicking a task in the **Database Tasks** view.

Available actions:

Action	How to use it
Add	Click the **View Menu** button ( ) in the view's toolbar and select **Create new task...**, or right-click to open the context menu and choose **Create new task...**
Edit	Select a task, then right-click and choose **Edit task**
Delete	Select a task, then right-click and choose **Delete**

> **Warning**: Deleted tasks can't be restored.

Run a task

1. Select a task in the **Database Tasks** view.
2. Right-click it to open the context menu.
3. Click **Run task**.
    - Alternatively, use **Run task** button ( ) on the view's toolbar.

Run tasks from the command line

You can run saved tasks using the DBeaver command line interface. Use the `-runTask` parameter like this:

```
-runTask "@projectName:taskName"
```

Tasks run in the background without keeping the client open.

If your workspace has only one project, you can omit the `@projectName:` part.

On Windows, use the `dbeaverc` executable (or the legacy name `dbeaver-cli`) to run tasks.

> **Tip**: Add the `-nosplash` parameter to skip the splash screen.

Adjusting task configurations

The process of editing a task is similar to the process of creating a task.

By double-clicking on a task or right-click and choose **Edit task**, the task edit wizard will be opened. In this wizard, you can:

- Change the task properties.
- Change the file format, either output for an export task or input for an import task.
- Alter the set of objects for data transfer.
- Adjust any export/import configuration.

After modifying the task settings, click the **Save** button.

Scheduling tasks

> **Note**: This feature is available in Lite, Enterprise, and Ultimate editions only.

You can schedule tasks for later/regular execution. See the Task Scheduler article.

Set task timeout

You can set how long a task can run before it stops automatically. This helps avoid cases where one long or stuck task prevents the next ones from running.

To configure it:

1. In the Database Tasks view, select or create a task.
2. Click the **Timeout** button ( ).

   > **Note**: The **Timeout** button is enabled after you finish setting up the task.

3. Choose a preset duration (1 min, 5 min, 30 min, 1 hour) or set a custom time.

Customize the Database Tasks view

You can personalize how tasks are displayed in the **Database Tasks** view. Group them by type or category, and choose which columns are visible.

Group tasks

To organize your task list:

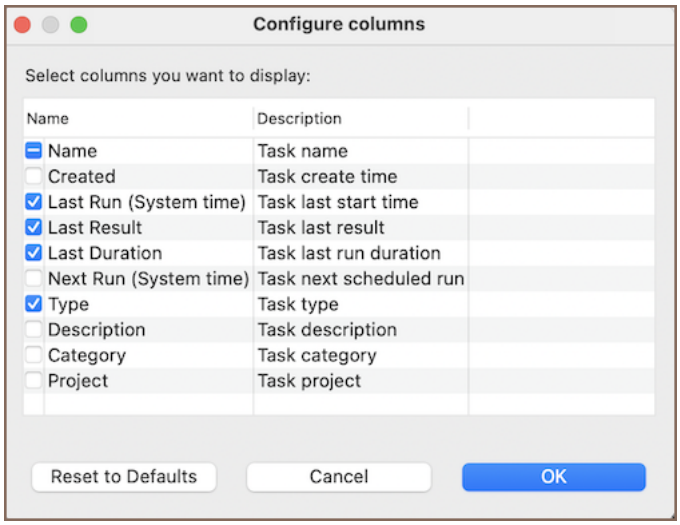
1. Right-click anywhere in the task list
2. Select **Group tasks by category** or **Group tasks by type** from the context menu

> **Tip**: You can enable both grouping options at the same time.

Configure columns

To show or hide columns:

1. Right-click anywhere in the task list
2. Click **Configure columns**
3. Select the columns to show or hide



Folders

You can organize tasks into folders. To create one:

1. Right-click in the **Database Tasks** view and select **Create new task folder**.
2. Enter a unique name and choose a project.

Move tasks between folders

You can:

- Open a task for editing, click **Back**, and change the folder.
- Drag and drop tasks between folders directly in the view.

Delete a folder

To remove a folder, right-click it and select **Delete**.

> **Warning**: All tasks inside will also be deleted. Deleted tasks can't be restored.

Task scheduler

> **Note**: This feature is available in Enterprise, and Ultimate editions only.

Table of contents

- Database Tasks view
- Schedule a task
- Update or cancel the schedule
- View schedule details
- Windows
- macOS or Linux
- Monitoring task execution
- Limitations
- Unix limitations
- Troubleshooting

DBeaver lets you automate routine tasks - like exporting data or running SQL scripts - by scheduling them to run at specific times.

Database Tasks view

Use the **Database Tasks** view to schedule and manage your tasks.

To open the view:

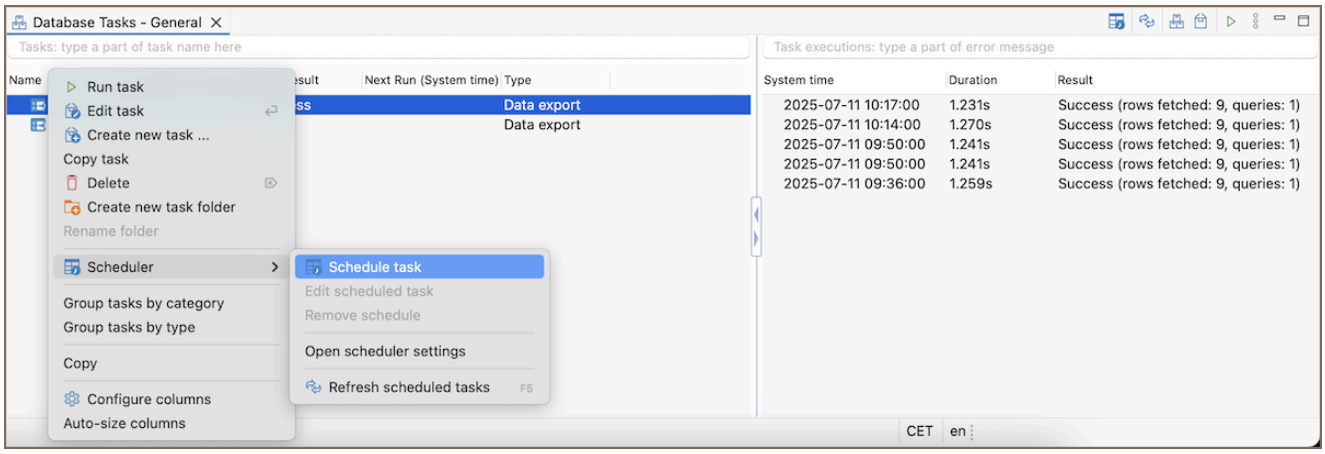
- navigate to **Database -> Tasks -> Database Tasks**
- or go to **Window -> Database Tasks**
- or click the **Show View (Database Tasks)** icon ( ) in the main toolbar

> **Note**: The toolbar is customizable. For further information, refer to Toolbar Customization article.

Schedule a task

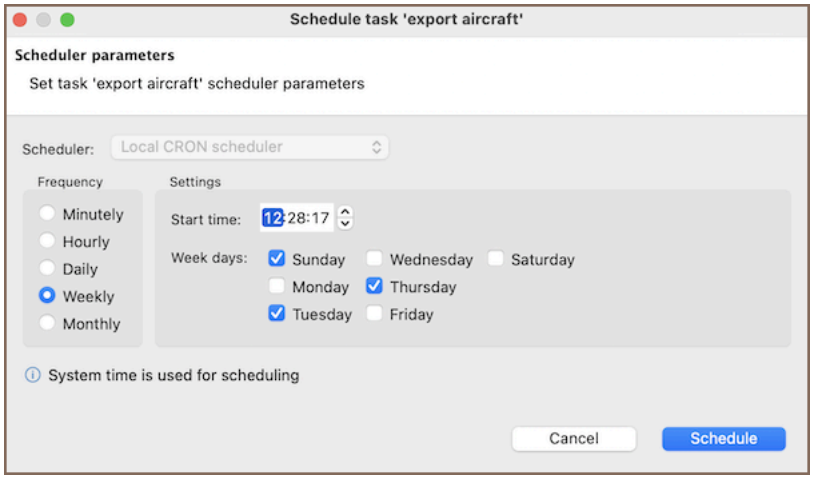
To schedule a task:

1. In the **Database Tasks** view, right-click the task you want to schedule
2. Select **Scheduler -> Schedule task**



   - or select the task you want to schedule and click the **Schedule task** icon ( ) in the **Database Tasks** toolbar.

3. You'll see the scheduler configuration dialog. Here, you can set:
   - task frequency (hourly, daily, weekly, etc.)
   - recurrence pattern
   - start time



> **Note**: macOS and GNU/Linux has some limitations.

4. To save the schedule click **Schedule** button
    - if successful, you'll see a confirmation dialog
    - if something goes wrong, you'll see an error dialog. Open the **Error Log** view to get more info about what went wrong.

> **Important**: When scheduling a task for the first time on macOS `10.15` or later, macOS will ask for permission. Click **Yes** to allow DBeaver to modify system `cron` settings.

Update or cancel the schedule

To update or cancel the schedule:

1. Right-click the task
2. Select **Scheduler -> Edit scheduled task** to update, or **Remove schedule** to cancel

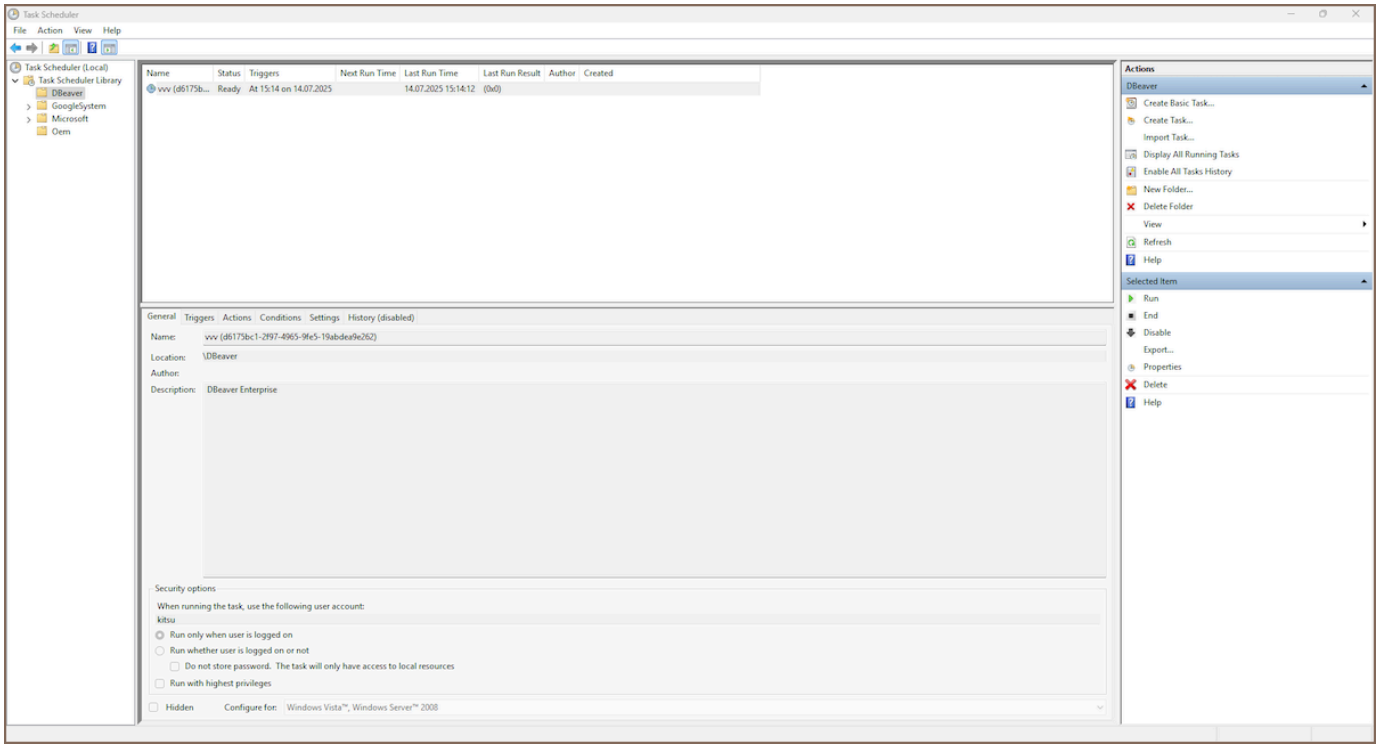
View schedule details

You can view and manage scheduled tasks using your system's built-in scheduler.

> Tasks are scheduled using:
>
> - Windows Task Scheduler on **Windows**
> - cron on **macOS** and **GNU/Linux**

Windows

To open **Windows Task Scheduler**, right-click a task in the task view and select **Scheduler** → **Open scheduler settings**.



DBeaver stores all scheduled tasks in a folder named `DBeaver`.

macOS or Linux

To view the `cron` entries used for task scheduling, right-click a task and select **Scheduler** → **Open scheduler settings**.

You can also use the terminal:

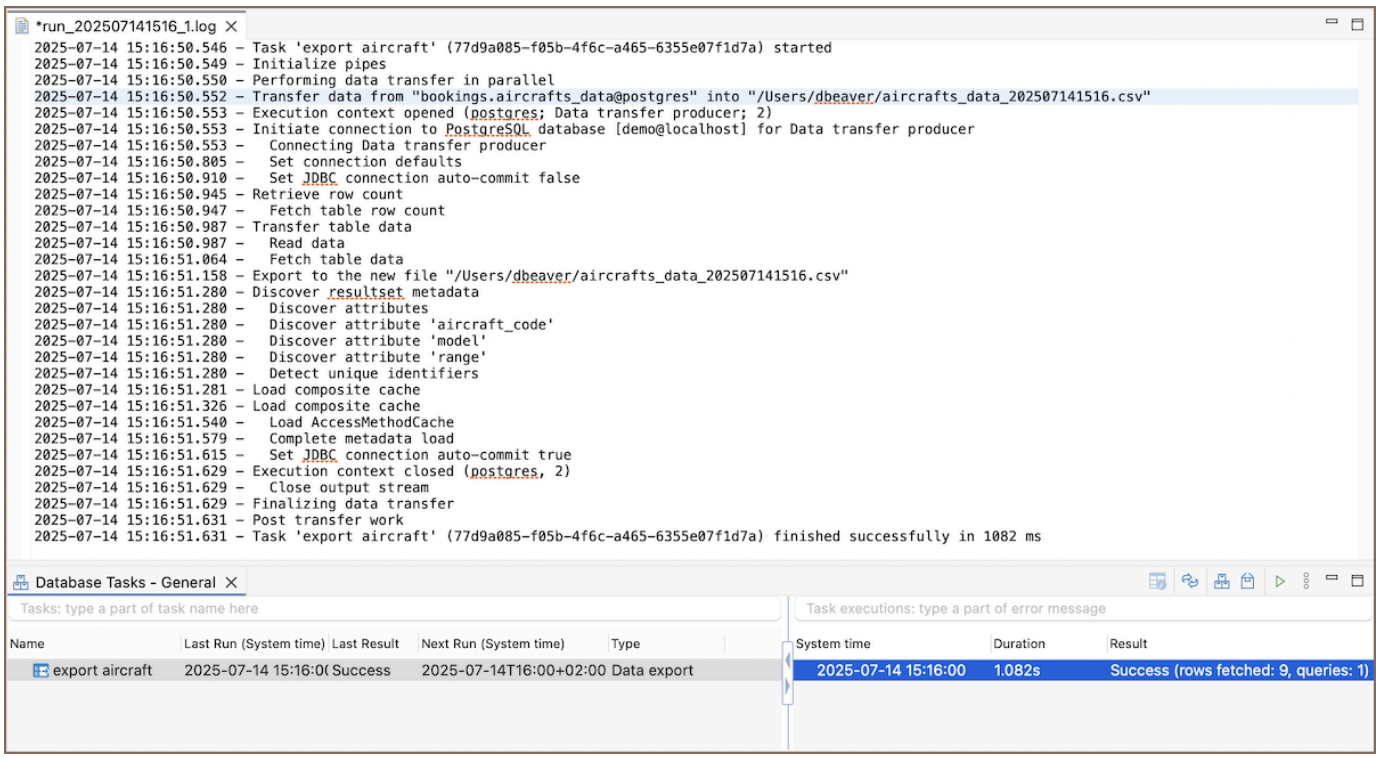
```
crontab -l
```

> **Warning**: While you can edit the crontab using `crontab -e`, we strongly recommend not doing so manually.

Monitoring task execution

You can view task execution logs on the right side of the tasks view.

Double-click a task run entry to see the full log, including detailed output, errors, and warnings.



> DBeaver stores task run logs in the workspace directory, under `.metadata/task-stats`.

Limitations

Unix limitations

macOS and GNU/Linux, which work on `cron` have some limitations:

- no start date option
- no start time for minutely tasks
- seconds are ignored even if you set them
- most `cron` implementations only allow minute-level granularity

If you set a task to run 42 minutes past the hour, it'll run at 1:42 PM, 2:42 PM, etc. You can't configure it to run at 1:42:15 PM or only on certain dates.

> **Warning**: Task start time is not guaranteed. The scheduler will try to run the task at the specified time, but execution may be delayed.

Troubleshooting

See Troubleshooting task scheduler issues for help with scheduled tasks.

Composite tasks

> **Note**: This feature is available in Enterprise, and Ultimate editions only.

Table of contents

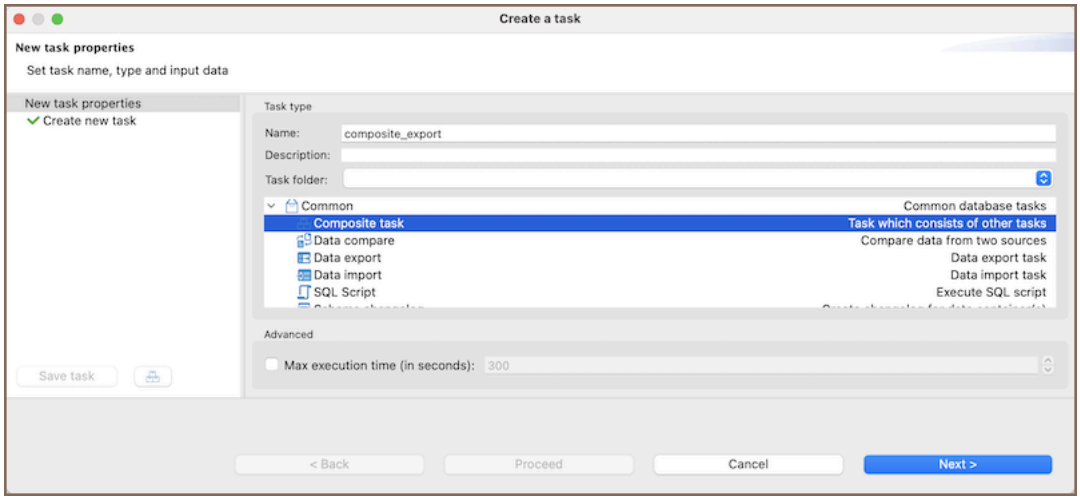
- Creating a Composite task
- Editing a Composite task

DBeaver allows you to create composite tasks that incorporate multiple other tasks. For example, you can group various data export tasks into a composite task, making managing and running them together at a set time simpler.

> For more details on creating and running tasks, see Task management.

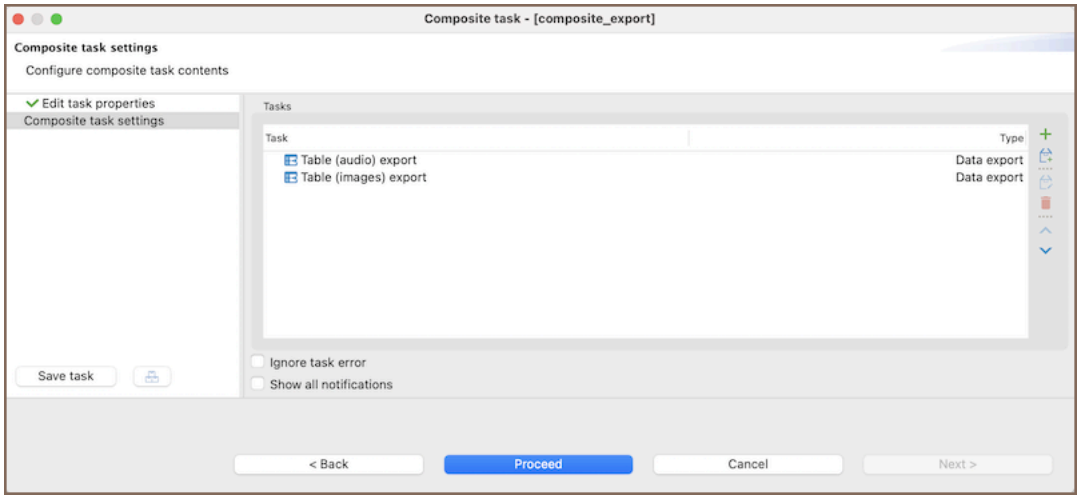
Creating a Composite task

1. To start creating a composite task, open the **Create a task** dialog. You can do this by:
    - Right-clicking in the Connections and selecting **Tools -> Create new task...**
    - Clicking on **Database -> Tasks -> Create new task...**
    - Navigating to **Database -> Tasks -> Database tasks**, right-clicking, and choosing **Create new task...**



    In the **Create a task** wizard, select **Composite task**. Enter a name, and optionally add a description and a task folder. Click **Next** to continue.

2. In the task configuration dialog, you can add tasks to your composite task using these controls:



Button/Checkbox Name	Button Image	Description
+ (Add)		Click to add existing tasks.
Create and add new task		Directly creates a new task and adds it to the composite task.
Edit task		Change the settings of a selected task.
Delete task		Removes the selected task from the composite task.
Drag-and-drop area		Drag tasks from **Database tasks** and drop them here to add them to the composite task.
Ignore task error		Allows the composite task to continue even if one task fails.
Show all notifications		Shows all notifications for the tasks within the composite task.

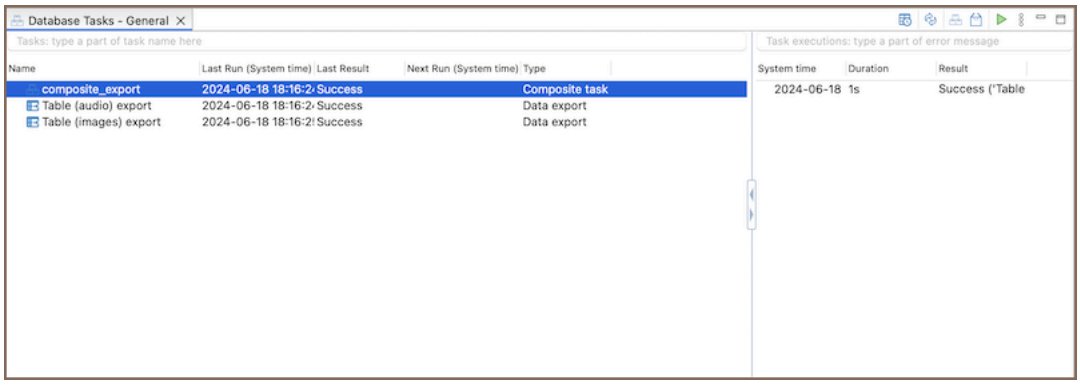
> **Note**: A composite task can also include other composite tasks.

3. After setting up the tasks, click **Proceed** to save and start the composite task.

> **Tip**: You can set a timeout for each task to avoid blocking others.

Editing a Composite task

To edit a composite task, go to **Database -> Tasks -> Database Tasks**. Here, you can edit, delete, or rearrange tasks in your composite task.



Integrated tools

GIT integration

> **Note**: This feature is available in Enterprise and Ultimate editions only. For Git functionality specific to the Team Edition, please refer to Team Edition Git documentation. For users of the Community edition to access **Git** feature, first install the required extension.

Table of contents

- Create and share a Project repository
- Cloning Existing Repositories
- Updating remote repository with changes
- Pulling new changes from remote branch
- Installing the Git extension

DBeaver supports full integration with Git, a version control system. This integration allows users to maintain project configurations, scripts, diagrams, bookmarks, and other artifacts in a Git repository. It facilitates project management by enabling version control and collaboration within DBeaver.

In this guide, we will cover the foundational Git operations in DBeaver, such as creating and cloning repositories, committing changes, and pulling updates.

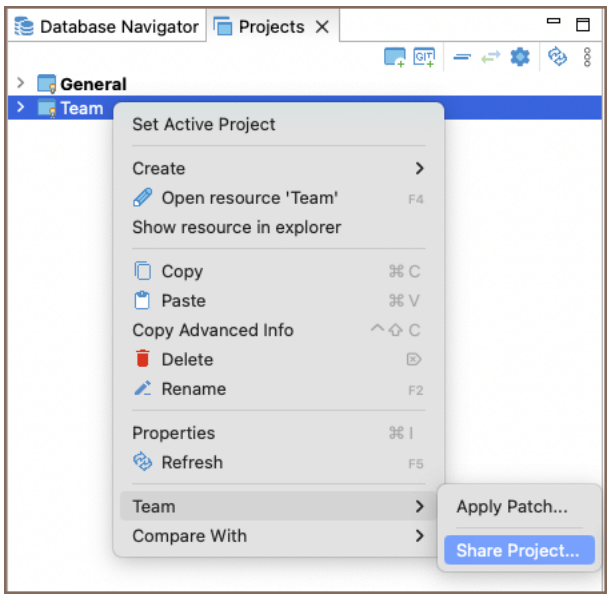
DBeaver's Git support is built on the Eclipse platform. For advanced Git functionalities like branching, and merging conflicts, refer to the Eclipse EGit documentation.

> For information on searching commit history, see Git search.

Create and share a Project repository

To share an existing project, follow these steps:

1. Navigate to the **Projects tab**, select the project you want to share, and choose **Team -> Share project** from the context menu.



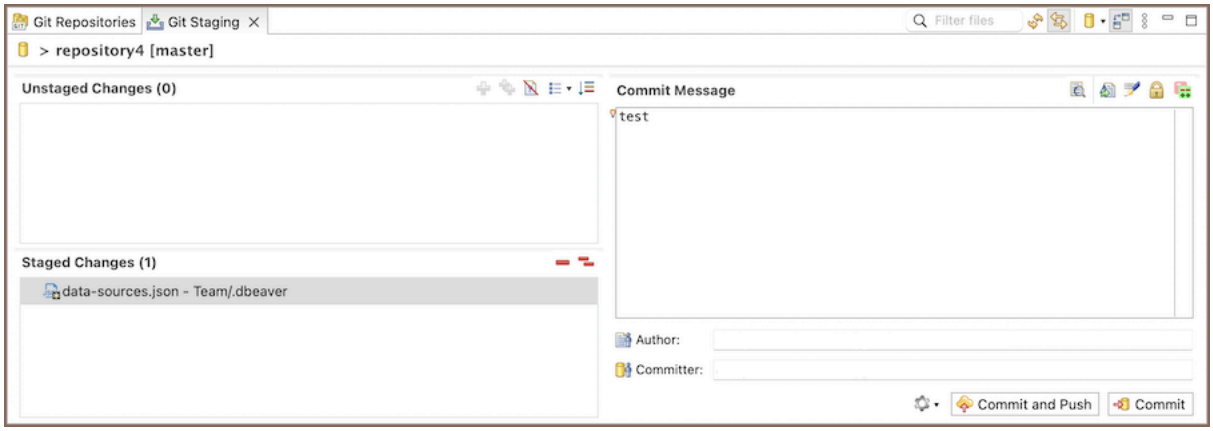
2. Create a new local repository.
3. On the repository page on GitHub copy the repository URL/SSH.

> **Tip**: For more details on creating GitHub repository you can refer to the official GitHub documentation.

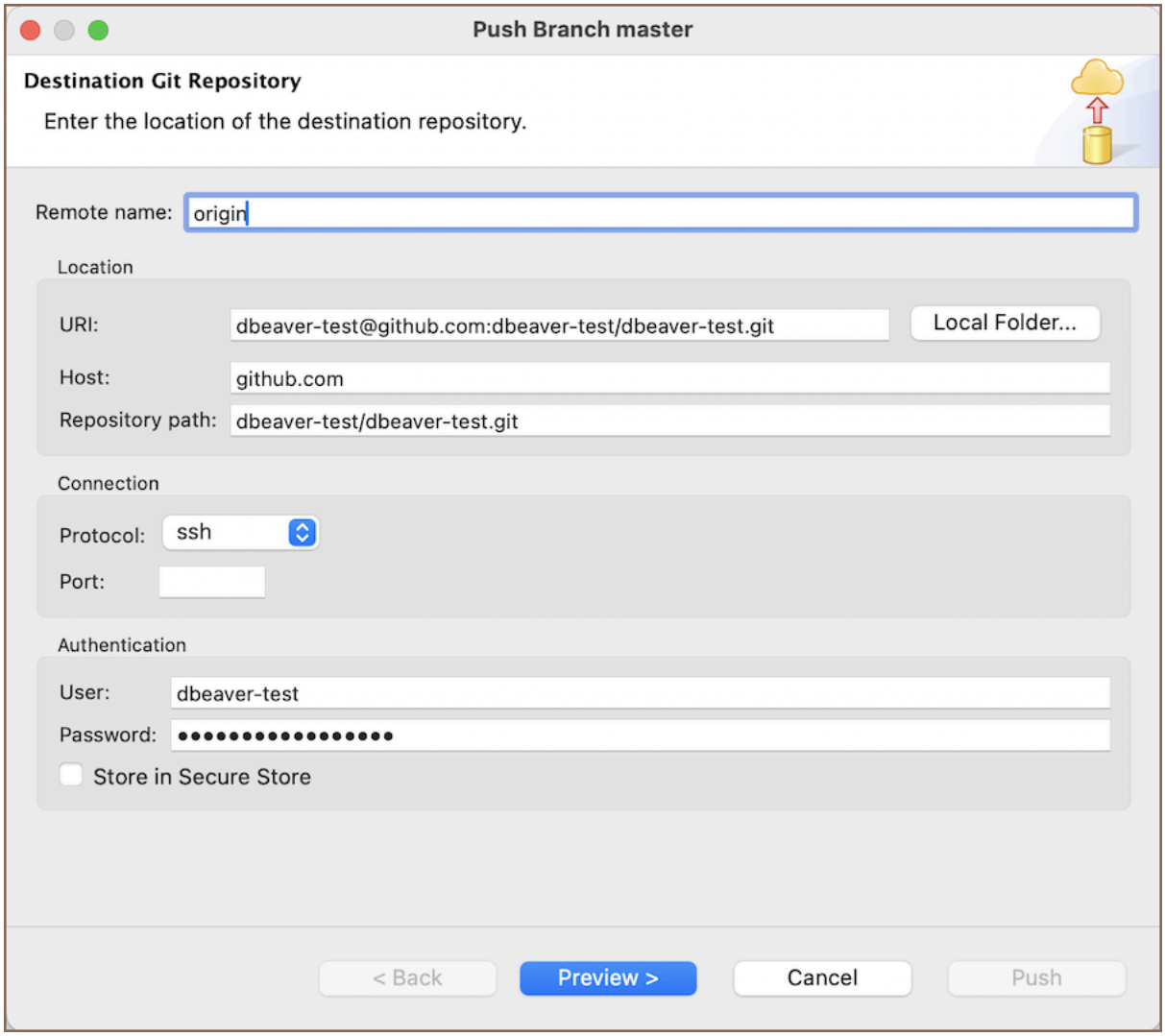
4. Generate a GitHub **access token** in **Settings -> Developer Settings -> Personal access tokens -> Tokens (classic) -> Generate new token (classic)**. Select all options in the `'repo'` section and `'read:org'` under `admin:org`.

> **Important**: DBeaver supports all types of Git tokens, including fine-grained and Personal access tokens (classic).

5. In DBeaver, right-click the project node and choose **Team -> Commit...** or use the shortcut `Ctrl+#`. Stage the files you wish to include in the commit by clicking **Add**. Enter a clear and concise commit message, then click **Commit** to finalize your changes.



6. Use the **Push HEAD** button and paste the GitHub repository URL/SSH and provide your GitHub **User** and **access token** as the **Password**.



7. Push changes to the remote repository by using the **Push** button.

Cloning Existing Repositories

To clone an existing repository, ensure you have a GitHub account and follow these steps:

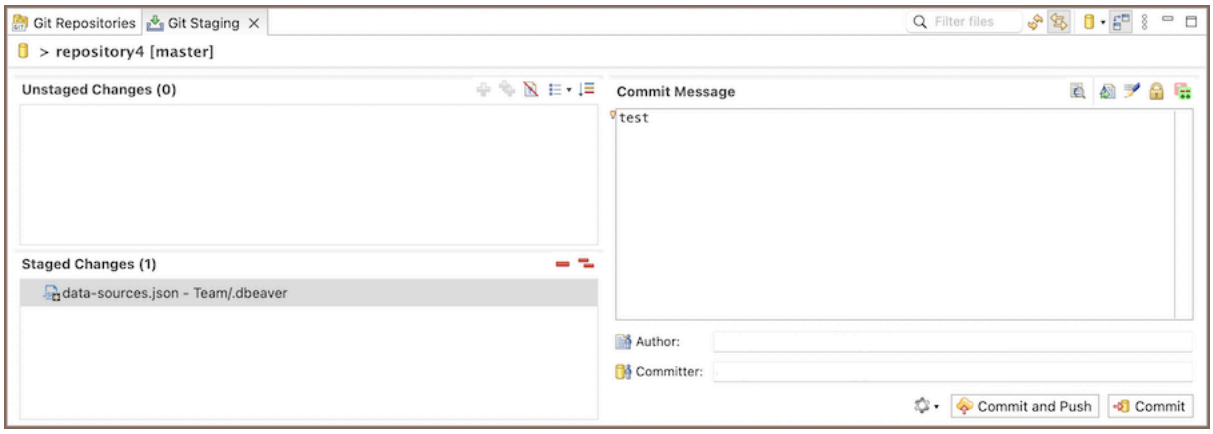
1. Click on **Create Project from Git** at the **Projects** Tab or navigate through **File -> Git -> Create Project from Git** in the Main menu.
2. Obtain the remote repository's URL from its GitHub page by clicking the **Code** button and copying the URL/SSH.
3. Generate a GitHub **access token** in **Settings -> Developer Settings -> Personal access tokens -> Tokens (classic) -> Generate new token (classic)**. Select all options in the `'repo'` section and `'read:org'` under `admin:org`.
4. In the wizard, enter the remote repository location. Use your GitHub **User** (it is commonly `git`) and **access token** instead of a **Password**.
5. Choose which branches to clone from the remote repository.
6. Define the local file system location for the repository storage and set initial configurations.

7. Select the desired wizard type for the project setup and complete the cloning process. The project will now be visible in the Project Explorer.

Updating remote repository with changes

After making changes to your project, such as creating, deleting, or modifying connections, scripts, or ERDs, follow these steps to update your remote repository:

1. Right-click on the project in the **Projects** tab and select **Team -> Commit** to open the commit interface.



2. In the **Git Staging** tab at the bottom, you will see **Unstaged Changes**. Use the **Add Selected files to the index** or **Add all files to the index** button to stage the changes you want to commit. Enter a commit message and then execute the commit using the **Commit** button or `Ctrl+Enter`. The commit will then be visible in the **History** tab (**Window -> Show view -> Other -> Version Control(Team) -> History**).

3. To push your commit to the remote repository, you can use the **Commit and Push** button. Alternatively, to push changes manually, navigate through the project's context menu (**Team -> Remote -> Push**) or from the **History** tab by right-clicking on the desired commit and selecting **Push commit**. This updates your git repository with the new changes.

Pulling new changes from remote branch

To incorporate new changes from the remote branch into your local project, follow these steps:

1. Right-click on the project in the **Project** tab and select **Team -> Pull**. Alternatively, if you are in the Git Repositories view, you can right-click on the repository and select **Pull**. This action fetches and merges any new changes from the branch your local repository is tracking into your current branch.

2. If you have not made any local changes, or if you wish to discard your local changes and synchronize with the remote branch, you can use the **Team -> Reset** command. This will reset your local repository to the state of the remote repository, discarding any local changes that have not been committed.

This process ensures your project is up to date with the latest changes from the remote branch, facilitating collaboration and avoiding potential merge conflicts.

Installing the Git extension

Community Edition users can manually install the Git extension by following these steps:

1. **Via Eclipse Marketplace:**
    - Visit the Eclipse Marketplace URL
    - Follow the instructions to install the Git support extension directly through your DBeaver installation.

2. **Via P2 Repository:**
    - Open DBeaver, and navigate to **Help -> Install New Software**.
    - Enter the P2 repository URL in the **Work with** field.
    - Select the Git extension from the list of available software and follow the on-screen instructions to complete the installation.

After installation, restart DBeaver for the changes to take effect, and you will have access to Git functionality within your DBeaver environment.

Tableau integration

> **Note**: This feature is available in Enterprise, and Ultimate editions only.

Table of contents

- Powering Tableau with DBeaver
- Easy connection to existing databases
- Working with custom SQL queries
- Managing charts
- Filling profile for Tableau connection
- Adding a Tableau Profile in DBeaver
- Creating a database connection
- Associate DBeaver and Tableau connection
- Editing Custom SQL Queries

Tableau is a leading data visualization and analytics platform that empowers you to connect, visualize, and share data in a way that makes it easier to understand. With its intuitive drag-and-drop interface, Tableau enables users to create interactive and shareable dashboards, providing insights into complex data sets.

Having a registered Tableau account is a prerequisite for those looking to leverage Tableau's capabilities. This article explores how DBeaver integrates with Tableau, enhancing the data manipulation process.

Powering Tableau with DBeaver

DBeaver's integration is compatible with both the On-premises and Cloud versions of Tableau, offering the following key features:

Easy connection to existing databases

DBeaver's integration with Tableau allows you to connect effortlessly to databases already present in Tableau. This seamless connection enables you to access and manage data without complex configurations or additional tools. Learn more about creating a connection.

Working with custom SQL queries

A standout feature of DBeaver's integration with Tableau is the ability to edit custom SQL queries stored in Tableau. You can modify these queries directly within DBeaver, providing a streamlined workflow for advanced data analysis. Learn more about editing custom SQL queries.

Managing charts

In addition to database management and query editing, DBeaver allows for the integration and manipulation of Tableau charts directly within its interface. Learn more about managing dashboards.

Filling profile for Tableau connection

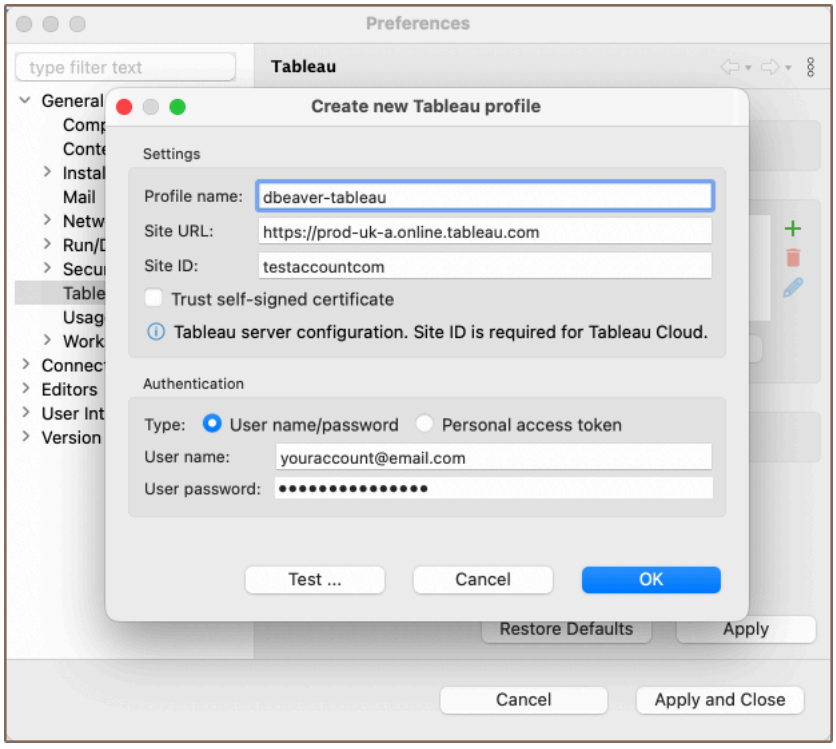
To configure Tableau credentials in DBeaver, follow these steps:

1. Navigate to **Window** -> **Preferences** -> **General** -> **Tableau**.
2. In the Tableau settings, you will find the following options:

Option	Description
Enable Tableau Integration	Enables Tableau in connections and in UI.
Tableau Profile	Manage Tableau profiles. User profiles are displayed here, and you can work with them using the buttons: - Add profile - Remove profile - Edit profile
Test Connection	Test the connection between DBeaver and Tableau.
Automatically associate Tableau connections	Automatically associate Tableau connections with DBeaver data sources. Learn more about associate connection.

Adding a Tableau Profile in DBeaver

When you click on **Add Profile**, a **Create new Tableau profile** window will appear. This is how you fill out the profile:



Field	Description
Profile name	The name of the profile in DBeaver (choose a convenient name for yourself).
Site URL	The URL should look like this: `https://prod-uk-a.online.tableau.com`.
SITE ID (required for Tableau Cloud)	Enter the correct `SITE ID` used during Tableau registration (e.g., copy `testaccountcom` from your Tableau home page URL like `https://prod-uk-a.online.tableau.com/#/site/testaccountcom/home`).
Trust self-signed certificate	Allows not to validate the server certificate. This option is useful if you are using a self-signed certificate and need it to pass validation. For more information on managing certificates, you can refer to our article about trust stores.
Authentication	Use the credentials of the account you use (login/password or token name/value).

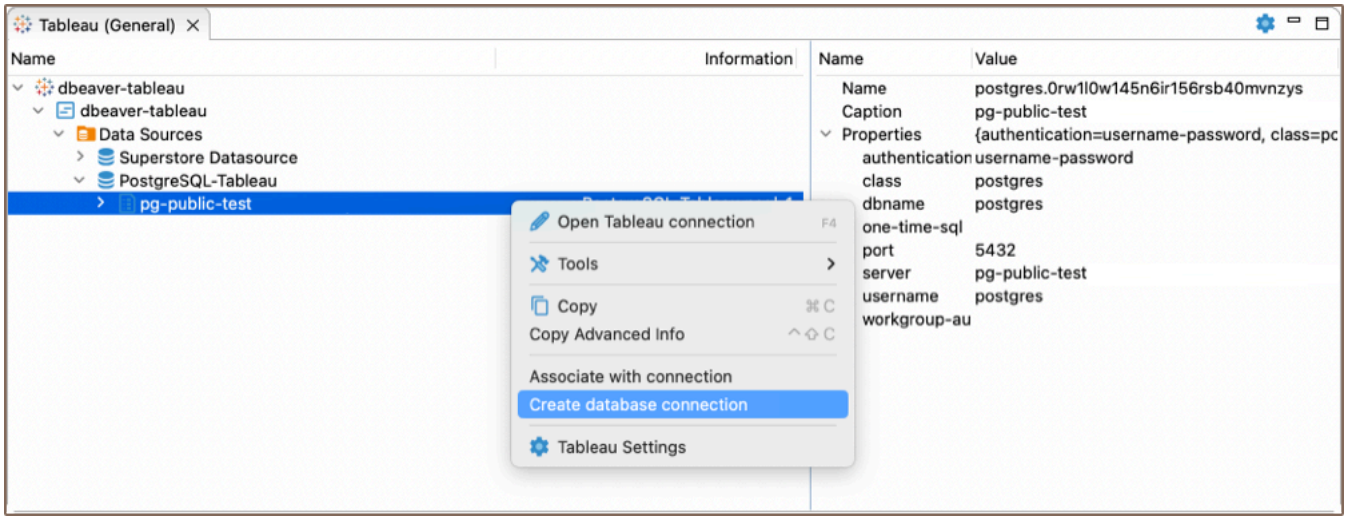
After filling out the profile details, you can test the connection to Tableau to ensure that everything is set up correctly. This step verifies that the provided information is accurate and that DBeaver can successfully communicate with Tableau.

Creating a database connection

Important: Before creating a connection in DBeaver, a connection must already be configured in Tableau. More details can be found in the official Tableau documentation.

Creating a connection between DBeaver and Tableau is a straightforward process:

1. In the DBeaver toolbar, click on the **Show View (Tableau)** button . The Tableau view will open in the lower part of DBeaver, displaying your profiles.
2. In your profile, select the desired database, right-click, and choose **Create Database Connection**.
3. DBeaver will prompt you to name the connection, and it will appear in the **Connections**.

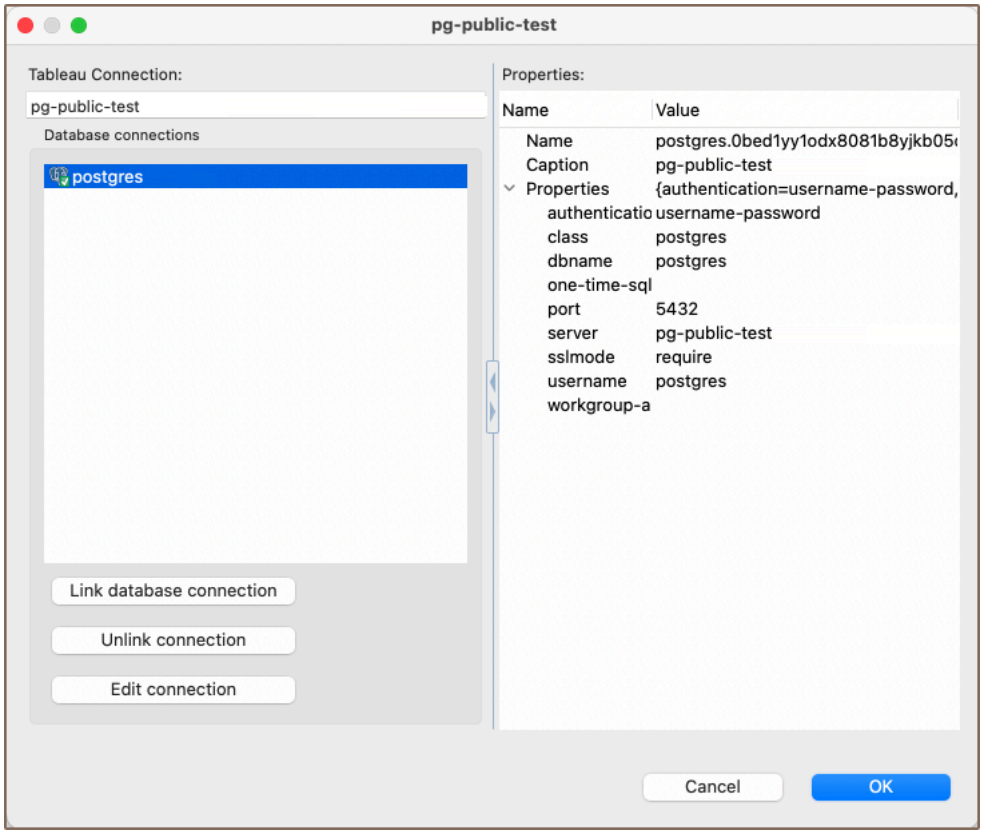


Associate DBeaver and Tableau connection

The **Associate with connection** button allows you to view a list of databases that can be associated with Tableau in DBeaver. It can automatically find the database based on the hosts.

> **Tip**: If the **Automatically associate Tableau connections** option is selected in the preferences, clicking on a table will automatically open it in DBeaver.

To configure the database that is associated with Tableau, you can use the **Open Tableau connections** option or double-click the database in Tableau view. A window with connections that are associated with Tableau will appear. From this window, you can choose to **Link database connection**, **Unlink connection**, or **Edit connection** as needed.



Editing Custom SQL Queries

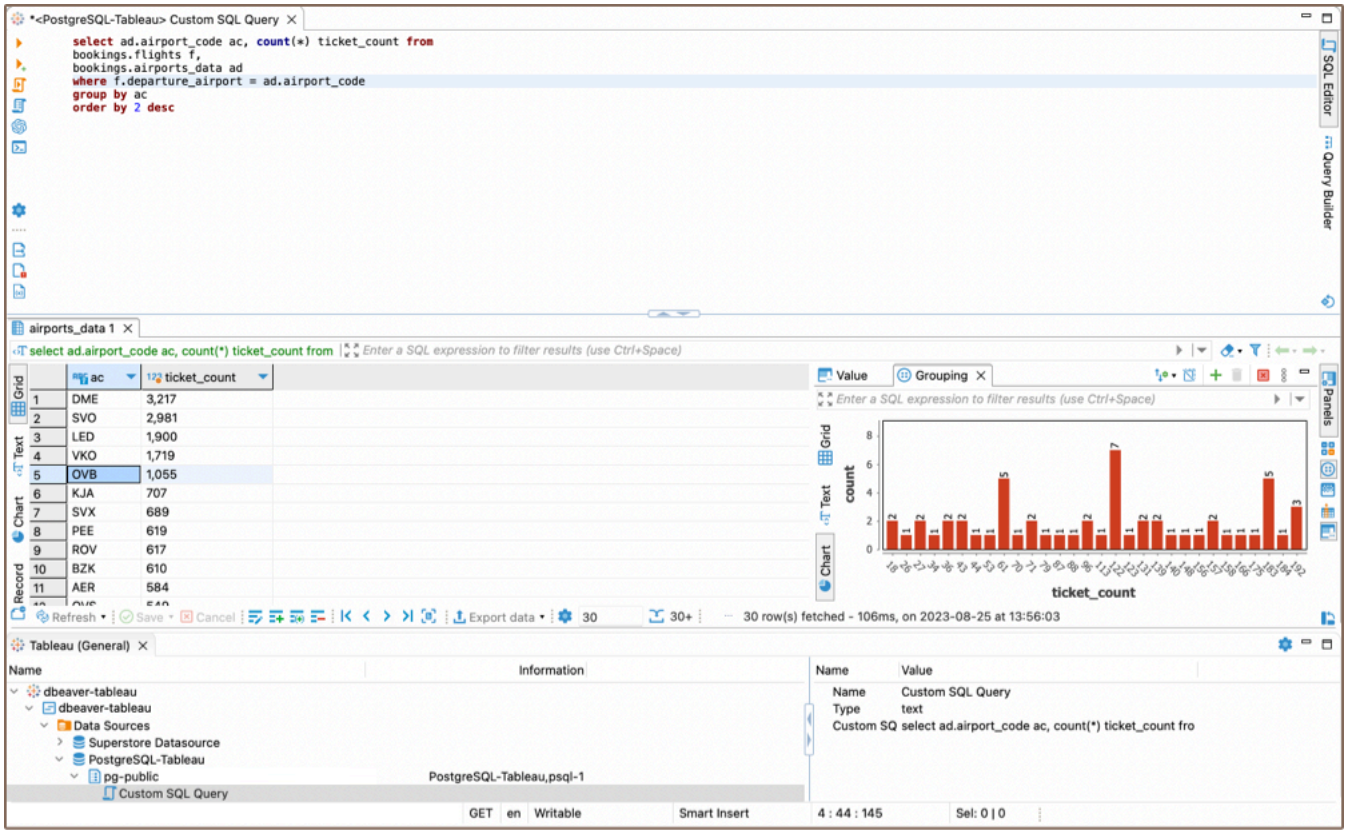
Working with custom SQL queries in Tableau can be challenging, especially when it comes to complex data manipulation. DBeaver's integration with Tableau offers a solution, allowing you to edit and manage custom SQL queries with ease. By connecting Tableau to DBeaver, you can leverage DBeaver's robust SQL editing tools to enhance your data analysis workflow in Tableau.

> **Important**: The custom SQL query must already be configured in Tableau. More details can be found in the official Tableau documentation.

This is how you can edit custom SQL queries using DBeaver:

1. In the Tableau view in DBeaver, select the Custom SQL query in your profile. You can double-click it, right-click and choose **Open Custom SQL**, or press the shortcut F4.
2. The SQL Editor will open, where you can work with the query just like any other query in DBeaver. You can make necessary changes, utilize grouping, filters, and other functionalities, and save and execute the query to see the changes reflected in Tableau.

> **Note**: It is essential to save the query in DBeaver to ensure that the changes are also reflected and saved in Tableau.



This integration simplifies the process of working with custom SQL queries in Tableau, making it more efficient and user-friendly.

Database debugger (PostgreSQL)

Table of contents

- Prerequisites for Debugging
- Installation
- PostgreSQL 12 on Ubuntu based distros
- Installation from source code
- Running debugger in the DBeaver interface

Prerequisites for Debugging

To enable interactive debugging of PL/pgSQL procedures on a Postgres server, you need to use the *plugin_debugger*. The *plugin_debugger* is a typical interactive debugger delivered as an extension. It requires a shared library preload in Postgres to operate the shared_preload_libraries parameter in the settings. The debugger is developed and maintained by EDB. Its source code is available for examination and improvement.

The debugger provides the required server API for debugging PL/SQL procedures with:

- Breakpoint management;
- Step-by-step tracing;
- Variable acquisition and management.

Installation

PostgreSQL 12 on Ubuntu based distros

If you happen to have a PostgreSQL 12 installed via *apt*, then the procedure is quite straightforward:

```
sudo apt install postgresql-12-pldebugger
sudo service postgresql restart
```

After that, run the following command in the database or databases that you wish to debug functions in:

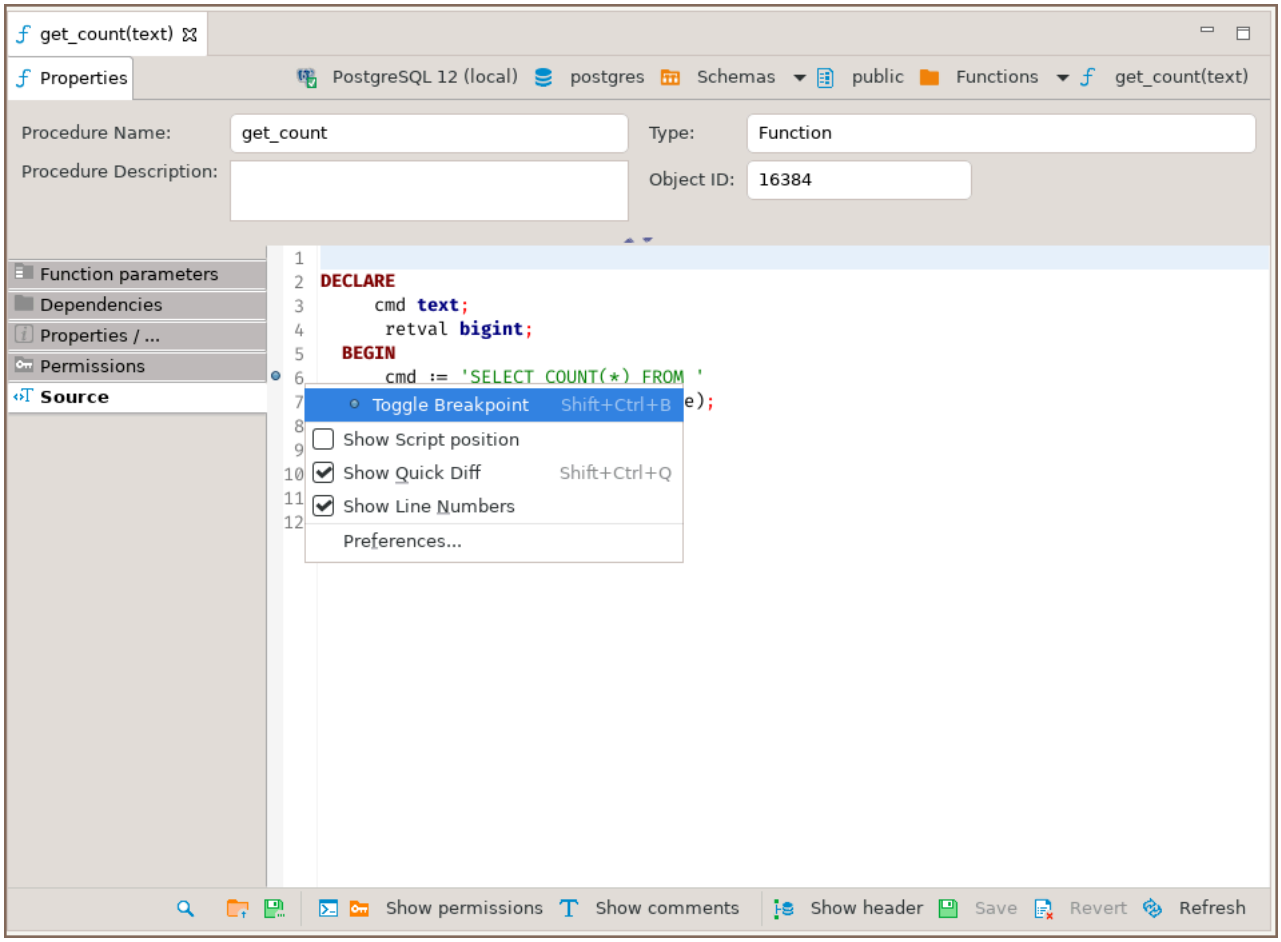
```
CREATE EXTENSION pldbgapi;
```

Installation from source code

You can find the source code in this repository. Installation instructions are located in the README file.

Running debugger in the DBeaver interface

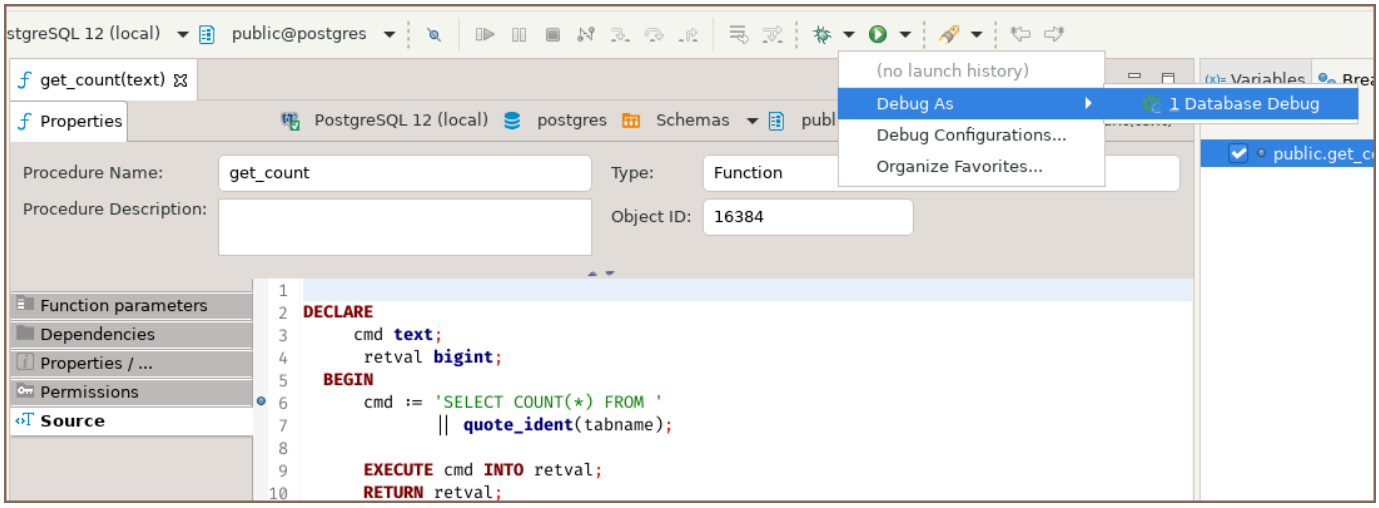
Open the source code of the function you want to debug. To toggle breakpoints, place the caret on the line you want the function to be stopped at and use a shortcut `Shift + Control + B`. Alternatively, you can toggle the breakpoint with your mouse by clicking on a ruler, as demonstrated in the screenshot below:



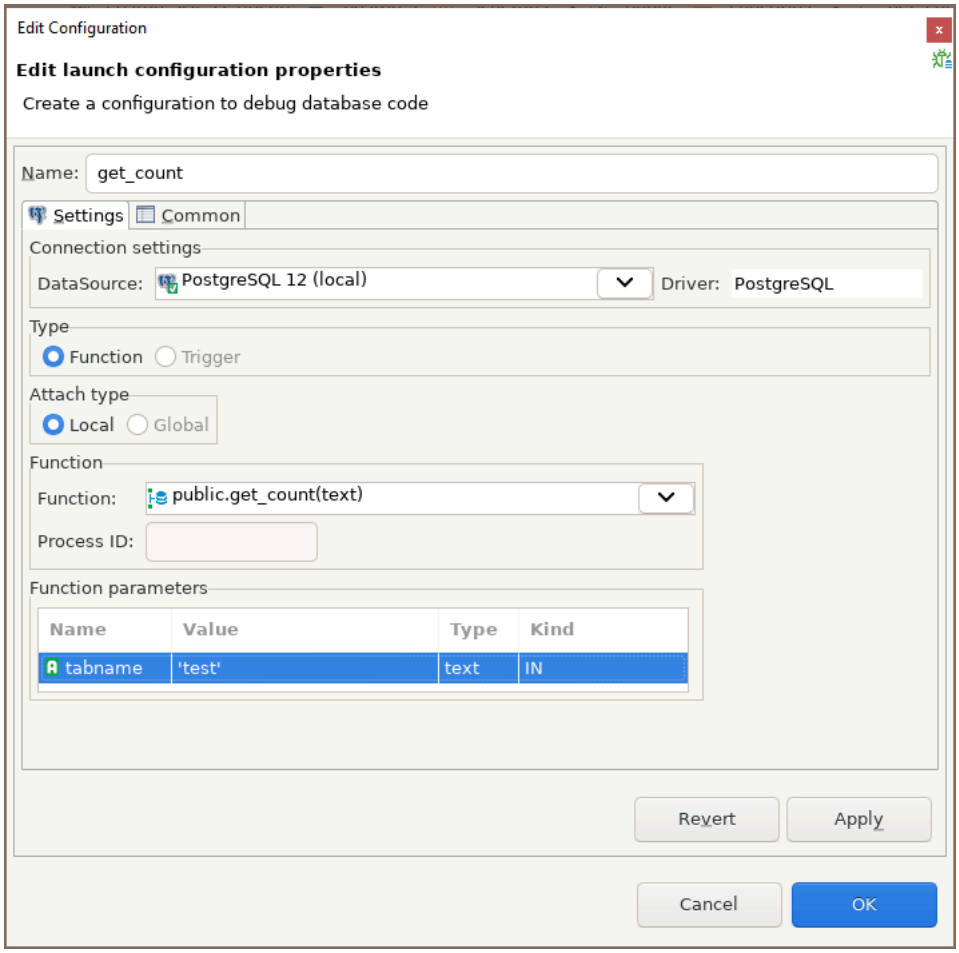
NB. You must only toggle the breakpoints when *Show header* option is **not** on.

Then you need to set up a debugging configuration. Locate the downward-facing arrow right to the bug icon, click on it, then

Debug As -> Database Debug:

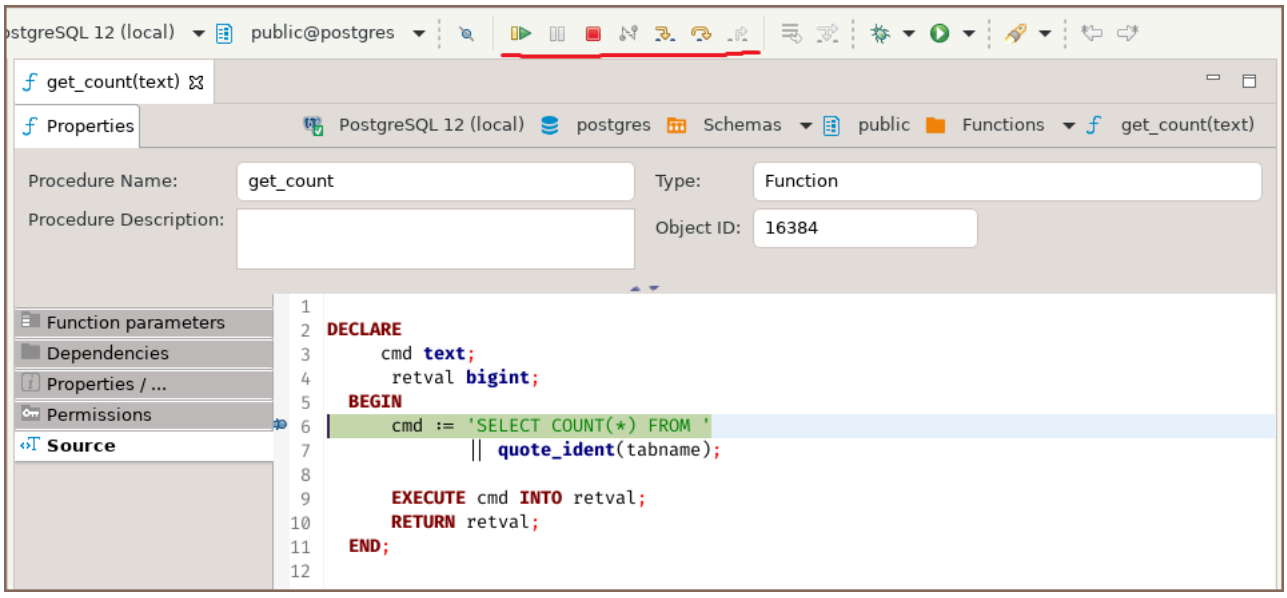


The *Edit Configuration* dialog opens. Set up input values in the table *Function parameters*.



Click on *OK* button, and you are ready to go!

The usual buttons essential for debugging such as *Step Over* and *Continue* are located here:



Session Manager

Table of contents

- Supported databases
- How to use

The Session Manager in DBeaver lets you view, search, and manage active database sessions - including terminating or disconnecting them.

Supported databases

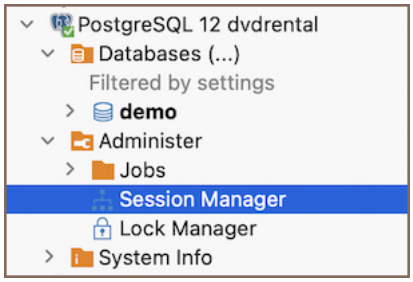
- Altibase
- AlloyDB ★

- Db2 for LUW
- Exasol
- Fujitsu Enterprise Postgres
- Greenplum
- MariaDB
- MongoDB ★
- MySQL
- MySQL on Google Cloud ★
- Netezza ★
- Oracle
- PostgreSQL
- PostgreSQL on Google Cloud
- Redshift ★
- Microsoft SQL Server
- Microsoft SQL Server on Google Cloud ★

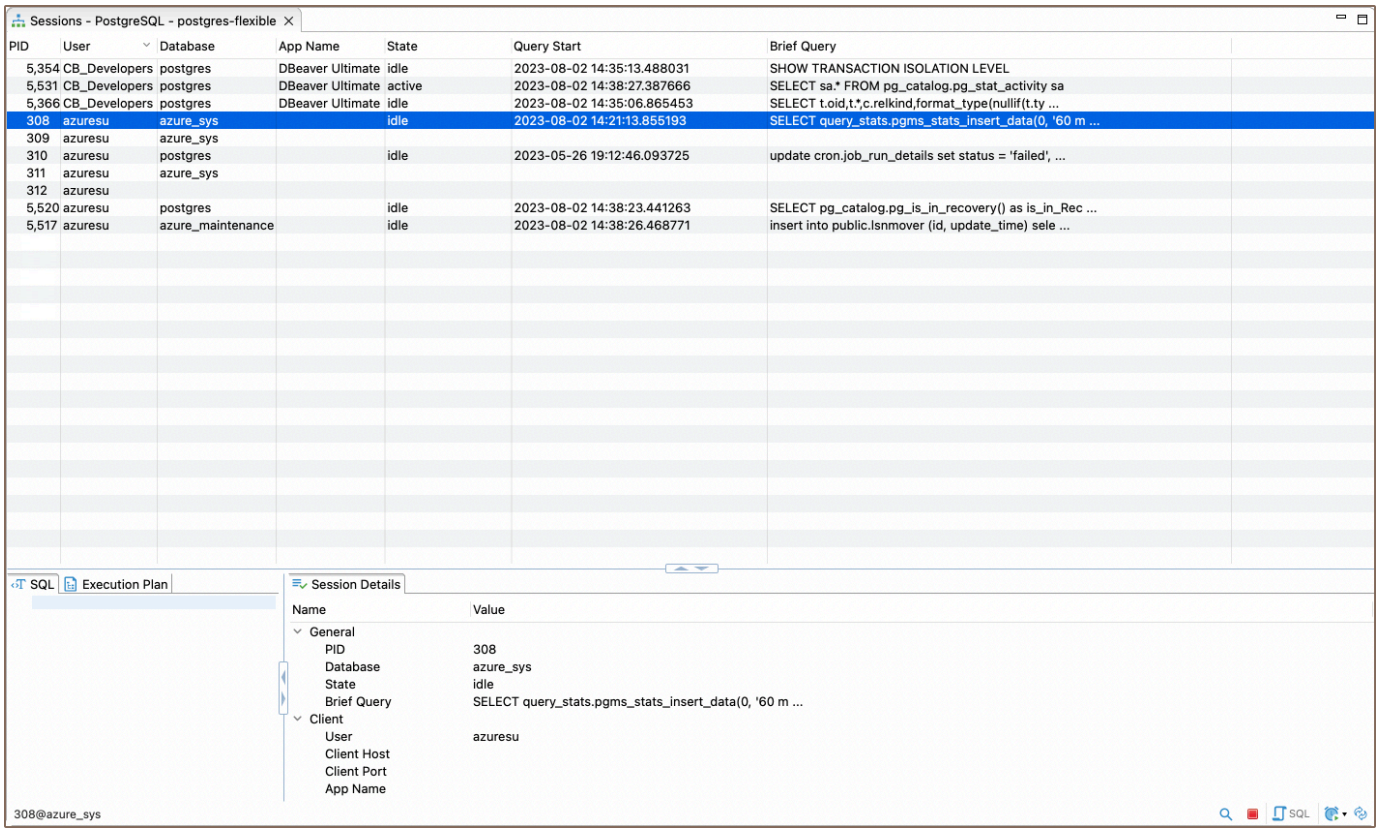
How to use

To open the Session Manager:

1. In the **Connections**, navigate to your desired database.
2. Go to the **Administer** section.



3. Double-click on **Session Manager** or right-click and select **View Session Manager**.



Right-click a session or select it and use the toolbar buttons at the bottom of the editor. Available actions depend on your database type.

Button	Image	Description
Kill Session	Kill Session	Terminates the selected session forcefully.
Disconnect Session	Disconnect session	Disconnects the selected session from the server.
Show Active/All Connections	Show Active/All Connections	Toggles between active sessions only and all sessions.

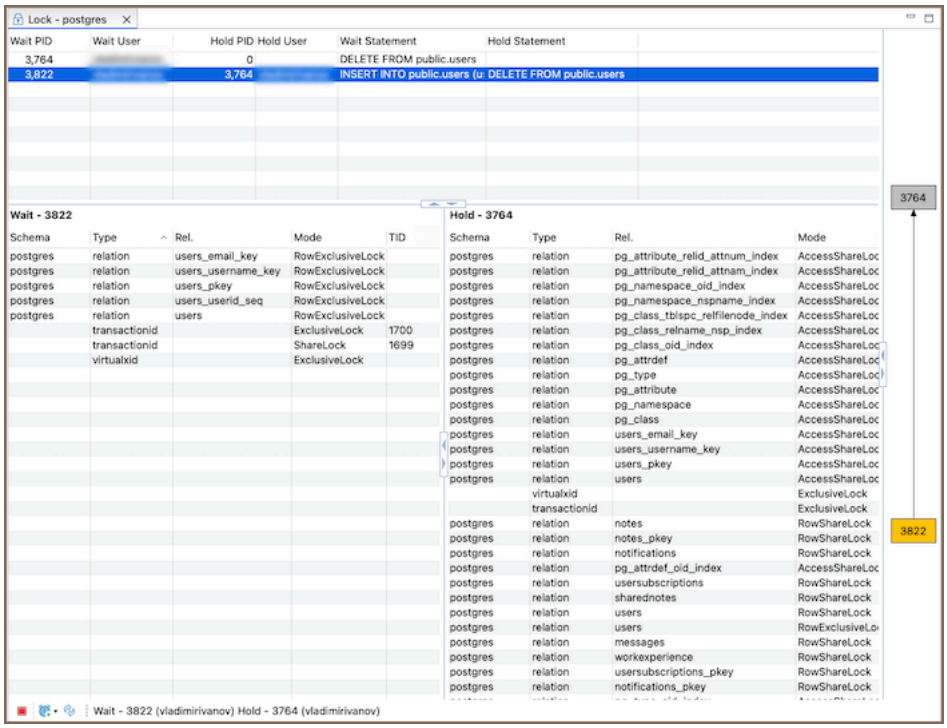
Button	Image	Description
Show Background	Show background	Shows background processes running in the session.
Show Inactive	Show inactive	Shows inactive sessions.
Refresh	Refresh sessions	Updates the session list.
Configure Auto-Refresh		Sets up automatic session list refresh.
SQL Script	SQL	Runs a SQL script that displays sessions as a table.

Lock Manager

Table of contents

- Supported databases
- How to use
- Managing locks

In DBeaver, you can use the Lock Manager to monitor and manage database locks. It helps identify blocking queries, and deadlocks, allowing you to resolve issues directly.



Supported databases

- AlloyDB
- Altibase
- Exasol
- Db2
- Greenplum
- Oracle
- PostgreSQL

How to use

To see the processes causing the blocks and their queries:

1. In the Connections, expand your database connection.
2. Go to the **Administer** tab and select **Lock Manager**.
3. Review the list of active locks to identify blocking and waiting sessions.
4. Analyze the **Wait Statement** and **Hold Statement** columns to understand the conflicting queries.

> **Tip**: Fields in the Lock Manager vary by database. Refer to your database's documentation for specifics.

Managing locks

To manage locks and resolve conflicts, use the options available bottom toolbar of the **Lock Manager**:

Button	Name	Description
	Kill waiting session	Terminates the session or process waiting for a lock, resolving blocking issues by stopping it.
	Configure auto-refresh	Sets up automatic refresh intervals to keep lock data updated without manual input.
	Refresh locks	Manually refreshes lock data to show the most recent information about active locks.

> **Tip**: You can right-click on a specific lock in the list and use the context menu to perform the same actions.

Administration

Admin preferences

Table of contents

- Locating preference files
- Reading and modifying preference files
- Preferences
- Default Connections view for new connections
- Miscellaneous Connections settings
- Default transaction settings
- Metadata settings
- Query manager settings
- Connections settings
- Error Logs settings
- User Interface settings
- SQL Editor settings
- Data Editor settings
- Drivers settings
- Disable Windows Defender pop-up
- Exporting and importing preferences

This guide describes how to set up an application preferences through configuration files without even booting it up.

> **Tip**: You can also specify parameters directly in your `data-sources.json` file.

Locating preference files

Preference files have the extension `.prefs` and can be found in the workspace directory under the following path:
`.metadata/.plugins/org.eclipse.core.runtime/.settings`

You might not find a preference file you are interested in, if it has not been created yet. In this case, you will need to create one yourself. See the examples below to figure out which preference file you need.

Reading and modifying preference files

Preference files consist of key-value pairs glued together with = and separated with a line break:

```
# file org.jkiss.dbeaver.test.prefs
some.key=value1
some.other.key=value2
```

Preferences

You can place the following preferences in the `org.jkiss.dbeaver.core.prefs` file:

Default Connections view for new connections

Name	Type	Default Value	Allowed Values	Description
navigator.settings.default.preset	string	`simple`	`simple`, `advanced`	Sets the default view mode for a new connections.Do not specify this preference if you want to configure a custom preset.
navigator.settings.default.showSystemObjects	boolean	`false`	`true`, `false`	Controls whether system objects must be shown.Used if preset is not specified.
navigator.settings.default.showUtilityObjects	boolean	`false`	`true`, `false`	Controls whether utility objects must be shown.Used if preset is not specified.
navigator.settings.default.showOnlyEntities	boolean	`false`	`true`, `false`	Controls whether only schemas and tables must be shown.Used if preset is not specified.
navigator.settings.default.mergeEntities	boolean	`false`	`true`, `false`	Controls whether all tables must be shown in a single list (without any catalog or database).Used if preset is not specified.

For more information, please see Simple and Advanced View.

Miscellaneous Connections settings

Name	Type	Default Value	Allowed Values	Description
navigator.settings.default.connectionPattern	string	`host_or_database`	expression with variables	Pattern for new connections title.
navigator.show.folder.placeholders	boolean	`true`	`true`, `false`	Displays placeholders for special folders. Show special folders (e.g. Scripts) even if they were not created yet.
navigator.sort.case.insensitive	boolean	`true`	`true`, `false`	Sorts items in case-insensitive mode.

Name	Type	Default Value	Allowed Values	Description
navigator.sort.forlers.first	boolean	`true`	`true`, `false`	Shows folders first.

Default transaction settings

Name	Type	Default Value	Allowed Values	Description
transaction.smart.commit	boolean	`false`	`true`, `false`	Enables smart commit mode.
transaction.smart.commit.recover	boolean	`true`	`true`, `false`	Returns to auto-commit mode after transaction end.
transaction.auto.close.enabled	boolean	`false`	`true`, `false`	Automatically ends (rollback) transaction after idle period.
transaction.auto.close.ttl	integer	`900`	integer value	Timeout before transaction close.
transaction.show.notifications	boolean	`true`	`true`, `false`	Shows transaction and notification.

Metadata settings

Name	Type	Default Value	Allowed Values	Description
database.meta.separate.connection	string	`DEFAULT`	`ALWAYS`, `NEVER`	Controls whether to open a separate connection for metadata read. Do not specify this preference if you want to use default settings.
database.meta.casesensitive	boolean	`false`	`true`, `false`	Specifies the usage of case-sensitive names in DDL statements.
database.props.expensive	boolean	`true`	`true`, `false`	Enables display of row count for tables.
database.meta.server.side.filters	boolean	`true`	`true`, `false`	Enables filtering on the server side.

Query manager settings

Name	Type	Default Value	Allowed Values	Description
qm.queryTypes	string	`USER`, `USER_FILTERED`, `USER_SCRIPT`	`USER`, `USER_FILTERED`, `USER_SCRIPT`, `UTIL`, `META`, `META_DDL`	View for different query types.
qm.objectTypes	string	`session,txn,query`	`session,txn,query`	Displays different object types. `txn` - Transactions.
qm.maxEntries	integer	`200`	integer value	Number of entries displayed per page.
qm.storeLogs	boolean	`false`	`true`, `false`	Enables the saving of logs.
qm.logDirectory	string	workspace directory	string value	Folder location for saving logs.
qm.historyDays	integer	`90`	integer value	Number of days for storing logs.

> For more details on Query Manager, see our article.

Connections settings

Name	Type	Default Value	Allowed Values
navigator.show.connection.host	boolean	`true`	`true`, `false`
navigator.show.objects.description	boolean	`false`	`true`, `false`
navigator.show.child.count	boolean	`false`	`true`, `false`
navigator.sort.case.insensitive	boolean	`false`	`true`, `false`
navigator.sort.case.insensitive.ignore.case	boolean	`false`	`true`, `false`
navigator.show.statistics.info	boolean	`true`	`true`, `false`
navigator.show.node.actions	boolean	`true`	`true`, `false`
navigator.show.folder.placeholders	boolean	`true`	`true`, `false`
navigator.sort.forlers.first	boolean	`true`	`true`, `false`
navigator.sort.case.insensitive	boolean	`false`	`true`, `false`
navigator.color.nodes.all	boolean	`false`	`true`, `false`
navigator.show.objects.tips	boolean	`true`	`true`, `false`
navigator.show.tooltips	boolean	`true`	`true`, `false`
navigator.show.tooltips.file.contents	boolean	`false`	`true`, `false`
navigator.object.doubleClick	string	`EDIT`	`EXPAND`
navigator.connection.doubleClick	string	`EXPAND`	`EDIT`, `CONNECT`, `SQL_EDITOR`, `SQL_EDITOR_NEW`
navigator.object.defaultEditorPage	string	`""`	`default.object.editor,org.jkiss.dbeaver.ui.editors.data.DatabaseDataEditor,org.jkiss.dbeaver.ui.ed`

Name	Type	Default Value	Allowed Values
navigator.expand.on.connect	boolean	`false`	`true`, `false`
navigator.restore.filters	boolean	`false`	`true`, `false`
navigator.long.list.fetch.size	integer	`5000`	integer values
navigator.restore.state.depth	integer	`0`	integer values

Error Logs settings

Name	Type	Default Value	Allowed Values	Description
logs.debug.enabled	boolean	`true`	`true`, `false`	Enables debug logs.
logs.debug.location	string	`${workspace}/.metadata/dbeaver-debug.log`	string values	Location of the log file.
logs.files.output.maxSize	integer	`10485760`	integer values	Maximum log file size (KB).
logs.files.output.maxCount	integer	`3`	integer values	Maximum backup log files count.

User Interface settings

Name	Type	Default Value	Allowed Values	Description
ui.auto.update.check	boolean	`true`	`true`, `false`	Enables automatic updates check.
platform.language	string	`en`	`en`, `zh_CN`, `ru`, `fr`, `de`, `it`, `ja`, `es`, `pt`, `ko`, `zh_TW`,	Platform language setting.
java.client.timezone	string	operating system time zone	time zones	Time zone setting.
notifications.enabled	boolean	`true`	`true`, `false`	Determines whether notification popups are allowed to appear.
notifications.closeDelay	integer	`3000`	integer values	Defines the timeout (in ms) before a notification will disappear.

SQL Editor settings

Name	Type	Default Value	Allowed Values	Description
database.editor.separate.connection	string	`DEFAULT`	`ALWAYS`, `NEVER`	Opening a separate connection for each editor.
database.editor.connect.on.activate	boolean	`true`	`true`, `false`	Connects on editors activation.
database.editor.connect.on.execute	boolean	`false`	`true`, `false`	Connects on query execution.
SQLEditor.autoSaveOnChange	boolean	`false`	`true`, `false`	Auto-saves after any modifications.
SQLEditor.autoSaveOnClose	boolean	`false`	`true`, `false`	Auto-saves editor on close.
SQLEditor.autoSaveOnExecute	boolean	`false`	`true`, `false`	Saves editor on query execution.
SQLEditor.autoSaveActiveSchema	boolean	`true`	`true`, `false`	Saves/restores active schema.
SQLEditor.resultSet.closeOnError	boolean	`false`	`true`, `false`	Closes results tab on error.
SQLEditor.resultSet.replaceCurrentTab	boolean	`true`	`true`, `false`	Replaces active result tab on single query execution.
SQLEditor.resultSet.orientation	string	`HORIZONTAL`	`VERTICAL`	Result view orientation.
SQLEditor.outputPanel.autoShow	boolean	`true`	`true`, `false`	Opens output viewer on new messages.
SQLEditor.outputPanel.autoShow	integer	`20`	integer values	Sets maximum of result tabs for single query.
SQLEditor.ContentAssistant.auto.activation.enable	boolean	`true`	`true`, `false`	Enables SQL Assistant auto-activation.
SQLEditor.ContentAssistant.activate.hippie	boolean	`false`	`true`, `false`	Uses Hippie Engine for autocompletion.
SQLEditor.ContentAssistant.auto.activation.delay	integer	`0`	integer values	Auto activation delay (ms).
SQLEditor.ContentAssistant.auto.keystrokes.activation	boolean	`true`	`true`, `false`	Activates SQL Assistant on typing.
SQLEditor.ContentAssistant.insert.single.proposal	boolean	`true`	`true`, `false`	Auto inserts proposal in SQL assistant if there is only one option.
SQLEditor.ContentAssistant.autocompletion.tab	boolean	`true`	`true`, `false`	Use Tab for autocompletion in SQL Assistant.
SQLEditor.ContentAssistant.insert.case	integer	`0`	`1`, `2`	SQL Assistant insertion case (1- Upper case, 2 - Lower case).
SQLEditor.ContentAssistant.replace.word	boolean	`false`	`true`, `false`	Sets up whether to replace the current word on accepting autocompletion proposal.
SQLEditor.ContentAssistant.proposals.short.name	boolean	`false`	`true`, `false`	Uses short object names in SQL Assistant.
SQLEditor.ContentAssistant.show.helpTopics	boolean	`false`	`true`, `false`	Shows server help topics in SQL Assistant.
SQLEditor.ContentAssistant.show.values	boolean	`true`	`true`, `false`	Proposes values for the given attribute. Note that it can drastically degrade performance when working with large tables, as it selects

Name	Type	Default Value	Allowed Values	Description
				distinct values.
sql.proposals.insert.table.alias	string	`PLAIN`	`NONE`, `EXTENDED`	Inserts table aliases (in FROM clauses).
SQLEditor.format.activeQuery	boolean	`true`	`true`, `false`	Enables formatting of the query under cursor only.
script.auto.folders	boolean	`false`	`true`, `false`	Creates script folder for each connection.
com.dbeaver.ee.vqb.disabled	boolean	`false`	`true`, `false`	Disables the Visual Query Builder in the SQL Editor.

Data Editor settings

Name	Type	Default Value	Allowed Values	Description
resultset.autofetch.next.segment	boolean	`true`	`true`, `false`	Automatically loads the next data segment when the current one is fully processed or viewed.
resultset.reread.on.scroll	boolean	`true`	`true`, `false`	Refreshes the data when moving to the next page.
resultset.maxrows.sql	boolean	`false`	`true`, `false`	Modifies the source SQL query to scroll through or limit results. This is typically achieved using the LIMIT/OFFSET SQL clause.
resultset.order.mode	string	`SMART`	`CLIENT_SIDE`, `SERVER_SIDE`, `SMART`	Specifies the method of sorting returned query results.
resultset.cancel.timeout	integer	`5000`	integer values	Defines the duration (in milliseconds) the system waits before cancelling a non-responsive query.
sql.query.filter.force.subselect	boolean	`false`	`true`, `false`	Forces the usage of subqueries when applying filters to the data.
resultset.column.label.ignore	boolean	`false`	`true`, `false`	Prioritizes column names over column labels when referencing data.
resultset.edit.key.use_all_columns	boolean	`false`	`true`, `false`	Forms a virtual composite key using all table columns for data saving. If set to `true`, this will determine the action taken if a key element is missing. If set to `false`, no action will be taken when a key element is missing.
resultset.edit.disableIfKeyMissing	boolean	`false`	`true`, `false`	Prevents data modification when a primary key is absent. This setting defines the action taken when the assumed primary key is missing.
keep.statement.open	boolean	`false`	`true`, `false`	Closes the result set after a selection. To view a cursor column value, the option should be enabled.
resultset.datetime.editor	boolean	`false`	`true`, `false`	Enables the advanced datetime editor in the data editor.
resultset.edit.new.row.after	boolean	`true`	`true`, `false`	Places newly inserted rows immediately following the currently selected row.
resultset.edit.refreshAfterUpdate	boolean	`true`	`true`, `false`	Automatically refreshes the data after making changes.
resultset.filter.use.navigator	boolean	`true`	`true`, `false`	Employs filters in Connections to refine the displayed data.
resultset.confirm.beforeSave	boolean	`false`	`true`, `false`	Displays a prompt asking for a confirmation before saving changes.
resultset.show.errorDialog	boolean	`false`	`true`, `false`	Presents error messages in a separate dialog box.
resultset.mark.cell.value.occurrences	boolean	`false`	`true`, `false`	Highlights instances of the selected cell's value within the data set.
resultset.image.browser	boolean	`true`	`true`, `false`	Displays images in the web browser.
resultset.behavior.autoSwitchMode	boolean	`false`	`true`, `false`	Allow to switch between **Record mode** and **Grid mode** when dealing with single or multiple rows. When set to `true`, the system will automatically switch modes based on the number of rows selected (**Record mode** for a single row, **Grid mode** for multiple rows).
resultset.calc.columnWidthByValues	boolean	`false`	`true`, `false`	Calculates column widths from displayed values
resultset.show.columnDescription	boolean	`false`	`true`, `false`	Shows column descriptions in headers
resultset.show.connectionName	boolean	`false`	`true`, `false`	Displays the name of the connection in the status section.
resultset.transform.complex.type	boolean	`true`	`true`, `false`	Enables the visualization of complex columns (such as objects, structures and arrays).
resultset.show.rightJustifyNumbers	boolean	`true`	`true`, `false`	Allows right-justify numbers.
resultset.show.rightJustifyDateTime	boolean	`true`	`true`, `false`	Aligns date and time values to the right side of their respective cells or columns.
resultset.behavior.autoCompleteProposial	boolean	`true`	`true`, `false`	Activates the auto-completion feature in the data filter text field.
resultset.show.oddRows	boolean	`true`	`true`, `false`	Marks odd and even rows in the grid.
resultset.show.colorizeDataTypes	boolean	`false`	`true`, `false`	Colorized data types in the grid.

Name	Type	Default Value	Allowed Values	Description
resultset.show.attIcons	boolean	`true`	`true`, `false`	Displays icons representing the data types of each column in the grid.
resultset.show.attFilters	boolean	`true`	`true`, `false`	Enables the display of attribute filters in the grid.
resultset.show.attrOrder	boolean	`false`	`true`, `false`	Shows asc/desc order icon in column header.
resultset.show.collections.inline	boolean	`false`	`true`, `false`	Enables the inline display of collections within the grid.
resultset.show.boolean.checkbox	boolean	`true`	`true`, `false`	Represents boolean values in the grid as checkboxes.
resultset.show.whitespace.characters	boolean	`false`	`true`, `false`	Displays whitespace characters within the grid.
resultset.show.column.position	boolean	`false`	`true`, `false`	Shows column order in the column headers.
resultset.grid.value.format	string	`UI`	`EDIT`, `NATIVE`, `UI`	Determines the format in which values are displayed within the grid.
resultset.max.column.def.width	integer	`30`	integer values (from `0` to `100`)	Determines the maximum percentage of width that a column can occupy in the grid
resultset.highlight.selectedRows	boolean	`true`	`true`, `false`	Enables the highlighting of entire rows when one or more cells within those rows are selected.
resultset.hScroll.smooth	boolean	`true`	`true`, `false`	Enables smooth scrolling within the DBeaver.
resultset.show.boolean.toggleOnClick	boolean	`true`	`true`, `false`	Allows the boolean values to be toggled or switched between true and false with a single click.
resultset.behavior.inlineEnter	boolean	`false`	`true`, `false`	Selects the next cell after the inline edit completed (or finished).
resultset.behavior.doubleClick	string	`INLINE_EDITOR`	`NONE`, `EDITOR`, `INLINE_EDITOR`, `COPY_VALUE`, `COPY_PASTE_VALUE`	Defines the action taken when a cell in a grid is double-clicked (Options: EDITOR, INLINE_EDITOR, COPY_VALUE, COPY_PASTE_VALUE, NONE).
resultset.show.row.batch.size	integer	`1`	integer values	Number of consequent rows with the same background color. Default = 1 means odd/even rows coloring.
resultset.text.tab.size	integer	`4`	integer values	Determines tab size in plain text.
resultset.text.max.column.size	integer	`255`	integer values	Determines the maximum length of a column's content when viewing data in plain text.
resultset.text.value.format	string	`EDIT`	`EDIT`,`UI`, `NATIVE`	Determines how plain text values are displayed in DBeaver.
resultset.text.show.nulls	boolean	`false`	`true`, `false`	Shows `NULL` if this setting is enabled or nothing if disabled.
resultset.text.line.number	boolean	`false`	`true`, `false`	Enables the display of line numbers.
resultset.text.delimiter.leading	boolean	`false`	`true`, `false`	Adds the leading delimiter in plain text grid representation
resultset.text.delimiter.trailing	boolean	`true`	`true`, `false`	Enables trailing delimiter in plain text for Grid Representation.
resultset.text.delimiter.top	boolean	`false`	`true`, `false`	Enables top delimiter in plain text for Grid Representation.
resultset.text.delimiter.bottom	boolean	`false`	`true`, `false`	Enables bottom delimiter in plain text for Grid Representation.
resultset.text.extra.spaces	boolean	`false`	`true`, `false`	Enables extra spaces in plain text for Grid Representation.
resultset.string.use.content.editor	boolean	`false`	`true`, `false`	Opens separate string editor instead of dialog.
resultset.binary.representation	string	`string`	`hex`, `base64`, `string`	Determines the format in which binary data is displayed.
resultset.binary.editor	string	`EDITOR`	`EDITOR`, `PANEL`	Enables to change the binary data editor. Choose between `EDITOR` (dedicated binary data editor) or `PANEL` (dialog window for viewing/editing binary data).
resultset.binary.stringMaxLength	integer	`32`	integer values (max: `3200`)	Determines the maximum length of binary strings that can be displayed in the binary editors.
content.memory.maxsize	integer	`10000`	integer values (max: `1000000000`)	Determines the maximum length of LOB (Large Object) data that will be kept in memory when working with binary editors.
content.hex.encoding	string	`UTF-8`	All available encodings from the current JVM	Allows to specify the character encoding used when interpreting binary data as strings.
hex.font.name	string	`Courier New`	`Courier New`, `Andale Mono`, `Menio`, `Monaco`, `PT Mono`	Allows to specify the name of the font to be used in the data editor.
hex.font.style	integer	`0`	`0` - Regular, `1` - Bold, `2` - Italic, `3` - Bold Italic	Allows to choose the font style for the data editor.
hex.font.size	integer	`10`	`6` - `14`, `16`, `18`, `22`, `32`, `72`	Sets the hex editor font size
default.hex.width	integer	`8`	`4`, `8`, `16`	Allows to choose the default width for the data editor.
resultset.json.maxObjectMembers	integer	`200` (max: `2000000000`)	integer values	Specifies the threshold for collapsing JSON objects in the data editor.
resultset.json.maxArrayElements	integer	`200` (max: `2000000000`)	integer values	Determines the threshold for collapsing arrays in the JSON data editor.

Name	Type	Default Value	Allowed Values	Description
resultset.json.maxElementNesting	integer	`4` (max: `400000000`)	integer values	Determines which objects in JSON will be collapsed (hidden) if their nesting level exceeds a specified threshold.
dataformat.type.number.nativeSpecialValues	boolean	`false`	`true`, `false`	Instructs the data editor to utilize the native formatting for special values.
dataformat.type.number.useGrouping	boolean	Locale-dependent	`true`, `false`	Indicates whether grouping is used to display numbers.
dataformat.type.number.useTypeScale	boolean	`false`	`true`, `false`	Uses column/attribute data type scale as minimum fraction digits number.
dataformat.type.number.roundingMode	string	No registered default	`UP`, `DOWN`, `CEILING`, `FLOOR`, `HALF_UP`, `HALF_DOWN`, `HALF_EVEN`, `UNNECESSARY`	Determines the rounding mode used for numerical values.
dataformat.type.number.minIntegerDigits	integer	`1`	integer values	Determines the minimum number of digits to display for integer values.
dataformat.type.number.minFractionDigits	integer	`0`	integer values	Determines the minimum number of fractional digits displayed.
dataformat.type.number.maxIntegerDigits	integer	`2147483647`	integer values	Defines the maximum number of digits allowed for an integer value.
dataformat.type.number.maxFractionDigits	integer	`10`	integer values	Specifies the maximum number of fraction digits allowed.
dataformat.type.number.groupingSize	integer	Locale-dependent	integer values	Determines the grouping size for displaying numeric values.
dataformat.type.timestamp.pattern	string	`yyyy-MM-dd HH:mm:ss.SSSS`	timestamp format	Determines the format for displaying and editing timestamp data.
dataformat.type.timestamptz.pattern	string	`yyyy-MM-dd HH:mm:ss.SSS Z`	timestamp with timezone format	Determines the format for displaying timestamp values with timezone information.
dataformat.type.date.pattern	string	`yyyy-MM-dd`	date format	Determines the format used for displaying and editing dates.
dataformat.type.time.pattern	string	`HH:mm:ss`	time format	Determines the format used for displaying and editing time.
dataformat.type.timetz.pattern	string	`HH:mm:ss Z`	time with timezone format	Determines the format used for displaying and editing time with timezone.

The preferences of **GIS Viewer** from **Data Editor** you can find in the `org.jkiss.dbeaver.data.gis.view.prefs` file:

Name	Type	Default Value	Allowed Values	Description
gis.view.minZoomLevel	integer	`0`	integer values	Determines the minimum zoom level
gis.view.maxObjectsRender	integer	`10000`	integer values	Determines the maximum number of objects that can be displayed.
gis.view.defaultSRID	integer	`4326`	integer values	Determines the default Spatial Reference Identifier (SRID).

Drivers settings

Name	Type	Default Value	Allowed Values	Description
ui.drivers.home	string	`path/to/DBeaverData/drivers`	string values	Specifies the local folder for driver storage.
ui.proxy.host	string	``	string values	Identifies the host for the HTTP proxy when downloading drivers.
ui.proxy.port	integer	`1080`	integer values	Specifies the port number used by the HTTP proxy during driver downloads.
ui.proxy.user	string	``	string values	Determines the username for accessing proxy during driver downloads.
ui.proxy.password	string	``	string values	Sets the password required for the proxy during driver downloads.
ui.drivers.sources	string	`https://dbeaver.io/files/jdbc/`	string values	Provides the URL where driver files can be downloaded.
ui.drivers.global.libraries	string	``	string values	Defines the global classpath from which global libraries can be loaded.
ui.drivers.version.update	boolean	`false`	`true`, `false`	Determines whether the system checks for new driver versions.

Disable Windows Defender pop-up

Name	Type	Default Value	Allowed Values	Description
`windows.defender.startup.check.skip`	boolean	`false`	`true`, `false`	Disables the Windows Defender status check and pop-up at startup. By default (false), the check is enabled.

> **Tip**: Add this preference to `DBeaverData/workspace6/.metadata/.plugins/org.eclipse.core.runtime/.settings/org.eclipse.ui.prefs` in your workspace.

Exporting and importing preferences

You can easily import and export DBeaver preferences. Follow the steps below to perform these actions:

1. Go to **File** -> **Import** (for importing) or **Export** (for exporting)

2. In the opened window, select **DBeaver**, then choose **Preferences** from the nested menu
3. In the new window:
4. for importing, navigate to the location where the configuration file is stored, select the file, and click **Finish**
5. for exporting, choose the destination where you want to save the configuration file, and click **Finish**
6. If prompted, restart DBeaver to apply the imported preferences.

Configuration files

Table of contents

- Configuration files
- dbeaver.ini
- How to Locate the dbeaver ini
- Other configuration files
- Location of the other configuration files
- Driver files
- Bundled drivers
- Maven-downloaded drivers

DBeaver utilizes several configuration files to tailor the application to user needs. These files include:

- `dbeaver.ini`
- `data-sources.json`
- `tasks.json`

Configuration files

Each of the listed files serves a specific purpose, from storing data sources and tasks to overall application settings.

dbeaver.ini

Through modifications to `dbeaver.ini`, you can apply licenses, manage drivers, enhance security, adjust user interface settings, and more.

Configuration	Description
License Management	Configure Enterprise Edition with a specific license.
Driver Management	Manage and automate the provisioning of database drivers.
Administrative Preference Restrictions	Implement restrictions to enhance application security and control over available functionalities.
AI-Smart Assistance	Control the AI-Smart Assistance feature, including the option to disable it.
Command-Line Parameters	Customize DBeaver startup and operational parameters through the command line.
JDBC Tracing	Enable tracing for JDBC operations to diagnose and troubleshoot issues.

> **Tip**: DBeaver also utilizes preference files to further customize and optimize the user experience. These preference files allow users to fine-tune the application settings and behaviors according to their specific needs.
> For detailed information, refer to the Admin manage preferences article.

How to Locate the dbeaver ini

The `dbeaver.ini` file's location varies depending on the operating system and the method of installation. As a general rule, you can find `dbeaver.ini` in the directory where DBeaver is installed. For macOS, specifically, it is located in the `Contents/Eclipse` subfolder of the application.

Here are the common locations based on the operating system and installation method:

Operating System	Location of `dbeaver.ini`
macOS	`./Applications/[DBeaver Edition].app/Contents/Eclipse/dbeaver.ini`
Linux	`/usr/share/dbeaver-[edition]/dbeaver.ini` for installations via `.deb`, `.rpm`, or PPA packages.
Windows	`C:\Program Files\[DBeaver Edition]\dbeaver.ini` for a default installation.

> Replace `[DBeaver Edition]` with your specific installation, such as `Dbeaver` for Community Edition, `DBeaverLite` for Lite Edition, `DBeaverEE` for Enterprise Edition, and `DBeaverUltimate` for Ultimate Edition.

Other configuration files

The `data-sources.json` and `tasks.json` files store database connection and task configurations. You can also use these files to transfer configurations between DBeaver installations.

Configuration file	Capabilities
`data-sources.json`	Stores database connection configurations and can also be used to transfer them between DBeaver installations
`tasks.json`	Stores task configurations, such as SQL script execution, data transfer operations, and other automated tasks, and can also be used to transfer them between DBeaver installations

> **Note**: Saved credentials may require re-entry or migration, depending on the selected credential storage provider.

Location of the other configuration files

The `data-sources.json` and `tasks.json` files are stored in project metadata under the `DBeaverData` product data root. The default workspace is `DBeaverData/workspace6`, and each project's configuration files are under `DBeaverData/workspace6/<project>/.dbeaver/`:

- **data-sources.json**: `DBeaverData/workspace6/General/.dbeaver/data-sources.json`
- **tasks.json**: `DBeaverData/workspace6/General/.dbeaver/tasks.json`

> **Tip**: Detailed instructions on finding your workspace are available in Workspace Location article.

Driver files

DBeaver stores driver `.jar` files in two locations depending on how the driver was installed:

> **Tip**: To find the exact path for a specific driver, open **Database -> Driver Manager**, select the driver, click **Edit -> Libraries tab**, then double-click the `.jar` file.

Bundled drivers

OS	Path
Windows	`C:\Program Files\[DBeaver Edition]\drivers\`
macOS	`/Applications/[DBeaver Edition].app/Contents/Eclipse/drivers/`
Linux	`/usr/share/dbeaver-[edition]/drivers/`

Maven-downloaded drivers

OS	Path
Windows	`%APPDATA%\DBeaverData\drivers\maven\maven-central\`
macOS	`~/Library/DBeaverData/drivers/maven/maven-central/`
Linux	`$XDG_DATA_HOME/DBeaverData/drivers/maven/maven-central/`

> **Tip**: For information on workspace location, see Workspace Location.

Command line

Table of contents

- Windows
- MacOS
- Linux
- Using dbeaver.ini
- Command line parameters
- DBeaver control
- System parameters
  - VM parameters
  - Connection parameters
- Install Eclipse extensions
- Exit codes

Command line parameters can be passed directly to the DBeaver executable. The way to do this depends on your operating system.

Windows

You can use the `dbeaverc.exe [parameters]` executable. This version doesn't spawn a new window, so you can see output messages in the same terminal.

MacOS

Parameters can be passed in two ways:

- Use the `open` command with the `-a` flag and the app name, followed by `--args` and your parameters.

  Depending on the edition, use `DBeaver.app`, `DBeaverLite.app`, `DBeaverEE.app`, or `DBeaverUltimate.app`.

> **Example**: `open -a "DBeaver.app" --args [parameters]`

This method does **not** redirect logs, `stdout`, or `stderr` to the terminal.

- Run the actual executable directly from inside the `.app` package.

> **Example**: `/Applications/DBeaver.app/Contents/MacOS/dbeaver [parameters]`

This method **does** redirect logs, `stdout`, and `stderr` to the terminal.

Linux

Pass parameters directly to the DBeaver executable in the terminal.

> **Example**: `/usr/bin/dbeaver-ce [parameters]`.

This method redirects logging messages, `stdout`, and `stderr` to the terminal.

Using dbeaver.ini

Parameters can also be added in the `dbeaver.ini` configuration file. These should be written at the beginning of the file, with each parameter on its own line.

> **Tip**: Detailed instructions on finding `dbeaver.ini` are available in our article.

Command line parameters

DBeaver control

Name	Value	Example
`-help`	Prints help message.	
`-stop`	Quits DBeaver.	
`-dump`	Prints DBeaver thread dump. Learn more about Thread Dump	
`-f`	Opens the file in DBeaver UI, if the command has `-con` argument, connects it to datasource.	`-f c:\some-path\some-file.sql`
`-connect`	Opens database connection in DBeaver UI. Learn more	See connection parameters table
`-closeTabs`	Closes all open editor tabs.	
`-disconnectAll`	Closes all open connections.	
`-reuseWorkspace`	Forces reuse of single workspace by multiple DBeaver instances.	
`-newInstance`	Forces new DBeaver instance creation (do not try to reuse already running one).	
`-bringToFront`	Brings the DBeaver window on top of other applications.	
`-link-project`	Links an existing external folder into the workspace as a project. The project contents stay in the original folder.	`-link-project "C:\Projects\Demo"`
`-name`	Sets the project name for `-link-project`. If you omit it, DBeaver uses the existing project name or the folder name.	`-link-project "C:\Projects\Demo" -name "Demo"`
`-unlink-project`	Unlinks a project from the workspace and keeps the project files on disk.	`-unlink-project "Demo"`
`-runTask` ★	Executes specified task. The command returns an exit code that indicates success or failure.	`-runTask @projectName:taskName.`

`-var` ★	Customs variables for runTask. You can add several parameters at once to the command line, each starting with `-var`. Used right after `-runTask`.	`-var variableName=variableValue`
		`-runTask exportFromSakila`
		`-var film=sakila.film`
		`-var actor=sakila.actor`

`-vars`	Path to a properties file for runtime variable resolving. Used for sensitive values like passwords, hosts, ports, and similar settings.	`-vars c:\path\to\file.properties` For more information, see Declare external variables in a file
`-license`	Path to the license file.	`-license "/etc/licenses/dbeaver.txt".`

> **Important**: Place task variables (`-var`) after the `-runTask` command. Example:
>
> `sh -runTask @General:export -var test=123`

System parameters

Name	Value	Example
`-nl`	UI language and locale (affects UI translation, number/date formats). For supported values, see Supported languages.	`-nl "en_US"`
`-data`	Workspace path.	`-data "c:\ProgramData\MyWorkspace"`
`-nosplash`	Omits splash screen.	
`-clean`	Clears all Eclipse caches. Use it if DBeaver fails to start after it upgrades.	
`-vmargs`	Passes VM parameters.	See VM arguments table

> **Important**: The `-data` parameter only affects the **workspace** location. Other folders are created in the default location.

VM parameters

You can pass any advanced Java parameters supported by your local JVM. Parameters supported by HotSpot JVM (25):
`https://docs.oracle.com/en/java/javase/25/docs/specs/man/java.html`

Parameters supported by all JVMs:

Name	Value	Example
`-Xms`	Sets initial memory available for DBeaver.	`-Xmx1000m`
`-Xmx`	Sets maximum memory available for DBeaver.	`-Xmx4000m`

Connection parameters

Use the `-con` argument to open a database connection in the DBeaver UI.

Pass all parameters as a **single string**, using:

- `|` to separate parameters
- `=` to separate names and values

> **Example**: `-con "driver=sqlite|database=C:\db\SQLite\Chinook.db|name=SQLite"`

See the table below for supported arguments.

Name	Description	Example
`name`	Connection name.	`Test connection`
`driver`	Driver name or ID.	`driver=sqlite`, `driver=mysql`, etc
`url`	Connection URL. Optional (JDBC URL may be constructed by a driver from other parameters).	`url=jdbc:sqlite:C:\db\SQLite\Chinook.db`
`host`	Database host name (optional).	`host=localhost`
`port`	Database port number (optional).	`port=1534`
`server`	Database server name (optional).	`server=myserver`
`database`	Database name or path (optional).	`database=db-name`
`user`	User name (optional).	`user=root`
`password`	User password (optional).	`password=mysecret`
`auth`	Authentication model ID. See Auth models.	`auth=postgres_pgpass`
`authProp.propName`	Custom authentication parameters (depends on the driver and auth model).	`authProp.oracle.net.wallet_location=C:/temp/ora-wallet`
`savePassword`	Does not ask user for a password on connection.	`savePassword=true`
`showSystemObjects`	Shows/Hides system schemas, tables, etc.	`showSystemObjects=true`
`showUtilityObjects`	Shows/Hides utility schemas, tables, etc.	`showUtilityObjects=true`
`folder`	Puts a new connection in a folder.	`folder=FolderName`
`autoCommit`	Sets connection auto commit flag (default value depends on driver).	`autoCommit=true`
`prop.propName`	Advanced connection parameters (depend on driver).	`prop.connectTimeout=30`
`id`	Connection id.	`oracle_thin-16a88e815bd-70598e648cedd28c` (useful in conjunction with `create=false`)
`connect`	Connects to this database.	`connect=false`
`openConsole`	Opens the SQL console for this database (sets `connect` to true).	`openConsole=true`

Name	Description	Example
`create`	Creates new connection.	`create=false` (true by default). If it is set as false, then an existing connection configuration will be used. The name or id parameter must be specified.
`save`	Saves new connection.	When `create=true`, then `save=false` (default) makes new connection temporary, `save=true` means that new connection will be saved and accessible between DBeaver launches.

> **Tip**: See how to declare external variables in a file

Install Eclipse extensions

You can install additional Eclipse-compatible extensions directly from the command line using the built-in P2 Director. This method doesn't require the graphical interface.

Name	Value	Example
`-application`	Defines which Eclipse application to run. Use `org.eclipse.equinox.p2.director` to install extensions via the built-in Eclipse P2 Director.	`-application org.eclipse.equinox.p2.director -repository https://wakatime.com/eclipse/update-site/ -repository https://download.eclipse.org/releases/2025-06/ -installIU com.wakatime.eclipse.feature.feature.group -followReferences`

> For details on Eclipse P2 Director syntax and available arguments, see the official Eclipse documentation

Exit codes

When DBeaver finishes executing a command, it returns an **exit code** to indicate the result.

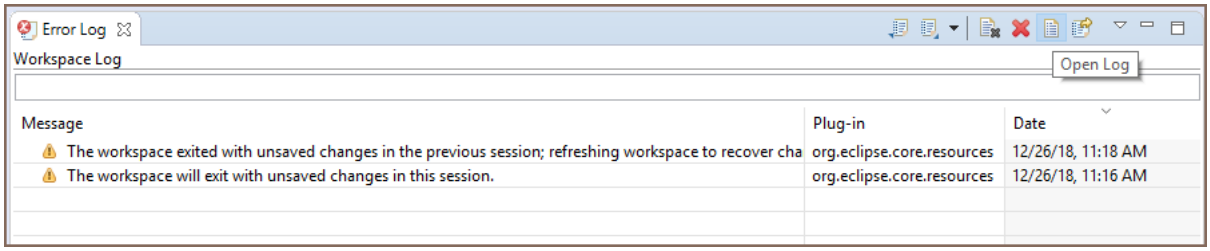
Code	Name	Description
`-1`	`EXIT_CODE_CONTINUE`	Indicates that the process continues running. You won't normally see this code when running commands.
`0`	`EXIT_CODE_OK`	Command executed successfully.
`1`	`EXIT_CODE_ERROR`	A general error occurred during execution.
`2`	`EXIT_CODE_ILLEGAL_ARGUMENTS`	Invalid command line arguments were provided.

Error log

Table of contents

- DBeaver log files
- Standard log file paths
- Customizing log files
- Gathering log files for error reporting
- Java fatal logs

There is an Error Log view (main menu **Window -> Show View -> Error Log**) which contains all errors which occur during the DBeaver runtime.
You can double click on the warning/error in the log viewer and see the error stacktrace. Please attach it to the bug report.
Also, you can open the full log (all error messages) if you need:



DBeaver log files

Standard log file paths

DBeaver writes different log files. Most of them are Eclipse logs. Log files usually reside in the `[your_workspace_location]/.metadata`.

> **Tip**: For detailed guidance on locating your workspace, refer to the workspace location article.

- On Windows, open Explorer and enter the path `%APPDATA%\DBeaverData\workspace6\.metadata`.
- On Linux, execute `cd $XDG_DATA_HOME/DBeaverData/workspace6/.metadata`.
- On MacOS, navigate to `~/Library/DBeaverData/workspace6/.metadata` in Finder. To view hidden folders, press `Cmd+Shift+.`.

Two standard log files:

- `[your_workspace_location]/.metadata/.log` - contains all warnings and errors occurring during normal operation.

- `[your_workspace_location]/.metadata/dbeaver-debug.log` - includes the same information as `.log` plus additional debug details.

> **Note**: The `dbeaver-debug.log` file has a maximum size limit of 10 MB. If the file size exceeds this limit, a new log file is created. Additionally, a new file is generated each time DBeaver is started or restarted.

In special cases, log files can be written in other directories. A special case is an emergency situation when DBeaver cannot start and there is no workspace. Two typical places to find emergency logs:

- `<install-path>/configuration`
- `${HOME}/.eclipse/org.jkiss.dbeaver.product_<dbeaver-version>`

Customizing log files

You can customize the settings for your `dbeaver-debug.log` file. Follow these steps to configure the log file settings:

1. Navigate to **Window -> Preferences -> User Interface -> Error Logs**.
2. In the **Error Logs** settings, you have the following options to customize:

Setting	Description
Enable debug logs	Toggles the recording of debug logs.
Log file location	The directory where `dbeaver-debug.log` files are stored.
Max log file size (KB)	The maximum size of the log file.
Max backup log files count	The number of backup logs to keep.

> **Note**: These settings take effect after DBeaver is restarted.

Gathering log files for error reporting

When encountering an issue with DBeaver, providing log files is essential for diagnosis. To gather the logs:

1. Navigate to **Help -> Collect Diagnostic info**.
2. Select the folder to save the collected log information.

> **Important**: Diagnostic info may contain sensitive information. You can disable sensitive data in the logs by following the guidelines in our guide. Be cautious when sharing logs with third parties.

Java fatal logs

On the rare occasion that the DBeaver process dies, it does not leave any valuable logs. This is caused by a Java VM crash.
JVM creates a fatal log file for each crash (log gile `hs_err_PID.log`). This log usually resides in the same directory where the DBeaver launcher is (e.g. dbeaver.exe).
But in some cases it is a write-protected directory and the log file will be created in other folder.
Instructions on how to find the Java fatal log file: https://docs.oracle.com/javase/9/troubleshoot/fatal-error-log.htm

Troubleshooting system issues

Table of contents

- Screen flickering
- Mixed dark and light theme elements
- Missing browser functionality
- Theme parsing errors
- Gtk box gadget distribute error
- Disabling the AT SPI accessibility bridge

DBeaver is an Eclipse RCP application, so it may encounter issues with GTK or certain Linux desktop environments. Below are common problems and their solutions:

Screen flickering

To fix screen flickering, add this line to your `~/.profile` file:

```
export GTK_IM_MODULE=ibus
```

Mixed dark and light theme elements

If parts of DBeaver show a mix of light and dark colors when using a system theme, switch to a theme that matches your system's color palette.

Missing browser functionality

If you see an error like `No more handles because there is no underlying browser available`, install the `libwebkit2gtk` library.

Theme parsing errors

If you get errors like `GTK-WARNING xxx:Theme parsing error`, try adjusting the GTK program style in your system settings:

- Open **System Settings -> Appearance -> Program Style** (the exact location may vary by desktop environment).
- Pick a style that works better with GTK apps.

Gtk box gadget distribute error

If you see the error `gtk_box_gadget_distribute: assertion size 'size >= 0' failed in GtkScrollbar`, turn off GTK overlay scrolling:

1. Open the file `~/.config/gtk-3.0/settings.ini`.
2. Add or update this setting: `[Settings] gtk-overlay-scrolling = false`

Disabling the AT SPI accessibility bridge

The AT-SPI accessibility bridge can sometimes cause crashes, especially on Linux with Cinnamon. To disable it:

1. Open a terminal and run: `bash export NO_AT_BRIDGE=1`
2. Make the change permanent:
    - Add the line to your shell configuration file (e.g., `~/.bashrc` or `~/.zshrc`): `bash export NO_AT_BRIDGE=1`
3. Restart your session to apply the change.

DBeaver Editions

Standalone

Enterprise edition

Table of contents

- Enterprise Edition features
- Subscription model
- Integrated database drivers
- Supported databases
  - Relational databases
  - Non-Relational databases

DBeaver Enterprise Edition is ideal for users who need a powerful set of tools for complex data environments. This edition goes beyond simple database tasks, enhancing data visualization, query development, security features, and administrative tasks. It is a vital tool for professionals looking to improve their productivity and manage data more effectively.

> **Note**: The DBeaver Enterprise Edition is designed for individual use only.

Enterprise Edition features

The Enterprise Edition provides specialized tools and features that cater to the needs of businesses:

- **Data visualization**
  - **Data filtering and sorting**: Provides tools to filter and sort data directly within the database interface. Learn more.
  - **Color coding and highlighting**: Simplifies data analysis by color-coding and highlighting table rows and fields based on conditions. Learn more.
  - **Functional panels**: Enhance data manipulation by offering additional space in the Data Editor for managing complex data types, accessible via tabs in a dedicated panel. Learn more.
  - **Tableau integration**: Facilitates integration with Tableau for advanced data visualization and analysis. Learn more.
- **Query development**
  - **SQL Editor**: Provides code completion for easier writing, auto-formatting, spelling checks, and support for variables. Learn more.
  - **Execution plan visualization**: Provides detailed visualizations of SQL execution plans. Learn more.
  - **Additional tools for query creation**:
    - **Visual Query Builder (VQB)**: Provides a graphical interface to construct SQL queries without writing code. Learn more.
    - **AI Assistant**: Integrates an AI Assistant and AI Chat to offer guidance and automation within the SQL Editor. Learn more.
  - **SQL debugger**: Offers an SQL debugger specifically designed for PostgreSQL, aiding in query development and troubleshooting. Learn more
- **Administrative tasks**
  - **Reverse engineering (ERD)**: Provides capabilities for reverse engineering databases into entity-relationship diagrams. Learn more.
  - **Forward engineering**:
    - **Metadata editor**: Enables the management and editing of database metadata. Learn more.
    - **Edit mode for ER-Diagrams**: Offers functionalities for editing entity-relationship diagrams to aid in database design. Learn more.
  - **Administrative tools**:
    - **Backup and Restore**: Offers functionalities for backing up and restoring databases, ensuring data safety. Learn more.
    - **Analyze and vacuum**: Tools for optimizing database performance by analyzing and vacuuming database objects.
  - **Permission and user management**: Manages database permissions and user accounts.
  - **Schema comparison**: Provides tools for comparing database schemas either within the same database or across different databases. Learn more.
  - **Data comparison**: Offers tools for comparing data across databases or within different parts of the same database. Learn more.
  - **Mock data generator**: Provides tools for generating mock data, useful for testing and development purposes. Learn more.
- **Data migration**
  - **Export and Import from files**: Supports data export and import in up to 10 different file formats. Learn more.
  - **Direct transfer between databases**: Provides tools for efficient data migration directly between different databases or systems. Learn more.
- **Security**
  - **Database authentication**: Implements database authentication mechanisms to secure access.
  - **Master and Project Password**:
    - **Master password**: Enables the use of a master password to enhance security across the DBeaver. Learn more.
    - **Project password**: Facilitates the use of project-specific password to secure projects. Learn more.
- **Automation**
  - **Task manager**: A tool for managing and monitoring database tasks, enhancing productivity and organization. Learn more.
  - **Task scheduler**: Allows for scheduling database tasks, automating routine operations. Learn more.
  - **Git integration**: Supports integration with Git for version control, and change tracking. Learn more.

Subscription model

The DBeaver Enterprise Edition is offered as an annual subscription, which should be renewed to continue receiving updates and support.

Discover more about the DBeaver Enterprise Edition and subscribe.

Integrated database drivers

The Enterprise Edition has a comprehensive set of JDBC drivers, offering connectivity to various databases without needing separate downloads and configurations. Additionally, a versatile ODBC driver is supported for broader database compatibility.

> For additional details on utilizing the ODBC driver, please consult the ODBC-JDBC driver article.

Supported databases

The Enterprise Edition provides support for a wide range of databases. Below are tables listing the databases available and specifying whether the Enterprise Edition offers enhanced features compared to the Community Edition.

> * Enhanced features may include additional metadata capabilities, such as the ability to manage and interact with specific database objects unique to certain systems. For instance, managing **Datashares** in Redshift or handling **Sequences** in Netezza. These features also cover various advanced authentication methods, and numerous other functionalities designed to improve security and data management.

Relational databases

Database name	Enhanced features* compared to the Community Edition
Altibase	-
Apache Arrow	Not available in Community Edition
Apache Calcite Avatica	✓
Apache Doris	-
Apache Kylin	-
Aurora DSQL	Not available in Community Edition
Azure SQL Server	✓
Babelfish via TDS (beta)	✓
CUBRID	✓
Cache	-
ClickHouse	-
CloudberryDB	-
ClickHouse (Legacy)	✓
CloudSQL - MySQL	Not available in Community Edition
CloudSQL - PostgreSQL	Not available in Community Edition
CloudSQL - SQL Server	Not available in Community Edition
CockroachDB	-
CrateDB	-
CrateDB (Legacy)	-
Databricks	✓
Dameng	-
Db2 for IBM i	✓
Db2 for LUW	-
Db2 for z/OS	✓
Denodo 8	-
Derby Embedded	-
Derby Server	-
Dremio	-
DuckDB	-
EDB	✓
Exasol	✓
FerretDB	Not available in Community Edition
Firebird	✓
Fujitsu Enterprise Postgres	Not available in Community Edition
GaussDB	-
GizmoSQL	-
Google AlloyDB	Not available in Community Edition

Database name	Enhanced features* compared to the Community Edition
Google Cloud Spanner	✓
Greenplum	✓
GreptimeDB	-
Greengage	Not available in Community Edition
H2 Embedded	✓
H2 Embedded V.2	-
H2 Server	✓
H2GIS Embedded	-
H2GIS Server	-
HANA	✓
HSQL Embedded	-
HSQL Server	-
Informix	✓
Ingres	-
InterSystems IRIS	-
JDBCX	-
Jennifer	-
MS Access (UCanAccess)	-
MariaDB	✓
Materialize	-
MaxDB	-
Microsoft Fabric	Not available in Community Edition
Mimer SQL	-
MonetDB	-
MySQL	✓
MySQL 5 (Legacy)	✓
NDB Cluster	-
NetSuite	Not available in Community Edition
Netezza	✓
NuoDB	-
ODBC	Not available in Community Edition
OceanBase	-
Ocient	-
OmniSci (formerly MapD)	-
OpenEdge	-
OpenSearch	✓
Oracle	✓
Pervasive SQL	-
PostgreSQL	✓
PrestoDB	-
PrestoSQL	-
Raima	Not available in Community Edition
Redshift	✓
Redshift Serverless	✓
RisingWave	-
SAP ASE jConnect	-
SQL Server	✓
SQL Server (Old driver, jTDS)	✓
SQLite	✓
SQLite Crypt	Not available in Community Edition
SQream DB	-
Salesforce	Not available in Community Edition
Salesforce Data Cloud	✓
SingleStore	✓

Database name	Enhanced features* compared to the Community Edition
Snowflake	✓
StarRocks	-
Sybase jConnect	✓
Sybase jTDS	✓
Teradata	✓
TiDB	-
Trino	-
Vertica	✓
Virtuoso	-
Yellowbrick	-
YugabyteDB	-

Non-Relational databases

Database name	Enhanced features* compared to the Community Edition
Apache Drill	✓
Apache Hive	✓
Apache Ignite	-
Apache IoTDB	-
Apache Kyuubi	-
Apache Phoenix	-
Apache Spark	-
Athena	✓
Azure Cosmos DB for Cassandra	Not available in Community Edition
Azure Cosmos DB for MongoDB	Not available in Community Edition
Azure Cosmos DB for NoSQL	Not available in Community Edition
Cassandra	Not available in Community Edition
Cloudera Impala	-
CouchDB	Not available in Community Edition
Couchbase	Not available in Community Edition
Couchbase 5+	Not available in Community Edition
CSV	+
DDL	-
DBF	-
DocumentDB	Not available in Community Edition
DynamoDB	Not available in Community Edition
Elasticsearch	✓
etcd	Not available in Community Edition
Firestore	Not available in Community Edition
Gemfire XD	-
Google BigQuery	✓
Google Cloud Bigtable	Not available in Community Edition
DolphinDB	-
InfluxDB	Not available in Community Edition
InfluxDB 2	Not available in Community Edition
TimechoDB	-
Timeplus	-
Timeplus Proton	-
InfluxDB 3	Not available in Community Edition
Keyspaces	Not available in Community Edition
Machbase	-
MongoDB	Not available in Community Edition
Neo4j	Not available in Community Edition
Neptune	Not available in Community Edition
Open Distro Elasticsearch	✓

Database name	Enhanced features* compared to the Community Edition
OrientDB	-
Parquet	Not available in Community Edition
Redis	Not available in Community Edition
ScyllaDB	Not available in Community Edition
SnappyData	-
Solr	-
TDengine	✓
TDengine Cloud	✓
TimescaleDB	-
Timestream	Not available in Community Edition
Valkey	Not available in Community Edition
XLSX	Not available in Community Edition
Yugabyte CQL	Not available in Community Edition

Lite edition

Table of contents

- Lite Edition features
- Subscription model
- Integrated database drivers
- Supported databases
  - Relational databases
  - Non-Relational databases

DBeaver Lite Edition is tailored for users who need basic tools to work with databases. This version offers a streamlined interface that focuses on simple database tasks like viewing data and running queries. It is especially useful for business users who need an easy-to-use tool.

> **Note**: The DBeaver Lite Edition is designed for individual use only.

Lite Edition features

Key features included in the Lite Edition are designed to support everyday database tasks with minimal hassle:

- **Data Visualization**
  - **Color coding and highlighting**: Simplifies data analysis by color-coding and highlighting table rows and fields based on conditions. Learn more.
  - **Data filtering and sorting**: Provides tools to filter and sort data directly within the database interface. Learn more.
  - **Analytical charts**: Offers tool for creating analytical charts to represent data visually. Learn more.
  - **Entity relationship diagrams**: Visualize database schemas through entity-relationship diagrams. Learn more.
- **Data import/export**
  - **Office formats support (XLS)**: Enables users to import and export data in Office formats, specifically Excel (XLS). Learn more.
- **Query development**
  - **AI Assistant in SQL Editor**: Integrates an AI Assistant and AI Chat to offer guidance and automation within the SQL Editor. Learn more.
  - **Search in query history**: Enables searching through previous queries for easier management and reuse.
  - **Spelling**: Provides spelling verification to improve the quality and accuracy of SQL scripts. Learn more.
  - **Visual query builder**: Provides a graphical interface to construct SQL queries without writing code. Learn more.
- **Comprehensive security support**: Ensures secure access to databases within your environment, even if they are protected by Single Sign-On (SSO), Kerberos, SSL, and other security measures.

Subscription model

The DBeaver Lite Edition is offered as an annual subscription, which should be renewed to continue receiving updates and support.

Discover more about the DBeaver Lite Edition and subscribe.

Integrated database drivers

The Lite Edition has a comprehensive set of JDBC drivers, offering connectivity to various databases without needing separate downloads and configurations. Additionally, a versatile ODBC driver is supported for broader database compatibility.

> For additional details on utilizing the ODBC driver, please consult the ODBC-JDBC driver article.

Supported databases

The Lite Edition provides support for a wide range of databases. Below are tables listing the databases available and specifying whether the Lite Edition offers enhanced features compared to the Community Edition.

> * Enhanced features may include additional metadata capabilities, such as the ability to manage and interact with specific database objects unique to certain systems. For instance, managing **Roles** in Redshift. These features also cover various advanced authentication methods, and numerous other functionalities designed to improve security and data management.

Relational databases

Database name	Enhanced features* compared to the Community Edition
Altibase	-
Apache Arrow	Not available in Community Edition
Apache Calcite Avatica	✓
Apache Doris	-
Apache Kylin	-
Aurora DSQL	Not available in Community Edition
Azure SQL Server	✓
Babelfish via TDS (beta)	✓
CUBRID	✓
Cache	-
ClickHouse	-
CloudberryDB	-
ClickHouse (Legacy)	✓
CloudSQL - MySQL	Not available in Community Edition
CloudSQL - PostgreSQL	Not available in Community Edition
CloudSQL - SQL Server	Not available in Community Edition
CockroachDB	-
CrateDB	-
CrateDB (Legacy)	-
Databricks	✓
Dameng	-
Db2 for IBM i	✓
Db2 for LUW	-
Db2 for z/OS	✓
Denodo 8	-
Derby Embedded	-
Derby Server	-
Dremio	-
DuckDB	-
EDB	✓
Exasol	✓
FerretDB	Not available in Community Edition
Firebird	✓
Firestore	Not available in Community Edition
Fujitsu Enterprise Postgres	Not available in Community Edition
GaussDB	-
GizmoSQL	-
Google AlloyDB	Not available in Community Edition
Google Cloud Spanner	✓
Greenplum	✓
GreptimeDB	-
Greengage	Not available in Community Edition
H2 Embedded	✓
H2 Embedded V.2	-
H2 Server	✓
H2GIS Embedded	-
H2GIS Server	-

Database name	Enhanced features* compared to the Community Edition
HANA	✓
HSQL Embedded	-
HSQL Server	-
Informix	✓
Ingres	-
InterSystems IRIS	-
JDBCX	-
Jennifer	-
MS Access (UCanAccess)	-
MariaDB	✓
Materialize	-
MaxDB	-
Microsoft Fabric	Not available in Community Edition
Mimer SQL	-
MonetDB	-
MySQL	✓
MySQL 5 (Legacy)	✓
NDB Cluster	-
NetSuite	Not available in Community Edition
Netezza	✓
NuoDB	-
ODBC	Not available in Community Edition
OceanBase	-
Ocient	-
OmniSci (formerly MapD)	-
OpenEdge	-
OpenSearch	✓
Oracle	✓
Pervasive SQL	-
PostgreSQL	✓
PrestoDB	-
PrestoSQL	-
Raima	Not available in Community Edition
Redshift	✓
Redshift Serverless	✓
RisingWave	-
SAP ASE jConnect	-
SQL Server	✓
SQL Server (Old driver, jTDS)	✓
SQLite	✓
SQLite Crypt	Not available in Community Edition
SQream DB	-
Salesforce	Not available in Community Edition
Salesforce Data Cloud	✓
SingleStore	✓
Snowflake	✓
StarRocks	-
Sybase jConnect	✓
Sybase jTDS	✓
Teradata	✓
TiDB	-
Trino	-
Vertica	✓
Virtuoso	-

Database name	Enhanced features* compared to the Community Edition
Yellowbrick	-
YugabyteDB	-

Non-Relational databases

Database name	Enhanced features* compared to the Community Edition
Apache Drill	✓
Apache Hive	✓
Apache Ignite	-
Apache IoTDB	-
Apache Kyuubi	-
Apache Phoenix	-
Apache Spark	-
Athena	✓
Azure Cosmos DB for Cassandra	Not available in Community Edition
Azure Cosmos DB for MongoDB	Not available in Community Edition
Azure Cosmos DB for NoSQL	Not available in Community Edition
Cassandra	Not available in Community Edition
Cloudera Impala	-
CouchDB	Not available in Community Edition
Couchbase	Not available in Community Edition
Couchbase 5+	Not available in Community Edition
CSV	+
DDL	-
DBF	-
DocumentDB	Not available in Community Edition
DynamoDB	Not available in Community Edition
Elasticsearch	✓
etcd	Not available in Community Edition
Gemfire XD	-
Google BigQuery	✓
Google Cloud Bigtable	Not available in Community Edition
DolphinDB	-
InfluxDB	Not available in Community Edition
InfluxDB 2	Not available in Community Edition
TimechoDB	-
Timeplus	-
Timeplus Proton	-
InfluxDB 3	Not available in Community Edition
Keyspaces	Not available in Community Edition
Machbase	-
MongoDB	Not available in Community Edition
Neo4j	Not available in Community Edition
Neptune	Not available in Community Edition
Open Distro Elasticsearch	✓
OrientDB	-
Parquet	Not available in Community Edition
Redis	Not available in Community Edition
ScyllaDB	Not available in Community Edition
SnappyData	-
Solr	-
TDengine	✓
TDengine Cloud	✓
TimescaleDB	-
Timestream	Not available in Community Edition

Database name	Enhanced features* compared to the Community Edition
Valkey	Not available in Community Edition
XLSX	Not available in Community Edition
Yugabyte CQL	Not available in Community Edition

Ultimate edition

Table of contents

- Ultimate Edition features
- Advanced Enterprise features in Ultimate Edition
- Subscription model
- Integrated database drivers
  - Cloud Explorer databases
- Supported databases
  - Relational databases
  - Non-Relational databases

DBeaver Ultimate Edition is perfect for users who require a powerful set of tools for managing complex data environments and are also seeking enhanced cloud capabilities. This edition builds on the features of the Enterprise Edition by adding specialized tools for cloud database management and Kafka data streams. It offers seamless integration and efficient management of cloud data sources, making it an excellent choice for users who need to extend their data handling into the cloud.

> **Note**: The DBeaver Ultimate Edition is designed for individual use only.

Ultimate Edition features

The Ultimate Edition offering unique capabilities designed for complex, multi-platform data management tasks with a strong emphasis on enhanced cloud integration:

- **Cloud Explorer**: A tool for managing and connecting cloud-hosted databases. Learn more
  It offers integration with various cloud services:
  - **Amazon Web Services** Learn more
  - **Google Cloud Platform** Learn more
  - **Microsoft Azure** Learn more
- **Cloud Storage support**: Enables file management within cloud environments of Amazon Web Services and Google Cloud Platform. Learn more
- **Kafka support**: Provides integration with Apache Kafka.

Advanced Enterprise features in Ultimate Edition

In addition to its unique features, the Ultimate Edition utilizes all the advanced capabilities from the Enterprise Edition, including:

- **Data visualization**
  - **Data filtering and sorting**: Provides tools to filter and sort data directly within the database interface. Learn more.
  - **Color coding and highlighting**: Simplifies data analysis by color-coding and highlighting table rows and fields based on conditions. Learn more.
  - **Functional panels**: Enhance data manipulation by offering additional space in the Data Editor for managing complex data types, accessible via tabs in a dedicated panel. Learn more.
  - **Tableau integration**: Facilitates integration with Tableau for advanced data visualization and analysis. Learn more.
- **Query development**
  - **SQL Editor**: Provides code completion for easier writing, auto-formatting, spelling checks, and support for variables. Learn more.
  - **Execution plan visualization**: Provides detailed visualizations of SQL execution plans. Learn more.
  - **Additional tools for query creation**:
    - **Visual Query Builder (VQB)**: Provides a graphical interface to construct SQL queries without writing code. Learn more.
    - **AI Assistant**: Integrates an AI Assistant and AI Chat to offer guidance and automation within the SQL Editor. Learn more.
  - **SQL debugger**: Offers an SQL debugger specifically designed for PostgreSQL, aiding in query development and troubleshooting. Learn more
- **Administrative tasks**
  - **Reverse engineering (ERD)**: Provides capabilities for reverse engineering databases into entity-relationship diagrams. Learn more.
  - **Forward engineering**:
    - **Metadata editor**: Enables the management and editing of database metadata. Learn more.
    - **Edit mode for ER-Diagrams**: Offers functionalities for editing entity-relationship diagrams to aid in database design. Learn more.
  - **Administrative tools**:
    - **Backup and Restore**: Offers functionalities for backing up and restoring databases, ensuring data safety. Learn more.
    - **Analyze and vacuum**: Tools for optimizing database performance by analyzing and vacuuming database objects.
  - **Permission and user management**: Manages database permissions and user accounts.
  - **Schema comparison**: Provides tools for comparing database schemas either within the same database or across different databases. Learn more.
  - **Data comparison**: Offers tools for comparing data across databases or within different parts of the same database. Learn more.
  - **Mock data generator**: Provides tools for generating mock data, useful for testing and development purposes. Learn more.
- **Data migration**
  - **Export and Import from files**: Supports data export and import in up to 10 different file formats. Learn more.

    - **Direct transfer between databases**: Provides tools for efficient data migration directly between different databases or systems. Learn more.

- **Security**
    - **Database authentication**: Implements database authentication mechanisms to secure access.
    - **Master and Project Password**:
        - **Master password**: Enables the use of a master password to enhance security across the DBeaver. Learn more.
        - **Project password**: Facilitates the use of project-specific password to secure projects. Learn more.

- **Automation**
    - **Task manager**: A tool for managing and monitoring database tasks, enhancing productivity and organization. Learn more.
    - **Task scheduler**: Allows for scheduling database tasks, automating routine operations. Learn more.
    - **Git integration**: Supports integration with Git for version control, and change tracking. Learn more.

Subscription model

The DBeaver Ultimate Edition is offered as an annual subscription, which should be renewed to continue receiving updates and support.

Discover more about the DBeaver Ultimate Edition and subscribe.

Integrated database drivers

The Ultimate Edition has a comprehensive set of JDBC drivers, offering connectivity to various databases without needing separate downloads and configurations. Additionally, a versatile ODBC driver is supported for broader database compatibility.

> For additional details on utilizing the ODBC driver, please consult the ODBC-JDBC driver article.

This edition also includes the Cloud Explorer feature, which provides access to cloud databases such as those on Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure. Cloud Explorer enables centralized authentication and automatic configuration, simplifying the management of cloud database connections.

Cloud Explorer databases

For a list of supported databases, please refer to the related section in the Cloud Explorer guide.

Supported databases

The Ultimate Edition provides support for a wide range of databases. Below are tables listing the databases available and specifying whether the Ultimate Edition offers enhanced features compared to the Community Edition.

> * Enhanced features may include additional metadata capabilities, such as the ability to manage and interact with specific database objects unique to certain systems. For instance, managing **Datashares** in Redshift or handling **Sequences** in Netezza. These features also cover various advanced authentication methods, and numerous other functionalities designed to improve security and data management.

Relational databases

Database name	Enhanced features* compared to the Community Edition	Available through Cloud Explorer
Altibase	-	-
Apache Arrow	Not available in Community Edition	-
Apache Calcite Avatica	✓	-
Apache Doris	-	-
Apache Kylin	-	-
Aurora DSQL	Not available in Community Edition	✓
Azure SQL Server	✓	✓
Babelfish via TDS (beta)	✓	-
CUBRID	✓	-
Cache	-	-
ClickHouse	-	-
CloudberryDB	-	-
ClickHouse (Legacy)	✓	-
CloudSQL - MySQL	Not available in Community Edition	✓
CloudSQL - PostgreSQL	Not available in Community Edition	✓
CloudSQL - SQL Server	Not available in Community Edition	✓
CockroachDB	-	-
CrateDB	-	-
CrateDB (Legacy)	-	-
Databricks	✓	-

Database name	Enhanced features* compared to the Community Edition	Available through Cloud Explorer
Dameng	-	-
Db2 for IBM i	✓	-
Db2 for LUW	-	-
Db2 for z/OS	✓	-
Denodo 8	-	-
Derby Embedded	-	-
Derby Server	-	-
Dremio	-	-
DuckDB	-	-
EDB	✓	-
Exasol	✓	-
Firebird	Not available in Community Edition	-
Fujitsu Enterprise Postgres	Not available in Community Edition	-
GaussDB	-	-
GizmoSQL	-	-
Google AlloyDB	Not available in Community Edition	✓
Google Cloud Spanner	✓	✓
Greenplum	✓	-
GreptimeDB	-	-
Greengage	Not available in Community Edition	-
H2 Embedded	✓	-
H2 Embedded V.2	-	-
H2 Server	✓	-
H2GIS Embedded	-	-
H2GIS Server	-	-
HANA	✓	-
HSQL Embedded	-	-
HSQL Server	-	-
Informix	✓	-
Ingres	-	-
InterSystems IRIS	-	-
JDBCX	-	-
Jennifer	-	-
MS Access (UCanAccess)	-	-
MariaDB	✓	-
Materialize	-	-
MaxDB	-	-
Microsoft Fabric	Not available in Community Edition	-
Mimer SQL	-	-
MonetDB	-	-
MySQL	✓	✓
MySQL 5 (Legacy)	✓	-
NDB Cluster	-	-
NetSuite	Not available in Community Edition	-
Netezza	✓	-
NuoDB	-	-
ODBC	Not available in Community Edition	-
OceanBase	-	-
Ocient	-	-
OmniSci (formerly MapD)	-	-
OpenEdge	-	-
OpenSearch	✓	-
Oracle	✓	✓
Pervasive SQL	-	-

Database name	Enhanced features* compared to the Community Edition	Available through Cloud Explorer
PostgreSQL	✓	✓
PrestoDB	-	-
PrestoSQL	-	-
Raima	Not available in Community Edition	-
Redshift	✓	-
Redshift Serverless	✓	✓
RisingWave	-	-
SAP ASE jConnect	-	-
SQL Server	✓	✓
SQL Server (Old driver, jTDS)	✓	-
SQLite	✓	-
SQLite Crypt	Not available in Community Edition	-
SQream DB	-	-
Salesforce	Not available in Community Edition	-
Salesforce Data Cloud	✓	-
SingleStore	✓	-
Snowflake	✓	-
StarRocks	-	-
Sybase jConnect	✓	-
Sybase jTDS	✓	-
Teradata	✓	-
TiDB	-	-
Trino	-	-
Vertica	✓	-
Virtuoso	-	-
Yellowbrick	-	-
YugabyteDB	-	-

Non-Relational databases

Database name	Enhanced features* compared to the Community Edition	Available through Cloud Explorer
Apache Drill	✓	-
Apache Hive	✓	-
Apache Ignite	-	-
Apache IoTDB	-	-
Apache Kyuubi	-	-
Apache Phoenix	-	-
Apache Spark	-	-
Athena	✓	✓
Azure Cosmos DB for Cassandra	Not available in Community Edition	✓
Azure Cosmos DB for MongoDB	Not available in Community Edition	✓
Azure Cosmos DB for NoSQL	Not available in Community Edition	✓
Cassandra	Not available in Community Edition	-
Cloudera Impala	-	-
CouchDB	Not available in Community Edition	-
Couchbase	Not available in Community Edition	-
Couchbase 5+	Not available in Community Edition	-
CSV	+	-
DDL	-	-
DBF	-	-
DocumentDB	Not available in Community Edition	✓
DynamoDB	Not available in Community Edition	✓
Elasticsearch	✓	-
etcd	Not available in Community Edition	-
Firestore	Not available in Community Edition	✓

Database name	Enhanced features* compared to the Community Edition	Available through Cloud Explorer
Gemfire XD	-	-
Google BigQuery	✓	✓
Google Cloud Bigtable	Not available in Community Edition	✓
DolphinDB	-	-
InfluxDB	Not available in Community Edition	-
InfluxDB 2	Not available in Community Edition	-
TimechoDB	-	-
Timeplus	-	-
Timeplus Proton	-	-
InfluxDB 3	Not available in Community Edition	-
Kafka (ksqlDB)	Not available in Community Edition	-
Keyspaces	Not available in Community Edition	✓
Machbase	-	-
MongoDB	Not available in Community Edition	-
Neo4j	Not available in Community Edition	-
Neptune	Not available in Community Edition	✓
Open Distro Elasticsearch	✓	-
OrientDB	-	-
Parquet	Not available in Community Edition	-
Redis	Not available in Community Edition	-
ScyllaDB	Not available in Community Edition	-
SnappyData	-	-
Solr	-	-
TDengine	✓	-
TDengine Cloud	✓	-
TimescaleDB	-	-
Timestream	Not available in Community Edition	✓
Valkey	Not available in Community Edition	-
XLSX	Not available in Community Edition	-
Yugabyte CQL	Not available in Community Edition	-

Cloud-hosted

Ultimate edition for AWS

Table of contents

- Ultimate Edition for AWS features
- Subscription model
- Installation and access
- Supported databases

DBeaver Ultimate Edition for AWS is built for companies that already run on AWS and want a desktop-grade toolset for managing data and cloud services in that environment. It's based on the Ultimate Edition and adds AWS-focused capabilities for working with cloud databases, cloud resources, and Kafka streams. You get the same familiar desktop workflow, but packaged for AWS infrastructure. It's distributed through AWS Marketplace, so you can deploy it in your AWS account and connect to it over RDP.

> **Note**: DBeaver Ultimate for AWS is designed for individual use only.

> When you deploy DBeaver Ultimate for AWS from AWS Marketplace, AWS authentication for the built-in cloud integrations is already configured on the instance. You don't need to enter access keys manually - you can start working right away after you connect.

Ultimate Edition for AWS features

DBeaver Ultimate for AWS includes all features from the Ultimate Edition.

Subscription model

DBeaver Ultimate for AWS is distributed through the AWS Marketplace and billed directly by AWS. You don't purchase a separate license key from DBeaver - subscription, activation, and billing are handled within your AWS account.

Installation and access

DBeaver Ultimate for AWS is delivered as a preconfigured AMI. You don't install it manually - you launch an EC2 instance based on the selected image in AWS Marketplace.

Two deployment variants are available:

- Windows-based image
- Linux-based image

> For information on deployment, see Set up Ultimate edition in AWS.

Supported databases

For a list of supported databases, see Ultimate Edition - Supported databases.

FAQ

Table of contents

- How to check driver version
- How to update driver version
- How to update driver version manually
- Microsoft Store
- How to get logs
- Why application-level access restrictions are not implemented

How to check driver version

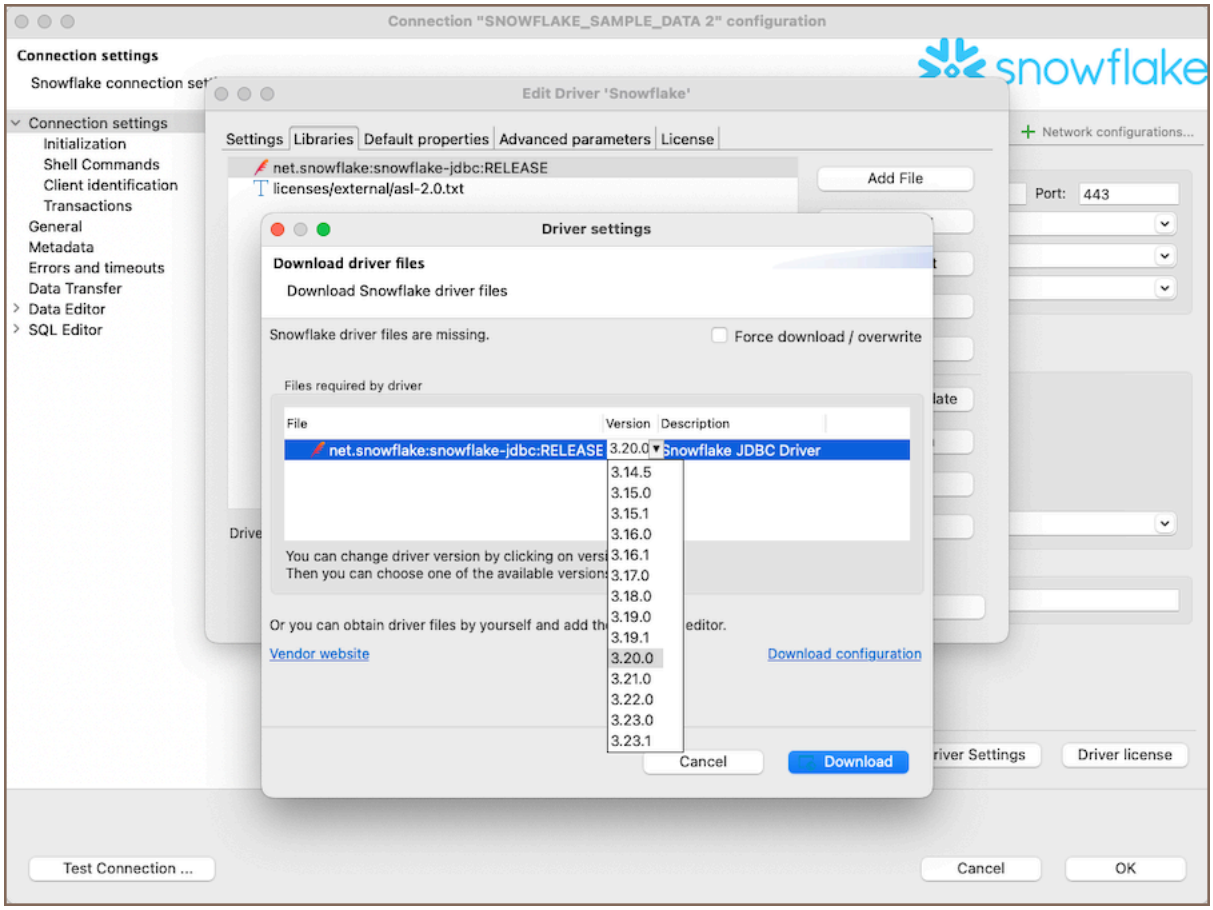
The easiest way to check your driver version.

1. In Connections, right-click your connection.
2. Click **Edit Connection**.
3. In the window that opens, click **Test Connection**.
4. In the pop-up, check the **Driver** field - that's your driver version.

How to update driver version

To update a driver version:

1. In Connections, right-click your connection.
2. Click **Edit Connection**.
3. In the window that opens, click **Driver Settings**.
4. Go to the **Libraries** tab.
5. Select the driver, then click **Download/Update**.
    - If you see a **Download/Update** button, it means the driver is available for download/update.
6. In the pop-up, click the arrow in the **Version** column.



7. Pick a version, then click **Download**.

> **Note**: DBeaver uses two types of drivers: built-in and downloadable (Maven-based).
> You can only change the version for downloadable drivers. To update a built-in driver, update it manually.

How to update driver version manually

To update a driver version manually:

1. In Connections, right-click your connection.
2. Click **Edit Connection**.
3. In the window that opens, click **Driver Settings**.
4. Go to the **Libraries** tab.
5. Double-click the driver file - this opens its folder.

   > If there's no driver listed (for example, you deleted it), click **Add File** or **Add Folder**, then select the driver file manually.

6. Replace the file with the new version. You need to download the JAR file manually. Sources vary depending on the driver:

   - **Official vendor site** - some drivers are only available on the database vendor's site.
   - **GitHub releases** - many open-source drivers publish binaries in the **Releases** section.
   - **Maven Central** - some drivers are available at search.maven.org.
   - **Other public repositories** - some projects host drivers on project-specific sites or custom registries.

   > **Tip**: You don't need to delete the whole folder - just replace or remove old JAR files to avoid conflicts.

7. Click **OK**, then **Test Connection** to verify it works.

> For details, see Driver Manager guide

Microsoft Store

- **Does a standard DBeaver license work with the Microsoft Store version?**

  Yes.

- **Can I use an Ultimate Edition add-on with Enterprise Edition or Lite?**

  No. Each add-on is tied to a specific product - an Ultimate Edition add-on works only with Ultimate Edition.

- **Where can I see my purchased add-ons?**

  Add-ons are managed by Microsoft, not DBeaver. You can view them on your Microsoft account Subscriptions page.

- **How do I cancel a subscription?**

  On your Microsoft account Subscriptions page.

How to get logs

To check logs for errors or connection issues navigate to **Window -> Show View -> Error Log**.

> For more info, see log files guide

Why application-level access restrictions are not implemented

Database permissions control which databases, schemas, tables, views, and other objects a user can see or change. They also control which SQL statements the user can run.

Application restrictions, including connection-level security restrictions and centrally managed administrative preferences, can broadly limit actions such as editing data or structure or executing SQL scripts, but they don't grant, replace, or bypass database permissions.

To restrict access to specific objects or operations, configure permissions for the database account used by the connection.

Development

Build from sources

Table of contents

- Prerequisites
- Build steps
- Dependencies

Prerequisites

Before proceeding with building DBeaver, ensure that your system meets the following prerequisites:

- **Java 25**: DBeaver currently uses OpenJDK 25 as the default Java Development Kit. Make sure you have JDK 25 or a newer version installed.

> **Important**: OpenJDK is an open-source version of Java. This means you do not need to purchase a separate Java license to use DBeaver.

- **Git**: Needed for cloning the DBeaver repository. If you do not have Git installed, visit the official Git website for download and installation instructions.
- **Internet Access**: Required for downloading dependencies and other necessary components during the build process.

Build steps

1. Clone the DBeaver repository: `sh git clone https://github.com/dbeaver/dbeaver.git`
2. To build the project, use the appropriate script for your operating system:

   On Windows, run: `bat .\dbeaver\tools\build.cmd`
   On Linux or macOS, run: `sh ./dbeaver/tools/build.sh`

After the build process completes, the binaries can be found in the `product/community/target/products` directory.

Dependencies

DBeaver has the following external Java dependencies: - `https://download.eclipse.org/releases/${eclipse-version}` - Eclipse P2 repository - https://repo.dbeaver.net/p2/ce/ - Additional dependencies from Maven (https://github.com/dbeaver/dbeaver-deps-ce) - https://github.com/dbeaver/dbeaver-common - common libraries for all dbeaver products (cloudbeaver, tools, etc)

Develop in Eclipse

Table of contents

- Opening workspace in Eclipse
- Running and Debugging in Eclipse
- Sources structure
- Notes
- Include Plugins in the Debug Info

DBeaver is an Eclipse RCP application. It consists of a set of Eclipse plugins, features, and products.

Opening workspace in Eclipse

You will need to set up Eclipse IDE for RCP and RAP Developers. The current default Eclipse version is `2025-06` (Actual version is specified in Maven property `eclipse-version` in the dbeaver-common repository). As the build may fail on older and newer versions, you will need to install a few additional Eclipse plugins which DBeaver depends on (and which are not included in the standard Eclipse distribution):

1. Install third-party dependencies from DBeaver P2 repository:
2. Open the menu **Help -> Install New Software**.
3. Copy `https://repo.dbeaver.net/p2/ce/latest` into the "Work with" field and press Enter.
4. Check all plugins and finish the installation process
5. Restart Eclipse.
6. Clone repositories:
7. `git clone https://github.com/dbeaver/dbeaver.git`.
8. `git clone https://github.com/dbeaver/dbeaver-common.git`.
9. Open project in cloned `dbeaver` repo with **File -> Open Project from File System**
10. You may need to execute **Maven -> Update project** from context menu to sync all dependencies

Running and Debugging in Eclipse

1. Locate file `product/community/DBeaver.product` and open it (double-click)
2. Click `Launch an Eclipse application in Debug mode`.

Sources structure

- `docs` - some documentations (mostly outdated)

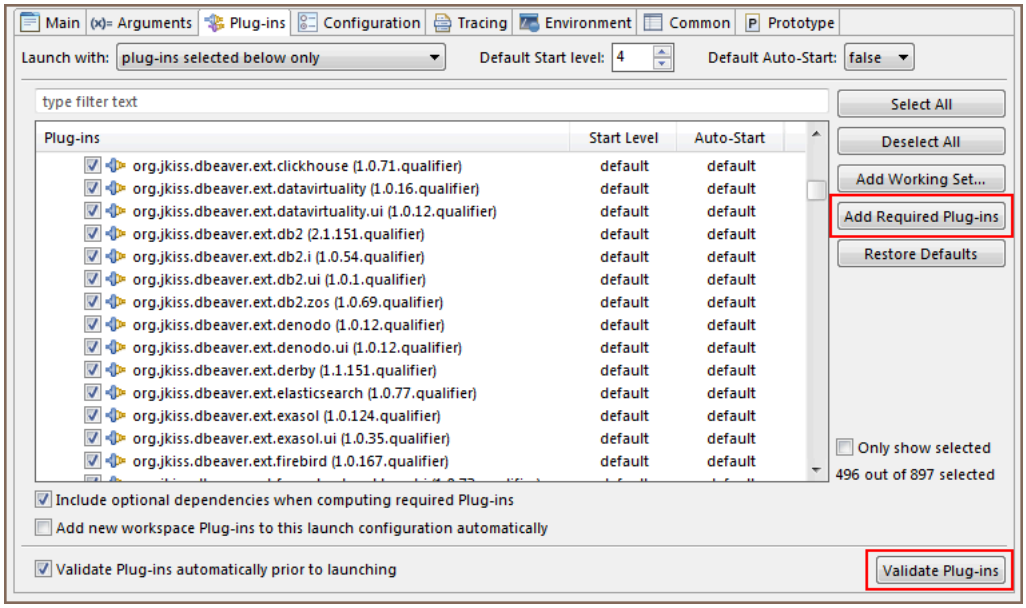
- `features` - feature descriptors. Does not contain any source code. Used to structurize the product plugins/dependencies.
- `bundles` - very base plugins (like utils)
- `plugins` - main sources location
- `org.jkiss.dbeaver.model` - model API and base classes. Does not contain any UI dependencies, just a pure data model.
- `org.jkiss.dbeaver.core` - main DBeaver module. Most of the base UI classes are here.
- `org.jkiss.dbeaver.ui.app.standalone` - a relatively small module which configures standalone DBeaver application.
- `org.jkiss.dbeaver.ui.app.eclipse` - the main Eclipse plugin. Adds some extra menus/views to the standard Eclipse IDE.
- `org.jkiss.dbeaver.ext.*` - DBeaver extensions
- `product` - the final products (standalone and Eclipse plugin) configuration.

Notes

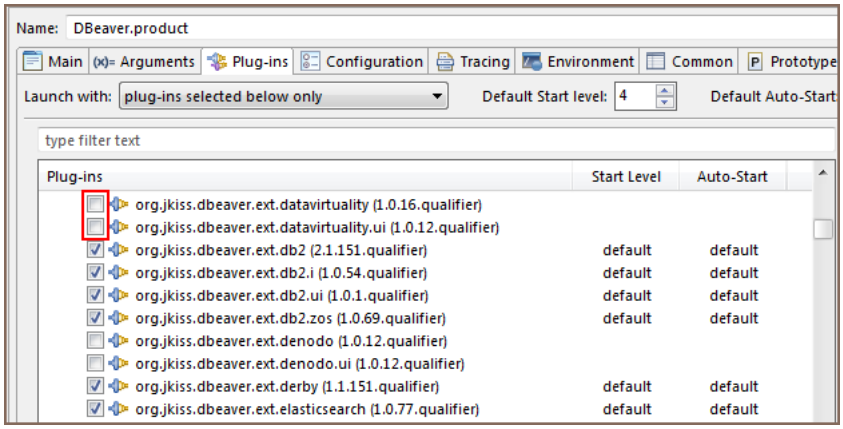
The instructions above cover the Community Edition version.

Include Plugins in the Debug Info

Sometimes, your plugins may depend on other plugins that are not included in the `DBeaver.product.launch` file. You can verify this in **Debug Configurations -> Plug-ins** tab by clicking the **Validate Plug-ins** button. If dependency problems appear in the dialogue, you can use the **Select Required** button. This action sometimes resolves all dependency issues.



Alternatively, you can manually add the necessary dependencies. A small tip: If you have added a new plugin and are confident it should work, but do not see the changes after the project build, it might be because the plugin is not marked in the `Plug-ins` tab.



Develop in IDEA

Table of contents

- Instructions
- Opening DBeaver in IDEA
- Additional steps for Cloudbeaver
- Final project structure for DBeaver workspace
- Configure code style
- Troubleshooting

DBeaver is built on the Eclipse Platform, making the Eclipse IDE a logical choice for its development. However, many developers prefer the IntelliJ IDEA.

In this guide, we aim to demonstrate how to open the DBeaver project in IDEA, write and compile code, and, most importantly, run and debug the application, leveraging the excellent Java debugger available in IDEA.

Instructions

Opening the project and understanding its module structure in IDEA is relatively straightforward. The complexity arises when attempting to run or debug the project, as this process requires an Eclipse runtime workspace, a feature exclusive to the Eclipse IDE that IntelliJ IDEA does not support.

Opening DBeaver in IDEA

1. Create a folder where you will clone DBeaver repositories.

   > **Note**: all repositories must be located in the same folder. For example, name it `dbeaver-ce`.

2. Navigate to the created folder using the command: `cd dbeaver-ce`.
3. Clone the DBeaver repository with: `git clone https://github.com/dbeaver/dbeaver.git`.
4. Clone the IDEA generator with: `git clone https://github.com/dbeaver/idea-rcp-launch-config-generator.git`.
5. Clone the DataDam API with: `git clone https://github.com/dbeaver/datadam-api.git`.
6. Clone the common components with: `git clone https://github.com/dbeaver/dbeaver-common.git`.

   > **Tip**: Optionally, for documentation purposes, clone the WIKI repository with: `git clone https://github.com/dbeaver/dbeaver.wiki.git` (skip this step if you do not need the WIKI).

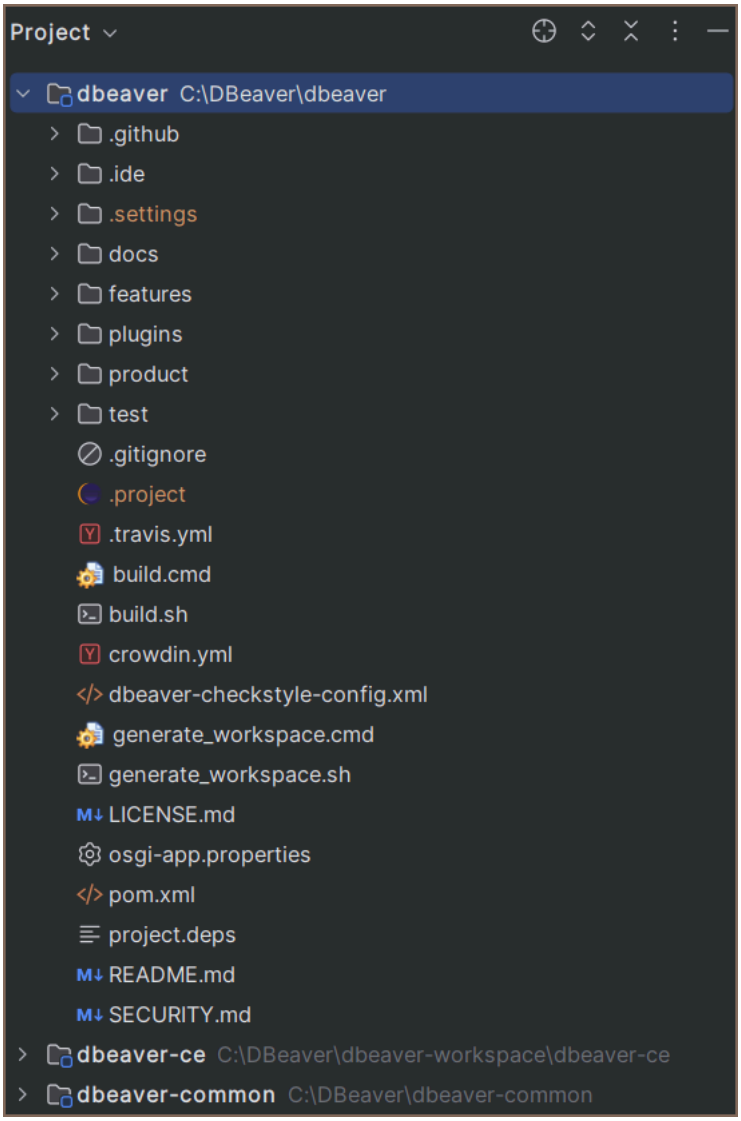
7. Clone the DBeaver osgi common repository with: `git clone https://github.com/dbeaver/dbeaver-osgi-common`.
8. (Optional) Clone the Cloudbeaver repository with `git clone https://github.com/dbeaver/cloudbeaver.git`
9. Open the `dbeaver/product/aggregate` folder in the console and run `mvn clean package`. Besides product build this will trigger Maven tools which generate dynamic source code.
10. Execute the `tools/generate_workspace` script in either the DBeaver or CloudBeaver repository, depending on the workspace you need. Once dependencies are downloaded and IDEA configurations are created, the `dbeaver-workspace` folder will appear. The IDEA configuration, by default, will be generated in `dbeaver-workspace/dbeaver-ce` or `dbeaver-workspace/cloudbeaver-ce`, which can be opened in IntelliJ IDEA.
11. Launch IntelliJ IDEA and open the project in the folder. It should contain all launch configurations.
12. If you see a popup about importing Maven projects, **press Skip**. Otherwise, IDEA might add all existing modules as Maven projects, breaking the project build.
13. For DBeaver, everything is set up and ready. Press `CTRL+F9` (or `Rebuild project`) and wait. CloudBeaver requires the following additional steps to launch properly

Additional steps for Cloudbeaver

1. To launch CloudBeaver, we first need to execute the `build.sh` or `build.bat` script in `cloudbeaver/deploy`.
2. After the script execution `cloudbeaver` folder will appear. Create the `opt` folder in the root directory and copy it there.

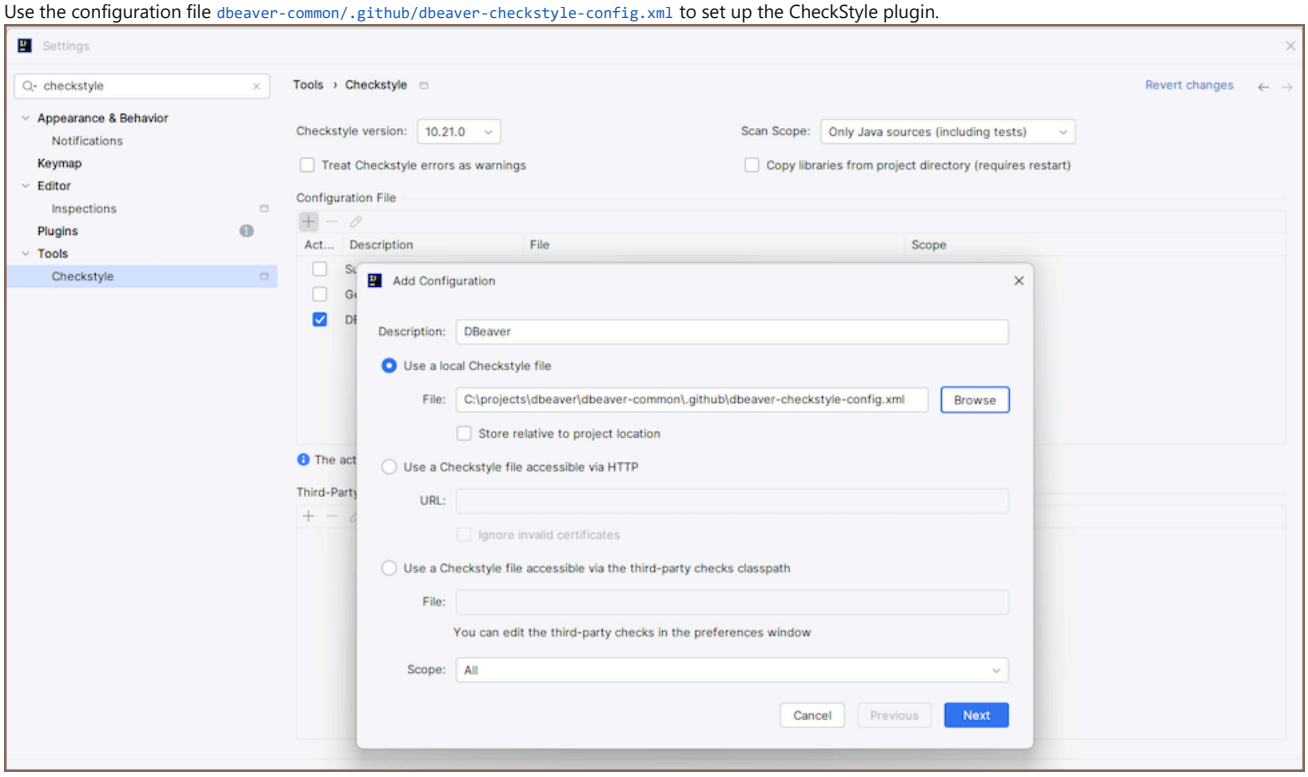
Final project structure for DBeaver workspace

The final project structure for the DBeaver workspace should look like this:

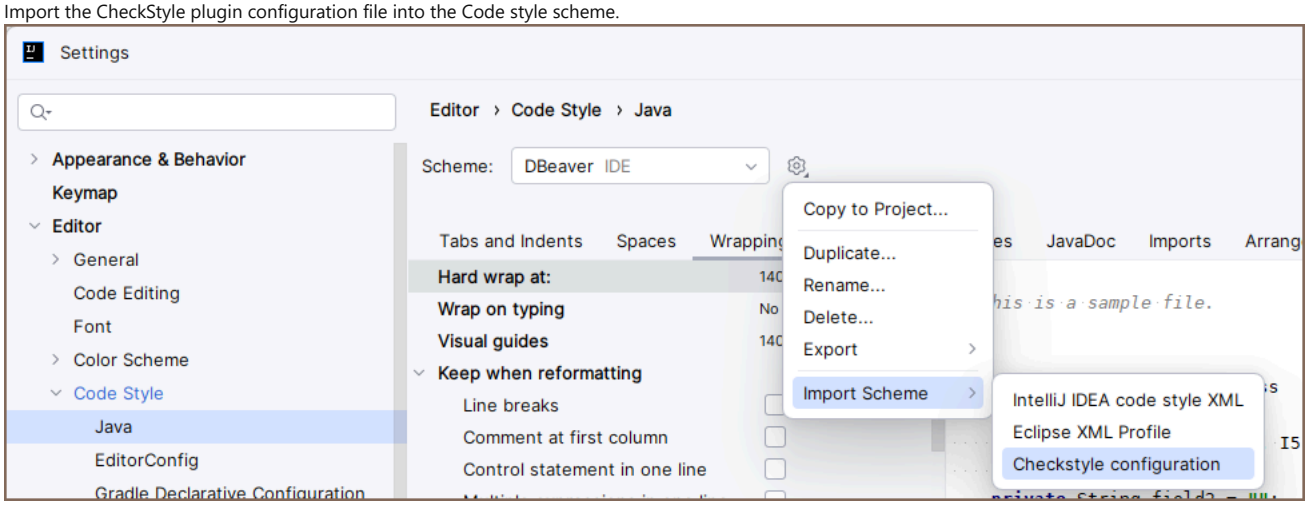


Configure code style

1. Install the CheckStyle plugin.
2. Use the configuration file `dbeaver-common/.github/dbeaver-checkstyle-config.xml` to set up the CheckStyle plugin.



3. Import the CheckStyle plugin configuration file into the Code style scheme.



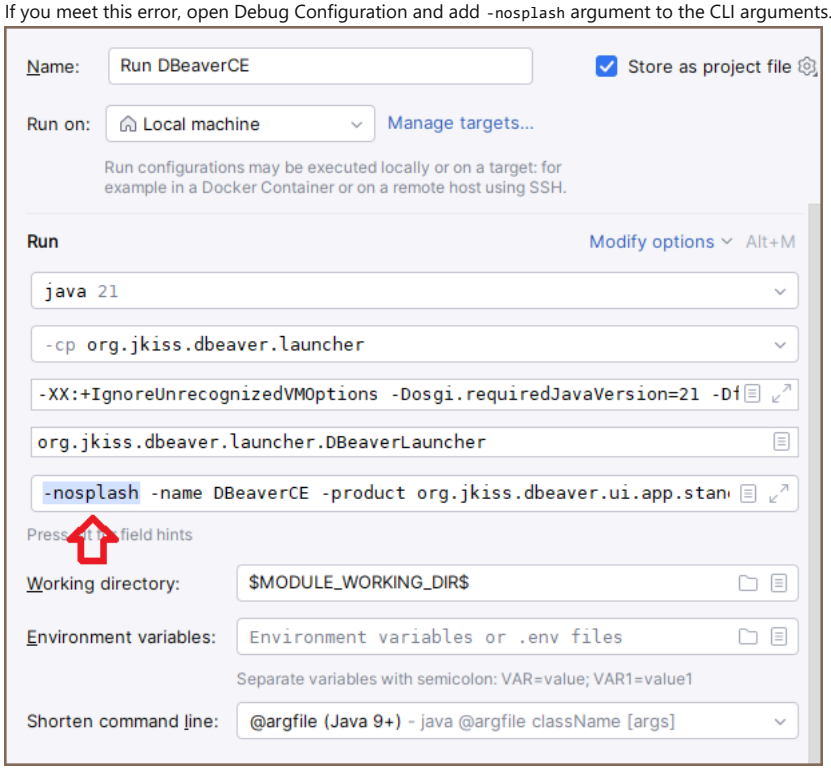
Troubleshooting

If you encounter problems, do not hesitate to double-check your configuration and try again. If you're still experiencing issues, feel free to ask for help at tickets.

Some Windows users may encounter the following error trying to debug the code.

```
Connected to the target VM, address: '127.0.0.1:50792', transport: 'socket'
#
# A fatal error has been detected by the Java Runtime Environment:
#
#  EXCEPTION_ACCESS_VIOLATION (0xc0000005) at pc=0x00007ffe0fdef633, pid=15196, tid=19604
#
# JRE version: OpenJDK Runtime Environment Temurin-21.0.7+6 (21.0.7+6) (build 21.0.7+6-LTS)
# Java VM: OpenJDK 64-Bit Server VM Temurin-21.0.7+6 (21.0.7+6-LTS, mixed mode, sharing, tiered, compressed oops, compressed class ptrs, g1 gc, windows-
amd64)
# Problematic frame:
# C  [libasyncProfiler.dll+0x8f633]
#
# No core dump will be written. Minidumps are not enabled by default on client versions of Windows
#
# An error report file with more information is saved as:
# D:\dbworkspace\dbeaver\plugins\org.jkiss.dbeaver.launcher\hs_err_pid15196.log
#
# If you would like to submit a bug report, please visit:
#   https://github.com/adoptium/adoptium-support/issues
# The crash happened outside the Java Virtual Machine in native code.
# See problematic frame for where to report the bug.
#
Disconnected from the target VM, address: '127.0.0.1:50792', transport: 'socket'
```

If you meet this error, open Debug Configuration and add `-nosplash` argument to the CLI arguments.



Resources localization

Table of contents

- Building and running DBeaver
- Finding bundles in IntelliJ IDEA
- Adding new languages
- Making localization changes
- Using placeholders in localization files
- Dealing with single quotes
- Pushing your changes
- Pull request guidelines
- Changing user interface language

DBeaver uses the standard properties-based i18n model. All translatable resources reside in the `*.properties` file. Each plugin (bundle) has its own set of resources. Almost all plugins have at least a `bundles.properties` resource. More extensive plugins have additional resources in the `src` folder. `bundle.properties` contains original English strings. Translated resources are stored in `bundle_XX.properties` files, where XX is a two-letter language code.

Building and running DBeaver

To set up your environment, follow the instructions in Develop in IDEA.

Finding bundles in IntelliJ IDEA

1. Open your cloned DBeaver project in IntelliJ IDEA.
2. Use `Find in Files` (Ctrl+Shift+F for Windows / Cmd+Shift+F on macOS) to search for `bundle.properties`.
3. Look for relevant `bundle.properties` files within the `OSGI-INF/l10n/` directory or `src` folders.

Adding new languages

If you want to add a new language that was not previously supported, you need to register it so it appears in the user interface:

1. Open the file `plugins/org.jkiss.dbeaver.registry/plugin.xml`
2. Locate `<extension point="org.jkiss.dbeaver.language">`
3. Copy an existing `<language>` element and update the `id` and `name` attributes. The `id` must match the language code.
4. Localize the language name in `OSGI-INF/l10n/bundle.properties`.

Example of adding Hebrew:

```
--- a/plugins/org.jkiss.dbeaver.registry/plugin.xml
+++ b/plugins/org.jkiss.dbeaver.registry/plugin.xml
@@ -36,6 +36,7 @@
         <language code="es" label="%language.es.label"/>
         <language code="tw" label="%language.tw.label"/>
         <language code="uk" label="%language.uk.label"/>
+        <language code="he" label="%language.he.label"/>
     </extension>
```

> **Important encoding requirements**:
>
> - All localization files must use UTF-8 encoding.
> - Ensure your editor is configured for UTF-8 and disable `native2ascii` if enabled.

Making localization changes

Using placeholders in localization files

Use text placeholders instead of concatenating strings in code. Placeholders are `{N}` markers, replaced at runtime.

```
// Outputs: "This is my text: hello from DBeaver"
NLS.bind("This is my text: {0} from {1}", "hello", "DBeaver");
```

Dealing with single quotes

Manually editing properties with single quotes requires doubling them:

- Wrong: `Can't delete '{0}'`
- Right: `Can''t delete ''{0}''`

Pushing your changes

1. Open GitHub Desktop.
2. Select **Current branch** and click **New Branch**.
3. Name your branch related to localization (`l10n`).
4. Commit your changes to the new branch.
5. Click **Publish branch** / **Push**.
6. Click **Fetch origin**.
7. In the main menu, select **Branch -> Create Pull Request**.

8. On GitHub, provide a meaningful title and description, then click **Create pull request**.
9. Switch back to the `devel` branch in GitHub Desktop.

Alternatively, create a Pull Request in the `devel` branch.

> For more details, see GitHub help.

Pull request guidelines

- Avoid multiple pull requests for the same issue.
- Don't force-push commits in the pull request branch to keep commit history readable.

Changing user interface language

See Changing interface language.

Unit Tests

Table of contents

- Using UNIX shell

DBeaver uses tycho-surefire to run JUnit tests in the OSGi environment. This fact may slightly complicate the process of writing and running unit tests for DBeaver. Let's take a look at possible ways to do that.

Using UNIX shell

Suppose we introduced some changes to our code and want to run the tests. It is actually quite easy:

```
$ cd /path/to/project/
$ mvn clean install
```

This compiles the project and runs unit tests.

If we only want to run unit tests:

```
$ cd test
$ mvn clean verify
```

The unit tests are separated into bundles. If you only want to run tests from a specific bundle, navigate to that bundle and run `mvn clean verify` there.

It is also worth pointing out that you need to recompile the whole project with `mvn clean install` after every change in the project's codebase before running unit tests, otherwise, you will be running tests against the previous version and it may lead to all sorts of confusion.

Code contribution

Table of contents

- What types of new features are acceptable
- Can I fix a bug by myself
- Code guidelines
- Nullability
- Formatting
- Conditions
- Code Quality and Readability
- Method and Class Design
- Abstraction and Encapsulation
- Specific Guidelines
- Copyright notice
- Include changes in the product
- Commit message rules
- Create pull request
- Pull request guidelines
- Can I use AI tools
- Guidelines for tagging reviewers
- Contribution rewards

This document provides guidelines for contributing to the DBeaver repository. It outlines the procedures for creating pull requests, fixing bugs, and maintaining consistency in code style within our codebase.

What types of new features are acceptable

Generally, you can suggest any feature you think is useful for the database management tool. However, we usually do not implement features that are too specific to your development process or specific to your internal company processes.

Good options include:

- Adding new database or driver support.
- Extending database metadata read/modify capabilities (e.g., adding trigger reading for a specific database).
- Adding a new SQL generator.
- Adding a new data export format.
- Localizing the DBeaver interface (extend existing localization or add a new language). For instructions, see Localization article.
- Adding a new database-specific tool (e.g., table analysis for a specific database).

You can find code examples of all these features in our codebase.

> **Important:** When suggesting new features, be aware that if a feature already exists in the Lite, Enterprise, Ultimate, or Team Editions, we may not merge it into the Community Edition.

Can I fix a bug by myself

If you identify a bug on our board that needs fixing, and it is not scheduled for the nearest milestone, you are welcome to attempt fixing it yourself. It is a good idea to ask in the ticket because some issues may not be resolved for specific reasons.

If you decide to fix the bug and submit a pull request (refer to the guidelines), the team will review it and provide feedback if necessary. If the PR is merged, the fix will be included in the next release of the DBeaver Community Edition.

> For more information in releases, see DBeaver Release Cycles

Code guidelines

The main rule is to use the same code style that is already applied in a particular source file.

- Historically, several slightly different code styles exist within our codebase. If you are modifying an old file, it is best to adhere to the code style already used in that file.
- Do not reformat code or optimize imports in a file you are changing, as it makes it very difficult to review your commits due to the volume of changes.
- Utilize our automatic code style checks for pull requests. Every PR you submit to the `dbeaver/dbeaver` repository will be checked, and you will be able to see the report of the checklist.

> IntelliJ IDEA code style is generated automatically with workspace generation according to the Develop in IDEA guide .

Nullability

To address nullability issues, we use `@org.jkiss.code.NotNull` and `@org.jkiss.code.Nullable` annotations throughout our codebase and annotate all API's return types and parameters with these.

- The usage of these annotations is **mandatory**.
- Carefully design your APIs to limit the utilization of nullable types.
- When dealing with a nullable type, you must explicitly check if it is `null` and gracefully handle such situations.

Formatting

- Line length must be 140 characters maximum.
- You must use 4-space for standard and continuation indentation.
- Add space between cast and variable (ex. `(String) object`).
- Arrays don't have space between brackets and braces (ex. `new int[]{0xFF, 0xFE, 0x00, 0x00}`).
- Declare each variable or field on its own line.
- Long-method signature formatting looks like this:

```
void test(
    int a,
    int b
) throws Something {
    ...
}
```

- Ternary operators formatting, if the whole expression doesn't fit one line, looks like this:

```
var result = test
    ? opt1
    : opt2;
```

Conditions

- The number of nested conditions should be limited. Ideally, there should be no more than 3.
- Don't forget about the break keyword in loops if further iteration is unplanned.
- Use the switch format by default in cases where it is convenient.

Code Quality and Readability

- Avoid copy-pasting in all places.
- Do not use busy waiting - not to have 100% CPU workload.

- No magic numbers.
- Don't forget `null` checks in all places where you can have a `null` argument.
- Avoid doing major refactoring within a small task because it will be difficult to find important changes in PR.
- Use more common type in declarations if possible, ex. `List<Something> name = new ArrayList<Something>()` instead of `ArrayList<Something> name = new ArrayList<Something>()`.
- Don't use wrapper classes for primitive data types if possible.
- Don't forget to add localization for strings visible in the UI.
- Pay attention to IDE hints on try-with-resources blocks. Use auto-closable constructions. Also, usually, we don't put more than one declaration to one try-with-resource
- Handle exceptional cases in the UI (throw an exception) when necessary.
- Use `CommonUtils` methods for basic Java conditions (ex. `CommonUtils.isNotEmpty()`).
- Use `var` only if you know the argument type. Use it only for simple types like `String` and `int` or for long name complex type. Unless it is not handy for code reading.
- Use records only for data holders (DTO), which will never have any logic inside.
- Avoid using `VoidProgressMonitor` if possible.
- Use static imports only in test classes.

Method and Class Design

- Carefully change the visibility of methods (`private -> public`). Do not forget about security levels.
- Naming of classes and methods matters.
- Don't forget about javadocs.
- If you've added extension points, provide clear and concise description for them.

Abstraction and Encapsulation

- Don't forget about abstraction levels. Don't try adding specific implementations at the wrong levels.
- Try not to violate encapsulation principles.
- If you can improve your small task at a more abstract level, and it's not expensive, then do it.

Specific Guidelines

- SWT and UI dependencies cannot be added directly to model bundles.
- Hardcoding colors are not allowed, especially in model plugins.
- `//$NON-NLS-1$` - use this for strings that do not require localization.

Copyright notice

Every file you add or modify must include the following copyright header at the top:

```
/*
 * DBeaver - Universal Database Manager
 * Copyright (C) 2010-2025 DBeaver Corp and others
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
```

> **Tip**: Ensure the copyright header is present and up to date for each file in your pull request.

Include changes in the product

When adding new plugins or making significant changes, ensure they are included in the product build. To do this, list your plugin in the following files:

- `features/org.jkiss.dbeaver.db.feature/feature.xml`
- `plugins/pom.xml`

> **Note**: If your changes aren't listed in these files, they'll be excluded from the final product, even if the code is functional.

Commit message rules

- You must use the GitHub issue prefix at the start of the commit message (like `#12345 Add support for super-duper database`).
- Summarize the changes in a clear and concise manner.
- Write your commit message as if you are giving a command. For example, use “Add feature” instead of “Added feature” or “Adding feature”.
- Always capitalize the first word of the commit message.
- Do not end commit message with a period.
- Do not force-push, if possible. Especially after the code review started.
- If your commit is mostly AI-generated (90%+), please include `Assisted by: {toolname}` in your commit message.

Create pull request

Before creating a pull request, you should first create a ticket in our issue tracker if it doesn't exist yet. Leave a comment indicating that you intend to implement the feature or fix the bug yourself. Once the DBeaver development or QA team has responded to your comment, you may begin your work.

You can follow the standard GitHub instructions for this process. Generally, you will need to:

- Create a fork of the `dbeaver/dbeaver` repository.
- Create a new branch in your fork.
- Commit your changes to this branch, following the commit message rules.
- Create a pull request to the upstream repository.

> **Important**: When committing changes in your branch, you must include the ticket number in the commit message like this: `dbeaver/dbeaver#issue-number Initial commit for my super-duper feature`. Otherwise, your Pull request can't be merged.

You may make any number of commits. We typically perform a squash merge before integrating changes into the main repository.

Pull request guidelines

- Please avoid creating multiple pull requests for a single issue.
- Provide a detailed description of the affected functionality.
- Avoid force-pushing commits in the PR branch, as this makes reading the commit history more difficult.
- If the UI is affected, it is recommended to provide screenshots or a video demonstrating the changes made.

Can I use AI tools

You can use AI tools just like any other tools that help you write code. However, you must remain the author of the code and take full responsibility for it. If AI generates code for you, you must fully understand it, review it, and verify it yourself before submitting a pull request.

Please follow these guidelines when using AI: - If AI tools were used to generate code, mention it in the PR description. Example: `This PR was generated with AI (GitHub Copilot).` - AI-generated contributions must comply with the project's contribution guidelines, including commit message conventions. - Carefully review the code before submitting it to the repository. Treat it the same way you would review code written by a colleague. - Test the changes in DBeaver. Make sure they work correctly in the product before submitting the PR. - Try to avoid contributions larger than ~300 new lines of code. Large AI-generated changes are difficult to review. With this in mind, large, mostly AI-generated PRs can be rejected without PR review. - Consider instructing your AI tool not to perform refactoring, unless it is absolutely necessary for logic-related changes.

If we see that AI is being used without understanding the generated code, we may close such PRs. These contributions are very difficult to review and usually do not bring useful improvements to the product.

Guidelines for tagging reviewers

- It's okay to tag someone if your ticket hasn't had any progress for about a week.
- Please avoid tagging when a PR is ready for re-review, a regular comment is enough.
- Please don't tag repeatedly (e.g., daily). Reviews will be done as time allows.

Contribution rewards

We appreciate your contributions and offer rewards for accepted pull requests. Keep contributing to improve the project and receive a reward from the team! For more details, visit Help the Beaver.